

Java Full Stack Training

Core Java Module — Extended Theory, Concepts & Demo Programs

(Viva / Interview Preparation Edition)

Accenture QuantumCrew Batch — Day-wise Training Reference

(Covers Day 1 through Performance Monitoring)

Prepared as a self-study companion to the classroom sessions

Every concept is explained in depth, backed by an actual demo program,
and followed by likely viva / interview questions with answers

Chapter 1: Java Introduction & Language Fundamentals

1.1 What is Full Stack Development?

“Full Stack” means being able to work across every layer of an application:

- **Front-end (client side):** HTML, CSS, JavaScript, React — what the user sees and interacts with.
- **Back-end (server side):** Java, Spring Boot — business logic, security, APIs.
- **Database layer:** JDBC, JPA/Hibernate, MySQL, MongoDB — where data is stored and retrieved.
- **DevOps/Cloud layer:** Git, Maven, Jenkins, Docker, Kubernetes, AWS — building, testing, and deploying the application.

This training starts at the very foundation of that stack: **Core Java**, because everything else (Spring, Hibernate, REST APIs, Microservices) is written in Java and depends on a solid grip on the language. A full-stack developer doesn’t need to be equally expert in all layers, but must understand how each layer talks to the next — e.g., how a React form submission becomes an HTTP request, how Spring routes that request to a controller method, how JPA turns a method call into SQL, and how the database returns rows that get serialized back to JSON.

1.2 Why Java? History & Design Goals

Java was released by Sun Microsystems in 1995, designed around five original goals that still shape the language today: it should be **simple, object-oriented, and familiar** (syntax deliberately close to C/C++ to ease adoption); **robust and secure** (no manual pointer arithmetic, automatic bounds checking, mandatory exception handling for risky operations); **architecture-neutral and portable** (“Write Once, Run Anywhere” via bytecode + JVM); **high performance** (Just-In-Time compilation translates hot bytecode to native machine code at runtime); and **interpreted, threaded, and dynamic** (supports multithreading and can load classes dynamically at runtime).

Java is a **platform-independent, object-oriented, statically-typed, compiled** language. Key characteristics, with the reasoning behind each:

Feature	Meaning	Why it matters in practice
Platform independent	Source compiles to bytecode (.class), executed by a JVM	The same .jar can run on Windows, Linux, macOS without recompiling
Object-Oriented	Everything (barring primitives) is modeled as objects	Encourages modular, reusable, maintainable design (see Chapter 3)
Statically typed	Every variable’s type is fixed and checked at compile time	Many bugs (type mismatches) are caught before the program ever runs
Automatic memory management	The Garbage Collector reclaims unused memory	No manual free()/delete like C/C++ — eliminates a whole class of memory-leak/dangling-pointer bugs
Robust & Secure	Strong compile-time checks, mandatory exception handling	Fewer runtime surprises; safer for enterprise/financial systems

Feature	Meaning	Why it matters in practice
Multithreaded	Native Thread support + <code>java.util.concurrent</code>	Enables applications to do multiple things “at once” (Chapter 9)

1.3 JDK, JRE, and JVM — The Three Layers

This trio is one of the most commonly asked Java basics questions, so it’s worth understanding precisely, not just as an acronym:

- **JVM (Java Virtual Machine):** the runtime engine that actually loads, verifies, and executes bytecode. It is platform-specific internally (a Windows JVM’s native implementation differs from a Linux JVM’s), but every JVM implementation honours the same bytecode contract, which is exactly what makes a compiled `.class` file portable. The JVM also owns memory management (heap, stack, garbage collection) and class loading.
- **JRE (Java Runtime Environment):** JVM + the core class libraries (`java.lang`, `java.util`, etc.) needed to *run* an already-compiled Java application. If you only need to run a `.jar`, JRE is technically sufficient.
- **JDK (Java Development Kit):** JRE + development tooling — the `javac` compiler, `javadoc` generator, `jdb` debugger, `jar` archiver, and (from Java 9+) `jshell` REPL. You need the JDK to *write and compile* Java source code.

Relationship: $\text{JDK} \supset \text{JRE} \supset \text{JVM}$ (each one is a superset containing the one before it).

1.4 Compilation & Execution Flow, In Detail

```

MyClass.java --(javac compiler)--> MyClass.class (bytecode)
                                |
                                (class loader loads it)
                                |
                                (bytecode verifier checks it)
                                |
                                (JVM interprets / JIT-compiles it)
                                |
                                Program output

```

Two additional pieces worth knowing for a viva:

- **Bytecode verification** — before running any class, the JVM checks the bytecode doesn’t violate access rules or corrupt memory — a key part of Java’s security model, since bytecode could in principle come from an untrusted source (e.g., a downloaded applet, historically).
- **JIT (Just-In-Time) compilation** — the JVM doesn’t purely interpret bytecode line-by-line forever; it profiles which methods run most frequently (“hot spots” — hence the name **HotSpot JVM**, Oracle’s reference implementation) and compiles those to native machine code on the fly, which is why long-running Java applications often get *faster* the longer they run.

1.5 Language Fundamentals — Data Types, Variables & Operators

Java has two categories of data types:

1. **Primitive types** (8 total) — store raw values directly, not objects, and always have a default value:

Type	Size	Default	Range (approx.)
byte	8-bit	0	-128 to 127
short	16-bit	0	-32,768 to 32,767
int	32-bit	0	~-2.1 billion to 2.1 billion
long	64-bit	0L	very large; needs L suffix for literals beyond int range
float	32-bit	0.0f	single-precision decimal; needs f suffix
double	64-bit	0.0	double-precision decimal (default for decimal literals)
char	16-bit	'\u0000'	single Unicode character
boolean	JVM-dependent	false	true/false only

2. **Reference types** — everything else: classes, interfaces, arrays, Strings. A reference variable holds the *memory address* of an object living on the heap, not the object itself; its default value (as an instance field) is `null`.

Variable categories and their default-value behaviour (a frequent viva trap):

- **Local variables** (declared inside a method) — **must be explicitly initialized** before use; Java gives them *no* default value, and the compiler flags “variable might not have been initialized” if you try to read one too early.
- **Instance variables** (fields belonging to an object) — automatically get the type’s default value shown in the table above if not explicitly initialized.
- **Static/class variables** (declared static) — shared across *all* objects of the class (only one copy exists no matter how many objects are created), also get default values, and are initialized once when the class is first loaded.

Type casting:

- **Widening (implicit)** — a smaller type is automatically promoted to a larger one with no data loss, e.g. `int` → `long` → `float` → `double`. No cast operator needed.
- **Narrowing (explicit)** — converting a larger type to a smaller one *can* lose data or precision, so Java forces you to write an explicit cast to acknowledge the risk: `int i = (int) someLong;`

Operators, grouped by family:

- **Arithmetic:** `+` `-` `*` `/` `%` (% is remainder, not strictly “modulo” for negative numbers in Java — the sign of the result follows the dividend).
- **Relational:** `==` `!=` `>` `<` `>=` `<=` — compare values (for primitives) or **references** (for objects — see the String-comparison discussion in Chapter 2).
- **Logical:** `&&` `||` `!` — `&&` and `||` are **short-circuit**: if the left operand already decides the result, the right operand is never evaluated (important for avoiding, e.g., a `NullPointerException` in `obj != null && obj.getValue() > 0`).
- **Bitwise:** `&` `|` `^` `~` `<<` `>>` `>>>` — operate on the individual bits of integer types; `>>>` is the *unsigned* right shift (fills with 0 regardless of sign), while `>>` is signed (fills with the sign bit) — a classic interview distinction.
- **Assignment:** `=` `+=` `-=` `*=` `/=` `%=` — compound assignment operators implicitly include a narrowing cast, so `byte b = 10; b += 5;` compiles even though `b = b + 5` (without the

compound operator) would not, since `b + 5` promotes to `int`.

- **Ternary:** `condition ? valueIfTrue : valueIfFalse` — a compact single-expression `if/else`.

The demo program below (`OperatorDemo.java`) exercises these operator families with real output, and `DatatypeDemo.java` shows the size/range/default behaviour of each primitive type.

File: `JavaIntroductionDemos/com/accenture/ltt/DatatypeDemo.java`

```
package com.accenture.ltt;

/**
 * Demonstrates different primitive data types in Java.
 */
public class DatatypeDemo {
    public static void main(String[] args) {
        int number = 25;           // Integer type
        double price = 19.99;      // Double (floating point number)
        char letter = 'A';         // Character type
        boolean flag = true;       // Boolean type

        // Printing each variable
        System.out.println("Int: " + number);
        System.out.println("Double: " + price);
        System.out.println("Char: " + letter);
        System.out.println("Boolean: " + flag);
    }
}
```

File: `JavaIntroductionDemos/com/accenture/ltt/OperatorDemo.java`

```
package com.accenture.ltt;

/**
 * Demonstrates various Java operators.
 */
public class OperatorDemo {
    public static void main(String[] args) {
        int a = 10, b = 5;

        // Arithmetic operators
        System.out.println("Addition: " + (a + b));           // 15
        System.out.println("Multiplication: " + (a * b));     // 50

        // Relational operator
        System.out.println("Is a > b? " + (a > b));           // true

        // Logical operator
        System.out.println("Logical AND: " + (a > 5 && b < 10)); // true
    }
}
```

1.6 Keywords

Java reserves about 50 keywords that cannot be used as identifiers (variable/class/method names). They fall into groups: **access modifiers** (`public`, `private`, `protected` — and the

implicit “package-private” when none is specified), **class-related** (class, interface, extends, implements, abstract, final), **control flow** (if, else, switch, for, while, do, break, continue, return), **exception handling** (try, catch, finally, throw, throws), and **others** (static, this, super, new, void, instanceof, synchronized, package, import, transient, volatile, native, strictfp).

Two frequently-confused keyword pairs worth memorizing for a viva:

- **final vs finally vs finalize()** — final prevents a variable from being reassigned, a method from being overridden, or a class from being subclassed; finally is a block that always executes after a try (Chapter 10); finalize() was a deprecated Object method the garbage collector used to call before reclaiming an object (no longer recommended/relied upon in modern Java).
- **this vs super** — this refers to the current object’s own members; super refers to the immediate parent class’s members (used to call a parent constructor or an overridden parent method — see Chapter 4).

KeywordDemo.java below demonstrates several of these keywords being used together in one class.

File: JavaIntroductionDemos/com/accenture/ltt/KeywordDemo.java

```
package com.accenture.ltt;

/**
 * Demonstrates basic Java keywords used in a class structure.
 */
public class KeywordDemo {
    public static void main(String[] args) {
        // 'public' means this method is accessible to all
        // 'static' allows the method to run without creating an instance
        // 'void' means the method doesn't return anything
        System.out.println("Demonstrating class, public, static, void, etc.");
    }
}
```

1.7 Control Flow — Conditionals, Switch, Loops

Decision-making: if / else if / else evaluates boolean expressions to choose exactly one branch of execution — nested ifs should generally be avoided beyond 2-3 levels for readability (refactor into separate methods or use early returns instead).

switch compares one variable against several constant values. Classic switch requires an explicit break in each case, otherwise execution **falls through** to the next case (sometimes intentional, often a bug). Java 14+ introduced the newer **arrow-style switch expression** (case X -> ...;) which does *not* fall through and can directly produce a value — covered again under Java 17 pattern matching in Chapter 15.

Looping constructs:

- **for** — best when the number of iterations (or a counter) is known upfront: for (init; condition; update) { ... }.
- **while** — condition is checked *before* each iteration; may execute the body **zero times** if the condition is false from the start.
- **do-while** — condition is checked *after* the body runs, so the body always executes **at least once**, even if the condition is false immediately.

- **Enhanced for-each** (for (Type item : collection)) — simplified iteration syntax for arrays and collections, hiding the index/iterator entirely; cannot be used when you need the index itself or need to remove elements while iterating (see the Collections chapter's Iterator discussion).

break vs continue: break exits the *entire* loop (or switch) immediately; continue skips only the *rest of the current iteration* and jumps to the loop's next condition check.

The demo files below show if-else decision trees, a switch statement, and all three loop types in action.

File: JavaIntroductionDemos/com/accenture/ltt/IfElseDemo.java

```
package com.accenture.ltt;

/**
 * Demonstrates if-else control structure in Java.
 */
public class IfElseDemo {
    public static void main(String[] args) {
        int number = 10;

        // Check if number is positive, negative, or zero
        if (number > 0) {
            System.out.println("Positive number");
        } else if (number < 0) {
            System.out.println("Negative number");
        } else {
            System.out.println("Zero");
        }
    }
}
```

File: JavaIntroductionDemos/com/accenture/ltt/LoopDemo.java

```
package com.accenture.ltt;

/**
 * Demonstrates different loop structures in Java.
 */
public class LoopDemo {
    public static void main(String[] args) {

        // For loop: Executes known number of times
        System.out.println("For loop:");
        for (int i = 1; i <= 5; i++) {
            System.out.println("Count: " + i);
        }

        // While loop: Executes as long as condition is true
        System.out.println("\nWhile loop:");
        int j = 1;
        while (j <= 5) {
            System.out.println("Count: " + j);
            j++;
        }
    }
}
```

```

// Do-while loop: Executes at least once
System.out.println("\nDo-While loop:");
int k = 1;
do {
    System.out.println("Count: " + k);
    k++;
} while (k <= 5);
}

```

File: JavaIntroductionDemos/com/accenture/ltt/SwitchDemo.java

```

package com.accenture.ltt;

/**
 * Demonstrates switch-case control structure in Java.
 */
public class SwitchDemo {
    public static void main(String[] args) {
        int day = 3;

        // Match value of 'day' with one of the cases
        switch (day) {
            case 1:
                System.out.println("Monday");
                break; // Prevents fall-through
            case 2:
                System.out.println("Tuesday");
                break;
            case 3:
                System.out.println("Wednesday");
                break;
            default:
                System.out.println("Invalid Day");
        }
    }
}

```

1.8 Practice Activities

The two activity files below were given to trainees as hands-on exercises applying the fundamentals covered above (data types, operators, conditionals and loops) to small real-world style problems.

File: JavaIntroductionDemos/com/accenture/ltt/activities/ActivityOne.java

```

package com.accenture.ltt.activities;

/**
 * TODO: Write a program that checks whether a given year is a leap year.
 * Hints:
 * - A leap year is divisible by 4
 * - But not divisible by 100 unless divisible by 400
 */
public class ActivityOne {

```

```

    public static void main(String[] args) {
        int year = 2024;

        // TODO: Write logic using if-else
        // Output should be "Leap Year" or "Not a Leap Year"
    }
}

```

File: JavaIntroductionDemos/com/accenture/ltt/activities/ActivityTwo.java

```

package com.accenture.ltt.activities;

/**
 * TODO: Implement a calculator that performs +, -, *, / operations using switch.
 */
public class ActivityTwo {
    public static void main(String[] args) {
        double a = 20;
        double b = 5;
        char operator = '+'; // Try with '-', '*', '/'

        // TODO: Use switch-case to perform operation
        // Output the result
    }
}

```

1.9 Viva Voce — Likely Questions & Answers

Q1. Why is Java called “platform independent”? Isn’t the JVM itself platform-specific? A: The *source code and compiled bytecode* are platform-independent — the same .class/.jar runs unmodified anywhere. What’s platform-specific is the *JVM implementation itself* (a native program you install per OS), which is precisely what translates that universal bytecode into instructions the local machine understands.

Q2. What is the difference between JDK, JRE, and JVM? A: JVM executes bytecode; JRE = JVM + runtime libraries (enough to *run* programs); JDK = JRE + development tools like javac (needed to *write and compile* programs). $JDK \supset JRE \supset JVM$.

Q3. Why does Java not support pointers directly like C/C++? A: To eliminate pointer-arithmetic bugs (buffer overruns, dangling pointers) and improve security — Java uses *references* instead, which can only refer to valid objects or null, and all array access is bounds-checked automatically at runtime.

Q4. What is the default value of a local variable? Of an instance variable? A: Local variables have **no default** — the compiler forces you to initialize before use. Instance variables (and static variables) get an automatic default (0/0.0/false/null depending on type).

Q5. What is the difference between == and .equals()? A: == compares primitive *values* directly, or for objects, compares whether two references point to the exact same object in memory. .equals() (inherited from Object, commonly overridden) is meant to compare logical/content equality — critical for String and custom objects (expanded in Chapter 12).

Q6. Difference between >> and >>>? A: Both are right-shift operators. >> is the *signed* right shift — it fills the vacated high bits with the original sign bit (preserving negative numbers as negative). >>> is the *unsigned* right shift — it always fills with 0, regardless of sign.

Q7. Why are &&|| called “short-circuit” operators? A: Because Java stops evaluating as soon as the overall result is already determined by the left operand — e.g., in a && b, if a is false, b is never evaluated at all. This is commonly used defensively, e.g. if (obj != null && obj.isValid()), to avoid a NullPointerException.

Q8. What happens if you don’t put a break in a switch case? A: Execution “falls through” into the next case’s statements, continuing until it hits a break or reaches the end of the switch — this can be a bug if unintended, but is occasionally used deliberately to group multiple case labels that should share the same logic.

Q9. What’s the difference between while and do-while? A: while checks its condition *before* the first iteration (may run zero times); do-while checks *after* the first iteration runs (always runs at least once).

Q10. Is Java purely object-oriented? A: No — Java has 8 **primitive types** that are not objects (for performance reasons, to avoid the overhead of object allocation for simple values like int). This is why “wrapper classes” (Integer, Double, etc., plus autoboxing/unboxing) exist to bridge primitives into the object-oriented collections framework when needed.

Chapter 2: Arrays & Strings

2.1 Arrays — Structure & Behaviour

An **array** is a fixed-size, ordered, index-based container of elements of the *same type*, stored in **contiguous memory**. This contiguity is exactly why array *index access* is $O(1)$ — the JVM can compute the exact memory address of `arr[i]` directly (`base_address + i * element_size`) without scanning anything.

Key points:

- Declared as `type[] name` (preferred, Java style) or `type name[]` (C-style, also legal but discouraged).
- Arrays must be created with `new` (e.g., `int[] arr = new int[5];`) or via an array literal (`int[] arr = {1, 2, 3};`) — **the size is fixed at creation and can never change afterward**. This is precisely why `ArrayList` (Chapter 12) is usually preferred when the final size isn't known upfront — it “grows” by silently allocating a *new*, larger backing array behind the scenes and copying elements over, which an array itself cannot do.
- Indexing starts at **0**; the last valid index is `length - 1`. Accessing `arr[length]` or beyond throws `ArrayIndexOutOfBoundsException` at runtime (this is *not* caught at compile time).
- Arrays can be **1-D**, **2-D (matrix, an “array of arrays”)**, or multi-dimensional.
- Default values on creation mirror the primitive-type defaults from Chapter 1: `0` for numeric types, `false` for boolean, `null` for reference-type arrays (e.g., `String[]`).
- `array.length` is a **field** (not a method call, unlike `String.length()`) — a subtle but frequently-tested distinction.
- Arrays are **objects** in Java (even arrays of primitives), which is why `array.length` works without a cast, and why you can pass an array by reference to a method.

Common array algorithms every developer should be comfortable with: **traversal**, **reversal**, **sum/average calculation**, and **searching**. The demo programs below implement each of these.

2.2 Strings — Immutability & the String Pool

A `String` in Java is an **immutable** object — once created, its character sequence can never be changed; every “modifying-looking” method (`concat`, `substring`, `replace`, `toUpperCase`, ...) actually returns a **brand-new** `String` object, leaving the original untouched.

Why immutability matters (a very common viva question):

1. **Thread-safety** — an immutable object can be freely shared across multiple threads with zero synchronization, since no thread can ever change it out from under another.
2. **The String Pool (a.k.a. String Constant Pool)** — a special memory region (part of the heap) that caches `String literals`. When you write `String a = "hello";` the JVM checks the pool first; if “hello” already exists there, `a` is pointed at the *same* existing object instead of allocating a new one. This only works because `Strings` are immutable — sharing would be unsafe otherwise. Using `new String("hello")` deliberately bypasses the pool and always creates a new object on the heap, which is why `"hello" == new String("hello")` is `false` even though `.equals()` returns `true`.
3. **Safe as `HashMap` keys** — since a `String`'s `hashCode()` cannot change after creation (nothing can mutate its characters), it's safe to cache the hash code the first time it's computed, and safe to use `Strings` as hash-based collection keys without the entry becoming “lost” if the key's hash later changed.
4. **Security** — `Strings` are used constantly for things like file paths, network hosts, and database URLs; immutability prevents malicious code from changing a `String`'s value after a security check has already validated it.

String vs StringBuilder vs StringBuffer:

	String	StringBuilder	StringBuffer
Mutable?	No	Yes	Yes
Thread-safe?	Yes (by virtue of immutability)	No	Yes (synchronized methods)
Performance	Slow for repeated concatenation (each + creates a new object)	Fast	Slower than StringBuilder due to synchronization overhead
Use when	Value won't change, or changes rarely	Building a string in a loop, single-threaded	Building a string in a loop, shared across threads

Common String operations: `length()` (a method, unlike array's `.length` field), `charAt(i)`, `substring(start, end)` (end index exclusive), `equals()` vs `==` (content vs reference comparison — **always** use `.equals()` for String content comparison), `equalsIgnoreCase()`, `toCharArray()`, `split(regex)`, `trim()/strip()`, `indexOf()/lastIndexOf()`, and `replace()`.

The demo files below cover array traversal/reversal/aggregation and String manipulation, including converting a String to a character array and counting words in a sentence.

File: `ArraysAndStringsDemos/com/accenture/ltt/ArrayDemo.java`

```
package com.accenture.ltt;

/**
 * Demonstrates different types of arrays in Java.
 */
public class ArrayDemo {
    public static void main(String[] args) {
        // 1. Declare and initialize an integer array
        int[] numbers = {10, 20, 30, 40, 50};

        // Accessing and printing elements
        System.out.println("Array Elements:");
        for (int i = 0; i < numbers.length; i++) {
            System.out.println("Index " + i + ": " + numbers[i]);
        }

        // 2. Calculate sum of array elements
        int sum = 0;
        for (int num : numbers) {
            sum += num;
        }
        System.out.println("Sum = " + sum);

        // 3. String array
        String[] fruits = {"Apple", "Banana", "Cherry"};
        System.out.println("Fruits:");
        for (String fruit : fruits) {
            System.out.println(fruit);
        }
    }
}
```



```
}  
}
```

File: ArraysAndStringsDemos/com/accenture/ltt/ArrayReverse.java

```
package com.accenture.ltt;  
  
/**  
 * Reverses an array and prints the result.  
 */  
public class ArrayReverse {  
    public static void main(String[] args) {  
        int[] numbers = {1, 2, 3, 4, 5};  
  
        System.out.println("Reversed Array:");  
        for (int i = numbers.length - 1; i >= 0; i--) {  
            System.out.print(numbers[i] + " ");  
        }  
    }  
}
```

File: ArraysAndStringsDemos/com/accenture/ltt/ArraySumAndAverage.java

```
package com.accenture.ltt;  
  
/**  
 * Calculates sum and average of array elements.  
 */  
public class ArraySumAndAverage {  
    public static void main(String[] args) {  
        int[] marks = {70, 80, 90, 100, 85};  
  
        int sum = 0;  
        for (int mark : marks) {  
            sum += mark;  
        }  
  
        double average = (double) sum / marks.length;  
  
        System.out.println("Sum = " + sum);  
        System.out.println("Average = " + average);  
    }  
}
```

File: ArraysAndStringsDemos/com/accenture/ltt/StringDemo.java

```
package com.accenture.ltt;  
  
/**  
 * Demonstrates common String operations in Java.  
 */  
public class StringDemo {  
    public static void main(String[] args) {  
        // Creating Strings  
        String name = "Alice";  
        String greeting = "Hello, " + name;  
    }  
}
```

```

// String length
System.out.println("Length of greeting: " + greeting.length());

// Concatenation
String fullName = "Alice" + " " + "Smith";
System.out.println("Full Name: " + fullName);

// Comparison
String s1 = "hello";
String s2 = "Hello";
System.out.println("Equals: " + s1.equals(s2)); // false
System.out.println("Equals Ignore Case: " + s1.equalsIgnoreCase(s2)); // true

// Character access
System.out.println("First char: " + name.charAt(0));

// Substring
System.out.println("Substring: " + fullName.substring(0, 5));
}
}

```

File: ArraysAndStringsDemos/com/accenture/ltt/StringToCharArray.java

```

package com.accenture.ltt;

/**
 * Converts a string into character array and prints characters.
 */
public class StringToCharArray {
    public static void main(String[] args) {
        String text = "HELLO";

        char[] chars = text.toCharArray();

        System.out.println("Characters in the string:");
        for (char c : chars) {
            System.out.println(c);
        }
    }
}

```

File: ArraysAndStringsDemos/com/accenture/ltt/StringWordCount.java

```

package com.accenture.ltt;

/**
 * Counts the number of words in a sentence.
 */
public class StringWordCount {
    public static void main(String[] args) {
        String sentence = "Java is a powerful programming language";

        String[] words = sentence.split(" "); // split by space
    }
}

```

```

        System.out.println("Word count: " + words.length);
    }
}

```

2.3 Practice Activities

File: ArraysAndStringsDemos/com/accenture/ltt/activities/ActivityOne.java

```

package com.accenture.ltt.activities;

/**
 * TODO: Find the maximum number in an array of integers.
 */
public class ActivityOne {
    public static void main(String[] args) {
        // TODO: Declare an array of 5 integers
        // TODO: Use loop to find the maximum value
        // TODO: Print the maximum value
    }
}

```

File: ArraysAndStringsDemos/com/accenture/ltt/activities/ActivityTwo.java

```

package com.accenture.ltt.activities;

/**
 * TODO: Check if a given string is a palindrome.
 * A palindrome reads the same forward and backward. e.g., "madam", "racecar"
 */
public class ActivityTwo {
    public static void main(String[] args) {
        // TODO: Declare a string
        // TODO: Reverse the string using loop or StringBuilder
        // TODO: Compare original and reversed string (ignore case)
        // TODO: Print "Palindrome" or "Not a palindrome"
    }
}

```

File: ArraysAndStringsDemos/com/accenture/ltt/activities/TestForLoop.java

```

package com.accenture.ltt.activities;

import java.util.Scanner;

public class TestForLoop {

    public static void main(String[] args) {

        // Step 1: Display message to user
        System.out.println("Enter count of elements of array");

        // Step 2: Create Scanner object and read user input
        Scanner scanner = new Scanner(System.in);
        int count = 0; // variable to store user input for array size
    }
}

```

```

// Step 3: Create String array using user input
String[] elements = null;

// Step 4: Use do-while loop to read and initialize array elements
// Loop should iterate as many times as total number of elements

// Step 5: Use for-each loop to iterate over array and print values

// Step 6: Close the scanner
scanner.close();
    }
}

```

2.4 Viva Voce — Likely Questions & Answers

Q1. Why are arrays fixed-size in Java, and how does ArrayList get around this? A: An array's size is baked into its allocation on the heap, so it physically cannot grow. ArrayList works around this by internally holding a *regular array* as backing storage; when it runs out of room, it allocates a **new, larger array** and copies all existing elements across — this is hidden from the caller, giving the *illusion* of a growable array.

Q2. What exception is thrown when you access an invalid array index, and when is it detected? A: `ArrayIndexOutOfBoundsException`, thrown at **runtime** — Java cannot catch this at compile time because the index is often a variable whose value is only known while the program runs.

Q3. Why is String immutable in Java? A: For thread-safety (safe to share without synchronization), to enable the String Pool (safe reuse/caching of literals), to allow safe use as `HashMap/HashSet` keys (hash code never changes), and for security (values can't be altered after validation).

Q4. What is the String Pool, and how does it differ from the regular heap? A: The String Pool is a special area of the heap that caches String *literals* so identical literal values are stored once and shared. `new String(...)` bypasses the pool and forces a fresh heap allocation, which is why reference (`==`) comparison behaves differently for literals vs. new-created Strings even when content is identical.

Q5. Difference between String, StringBuilder, and StringBuffer? A: String is immutable. `StringBuilder` is mutable but *not* thread-safe (faster). `StringBuffer` is mutable *and* thread-safe (its methods are synchronized, making it slightly slower than `StringBuilder`). Use `StringBuilder` unless you specifically need thread safety.

Q6. Why shouldn't you concatenate Strings with + inside a loop? A: Every + concatenation on immutable Strings creates a brand-new String object (the old ones become garbage), so doing this repeatedly inside a loop creates many short-lived objects and wastes CPU/memory — a `StringBuilder.append()` inside the loop, converted to a String once at the end, is far more efficient.

Q7. What does array.length return, and how is it different from String.length()? A: `array.length` is a **field** access (no parentheses) giving the array's fixed size. `String.length()` is a **method call** (with parentheses) returning the number of characters — a classic "gotcha" for beginners who mix the two up.

Q8. How would you reverse a String in Java? A: Several ways — manually iterate from the last character to the first building a new String/ `StringBuilder`, or simply use `new StringBuilder(str).reverse().toString()`, which is the idiomatic, most concise approach.

Q9. What's the time complexity of accessing an array element by index vs. searching for a value? A: Access by index is $O(1)$ — direct memory-address computation. Searching for a specific *value* (without knowing its index) is $O(n)$ via linear search unless the array is sorted, in which case binary search achieves $O(\log n)$ (see Chapter 11).

Chapter 3: Classes & Objects (Core OOP)

3.1 Object-Oriented Programming — Why?

Object-Oriented Programming (OOP) organizes software as a collection of **objects**, each bundling *data* (fields) with the *behaviour* that operates on that data (methods) — as opposed to procedural programming, where data and the functions that act on it are entirely separate, global, and easy to misuse. OOP gives us four pillars, all of which this training covers with dedicated demos:

1. **Encapsulation** — hiding internal state behind methods (this chapter).
2. **Inheritance** — reusing/extending behaviour from a parent class (Chapter 4).
3. **Polymorphism** — one interface, many implementations (Chapter 4 & 5).
4. **Abstraction** — exposing only essential details, hiding implementation complexity (Chapter 5).

Why does this matter in practice? Real systems evolve constantly — new requirements, new edge cases, new integrations. OOP's discipline (bundling data with behaviour, hiding internals, allowing extension without modification) is what makes large codebases *changeable* over years without collapsing into unmanageable spaghetti. This is why virtually every enterprise Java, Spring, and microservice codebase you'll encounter after this training is built on these same four pillars.

3.2 Class vs Object — Blueprint vs Instance

A **class** is a *blueprint/template* — it defines what fields and methods every object of that type will have, but a class itself occupies no memory for *instance* data until you actually instantiate it. An **object** is a concrete *instance* of a class, created using the `new` keyword, living on the **heap** at runtime.

```
Car myCar = new Car();    // myCar is a reference variable pointing to a Car object on the heap
```

Memory model, precisely:

- The reference variable `myCar` itself lives on the **stack** (if it's a local variable) and holds the *memory address* of the actual `Car` object.
- The actual `Car` object — its fields and their current values — lives on the **heap**.
- Multiple reference variables can point to the *same* object (`Car anotherRef = myCar;` — now both variables refer to the identical object; changing a field through one reference is visible through the other too, since there's only ever one object).

A class typically has:

- **Fields (instance variables):** the state/data — each object gets its own independent copy.
- **Constructors:** special methods (same name as the class, no return type — not even `void`) that initialize a new object's state at the moment of creation. If you write *no* constructor at all, Java automatically supplies a no-argument **default constructor**. The moment you write *any* constructor yourself, that automatic default disappears — a common source of confusion when code that relied on the implicit no-arg constructor suddenly fails to compile after a custom constructor is added.
- **Constructor overloading:** multiple constructors in the same class with different parameter lists, letting an object be created in more than one way (e.g., with all fields specified, or with some defaulted).
- **Methods:** the behaviour operating on the object's fields.
- **this keyword:** refers to the *current* object. Two common uses: disambiguating a field from a same-named constructor/method parameter (`this.name = name;`), and calling

another constructor of the same class from within a constructor (`this(...)`), must be the very first statement — known as **constructor chaining**).

3.3 Encapsulation — Data Hiding

Encapsulation means declaring fields **private** and exposing controlled access through **public getter and setter methods**. This lets a class:

- **Validate** data before accepting it (e.g., reject a negative age or an empty name inside the setter, rather than trusting external code to always pass sane values).
- **Change internal representation later** without breaking code that uses the class — e.g., internally switching from storing age as an `int` to computing it from a `birthDate`, while the public `getAge()` method's contract stays identical, so no calling code needs to change.
- **Control read/write access independently** — e.g., a read-only field can expose only a getter, with no corresponding setter at all.

This is sometimes summarized as: *“tell, don't ask”* — well-encapsulated objects expose behaviour (methods that do something), not raw internal state, whenever possible, minimizing how much outside code needs to know about an object's internals.

Access modifiers, from most to least restrictive:

Modifier	Same class	Same package	Subclass (different package)	Everywhere
<code>private</code>	Yes	No	No	No
(default/package-private, no keyword)	Yes	Yes	No	No
<code>protected</code>	Yes	Yes	Yes	No
<code>public</code>	Yes	Yes	Yes	Yes

The demo classes below (`Book`, `Calculator`, `Car`) each show fields, constructors (including overloaded ones), getters/setters, and a `Demo/tester` class with a `main` method that creates objects and calls their methods.

File: `ClassesAndObjectDemos/com/accenture/ltt/oops/Book.java`

```
package com.accenture.ltt.oops;

/**
 * Represents a book using constructor initialization.
 */
public class Book {
    String title;
    String author;

    // Constructor
    public Book(String t, String a) {
        title = t;
        author = a;
    }

    public void display() {
```

```

        System.out.println("Book: " + title + " by " + author);
    }
}

```

File: ClassesAndObjectDemos/com/accenture/ltt/oops/BookDemo.java

```

package com.accenture.ltt.oops;

/**
 * Demonstrates constructor usage with Book class.
 */
public class BookDemo {
    public static void main(String[] args) {
        Book b1 = new Book("Java Basics", "John Smith");
        Book b2 = new Book("OOP Concepts", "Jane Doe");

        b1.display();
        b2.display();
    }
}

```

File: ClassesAndObjectDemos/com/accenture/ltt/oops/Calculator.java

```

package com.accenture.ltt.oops;

/**
 * A simple calculator class with basic operations.
 */
public class Calculator {

    public int add(int a, int b) {
        return a + b;
    }

    public int subtract(int a, int b) {
        return a - b;
    }

    public int multiply(int a, int b) {
        return a * b;
    }
}

```

File: ClassesAndObjectDemos/com/accenture/ltt/oops/CalculatorDemo.java

```

package com.accenture.ltt.oops;

/**
 * Demonstrates the Calculator class with multiple methods.
 */
public class CalculatorDemo {
    public static void main(String[] args) {
        Calculator calc = new Calculator();
        System.out.println("Addition: " + calc.add(10, 5));
        System.out.println("Subtraction: " + calc.subtract(10, 5));
        System.out.println("Multiplication: " + calc.multiply(10, 5));
    }
}

```



```
}  
}
```

File: ClassesAndObjectDemos/com/accenture/ltt/oops/Car.java

```
package com.accenture.ltt.oops;  
  
/**  
 * This class represents a Car with a color and a behavior to drive.  
 */  
public class Car {  
    // Fields / attributes  
    String color;  
    String brand;  
  
    // Method / behavior  
    public void drive() {  
        System.out.println(brand + " car is driving in " + color + " color.");  
    }  
}
```

File: ClassesAndObjectDemos/com/accenture/ltt/oops/CarDemo.java

```
package com.accenture.ltt.oops;  
  
/**  
 * This class demonstrates how to create and use objects from the Car class.  
 */  
public class CarDemo {  
    public static void main(String[] args) {  
        // Creating first car object  
        Car car1 = new Car();  
        car1.color = "Red";  
        car1.brand = "Toyota";  
        car1.drive(); // Output: Toyota car is driving in Red color.  
  
        // Creating second car object  
        Car car2 = new Car();  
        car2.color = "Blue";  
        car2.brand = "BMW";  
        car2.drive(); // Output: BMW car is driving in Blue color.  
    }  
}
```

3.4 Practice Activities

Hands-on exercises reinforcing class design, constructors, and encapsulation with Car and Employee classes.

File: ClassesAndObjectDemos/com/accenture/ltt/oopsActivity/ActivityOne.java

```
package com.accenture.ltt.oopsActivity;  
  
/**  
 * Activity 1: Define a Student class with name, age and a method to display details.
```

```

*/
public class ActivityOne {

    // TODO: Define class 'Student' with fields: name (String), age (int)
    // TODO: Add a method called displayDetails() that prints student name and age

    public static void main(String[] args) {
        // TODO: Create 2 Student objects
        // TODO: Set their names and ages
        // TODO: Call displayDetails() for both
    }
}

```

File: ClassesAndObjectDemos/com/accenture/ltt/oopsActivity/ActivityTwo.java

```

package com.accenture.ltt.oopsActivity;

/**
 * Activity 2: Define a BankAccount class with account number and balance, and a method to show
 */
public class ActivityTwo {

    // TODO: Create class 'BankAccount' with fields: accountNumber (String), balance (double)
    // TODO: Add method displayBalance() that prints account details

    public static void main(String[] args) {
        // TODO: Create 2 BankAccount objects
        // TODO: Set accountNumber and balance
        // TODO: Call displayBalance() for both
    }
}

```

File: ClassesAndObjectDemos/com/accenture/ltt/oopsActivity/Car.java

```

package com.accenture.ltt.oopsActivity;

public class Car {

    // Member variables
    String model;
    String color;
    String manufacturer;

    // You can optionally add constructors, getters, and setters here later
}

```

File: ClassesAndObjectDemos/com/accenture/ltt/oopsActivity/Employee.java

```

package com.accenture.ltt.oopsActivity;

public class Employee {

    // Instance variables
    int employeeId;
    int age;
    double salary;
}

```

```
double variablePay;

// You can later add constructors, methods, or getters/setters here
}
```

File: ClassesAndObjectDemos/com/accenture/ltt/oopsActivity/TestCar.java

```
package com.accenture.ltt.oopsActivity;

public class TestCar {

    public static void main(String[] args) {
        // Create Car objects here

        // Assign values to each object's instance variables

        // Print the values using dot operator
    }
}
```

File: ClassesAndObjectDemos/com/accenture/ltt/oopsActivity/TestEmployee.java

```
package com.accenture.ltt.oopsActivity;

public class TestEmployee {

    public static void main(String[] args) {
        // Create Employee objects here

        // Assign values to instance variables

        // Print or display employee details
    }
}
```

3.5 Viva Voce — Likely Questions & Answers

Q1. What is the difference between a class and an object? A: A class is a compile-time blueprint/template — it defines structure but consumes no memory for instance data by itself. An object is a runtime *instance* of that class, created with `new`, occupying actual memory on the heap with its own copy of the instance fields.

Q2. Where do objects live in memory, versus their reference variables? A: The object itself (its fields and data) lives on the **heap**. The reference variable pointing to it (if it's a local variable) lives on the **stack**, holding just the memory address of the heap object.

Q3. What happens if a class defines no constructor at all? A: Java automatically provides a public, no-argument **default constructor** that does nothing beyond default field initialization. The moment you write *any* constructor yourself, this automatic one disappears.

Q4. What is constructor overloading? A: Defining multiple constructors in the same class with different parameter lists (different number/type of parameters), allowing an object to be constructed in more than one way.

Q5. What is constructor chaining, and how do you do it? A: Calling one constructor from another *within the same class* using `this(...)`, which must be the first statement in the calling constructor — avoids duplicating initialization logic across multiple constructors.

Q6. Why do we make fields private and expose getters/setters instead of making fields public directly? A: This is Encapsulation — it lets the class validate incoming data, hide/change its internal representation without breaking external code, and control whether a field is read-only, write-only, or both, independent of each other.

Q7. What is the difference between private, default (package-private), protected, and public access modifiers? A: private — only the same class. Default (no keyword) — same package only. protected — same package, plus subclasses even in other packages. public — accessible from anywhere.

Q8. Can two reference variables point to the same object? What happens if you modify a field through one of them? A: Yes — assigning one reference variable to another (Car b = a;) makes both point to the *same* heap object. Since there's only one actual object, modifying a field through either reference is immediately visible through the other, because both are looking at the same data.

Q9. What is this used for? A: To refer to the *current* object — most commonly to disambiguate an instance field from a same-named parameter (this.name = name;), and to call another constructor in the same class (constructor chaining).

Q10. What are the four pillars of OOP, and which Java mechanism implements each? A: Encapsulation (private fields + public getters/setters), Inheritance (extends), Polymorphism (method overriding/overloading, dynamic binding), and Abstraction (abstract classes and interfaces).

Chapter 4: Class Relationships — Inheritance, Polymorphism, Composition & Aggregation

4.1 Inheritance — Reusing and Extending Behaviour

Inheritance lets one class (**subclass/child/derived class**) acquire the fields and methods of another (**superclass/parent/base class**) using the `extends` keyword. It models an “**is-a**” relationship (e.g., a Manager *is an* Employee, a Car *is a* Vehicle). Benefits: code reuse (shared behaviour written once in the parent), and the ability to override/specialize behaviour in children.

Key rules and mechanics:

- Java supports **single inheritance** for classes — a class can extend only **one** other class. This deliberate restriction avoids the “Diamond Problem” (ambiguity when a class would inherit conflicting implementations of the same method from two different parents) that exists in languages like C++.
- Java **does** support multiple inheritance **of interfaces** — a class can implement several interfaces at once (Chapter 5), because interfaces (pre-Java 8) contained no implementation to conflict over, and even now with default methods, Java has explicit rules requiring the implementing class to resolve any clash itself.
- **super keyword** has two uses: (1) `super(...)` calls the parent class’s constructor — if used, it **must be the first statement** in the child’s constructor; if you don’t call it explicitly, Java automatically inserts a call to the parent’s *no-argument* constructor for you (which fails to compile if the parent has no no-arg constructor available); (2) `super.method()` explicitly invokes the parent’s version of a method the child has overridden.
- **Method Overriding** — the child redefines a method with the **exact same signature** (name, parameter types, and return type, or a covariant return type) as one in the parent. This is the mechanism behind **runtime polymorphism** (4.2). Rules: the overriding method cannot reduce the visibility of the parent’s method (can’t override a public method as private), and cannot throw new/broader checked exceptions than the parent version.
- **Method Overloading** — multiple methods in the **same class** (or class hierarchy) with the **same name but different parameter lists** (different number, types, or order of parameters) — this is resolved entirely at **compile time** based on the declared argument types, which is why it’s called **compile-time/static polymorphism**.

Overriding vs Overloading — the single most commonly asked OOP interview question:

	Overriding	Overloading
Where	Between parent and child class	Within the same class (or inherited)
Signature	Must be identical	Must differ (params)
Binding	Dynamic (runtime)	Static (compile-time)
Return type	Same or covariant	Can differ freely
Purpose	Specialize inherited behaviour	Provide convenience variants of an operation

4.2 Polymorphism — “Many Forms”

Polymorphism allows a single reference type to behave differently based on the *actual object* it refers to at runtime, not the *declared type* of the reference variable.

```
Vehicle v = new Car();    // Vehicle reference, but a Car object
v.start();                // calls Car's overridden start() – decided at RUNTIME, not compile
```

This works because the JVM maintains, for every object, a hidden pointer to its **actual class's method table**. When you call an overridden method through a parent-typed reference, the JVM looks up *that specific object's* method table at the moment of the call (not the reference's declared type) — this lookup is what “dynamic binding” (Chapter 5) refers to.

Why is this useful? It lets you write code against the general parent type (`List<Vehicle> fleet`) while still getting each object's specific, correct behaviour automatically — you don't need `if (v instanceof Car) ... else if (v instanceof Bus) ...` checks scattered through your code; the right method simply runs.

This is demonstrated below using a `Vehicle → Car/Bus` hierarchy and an `Employee → Manager` hierarchy, where an array/list of the parent type holds different subclass objects and the correct overridden method fires for each.

File: Activity_Polymorphism_Inheritance/com/accenture/ltt/activity/Bus.java

```
package com.accenture.ltt.activity;

public class Bus extends Vehicle {

    // No-argument constructor
    public Bus() {
        // SOP statement (e.g., System.out.println("Bus constructor called"));
    }

    // Implement abstract method drive()
    @Override
    public void drive() {
        // SOP statement (e.g., System.out.println("Bus is being driven"));
    }
}
```

File: Activity_Polymorphism_Inheritance/com/accenture/ltt/activity/Car.java

```
package com.accenture.ltt.activity;

public class Car extends Vehicle {

    // No-argument constructor
    public Car() {
        // SOP statement (e.g., System.out.println("Car constructor called"));
    }

    // Implement abstract method drive()
    @Override
    public void drive() {
        // SOP statement (e.g., System.out.println("Car is being driven"));
    }
}
```

File: Activity_Polymorphism_Inheritance/com/accenture/ltt/activity/Employee.java

```
package com.accenture.ltt.activity;
```

```

public class Employee {

    // Instance variables
    int employeeId;
    int age;
    double salary;
    double variablePay;

    // Method to print employee details
    public void printEmployeeDetails() {
        // Print employee details here
    }
}

```

File: Activity_Polymorphism_Inheritance/com/accenture/ltt/activity/Manager.java

```

package com.accenture.ltt.activity;

public class Manager extends Employee {

    // Additional instance variable
    int numberOfReportees;

    // Getter method
    public int getNumberOfReportees() {
        // return numberOfReportees;
        return 0; // placeholder
    }

    // Setter method
    public void setNumberOfReportees(int numberOfReportees) {
        // this.numberOfReportees = numberOfReportees;
    }

    // Overriding printEmployeeDetails method
    @Override
    public void printEmployeeDetails() {
        // Call super.printEmployeeDetails();
        // Print numberOfReportees here
    }
}

```

File: Activity_Polymorphism_Inheritance/com/accenture/ltt/activity/TestEmployee.java

```

package com.accenture.ltt.activity;

public class TestEmployee {

    public static void main(String[] args) {
        // Create Manager object
        Manager manager = new Manager();

        // Set values for inherited fields (e.g., employeeId, salary, etc.)

        // Set numberOfReportees using setter method
    }
}

```

```

        // Call printEmployeeDetails() using Manager object
    }
}

```

File: Activity_Polymorphism_Inheritance/com/accenture/ltt/activity/TestVehicle.java

```

package com.accenture.ltt.activity;

public class TestVehicle {

    public static void main(String[] args) {
        // Create object of Car class
        Car car = new Car();
        // Invoke drive() method
        car.drive();

        // Create object of Bus class
        Bus bus = new Bus();
        // Invoke drive() method
        bus.drive();
    }
}

```

File: Activity_Polymorphism_Inheritance/com/accenture/ltt/activity/Vehicle.java

```

package com.accenture.ltt.activity;

public abstract class Vehicle {

    // No-argument constructor
    public Vehicle() {
        // SOP statement (e.g., System.out.println("Vehicle constructor called"));
    }

    // Abstract method
    public abstract void drive();
}

```

File: Demo100_Composition/com/accenture/ltt/activity/CompositionTester.java

```

package com.accenture.ltt.activity;

public class CompositionTester {

    public static void main(String[] args) {
        //Create a Student Object and Set the Values
        System.out.println("Student details are:");
        //Get Both Student and course details for particular student
    }
}

```

File: Demo100_Composition/com/accenture/ltt/activity/Course.java

```

package com.accenture.ltt.activity;

```



```

public class Course {
    private String courseName;
    private int courseId;

    public String getCourseName() {
        return courseName;
    }

    public void setCourseName(String courseName) {
        this.courseName = courseName;
    }

    public int getCourseId() {
        return courseId;
    }

    public void setCourseId(int courseId) {
        this.courseId = courseId;
    }
}

```

File: Demo100_Composition/com/accenture/ltt/activity/Student.java

```

package com.accenture.ltt.activity;

public class Student {
    private String studentName;
    private Course course;

    public Student() {
        //Create a object of Course and invoke setters to set the values
    }

    public String getStudentName() {
        return studentName;
    }

    public void setStudentName(String studentName) {
        this.studentName = studentName;
    }

    public Course getCourse() {
        return course;
    }
}

```

File: Demo101_Aggregation/com/accenture/ltt/activity/AggregationTester.java

```

package com.accenture.ltt.activity;

public class AggregationTester {
    public static void main(String[] args) {
        // 1. Create a Object of Coach and Invoke Setters
        // 2. Create a Object of Team and Invoke Setters
        // 3. Pass Coach Instance to the Team
    }
}

```

```
// Aggregation: team has a coach
// Display all the details
}
}
```

File: Demo101_Aggregation/com/accenture/ltt/activity/Coach.java

```
package com.accenture.ltt.activity;

public class Coach {
    private String coachName;
    private int coachId;

    public String getCoachName() {
        return coachName;
    }

    public void setCoachName(String coachName) {
        this.coachName = coachName;
    }

    public int getCoachId() {
        return coachId;
    }

    public void setCoachId(int coachId) {
        this.coachId = coachId;
    }
}
```

File: Demo101_Aggregation/com/accenture/ltt/activity/Team.java

```
package com.accenture.ltt.activity;

public class Team {
    private String teamName;
    private Coach coach;

    public String getTeamName() {
        return teamName;
    }

    public void setTeamName(String teamName) {
        this.teamName = teamName;
    }

    public Coach getCoach() {
        return coach;
    }

    public void setCoach(Coach coach) {
        this.coach = coach;
    }
}
```

4.3 Composition (“has-a”, strong ownership)

Composition models a **strong “has-a”** relationship where the contained (child) object’s lifecycle is entirely owned by, and bound to, the container — if the container object is destroyed/garbage collected, the contained object has no independent existence or reason to exist either. Example: a Course composed of a Student created *within* the owning class’s own constructor — the Student detail object here belongs exclusively to that one Course instance.

Implementation signature: the contained object is instantiated **inside** the owning class, typically directly in its constructor or field initializer — the owning class controls the contained object’s entire lifecycle from birth.

Real-world analogies: a House and its Rooms (a Room doesn’t meaningfully exist detached from its specific house); an Order and its OrderLineItems.

File: Demo100_Composition/com/accnture/ltt/Account.java

```
package com.accnture.ltt;

public class Account {
    private int accountNo;
    private double amount;

    public int getAccountNo() {
        return accountNo;
    }
    public void setAccountNo(int accountNo) {
        this.accountNo = accountNo;
    }
    public double getAmount() {
        return amount;
    }
    public void setAmount(double amount) {
        this.amount = amount;
    }
}
```

File: Demo100_Composition/com/accnture/ltt/CompositionTester.java

```
package com.accnture.ltt;

public class CompositionTester {

    public static void main(String[] args) {
        Customer customer = new Customer();

        customer.setCustomerName("John");

        System.out.println("Account details are:");
        System.out.println("Customer Name: " + customer.getCustomerName());
        System.out.println("Account Number: " + customer.getAccount().getAccountNo()); // Pay a
        // unde
        System.out.println("Balance Amount: " + customer.getAccount().getAmount()); // Pay atte
        // understa
    }
```

```
}
```

File: Demo100_Composition/com/accenture/ltt/Customer.java

```
package com.accenture.ltt;

public class Customer {
    private String customerName;
    private Account account;

    public Customer(){
        account = new Account();
        account.setAccountNo(9001);
        account.setAmount(5000.0);
    }
    public String getCustomerName() {
        return customerName;
    }
    public void setCustomerName(String customerName) {
        this.customerName = customerName;
    }
    public Account getAccount() {
        return account;
    }
}
```

File: Demo100_Composition/com/accenture/ltt/activity/CompositionTester.java

```
package com.accenture.ltt.activity;

public class CompositionTester {

    public static void main(String[] args) {
        //Create a Student Object and Set the Values
        System.out.println("Student details are:");
        //Get Both Student and course details for particular student
    }
}
```

File: Demo100_Composition/com/accenture/ltt/activity/Course.java

```
package com.accenture.ltt.activity;

public class Course {
    private String courseName;
    private int courseId;

    public String getCourseName() {
        return courseName;
    }

    public void setCourseName(String courseName) {
        this.courseName = courseName;
    }
}
```

```

    }

    public int getCourseId() {
        return courseId;
    }

    public void setCourseId(int courseId) {
        this.courseId = courseId;
    }
}

```

File: Demo100_Composition/com/accenture/ltt/activity/Student.java

```

package com.accenture.ltt.activity;

public class Student {
    private String studentName;
    private Course course;

    public Student() {
        //Create a object of Course and invoke setters to set the values
    }

    public String getStudentName() {
        return studentName;
    }

    public void setStudentName(String studentName) {
        this.studentName = studentName;
    }

    public Course getCourse() {
        return course;
    }
}

```

4.4 Aggregation (“has-a”, weak ownership)

Aggregation is a **weaker “has-a”** relationship: the contained object *can* exist independently of the container, may have been created *before* the container even existed, and can even be **shared** across multiple containers simultaneously. Example: a Department *has* Faculty members, but a Faculty object is created independently and would continue to exist (e.g., transfer to a different department) even if this particular Department object were discarded.

Implementation signature: the contained object is **passed in from outside** (via a constructor parameter or a setter method) rather than instantiated internally — this is the concrete code-level difference that distinguishes aggregation from composition, even though both look superficially similar (“one class holds a reference to another”).

Composition vs Aggregation — summary table (frequently tested):

	Composition	Aggregation
Ownership strength	Strong (“owns”)	Weak (“uses”/“has”)
Contained object’s lifecycle	Tied to the container	Independent

	Composition	Aggregation
Can be shared across containers?	No	Yes
Typical implementation	Instantiated inside the owner	Passed in via constructor/setter
Example	Car and its Engine	Department and its Faculty

The demos below use a Department/Faculty and a Team/Coach example for aggregation.

File: Demo101_Aggregation/com/accenture/ltt/AggregationTester.java

```
package com.accenture.ltt;

public class AggregationTester {
    public static void main(String[] args){
        Department department = new Department();
        department.setDepartmentName("Computer Science");

        Faculty faculty = new Faculty();
        faculty.setFacultyId(1001);
        faculty.setFacultyName("Smith");
        department.setFaculty(faculty);

        System.out.println("Department Name: " + department.getDepartmentName());
        System.out.println("Faculty Name: " + department.getFaculty().getFacultyName());
        System.out.println("Faculty ID: " + department.getFaculty().getFacultyId());
    }
}
```

File: Demo101_Aggregation/com/accenture/ltt/Department.java

```
package com.accenture.ltt;

public class Department {
    private String departmentName;
    private Faculty faculty;

    public String getDepartmentName() {
        return departmentName;
    }
    public void setDepartmentName(String departmentName) {
        this.departmentName = departmentName;
    }
    public Faculty getFaculty() {
        return faculty;
    }
    public void setFaculty(Faculty faculty) {
        this.faculty = faculty;
    }
}
```

File: Demo101_Aggregation/com/accenture/ltt/Faculty.java

```

package com.accenture.ltt;

public class Faculty {
    private String facultyName;
    private int facultyId;

    public String getFacultyName() {
        return facultyName;
    }
    public void setFacultyName(String facultyName) {
        this.facultyName = facultyName;
    }
    public int getFacultyId() {
        return facultyId;
    }
    public void setFacultyId(int facultyId) {
        this.facultyId = facultyId;
    }
}

```

File: Demo101_Aggregation/com/accenture/ltt/activity/AggregationTester.java

```

package com.accenture.ltt.activity;

public class AggregationTester {
    public static void main(String[] args) {
        // 1. Create a Object of Coach and Invoke Setters
        // 2. Create a Object of Team and Invoke Setters
        // 3. Pass Coach Instance to the Team
        // Aggregation: team has a coach
        // Display all the details
    }
}

```

File: Demo101_Aggregation/com/accenture/ltt/activity/Coach.java

```

package com.accenture.ltt.activity;

public class Coach {
    private String coachName;
    private int coachId;

    public String getCoachName() {
        return coachName;
    }

    public void setCoachName(String coachName) {
        this.coachName = coachName;
    }

    public int getCoachId() {
        return coachId;
    }

    public void setCoachId(int coachId) {

```

```

        this.coachId = coachId;
    }
}

```

File: Demo101_Aggregation/com/accenture/ltt/activity/Team.java

```

package com.accenture.ltt.activity;

public class Team {
    private String teamName;
    private Coach coach;

    public String getTeamName() {
        return teamName;
    }

    public void setTeamName(String teamName) {
        this.teamName = teamName;
    }

    public Coach getCoach() {
        return coach;
    }

    public void setCoach(Coach coach) {
        this.coach = coach;
    }
}

```

4.5 Viva Voce — Likely Questions & Answers

Q1. What is the difference between method overloading and method overriding? A: Overloading is multiple methods in the *same* class with the same name but *different parameter lists*, resolved at *compile time*. Overriding is a *subclass* redefining a parent method with the *identical signature*, resolved at *runtime* based on the actual object type.

Q2. Why does Java not support multiple inheritance of classes? A: To avoid the “Diamond Problem” — ambiguity over which parent’s implementation should apply if two parent classes provided conflicting implementations of the same inherited method. Java sidesteps this entirely by allowing a class to extend only one class, while still allowing multiple interfaces to be implemented.

Q3. What is the difference between composition and aggregation? A: Both are “has-a” relationships, but composition implies *strong ownership* (the contained object’s lifecycle is tied to the container, typically instantiated inside it), while aggregation implies *weak ownership* (the contained object exists independently and is passed in from outside, possibly shared across multiple containers).

Q4. What is runtime polymorphism, and what makes it possible? A: Calling an overridden method through a parent-type reference and having the *actual object’s* version execute — made possible by dynamic (late) binding, where the JVM resolves which method body to run based on the real object’s type at the moment of the call, not the reference’s declared type.

Q5. If class B extends class A, and both define a method foo(), which one runs when called via an A reference pointing to a B object? A: B’s version runs — because instance

method calls on non-final/non-static/non-private methods are dynamically bound to the actual object's type, not the reference's declared type.

Q6. What must you keep in mind when overriding a method regarding access modifiers and exceptions? A: The overriding method cannot have a *more restrictive* access modifier than the parent's version (e.g., can't override public as private), and cannot declare new or broader checked exceptions than the parent method declared.

Q7. What happens if a child class constructor doesn't explicitly call `super(...)`? A: The compiler automatically inserts an implicit call to the parent's no-argument constructor as the very first statement. If the parent class has no no-argument constructor available, this causes a compile error, forcing you to call `super(...)` with matching arguments explicitly.

Q8. Give a real-world example each of composition and aggregation. A: Composition: a House and its Rooms — rooms don't exist independently of that specific house. Aggregation: a Department and its Faculty — a faculty member exists independently and could move to a different department.

Q9. Can a static method be overridden? A: No — static methods are resolved via **static/early binding** based on the reference type at compile time, not the actual object at runtime, so a subclass "redefining" a static method with the same signature is actually **method hiding**, not overriding (further explored in Chapter 5).

Chapter 5: Static/Dynamic Binding, Abstraction & Interfaces

5.1 Static Binding vs Dynamic Binding

Binding refers to when the JVM decides *which method implementation to actually call* for a given method invocation expression.

	Static (Early) Binding	Dynamic (Late) Binding
Resolved at	Compile time	Runtime
Applies to	private, static, and final methods; overloaded methods; fields (field access is always statically bound in Java!)	Overridden (@Override) instance methods, accessed via any reference
Mechanism	Compiler resolves the call directly based on the reference's declared type	JVM consults the actual object's runtime method table (a mechanism sometimes called a "vtable")
Speed	Faster — no lookup needed at runtime	Marginally slower — needs a runtime lookup, though JIT optimizes this heavily in practice
Example	<code>obj.staticMethod()</code> behaves per reference type	<code>parentRef.overriddenMethod()</code> behaves per actual object type

A crucial, often-overlooked detail: in Java, **fields (instance variables) are always resolved via static binding**, even if a subclass declares a field with the same name as one in the parent (known as "field hiding" or "field shadowing") — this is very different from method overriding, and is a classic "gotcha" question: accessing `parentRef.someField` always returns the *parent's* field value, even if `parentRef` actually points to a child object that redeclared that field name, because field access doesn't go through the dynamic dispatch mechanism that methods do.

Static binding is faster (no lookup needed) but inflexible; dynamic binding is what makes runtime polymorphism possible.

5.2 Abstraction

Abstraction means exposing *only what is necessary* to the outside world and hiding internal implementation complexity. Java offers two mechanisms:

1. **Abstract classes** (`abstract class`) — can contain both **abstract** (unimplemented, `abstract returnType methodName();` with no body) and **concrete** (fully implemented) methods. Cannot be instantiated directly with `new` (you can't create "a generic Shape" — only a specific `Circle` or `Triangle`). A subclass must implement *all* inherited abstract methods, or itself be declared abstract too (deferring the obligation further down the hierarchy).
2. **Interfaces** (`interface`) — a pure behavioural contract: (historically, pre-Java 8) *only* abstract method signatures, with **no** implementation and **no** instance fields (only public static final constants). Since Java 8, interfaces can also carry default and static methods with bodies (explored further in Chapter 13).

Abstract class vs Interface — the classic comparison table:

	Abstract Class	Interface
Keyword	<code>abstract class</code>	<code>interface</code>
Method bodies	Can mix abstract + concrete	Traditionally none; default/static allowed since Java 8
Fields	Any kind (instance, static, final, etc.)	Only public static final constants
Multiple inheritance	A class can extend only ONE abstract class	A class can implement MANY interfaces
Constructors	Yes (called via subclass's <code>super()</code>)	No
When to use	When subclasses share significant common code/state ("is-a" with shared implementation)	When unrelated classes need to guarantee the same capability/contract ("can-do")

5.3 Interfaces in Practice

An interface defines *what* a class must do, not *how*. Any class using implements Interface-Name must provide concrete bodies for all abstract methods in that interface (or be declared abstract itself). A class can implement **multiple interfaces** simultaneously — this is precisely how Java compensates for disallowing multiple class inheritance, since interfaces (at least pre-default- methods) carried no implementation to create ambiguity over.

Interface as a “contract” — why this matters architecturally: in real systems, code is written against interface types (e.g., `List<String> list = new ArrayList<>();` rather than `ArrayList<String> list = new ArrayList<>();`), so the concrete implementation can be swapped later (e.g., to a `LinkedList`) with zero changes to any code that only ever referred to the `List` interface type. This principle — “program to an interface, not an implementation” — underlies virtually all of Spring’s dependency injection design later in this course.

The demos below use a `Shape` interface implemented by `Circle` and `Triangle` (classic abstraction/ polymorphism example), and `Employee/Manager` classes to concretely contrast **static binding** (with static methods resolved by reference type) versus **dynamic binding** (overridden instance methods resolved by actual object type at runtime).

File: `Activity/com/accenture/ltt/Circle.java`

```
package com.accenture.ltt;

public class Circle implements Shape {

    // Implement draw() method
    @Override
    public void draw() {
        // SOP statement (e.g., System.out.println("Drawing Circle"));
    }

    // Additional method
    public void rotate() {
        // SOP statement (e.g., System.out.println("Rotating Circle"));
    }
}
```

File: Activity/com/accenture/ltt/Employee.java

```
package com.accenture.ltt;

public class Employee {

    // Method to calculate salary
    public void calculateSalary() {
        // SOP statement (e.g., System.out.println("Employee salary calculated"));
    }

    // Method to get bonus
    public void getBonus() {
        // SOP statement (e.g., System.out.println("Employee bonus calculated"));
    }
}
```

File: Activity/com/accenture/ltt/Manager.java

```
package com.accenture.ltt;

public class Manager extends Employee {

    // Overriding calculateSalary() method
    @Override
    public void calculateSalary() {
        // SOP statement (e.g., System.out.println("Manager salary calculated"));
    }

    // Overriding getBonus() method
    @Override
    public void getBonus() {
        // SOP statement (e.g., System.out.println("Manager bonus calculated"));
    }
}
```

File: Activity/com/accenture/ltt/Shape.java

```
package com.accenture.ltt;

public interface Shape {
    // Abstract method
    void draw();
}
```

File: Activity/com/accenture/ltt/TestEmployee.java

```
package com.accenture.ltt;

public class TestEmployee {

    public static void main(String[] args) {

        // Create Employee reference and Manager object
        Employee emp = new Manager();

        // Call calculateSalary() method
    }
}
```

```

        emp.calculateSalary(); // Notice which class's method is executed

        // Call getBonus() method
        emp.getBonus(); // Notice which class's method is executed
    }
}

```

File: Activity/com/accenture/ltt/TestShape.java

```

package com.accenture.ltt;

public class TestShape {

    public static void main(String[] args) {

        // Create Circle object and call methods
        Circle circle = new Circle();
        circle.draw();
        circle.rotate();

        // Create Triangle object and call methods
        Triangle triangle = new Triangle();
        triangle.draw();
        triangle.calculatePerimeter();

        // Shape reference pointing to Circle object
        Shape shape1 = new Circle();
        shape1.draw();
        // shape1.rotate(); // Not accessible via Shape reference

        // Shape reference pointing to Triangle object
        Shape shape2 = new Triangle();
        shape2.draw();
        // shape2.calculatePerimeter(); // Not accessible via Shape reference
    }
}

```

File: Activity/com/accenture/ltt/Triangle.java

```

package com.accenture.ltt;

public class Triangle implements Shape {

    // Implement draw() method
    @Override
    public void draw() {
        // SOP statement (e.g., System.out.println("Drawing Triangle"));
    }

    // Additional method
    public void calculatePerimeter() {
        // SOP statement (e.g., System.out.println("Calculating perimeter of Triangle"));
    }
}

```

File: Demo102_StaticBinding/com/accnture/ltt/Employee.java

```
package com.accnture.ltt;

public class Employee {
    private int employeeId;
    private String employeeName;

    public Employee(String employeeName, int employeeId) {
        this.employeeId = employeeId;
        this.employeeName = employeeName;
    }

    public int getEmployeeId() {
        return employeeId;
    }

    public void setEmployeeId(int employeeId) {
        this.employeeId = employeeId;
    }

    public String getEmployeeName() {
        return employeeName;
    }

    public void setEmployeeName(String employeeName) {
        this.employeeName = employeeName;
    }

    public void displayDetails() {
        System.out.println("Employee Name: " + this.employeeName);
        System.out.println("Employee Id: " + this.employeeId);
    }
}
```

File: Demo102_StaticBinding/com/accnture/ltt/Manager.java

```
package com.accnture.ltt;

public class Manager extends Employee {
    private int numberOfReportees;

    public Manager(String employeeName, int employeeId, int numberOfReportees) {
        super(employeeName, employeeId);
        this.numberOfReportees = numberOfReportees;
    }

    public int getNumberOfReportees() {
        return numberOfReportees;
    }

    public void setNumberOfReportees(int numberOfReportees) {
        this.numberOfReportees = numberOfReportees;
    }
}
```

```

    public void displayDetails() {
        System.out.println("Number of reportees: " + this.numberOfReportees);
    }
}

```

File: Demo102_StaticBinding/com/accenture/ltt/StaticBindingTester.java

```

package com.accenture.ltt;

public class StaticBindingTester {
    public static void main(String[] args) {
        Manager manager = new Manager("Cooper", 3001, 4);
        manager.displayDetails(); // Modify the code to display employee name and employee id
        System.out.println(manager.getEmployeeId());
        System.out.println(manager.getEmployeeName());
        System.out.println(manager.getNumberOfReportees());
    }
}

```

File: Demo103_DynamicBinding/com/accenture/ltt/DynamicBindingTester.java

```

package com.accenture.ltt;

public class DynamicBindingTester {
    public static void main(String[] args){
        Employee manager = new Manager("Cooper",3001,4);
        manager.displayDetails(); //Modify the code to display employee name and employee id
        System.out.println(manager.getEmployeeId());
        System.out.println(manager.getEmployeeName());
        // System.out.println(manager.getNumberOfReportees()); // uncomment and try

    }
}

```

File: Demo103_DynamicBinding/com/accenture/ltt/activity/DynamicBindingTester.java

```

package com.accenture.ltt.activity;

public class DynamicBindingTester {
    public static void main(String[] args){

        // This Activity will help in understanding dynamic binding and Polymorphism
        // 1. In the main() method, create a Employee Class reference and Manager Object
        // 2. Call the calculateSalary() method and notice which class's method is called
        // 3. Call the getBonus() method and observe the output

    }
}

```

File: Demo103_DynamicBinding/com/accenture/ltt/activity/Employee.java

```

package com.accenture.ltt.activity;

public class Employee {
    private int employeeId;
    private String employeeName;
}

```

```

public Employee(String employeeName, int employeeId) {
    this.employeeId = employeeId;
    this.employeeName = employeeName;
}

public int getEmployeeId() {
    return employeeId;
}

public void setEmployeeId(int employeeId) {
    this.employeeId = employeeId;
}

public String getEmployeeName() {
    return employeeName;
}

public void setEmployeeName(String employeeName) {
    this.employeeName = employeeName;
}

public void displayDetails() {
    System.out.println("Employee Name: " + this.employeeName);
    System.out.println("Employee Id: " + this.employeeId);
}
}

```

File: Demo103_DynamicBinding/com/accenture/ltt/activity/Manager.java

```

package com.accenture.ltt.activity;

public class Manager extends Employee {
    private int numberOfReportees;

    public Manager(String employeeName, int employeeId, int numberOfReportees) {
        super(employeeName, employeeId);
        this.numberOfReportees = numberOfReportees;
    }

    public int getNumberOfReportees() {
        return numberOfReportees;
    }

    public void setNumberOfReportees(int numberOfReportees) {
        this.numberOfReportees = numberOfReportees;
    }

    public void displayDetails() {
        System.out.println("Number of reportees: " + this.numberOfReportees);
    }
}

```


5.4 Viva Voce — Likely Questions & Answers

Q1. What is the difference between static binding and dynamic binding? A: Static binding resolves *which method to call* at compile time, based on the reference's declared type — applies to private, static, and final methods. Dynamic binding resolves it at runtime, based on the actual object's type — applies to overridden instance methods, and is the mechanism behind runtime polymorphism.

Q2. Are fields (instance variables) statically or dynamically bound in Java? A: Statically bound — always. Even if a subclass “hides” a parent field by redeclaring one with the same name, accessing that field through a parent-typed reference always returns the *parent's* field, regardless of the actual runtime object type. This is different from method overriding and is a classic exam trap.

Q3. Can a static method be overridden? What actually happens if a subclass declares a static method with the same signature as the parent's? A: No, it cannot truly be *overridden* — this is called **method hiding**. Which version runs is decided at compile time based on the reference type, not the actual object, unlike true (instance method) overriding.

Q4. What is the difference between an abstract class and an interface? A: An abstract class can mix abstract and concrete methods and any kind of field, supports constructors, but a class can extend only one. An interface (traditionally) only declared method signatures and constants, has no constructors, but a class can implement many interfaces at once. Since Java 8, interfaces can also have default/static method bodies, narrowing (but not eliminating) this gap.

Q5. Can you instantiate an abstract class directly? A: No — `new AbstractClassName()` does not compile. You can only instantiate a concrete subclass that has implemented all abstract methods (or an anonymous inner subclass that supplies bodies for them).

Q6. Why can a class implement multiple interfaces but extend only one class? A: Because interfaces (historically) carried no implementation, there was no ambiguity if a class implemented several — there was nothing conflicting to inherit. Classes carry actual field/method implementation, so allowing multiple class inheritance risks the “Diamond Problem” ambiguity, which Java deliberately avoids.

Q7. What does “program to an interface, not an implementation” mean, and why is it good practice? A: It means declaring variables/parameters/return types using the interface type (e.g. `List`) rather than a specific concrete class (e.g. `ArrayList`), so the actual implementation can be swapped later without changing any code that depends only on the interface's contract — this underlies dependency injection patterns used heavily in Spring.

Q8. What kind of fields can an interface have? A: Only public static final (i.e., constants) — even if you omit these modifiers explicitly, the compiler adds them implicitly, since interface fields cannot represent per-instance mutable state.

Chapter 6: Method-1 Coding Standards

6.1 Why Coding Standards Matter

In a professional/enterprise codebase, code is read far more often than it is written — by teammates, reviewers, and your future self, sometimes years later. Consistent standards reduce onboarding time for new team members, prevent bugs caused by ambiguous naming or formatting, make code reviews faster and more focused on *logic* rather than style bikeshedding, and are usually enforced automatically by tools (linters, SonarQube — Chapter 8) as part of a CI/CD pipeline so violations are caught before merge, not after. “Method-1” here refers to Accenture’s internal baseline coding-standard module covering four pillars:

1. Project structure & naming conventions
2. Code formatting & layout
3. Comments & documentation
4. General readability practices

6.2 Naming Conventions

Element	Convention	Example
Class / Interface Method	PascalCase, noun camelCase, verb/verb-phrase	TaskService, Runnable calculateTotal(), getName()
Variable Constant (static final) Package	camelCase, descriptive noun UPPER_SNAKE_CASE all lowercase, reverse-domain	taskList, totalAmount MAX_RETRY_COUNT com.todoapp.taskmanager

Good naming eliminates the need for many comments — `int d;` tells you nothing, `int daysUntilDue;` tells you everything without a single comment. Names should be **intention-revealing**: a reader should be able to guess a variable’s purpose or a method’s effect from its name alone, without reading its implementation.

Additional naming guidelines worth knowing:

- Avoid abbreviations unless universally understood (`num` is borderline, `usrNm` is not).
- Boolean variables/methods read naturally in an `if`: `isValid`, `hasPermission`, `canRetry`.
- Avoid encoding type information in names (Hungarian notation, e.g. `strName`) — Java’s static typing already tells you the type; this style is considered outdated/redundant in modern Java.

6.3 Code Formatting & Layout

Consistent indentation (commonly 4 spaces, **never mixed with tabs** — mixing causes misaligned code across different editors/viewers), consistent brace placement (Java convention: opening brace on the same line as the declaration), sensible line-length limits (commonly 80–120 characters, wrapped otherwise so code doesn’t require horizontal scrolling), one statement per line, and logical blank-line grouping (visually separating a method’s “setup”, “core logic”, and “return” sections) all make code easier to scan quickly. Package organization should mirror logical layering — e.g., separate `model`, `service`, and `application-entry` packages rather than dumping every class into one package — this mirrors the layered architecture (Controller-Service-Repository) used throughout the Spring portion of this course.

6.4 Comments & Documentation

- **Javadoc comments** (`/** ... */`) document the *public contract* of a class/method — used by tooling (javadoc command, IDEs) to auto-generate browsable HTML API documentation. They use structured tags like `@param name description`, `@return description`, `@throws ExceptionType when`, and `@author`. Every public class/method in a shared library should ideally carry a Javadoc comment, since callers may never see the implementation, only the generated docs.
- **Block comments** (`/* ... */`) explain a chunk of logic — typically a multi-line, non-obvious algorithm or business rule.
- **Line comments** (`//`) clarify a single line or a very short group of lines.
- **The golden rule of commenting:** comments should explain **why**, not restate **what** the code already says. `// increment i by 1 above i++;` is noise; `// retry once, since the upstream service occasionally times out on cold start` explains a *non-obvious reason* and earns its place. Over-commenting obvious code is itself considered a code smell — it adds maintenance burden (comments can silently go stale/wrong as code changes) without adding real understanding.

The demo files below walk through: readable naming in a small Task Manager mini-app, indentation and JavaDoc examples, line-length problems and their fixes, package/import organization (including static import), and structured comment tags.

File: `Method1Standards_Demos/Demo1001_Naming_Conventions_InJava/com/acnture/ltt/CodeReadability`

```
package com.acnture.ltt;
```

```
public class CodeReadabilityExample {
```

```
    // Constants for price and quantity
```

```
    public static final double PRICE_PER_ITEM = 20.5;
```

```
    public static final int QUANTITY = 5;
```

```
    public static void main(String[] args) {
```

```
        // Calculate the total cost
```

```
        int total = calculateTotal(PRICE_PER_ITEM, QUANTITY);
```

```
        // If the user is active, send a notification
```

```
        User user = new User(true); // Assuming a User class with a constructor
```

```
        if (user.isActive()) {
```

```
            sendNotification();
```

```
        }
```

```
        // Output the total
```

```
        System.out.println("Total Price: " + total);
```

```
    }
```

```
    // Method to calculate the total cost
```

```
    public static int calculateTotal(double price, int quantity) {
```

```
        return (int) (price * quantity);
```

```
    }
```

```
    // Method to send a notification
```

```
    public static void sendNotification() {
```

```
        System.out.println("Notification sent!");
```

```

    }
}

class User {
    private boolean active;

    // Constructor to initialize the user's status
    public User(boolean active) {
        this.active = active;
    }

    // Method to check if the user is active
    public boolean isActive() {
        return active;
    }
}

```

File: Method1Standards_Demos/Demo1001_Naming_Conventions_InJava/com/todoapp/taskmanager/MainApp.

```

package com.todoapp.taskmanager;

import java.util.Scanner;

import com.todoapp.taskmanager.service.TaskService;

public class MainApp {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        TaskService taskService = new TaskService();

        while (true) {
            System.out.println("\n--- To-Do List Menu ---");
            System.out.println("1. Add Task");
            System.out.println("2. Show Tasks");
            System.out.println("3. Complete Task");
            System.out.println("4. Delete Task");
            System.out.println("5. Exit");

            System.out.print("Choose an option: ");
            String input = scanner.nextLine();

            switch (input) {
                case "1":
                    System.out.print("Enter task description: ");
                    String desc = scanner.nextLine();
                    taskService.addTask(desc);
                    break;
                case "2":
                    taskService.showTasks();
                    break;
                case "3":
                    System.out.print("Enter task ID to mark as completed: ");
                    int completeId = Integer.parseInt(scanner.nextLine());
                    taskService.completeTask(completeId);

```

```

        break;
    case "4":
        System.out.print("Enter task ID to delete: ");
        int deleteId = Integer.parseInt(scanner.nextLine());
        taskService.deleteTask(deleteId);
        break;
    case "5":
        System.out.println("Exiting. Goodbye!");
        return;
    default:
        System.out.println("Invalid option.");
    }
}
}
}
}

```

File: Method1Standards_Demos/Demo1001_Naming_Conventions_InJava/com/todoapp/taskmanager/model/Task.java

```

package com.todoapp.taskmanager.model;

public class Task {
    private int id;
    private String description;
    private boolean completed;

    public Task(int id, String description) {
        this.id = id;
        this.description = description;
        this.completed = false;
    }

    // Getters and setters
    public int getId() { return id; }
    public String getDescription() { return description; }
    public boolean isCompleted() { return completed; }

    public void setCompleted(boolean completed) { this.completed = completed; }

    @Override
    public String toString() {
        return (completed ? "[x] " : "[ ] ") + id + ": " + description;
    }
}

```

File: Method1Standards_Demos/Demo1001_Naming_Conventions_InJava/com/todoapp/taskmanager/service/TaskService.java

```

package com.todoapp.taskmanager.service;

import java.util.ArrayList;
import java.util.List;

import com.todoapp.taskmanager.model.Task;

public class TaskService {
    private List<Task> taskList = new ArrayList<>();
}

```

```

private int nextId = 1;

public void addTask(String description) {
    Task task = new Task(nextId++, description);
    taskList.add(task);
    System.out.println("Task added.");
}

public void showTasks() {
    if (taskList.isEmpty()) {
        System.out.println("No tasks found.");
    } else {
        taskList.forEach(System.out::println);
    }
}

public void completeTask(int id) {
    for (Task task : taskList) {
        if (task.getId() == id) {
            task.setCompleted(true);
            System.out.println("Task marked as completed.");
            return;
        }
    }
    System.out.println("Task not found.");
}

public void deleteTask(int id) {
    taskList.removeIf(task -> task.getId() == id);
    System.out.println("Task deleted if it existed.");
}
}

```

File: Method1Standards_Demos/Demo1003_CodeFormating/com/accenture/ltt/Greeting.java

```

package com.accenture.ltt;

/**
 * @author r.roopavathi.n
 *
 */
public class Greeting {

    public static void main(String[] args) {
        String name = "Alice";

        // Proper whitespace and braces usage
        if (name != null && !name.isEmpty()) {
            System.out.println("Hello, " + name + "!");
        } else {
            System.out.println("Hello, guest!");
        }
    }
}

```

```
//Bad Practice (Poor Braces & Whitespace Usage)
/*public class Greeting1{
public static void main(String[]args){
String name="Alice";
if(name!=null&&!name.isEmpty()){
System.out.println("Hello,"+name+"!");
}else{
System.out.println("Hello, guest!");
}
}} */
```

File: Method1Standards_Demos/Demo1003_CodeFormating/com/accenture/ltt/IndentationExample.java

```
package com.accenture.ltt;

public class IndentationExample {
    // This is the main method
    public static void main(String[] args) {
        int number = 10;
        // Conditional block with correct indentation
        if (number > 5) {
            System.out.println("Number is greater than 5");
            // Nested if statement
            if (number == 10) {
                System.out.println("Number is 10");
            }
            else {
                System.out.println("Number is 5 or less");
            }
        }
    }
    //The code inside the if and else blocks is indented with 4 spaces.
    //The code inside the nested if block is indented further with 4 more spaces.
```

File: Method1Standards_Demos/Demo1003_CodeFormating/com/accenture/ltt/JavaDoc/Calculator.java

```
package com.accenture.ltt.JavaDoc;

/**
 * The Calculator class provides basic arithmetic operations.
 * <p>
 * This class supports addition, subtraction, multiplication, and division
 * for integers and doubles.
 * </p>
 *
 * @author r.roopavathi.n
 *
 * @since 2025-08-13
 */
public class Calculator {

    /**
     * Adds two integers.
     *
     * @param a the first integer
```

```

    * @param b the second integer
    * @return the sum of a and b
    */
    public int add(int a, int b) {
        return a + b;
    }

    /**
     * Divides one number by another.
     *
     * @param numerator the number to be divided
     * @param denominator the number to divide by
     * @return the result of division
     * @throws ArithmeticException if denominator is zero
     */
    public double divide(double numerator, double denominator) {
        if (denominator == 0) {
            throw new ArithmeticException("Cannot divide by zero");
        }
        return numerator / denominator;
    }

    /**
     * The entry point of the Calculator application.
     * <p>
     * This method demonstrates the usage of the {@link #add(int, int)} method
     * by performing a simple addition operation and printing the result.
     * </p>
     *
     * @param args command-line arguments (not used in this program)
     */

    public static void main(String[] args) {
        Calculator calculator = new Calculator();

        // Perform addition of two numbers
        int sum = calculator.add(5, 10);

        // Print the result
        System.out.println("Sum: " + sum);
    }
}

```

File: Method1Standards_Demos/Demo1003_CodeFormating/com/accenture/ltt/linelength/LongLineExample

```

package com.accenture.ltt.linelength;

public class LongLineExample {
    public static void main(String[] args) {
        String message = "This is a very long string that is exceeding the recommended line length";
        System.out.println(message);
    }
}

```



```

    }
}
//Observe the String Line Length

```

File: Method1Standards_Demos/Demo1003_CodeFormating/com/accenture/ltt/linelength/LongLineExample

```

package com.accenture.ltt.linelength;

public class LongLineExample1 {
    public static void main(String[] args) {
        String message = "This is a very long string that is exceeding the recommended line "
            + "length in Java and makes the code harder to read and maintain "
            + "because you have to scroll horizontally.";
        System.out.println(message);
    }
}

//Line Length: Lines longer than 80-100 characters are split into multiple lines.
//String Concatenation: If a string is too long, you can break it into multiple strings and con

```

File: Method1Standards_Demos/Demo1003_CodeFormating/com/accenture/ltt/linelength/TestUtil.java

```

package com.accenture.ltt.linelength;

public class TestUtil {

    public static void main(String[] args) {
        // Create a UserService object
        UserService userService = new UserService();

        // Create a user, set their address, assign a role, and save the user
        User user = new User("John Doe");

        // Using method chaining to create, set address, set role, and save
        userService.createUser(user) // Create the user
            .setAddress("1234 Elm Street") // Set the address
            .setRole("Admin"); // Set the role

        // Now save the user
        userService.save(user); // Save the user after all properties have been set
    }
}

```

File: Method1Standards_Demos/Demo1003_CodeFormating/com/accenture/ltt/linelength/User.java

```

package com.accenture.ltt.linelength;

public class User {

    private String name;
    private String address;
    private String role;
}

```

```

public User(String name) {
    this.name = name;
}

public String getName() {
    return name;
}

public String getAddress() {
    return address;
}

public String getRole() {
    return role;
}

// Setters for method chaining
public User setAddress(String address) {
    this.address = address;
    return this; // Return the current object (this) for chaining
}

public User setRole(String role) {
    this.role = role;
    return this; // Return the current object (this) for chaining
}
}

```

File: Method1Standards_Demos/Demo1003_CodeFormating/com/accenture/ltt/linelength/UserGreeting.java

```

package com.accenture.ltt.linelength;

//This class demonstrates good Java code formatting and best practices.
import java.util.Scanner;

public class UserGreeting {

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.print("Enter your first name: ");
        String firstName = scanner.nextLine();

        System.out.print("Enter your last name: ");
        String lastName = scanner.nextLine();

        // Proper spacing, line length, and indentation
        String greetingMessage = generateGreeting(firstName, lastName);

        System.out.println(greetingMessage);

        scanner.close();
    }
}

```

```

    // Properly indented method with consistent spacing and braces
    private static String generateGreeting(String firstName, String lastName) {
        return "Hello, " + firstName + " " + lastName + "! Welcome to the Java formatting demo."
    }
}

```

File: Method1Standards_Demos/Demo1003_CodeFormating/com/accenture/ltt/linelength/UserService.java

```

package com.accenture.ltt.linelength;

public class UserService {

    // Create a new User object
    public User createUser(User user) {
        // Logic for creating a user (like saving it to a database)
        System.out.println("Creating user: " + user.getName());
        return user; // Return the user object for chaining
    }

    // Save the user to the database (or perform some operation)
    public void save(User user) {
        System.out.println(
            "Saving user: " + user.getName() + ", Address: " + user.getAddress() + ", Role:
        }
    }
}

```

File: Method1Standards_Demos/Demo1003_CodeFormating/com/accenture/ltt/packages/StaticImportExample.java

```

package com.accenture.ltt.packages;

import static java.lang.Math.*; // Static import of all static members of Math class

public class StaticImportExample {
    public static void main(String[] args) {
        // We can directly call sqrt() without Math.
        double result = sqrt(16);
        System.out.println("Square root of 16: " + result);

        // Other static methods/constants can also be used directly
        System.out.println("Value of PI: " + PI);
    }
}

```

File: Method1Standards_Demos/Demo1003_CodeFormating/com/accenture/ltt/packages/TemperatureConverter.java

```

package com.accenture.ltt.packages;

import java.util.Scanner; // Specific import, not wildcard

public class TemperatureConverter {

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in); // Using built-in Java package

        System.out.print("Enter temperature in Celsius: ");
        double celsius = scanner.nextDouble();
    }
}

```

```

        double fahrenheit = convertToFahrenheit(celsius);

        System.out.println("Temperature in Fahrenheit: " + fahrenheit);

        scanner.close(); // Clean code: resource management
    }

    // Clean, reusable method with a meaningful name
    private static double convertToFahrenheit(double celsius) {
        return (celsius * 9 / 5) + 32;
    }
}

```

File: Method1Standards_Demos/Demo1003_CodeFormating/com/accenture/ltt/packages/UserGreeting.java

```

package com.accenture.ltt.packages;

//Good import practice: only import what's needed
//import java.util.Scanner; // Regular/default import
import java.util.*;
public class UserGreeting {

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in); // Using built-in Java package (java.util)

        System.out.print("Enter your name: ");
        String name = scanner.nextLine();

        greetUser(name);

        scanner.close(); // Clean code: closing resources
    }

    // Clean code: meaningful method name and proper documentation
    private static void greetUser(String name) {
        if (name != null && !name.trim().isEmpty()) {
            System.out.println("Hello, " + name + "!");
        } else {
            System.out.println("Hello, guest!");
        }
    }
}

```

File: Method1Standards_Demos/Demo1004_Comments_Documentation/com/accenture/ltt/comments/CommentTagDemo.java

```

package com.accenture.ltt.comments;

public class CommentTagDemo {

    public static void main(String[] args) {
        String userInput = null;

        // TODO: Get user input from console or UI
        // Currently hardcoded for demonstration
    }
}

```

```

        userInput = " John Doe ";

        System.out.println(userInput.length());
        // FIXME: This will throw an exception if userInput is null
        // Should add a null check before calling trim()
        String trimmedName = userInput.trim();

        // NOTE: Using toUpperCase to normalize name for display
        System.out.println("Welcome, " + trimmedName.toUpperCase()+ " " +trimmedName.length());
    }
}

```

File: Method1Standards_Demos/Demo1004_Comments_Documentation/com/accenture/ltt/comments/Password

```

package com.accenture.ltt.comments;

/**
 * // JavaDoc Comment
 * This class provides basic password validation utility.
 * It checks for minimum length and presence of a special character.
 */
public class PasswordValidator {

    /**
     * Validates the password based on predefined rules. // JavaDoc Comment
     * Rules:
     * - Minimum 8 characters
     * - Must contain at least one special character (!, @, #, $, %, etc.)
     *
     * @param password the password string to validate
     * @return true if password is valid, false otherwise
     */
    public boolean isValid(String password) {
        // Check if password is too short // Single-line Comment
        if (password.length() < 8) {
            return false;
        }

        /* // Multi-line Comment
         * Check if password contains at least one special character.
         * This is a basic check using regex pattern.
         */
        if (!password.matches(".*[!@#$$%^&*()]*.*")) {
            return false;
        }

        return true;
    }

    public static void main(String[] args) {
        PasswordValidator validator = new PasswordValidator();

        // Sample test cases // Single-line Comment
        String pw1 = "welcome123";
    }
}

```

```
String pw2 = "welcome@123";

System.out.println(pw1 + " => " + validator.isValid(pw1)); // false
System.out.println(pw2 + " => " + validator.isValid(pw2)); // true
    }
}
```

6.5 Viva Voce — Likely Questions & Answers

Q1. Why do coding standards matter in a team environment? A: They make code predictable and easier to read across the whole team, reduce onboarding time, speed up code reviews (since reviewers focus on logic rather than style disagreements), and are often enforced automatically via static-analysis tooling in CI/CD pipelines.

Q2. What naming convention does Java use for classes vs. methods vs. constants? A: Classes/interfaces use PascalCase (e.g. TaskService); methods and variables use camel-Case (e.g. calculateTotal); constants (static final fields) use UPPER_SNAKE_CASE (e.g. MAX_RETRY_COUNT).

Q3. What is the difference between a Javadoc comment, a block comment, and a line comment? A: A Javadoc comment (`/** ... */`) documents a public API element for auto-generated documentation tooling, using tags like `@param/@return`. A block comment (`/* ... */`) explains a larger chunk of logic inline. A line comment (`//`) is a brief, single-line clarification.

Q4. What's the rule of thumb for what comments should (and shouldn't) explain? A: Comments should explain *why* something is done a certain (possibly non-obvious) way, not restate *what* the code already visibly does — restating obvious code in comments is itself considered a code smell, and stale comments (that drift out of sync with the code they describe) can actively mislead readers.

Q5. Why should tabs and spaces not be mixed for indentation? A: Because different editors/tools render tab width differently, mixing them causes code to appear misaligned depending on the viewer's settings — a purely-spaces (or purely-tabs) convention, consistently applied, avoids this entirely.

Chapter 7: Debugging

7.1 Understanding Errors & Exceptions

Before debugging effectively, you need to be able to read a **stack trace** confidently: the top line names the exception type and its message; each subsequent `package.Class.method(File:line)` line shows one frame of the call chain, listed **most-recent-call-first**. The *root cause* of a bug is usually the **deepest frame that belongs to your own application code** (as opposed to deep inside a JDK or third-party library class) — reading top-to-bottom, you’re looking for where your own logic first went wrong, not necessarily the very first line of the trace.

7.2 “Print-Statement” Debugging, Tracing & Logging

The simplest debugging technique is inserting `System.out.println()` at key points to inspect variable values as the program runs — quick to add, but messy (clutters real output), easy to forget to remove before committing, and completely impractical in a production system where you cannot recompile/redeploy on a whim just to add a print statement.

A more disciplined alternative is a **Logger** (Java’s built-in `java.util.logging`, or in real projects the more commonly used SLF4J/Log4j2 frameworks), which supports **log levels**:

Level	When to use
TRACE/DEBUG	Fine-grained diagnostic detail, normally off in production
INFO	Notable application events (startup, a request completed)
WARN	Something unexpected but recoverable happened
ERROR	A failure occurred that needs attention

Levels can be turned on/off (or filtered above a threshold) **without touching code or recompiling**, simply by changing a configuration file — this is precisely why logging, not `println`, is the standard in real production systems: you can enable verbose DEBUG logging temporarily to diagnose a live issue, then turn it back down, all without a code deployment.

7.3 Code Review & Problem Isolation

Systematic problem isolation means narrowing down *where* a bug lives before attempting to fix it:

1. **Reproduce reliably** — a bug you can’t reproduce on demand is very hard to confirm you’ve actually fixed.
2. **Bisect the code** — comment out or isolate halves of the suspect logic to narrow down which section actually contains the fault (similar in spirit to a binary search over your own code).
3. **Check assumptions about input data** — many “impossible” bugs turn out to be a subtly unexpected input value (null, empty string, unexpected format) that the code never accounted for.
4. **Read the stack trace bottom-up from your own code outward** — start at the deepest frame that’s *your* code, since library/framework internals are rarely themselves the bug.

7.4 The Eclipse Debugger

Eclipse’s graphical debugger turns guesswork into direct, live inspection of a running program:

- **Breakpoints** — a marker on a specific line that pauses execution the *instant* that line is about to run, letting you inspect the program’s exact state at that point (switches the IDE into the “Debug perspective”). Toggled by double-clicking in the editor’s left margin.
- **Step Controls**, once paused at a breakpoint:
 - **Step Into (F5)** — enters the very next method call, moving into its internals line-by-line.
 - **Step Over (F6)** — executes the current line (including any method call it makes) as a single atomic step, *without* diving into that called method’s internals — used when you trust a method and just want to see its result.
 - **Step Return (F7)** — finishes executing the *current* method entirely and returns control to wherever it was called from, useful once you’ve seen enough of the current method’s internals.
- **Inspecting Runtime State** — the *Variables* view shows the live values of all local and instance variables visible at the current breakpoint; hovering the mouse over any expression in the editor while paused shows a tooltip with its current value.
- **Call Stack (Debug view)** — shows every method call currently “in progress” (i.e., that hasn’t yet returned), letting you click any frame to inspect *that* frame’s local variable state — vital for understanding *how* execution arrived at the current point.
- **Exception Breakpoints** — rather than a breakpoint tied to a specific line, this pauses execution the *instant* a specified exception type is thrown **anywhere**, even deep inside library code you didn’t write — invaluable when you know *what* exception is happening but not *where* it originates.
- **Debug Configurations** — control program arguments, JVM arguments (like -Xmx heap size), and environment variables for a specific debug run, without changing the actual source code.
- **Advanced tools: conditional breakpoints** (only pause when a boolean expression you specify evaluates to true — e.g., only pause on the 500th loop iteration, or only for a specific employee ID), **watchpoints** (pause the instant a specific *field’s* value changes, anywhere in the code, rather than at a specific line), and the **Expressions view** (evaluate arbitrary snippets of code live while paused, without modifying the source).

The demo programs below intentionally contain bugs/logic to trace, matching each debugging technique above — from a simple Demo1_Errors (a program containing an error to locate), through tracing with print statements and a Logger, to a StackTrace-driven fix, and finally the Eclipse-breakpoint- oriented demos (BreakpointDemo, StepControlDemo, runtime-state inspection, debug configuration, and advanced tooling).

7.4.1 Part 1 — Errors, Tracing, Logging, Stack Traces

File: Part-1-Demos/Demo_1000_Debugger_Errors/com/accenture/ltt/Demo1_Errors.java

```
package com.accenture.ltt;
```

```
import java.util.Arrays;
```

```
//Demo 1: Syntax vs Runtime vs Logical Error
```

```
//HOW TO USE:
```

```
//1) Start with the SYNTAX ERROR section: uncomment the bad line to show a compile-time error,
```

```
//2) Run the RUNTIME ERROR: observe ArithmeticException (division by zero), then fix 'b' to non
```

```
//3) Run the LOGICAL ERROR: observe wrong sum, use println tracing to find and fix the bug.
```



```

public class Demo1_Errors {
    public static void main(String[] args) {
        System.out.println("=== Demo 1: Syntax vs Runtime vs Logical Error ===");

        // ----- SYNTAX ERROR (Compile-time) -----
        // Uncomment the next line to PRODUCE a syntax error (missing semicolon):
        // int x = 10 // ❌ Missing semicolon -> compiler error
        // Fix: add semicolon -> int x = 10;

        // ----- RUNTIME EXCEPTION (Execute-time) -----
        int a = 10;
        int b = 0; // ❌ Will cause division by zero
        try {
            System.out.println("Runtime test: a / b = " + (a / b));
        } catch (ArithmeticException ex) {
            System.out.println("Caught ArithmeticException: " + ex.getMessage());
            // Fix suggestion: set b = 2; then rerun
        }

        // ----- LOGICAL ERROR (Program runs but wrong output) -----
        // Goal: sum the first 5 numbers (1..5 = 15). Bug: starts from 0 and uses '<' wrong bound.
        int sum = 0;
        for (int i = 0; i < 5; i++) { // ❌ Off-by-one logic if intended 1..5 (prints 10)
            // TRACE with println to see i and sum
            System.out.println("[TRACE] i=" + i + ", sum(before)+=i -> " + sum + " + " + i);
            sum += i;
        }
        System.out.println("Logical error result (expected 15): " + sum);
        // Fix: for (int i = 1; i <= 5; i++) sum += i;
    }
}

```

File: Part-1-Demos/Demo_1001_Debugger_TracingVariables/com/accnture/ltt/TracingVariables.java

```

package com.accnture.ltt;

//Demo 1: Syntax vs Runtime vs Logical Error
//HOW TO USE:
//1) Start with the SYNTAX ERROR section: uncomment the bad line to show a compile-time error,
//2) Run the RUNTIME ERROR: observe ArithmeticException (division by zero), then fix 'b' to non
//3) Run the LOGICAL ERROR: observe wrong sum, use println tracing to find and fix the bug.

```

```

public class TracingVariables {
    public static void main(String[] args) {
        System.out.println("=== Demo 1: Syntax vs Runtime vs Logical Error ===");

        // ----- SYNTAX ERROR (Compile-time) -----
        // Uncomment the next line to PRODUCE a syntax error (missing semicolon):
        // int x = 10 // ❌ Missing semicolon -> compiler error
        // Fix: add semicolon -> int x = 10;

        // ----- RUNTIME EXCEPTION (Execute-time) -----
        int a = 10;
        int b = 0; // ❌ Will cause division by zero
    }
}

```

```

try {
    System.out.println("Runtime test: a / b = " + (a / b));
} catch (ArithmeticException ex) {
    System.out.println("Caught ArithmeticException: " + ex.getMessage());
    // Fix suggestion: set b = 2; then rerun
}

// ----- LOGICAL ERROR (Program runs but wrong output) -----
// Goal: sum the first 5 numbers (1..5 = 15). Bug: starts from 0 and uses '<' wrong bound.
int sum = 0;
for (int i = 0; i < 5; i++) { // ☐ Off-by-one logic if intended 1..5 (prints 10)
    // TRACE with println to see i and sum
    System.out.println("[TRACE] i=" + i + ", sum(before)+=i -> " + sum + " + " + i);
    sum += i;
}
System.out.println("Logical error result (expected 15): " + sum);
// Fix: for (int i = 1; i <= 5; i++) sum += i;
}
}

```

File: Part-1-Demos/Demo_1002_Debugger_TracingWithLogger/com/accnture/ltt/TracingWithLogger.java

```

package com.accnture.ltt;

//Demo 3: Tracing with Logger
//
//PURPOSE: Show multi-method flow and how INFO/WARNING appear differently.
//
//HOW TO VIEW:
//- Run normally -> see log levels in console.
//- Adjust logger level if needed.

import java.util.logging.Logger;
import java.util.logging.Level;

public class TracingWithLogger {
    private static final Logger LOG = Logger.getLogger(TracingWithLogger.class.getName());

    public static void main(String[] args) {
        // Optionally set a global level:
        LOG.setLevel(Level.ALL);

        LOG.info("=== Demo 3: Tracing With Logger START ===");
        int[] numbers = {2, -1, 3};

        int total = process(numbers);
        LOG.info("Final total = " + total);
        LOG.info("=== Demo 3: END ===");
    }

    static int process(int[] arr) {
        LOG.info("process() called with length=" + arr.length);
        int sum = 0;
        for (int i = 0; i < arr.length; i++) {

```

```

        LOG.fine("At index " + i + ", value=" + arr[i]); // FINE may be hidden if console defa
        sum = add(sum, arr[i]);
    }
    return sum;
}

static int add(int a, int b) {
    if (b < 0) {
        LOG.warning("add() received negative value: " + b); // WARNING stands out
    } else {
        LOG.info("add() adding " + b);
    }
    return a + b;
}
}
}

```

File: Part-1-Demos/Demo_1003_Debugger_StackTraceAndFix/com/accenture/ltt/StackTraceAndFix.java

```

package com.accenture.ltt;

//Demo 4: Reading Stack Trace & Isolating Error
//
//PURPOSE: Trigger NullPointerException, read stack trace, then fix via null check.
//STEPS:
//1) Run as-is -> observe NPE stack trace pointing to getLength(name).
//2) Add a null-check in getLength before calling length().

public class StackTraceAndFix {
    public static void main(String[] args) {
        System.out.println("=== Demo 4: Stack Trace & Isolating Error ===");
        String name = null; // ☐ Intentional null to trigger NPE
        try {
            int len = getLength(name); // Stack trace will point here and into getLength
            System.out.println("Name length = " + len);
        } catch (NullPointerException ex) {
            ex.printStackTrace(); // Print full stack trace to console
            System.out.println("Fix: add null-check in getLength().");
        }
    }

    static int getLength(String s) {
        // ☐ This will throw NPE when s is null
        return s.length();
        // Fix:
        // if (s == null) return 0;
        // return s.length();
    }
}

```

7.4.2 Part 2 — Eclipse Breakpoints, Step Control, Runtime State, Debug Config, Advanced Tools

File: Part-2-Demos/Demo_2000_Eclipse_Debugger_Breakpoints/com/accenture/ltt/BreakpointDemo.java

```

// Demo_20001_Eclipse_Debugger_Breakpoints
// Demonstrates Line, Conditional, Exception, Method, and Class Load Breakpoints.
package com.accenture.ltt;
public class BreakpointDemo {

    // Step 1: Set a CLASS LOAD BREAKPOINT here
    static {
        System.out.println("Class BreakpointDemo loaded into memory");
    }

    public static void main(String[] args) {
        System.out.println("=== Demo: All Types of Breakpoints ===");

        // Step 2: Set a LINE BREAKPOINT on the next line
        int[] numbers = {2, 4, 6, 8, 10};

        for (int i = 0; i < numbers.length; i++) {

            // Step 3: Add a CONDITIONAL BREAKPOINT -> Condition: i == 2
            System.out.println("Processing index: " + i + " -> Value: " + numbers[i]);

            if (numbers[i] == 8) {
                // Step 4: Trigger an EXCEPTION BREAKPOINT for NullPointerException
                throw new NullPointerException("Simulated Exception at value: " + numbers[i]);
            }
        }

        // Step 5: Add a METHOD BREAKPOINT on sortNumbers()
        sortNumbers();
        System.out.println("Program Completed.");
    }

    public static void sortNumbers() {
        System.out.println("Inside sortNumbers() method.");
        int[] nums = {5, 1, 3, 2, 4};
        java.util.Arrays.sort(nums);
        System.out.println("Sorted Numbers: " + java.util.Arrays.toString(nums));
    }
}

```

File: Part-2-Demos/Demo_2000_Eclipse_Debugger_Breakpoints/com/accenture/ltt/DebugPractice.java

```

// Demo_20001_Eclipse_Debugger_Breakpoints
// Demonstrates Exception and Method Breakpoints for debugging practice.
package com.accenture.ltt;
public class DebugPractice {

    public static void main(String[] args) {
        System.out.println("=== Demo: Exception & Method Breakpoints ===");

        // Step 1: Set an EXCEPTION BREAKPOINT for ArithmeticException
        try {
            int result = divide(10, 0); // Intentional division by zero
            System.out.println("Result: " + result);
        }
    }
}

```

```

    } catch (ArithmeticException e) {
        System.out.println("Caught Exception: " + e.getMessage());
    }

    // Step 2: Add a METHOD BREAKPOINT on greetUser()
    greetUser("Ravi");

    System.out.println("Program Completed.");
}

public static int divide(int a, int b) {
    // Step 3: Step Into this method to observe the call stack
    System.out.println("Dividing " + a + " by " + b);
    return a / b; // Will cause ArithmeticException
}

public static void greetUser(String name) {
    System.out.println("Hello, " + name + "! Welcome to the Debugger Demo.");
}
}

```

File: Part-2-Demos/Demo_2000_Eclipse_Debugger_Breakpoints/com/accenture/ltt/EmployeeList.java

```

// Demo_20001_Eclipse_Debugger_Breakpoints
// Demonstrates Line and Conditional Breakpoints in a collection loop.
package com.accenture.ltt;
import java.util.ArrayList;
import java.util.List;

public class EmployeeList {

    public static void main(String[] args) {
        System.out.println("=== Demo: Employee List Debugging ===");

        // Step 1: Observe List creation in Variables View
        List<String> employees = new ArrayList<>();
        employees.add("Ravi");
        employees.add("Meena");
        employees.add("Karthik");
        employees.add("Divya");

        // Step 2: Set a LINE BREAKPOINT inside the loop below
        for (int i = 0; i < employees.size(); i++) {
            String name = employees.get(i);

            // Step 3: Add a CONDITIONAL BREAKPOINT -> Condition: name.equals("Meena")
            System.out.println("Employee " + (i + 1) + ": " + name);

            if (name.equals("Meena")) {
                System.out.println(name + " is a Team Lead");
            }
        }

        System.out.println("After loop execution: " + employees);
    }
}

```

```

        System.out.println("Demo Completed.");
    }
}

```

File: Part-2-Demos/Demo_2001_Debugger_StepControl/com/accenture/ltt/StepControlDemo.java

```
package com.accenture.ltt;
```

```
//Demo 1: Syntax vs Runtime vs Logical Error
```

```
//HOW TO USE:
```

```
//(1) Start with the SYNTAX ERROR section: uncomment the bad line to show a compile-time error,
```

```
//(2) Run the RUNTIME ERROR: observe ArithmeticException (division by zero), then fix 'b' to non
```

```
//(3) Run the LOGICAL ERROR: observe wrong sum, use println tracing to find and fix the bug.
```

```
public class Demo1_Errors {
    public static void main(String[] args) {
        System.out.println("=== Demo 1: Syntax vs Runtime vs Logical Error ===");
```

```
        // ----- SYNTAX ERROR (Compile-time) -----
```

```
        // Uncomment the next line to PRODUCE a syntax error (missing semicolon):
```

```
        // int x = 10 // □ Missing semicolon -> compiler error
```

```
        // □ Fix: add semicolon -> int x = 10;
```

```
        // ----- RUNTIME EXCEPTION (Execute-time) -----
```

```
        int a = 10;
```

```
        int b = 0; // □ Will cause division by zero
```

```
        try {
```

```
            System.out.println("Runtime test: a / b = " + (a / b));
```

```
        } catch (ArithmeticException ex) {
```

```
            System.out.println("Caught ArithmeticException: " + ex.getMessage());
```

```
            // □ Fix suggestion: set b = 2; then rerun
```

```
        }
```

```
        // ----- LOGICAL ERROR (Program runs but wrong output) -----
```

```
        // Goal: sum the first 5 numbers (1..5 = 15). Bug: starts from 0 and uses '<' wrong bound.
```

```
        int sum = 0;
```

```
        for (int i = 0; i < 5; i++) { // □ Off-by-one logic if intended 1..5 (prints 10)
```

```
            // TRACE with println to see i and sum
```

```
            System.out.println("[TRACE] i=" + i + ", sum(before)+=i -> " + sum + " + " + i);
```

```
            sum += i;
```

```
        }
```

```
        System.out.println("Logical error result (expected 15): " + sum);
```

```
        // □ Fix: for (int i = 1; i <= 5; i++) sum += i;
```

```
    }
```

```
}
```

File: Part-2-Demos/Demo_2002_Viewing_RunTime_State/com/accenture/ltt/EmployeeList.java

```
package com.accenture.ltt;
```

```
import java.util.ArrayList;
```

```
import java.util.List;
```

```
public class EmployeeList {
```

```
    public static void main(String[] args) {
```

```

System.out.println("Runtime State Demo Started");

// Step 1: Observe the List creation and element addition in Variables View
List<String> employees = new ArrayList<>();
employees.add("Ravi");
employees.add("Meena");
employees.add("Karthik");
employees.add("Divya");

// Step 2: Set a Line Breakpoint on the next line to inspect runtime state
for (int i = 0; i < employees.size(); i++) {

    // Step 3: When paused here, open Variables View
    // Inspect variables: i, employees, name
    String name = employees.get(i);

    System.out.println("Employee " + (i + 1) + ": " + name);

    // Step 4: Conditional logic – monitor this condition in Expressions View
    if (name.equals("Meena")) {
        System.out.println(name + " is a Team Lead");
    }

    // Step 5 (Optional): While paused, try modifying variable value in Variables View
    // Example: Change name = "Meena Sharma" and resume to see the effect
}

// Step 6: Inspect final state of the List after loop completes
System.out.println("After loop execution: " + employees);

// Step 7: End of demo – verify console output after modifications
System.out.println("Runtime State Demo Completed");
}
}

```

File: Part-2-Demos/Demo_2003_DebugConfiguration/com/accenture/ltt/DebugConfigDemo.java

```

package com.accenture.ltt;
public class DebugConfigDemo {

    public static void main(String[] args) {

        // Step 1: Open Eclipse → Run → Debug Configurations...
        // Step 2: Select "Java Application" → Click "New Configuration"
        // Step 3: In Main tab → Browse and select this class (DebugConfigDemo) as Main Class
        // Step 4: (Optional) Add Program Arguments, e.g., "Ravi 50000"
        // Step 5: Enable "Stop in main()" to pause at the first line of execution
        // Step 6: Click Apply → Debug → Program pauses before executing first statement

        System.out.println("Debug Configuration Demo Started");

        // Step 7: Inspect Variables View after program starts
        String name = "Ravi";
        double salary = 50000;
    }
}

```

```

        System.out.println("Employee Name: " + name);
        System.out.println("Salary: " + salary);

        double bonus = calculateBonus(salary);

        System.out.println("Final Salary (with Bonus): " + (salary + bonus));

        System.out.println("Debug Configuration Demo Completed");
    }

    // Step 8: Step Into this method using Debug Controls to see how call stack changes
    public static double calculateBonus(double salary) {
        double bonus = salary * 0.10;
        return bonus;
    }
}

```

File: Part-2-Demos/Demo_2004_AdvancedTools/com/accenture/ltt/AdvancedDebugDemo.java

```

package com.accenture.ltt;
public class AdvancedDebugDemo {

    // Use a WATCHPOINT on this field to break whenever it is read or written
    private static double bonusRate = 0.10; // Right-click margin → Toggle Watchpoint (field)

    public static void main(String[] args) {
        System.out.println("Advanced Debugging Tools Demo Started");

        Employee e = new Employee("Ravi", 50000);
        applyAnnualIncrement(e); //Try a METHOD BREAKPOINT here
        applyBonus(e); //Step Filters can skip into JDK/internal libs

        //DROP TO FRAME: After stepping past applyBonus, use 'Drop to Frame' on main()
        // to re-run from this point and observe state reset.

        System.out.println("Final: " + e);

        // HOT CODE REPLACE (HCR):
        // While paused in debug, edit the body of formatEmployee() to change output
        // and save. Eclipse automatically applies the change using Hot Code Replace.
        // No need to restart the program.
        System.out.println(formatEmployee(e));

        System.out.println("Advanced Debugging Tools Demo Completed");
    }

    private static void applyAnnualIncrement(Employee e) {
        // CONDITIONAL BREAKPOINT / HIT COUNT:
        // Put a breakpoint that hits only when salary > 60000 or after N hits.
        if (e.getSalary() < 60000) {
            e.setSalary(e.getSalary() * 1.10);
        }
    }
}

```



```

private static void applyBonus(Employee e) {
    // WATCHPOINT will trigger when bonusRate is READ here
    double bonus = e.getSalary() * bonusRate;
    e.setSalary(e.getSalary() + bonus);
}

private static String formatEmployee(Employee e) {
    // DETAIL FORMATTER idea: You can configure how Employee is displayed in Variables View
    return "Employee{name='" + e.getName() + "', salary=" + e.getSalary() + "}";
}

// ===== Helper Type =====
static class Employee {
    private String name;
    private double salary; // You can also add a WATCHPOINT on this field

    Employee(String name, double salary) {
        this.name = name;
        this.salary = salary;
    }

    public String getName() { return name; }
    public double getSalary() { return salary; }

    public void setSalary(double salary) {
        // TRACEPOINT alternative: log automatically without pausing (configure from breakpoint)
        this.salary = salary;
    }

    @Override
    public String toString() {
        return "Employee(" + name + ", " + salary + ")";
    }
}
}

```

7.5 Viva Voce — Likely Questions & Answers

Q1. How do you read a Java stack trace to find the root cause of a bug? A: The top line gives the exception type and message; each subsequent frame shows one call in the chain, most-recent-call-first. Look for the *deepest frame belonging to your own application code* (not JDK/library internals) — that’s usually where the actual root cause lies.

Q2. Why is logging generally preferred over `System.out.println()` for real applications? A: Logging supports levels (DEBUG/INFO/WARN/ERROR) that can be turned on or off via configuration without recompiling or redeploying, can be routed to files/monitoring systems, and doesn’t clutter production output — `println` offers none of this and must be manually removed before shipping.

Q3. What is the difference between “Step Into” and “Step Over” in a debugger? A: Step Into (F5) enters the very next method call and lets you trace its internals line by line. Step Over (F6) executes the current line (including any method it calls) as one atomic step without descending into that method’s internals.

Q4. What is an exception breakpoint, and when would you use one instead of a normal line breakpoint? A: An exception breakpoint pauses execution the instant a specified exception type is thrown *anywhere* in the code (even inside library internals), rather than at one specific line — useful when you know which exception is happening but don't yet know where it's being thrown from.

Q5. What is a conditional breakpoint, and why is it useful? A: A breakpoint that only pauses execution when a boolean expression you specify evaluates to true — e.g., pausing only on the 500th loop iteration or only for a specific problematic input value, avoiding having to manually step through hundreds of irrelevant iterations first.

Q6. What does the Call Stack / Debug view show while paused at a breakpoint? A: Every method invocation currently in progress (not yet returned), in order from the most recent call back to the program's entry point — clicking any frame lets you inspect that frame's own local variable state, showing exactly how execution arrived at the current point.

Chapter 8: Code Quality with SonarQube

(No demo Java project was supplied for this topic — this section covers the classroom theory only. Practically, this topic is a hands-on tool walkthrough rather than a coding exercise.)

8.1 Why Code Quality Matters

Code that merely “works” today is not the same as code that is *maintainable, secure, and efficient* over its whole lifetime. Poor code quality — duplicated logic, high cyclomatic complexity, unhandled edge cases, security vulnerabilities like SQL injection or hardcoded credentials — accumulates as **technical debt**: like financial debt, it doesn’t disappear on its own, and the longer it’s left unaddressed, the more “interest” it costs in the form of slower future changes and higher production-defect risk.

8.2 Real-Time Static Code Analysis

Static analysis tools scan source code **without executing it**, categorizing issues into three buckets:

- **Bugs** — code that is very likely to fail at runtime under some condition (e.g., a possible null dereference, a resource that’s never closed).
- **Vulnerabilities** — security-exploitable patterns (e.g., unsanitized SQL query construction enabling SQL injection, use of weak/broken cryptographic algorithms, hardcoded passwords/API keys in source).
- **Code Smells** — maintainability issues that aren’t strictly *wrong* yet, but make future changes riskier and slower (duplicated blocks of logic, overly long methods/classes, deeply nested conditionals, dead/unreachable code, magic numbers instead of named constants).

8.3 SonarQube — Core Concepts

SonarQube is a widely used platform for continuous code-quality inspection integrated into the development workflow. Key concepts:

- **Quality Gate** — a pass/fail threshold a build must satisfy to be considered releasable, e.g. “0 new bugs introduced”, “code coverage on new code $\geq$ 80%”, “no new critical/blocker vulnerabilities”. A failing Quality Gate can be configured to actually **block** a merge/deployment in the CI/CD pipeline (Jenkins, covered later in the course).
- **SonarScanner** — the command-line tool that statically analyzes a project’s source code and uploads the results to the central SonarQube server for reporting and gate evaluation.
- **SonarLint** — an IDE plugin (Eclipse/IntelliJ/VS Code) that surfaces the *same* rule violations live, **as you type**, giving immediate feedback before you even commit the code — shifting quality feedback as early as possible in the development cycle (“shift-left”).
- **Technical Debt Ratio** — SonarQube estimates the *time* it would take to fix all currently-known issues, expressed as a ratio against the estimated time it took to develop the code, giving a rough, trackable proxy for overall codebase health over time.
- **Automation in CI/CD** — in a mature pipeline, the SonarQube scan and Quality Gate check run automatically on every push/pull-request, so quality regressions are caught mechanically rather than relying purely on manual code review.

Because SonarQube’s automated Quality Gate features and scanner rely on newer language constructs, this module notes it needs **Java 17** as the analyzed project’s/tooling baseline.

8.4 Viva Voce — Likely Questions & Answers

Q1. What’s the difference between a “bug”, a “vulnerability”, and a “code smell” in static analysis terminology? A: A bug is code very likely to fail at runtime (e.g. possible null dereference). A vulnerability is a security-exploitable pattern (e.g. SQL injection risk). A code smell isn’t broken yet but hurts future maintainability (duplication, overly long methods, dead code).

Q2. What is a Quality Gate in SonarQube? A: A configurable set of pass/fail conditions (e.g. zero new bugs, minimum code coverage on new code, no new critical vulnerabilities) that a build must satisfy to be considered releasable — often wired into CI/CD to actually block a failing merge/deploy.

Q3. What’s the difference between SonarScanner and SonarLint? A: SonarScanner is a CLI tool that analyzes a whole project and pushes results to the central SonarQube server (typically run in CI). SonarLint is an IDE plugin giving the *same* feedback live, as the developer types, before the code is even committed.

Q4. Why is static analysis run “without executing” the code — what does that mean practically? A: It means the tool parses and inspects the source/bytecode structurally (control flow, data flow, known bad patterns) rather than running the program with test inputs — this lets it flag potential issues across every code path, including ones that might rarely execute in normal testing.

Q5. What is technical debt, and how does SonarQube attempt to quantify it? A: Technical debt is the implied cost of extra future work caused by choosing an easy/quick solution now instead of a better approach that would take longer. SonarQube estimates it as the time needed to fix all currently known issues, often expressed as a ratio against estimated development time.

Chapter 9: Multithreading

9.1 Process vs Thread

A **process** is an independent running program instance with its own dedicated memory space, managed and isolated by the operating system — one process cannot directly read another's memory. A **thread** is a lightweight sub-unit of execution *within* a process — multiple threads in the same process **share the same heap memory and open resources** (files, sockets), which enables very fast communication between them (no serialization/IPC needed, unlike between separate processes) but also opens the door to **data-inconsistency bugs** if two threads read/modify shared state at the same time without coordination (9.5).

Each thread does, however, get its **own separate stack** — this is why local variables are inherently thread-safe (each thread has its own copy on its own stack), while instance/static fields are **not** automatically thread-safe (they live once on the shared heap).

9.2 Creating Threads in Java

There are two standard ways to define the work a thread should perform:

1. **Extending Thread** and overriding `run()` — simplest to write, but “spends” Java's single class-inheritance slot on threading machinery rather than on meaningful business-logic inheritance, and tightly couples your task to being a Thread specifically.
2. **Implementing Runnable** and passing the instance to `new Thread(runnableInstance)` — **generally preferred**, because your class remains free to extend something else if needed, the same Runnable instance can be handed to multiple threads or reused with a thread-pool executor (`ExecutorService`, a more advanced topic beyond this module), and it cleanly separates “*what work to do*” (the Runnable) from “*how it's executed*” (the Thread/executor).

A classic beginner mistake, frequently tested: calling `.start()` schedules the thread to run **concurrently** — the JVM allocates a *new call stack* and the `run()` method executes on a separate thread of control. Calling `.run()` **directly** (instead of `.start()`) does **not** create a new thread at all — it just executes `run()`'s body synchronously on the *current* calling thread, exactly like any ordinary method call, defeating the entire purpose of threading.

9.3 Thread Lifecycle & Core APIs

A thread moves through states: **New** (created but not started) → **Runnable** (eligible to run, either actually running or waiting for CPU time) → **Blocked/Waiting** (waiting for a lock or a signal) → **Terminated** (finished executing `run()`).

- **`start()`** — transitions a New thread to Runnable, beginning concurrent execution.
- **`join()`** — makes the *calling* thread pause and wait until the *target* thread finishes (transitions to Terminated) before the caller proceeds — used to synchronize results, e.g. “wait for all worker threads to finish before printing the combined total”.
- **`sleep(ms)`** — pauses the **current** thread for at least the given duration, *without* releasing any locks it currently holds (an important distinction from `wait()`, a more advanced concurrency primitive that *does* release its lock while waiting).
- **Thread priority (`setPriority(1–10)`)** — merely a *hint* to the OS/JVM scheduler about relative importance; it is **not** a guarantee of execution order or CPU share, and behaviour can vary across operating systems.
- **CPU cores & concurrency vs parallelism** — `Runtime.getRuntime().availableProcessors()` reveals how many CPU cores the JVM sees. True **parallelism** (multiple threads executing *simultaneously*, at the exact same instant) requires multiple cores; **concurrency** (multiple threads making *interleaved* progress, taking turns) can happen even on a

single core via rapid time-slicing by the OS scheduler — concurrency is about *dealing with* many things at once, parallelism is about *doing* many things at the exact same instant.

9.4 Runnable-based Patterns

The demo set below shows classic sender/receiver coordination and combined Thread+Runnable patterns used to hand off work between concurrently running units — a simplified precursor to the producer-consumer pattern common in real concurrent systems (e.g., message queues).

9.5 Data Inconsistency & the Need for Synchronization

When two threads perform a **read-modify-write** sequence on a shared variable (e.g., a bank account balance: read current balance, compute new balance, write it back) without any coordination, their individual steps can **interleave** unpredictably. For example, both threads might read the *same* starting balance before either has written back its update — one thread's update is then silently **lost** when the other thread overwrites it with a value computed from the same stale starting point. This is called a **race condition**, and it's one of the most important — and most subtle — bug classes in concurrent programming, because it may not reproduce reliably (it depends on precise timing) and can pass all your tests yet still fail intermittently in production under real load.

The DataInconsistencyIssue/SavingAccount demo below reproduces this bug concretely with two threads hitting the same account simultaneously. (The synchronized keyword — which fixes this by allowing only **one thread at a time** into a “critical section” of code, forcing the others to wait their turn — is the natural next topic once this specific problem has been demonstrated and felt firsthand, and is typically covered as the very next lesson after this one.)

All multithreading demo programs (Thread creation via both approaches, Runnable, thread joining, priorities, CPU-core awareness, sender-receiver coordination, and the data-inconsistency race-condition example) are below.

File: Multithreading Demos/ATCILTT_Multithreading_Faculty/ATCILTT_Multithreading_Faculty/com/acc

```
package com.accenture.ltt.threading.activity;

//Code that generates output synchronized from 2 threads
//First thread prints a number from 1-26 and the other prints alphabets against those numbers

public class AlphabetNumberTest {
    public static void main(String[] args)
    {
        Generator obj = new Generator();

        //TODO 1 - Create objects of both the thread classes - NumGenerator and AlphaGenerator
        //    and pass obj as a parameter to them

        NumGenerator numGen = new NumGenerator(obj);
        AlphaGenerator alphaGen = new AlphaGenerator(obj);

        //TODO 2 - Start both the threads
        alphaGen.start();
        numGen.start();
    }
}
```

```

    }
}

class NumGenerator extends Thread
{
    public Generator obj;
    public NumGenerator(Generator obj)
    {
        this.obj = obj;
    }
    public void run()
    {
        //TODO 3 - Run a loop from 1-26 (say i is the variable) and
        //call printNumber method passing i as its parameter
        int i = 1;
        while (i <= 26)
        {
            obj.printNumber(i);
            i = i + 1;
        }
    }
}

class AlphaGenerator extends Thread
{
    public Generator obj;
    public AlphaGenerator(Generator obj)
    {
        this.obj = obj;
    }
    public void run()
    {
        //TODO 4 - Run a loop from A-Z (say c is the variable) and
        //call printAlphabet method passing c as its parameter
        char c = 'A';
        while (c <= 'Z')
        {
            obj.printAlphabet(c);
            c++;
        }
    }
}

class Generator {
    boolean numPrinted = false;
    public synchronized void printNumber(int number)
    {
        //TODO 5 - Check if numPrinted is true, then call wait()
        if (numPrinted == true)
        {
            try

```

```

        {
            wait();
        } catch (InterruptedException e)
        { }
    }

    //TODO 6 - print the number here
    System.out.print(number + ": ");

    //TODO 7 - numPrinted should be assigned true here
    numPrinted = true;

    //TODO 8 - Notify the waiting thread
    notify();
}
public synchronized void printAlphabet(char alphabet)
{
    //TODO 9 - Check if numPrinted is false, then call wait()
    if (numPrinted == false)
    {
        try {
            wait();
        } catch (InterruptedException e) { }
    }

    //TODO 10 - print the alphabet here
    System.out.println(alphabet);

    //TODO 11 - numPrinted should be assigned false here
    numPrinted = false;

    //TODO 12 - Notify the waiting thread using notify()
    notify();
}
}

```

File: Multithreading Demos/ATCILTT_Multithreading_Faculty/ATCILTT_Multithreading_Faculty/com/acc

```
package com.accenture.ltt.threading.activity;
```

```
//TODO 1 - Implement Runnable interface
```

```
class MyRunnableThread implements Runnable {
```

```
//Overriding run method
```

```
public void run() {
```

```

    //TODO 2 - Write a for loop which runs 5 times and each time it is called, print three
    // 1. Entering into the method
    // 2. Inside the method
    // 3. Exiting from the method

```

```
for(int i = 0; i<5; i++) {
```

```
    System.out.println("Entering method by ----- " + Thread.currentThread().getName())
```

```
    System.out.println("inside method " + Thread.currentThread().getName()+ " value of
```



```

        System.out.println("Existing method by ----- " + Thread.currentThread().getName())
    }
}

```

File: Multithreading Demos/ATCILTT_Multithreading_Faculty/ATCILTT_Multithreading_Faculty/com/acc

```

package com.accenture.ltt.threading.activity;

//TODO 1 - extend Thread class
class MyThread extends Thread{

    //Overriding run method
    public void run() {

        //TODO 2 - Write a for loop which runs 5 times and each time it is called, print three
        // 1. Entering into the method
        // 2. Inside the method
        // 3. Exiting from the method
        for(int i = 0; i<5; i++) {
            System.out.println("Entering method ----- " + Thread.currentThread().getName() + "
            System.out.println("inside method " + Thread.currentThread().getName()+ " value of
            System.out.println("Existing method ----- " + Thread.currentThread().getName() +
        }
    }
}

```

File: Multithreading Demos/ATCILTT_Multithreading_Faculty/ATCILTT_Multithreading_Faculty/com/acc

```

package com.accenture.ltt.threading.activity;

public class TestMyRunnable {
    public static void main(String args[]) {
        System.out.println("Thread Name :" + Thread.currentThread().getName());

        // TODO 1 - Instantiate an object (say m1) of MyRunnableThread class
        MyRunnableThread r = new MyRunnableThread();

        // TODO 2 - Create 2 objects of Thread class and pass m1 and a thread name as its param
        Thread thread1 = new Thread(r, "THREADX");
        Thread thread2 = new Thread(r, "THREADY");
        // TODO 3 - Start both the threads using start()
        thread1.start();
        thread2.start();
    }
}

```

File: Multithreading Demos/ATCILTT_Multithreading_Faculty/ATCILTT_Multithreading_Faculty/com/acc

```

package com.accenture.ltt.threading.activity;

public class TestMyThread {
    public static void main(String args[]) {
        System.out.println("Thread Name :" + Thread.currentThread().getName());
    }
}

```

```

        //TODO 1 - create 2 threads by creating objects of MyThread class
        MyThread thread1 = new MyThread();
        thread1.setName("THREADX");
        MyThread thread2 = new MyThread();
        thread2.setName("THREADY");
        //TODO 2 - Start both the threads by using start()
        thread1.start();
        thread2.start();
    }
}

```

File: Multithreading Demos/ATCILTT_Multithreading_Faculty/ATCILTT_Multithreading_Faculty/com/acc

```

package com.accenture.ltt.threading.activity;

/**
 * This class will create 2 thread object.
 * It will start the second thread once the first thread executed for 2 ms.
 * When both the threads complete their execution it will print message "All threads are dead,
 *
 */
public class TestThreadJoin {

    public static void main(String[] args) throws InterruptedException{

        //TODO 1- creating two instances of Thread by passing ThreadJoinTryIt class and thread
        ThreadJoinTryIt thread1 = new ThreadJoinTryIt();
        ThreadJoinTryIt thread2 = new ThreadJoinTryIt();

        //TODO 2- call start method on first thread
        thread1.start();
        //TODO 3 - call join method on thread 1 object by passing 2000 as its argument
        thread1.join(2000);
        //TODO 4 - call start method on second thread object
        thread2.start();
        //TODO 5 - call join method on thread 1 and thread 2 object
        thread1.join();
        thread2.join();
        System.out.println("All threads are dead, exiting main thread");
    }
}

```

File: Multithreading Demos/ATCILTT_Multithreading_Faculty/ATCILTT_Multithreading_Faculty/com/acc

```

package com.accenture.ltt.threading.activity;
//TODO 1 - extends the class with Thread class
public class ThreadJoinTryIt extends Thread{

    public void run() {
        //printing the thread name on entering the method
        System.out.println("Thread started:::" + Thread.currentThread().getName());

        // TODO 2 -Call Thread.sleep method for 4 sec
        try {

```

```

        Thread.sleep(4000);
    } catch (InterruptedException e) {
        // TODO Auto-generated catch block
        e.printStackTrace();
    }

    //printing the thread name on exiting from the method
    System.out.println("Thread ended:::" + Thread.currentThread().getName());
}
}

```

File: Multithreading Demos/ATCILTT_Multithreading_Faculty/ATCILTT_Multithreading_Faculty/com/acc

```
package com.accenture.ltt.threading.cpuscore;
```

```
public class ThreadDemo {
```

```

    public static void main(String[] args) {
        // Display the number of available CPU cores
        int cores = Runtime.getRuntime().availableProcessors();
        System.out.println("Number of CPU Cores: " + cores);
        System.out.println("-----");

        // Create and start multiple threads
        for (int i = 1; i <= cores * 2; i++) { // creating 2x threads than cores
            Thread worker = new Thread(new WorkerTask(), "Thread-" + i);
            worker.start();
        }
    }
}

```

```

class WorkerTask implements Runnable {
    @Override
    public void run() {
        // Each thread prints its name and simulates work
        System.out.println(Thread.currentThread().getName() + " is running on " +
            Thread.currentThread().getThreadGroup().getName());

        try {
            // Simulate some workload
            for (int i = 1; i <= 3; i++) {
                System.out.println("    " + Thread.currentThread().getName() + " is working...");
                Thread.sleep(500); // simulate computation time
            }
        } catch (InterruptedException e) {
            System.out.println(Thread.currentThread().getName() + " interrupted.");
        }

        System.out.println(Thread.currentThread().getName() + " has finished execution.\n");
    }
}

```

File: Multithreading Demos/ATCILTT_Multithreading_Faculty/ATCILTT_Multithreading_Faculty/com/acc

```

package com.accenture.ltt.threading.sample;

public class DataInconsistencyIssue extends Thread{

    float amount;
    SavingAccount savingAccount;

    //constructor of DataInconsistencyIssue class
    public DataInconsistencyIssue(float amount, SavingAccount savingAccount) {
        this.amount = amount;
        this.savingAccount = savingAccount;
    }

    @Override
    public void run() {
        //called withdraw method of SavingAccount
        savingAccount.withdraw(amount);
    }

    public static void main(String[] args) throws InterruptedException {
        //Created object of SavingAccount
        SavingAccount sa = new SavingAccount();

        //created two threads
        DataInconsistencyIssue thread1 = new DataInconsistencyIssue(400.0f, sa);
        DataInconsistencyIssue thread2 = new DataInconsistencyIssue(800.0f, sa);

        //starting both the threads
        thread1.start();
        thread2.start();

        /*calling join on both the threads so that main method
        * will print the value of balance when
        * both the threads finish their task
        */
        thread1.join();
        thread2.join();

        System.out.println("final balance is : " + sa.balance);
    }

}

```

File: Multithreading Demos/ATCILTT_Multithreading_Faculty/ATCILTT_Multithreading_Faculty/com/acc

```

package com.accenture.ltt.threading.sample;

public class RunnableDemo implements Runnable {

    // implements run method of Runnable interface

```

```

@Override
public void run() {
    // print out the name of the thread which is executing this method
    System.out.println("Runnable Thread name :" + Thread.currentThread().getName());
}
}

```

File: Multithreading Demos/ATCILTT_Multithreading_Faculty/ATCILTT_Multithreading_Faculty/com/acc

```

package com.accenture.ltt.threading.sample;

public class SavingAccount {

    // instance variable
    float balance = 1000f;

    // method to withdraw the amount from Saving Account
    //Note - Add synchronization in method signature to show the example of synchronization
    public void withdraw(float amount) {
        if (amount > 0 && amount <= balance) {
            balance = balance - amount;
        }
    }

    // method to deposit the amount into Saving Account
    //Note - Add synchronization in method signature to show the example of synchronization
    public void deposit(float amount) {
        if (amount < 0) {
            throw new IllegalArgumentException("Can not withdraw negative amount");
        }
        balance = balance + amount;
    }
}

```

File: Multithreading Demos/ATCILTT_Multithreading_Faculty/ATCILTT_Multithreading_Faculty/com/acc

```

package com.accenture.ltt.threading.sample;

/**
 * @author ishika.agarwal
 * Sender - Receiver problem
 * The Sender is supposed to send a data packet to the Receiver
 * The Receiver cannot process the data packet until the Sender is finished sending it
 * Similarly, the Sender mustnt attempt to send another packet unless the Receiver has already
 *
 */
public class SenderReceiverTest {
    public static void main(String[] args) {
        Data data = new Data();

        //create object of Sender and Receiver class and start both the threads
        Sender sender = new Sender(data);

```

```

        Receiver receiver = new Receiver(data);

        sender.start();
        receiver.start();
    }
}
// Sender class to send the data
class Sender extends Thread {
    private Data data;

    // standard constructors
    public Sender(Data data) {
        this.data = data;
    }

    public void run() {
        String packets[] = { "First packet", "Second packet", "Third packet", "Fourth packet",

        for (String packet : packets) {
            //call send method of Data class to send the data
            data.send(packet);
            try {
                Thread.sleep(1000);
            } catch (InterruptedException e) {
                // TODO Auto-generated catch block
                e.printStackTrace();
            }
        }
    }
}
//Receiver class to receive the data
class Receiver extends Thread {
    private Data load;

    //standard constructors
    public Receiver(Data load) {
        this.load = load;
    }

    public void run() {
        //call receive method of Data class to receive the data
        String receivedMessage = load.receive();
        //terminating condition : stop when the received message is 'End'
        while(!"End".equals(receivedMessage)){
            System.out.println(receivedMessage);
            //call receive method of Data class to receive the data
            receivedMessage = load.receive();
        }
    }
}
class Data {
    private String packet;

```

```

// True if receiver should wait
// False if sender should wait
private boolean transfer = true;

// method to transfer the data
public synchronized void send(String packet) {
    while (!transfer) {
        try {
            //call wait so that receiver can receive the message before sending another mes
            wait();
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }
    transfer = false;

    this.packet = packet;

    //notifying receiver that message is sent .. please receive it...
    notifyAll();
}

//method to receive the data
public synchronized String receive() {
    while (transfer) {
        try {
            //call wait so that sender can send the next message
            wait();
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }
    transfer = true;

    //notify the sender that receiver has processed the data .. send another one
    notifyAll();
    return packet;
}
}

```

File: Multithreading Demos/ATCILTT_Multithreading_Faculty/ATCILTT_Multithreading_Faculty/com/acc

```

package com.accenture.ltt.threading.sample;

public class TestRunnable {

    public static void main(String args[]) {
        // printing current thread name
        System.out.println(Thread.currentThread().getName());

        // creating two thread objects
        Thread thread1 = new Thread(new RunnableDemo());
        Thread thread2 = new Thread(new RunnableDemo());
    }
}

```

```

        // starting both the threads
        thread1.start();
        thread2.start();
    }
}

```

File: Multithreading Demos/ATCILTT_Multithreading_Faculty/ATCILTT_Multithreading_Faculty/com/acc

```

package com.accenture.ltt.threading.sample;

public class TestThread {

    public static void main(String args[]) {
        // printing current thread name
        System.out.println(Thread.currentThread().getName());

        // creating two thread objects
        ThreadDemo thread1 = new ThreadDemo();
        ThreadDemo thread2 = new ThreadDemo();

        // starting both the threads
        thread1.start();
        thread2.start();
    }

}

```

File: Multithreading Demos/ATCILTT_Multithreading_Faculty/ATCILTT_Multithreading_Faculty/com/acc

```

package com.accenture.ltt.threading.sample;

public class ThreadDemo extends Thread {

    public void run() {
        // print out the name of the thread which is executing this method
        System.out.println("Thread name :" + Thread.currentThread().getName());
    }

}

```

File: Multithreading Demos/ATCILTT_Multithreading_Faculty/ATCILTT_Multithreading_Faculty/com/acc

```

package com.accenture.ltt.threading.sample;

public class ThreadJoinDemo extends Thread {

    @Override
    public void run() {
        //loop from number 0 to 4
        for (int i = 0; i < 5; i++) {

```



```

        System.out.println(i);
        //calling Thread sleep method so that a thread sleep for 2 sec
        try {
            Thread.sleep(2000);
        } catch (InterruptedException e) {
            // TODO Auto-generated catch block
            e.printStackTrace();
        }
    }
    System.out.println("Exiting newly created thread");
}

public static void main(String[] args) throws InterruptedException {
    // creating an instance of ThreadJoinMainClass class
    ThreadJoinDemo thread = new ThreadJoinDemo();
    // calling start so that a new thread create and call run method on that
    thread.start();
    // execute main method once the thread method finish its execution
    //NOTE: execute this program after commenting call to join method
    thread.join();

    System.out.println("Exiting main thread");
}
}

```

File: Multithreading Demos/ATCILTT_Multithreading_Faculty/ATCILTT_Multithreading_Faculty/com/acc

package com.accenture.ltt.threading.sample;

public class ThreadPriorityDemo **extends** Thread{

//overriding run method of Thread class

@Override

public void run() {

//printing thread name and priority

System.out.println("Entering method ----- " + Thread.currentThread().getName());

System.out.println("inside method " + Thread.currentThread().getName() + " having threa

System.out.println("Existing method ----- " + Thread.currentThread().getName());

}

public static void main(String[] args) {

//creating ThreadPriorityDemo class object

ThreadPriorityDemo threadX = new ThreadPriorityDemo();

threadX.setName("THREADX");

//setting the thread priority to Maximum

threadX.setPriority(MAX_PRIORITY);

//creating another object

ThreadPriorityDemo threadY = new ThreadPriorityDemo();

threadY.setName("THREADY");

```

//setting the thread priority to minimum
threadY.setPriority(MIN_PRIORITY);

//creating another object
ThreadPriorityDemo threadZ = new ThreadPriorityDemo();
threadZ.setName("THREADZ");
//setting the thread priority to 4
threadZ.setPriority(4);

//starting the new thread
threadX.start();
threadY.start();
threadZ.start();

}

}

```

9.6 Viva Voce — Likely Questions & Answers

Q1. What is the difference between a process and a thread? A: A process is an independently isolated running program with its own memory space, managed by the OS. A thread is a lightweight unit of execution within a process; multiple threads in the same process share the same heap memory (fast communication, but risk of shared-state bugs) while each still gets its own separate call stack.

Q2. What is the difference between extending Thread and implementing Runnable? Which is preferred and why? A: Extending Thread overrides run() directly but consumes Java's single-inheritance slot. Implementing Runnable and passing it to a Thread is generally preferred — it leaves the class free to extend something else, and cleanly separates “what work to do” from “how it's run” (e.g., allowing reuse with thread pools).

Q3. What happens if you call .run() directly instead of .start()? A: No new thread is created at all — run() executes synchronously on the current calling thread, just like a normal method call, so none of the intended concurrency actually happens. This is one of the most common beginner mistakes.

Q4. What does Thread.join() do? A: It makes the calling thread pause and wait until the target thread has fully finished executing before the caller continues — commonly used to ensure all worker threads complete before combining or reporting their results.

Q5. Does sleep() release any locks the thread is holding? A: No — sleep() pauses the thread but keeps any locks it currently holds. This is different from wait() (a more advanced synchronization primitive), which does release its lock while waiting.

Q6. What is the difference between concurrency and parallelism? A: Concurrency is multiple threads making interleaved progress by taking turns — achievable even on a single CPU core via rapid time-slicing. Parallelism is multiple threads executing at the exact same physical instant, which requires multiple CPU cores.

Q7. What is a race condition? Give a concrete example. A: A bug that occurs when multiple threads perform an unsynchronized read-modify-write on shared data, and their operations interleave such that one thread's update is silently lost. Example: two threads both

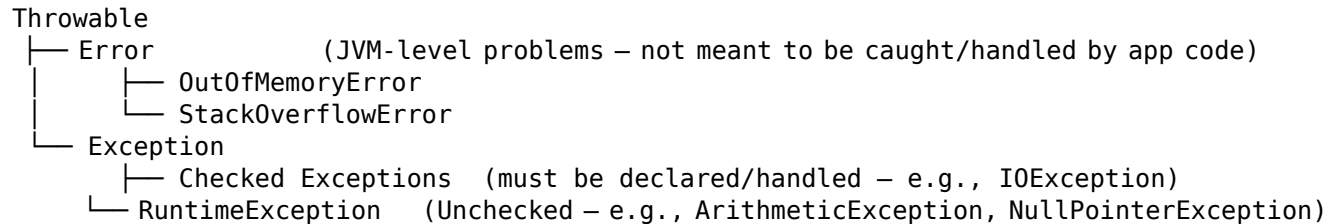
read a bank account's balance as \$100 before either writes back an update — one thread's deposit can be overwritten and lost because both started from the same stale value.

Q8. Does setting a higher thread priority guarantee it runs first or gets more CPU time? A: No — thread priority is only a *hint* to the OS/JVM scheduler; it does not guarantee execution order or CPU share, and its actual effect can vary across different operating systems.

Chapter 10: Exception Handling

10.1 The Exception Hierarchy

Every abnormal event in Java is represented by a class under `java.lang.Throwable`:



- **Checked exceptions** are checked by the *compiler* at compile time — a method that can throw one must either handle it (`try/catch`) or declare it (`throws`), otherwise the code simply won't compile. These represent conditions the caller is expected to *anticipate and plan for* (e.g., a file that might not exist, a network call that might fail) — the compiler forcing acknowledgment is a deliberate design choice to make failure handling part of the API contract.
- **Unchecked (Runtime) exceptions** are **not** checked at compile time — typically these represent *programming bugs* (dividing by zero, calling a method on a null reference, an invalid array index) that, in principle, could have been avoided entirely with correct code/validation, so Java doesn't force every caller to handle them explicitly.
- **Errors** represent serious, usually unrecoverable problems at the JVM level itself (running out of heap memory, an infinite recursion blowing the call stack) that application code is generally **not** expected to catch or recover from — catching an `Error` is technically legal but rarely meaningful, since the JVM itself may already be in an unstable state.

Why does Java distinguish checked vs. unchecked at all (a classic viva question)?

The design intent is: checked exceptions model *recoverable* conditions external to your own code's correctness (I/O, network) where the caller genuinely has a decision to make about how to respond; unchecked exceptions model *programming errors* where the "fix" is to correct the code, not to write a `try/catch` that papers over a bug.

10.2 try / catch / finally

```
try {  
    // risky code  
} catch (SpecificException e) {  
    // handle it  
} finally {  
    // ALWAYS runs – whether an exception was thrown or not, even if the try/catch block itself  
    // returns from the method  
}
```

- **Multiple catch blocks** let you handle different exception types differently — Java evaluates them **top-to-bottom**, so more *specific* exception types must be caught **before** more *general* ones (e.g., `catch (ArithmeticException e)` before `catch (Exception e)`) — putting the general type first would make the specific catch block unreachable, which the compiler actually flags as an error.
- **Multi-catch (Java 7+)** — `catch (IOException | SQLException e)` handles several *unrelated* exception types with one shared block, when the handling logic is identical for both — reduces code duplication versus writing near-identical separate catch blocks.

- **finally** is guaranteed to execute — used almost exclusively for **cleanup** (closing files, releasing database connections/locks) that must happen *regardless* of whether the try block succeeded, failed, or even returned early. The only real exceptions to “always runs” are extreme cases like the JVM itself being forcibly terminated (`System.exit()`), a crash, or the machine losing power).
- **try-with-resources (Java 7+)** — automatically closes any resource implementing `AutoCloseable/Closeable` when the try block exits (whether normally or via exception), completely removing the need for a manual `finally { resource.close(); }` block, and correctly handling the tricky case of an exception being thrown *during* the `close()` call itself (suppressed exceptions).

10.3 throw vs throws — A Frequently Confused Pair

	throw	throws
What it is	A statement , used inside a method body	A clause in a method’s signature
Purpose	Actually raises/triggers a specific exception instance right now	Declares that this method <i>might</i> propagate a checked exception up to its caller
Example	<code>throw new IllegalArgumentException("bad input");</code>	<code>public void readFile() throws IOException { ... }</code>
Count	Exactly one exception object per throw statement	Can list multiple exception types, comma-separated

10.4 Custom (User-Defined) Exceptions

You create a custom exception by extending `Exception` (making it **checked**) or `RuntimeException` (making it **unchecked**), typically adding a constructor that forwards a message (and optionally a cause) to `super(message)`. This is used to model *domain-specific* failure conditions clearly and expressively — e.g., `EmployeeDoesNotExistException` or `InvalidCredentialsException` communicate intent far better to a caller (and in a stack trace) than a generic `Exception` or `RuntimeException` would, and let calling code selectively catch just that specific domain failure without accidentally swallowing unrelated errors too.

Design guideline: extend `RuntimeException` for conditions that represent a programming mistake/invalid state the caller should generally not be forced to handle everywhere; extend `Exception` for conditions a well-behaved caller genuinely needs to anticipate and is expected to recover from.

10.5 Common Runtime Exceptions & Errors Demonstrated

- **ArithmeticException** — e.g., integer division by zero (`5 / 0` throws this; note that *floating-point* division by zero, e.g. `5.0 / 0`, does **not** throw — it silently produces Infinity or NaN instead, per IEEE-754 rules, which is itself a common interview trick question).
- **StackOverflowError** — uncontrolled/infinite recursion exhausts the fixed-size call-stack memory allotted to a thread; each recursive call pushes a new stack frame, and without a proper base case the stack eventually runs out of room.
- **OutOfMemoryError** — the heap fills up with live (still-referenced) objects faster than the garbage collector can reclaim space, typically indicating either a genuine memory leak (objects unintentionally kept referenced forever) or simply processing more data than the configured heap size (`-Xmx`) can hold.

All demo files below are grouped by category: **runtime (unchecked) exceptions** (try/catch, multi-catch styles across JDK versions, finally, throw, try-with-resources), **checked exceptions** (try/catch, throws, multi-catch), **custom exceptions**, and **Errors**. Each topic includes both the original “**practice**” version (the problem as posed to trainees) and the instructor’s “**solutions**” version where relevant, so you can compare your own attempt against the reference solution.

10.5.1 Runtime (Unchecked) Exceptions

File: ATCILT_Exception_Faculty/com/accenture/ltt/exceptions/runtime/Arithmetic.java

```
package com.accenture.ltt.exceptions.runtime;

public class Arithmetic {
    public static void main(String[] args) {
        int a = 10;
        int b = 0;
        int c = 0;
        try {
            c = a / b;
        } catch (ArithmeticException e) {
            System.out.println(e.getMessage());
        }
        System.out.println("c = " + c);
    }
}
```

File: ATCILT_Exception_Faculty/com/accenture/ltt/exceptions/runtime/FinallyClause.java

```
package com.accenture.ltt.exceptions.runtime;

import java.util.InputMismatchException;
import java.util.Scanner;

public class FinallyClause {
    public static void division() {
        Scanner sc = new Scanner(System.in);
        int a;
        int b;
        int c;

        try {
            System.out.println("Enter a and b ");
            a = sc.nextInt();
            b = sc.nextInt();
            c = a / b;
            System.out.println("result = " + c);
        } catch (InputMismatchException | ArithmeticException e) {
            e.printStackTrace();
        } finally {
            System.out.println("finally block executed");
            sc.close();
        }
    }
}
```

```

        public static void main(String[] args) {
            division();
        }
    }
}

```

File: ATCILTT_Exception_Faculty/com/accenture/ltt/exceptions/runtime/MultipleCatchJDK7.java

```

package com.accenture.ltt.exceptions.runtime;

import java.util.InputMismatchException;
import java.util.Scanner;

public class MultipleCatchJDK7 {
    public void division() {

        System.out.println("Enter Nr and dr");

        int q = 0;
        Scanner sc = new Scanner(System.in);
        try {

            int nr = sc.nextInt();
            int dr = sc.nextInt();

            q = nr / dr;

        } catch (ArithmeticException | InputMismatchException e) {
            e.printStackTrace();
        }
        sc.close();
        System.out.println("quotient = " + q);

    }

    public static void main(String[] args) {
        MultipleCatchJDK7 a = new MultipleCatchJDK7();
        a.division();
    }
}

```

File: ATCILTT_Exception_Faculty/com/accenture/ltt/exceptions/runtime/MutipleCatch.java

```

package com.accenture.ltt.exceptions.runtime;

import java.util.InputMismatchException;
import java.util.Scanner;

public class MutipleCatch {
    public void division() {

        System.out.println("Enter Nr and dr");

        int q = 0;
        Scanner sc = new Scanner(System.in);

```

```

    try {
        int nr = sc.nextInt();
        int dr = sc.nextInt();

        q = nr / dr;

    } catch (InputMismatchException i) {
        i.printStackTrace();
    } catch (ArithmeticException e) {
        e.printStackTrace();
    }
    sc.close();
    System.out.println("quotient = " + q);

}

public static void main(String[] args) {
    MutipleCatch a = new MutipleCatch();
    a.division();
}
}

```

File: ATCILTT_Exception_Faculty/com/accenture/ltt/exceptions/runtime/ThrowClause.java

```

package com.accenture.ltt.exceptions.runtime;

import java.util.InputMismatchException;
import java.util.Scanner;

public class ThrowClause {
    public void division() {

        System.out.println("Enter Nr and dr");

        int q = 0;
        Scanner sc = new Scanner(System.in);
        try {
            int nr = sc.nextInt();
            int dr = sc.nextInt();

            q = nr / dr;

        } catch (ArithmeticException | InputMismatchException e) {
            throw e;
        }
        sc.close();
        System.out.println("quotient = " + q);

    }

    public static void main(String[] args) {
        MutipleCatch a = new MutipleCatch();
        a.division();
    }
}

```



```
}  
}
```

File: ATCILT_Exception_Faculty/com/accenture/ltt/exceptions/runtime/TryCatch.java

```
package com.accenture.ltt.exceptions.runtime;  
  
import java.util.Scanner;  
  
public class TryCatch {  
  
    public void division(byte nr, byte dr) {  
        try {  
            System.out.println("Quotient = " + nr/dr);  
        } catch (ArithmeticException ae) {  
            System.out.println(ae.getMessage());  
        }  
    }  
  
    public static void main(String[] args) {  
        TryCatch a = new TryCatch();  
        Scanner sc = new Scanner(System.in);  
        char ch;  
        do {  
            System.out.println("Enter Numerator [-128 to 127]");  
            byte nr = sc.nextByte();  
  
            System.out.println("Enter Dr [-128 to 127]");  
            byte dr = sc.nextByte();  
            a.division(nr, dr);  
  
            System.out.print("Enter [ Y ] to continue OR --> any key to exit\n");  
            ch = sc.next().charAt(0);  
            System.out.println("*****");  
  
        } while (ch == 'y' || ch == 'Y');  
        sc.close();  
    }  
}
```

File: ATCILT_Exception_Faculty/com/accenture/ltt/exceptions/runtime/TryWithResources.java

```
package com.accenture.ltt.exceptions.runtime;  
  
import java.util.InputMismatchException;  
import java.util.Scanner;  
  
public class TryWithResources {  
  
    public void division() {  
        System.out.println("Enter Nr and dr");  
    }  
}
```

```

int q = 0;
try (Scanner sc = new Scanner(System.in);) {

    int nr = sc.nextInt();
    int dr = sc.nextInt();

    q = nr / dr;

} catch (ArithmeticException | InputMismatchException e) {
    e.printStackTrace();
}

System.out.println("quotient = " + q);

}

public static void main(String[] args) {
    TryWithResources a = new TryWithResources();
    a.division();
}
}

```

File: ATCILTT_Exception_Faculty/com/accenture/ltt/exceptions/runtime/solutions/FinallyClause.java

```

package com.accenture.ltt.exceptions.runtime.solutions;

import java.util.InputMismatchException;
import java.util.Scanner;

public class FinallyClause {

    public static void input() {
        int numbers[] = { 10, 20, 30, 40, 50 };
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter index to [1 - 6] to find the element stored at that index");
        try {
            int index = sc.nextInt();
            System.out.println(numbers[index]);
        } catch (IndexOutOfBoundsException | InputMismatchException e) {
            throw e;
        } finally {
            System.out.println("finally block executed");
            sc.close();
        }
    }

    public static void main(String[] args) {
        input();
    }
}

```

File: ATCILTT_Exception_Faculty/com/accenture/ltt/exceptions/runtime/solutions/MultipleCatch.java

```

package com.accenture.ltt.exceptions.runtime.solutions;

import java.util.InputMismatchException;
import java.util.Scanner;

public class MultipleCatch {
    public static void input() {

        int numbers[] = { 10, 20, 30, 40, 50 };
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter index to [1 - 6] to find the element stored at that index");
        try {
            int index = sc.nextInt();
            System.out.println(numbers[index]);
        } catch (InputMismatchException i) {
            i.printStackTrace();
        } catch (IndexOutOfBoundsException iob) {
            iob.printStackTrace();
        }
        sc.close();
    }

    public static void main(String[] args) {
        input();
    }
}

```

File: ATCILTT_Exception_Faculty/com/accenture/ltt/exceptions/runtime/solutions/MultipleCatchJDK7

```

package com.accenture.ltt.exceptions.runtime.solutions;

import java.util.InputMismatchException;
import java.util.Scanner;

public class MultipleCatchJDK7 {
    public static void input() {

        int numbers[] = { 10, 20, 30, 40, 50 };
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter index to [1 - 6] to find the element stored at that index");
        try {
            int index = sc.nextInt();
            System.out.println(numbers[index]);
        } catch (InputMismatchException | IndexOutOfBoundsException e) {
            e.printStackTrace();
        }
        sc.close();
    }

    public static void main(String[] args) {
        input();
    }
}

```

```
}
```

File: ATCILTT_Exception_Faculty/com/accenture/ltt/exceptions/runtime/solutions/ThrowClause.java

```
package com.accenture.ltt.exceptions.runtime.solutions;

import java.util.InputMismatchException;
import java.util.Scanner;

public class ThrowClause {
    public static void input() {
        int numbers[] = { 10, 20, 30, 40, 50 };
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter index to [1 - 6] to find the element stored at that index");
        try {
            int index = sc.nextInt();
            System.out.println(numbers[index]);
        } catch (InputMismatchException | IndexOutOfBoundsException e) {
            throw e;
        }
        sc.close();
    }

    public static void main(String[] args) {
        input();
    }
}
```

File: ATCILTT_Exception_Faculty/com/accenture/ltt/exceptions/runtime/solutions/TryCatch.java

```
package com.accenture.ltt.exceptions.runtime.solutions;

import java.util.Scanner;

public class TryCatch {

    public static void input() {
        int numbers[] = { 10, 20, 30, 40, 50 };
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter index to [1 - 6] to find the element stored at that index");
        try {
            int index = sc.nextInt();
            System.out.println(numbers[index]);
        } catch (IndexOutOfBoundsException e) {
            e.printStackTrace();
        }
        sc.close();
    }

    public static void main(String[] args) {
        input();
    }
}
```

File: ATCILTT_Exception_Faculty/com/accenture/ltt/exceptions/runtime/solutions/TryWithResources.

```

package com.accenture.ltt.exceptions.runtime.solutions;

import java.util.InputMismatchException;
import java.util.Scanner;

public class TryWithResources {

    public static void input() {
        int numbers[] = { 10, 20, 30, 40, 50 };

        System.out.println("Enter index to [1 - 6] to find the element stored at that index");
        try (Scanner sc = new Scanner(System.in);) {
            int index = sc.nextInt();
            System.out.println(numbers[index]);
        } catch (InputMismatchException | IndexOutOfBoundsException e) {
            e.printStackTrace();
        }
    }

    public static void main(String[] args) {
        input();
    }
}

```

10.5.2 Checked Exceptions

File: ATCILT_Exception_Faculty/com/accenture/ltt/exceptions/checked/MultipleCatch.java

```

package com.accenture.ltt.exceptions.checked;

import java.io.FileNotFoundException;
import java.io.FileReader;
import java.io.IOException;

public class MultipleCatch {
    public static void read() {
        try {
            FileReader fr = new FileReader("src/files/users.txt");
            int ch;
            while ((ch = fr.read()) != -1) {
                System.out.print((char) ch);
            }

        } catch (FileNotFoundException e) {
            e.printStackTrace();
        } catch (IOException e) {
            e.printStackTrace();
        }
    }

    public static void main(String[] args) {
        read();
    }
}

```

```
}  
}
```

File: ATCILTT_Exception_Faculty/com/accenture/ltt/exceptions/checked/ThrowsClause.java

```
package com.accenture.ltt.exceptions.checked;  
  
import java.io.FileReader;  
import java.io.IOException;  
  
public class ThrowsClause {  
  
    public static void read() throws IOException {  
        try (FileReader fr = new FileReader("src/files/users.txt");) {  
            int ch;  
            while ((ch = fr.read()) != -1) {  
                System.out.print((char) ch);  
            }  
        }  
    }  
  
    public static void main(String[] args) {  
        try {  
            ThrowsClause.read();  
        } catch (IOException e) {  
            e.printStackTrace();  
        }  
    }  
}
```

File: ATCILTT_Exception_Faculty/com/accenture/ltt/exceptions/checked/TryCatch.java

```
package com.accenture.ltt.exceptions.checked;  
  
import java.io.FileNotFoundException;  
import java.io.FileReader;  
  
public class TryCatch {  
    public static void read() {  
        try {  
            FileReader fr = new FileReader("src/files/welcome.txt");  
        } catch (FileNotFoundException e) {  
            e.printStackTrace();  
        }  
    }  
  
    public static void main(String[] args) {  
        read();  
    }  
}
```

File: ATCILTT_Exception_Faculty/com/accenture/ltt/exceptions/checked/solutions/MultipleCatch.java

```

package com.accenture.ltt.exceptions.checked.solutions;

import java.io.FileNotFoundException;
import java.io.FileWriter;
import java.io.IOException;

public class MultipleCatch {
    public static void write() {

        try {
            FileWriter fw = new FileWriter("d://src/files/users.txt", true);
            fw = new FileWriter("d://src/files/users.txt");
            fw.write("john@gmail.com");
            fw.flush();
        } catch (FileNotFoundException f) {
            System.out.println(f.getMessage());
        } catch (IOException e) {
            System.out.println(e.getMessage());
        }
    }

    public static void main(String[] args) {
        write();
    }
}

```

File: ATCILTT_Exception_Faculty/com/accenture/ltt/exceptions/checked/solutions/ThrowsClause.java

```

package com.accenture.ltt.exceptions.checked.solutions;

import java.io.FileWriter;
import java.io.IOException;

public class ThrowsClause {
    public static void write() throws IOException {

        try (FileWriter fw = new FileWriter("src/files/users.txt", true);) {
            fw.write("john@gmail.com");
            fw.write(System.lineSeparator());
            fw.flush();
            System.out.println("record created");
        }

    }

    public static void main(String[] args) {
        try {
            write();
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}

```

File: ATCILTT_Exception_Faculty/com/accenture/ltt/exceptions/checked/solutions/TryCatch.java

```
package com.accenture.ltt.exceptions.checked.solutions;

import java.io.FileWriter;
import java.io.IOException;

public class TryCatch {
    public static void write() {
        try {
            FileWriter fw = new FileWriter("src/files/users.txt", true);

        } catch (IOException e) {
            System.out.println(e.getMessage());
        }
    }

    public static void main(String[] args) {
        write();
    }
}
```

10.5.3 Custom Exceptions

File: ATCILTT_Exception_Faculty/com/accenture/ltt/exceptions/customexceptions/EmployeeDoesNotExistException.java

```
package com.accenture.ltt.exceptions.customexceptions;

public class EmployeeDoesNotExistException extends Exception{

    private static final long serialVersionUID = 1L;

    public EmployeeDoesNotExistException(String message) {
        super(message);
    }
}
```

File: ATCILTT_Exception_Faculty/com/accenture/ltt/exceptions/customexceptions/EmployeeTest.java

```
package com.accenture.ltt.exceptions.customexceptions;

import java.util.Scanner;

public class EmployeeTest {

    public static void findEmp(int index) throws EmployeeDoesNotExistException {
        String names[] = { "tom", "jack", "mike" };
        if (index < 0 || index > 2) {
            throw new EmployeeDoesNotExistException("Employee does not exist");
        }
        System.out.println(names[index]);
    }

    public static void main(String[] args) throws EmployeeDoesNotExistException {
```



```

        System.out.println("Enter index [1-5] to find emp name");

        try (Scanner sc = new Scanner(System.in));) {

            int index = sc.nextInt();
            findEmp(index);

        }

    }
}

```

File: ATCILT_Exception_Faculty/com/accenture/ltt/exceptions/customexceptions/solutions/InvalidC

```

package com.accenture.ltt.exceptions.customexceptions.solutions;

public class InvalidCredentialsException extends Exception {

    private static final long serialVersionUID = 1L;

    public InvalidCredentialsException() {
        super();
    }

    public InvalidCredentialsException(String message) {
        super(message);
    }

}

```

File: ATCILT_Exception_Faculty/com/accenture/ltt/exceptions/customexceptions/solutions/User.java

```

package com.accenture.ltt.exceptions.customexceptions.solutions;

import java.util.Scanner;

public class User {
    String username = null;
    String password = null;

    public void validate() throws InvalidCredentialsException {

        if (username.equals("tom") && password.equals("tom@123")) {
            System.out.println("valid password");
        } else {
            throw new InvalidCredentialsException("username or password is invalid");
        }
    }

    public void input() {
        try (Scanner sc = new Scanner(System.in));) {
            System.out.println("Enter username");
            username = sc.next();

            System.out.println("Enter password");

```

```

        password = sc.next();
    }
}

public static void main(String[] args) throws InvalidCredentialsException {
    User user = new User();
    user.input();
    user.validate();
}
}

```

10.5.4 Errors (OutOfMemoryError, StackOverflowError)

File: ATCILTT_Exception_Faculty/com/accenture/ltt/exceptions/errors/OutOfMemoryErrorEx.java

```

package com.accenture.ltt.exceptions.errors;

public class OutOfMemoryErrorEx {
    public static void main(String[] args) {
        int j = 1;
        while(true) {
            int i[] = new int[Integer.MAX_VALUE];
            System.out.println(j++);
        }
    }
}

```

File: ATCILTT_Exception_Faculty/com/accenture/ltt/exceptions/errors/StackOverflowErrorEx.java

```

package com.accenture.ltt.exceptions.errors;

public class StackOverflowErrorEx {

    public static void main(String[] args) {
        String s[] = {"asdf"};
        main(s);
    }
}

```

File: ATCILTT_Exception_Faculty/com/accenture/ltt/exceptions/errors/solutions/OutOfMemoryErrorEx

```

package com.accenture.ltt.exceptions.errors.solutions;

import java.util.Calendar;
import java.util.Date;

public class OutOfMemoryErrorEx {
    public static void main(String[] args) {
        Employee e[] = new Employee[5];
        for(int i = 0 ; i < e.length; i++) {
            e[i] = new Employee();
            System.out.println(e[i]);
        }
    }
}

```

```

}

class Employee {
    int id;
    String name;
    String lname;
    String password;
    String email;
    Date dob;
    String skills[] = { "Java", "C", "C++", "C#", "Python", "SQL" };
    Address address = null;
    public Employee() {
        this(100, "John", "Michael", "secret", "john@gmail.com", Calendar.getInstance().getTime(),
            address = new Address());
    }

    public Employee(int id, String name, String lname, String password, String email, Date dob) {
        super();
        this.id = id;
        this.name = name;
        this.lname = lname;
        this.password = password;
        this.email = email;
        this.dob = dob;
    }

    @Override
    public String toString() {
        return "Employee [id=" + id + ", name=" + name + ", lname=" + lname + ", password=" + password + ", email=" + email + ", dob=" + dob + "]";
    }
}

class Address{
    String city ;
    String state;
    String country;
    int x[] = new int[99999999];

    public Address() {
        this("Hyderabad", "Telangana", "Country") ;
    }
    public Address(String city, String state, String country) {
        super();
        this.city = city;
        this.state = state;
        this.country = country;
    }
}

```

File: ATCILTT_Exception_Faculty/com/accenture/ltt/exceptions/errors/solutions/StackOverflowError

```

package com.accenture.ltt.exceptions.errors.solutions;

public class StackOverflowErrorEx {

    public static void main(String[] args) {
        System.out.println(factorial(5));
    }

    public static int factorial(int n) {
        return n * factorial((n - 1));
    }
}

```

10.6 Viva Voce — Likely Questions & Answers

Q1. What is the difference between checked and unchecked exceptions? A: Checked exceptions are verified by the compiler — a method that might throw one must either handle it or declare it with throws, otherwise it won't compile (they model recoverable external conditions like I/O failures). Unchecked exceptions (subclasses of RuntimeException) are not checked at compile time and typically represent programming bugs.

Q2. What is the difference between Exception and Error? A: Exceptions (checked or unchecked) are conditions an application might reasonably want to catch and handle. Errors represent serious JVM-level problems (like running out of memory) that application code generally isn't expected to recover from.

Q3. What is the difference between throw and throws? A: throw is a statement inside a method body that actually raises a specific exception instance right now. throws is a clause in a method's *signature* declaring that the method might propagate a checked exception to its caller.

Q4. Does finally always execute? Are there exceptions to this? A: Yes, essentially always — whether the try block succeeds, throws, or even returns early. The only exceptions are extreme JVM-terminating events like System.exit(), a JVM crash, or a power loss.

Q5. What is try-with-resources, and what problem does it solve? A: A try block that automatically closes any resource(s) implementing AutoCloseable when the block exits (whether normally or via exception), eliminating the need for a manual finally { resource. close(); } and correctly handling exceptions thrown during the close() call itself.

Q6. Why must more specific catch blocks come before more general ones? A: Because Java evaluates catch blocks top-to-bottom and uses the *first* matching one; placing a general type (like Exception) before a more specific one would make the specific catch block unreachable, which the compiler actually flags as an error.

Q7. How do you create a custom exception, and when would you make it checked vs. unchecked? A: Extend Exception (checked) or RuntimeException (unchecked), usually with a constructor forwarding a message to super(message). Use checked for conditions a well-behaved caller should anticipate and can meaningfully recover from; use unchecked for conditions representing a programming/state error the caller shouldn't be forced to handle everywhere.

Q8. Does 5 / 0 and 5.0 / 0 behave the same way in Java? A: No — integer division 5 / 0 throws ArithmeticException. Floating-point division 5.0 / 0 does *not* throw; it evaluates to Infinity (or NaN for 0.0/0.0), per IEEE-754 floating-point rules. This distinction is a common trick question.

Q9. What causes a `StackOverflowError`, and how would you generally fix it? A: Uncontrolled or infinite recursion, where each recursive call pushes another frame onto a thread's fixed-size call stack until it runs out of room. The fix is ensuring the recursion has a correct, reachable base case (or converting the algorithm to an iterative approach).

Q10. What is multi-catch, and what's its benefit? A: `catch (TypeA | TypeB e)` handles multiple unrelated exception types in a single catch block — useful when the handling logic is identical for both, reducing duplicated catch-block code.

Chapter 11: Data Structures (Without Collections API)

11.1 What is a Data Structure, and Why Learn This Manually First?

A data structure is a way of organizing and storing data so it can be accessed and modified efficiently for a given set of operations. Before learning Java’s built-in Collections framework (Chapter 12), it’s essential to understand the fundamental structures **conceptually and mechanically** — this chapter implements them manually (using plain arrays, Node objects, and pointer/reference chaining) precisely so that when you later use `ArrayList`, `LinkedList`, `Stack`, etc. from `java.util`, you understand *what those classes are actually doing internally*, rather than treating them as opaque magic.

11.2 Searching & Sorting — Conceptual Foundation

- **Linear Search** — check every element one by one from the start until a match is found (or the end is reached); **$O(n)$** time complexity in the worst case; works on **unsorted** data since it makes no assumptions about ordering.
- **Binary Search** — repeatedly compares the target to the *middle* element and discards the half of the search range that cannot contain it, halving the remaining search space each step; **$O(\log n)$** time complexity — dramatically faster than linear search for large datasets — but **strictly requires the data to already be sorted**, otherwise the “discard half” logic is invalid.
- **Time-complexity intuition (a common viva ask)**: for 1,000,000 sorted elements, linear search could need up to 1,000,000 comparisons in the worst case, while binary search needs at most about 20 (since $2^{20} \approx 1,000,000$) — this is the entire *point* of maintaining sorted order.
- **Sorting algorithms** rearrange elements into order so binary search and other optimizations become possible. At a basic level: **Bubble Sort** (repeatedly swap adjacent out-of-order pairs, $O(n^2)$), **Selection Sort** (repeatedly find the minimum remaining element and place it, $O(n^2)$), **Insertion Sort** (build up a sorted section one element at a time, $O(n^2)$ worst case but fast on nearly-sorted data). More advanced, better-performing sorts (**Merge Sort**, **Quick Sort** — both $O(n \log n)$ average case) are typically covered as a follow-on/interview-prep topic beyond this introductory module, and are what Java’s own `Collections.sort()`/`Arrays.sort()` actually use internally (a hybrid of these, tuned per data type).

11.3 Core Structures Implemented From Scratch

- **Array** — fixed size, contiguous memory, **$O(1)$** index access (direct address computation), but **$O(n)$** insertion/deletion in the middle (every subsequent element must be shifted to make or close a gap).
- **Node & Linked List** — each Node object holds a piece of data **plus a reference to the next node** in the chain; a `LinkedList` is simply a chain of these nodes starting from a head reference. Insertion/deletion is **$O(1)$** *once you already have a reference to the correct position* (you just relink a couple of pointers), but **finding** that position first is **$O(n)$** since — unlike an array — a linked list offers **no random access**; you must walk the chain node-by-node from the head to reach any given position.
- **Array vs Linked List — the classic head-to-head comparison:**

	Array	Linked List
Memory layout	Contiguous	Scattered, linked via references

	Array	Linked List
Random access (<code>get(i)</code>)	$O(1)$	$O(n)$ — must traverse from head
Insert/delete at a known position	$O(n)$ — must shift elements	$O(1)$ — just relink pointers
Size	Fixed at creation	Grows/shrinks dynamically, node by node
Memory overhead	None extra per element	Extra reference field per node

- **Stack — LIFO** (Last-In-First-Out) behaviour: `push()` adds to the top, `pop()` removes from the top, `peek()` looks at the top without removing it. Real-world uses: undo/redo functionality, expression evaluation (converting infix to postfix, balancing parentheses), and — notably — the JVM's own **call stack** itself is conceptually a stack of method-invocation frames (which is exactly why uncontrolled recursion causes a `StackOverflowError`, tying directly back to Chapter 10).
- **Queue — FIFO** (First-In-First-Out) behaviour: `enqueue()/offer()` adds at the rear, `dequeue()/poll()` removes from the front. Real-world uses: task scheduling (first request in is the first processed), breadth-first traversal of trees/graphs, and print/job queues.
- **Binary Tree** — a hierarchical (non-linear) structure where each `TreeNode` has at most **two** children, conventionally called `left` and `right`. This enables efficient hierarchical data representation; when additionally kept *ordered* (a **Binary Search Tree**, where every left descendant is smaller and every right descendant is larger than its parent) and reasonably *balanced*, it enables **$O(\log n)$** search, insertion, and deletion — combining the fast-search benefit of sorted arrays with the fast-insertion benefit of linked structures.

The demo programs below implement each of these structures manually using plain arrays and a custom `Node/TreeNode` class — no `java.util.*` collection classes are used, by design, so the underlying mechanics remain fully visible rather than hidden behind a library API.

File: `Demo_data_structures_withoout_Collection/com/accentur/ltt/ArrayExample.java`

```
package com.accentur.ltt;

public class ArrayExample {
    public static void main(String[] args) {
        int[] numbers = new int[5]; // Fixed-size array
        numbers[0] = 10;
        numbers[1] = 20;
        numbers[2] = 30;

        System.out.println("Array elements:");
        for (int num : numbers) {
            System.out.println(num);
        }
    }
}

/*
=>Can be replaced with:
List<Integer> numbers = new ArrayList<>();
numbers.add(10);
numbers.add(20);
*/
```

```
numbers.add(30);
*/
```

File: Demo_data_structures_withoout_Collection/com/accentur/ltt/BinaryTreeExample.java

```
package com.accentur.ltt;

public class BinaryTreeExample {
    TreeNode root;

    void inorder(TreeNode node) {
        if (node == null) return;
        inorder(node.left);
        System.out.print(node.data + " ");
        inorder(node.right);
    }

    public static void main(String[] args) {
        BinaryTreeExample tree = new BinaryTreeExample();
        tree.root = new TreeNode(10);
        tree.root.left = new TreeNode(5);
        tree.root.right = new TreeNode(15);
        tree.root.left.left = new TreeNode(2);
        tree.root.left.right = new TreeNode(7);

        System.out.println("Inorder Traversal:");
        tree.inorder(tree.root);
    }
}

/*
Can be replaced with:
TreeSet<Integer> tree = new TreeSet<>();
tree.add(10);
tree.add(5);
tree.add(15);
tree.add(2);
tree.add(7);
System.out.println(tree); // Sorted automatically
*/
```

File: Demo_data_structures_withoout_Collection/com/accentur/ltt/LinkedListExample.java

```
package com.accentur.ltt;

public class LinkedListExample {
    Node head;

    // Insert new node at end
    void insert(int data) {
        Node newNode = new Node(data);
        if (head == null) {
            head = newNode;
            return;
        }
    }
}
```



```

        Node temp = head;
        while (temp.next != null)
            temp = temp.next;
        temp.next = newNode;
    }

    // Display list
    void display() {
        Node current = head;
        while (current != null) {
            System.out.print(current.data + " -> ");
            current = current.next;
        }
        System.out.println("null");
    }

    public static void main(String[] args) {
        LinkedListExample list = new LinkedListExample();
        list.insert(10);
        list.insert(20);
        list.insert(30);
        list.display();
    }
}

/*Can be replaced with:
LinkedList<Integer> list = new LinkedList<>();
list.add(10);
list.add(20);
list.add(30);
*/

```

File: Demo_data_structures_withoout_Collection/com/accentur/ltt/Node.java

```

package com.accentur.ltt;

//Node class for Linked List
class Node {
    int data;
    Node next;

    Node(int data) {
        this.data = data;
        this.next = null;
    }
}

```

File: Demo_data_structures_withoout_Collection/com/accentur/ltt/QueueExample.java

```

package com.accentur.ltt;

//Simple Queue using Array
public class QueueExample {
    int[] queue = new int[5];
    int front = -1, rear = -1;
}

```

```

void enqueue(int data) {
    if (rear == queue.length - 1) {
        System.out.println("Queue Full");
        return;
    }
    if (front == -1) front = 0;
    queue[++rear] = data;
}

int dequeue() {
    if (front == -1 || front > rear) {
        System.out.println("Queue Empty");
        return -1;
    }
    return queue[front++];
}

void display() {
    System.out.print("Queue elements: ");
    for (int i = front; i <= rear; i++)
        System.out.print(queue[i] + " ");
    System.out.println();
}

public static void main(String[] args) {
    QueueExample q = new QueueExample();
    q.enqueue(10);
    q.enqueue(20);
    q.enqueue(30);
    q.display();
    System.out.println("Dequeued: " + q.dequeue());
    q.display();
}
}

/*
    Can be replaced with:
    Queue<Integer> queue = new LinkedList<>();
    queue.add(10);
    queue.add(20);
    queue.add(30);
    queue.remove();
*/

```

File: Demo_data_structures_withoout_Collection/com/accentur/ltt/StackExample.java

```

package com.accentur.ltt;

//Simple Stack using Array
public class StackExample {
    private int[] stack = new int[5];
    private int top = -1;

    void push(int data) {

```

```

        if (top == stack.length - 1) {
            System.out.println("Stack Overflow");
            return;
        }
        stack[++top] = data;
    }

    int pop() {
        if (top == -1) {
            System.out.println("Stack Underflow");
            return -1;
        }
        return stack[top--];
    }

    void display() {
        System.out.println("Stack elements:");
        for (int i = top; i >= 0; i--)
            System.out.println(stack[i]);
    }

    public static void main(String[] args) {
        StackExample stack = new StackExample();
        stack.push(10);
        stack.push(20);
        stack.push(30);
        stack.display();
        System.out.println("Popped: " + stack.pop());
        stack.display();
    }
}

/*
Can be replaced with:
Stack<Integer> stack = new Stack<>();
stack.push(10);
stack.push(20);
stack.push(30);
stack.pop();
*/

```

File: Demo_data_structures_withoout_Collection/com/accentur/ltt/TreeNode.java

```

package com.accentur.ltt;

//Simple Binary Tree Example
class TreeNode {
    int data;
    TreeNode left, right;

    TreeNode(int data) {
        this.data = data;
        left = right = null;
    }
}

```

}

11.4 Viva Voce — Likely Questions & Answers

Q1. What is the time complexity of linear search vs. binary search, and what precondition does binary search require? A: Linear search is $O(n)$ worst case and works on unsorted data. Binary search is $O(\log n)$ but **requires the data to already be sorted** — it works by repeatedly discarding the half of the remaining range that cannot contain the target.

Q2. Compare arrays and linked lists in terms of access time and insertion/deletion time. A: Arrays give $O(1)$ random access by index but $O(n)$ insertion/deletion in the middle (elements must shift). Linked lists give $O(n)$ access (must traverse from the head, no random access) but $O(1)$ insertion/deletion *once you already hold a reference to the correct position*, since only a couple of pointers need relinking.

Q3. What is the difference between a Stack and a Queue? A: A Stack is LIFO (Last-In-First-Out) — the most recently added element is removed first (think of a stack of plates). A Queue is FIFO (First-In-First-Out) — the earliest added element is removed first (think of a checkout line).

Q4. How does the JVM's own call stack relate to the Stack data structure, and what error results from misusing it? A: Each method call pushes a new frame onto the thread's call stack, and returning pops it off — a direct real-world instance of a Stack (LIFO). Uncontrolled/infinite recursion keeps pushing frames without ever popping, eventually exhausting the stack's fixed memory and throwing a `StackOverflowError`.

Q5. What is a Binary Search Tree, and why is it more efficient than a plain sorted array for insertions? A: A tree where every node's left subtree contains only smaller values and its right subtree only larger values, enabling $O(\log n)$ search when reasonably balanced. Unlike a sorted array, inserting a new value doesn't require shifting existing elements — you simply attach a new node at the correct leaf position, which is why BSTs suit datasets with frequent insertions better than sorted arrays.

Q6. Why does this chapter avoid using `java.util.*` collection classes? A: Deliberately, so the underlying mechanics of these fundamental structures (pointer chaining, manual array shifting, node-based hierarchy) remain visible — this builds the conceptual foundation needed to understand what Collections-framework classes like `ArrayList` and `LinkedList` (Chapter 12) are actually doing internally, rather than treating them as an opaque black box.

Chapter 12: Collections API

12.1 Why the Collections Framework?

The `java.util` Collections Framework provides ready-made, thoroughly-tested, resizable data structures so you rarely need to hand-roll a linked list or hash table (as done manually in Chapter 11) for everyday application code. At the top of the hierarchy sits the `Collection` interface, with three major sub-families: **List**, **Set**, and (as a close relative, though technically *not* a `Collection`, since it deals in key-value pairs rather than single elements) **Map**.

```
Collection (interface)
├── List    (ordered, duplicates allowed)    – ArrayList, LinkedList
├── Set     (no duplicates)                  – HashSet, TreeSet, LinkedHashSet
Map (separate interface, key-value pairs)    – HashMap, TreeMap, LinkedHashMap
```

12.2 List — Ordered, Allows Duplicates

`ArrayList` is the most-used `List` implementation: internally backed by a resizable array (exactly as discussed conceptually in Chapter 11), giving **O(1)** access by index but **O(n)** insertion/ removal in the middle (since subsequent elements must shift). `LinkedList` is the alternative implementation, backed by a doubly-linked node chain — better for frequent insertions/deletions at the ends but $O(n)$ for indexed access.

Ways to iterate a List, each with distinct trade-offs:

1. **Classic indexed for loop** (`for (int i = 0; i < list.size(); i++)`) — gives you the index, useful when you need position-aware logic.
2. **Enhanced for-each loop** (`for (Type item : list)`) — cleanest syntax when you don't need the index, but you **cannot safely remove elements** while iterating this way.
3. **forEach() method with a lambda** (Java 8+) (`list.forEach(item -> ...)`) — functional-style, concise, same restriction on structural modification as for-each.
4. **Explicit Iterator** — **required** when you need to `remove()` elements *while* iterating; using `list.remove(item)` directly inside a for-each loop throws `ConcurrentModificationException`, because the for-each loop's hidden iterator detects the list was structurally modified outside of its own control. `Iterator.remove()` is the *only* safe way to remove during iteration.

12.3 Set — No Duplicates

`HashSet` stores unique elements with **no guaranteed iteration order**, backed internally by a `HashMap` (each element is stored as a key, mapped to a dummy constant value). Uniqueness is determined via each element's `equals()`/`hashCode()` contract (12.6) — this is exactly why custom objects placed in a `HashSet` (or used as `HashMap` keys) **must** correctly override both methods, or duplicate-looking objects will not be recognized as duplicates. `LinkedHashSet` preserves insertion order at a small extra memory cost; `TreeSet` keeps elements in *sorted* order (requiring either `Comparable` elements or an explicit `Comparator`) at $O(\log n)$ operations instead of `HashSet`'s average $O(1)$.

12.4 Map — Key/Value Pairs

`HashMap` stores unique keys mapped to values; lookups, insertions, and removals by key are **O(1) average case** (achieved via **hashing** — the key's `hashCode()` determines which internal "bucket" it's stored in, so the map doesn't need to scan every entry to find a match). Common iteration patterns: iterating `.entrySet()` (retrieves both key and value together per iteration

— the most efficient pattern, since it avoids a separate `get(key)` lookup for each key), `.keySet()` (if you only need keys), or `.forEach((k, v) -> ...)` with a lambda (Java 8+, most concise).

12.5 Sorting Collections — Comparable vs Comparator

	Comparable	Comparator
Defines	The class's single, natural, intrinsic ordering	An external, pluggable ordering, defined outside the class
Method	<code>compareTo(other)</code> implemented inside the class itself	<code>compare(a, b)</code> implemented in a separate class or lambda
How many orderings possible?	Exactly one per class	As many as you want (sort by name, by fees, by duration, ...)
Used via	<code>Collections.sort(list)</code> (uses the natural ordering automatically)	<code>Collections.sort(list, comparator)</code> or <code>list.sort(comparator)</code>
Typical use case	A truly intrinsic ordering exists (e.g., numbers sort by value, dates sort chronologically)	You need several <i>different, situational</i> orderings of the same objects

The `compareTo/compare` contract: return a **negative** number if the first argument is “less than” the second, **zero** if equal, and a **positive** number if “greater than” — the exact magnitude doesn't matter, only the sign.

The demo set below sorts `Employee/Course` objects both ways: implementing `Comparable` for a natural ordering, and writing multiple separate `Comparator` classes (`SortByName`, `SortByFees`, `SortByDuration`) to sort the *same* objects differently depending on the use case.

12.6 `equals()` and `hashCode()` — The Contract That Powers Hash-Based Collections

The default `equals()` (inherited from `Object`) compares **references** (are these two variables pointing to the *exact same* object in memory?), **not** logical content. For two different `Employee` objects that happen to represent the same employee (say, same ID) to be considered “equal” by your application's logic, you must **override both `equals()` and `hashCode()`**.

The `equals/hashCode` contract (a near-guaranteed viva question):

1. If two objects are equal (per `.equals()`), they **must** produce the **same** `hashCode()`.
2. If two objects have the same `hashCode()`, they are **not** required to be `.equals()` (this is called a **hash collision** — different objects can share a hash bucket; the hash-based collection then falls back to `.equals()` to distinguish them within that bucket).
3. `hashCode()` must be **consistent** — calling it multiple times on the same unmodified object must always return the same value within a single program run.

Why does violating this contract silently break `HashMap/HashSet`? If you override `equals()` to say two objects are logically equal, but *don't* also override `hashCode()` to match, they may land in *different* hash buckets — a `HashSet.contains()` check or a `HashMap.get()` lookup can then fail to find an object that, by `.equals()`, clearly “is” already in the collection,

because the lookup never even looks in the bucket where the equal object actually landed. This is why the two methods must always be overridden **together**, never just one.

All demo files below (spanning List, Set, Map, Comparable, Comparator, and a dedicated equals()/hashCode() example, plus real-world style Transaction/Course assignments) are grouped by sub-topic.

12.6.1 List — ArrayList (creation, iteration styles)

File: ATCILTT_Collection_Faculty/ATCILTT_Collection_Faculty/com/accenture/ltt/collection/activity

```
package com.accenture.ltt.collection.activity.list;

import java.util.ArrayList;

public class ArrayListExample {

    public static void main(String[] args) {
        // Create an ArrayList to total marks scored by students.
        ArrayList<Integer> studentMarks = new ArrayList<>();
        studentMarks.add(87);
        studentMarks.add(98);
        studentMarks.add(80);
        studentMarks.add(68);
        studentMarks.add(52);
        //Print the marks
        System.out.println(studentMarks);
    }
}
```

File: ATCILTT_Collection_Faculty/ATCILTT_Collection_Faculty/com/accenture/ltt/collection/assignment

```
package com.accenture.ltt.collection.assignment.list;

public class Transaction {

    private String transactionId;
    private long senderAccountNumber;
    private long recipientAccountNumber;
    private long transactionAmount;

    public Transaction(String transactionId, long senderAccountNumber, long recipientAccountNumber,
        long transactionAmount) {
        this.transactionId = transactionId;
        this.senderAccountNumber = senderAccountNumber;
        this.recipientAccountNumber = recipientAccountNumber;
        this.transactionAmount = transactionAmount;
    }

    public String getTransactionId() {
        return transactionId;
    }

    public long getSenderAccountNumber() {
        return senderAccountNumber;
    }
}
```

```

    public long getRecipientAccountNumber() {
        return recipientAccountNumber;
    }

    public long getTransactionAmount() {
        return transactionAmount;
    }

    @Override
    public String toString() {
        return "\n senderAccountNumber=" + senderAccountNumber + "\n recipientAccountNumber=" +
            + "\n transactionAmount=" + transactionAmount + "]";
    }
}

```

File: ATCILTT_Collection_Faculty/ATCILTT_Collection_Faculty/com/accenture/ltt/collection/assignment

```

package com.accenture.ltt.collection.assignment.list;

import java.util.List;
import java.util.stream.Collectors;

public class TransactionService {

    //Method which helps print transactions corresponding to a particular account

    public void printAllTransactions(List<Transaction> transactions, long accountNumber) {
        //Create a stream on the collection which contains all transaction objects.
        //Use a Filter funtional interface to filter transactions corresponding to the account
        List<Transaction> list = transactions.stream().filter(x -> x.getSenderAccountNumber() =
            .collect(Collectors.toList());
        System.out.println(list);
    }
}

```

File: ATCILTT_Collection_Faculty/ATCILTT_Collection_Faculty/com/accenture/ltt/collection/assignment

```

package com.accenture.ltt.collection.assignment.list;

import java.util.ArrayList;
import java.util.List;

public class TransactionTester {

    public static void main(String[] args) {
        //ArrayList to store transaction objects
        List<Transaction> transactions = new ArrayList<>();

        //Create transaction objects and add them to the ArrayList
        Transaction t1 = new Transaction("TD5678", 110095, 345678, 20000);
        transactions.add(t1);
    }
}

```



```

Transaction t2 = new Transaction("TD6764", 149352, 577321, 100000);
transactions.add(t2);

Transaction t3 = new Transaction("TD6785", 210995, 673567, 50000);
transactions.add(t3);

//Call print method of service class which prints transactions corresponding to the acc
TransactionService service = new TransactionService();
service.printAllTransactions(transactions, 110095);

    }
}

```

File: ATCILTT_Collection_Faculty/ATCILTT_Collection_Faculty/com/accenture/ltt/collection/demo/li

```

package com.accenture.ltt.collection.demo.list;

import java.util.ArrayList;

public class ArrayListExample {

    public static void main(String[] args) {
        // Create an ArrayList to store names of employees.
        ArrayList<String> empNames = new ArrayList<>();
        empNames.add("John");
        empNames.add("Riya");
        empNames.add("Ram");
        empNames.add("Ali");
        empNames.add("Jess");
        System.out.println(empNames);
    }
}

```

File: ATCILTT_Collection_Faculty/ATCILTT_Collection_Faculty/com/accenture/ltt/collection/demo/li

```

package com.accenture.ltt.collection.demo.list;

import java.util.ArrayList;

public class ArrayListForEachLoopExample {
    public static void main(String[] args) {
        // Create an ArrayList to store names of employees.
        ArrayList<String> empNames = new ArrayList<>();
        empNames.add("John");
        empNames.add("Riya");
        empNames.add("Ram");
        empNames.add("Ali");
        empNames.add("Jess");
        // Iterate ArrayList using for-each loop
        for (String name : empNames) {
            System.out.println(name);
        }
    }
}

```

File: ATCILTT_Collection_Faculty/ATCILTT_Collection_Faculty/com/accenture/ltt/collection/demo/li

```
package com.accenture.ltt.collection.demo.list;

import java.util.ArrayList;

public class ArrayListForEachMethodExample {
    public static void main(String[] args) {
        // Create an ArrayList to store names of employees.
        ArrayList<String> empNames = new ArrayList<>();
        empNames.add("John");
        empNames.add("Riya");
        empNames.add("Ram");
        empNames.add("Ali");
        empNames.add("Jess");
        // Iterate ArrayList using for-each method
        empNames.forEach((x) -> System.out.println(x));
    }
}
```

File: ATCILTT_Collection_Faculty/ATCILTT_Collection_Faculty/com/accenture/ltt/collection/demo/li

```
package com.accenture.ltt.collection.demo.list;

import java.util.ArrayList;
import java.util.Iterator;

public class ArrayListIteratorExample {
    public static void main(String[] args) {
        ArrayList<String> empNames = new ArrayList<>();
        empNames.add("John");
        empNames.add("Riya");
        empNames.add("Ram");
        empNames.add("Ali");
        empNames.add("Jess");
        // Iterate ArrayList using Iterator
        Iterator<String> itr = empNames.iterator();
        while (itr.hasNext()) {
            String name = itr.next();
            System.out.println(name);
        }
    }
}
```

12.6.2 Set — HashSet

File: ATCILTT_Collection_Faculty/ATCILTT_Collection_Faculty/com/accenture/ltt/collection/activit

```
package com.accenture.ltt.collection.activity.set;

import java.util.HashSet;

public class HashSetExample {
```

```

public static void main(String[] args) {
    // Create a set to store account numbers
    HashSet<Integer> accountNumbers = new HashSet<>();
    accountNumbers.add(230041988);
    accountNumbers.add(230019090);
    accountNumbers.add(230091191);
    accountNumbers.add(230187110);
    accountNumbers.add(230032498);
    //Print account numbers
    System.out.println(accountNumbers);
}
}

```

File: ATCILTT_Collection_Faculty/ATCILTT_Collection_Faculty/com/accenture/ltt/collection/demo/set

```

package com.accenture.ltt.collection.demo.set;

import java.util.HashSet;

public class HashSetExample {

    public static void main(String[] args) {
        // Create a set to store user ids
        HashSet<String> userIds = new HashSet<>();
        userIds.add("John1988");
        userIds.add("Riya1990");
        userIds.add("Ram1991");
        userIds.add("Ali1990");
        userIds.add("Jess1987");
        System.out.println(userIds);
    }
}

```

12.6.3 Map — HashMap

File: ATCILTT_Collection_Faculty/ATCILTT_Collection_Faculty/com/accenture/ltt/collection/assignment

```

package com.accenture.ltt.collection.assignment.map;

public class Transaction {

    private String transactionId;
    private long senderAccountNumber;
    private long recipientAccountNumber;
    private long transactionAmount;

    public Transaction(String transactionId, long senderAccountNumber, long recipientAccountNumber,
        long transactionAmount) {
        this.transactionId = transactionId;
        this.senderAccountNumber = senderAccountNumber;
        this.recipientAccountNumber = recipientAccountNumber;
        this.transactionAmount = transactionAmount;
    }
}

```

```

    public String getTransactionId() {
        return transactionId;
    }

    public long getSenderAccountNumber() {
        return senderAccountNumber;
    }

    public long getRecipientAccountNumber() {
        return recipientAccountNumber;
    }

    public long getTransactionAmount() {
        return transactionAmount;
    }

    @Override
    public String toString() {
        return "\n senderAccountNumber=" + senderAccountNumber + "\n recipientAccountNumber=" +
            + "\n transactionAmount=" + transactionAmount + "];"
    }
}

```

File: ATCILTT_Collection_Faculty/ATCILTT_Collection_Faculty/com/accenture/ltt/collection/assignment

```

package com.accenture.ltt.collection.assignment.map;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

public class TransactionMapTester {

    public static void main(String[] args) {
        // Create a list to hold transaction
        List<Transaction> transactions = new ArrayList<>();
        // Add transaction objects to the list
        Transaction t1 = new Transaction("TD5678", 110095, 345678, 20000);
        transactions.add(t1);

        Transaction t2 = new Transaction("TD6764", 149352, 577321, 100000);
        transactions.add(t2);

        Transaction t3 = new Transaction("TD6785", 210995, 673567, 50000);
        transactions.add(t3);

        // Create a map to store transactionId and transaction object
        Map<String, Transaction> allTransactions = new HashMap<>();
        transactions.forEach(x -> {
            String transactionId = x.getTransactionId();
            allTransactions.put(transactionId, x);
        });
    }
}

```

```

    });
    // Call print method
    TransactionService service = new TransactionService();
    service.printAllTransactions(allTransactions, 210995);
}
}

```

File: ATCILTT_Collection_Faculty/ATCILTT_Collection_Faculty/com/accenture/ltt/collection/assignment

```

package com.accenture.ltt.collection.assignment.map;

import java.util.Map;

public class TransactionService {
    // Method to print transaction ids for an account number
    public void printAllTransactions(Map<String, Transaction> transactions, long accountNumber) {
        //Check whether account number of each element in the collection is matching the provided
        transactions.forEach((k, v) -> {
            if (v.getSenderAccountNumber() == accountNumber) {
                //Print transaction id if a match of account number is found
                System.out.println(k);
            }
        });
    }
}

```

File: ATCILTT_Collection_Faculty/ATCILTT_Collection_Faculty/com/accenture/ltt/collection/demo/map

```

package com.accenture.ltt.collection.demo.map;

import java.util.HashMap;
import java.util.Map;

public class HashMapExample {
    public static void main(String[] args) {
        // Create a map to store telephone numbers of users
        Map<Integer, String> directory = new HashMap<>();
        directory.put(404232, "Venkat Rajaram");
        directory.put(502113, "Tia Mathew");
        directory.put(562190, "Ali Mohammed");
        directory.put(764342, "Fatimah Ibrahim");
        directory.put(232428, "Dennis Jacob");
        // Print Map elements
        System.out.println(directory);
    }
}

```

File: ATCILTT_Collection_Faculty/ATCILTT_Collection_Faculty/com/accenture/ltt/collection/demo/map

```

package com.accenture.ltt.collection.demo.map;

import java.util.HashMap;
import java.util.Map;

```

```

public class HashMapForEachMethodExample {
    public static void main(String[] args) {
        // Create a map to store telephone numbers of users
        Map<Integer, String> directory = new HashMap<>();
        directory.put(404232, "Venkat Rajaram");
        directory.put(502113, "Tia Mathew");
        directory.put(562190, "Ali Mohammed");
        directory.put(764342, "Fatemah Ibrahim");
        directory.put(232428, "Dennis Jacob");
        // Iterate the Map using forEach method
        directory.forEach((K, V) -> System.out.println("Name is " + V + " Contact is " + K));
    }
}

```

File: ATCILTT_Collection_Faculty/ATCILTT_Collection_Faculty/com/accenture/ltt/collection/demo/ma

```

package com.accenture.ltt.collection.demo.map;

import java.util.HashMap;
import java.util.Iterator;
import java.util.Map;
import java.util.Set;

public class HashMapIteratorExample {
    public static void main(String[] args) {
        // Create a map to store telephone numbers of users
        Map<Integer, String> directory = new HashMap<>();
        directory.put(404232, "Venkat Rajaram");
        directory.put(502113, "Tia Mathew");
        directory.put(562190, "Ali Mohammed");
        directory.put(764342, "Fatemah Ibrahim");
        directory.put(232428, "Dennis Jacob");
        // Get all keys into a Set
        Set<Integer> keys = directory.keySet();
        // Create an Iterator on keySet
        Iterator<Integer> itr = keys.iterator();
        while (itr.hasNext()) {
            Integer key = itr.next();
            String value = directory.get(key);
            System.out.println("Name is " + value + " Contact is " + key);
        }
    }
}

```

12.6.4 User-Defined Objects in Collections

File: ATCILTT_Collection_Faculty/ATCILTT_Collection_Faculty/com/accenture/ltt/collection/demo/us

```

package com.accenture.ltt.collection.demo.userdefined;

public class Employee {
    private int employeeId;
    private String employeeName;
    private int yearsOfExperience;
}

```

```

private double salary;

public Employee(int employeeId, String employeeName, int yearsOfExperience, double salary) {
    this.employeeId = employeeId;
    this.employeeName = employeeName;
    this.yearsOfExperience = yearsOfExperience;
    this.salary = salary;
}

public int getEmployeeId() {
    return employeeId;
}

public String getEmployeeName() {
    return employeeName;
}

public int getYearsOfExperience() {
    return yearsOfExperience;
}

public double getSalary() {
    return salary;
}
}

```

File: ATCILTT_Collection_Faculty/ATCILTT_Collection_Faculty/com/accenture/ltt/collection/demo/us

```

package com.accenture.ltt.collection.demo.userdefined;

import java.util.List;

public class EmployeeService {
    public void printEmployeeDetails(List<Employee> employeeList) {
        employeeList.forEach(employee -> {
            System.out.println("Emp id is " + employee.getEmployeeId() + " Emp name is " + empl
                + " Emp experience is " + employee.getYearsOfExperience() + " Emp salary is
                + employee.getSalary());
        });
    }
}

```

File: ATCILTT_Collection_Faculty/ATCILTT_Collection_Faculty/com/accenture/ltt/collection/demo/us

```

package com.accenture.ltt.collection.demo.userdefined;

import java.util.ArrayList;
import java.util.List;

public class EmployeeTester {
    public static void main(String[] args) {
        Employee emp1 = new Employee(1001, "Employee1", 5, 60000);
        Employee emp2 = new Employee(1011, "Employee2", 8, 100000);
        Employee emp3 = new Employee(1006, "Employee3", 7, 80000);
        Employee emp4 = new Employee(1201, "Employee4", 3, 40000);
    }
}

```

```

        List<Employee> empList = new ArrayList<>();
        empList.add(emp1);
        empList.add(emp2);
        empList.add(emp3);
        empList.add(emp4);
        EmployeeService service = new EmployeeService();
        service.printEmployeeDetails(empList);
    }
}

```

12.6.5 Comparable

File: ATCILTT_Collection_Faculty/ATCILTT_Collection_Faculty/com/accenture/ltt/collection/assignment

```
package com.accenture.ltt.collection.assignment.comparable;
```

```

public class Course implements Comparable<Course> {

    private int courseId;
    private String courseName;
    private int courseDurationInHours;
    private int courseFee;

    public Course(int courseId, String courseName, int courseDurationInHours, int courseFee) {
        this.courseId = courseId;
        this.courseName = courseName;
        this.courseDurationInHours = courseDurationInHours;
        this.courseFee = courseFee;
    }

    public int getCourseId() {
        return courseId;
    }

    public String getCourseName() {
        return courseName;
    }

    public int getCourseDurationInHours() {
        return courseDurationInHours;
    }

    public int getCourseFee() {
        return courseFee;
    }

    @Override
    public int compareTo(Course o) {
        if (this.getCourseFee() > o.getCourseFee())
            return 1;
        else if (this.getCourseFee() == o.getCourseFee())
            return 0;
        else
            return -1;
    }
}

```



```

    }
}

```

File: ATCILTT_Collection_Faculty/ATCILTT_Collection_Faculty/com/accenture/ltt/collection/assignment

```

package com.accenture.ltt.collection.assignment.comparable;

import java.util.ArrayList;
import java.util.List;

public class CourseComparableTester {

    public static void main(String[] args) {
        //ArrayList to store transaction objects
        List<Course> courses = new ArrayList<>();

        //Create courses and add them to the ArrayList
        Course t1 = new Course(1056, "C#.Net", 72, 20000);
        courses.add(t1);

        Course t2 = new Course(1045, "Java", 48, 15000);
        courses.add(t2);

        Course t3 = new Course(4532, "Python", 40, 20500);
        courses.add(t3);

        //Call print method of service class which prints
        CourseService service = new CourseService();
        service.printAllCoursesSorted(courses);

    }
}

```

File: ATCILTT_Collection_Faculty/ATCILTT_Collection_Faculty/com/accenture/ltt/collection/assignment

```

package com.accenture.ltt.collection.assignment.comparable;

import java.util.Collections;
import java.util.List;

public class CourseService {

    //Method which helps to print courses sorted based on course fee

    public void printAllCoursesSorted(List<Course> courses) {
        //Sort the collection by calling Collections.sort method
        Collections.sort(courses);
        //Print course name and fee using foreach method
        courses.stream().forEach(x->System.out.println("Course is "+x.getCourseName()+" "
            + ": Course Fee is Rs." +x.getCourseFee()));
    }
}

```

```
}
```

File: ATCILTT_Collection_Faculty/ATCILTT_Collection_Faculty/com/accenture/ltt/collection/demo/co

```
package com.accenture.ltt.collection.demo.comparable;
```

```
public class Employee implements Comparable<Employee> {  
    private int employeeId;  
    private String employeeName;  
    private int yearsOfExperience;  
    private double salary;
```

```
    public int compareTo(Employee emp) {  
        if (this.employeeId > emp.employeeId)  
            return 1;  
        else if (this.employeeId < emp.employeeId)  
            return -1;  
        else  
            return 0;  
    }
```

```
    public Employee(int employeeId, String employeeName, int yearsOfExperience, double salary) {  
        this.employeeId = employeeId;  
        this.employeeName = employeeName;  
        this.yearsOfExperience = yearsOfExperience;  
        this.salary = salary;  
    }
```

```
    public int getEmployeeId() {  
        return employeeId;  
    }
```

```
    public String getEmployeeName() {  
        return employeeName;  
    }
```

```
    public int getYearsOfExperience() {  
        return yearsOfExperience;  
    }
```

```
    public double getSalary() {  
        return salary;  
    }
```

```
}
```

File: ATCILTT_Collection_Faculty/ATCILTT_Collection_Faculty/com/accenture/ltt/collection/demo/co

```
package com.accenture.ltt.collection.demo.comparable;
```

```
import java.util.ArrayList;  
import java.util.Collections;  
import java.util.List;
```

```

public class EmployeeComparableTester {
    public static void main(String[] args) {
        Employee emp1 = new Employee(1001, "Employee1", 5, 60000);
        Employee emp2 = new Employee(1011, "Employee2", 8, 100000);
        Employee emp3 = new Employee(1006, "Employee3", 7, 80000);
        Employee emp4 = new Employee(1201, "Employee4", 3, 40000);
        List<Employee> empList = new ArrayList<>();
        empList.add(emp1);
        empList.add(emp2);
        empList.add(emp3);
        empList.add(emp4);
        Collections.sort(empList);
        EmployeeService service = new EmployeeService();
        service.printEmployeeDetails(empList);
    }
}

```

File: ATCILTT_Collection_Faculty/ATCILTT_Collection_Faculty/com/accenture/ltt/collection/demo/comparable

```

package com.accenture.ltt.collection.demo.comparable;
import java.util.List;

public class EmployeeService {
    public void printEmployeeDetails(List<Employee> employeeList) {
        employeeList.forEach(employee -> {
            System.out.println("Emp id is " + employee.getEmployeeId() + " Emp name is " + employee.getEmployeeName() + " Emp experience is " + employee.getYearsOfExperience() + " Emp salary is " + employee.getSalary());
        });
    }
}

```

12.6.6 Comparator

File: ATCILTT_Collection_Faculty/ATCILTT_Collection_Faculty/com/accenture/ltt/collection/assignment/comparator

```

package com.accenture.ltt.collection.assignment.comparator;

public class Course {

    private int courseId;
    private String courseName;
    private int courseDurationInHours;
    private int courseFee;

    public Course(int courseId, String courseName, int courseDurationInHours, int courseFee) {
        this.courseId = courseId;
        this.courseName = courseName;
        this.courseDurationInHours = courseDurationInHours;
        this.courseFee = courseFee;
    }

    public int getCourseId() {
        return courseId;
    }
}

```

```

    }

    public String getCourseName() {
        return courseName;
    }

    public int getCourseDurationInHours() {
        return courseDurationInHours;
    }

    public int getCourseFee() {
        return courseFee;
    }

}

```

File: ATCILTT_Collection_Faculty/ATCILTT_Collection_Faculty/com/accenture/ltt/collection/assignment

```

package com.accenture.ltt.collection.assignment.comparator;

import java.util.ArrayList;
import java.util.List;

public class CourseComparatorTester {

    public static void main(String[] args) {
        //ArrayList to store transaction objects
        List<Course> courses = new ArrayList<>();

        //Create courses and add them to the ArrayList
        Course t1 = new Course(1056, "C#.Net", 72, 20000);
        courses.add(t1);

        Course t2 = new Course(1045, "Java", 48, 15000);
        courses.add(t2);

        Course t3 = new Course(4532, "Python", 40, 20500);
        courses.add(t3);

        //Call print method of service class which prints transactions corresponding to the acc
        CourseService service = new CourseService();
        service.printAllCoursesDurationSorted(courses);

    }
}

```

File: ATCILTT_Collection_Faculty/ATCILTT_Collection_Faculty/com/accenture/ltt/collection/assignment

```

package com.accenture.ltt.collection.assignment.comparator;

import java.util.Collections;
import java.util.List;

```

```

public class CourseService {

    //Method which helps to print courses sorted based on course fee

    public void printAllCoursesFeesSorted(List<Course> courses) {
        //Sort the collection by calling Collections.sort method
        Collections.sort(courses,new SortByFeesComparator());
        //Print course name and fee using foreach method
        courses.stream().forEach(x->System.out.println("Course is "+x.getCourseName()+" "
            + ": Course Fee is Rs." +x.getCourseFee()));
    }
    public void printAllCoursesNamesSorted(List<Course> courses) {
        //Sort the collection by calling Collections.sort method
        Collections.sort(courses,new SortByNameComparator());
        //Print course name and fee using foreach method
        courses.stream().forEach(x->System.out.println("Course is "+x.getCourseName()+" "
            + ": Course Fee is Rs." +x.getCourseFee()));
    }public void printAllCoursesDurationSorted(List<Course> courses) {
        //Sort the collection by calling Collections.sort method
        Collections.sort(courses,new SortByDurationComparator());
        //Print course name and fee using foreach method
        courses.stream().forEach(x->System.out.println("Course is "+x.getCourseName()+" "
            + ": Course Fee is Rs." +x.getCourseFee()));
    }

}

```

File: ATCILTT_Collection_Faculty/ATCILTT_Collection_Faculty/com/accenture/ltt/collection/assignment

```

package com.accenture.ltt.collection.assignment.comparator;

import java.util.Comparator;

public class SortByDurationComparator implements Comparator<Course> {

    @Override
    public int compare(Course o1, Course o2) {
        return o1.getCourseDurationInHours()-o2.getCourseDurationInHours();
    }

}

```

File: ATCILTT_Collection_Faculty/ATCILTT_Collection_Faculty/com/accenture/ltt/collection/assignment

```

package com.accenture.ltt.collection.assignment.comparator;

import java.util.Comparator;

public class SortByFeesComparator implements Comparator<Course> {

    @Override
    public int compare(Course o1, Course o2) {

```

```

        return o1.getCourseFee() - o2.getCourseFee();
    }
}

```

File: ATCILTT_Collection_Faculty/ATCILTT_Collection_Faculty/com/accenture/ltt/collection/assignment/comparator

```

package com.accenture.ltt.collection.assignment.comparator;

import java.util.Comparator;

public class SortByNameComparator implements Comparator<Course> {

    @Override
    public int compare(Course o1, Course o2) {
        return o1.getCourseName().compareTo(o2.getCourseName());
    }
}

```

File: ATCILTT_Collection_Faculty/ATCILTT_Collection_Faculty/com/accenture/ltt/collection/demo/comparator

```

package com.accenture.ltt.collection.demo.comparator;

public class Employee {
    private int employeeId;
    private String employeeName;
    private int yearsOfExperience;
    private double salary;

    public Employee(int employeeId, String employeeName, int yearsOfExperience, double salary) {
        this.employeeId = employeeId;
        this.employeeName = employeeName;
        this.yearsOfExperience = yearsOfExperience;
        this.salary = salary;
    }

    public int getEmployeeId() {
        return employeeId;
    }

    public String getEmployeeName() {
        return employeeName;
    }

    public int getYearsOfExperience() {
        return yearsOfExperience;
    }

    public double getSalary() {
        return salary;
    }
}

```

```

    }
}

```

File: ATCILTT_Collection_Faculty/ATCILTT_Collection_Faculty/com/accenture/ltt/collection/demo/co

```

package com.accenture.ltt.collection.demo.comparator;

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

public class EmployeeComparatorTester{
    public static void main(String[] args) {
        Employee emp1 = new Employee(1001, "Employee1", 5, 60000);
        Employee emp2 = new Employee(1011, "Employee2", 8, 100000);
        Employee emp3 = new Employee(1006, "Employee3", 7, 80000);
        Employee emp4 = new Employee(1201, "Employee4", 3, 40000);
        List<Employee> empList = new ArrayList<>();
        empList.add(emp1);
        empList.add(emp2);
        empList.add(emp3);
        empList.add(emp4);
        Collections.sort(empList, new EmployeeIdComparator());
        EmployeeService service = new EmployeeService();
        service.printEmployeeDetails(empList);
    }
}

```

File: ATCILTT_Collection_Faculty/ATCILTT_Collection_Faculty/com/accenture/ltt/collection/demo/co

```

package com.accenture.ltt.collection.demo.comparator;

import java.util.Comparator;

public class EmployeeIdComparator implements Comparator<Employee> {

    public int compare(Employee o1, Employee o2) {
        if (o1.getEmployeeId() > o2.getEmployeeId())
            return 1;
        else if (o1.getEmployeeId() < o2.getEmployeeId())
            return -1;
        else
            return 0;
    }
}

```

File: ATCILTT_Collection_Faculty/ATCILTT_Collection_Faculty/com/accenture/ltt/collection/demo/co

```

package com.accenture.ltt.collection.demo.comparator;
import java.util.List;

public class EmployeeService {
    public void printEmployeeDetails(List<Employee> employeeList) {
        employeeList.forEach(employee -> {
            System.out.println("Emp id is " + employee.getEmployeeId() + " Emp name is " + empl
                + " Emp experience is " + employee.getYearsOfExperience() + " Emp salary is

```

```

        + employee.getSalary());
    });
}
}

```

12.6.7 equals() / hashCode()

File: ATCILT_Collection_Faculty/com/accenture/ltt/collection/demo/equalshashcode/EqualsHashCode

```

package com.accenture.ltt.collection.demo.equalshashcode;
import java.util.Objects;

class Employee {
    private int id;
    private String name;

    public Employee(int id, String name) {
        this.id = id;
        this.name = name;
    }

    // Override equals()
    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true; // Reflexive
        if (obj == null || getClass() != obj.getClass()) return false; // Null & type check
        Employee other = (Employee) obj;
        return id == other.id && Objects.equals(name, other.name);
    }

    // Override hashCode()
    @Override
    public int hashCode() {
        return Objects.hash(id, name);
    }
}

public class EqualsHashCodeDemo {
    public static void main(String[] args) {
        Employee x = new Employee(101, "Priya");
        Employee y = new Employee(101, "Priya");
        Employee z = new Employee(101, "Priya");

        // 1. Reflexive
        System.out.println("Reflexive: x.equals(x) = " + x.equals(x)); // true

        // 2. Symmetric
        System.out.println("Symmetric: x.equals(y) = " + x.equals(y)); // true
        System.out.println("Symmetric: y.equals(x) = " + y.equals(x)); // true

        // 3. Transitive
        System.out.println("Transitive: x.equals(y) = " + x.equals(y)); // true
        System.out.println("Transitive: y.equals(z) = " + y.equals(z)); // true
        System.out.println("Transitive: x.equals(z) = " + x.equals(z)); // true
    }
}

```



```

// 4. Consistent
System.out.println("Consistent: x.equals(y) repeated = " + x.equals(y)); // true
System.out.println("Consistent: x.equals(y) repeated again = " + x.equals(y)); // true

// 5. Null Comparison
System.out.println("Null Comparison: x.equals(null) = " + x.equals(null)); // false

// hashCode consistency
System.out.println("x.hashCode() = " + x.hashCode());
System.out.println("y.hashCode() = " + y.hashCode());
System.out.println("z.hashCode() = " + z.hashCode());
    }
}

```

12.6.8 Assignment Exercises — Transactions & Courses

File: ATCILTT_Collection_Faculty/ATCILTT_Collection_Faculty/com/accenture/ltt/collection/assignment

```

package com.accenture.ltt.collection.assignment.comparable;

public class Course implements Comparable<Course> {

    private int courseId;
    private String courseName;
    private int courseDurationInHours;
    private int courseFee;

    public Course(int courseId, String courseName, int courseDurationInHours, int courseFee) {
        this.courseId = courseId;
        this.courseName = courseName;
        this.courseDurationInHours = courseDurationInHours;
        this.courseFee = courseFee;
    }

    public int getCourseId() {
        return courseId;
    }

    public String getCourseName() {
        return courseName;
    }

    public int getCourseDurationInHours() {
        return courseDurationInHours;
    }

    public int getCourseFee() {
        return courseFee;
    }

    @Override
    public int compareTo(Course o) {
        if (this.getCourseFee() > o.getCourseFee())

```

```

        return 1;
    else if (this.getCourseFee() == o.getCourseFee())
        return 0;
    else
        return -1;
    }
}

```

File: ATCILTT_Collection_Faculty/ATCILTT_Collection_Faculty/com/accenture/ltt/collection/assignment

```

package com.accenture.ltt.collection.assignment.comparable;

import java.util.ArrayList;
import java.util.List;

public class CourseComparableTester {

    public static void main(String[] args) {
        //ArrayList to store transaction objects
        List<Course> courses = new ArrayList<>();

        //Create courses and add them to the ArrayList
        Course t1 = new Course(1056, "C#.Net", 72, 20000);
        courses.add(t1);

        Course t2 = new Course(1045, "Java", 48, 15000);
        courses.add(t2);

        Course t3 = new Course(4532, "Python", 40, 20500);
        courses.add(t3);

        //Call print method of service class which prints
        CourseService service = new CourseService();
        service.printAllCoursesSorted(courses);

    }
}

```

File: ATCILTT_Collection_Faculty/ATCILTT_Collection_Faculty/com/accenture/ltt/collection/assignment

```

package com.accenture.ltt.collection.assignment.comparable;

import java.util.Collections;
import java.util.List;

public class CourseService {

    //Method which helps to print courses sorted based on course fee

    public void printAllCoursesSorted(List<Course> courses) {
        //Sort the collection by calling Collections.sort method
    }
}

```

```

        Collections.sort(courses);
        //Print course name and fee using foreach method
        courses.stream().forEach(x->System.out.println("Course is "+x.getCourseName()+" "
            + ": Course Fee is Rs." +x.getCourseFee()));
    }
}

```

File: ATCILTT_Collection_Faculty/ATCILTT_Collection_Faculty/com/accenture/ltt/collection/assignment

```

package com.accenture.ltt.collection.assignment.comparator;

public class Course {

    private int courseId;
    private String courseName;
    private int courseDurationInHours;
    private int courseFee;

    public Course(int courseId, String courseName, int courseDurationInHours, int courseFee) {
        this.courseId = courseId;
        this.courseName = courseName;
        this.courseDurationInHours = courseDurationInHours;
        this.courseFee = courseFee;
    }

    public int getCourseId() {
        return courseId;
    }

    public String getCourseName() {
        return courseName;
    }

    public int getCourseDurationInHours() {
        return courseDurationInHours;
    }

    public int getCourseFee() {
        return courseFee;
    }

}

```

File: ATCILTT_Collection_Faculty/ATCILTT_Collection_Faculty/com/accenture/ltt/collection/assignment

```

package com.accenture.ltt.collection.assignment.comparator;

import java.util.ArrayList;
import java.util.List;

public class CourseComparatorTester {

```

```

public static void main(String[] args) {
    //ArrayList to store transaction objects
    List<Course> courses = new ArrayList<>();

    //Create courses and add them to the ArrayList
    Course t1 = new Course(1056, "C#.Net", 72, 20000);
    courses.add(t1);

    Course t2 = new Course(1045, "Java", 48, 15000);
    courses.add(t2);

    Course t3 = new Course(4532, "Python", 40, 20500);
    courses.add(t3);

    //Call print method of service class which prints transactions corresponding to the acc
    CourseService service = new CourseService();
    service.printAllCoursesDurationSorted(courses);

}
}

```

File: ATCILTT_Collection_Faculty/ATCILTT_Collection_Faculty/com/accenture/ltt/collection/assignment

```

package com.accenture.ltt.collection.assignment.comparator;

import java.util.Collections;
import java.util.List;

public class CourseService {

    //Method which helps to print courses sorted based on course fee

    public void printAllCoursesFeesSorted(List<Course> courses) {
        //Sort the collection by calling Collections.sort method
        Collections.sort(courses, new SortByFeesComparator());
        //Print course name and fee using foreach method
        courses.stream().forEach(x->System.out.println("Course is "+x.getCourseName()+" "
            + ": Course Fee is Rs." +x.getCourseFee()));
    }

    public void printAllCoursesNamesSorted(List<Course> courses) {
        //Sort the collection by calling Collections.sort method
        Collections.sort(courses, new SortByNameComparator());
        //Print course name and fee using foreach method
        courses.stream().forEach(x->System.out.println("Course is "+x.getCourseName()+" "
            + ": Course Fee is Rs." +x.getCourseFee()));
    }

    public void printAllCoursesDurationSorted(List<Course> courses) {
        //Sort the collection by calling Collections.sort method
        Collections.sort(courses, new SortByDurationComparator());
        //Print course name and fee using foreach method
        courses.stream().forEach(x->System.out.println("Course is "+x.getCourseName()+" "
            + ": Course Fee is Rs." +x.getCourseFee()));
    }

}

```

```
}
```

File: ATCILTT_Collection_Faculty/ATCILTT_Collection_Faculty/com/accenture/ltt/collection/assignment

```
package com.accenture.ltt.collection.assignment.comparator;

import java.util.Comparator;

public class SortByDurationComparator implements Comparator<Course> {

    @Override
    public int compare(Course o1, Course o2) {
        return o1.getCourseDurationInHours()-o2.getCourseDurationInHours();
    }

}
```

File: ATCILTT_Collection_Faculty/ATCILTT_Collection_Faculty/com/accenture/ltt/collection/assignment

```
package com.accenture.ltt.collection.assignment.comparator;

import java.util.Comparator;

public class SortByFeesComparator implements Comparator<Course> {

    @Override
    public int compare(Course o1, Course o2) {

        return o1.getCourseFee() - o2.getCourseFee();
    }

}
```

File: ATCILTT_Collection_Faculty/ATCILTT_Collection_Faculty/com/accenture/ltt/collection/assignment

```
package com.accenture.ltt.collection.assignment.comparator;

import java.util.Comparator;

public class SortByNameComparator implements Comparator<Course> {

    @Override
    public int compare(Course o1, Course o2) {
        return o1.getCourseName().compareTo(o2.getCourseName());
    }

}
```

File: ATCILTT_Collection_Faculty/ATCILTT_Collection_Faculty/com/accenture/ltt/collection/assignment

```

package com.accenture.ltt.collection.assignment.list;

public class Transaction {

    private String transactionId;
    private long senderAccountNumber;
    private long recipientAccountNumber;
    private long transactionAmount;

    public Transaction(String transactionId, long senderAccountNumber, long recipientAccountNumber, long transactionAmount) {
        this.transactionId = transactionId;
        this.senderAccountNumber = senderAccountNumber;
        this.recipientAccountNumber = recipientAccountNumber;
        this.transactionAmount = transactionAmount;
    }

    public String getTransactionId() {
        return transactionId;
    }

    public long getSenderAccountNumber() {
        return senderAccountNumber;
    }

    public long getRecipientAccountNumber() {
        return recipientAccountNumber;
    }

    public long getTransactionAmount() {
        return transactionAmount;
    }

    @Override
    public String toString() {
        return "\n senderAccountNumber=" + senderAccountNumber + "\n recipientAccountNumber=" + recipientAccountNumber + "\n transactionAmount=" + transactionAmount + "]";
    }

}

```

File: ATCILTT_Collection_Faculty/ATCILTT_Collection_Faculty/com/accenture/ltt/collection/assignment.list

```

package com.accenture.ltt.collection.assignment.list;

import java.util.List;
import java.util.stream.Collectors;

public class TransactionService {

    //Method which helps print transactions corresponding to a particular account

    public void printAllTransactions(List<Transaction> transactions, long accountNumber) {

```

```

        //Create a stream on the collection which contains all transaction objects.
        //Use a Filter funtional interface to filter transactions corresponding to the account
        List<Transaction> list = transactions.stream().filter(x -> x.getSenderAccountNumber() == 110095)
        .collect(Collectors.toList());
        System.out.println(list);
    }
}

```

File: ATCILTT_Collection_Faculty/ATCILTT_Collection_Faculty/com/accenture/ltt/collection/assignment

```

package com.accenture.ltt.collection.assignment.list;

import java.util.ArrayList;
import java.util.List;

public class TransactionTester {

    public static void main(String[] args) {
        //ArrayList to store transaction objects
        List<Transaction> transactions = new ArrayList<>();

        //Create transaction objects and add them to the ArrayList
        Transaction t1 = new Transaction("TD5678", 110095, 345678, 20000);
        transactions.add(t1);

        Transaction t2 = new Transaction("TD6764", 149352, 577321, 100000);
        transactions.add(t2);

        Transaction t3 = new Transaction("TD6785", 210995, 673567, 50000);
        transactions.add(t3);

        //Call print method of service class which prints transactions corresponding to the account number
        TransactionService service = new TransactionService();
        service.printAllTransactions(transactions, 110095);
    }
}

```

File: ATCILTT_Collection_Faculty/ATCILTT_Collection_Faculty/com/accenture/ltt/collection/assignment

```

package com.accenture.ltt.collection.assignment.map;

public class Transaction {

    private String transactionId;
    private long senderAccountNumber;
    private long recipientAccountNumber;
    private long transactionAmount;

    public Transaction(String transactionId, long senderAccountNumber, long recipientAccountNumber, long transactionAmount) {
        this.transactionId = transactionId;
        this.senderAccountNumber = senderAccountNumber;
        this.recipientAccountNumber = recipientAccountNumber;
    }
}

```

```

        this.transactionAmount = transactionAmount;
    }

    public String getTransactionId() {
        return transactionId;
    }

    public long getSenderAccountNumber() {
        return senderAccountNumber;
    }

    public long getRecipientAccountNumber() {
        return recipientAccountNumber;
    }

    public long getTransactionAmount() {
        return transactionAmount;
    }

    @Override
    public String toString() {
        return "\n senderAccountNumber=" + senderAccountNumber + "\n recipientAccountNumber=" +
            + "\n transactionAmount=" + transactionAmount + "];"
    }
}

```

File: ATCILTT_Collection_Faculty/ATCILTT_Collection_Faculty/com/accenture/ltt/collection/assignment

```

package com.accenture.ltt.collection.assignment.map;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

public class TransactionMapTester {

    public static void main(String[] args) {
        // Create a list to hold transaction
        List<Transaction> transactions = new ArrayList<>();
        // Add transaction objects to the list
        Transaction t1 = new Transaction("TD5678", 110095, 345678, 20000);
        transactions.add(t1);

        Transaction t2 = new Transaction("TD6764", 149352, 577321, 100000);
        transactions.add(t2);

        Transaction t3 = new Transaction("TD6785", 210995, 673567, 50000);
        transactions.add(t3);

        // Create a map to store transactionId and transaction object
        Map<String, Transaction> allTrasactions = new HashMap<>();
    }
}

```



```

        transactions.forEach(x -> {
            String transactionId = x.getTransactionId();
            allTrasactions.put(transactionId, x);
        });
        // Call print method
        TransactionService service = new TransactionService();
        service.printAllTransactions(allTrasactions, 210995);
    }
}

```

File: ATCILTT_Collection_Faculty/ATCILTT_Collection_Faculty/com/accenture/ltt/collection/assignment

```

package com.accenture.ltt.collection.assignment.map;

import java.util.Map;

public class TransactionService {
    // Method to print transaction ids for an account number
    public void printAllTransactions(Map<String, Transaction> transactions, long accountNumber) {
        //Check whether account number of each element in the collection is matching the provided
        transactions.forEach((k, v) -> {
            if (v.getSenderAccountNumber() == accountNumber) {
                //Print transaction id if a match of account number is found
                System.out.println(k);
            }
        });
    }
}

```

12.7 Viva Voce — Likely Questions & Answers

Q1. What is the difference between List, Set, and Map? A: List is an ordered collection that allows duplicates, accessed by index. Set holds only unique elements with (in the case of HashSet) no guaranteed order. Map stores key-value pairs with unique keys — it is not technically a subtype of Collection, since it deals with pairs, not single elements.

Q2. Why does removing an element with list.remove(item) inside an enhanced for-each loop throw ConcurrentModificationException? A: The for-each loop uses a hidden Iterator internally to track its position. Calling list.remove() directly structurally modifies the list *outside* that iterator's awareness, so the iterator detects the mismatch on its next step and fails fast rather than risking undefined behaviour. The safe fix is calling iterator.remove() explicitly via your own Iterator.

Q3. What is the difference between Comparable and Comparator? A: Comparable (via compareTo) is implemented *inside* the class itself and defines exactly one “natural” ordering. Comparator (via compare) is implemented *externally*, letting you define as many different orderings of the same objects as needed, chosen at the call site.

Q4. Why must you override both equals() and hashCode() together, never just one? A: The contract requires that equal objects (per equals()) *must* produce the same hashCode(). If you override only equals(), two logically-equal objects can still land in different hash buckets, causing HashSet.contains() or HashMap.get() to fail to find an object that should clearly match — silently breaking hash-based collections.

Q5. Can two unequal objects have the same hashCode()? What is this called, and how do hash-based collections handle it? A: Yes — this is called a **hash collision**. Since multiple distinct objects can legitimately share the same hash bucket, HashMap/HashSet fall back to calling `.equals()` to distinguish between objects that landed in the same bucket, ensuring correctness even when collisions occur.

Q6. What is the average time complexity of get/put on a HashMap, and why? A: $O(1)$ average case — the key's `hashCode()` determines which internal bucket to look in directly, avoiding the need to scan every entry. (Worst case can degrade toward $O(n)$ if many keys collide into the same bucket, though modern JDK versions mitigate this by converting densely-populated buckets into balanced trees.)

Q7. What's the difference between HashSet, LinkedHashSet, and TreeSet? A: HashSet gives no guaranteed order (fastest, $O(1)$ average operations). LinkedHashSet preserves insertion order at a small extra memory cost. TreeSet keeps elements in sorted order (via Comparable/Comparator) at $O(\log n)$ per operation.

Q8. Why is it important to override equals()/hashCode() for a custom class you plan to store in a HashSet or use as a HashMap key? A: Without overriding, the inherited Object versions compare by reference identity — two distinct objects representing the “same” logical entity (e.g., two Employee objects with the same ID) would be treated as different, defeating the purpose of a uniqueness-based collection like HashSet, or causing HashMap lookups by an “equivalent” key object to fail.

Chapter 13: Java 8 Features

Java 8 (2014) was one of the most transformative releases in the language’s history, introducing functional-style programming into a previously purely object-oriented language. Almost every “modern Java” idiom you’ll see in Spring Boot code later in this course (stream pipelines, lambda-based configuration, Optional) traces back to this single release.

13.1 Functional Interfaces

A **functional interface** is any interface with **exactly one abstract method** (it may have any number of default/static methods too, since those aren’t abstract) — this single-method shape is precisely what allows a **lambda expression** to be used anywhere that interface type is expected (the lambda’s body simply *becomes* that one abstract method’s implementation). Java marks these with the `@FunctionalInterface` annotation (optional but recommended — the compiler will then produce a compile error if a second abstract method is accidentally added later, catching an otherwise silent design mistake early).

`java.util.function` ships several ready-made functional interfaces so you rarely need to write your own from scratch:

Interface	Abstract method	Purpose	Typical use
<code>Consumer<T></code>	<code>void accept(T t)</code>	Takes a value, returns nothing	<code>list.forEach(item -> print(item))</code>
<code>Supplier<T></code>	<code>T get()</code>	Takes nothing, produces a value	Lazy/deferred value creation
<code>Predicate<T></code>	<code>boolean test(T t)</code>	Takes a value, returns true/false (a condition)	<code>stream.filter(x -> x > 0)</code>
<code>Function<T,R></code>	<code>R apply(T t)</code>	Takes a value, transforms it into another (possibly different) type	<code>stream.map(String::toUpperCase)</code>
<code>BiConsumer/BiPredicate/BiFunction</code>	versions of the above	Same idea, but with 2 inputs	<code>map.forEach((k, v) -> ...)</code>
<code>UnaryOperator<T></code>	<code>T apply(T t)</code>	A <code>Function<T,T></code> — input and output are the <i>same</i> type	<code>list.replaceAll(x -> x * 2)</code>
<code>BinaryOperator<T></code>	<code>T apply(T t1, T t2)</code>	A <code>BiFunction<T,T,T></code> — combines two values of the same type into one	<code>stream.reduce(0, Integer::sum)</code>

Why this matters for the viva: interviewers often ask you to *identify* which standard functional interface fits a given scenario (e.g., “validate an object and return true/false” → `Predicate`), so recognizing the shape (how many inputs, whether it returns something) is the key skill, not memorizing every name.

13.2 Default and Static Methods in Interfaces

Before Java 8, adding a new method to an interface would **break every existing implementing class** (they’d suddenly fail to compile, missing an implementation for the new method) — effectively making interfaces impossible to evolve once published and widely implemented.

default methods solve this — an interface can now provide a method **body**, which implementing classes inherit automatically and can use as-is, unless they choose to override it with their own logic. **static methods** on an interface belong to the interface itself (called as `InterfaceName.method()`, never via an instance), useful for utility/helper logic conceptually related to the interface that doesn't depend on any particular implementing instance's state.

A subtlety worth knowing: if a class implements two interfaces that both provide a default method with the *same signature*, the class **must** explicitly override that method itself to resolve the ambiguity (often delegating to one via `InterfaceName.super.method()`) — the compiler refuses to guess which default implementation “wins”.

13.3 Lambda Expressions

A lambda is a compact, anonymous implementation of a functional interface's single abstract method:

```
Runnable r = () -> System.out.println("Running!");
Calculator add = (a, b) -> a + b;
```

Syntax: (parameters) -> expression-or-block. If the body is a single expression, its value is implicitly returned (no return keyword or braces needed); a multi-statement body needs { } and an explicit return if a value is produced. Lambdas eliminate the verbosity of **anonymous inner classes** for simple, single-method behaviour — compare:

```
// Pre-Java 8, anonymous inner class:
Runnable r1 = new Runnable() {
    @Override
    public void run() { System.out.println("Running!"); }
};
// Java 8+, equivalent lambda:
Runnable r2 = () -> System.out.println("Running!");
```

Variable capture: a lambda can reference local variables from its enclosing scope, but only if they are **effectively final** (never reassigned after initialization) — this is required because the lambda may be invoked later, possibly on a different thread, and Java needs a stable, immutable snapshot of the captured value rather than a reference to a variable that could change underneath it.

13.4 Method References

A method reference (`::`) is shorthand for a lambda that does nothing but call an *already-existing* method. Four forms, each demonstrated in the code below:

1. **Static method reference** — `ClassName::staticMethod`, equivalent to `args -> ClassName.staticMethod(args)`.
2. **Instance method reference of a particular object** — `instance::method`, equivalent to `args -> instance.method(args)`.
3. **Instance method reference of an arbitrary object of a particular type** — `ClassName::instanceMethod`, equivalent to `(obj, args) -> obj.instanceMethod(args)` — here, the *first* lambda parameter itself becomes the object the method is called *on*.
4. **Constructor reference** — `ClassName::new`, equivalent to `args -> new ClassName(args)`.

There are well-defined “reduction rules” for converting a lambda that merely forwards its arguments into the equivalent, more concise method reference — the demo set below works through these rules explicitly with worked examples for both custom and standard functional interfaces.

13.5 Stream API

A Stream is a **pipeline** over a source of data (a collection, array, or generator) that supports **declarative**, chainable operations — you describe *what* transformation/result you want, not the step-by-step *how* (the imperative for-loop equivalent), split into two categories:

- **Intermediate (non-terminal) operations** — return *another* Stream, so they can be chained one after another: `filter` (keep only elements matching a predicate), `map` (transform each element into something else, possibly a different type), `peek` (inspect elements without altering the stream, mainly useful for debugging a pipeline), `distinct` (remove duplicate elements, based on `equals()`), `limit/skip` (truncate the stream to the first/remaining N elements), `sorted`.
- **Terminal operations** — actually **trigger execution** of the whole pipeline and produce a final result, ending the stream: `forEach` (side-effecting action per element), `toArray/collect` (gather results into an array or a collection like a `List/Map`), `max/min/count`, and the `summarizing*` family (computing statistics like average/sum/count/min/max in a **single pass** over the data).

Two crucial, frequently-tested properties of streams:

1. **Laziness** — intermediate operations don't actually execute *anything* until a terminal operation is invoked; building a pipeline of `filter().map().sorted()` does no work by itself until, say, `.collect()` or `.forEach()` is finally called — this allows the JVM to optimize the whole pipeline (e.g., fusing operations, short-circuiting with `limit()`) rather than materializing an intermediate collection after every single step.
2. **Single-use** — a stream can only be **consumed once**; calling a terminal operation on it and then trying to reuse the same stream object throws `IllegalStateException`. If you need to process the same data multiple ways, you must create a **fresh stream** from the source collection each time (the source collection itself is unaffected by stream operations, only the stream object itself is single-use).

The demo programs below are organized exactly along these five sub-topics, in the order the topics were taught: functional interfaces via default/static interface methods, lambdas, method references, stream creation, and finally the two families of stream operations.

13.5.1 Default & Static Methods in Interfaces

File: `ATCILKM_LambdaExpression_Faculty/com/accenture/ltt/java8features/DefaultMethodImpl.java`

```
package com.accenture.ltt.java8features;

public class DefaultMethodImpl implements IDefaultMethod {

    //Default can be overridden in the sub class
    //Comment the below print() method and try
    public void print() {
        System.out.println("Default method overridden in the implemented class");
    }

}
```

File: `ATCILKM_LambdaExpression_Faculty/com/accenture/ltt/java8features/DefaultMethodImpl1.java`

```
package com.accenture.ltt.java8features;

public class DefaultMethodImpl1 implements IDefaultMethodOne, IDefaultMethodTwo {
```

```

        //Duplicate default methods named print with the parameters () and () are inherited from th

@Override
public void print() {

    IDefaultMethodOne.super.print(); // change to IDefaultMethodTwo.super.print()
}

}

```

File: ATCILKM_LambdaExpression_Faculty/com/accenture/ltt/java8features/DefaultMethodTest.java

```

package com.accenture.ltt.java8features;

public class DefaultMethodTest {

    public static void main(String[] args) {
        IDefaultMethod defaultObj = new DefaultMethodImpl();
        defaultObj.print();
    }

}

```

File: ATCILKM_LambdaExpression_Faculty/com/accenture/ltt/java8features/DefaultMethodTest1.java

```

package com.accenture.ltt.java8features;

public class DefaultMethodTest1 {

    public static void main(String[] args) {
        // TODO Auto-generated method stub

        IDefaultMethodOne defaultObj = new DefaultMethodImpl1();
        defaultObj.print();
    }

}

```

File: ATCILKM_LambdaExpression_Faculty/com/accenture/ltt/java8features/IDefaultMethod.java

```

package com.accenture.ltt.java8features;

public interface IDefaultMethod {
    public default void print() {
        System.out.println("Default Method !!");
    }
}

```

File: ATCILKM_LambdaExpression_Faculty/com/accenture/ltt/java8features/IDefaultMethodOne.java

```
package com.accenture.ltt.java8features;

public interface IDefaultMethodOne {
    public default void print() {
        System.out.println("Default Method One !!");
    }
}
```

File: ATCILKM_LambdaExpression_Faculty/com/accenture/ltt/java8features/IDefaultMethodTwo.java

```
package com.accenture.ltt.java8features;

public interface IDefaultMethodTwo {
    public default void print() {
        System.out.println("Default Method Two!!");
    }
}
```

File: ATCILKM_LambdaExpression_Faculty/com/accenture/ltt/java8features/IStaticMethod.java

```
package com.accenture.ltt.java8features;

public interface IStaticMethod {
    static void print() {
        System.out.println("print");
    }
}
```

File: ATCILKM_LambdaExpression_Faculty/com/accenture/ltt/java8features/StaticMethodImpl.java

```
package com.accenture.ltt.java8features;

public class StaticMethodImpl implements IStaticMethod {

    // @Override
    //Static Method cannot be overridden
    public static void print() {
        System.out.println("Implemented class");
    }
}
```

File: ATCILKM_LambdaExpression_Faculty/com/accenture/ltt/java8features/TestInterfaceStaticMethod

```
package com.accenture.ltt.java8features;

public class TestInterfaceStaticMethod {

    public static void main(String[] args) {
        IStaticMethod.print();
    }
}
```

```

        StaticMethodImpl.print();
    }
}

```

13.5.2 Lambda Expressions (Custom & Standard Functional Interfaces)

File: ATCILKM_LambdaExpression_Faculty/com/accenture/ltt/lambdaexp/activity/Calculator.java

```

package com.accenture.ltt.lambdaexp.activity;

// Functional interface having only one method
public interface Calculator {

    /**
     * @param a
     * @param b
     * @return
     */
    int operation(int a, int b);
}

```

File: ATCILKM_LambdaExpression_Faculty/com/accenture/ltt/lambdaexp/activity/CalculatorTest.java

```

package com.accenture.ltt.lambdaexp.activity;

/**
 * CalculatorTest class which perform addition, subtraction, multiplication
 * and division depending upon the implementation of Calculator.
 */
public class CalculatorTest {

    /**
     * Calling Calculator.operation method by passing value of a, b
     * @param a
     * @param b
     * @param op
     * @return
     */
    public int operate(int a, int b, Calculator op) {
        return op.operation(a, b);
    }

    public static void main(String[] args) {

        //TODO 1 - Create CalculatorTest object
        CalculatorTest calculatorTest = new CalculatorTest();
        //TODO 2 - Create Calculator implementation addition using lambda expression and add bo
        Calculator addition = (a,b) -> a+b;
        //TODO 3 - Create Calculator implementation subtraction using lambda expression and sub
        Calculator subtraction = (a,b) -> a-b;
    }
}

```



```

//TODO 4 - Create Calculator implementation multiplication using lambda expression and
Calculator multiplication = (a,b) -> a*b;
//TODO 5 - Create Calculator implementation division using lambda expression and divide
Calculator division = (a,b) -> a/b;
//TODO 6 - Assign the value to int variable by calling operate method using object of C
int addValue = calculatorTest.operate(30, 20, addition);
//TODO 7 - Assign the value to int variable by calling operate method using object of C
int subValue = calculatorTest.operate(30, 20, subtraction);
//TODO 8 - Assign the value to int variable by calling operate method using object of C
int mulValue = calculatorTest.operate(30, 20, multiplication);
//TODO 9 - Assign the value to int variable by calling operate method using object of C
int divValue = calculatorTest.operate(30, 20, division);
//TODO 10 - Print the values found in step 6 to 9
System.out.println("addValue = " + addValue + " subValue = " + subValue + " mulValue = " + mulValue);
}
}

```

File: ATCILKM_LambdaExpression_Faculty/com/accenture/ltt/lambdaexp/sample/Drawable.java

```

package com.accenture.ltt.lambdaexp.sample;
//Functional Interface with one method
public interface Drawable {

    void draw();

}

```

File: ATCILKM_LambdaExpression_Faculty/com/accenture/ltt/lambdaexp/sample/DrawableTest.java

```

package com.accenture.ltt.lambdaexp.sample;

public class DrawableTest {

    //method to draw a shape by calling Drawable draw method
    void drawShape(Drawable drawable) {
        drawable.draw();
    }

    public static void main(String[] args) {

        DrawableTest drawableTest = new DrawableTest();

        //Circle implementation of Drawable interface
        Drawable circle = () -> {
            System.out.println("Drawing circle");
        };
        //Rectangle implementation of Drawable interface
        Drawable rectangle = () -> {
            System.out.println("Drawing rectangle");
        };
        //Traingle implementation of Drawable interface
        Drawable traingle = () -> {
            System.out.println("Drawing traingle");
        };
    }
}

```

```

        //need circle implementation so passing circle reference in drawShape method
        drawableTest.drawShape(circle);

        //need rectangle implementation so passing rectangle reference in drawShape method
        drawableTest.drawShape(rectangle);

        //need traingle implementation so passing traingle reference in drawShape method
        drawableTest.drawShape(traingle);
    }
}

```

File: ATCILKM_LambdaExpression_Faculty/com/accenture/ltt/lambdaexp/sample/Greeting.java

```

package com.accenture.ltt.lambdaexp.sample;

//Functional interface with one method only
public interface Greeting {

    public void greet(String name);
}

```

File: ATCILKM_LambdaExpression_Faculty/com/accenture/ltt/lambdaexp/sample/Welcome.java

```

package com.accenture.ltt.lambdaexp.sample;

/**
 * Demonstration of how anonymous class is written
 *
 */
public class Welcome {

    public static void main(String[] args) {
        //creating anonymous class for Greeting implementation
        Greeting greeting = new Greeting() {
            public void greet(String name) {
                System.out.println("Welcome "+name);
            }
        };

        //calling greet method of Greeting interface
        greeting.greet("Seeta");
    }
}

```

File: ATCILKM_LambdaExpression_Faculty/com/accenture/ltt/lambdaexp/sample/WelcomeUsers.java

```

package com.accenture.ltt.lambdaexp.sample;

public class WelcomeUsers {

    public static void main(String[] args) {
        Greeting greeting = (name) -> {System.out.println("Welcome "+name)};;
        greeting.greet("Seeta");
    }
}

```

```
}
```

File: Demo10001_Lambdas_Using_StandardInterfaces/com/accenture/ltt/a/pojo/Customer.java

```
package com.accenture.ltt.a.pojo;

public class Customer {
    public void printCustomerNameInUpper(String cname) {
        System.out.println(cname.toUpperCase());
    }
    public String returnCustomerName() {
        return "customer name";
    }
    public Integer returnCustomerNameLength(String customerName) {
        return customerName.length();
    }
    public void printCustomerDetails(int customerId, String customerName) {
        System.out.println("Customer Details");
        System.out.println("-----");
        System.out.println("CID: " + customerId);
        System.out.println("Customer Name: " + customerName);
    }
}
```

File: Demo10001_Lambdas_Using_StandardInterfaces/com/accenture/ltt/a/pojo/Product.java

```
package com.accenture.ltt.a.pojo;

public class Product {
    public void printProductNameUpper(String pname) {
        System.out.println(pname.toUpperCase());
    }
    public boolean productCategory(String category) {
        return (category.equals("electronics")) ? true : false;
    }
    public String returnProductName() {
        return "product name";
    }
    public Integer returnProductNameLength(String productName) {
        return productName.length();
    }
    public void printProductDetails(int productId, String productName) {
        System.out.println("Product Details");
        System.out.println("-----");
        System.out.println("PID: " + productId);
        System.out.println("PName: " + productName);
    }

    public double returnDiscountedPrice(String productName, double price) {
        return productName.startsWith("A") ? (price * (15.0 / 100)) : price;
    }

    public boolean productNameContains(String name, String str) {
        return name.contains(str);
    }
}
```

```

    public double computeFinalPriceAfterFixedDiscount(double price) {
        return price + (price * (15.0 / 100));
    }

    public double computeFinalPriceAfterCustomDiscount(double price, Double discount) {
        return price + (price * (discount / 100));
    }
}

```

File: Demo10001_Lambdas_Using_StandardInterfaces/com/accenture/ltt/custom/interfaces/Tester1001P

```

package com.accenture.ltt.custom.interfaces;

import com.accenture.ltt.a.pojo.Product;

interface ProductConsumerInterface {

    void printNameInUpper(String name);

}

public class Tester1001ProductConsumerClass {

    public static void main(String[] args) {
        // Product
        ProductConsumerInterface pciProduct = (name) -> new Product().printProductNameUpper(name);
        pciProduct.printNameInUpper("samsung");
    }

}

```

File: Demo10001_Lambdas_Using_StandardInterfaces/com/accenture/ltt/custom/interfaces/Tester1002P

```

package com.accenture.ltt.custom.interfaces;

import com.accenture.ltt.a.pojo.Product;

interface ProductPredicateInterface {
    boolean productCategory(String name);
}

public class Tester1002ProductPredicateClass {

    public static void main(String[] args) {
        // Product
        ProductPredicateInterface ppi = (name) -> new Product().productCategory(name);
        System.out.println("Is product electronics category? " + ppi.productCategory("electronics"));
    }

}

```

File: Demo10001_Lambdas_Using_StandardInterfaces/com/accenture/ltt/custom/interfaces/Tester1003P

```

package com.accenture.ltt.custom.interfaces;

```

```

import com.accenture.ltt.a.pojo.Product;

interface ProductSupplierInterface {
    String returnName();
}

public class Tester1003ProductSupplierClass {
    public static void main(String[] args) {
        ProductSupplierInterface msiProduct = () -> new Product().returnProductName();
        System.out.println(msiProduct.returnName());
    }
}

```

File: Demo10001_Lambdas_Using_StandardInterfaces/com/accenture/ltt/custom/interfaces/Tester1004P

```

package com.accenture.ltt.custom.interfaces;

import com.accenture.ltt.a.pojo.Product;

interface ProductFunctionalInterface {
    Integer returnLengthOfName(String name);
}

public class Tester1004ProductFunctionClass {
    public static void main(String[] args) {
        ProductFunctionalInterface pfi = (name) -> new Product().returnProductNameLength(name);
        Integer result1 = pfi.returnLengthOfName("Samsung");
        System.out.println("Length of product name: " + result1);
    }
}

```

File: Demo10001_Lambdas_Using_StandardInterfaces/com/accenture/ltt/custom/interfaces/Tester1005P

```

package com.accenture.ltt.custom.interfaces;

import com.accenture.ltt.a.pojo.Product;

interface ProductBiConsumerInterface {
    void printDetails(int id, String name);
}

public class Tester1005ProductBiConsumerClass {

    public static void main(String[] args) {
        // Product
        ProductBiConsumerInterface pbci = (id, name) -> new Product().printProductDetails(id, name);
        pbci.printDetails(111, "SONY");
    }
}

```

File: Demo10001_Lambdas_Using_StandardInterfaces/com/accenture/ltt/standard/interfaces/Tester100

```

package com.accenture.ltt.standard.interfaces;

import java.util.function.BiConsumer;
import java.util.function.Consumer;

import com.accenture.ltt.a.pojo.Product;

public class Tester1006Consumer {
    public static void main(String[] args) {

        // Product - print Product Name in upper case
        Consumer<String> product4 = (pname) -> new Product().printProductNameUpper(pname);
        product4.accept("LG");

        // Its not a good practice to tie to an object
        // This can be resolved by using by using accept() of BiConsumer by passing object and

        // Alternative:
        BiConsumer<Product, String> product5 = (productL, name) -> productL.printProductNameUpper(name);
        // class on which to invoke the method
        // parameter data type to be passed to function
        product5.accept(new Product(), "Samsung");
    }
}

```

File: Demo10001_Lambdas_Using_StandardInterfaces/com/accenture/ltt/standard/interfaces/Tester1007Predicate.java

```

package com.accenture.ltt.standard.interfaces;

import java.util.function.BiPredicate;
import java.util.function.Predicate;

import com.accenture.ltt.a.pojo.Product;

public class Tester1007Predicate {
    public static void main(String[] args) {

        // Product
        // checks the name of product contains minimum of 5 characters or not
        System.out.println("Product name contains minimum of 5 characters");
        System.out.println("=====");

        Predicate<String> predicateLength = (pname) -> new Product().returnProductNameLength(pname) >= 5;
        System.out.println(predicateLength.test("Samsung"));

        // Its not a good practice to tie to an object
        // This can be resolved by using by using accept() of BiConsumer by passing object and

        // Alternative:
        BiPredicate<Product, String> biPredicateLength = (productL, name) -> productL.returnProductNameLength(name) >= 5;
        // class on which to invoke the method
        // parameter data type to be passed to function
    }
}

```

```

        System.out.println(biPredicateLength.test(new Product(), "Samsung"));
    }
}

```

File: Demo10001_Lambdas_Using_StandardInterfaces/com/accenture/ltt/standard/interfaces/Tester100

```

package com.accenture.ltt.standard.interfaces;

import java.util.function.BiFunction;
import java.util.function.Function;

import com.accenture.ltt.a.pojo.Product;

public class Tester1008Function {
    public static void main(String[] args) {

        System.out.println("Function Interface\n-----");
        Function<String, Integer> product1 = (name) -> new Product().returnProductNameLength(name);
        Integer result1 = product1.apply("Samsung");
        System.out.println(result1);

        // Its not a good practice to tie Lambda to an object
        // This can be resolved by using accept() of Function by passing the object and
        // data to the Lambda as below

        System.out.println("BiFunction Interface\n-----");
        BiFunction<Product, String, Integer> product5 = (product, productName) -> product.returnProductNameLength(productName);
        int total = product5.apply(new Product(), "Alexa");
        System.out.println(total);
    }
}

```

File: Demo10001_Lambdas_Using_StandardInterfaces/com/accenture/ltt/standard/interfaces/Tester100

```

package com.accenture.ltt.standard.interfaces;

import java.util.function.Function;
import java.util.function.Supplier;

import com.accenture.ltt.a.pojo.Product;

public class Tester1009Supplier {
    public static void main(String[] args) {
        // Product
        // not good implementation as tied to the Object
        System.out.println("returnProductName");
        System.out.println("=====");
        Supplier<String> product2 = () -> new Product().returnProductName();
        // The method is invoked by get() of Supplier
        System.out.println(product2.get());
    }
}

```

```

        Function<Product, String> funProduct= (product) -> product.returnProductName();
        String productName = funProduct.apply(new Product());

        System.out.println("Product Name: " + productName);
    }
}

```

File: Demo10001_Lambdas_Using_StandardInterfaces/com/accenture/ltt/standard/interfaces/exact/sig

```

package com.accenture.ltt.standard.interfaces.exact.signature.match;

import java.util.function.BiConsumer;

import com.accenture.ltt.a.pojo.Product;

public class Tester1010BiConsumer {
    public static void main(String[] args) {

        // Product
        // printProductDetails
        // RHS Method signature matches with LHS
        System.out.println("printProductDetails");
        System.out.println("=====");
        BiConsumer<Integer, String> product = (productId, productName) -> new Product().printPr
            productName);
        product.accept(12, "Sony");

    }
}

```

File: Demo10001_Lambdas_Using_StandardInterfaces/com/accenture/ltt/standard/interfaces/exact/sig

```

package com.accenture.ltt.standard.interfaces.exact.signature.match;

import java.util.function.BiPredicate;

import com.accenture.ltt.a.pojo.Product;

public class Tester1011BiPredicate {
    public static void main(String[] args) {

        // RHS Method signature matches with LHS
        BiPredicate<String, String> biPredicateLength = (pname, name) -> new Product().productN
        System.out.println(biPredicateLength.test("Samsung", "sun"));

    }
}

```

File: Demo10001_Lambdas_Using_StandardInterfaces/com/accenture/ltt/standard/interfaces/exact/sig

```

package com.accenture.ltt.standard.interfaces.exact.signature.match;

import java.util.function.BiFunction;

```



```
import com.accenture.ltt.a.pojo.Product;

public class Tester1012BiFunction {
    public static void main(String[] args) {

        System.out.println("BiFunction Interface\n-----");
        // RHS Method signature matches with LHS
        BiFunction<String, Double, Double> product4 = (productName, price) -> new Product().ret
        Double total = product4.apply("Alexa", 500000.2);
        System.out.println(total);
    }
}
```

File: Demo10001_Lambdas_Using_StandardInterfaces/com/accenture/ltt/standard/interfaces/exact/sig

```
package com.accenture.ltt.standard.interfaces.exact.signature.match;

import java.util.function.UnaryOperator;

import com.accenture.ltt.a.pojo.Product;

public class Tester1013UnaryOperator {

    public static void main(String[] args) {
        UnaryOperator<Double> finalPrice = (price) -> new Product().computeFinalPriceAfterFixed
        System.out.println(finalPrice.apply(1000.0));
    }
}
```

File: Demo10001_Lambdas_Using_StandardInterfaces/com/accenture/ltt/standard/interfaces/exact/sig

```
package com.accenture.ltt.standard.interfaces.exact.signature.match;

import java.util.function.BinaryOperator;

import com.accenture.ltt.a.pojo.Product;

public class Tester1014BinaryOperator {
    public static void main(String[] args) {
        BinaryOperator<Double> finalPrice = (price, discount) -> new Product().computeFinalPric
        System.out.println(finalPrice.apply(1000.0,10.0));
    }
}
```

File: Demo10001_Lambdas_Using_StandardInterfaces/com/accenture/ltt/z/assignment/Tester10008_Assi

```
package com.accenture.ltt.z.assignment;
interface MyInterface11{
    void service(int i, String str, double d); // should display all the details
}
interface MyInterface22{
    double computeTax(double salary, double taxPer); // return tax
}
//(salary+salary*tax/100)
interface MyInterface33{
```

```

    void dummyFunction();// prints hello world
}
interface MyInterface444{
    int dummyFunction2();// should return a random value
}
interface MyInterface5{
    int dummyFunction2(int x);// should return a random value
}

public class Tester10008_Assignment {

    public static void main(String[] args) {
        //Line1
        /*MyInterface11 my1 = (var1, var2,var3)->{
            System.out.println(var1);
            System.out.println(var2);
            System.out.println(var3);

        },*/

        MyInterface11 my1 = (var1, var2,var3)->System.out.println(var1+" "+var2+" "+var3);

        my1.service(12, "Jack", 123);

        /*MyInterface22 my2 = (var2,var3)->{
            System.out.println(var2);
            System.out.println(var3);
            double tax= var2+var2*var3/100;
            return (int)tax;
        },*/

        MyInterface22 my2=(var2,var3)->var2+var2*var3/100;
        my2.computeTax(123,10);

        //MyInterface33 my3 = ()->{System.out.println("Hello world");};

        MyInterface33 my3 = ()->System.out.println("Hello world");
        my3.dummyFunction();

        MyInterface444 my4 = ()->{System.out.println("REtunring value");
            return (int)(Math.random()*10);
        };

        my4.dummyFunction2();

        MyInterface5 my5 = (x)->{return x+10;};

        MyInterface5 my6 = x->x+10;
    }
}

```

```

    }
}

```

13.5.3 Method References

File: Demo10002_MethodReference/com/accenture/ltt/Customer.java

```

package com.accenture.ltt;

public class Customer {
    private int customerID;
    private String customerName;

    public Customer() {
        super();
        // TODO Auto-generated constructor stub
    }
    public Customer(int customerID, String customerName) {
        super();
        this.customerID = customerID;
        this.customerName = customerName;
    }
    public int getCustomerID() {
        return customerID;
    }
    public void setCustomerID(int customerID) {
        this.customerID = customerID;
    }
    public String getCustomerName() {
        return customerName;
    }
    public void setCustomerName(String customerName) {
        this.customerName = customerName;
    }

    //*****
    //*****Static*****

    public static void printNameUpperStatic(String cname) {
        System.out.println(cname.toUpperCase());
    }

    public static void displayDetails(int customerId, String customerName){
        System.out.println("Customer Details Static");
        System.out.println("-----");
        System.out.println("CID: " + customerId);
        System.out.println("CName: " + customerName);
    }

    // *****
    //*****non-Static*****

```

```

        public void printCustomerDetails(int customerId, String customerName) {
            System.out.println("Customer Details");
            System.out.println("-----");
            System.out.println("CID: " + customerId);
            System.out.println("CName: " + customerName);
        }

        public void printNameUpper(String cname) {
            System.out.println(cname.toUpperCase());
        }

        // *****
    }

```

File: Demo10002_MethodReference/com/accenture/ltt/Product.java

```

package com.accenture.ltt;

public class Product {
    private int productId;
    private String productName;

    public Product() { }

    public Product(int productId, String productName) {
        this.productId = productId;
        this.productName = productName;
    }

    public int getProductId() {
        return productId;
    }

    public void setProductId(int productId) {
        this.productId = productId;
    }

    public String getProductName() {
        return productName;
    }

    public void setProductName(String productName) {
        this.productName = productName;
    }

    //*****
    //*****Static*****

    public static void printNameUpperStatic(String pname) {
        System.out.println(pname.toUpperCase());
    }

    public static void displayDetails(int productId, String productName){

```

```

        System.out.println("Product Details Static");
        System.out.println("-----");
        System.out.println("PID: " + productId);
        System.out.println("PName: " + productName);
    }

    // *****
    //*****non-Static*****
    public void printProductDetails(int productId, String productName) {
        System.out.println("Product Details");
        System.out.println("-----");
        System.out.println("PID: " + productId);
        System.out.println("PName: " + productName);
    }

    public void printNameUpper(String pname) {
        System.out.println(pname.toUpperCase());
    }

    // *****
}

```

File: Demo10002_MethodReference/com/accenture/ltt/a/interfaces/DefaultCustomerConstructorInterface

```

package com.accenture.ltt.a.interfaces;

import com.accenture.ltt.Customer;

public interface DefaultCustomerConstructorInterface {
    Customer getCustomerObject();
}

```

File: Demo10002_MethodReference/com/accenture/ltt/a/interfaces/DefaultProductConstructorInterface

```

package com.accenture.ltt.a.interfaces;

import com.accenture.ltt.Product;

public interface DefaultProductConstructorInterface {
    Product getProductObject();
}

```

File: Demo10002_MethodReference/com/accenture/ltt/a/interfaces/OneParameterCustomerObjectInterface

```

package com.accenture.ltt.a.interfaces;

import com.accenture.ltt.Customer;

public interface OneParameterCustomerObjectInterface {
    void printNameInUpperCase(Customer customer, String name);
}

```

File: Demo10002_MethodReference/com/accenture/ltt/a/interfaces/OneParameterInterface.java

```

package com.accenture.ltt.a.interfaces;

```

```
public interface OneParameterInterface {
    void printNameInUpperCase(String name);
}
```

File: Demo10002_MethodReference/com/accenture/ltt/a/interfaces/OneParameterProductObjectInterface

```
package com.accenture.ltt.a.interfaces;

import com.accenture.ltt.Product;

public interface OneParameterProductObjectInterface {
    void printNameInUpperCase(Product product, String name);
}
```

File: Demo10002_MethodReference/com/accenture/ltt/a/interfaces/ParameterizedCustomerConstructorInterface

```
package com.accenture.ltt.a.interfaces;

import com.accenture.ltt.Customer;

public interface ParameterizedCustomerConstructorInterface {
    Customer getCustomerObject(Integer id, String name);
}
```

File: Demo10002_MethodReference/com/accenture/ltt/a/interfaces/ParameterizedProductConstructorInterface

```
package com.accenture.ltt.a.interfaces;

import com.accenture.ltt.Product;

public interface ParameterizedProductConstructorInterface {
    Product getProductObject(int id, String name);
}
```

File: Demo10002_MethodReference/com/accenture/ltt/a/interfaces/TwoParameterCustomerObjectInterface

```
package com.accenture.ltt.a.interfaces;

import com.accenture.ltt.Customer;

public interface TwoParameterCustomerObjectInterface {
    void display(Customer customer, int id, String name);
}
```

File: Demo10002_MethodReference/com/accenture/ltt/a/interfaces/TwoParameterInterface.java

```
package com.accenture.ltt.a.interfaces;

public interface TwoParameterInterface {
    void display(int id, String name);
}
```

File: Demo10002_MethodReference/com/accenture/ltt/a/interfaces/TwoParameterProductObjectInterface

```
package com.accenture.ltt.a.interfaces;

import com.accenture.ltt.Product;
```

```
public interface TwoParameterProductObjectInterface {
    void display(Product product, int id, String name);
}
```

File: Demo10002_MethodReference/com/accenture/ltt/mr/custom/interfaces/Tester1001NeedOfMethodRef

```
package com.accenture.ltt.mr.custom.interfaces;

import com.accenture.ltt.Customer;
import com.accenture.ltt.Product;
import com.accenture.ltt.a.interfaces.OneParameterInterface;
import com.accenture.ltt.a.interfaces.TwoParameterInterface;

public class Tester1001NeedOfMethodReferenceStatic {

    // Need of MR for Static methods
    public static void req1() {
        /*
         * Req1: Lambda Expression to invoke a static method
         * printNameUpperStatic() of the class Product to print the product name in
         * upper case
         */
        // Product
        OneParameterInterface printProduct = (name) -> Product.printNameUpperStatic(name);
        printProduct.printNameInUpperCase("samsung");

        /*
         * Req1: Method Reference to invoke a static method
         * printNameUpperStatic() of the class Product to print the product name in
         * upper case
         */
        OneParameterInterface printProduct2 = Product::printNameUpperStatic;
        printProduct2.printNameInUpperCase("samsung");

        /*
         * Req1: Lambda Expression to invoke a static method
         * printNameUpperStatic() of the class Customer to print the customer name in
         * upper case
         */
        // Customer
        OneParameterInterface printCustomer = (name) -> Customer.printNameUpperStatic(name);
        printCustomer.printNameInUpperCase("Jack");

        /*
         * Req1: Method Reference to invoke a static method
         * printNameUpperStatic() of the class Customer to print the customer name in
         * upper case
         */
        OneParameterInterface printCustomer2 = Customer::printNameUpperStatic;
        printCustomer2.printNameInUpperCase("Jack");
    }
}
```

```

public static void req2() {
    /*
     * Req2: Lambda Expression to invoke a static method
     * displayDetails() of the class Product to print the product details
     */
    //Product
    TwoParameterInterface printProduct3 = (productId, name) -> Product.displayDetails(productId, name);
    printProduct3.display(11, "LG");

    /*
     * Req2: Method Reference to invoke a static method
     * displayDetails() of the class Product to print the product details
     */
    TwoParameterInterface printProduct4 = Product::displayDetails;
    printProduct4.display(11, "LG");

    /*
     * Req2: Lambda Expression to invoke a static method
     * displayDetails() of the class Customer to print the customer details
     */
    // customer
    TwoParameterInterface printCustomer3 = (customerId, name) -> Customer.displayDetails(customerId, name);
    printCustomer3.display(101, "Jack");

    /*
     * Req2: Method Reference to invoke a static method
     * displayDetails() of the class Customer to print the customer details
     */
    TwoParameterInterface printCustomer4 = Customer::displayDetails;
    printCustomer4.display(101, "Jack");
}

@SuppressWarnings("unused")
public static void main(String[] args) {

    req1();

    //    req2();

}
}

```

File: Demo10002_MethodReference/com/accenture/ltt/mr/custom/interfaces/Tester1002NeedOfMethodReferenceNonStatic.java

```

package com.accenture.ltt.mr.custom.interfaces;

import com.accenture.ltt.Customer;
import com.accenture.ltt.Product;
import com.accenture.ltt.a.interfaces.OneParameterInterface;
import com.accenture.ltt.a.interfaces.TwoParameterInterface;

public class Tester1002NeedOfMethodReferenceNonStatic {

```



```

// Need of MR for Instance methods
public static void req1() {
    /*
     * Req1: Lambda Expression to invoke a instance method
     * printNameUpper() of the class Product to print the product name in
     * upper case
     */
    OneParameterInterface printProduct = (name) -> new Product().printNameUpper(name);
    printProduct.printNameInUpperCase("samsung");

    /*
     * Req1: Method Reference to invoke a instance method
     * printNameUpper() of the class Product to print the product name in
     * upper case
     */
    OneParameterInterface printProduct2 = new Product()::printNameUpper;
    printProduct2.printNameInUpperCase("samsung");

    /*
     * Req1: Lambda Expression to invoke a instance method
     * printNameUpper() of the class Customer to print the customer name in
     * upper case
     */
    OneParameterInterface printCustomer = (name) -> new Customer().printNameUpper(name);
    printCustomer.printNameInUpperCase("Jack");

    /*
     * Req1: Method Reference to invoke a instance method
     * printNameUpper() of the class Customer to print the customer name in
     * upper case
     */
    OneParameterInterface printCustomer2 = new Customer()::printNameUpper;
    printCustomer2.printNameInUpperCase("Jack");
}

public static void req2() {
    /*
     * Req2: Lambda Expression to invoke a instance method
     * printProductDetails() of the class Product to print the product details
     */
    TwoParameterInterface printProduct3 = (productId, name) -> new Product().printProductDe
    printProduct3.display(11, "Jackson");

    /*
     * Req2: Method Reference to invoke a instance method
     * printProductDetails() of the class Product to print the product details
     */
    TwoParameterInterface printProduct4 = new Product()::printProductDetails;
    printProduct4.display(11, "Jackson");

    /*
     * Req2: Lambda Expression to invoke a instance method

```

```

        /* printCustomerDetails() of the class Customer to print the customer details
        */
TwoParameterInterface printCustomer3 = (customerId, name) -> new Customer().printCustomerDetails();
printCustomer3.display(101, "Jackson");

/*
    * Req2: Method Reference to invoke a instance method
    * printCustomerDetails() of the class Customer to print the customer details
    */
TwoParameterInterface printCustomer4 = new Customer()::printCustomerDetails;
printCustomer4.display(101, "Jackson");
}

@SuppressWarnings("unused")
public static void main(String[] args) {

    req1();

    //    req2();

}
}

```

File: Demo10002_MethodReference/com/accenture/ltt/mr/custom/interfaces/Tester1003NeedOfMethodReferenceConstructor.java

```

package com.accenture.ltt.mr.custom.interfaces;

import com.accenture.ltt.Customer;
import com.accenture.ltt.Product;
import com.accenture.ltt.a.interfaces.DefaultCustomerConstructorInterface;
import com.accenture.ltt.a.interfaces.DefaultProductConstructorInterface;
import com.accenture.ltt.a.interfaces.ParameterizedCustomerConstructorInterface;
import com.accenture.ltt.a.interfaces.ParameterizedProductConstructorInterface;

public class Tester1003NeedOfMethodReferenceConstructor {

    // Need of MR for Constructor
    public static void req1() {
        // Req1: Lambda Expression to get the product instance (object) using
        // Default Constructor
        DefaultProductConstructorInterface productObj = () -> new Product();
        Product p = productObj.getProductObject();

        // Req1: Method Reference to get the product instance (object) using
        // Default Constructor
        DefaultProductConstructorInterface productObj2 = Product::new;
        Product p2 = productObj2.getProductObject();

        // Req1: Lambda Expression to get the customer instance (object) using
        // Default Constructor
        DefaultCustomerConstructorInterface customerObj = () -> new Customer();
        Customer c = customerObj.getCustomerObject();
    }
}

```

```

        // Req1: Method Reference to get the customer instance (object) using
        // Default Constructor
        DefaultCustomerConstructorInterface customerObj2 = Customer::new;
        Customer c2 = customerObj2.getCustomerObject();
    }

    public static void req2() {
        // Req2: Lambda Expression to get the product instance (object) using
        // parameterized Constructor
        ParameterizedProductConstructorInterface productObj2 = (id, name) -> new Product(id, name);
        Product p2 = productObj2.getProductObject(11, "Jackson");

        // Req2: Method Reference to get the product instance (object) using
        // parameterized Constructor
        ParameterizedProductConstructorInterface productObj3 = Product::new;
        Product p3 = productObj3.getProductObject(11, "Jackson");

        // Req2: Lambda Expression to get the customer instance (object) using
        // parameterized Constructor
        ParameterizedCustomerConstructorInterface customerObj2 = (id, name) -> new Customer(id, name);
        Customer c2 = customerObj2.getCustomerObject(101, "Jackson");

        // Req2: Method Reference to get the customer instance (object) using
        // parameterized Constructor
        ParameterizedCustomerConstructorInterface customerObj3 = Customer::new;
        Customer c3 = customerObj3.getCustomerObject(101, "Jackson");
    }

    @SuppressWarnings("unused")
    public static void main(String[] args) {

        req1();

        // req2();

    }
}

```

File: Demo10002_MethodReference/com/accenture/ltt/mr/reducedmr/Tester1004LambdaToMRReductionRule

```

package com.accenture.ltt.mr.reducedmr;

import com.accenture.ltt.Customer;
import com.accenture.ltt.Product;
import com.accenture.ltt.a.interfaces.OneParameterInterface;
import com.accenture.ltt.a.interfaces.TwoParameterInterface;
// Rule 1 Reduction: before arrow and after arrow signature are same and after arrow it invokes
public class Tester1004LambdaToMRReductionRule1 {

    public static void req1() {
        /*
         * Req1: Lambda Expression to invoke a instance method
         * printNameUpper() of the class Product to print the product name in

```

```

    * upper case
    */
OneParameterInterface printProduct = (name) -> new Product().printNameUpper(name);
printProduct.printNameInUpperCase("samsung");

/*
 * Req1: Method Reference to invoke a instance method
 * printNameUpper() of the class Product to print the product name in
 * upper case
 */
OneParameterInterface printProduct2 = new Product()::printNameUpper;
printProduct2.printNameInUpperCase("samsung");

/*
 * Req1: Lambda Expression to invoke a instance method
 * printNameUpper() of the class Customer to print the customer name in
 * upper case
 */
OneParameterInterface printCustomer = (name) -> new Customer().printNameUpper(name);
printCustomer.printNameInUpperCase("Jack");

/*
 * Req1: Method Reference to invoke a instance method
 * printNameUpper() of the class Customer to print the customer name in
 * upper case
 */
OneParameterInterface printCustomer2 = new Customer()::printNameUpper;
printCustomer2.printNameInUpperCase("Jack");
}

public static void req2() {
    /*
     * Req2: Lambda Expression to invoke a instance method
     * printProductDetails() of the class Product to print the product details
     */
    TwoParameterInterface printProduct3 = (productId, name) -> new Product().printProductDe
    printProduct3.display(11, "Jackson");

    /*
     * Req2: Method Reference to invoke a instance method
     * printProductDetails() of the class Product to print the product details
     */
    TwoParameterInterface printProduct4 = new Product()::printProductDetails;
    printProduct4.display(11, "Jackson");

    /*
     * Req2: Lambda Expression to invoke a instance method
     * printCustomerDetails() of the class Customer to print the customer details
     */
    TwoParameterInterface printCustomer3 = (customerId, name) -> new Customer().printCustom
    printCustomer3.display(101, "Jackson");
}

```

```

    /*
    * Req2: Method Reference to invoke a instance method
    * printCustomerDetails() of the class Customer to print the customer details
    */
    TwoParameterInterface printCustomer4 = new Customer().printCustomerDetails;
    printCustomer4.display(101, "Jackson");
}

@SuppressWarnings("unused")
public static void main(String[] args) {

    req1();

    // req2();

}
}

```

File: Demo10002_MethodReference/com/accenture/ltt/mr/reducedmr/Tester1005LambdaToMRReductionRule

```

package com.accenture.ltt.mr.reducedmr;

import com.accenture.ltt.Customer;
import com.accenture.ltt.Product;
import com.accenture.ltt.a.interfaces.OneParameterCustomerObjectInterface;
import com.accenture.ltt.a.interfaces.OneParameterProductObjectInterface;
import com.accenture.ltt.a.interfaces.TwoParameterCustomerObjectInterface;
import com.accenture.ltt.a.interfaces.TwoParameterProductObjectInterface;

// Rule 2 Reduction: before arrow and after arrow signature are different and after arrow it in
// using the object passed as parameter
public class Tester1005LambdaToMRReductionRule2 {

    public static void req1() {
        // write the LE for the methods which take 1 parameter and does not return anything
        // Product
        OneParameterProductObjectInterface printProduct = (product, name) -> product.printNameUpper;
        printProduct.printNameInUpperCase(new Product(), "samsung");

        OneParameterProductObjectInterface printProduct2 = Product::printNameUpper;
        printProduct2.printNameInUpperCase(new Product(), "samsung");

        // Customer
        OneParameterCustomerObjectInterface printCustomer = (customer, name) -> customer.printNameUpper;
        printCustomer.printNameInUpperCase(new Customer(), "Jack");

        OneParameterCustomerObjectInterface printCustomer2 = Customer::printNameUpper;
        printCustomer2.printNameInUpperCase(new Customer(), "Jack");
    }

    public static void req2() {
        // write the LE for the methods which take 2 parameters and does not return anything
        // Product

```

```

TwoParameterProductObjectInterface printProduct3 = (product, productId, name) -> product
    .printProductDetails(productId, name);
printProduct3.display(new Product(), 11, "Jackson");

TwoParameterProductObjectInterface printProduct4 = Product::printProductDetails;
printProduct4.display(new Product(), 11, "Jackson");
// customer
TwoParameterCustomerObjectInterface printCustomer3 = (customer, customerId, name) -> customer
    .printCustomerDetails(customerId, name);
printCustomer3.display(new Customer(), 101, "Jackson");

TwoParameterCustomerObjectInterface printCustomer4 = Customer::printCustomerDetails;
printCustomer4.display(new Customer(), 101, "Jackson");
}

@SuppressWarnings("unused")
public static void main(String[] args) {

    req1();
//    req2();

}

}

/*
 * carrying the interface along with the application is not a good practice.
 * this is where standard functional interfaces are used.
 */

```

File: Demo10002_MethodReference/com/accenture/ltt/mr/standard/interfaces/Tester1006NeedOfMethodReferenceStatic

```

package com.accenture.ltt.mr.standard.interfaces;

import java.util.function.BiConsumer;
import java.util.function.Consumer;

import com.accenture.ltt.Customer;
import com.accenture.ltt.Product;

public class Tester1006NeedOfMethodReferenceStatic {

    // Need of MR for Static methods
    public static void req1() {
        // Product
        Consumer<String> printProduct = (name) -> Product.printNameUpperStatic(name);
        printProduct.accept("samsung");

        Consumer<String> printProduct2 = Product::printNameUpperStatic;
        printProduct2.accept("samsung");

        // Customer
    }
}

```

```

        Consumer<String> printCustomer = (name) -> Customer.printNameUpperStatic(name);
        printCustomer.accept("Jack");

        Consumer<String> printCustomer2 = Customer::printNameUpperStatic;
        printCustomer2.accept("Jack");
    }

    public static void req2() {
        //Product
        BiConsumer<Integer, String> printProduct3 = (productId, name) -> Product.displayDetails;
        printProduct3.accept(11, "LG");

        BiConsumer<Integer, String> printProduct4 = Product::displayDetails;
        printProduct4.accept(11, "LG");

        // customer
        BiConsumer<Integer, String> printCustomer3 = (customerId, name) -> Customer.displayDetails;
        printCustomer3.accept(101, "Jack");

        BiConsumer<Integer, String> printCustomer4 = Customer::displayDetails;
        printCustomer4.accept(101, "Jack");
    }

    @SuppressWarnings("unused")
    public static void main(String[] args) {

        req1();

        // req2();

    }
}

```

File: Demo10002_MethodReference/com/accnture/ltt/mr/standard/interfaces/Tester1007NeedOfMethodReferenceNonStatic

```

package com.accnture.ltt.mr.standard.interfaces;

import java.util.function.BiConsumer;
import java.util.function.Consumer;

import com.accnture.ltt.Customer;
import com.accnture.ltt.Product;

public class Tester1007NeedOfMethodReferenceNonStatic {

    // Need of MR for Instance methods
    public static void req1() {
        // write the LE for the methods which take 1 parameter and does not return anything
        // Product
        Consumer<String> printProduct = (name) -> new Product().printNameUpper(name);
        printProduct.accept("samsung");

        Consumer<String> printProduct2 = new Product()::printNameUpper;
    }
}

```

```

        printProduct2.accept("samsung");

        // Customer
        Consumer<String> printCustomer = (name) -> new Customer().printNameUpper(name);
        printCustomer.accept("Jack");

        Consumer<String> printCustomer2 = new Customer()::printNameUpper;
        printCustomer2.accept("Jack");
    }

    public static void req2() {
        // write the LE for the methods which take 2 parameters and does not return anything
        //Product
        BiConsumer<Integer, String> printProduct3 = (productId, name) -> new Product().printProductDetails;
        printProduct3.accept(11, "Jackson");

        BiConsumer<Integer, String> printProduct4 = new Product()::printProductDetails;
        printProduct4.accept(11, "Jackson");

        // customer
        BiConsumer<Integer, String> printCustomer3 = (customerId, name) -> new Customer().printCustomerDetails;
        printCustomer3.accept(101, "Jackson");

        BiConsumer<Integer, String> printCustomer4 = new Customer()::printCustomerDetails;
        printCustomer4.accept(101, "Jackson");
    }

    @SuppressWarnings("unused")
    public static void main(String[] args) {

        req1();

        // req2();

    }
}

```

File: Demo10002_MethodReference/com/accnture/ltt/mr/standard/interfaces/Tester1008NeedOfMethodReferenceConstructor.java

```

package com.accnture.ltt.mr.standard.interfaces;

import java.util.function.BiFunction;
import java.util.function.Supplier;

import com.accnture.ltt.Customer;
import com.accnture.ltt.Product;

public class Tester1008NeedOfMethodReferenceConstructor {

    // Need of MR for Constructor
    public static void req1() {
        // write the LE for the Default Constructor
        Supplier<Product> productObj = () -> new Product();
    }
}

```



```

        Product p = productObj.get();

        Supplier<Product> productObj2 = Product::new;
        Product p2 = productObj2.get();

        // Customer
        Supplier<Customer> customerObj = () -> new Customer();
        Customer c = customerObj.get();

        Supplier<Customer> customerObj2 = Customer::new;
        Customer c2 = customerObj2.get();
    }

    public static void req2() {
        // write the LE for the parameterized Constructor
        BiFunction<Integer, String, Product> productObj2 = (id, name) -> new Product(id, name);
        Product p2 = productObj2.apply(11, "Jackson");

        BiFunction<Integer, String, Product> productObj3 = Product::new;
        Product p3 = productObj3.apply(11, "Jackson");

        // Customer
        BiFunction<Integer, String, Customer> customerObj2 = (id, name) -> new Customer(id, name);
        Customer c2 = customerObj2.apply(101, "Jackson");

        BiFunction<Integer, String, Customer> customerObj3 = Customer::new;
        Customer c3 = customerObj3.apply(101, "Jackson");
    }

    @SuppressWarnings("unused")
    public static void main(String[] args) {

        req1();

        // req2();
    }
}

```

13.5.4 Stream Creation

File: Demo10003_Streams_StreamCreations/com/accenture/ltt/sampleclasses/Product.java

```

package com.accenture.ltt.sampleclasses;

import java.time.LocalDate;

public class Product {
    private Integer productId;
    private String productName;
    private String productCategory;
    private Double productPrice;
}

```

```

private LocalDate dateOfManufacture;

public Product(Integer productId, String productName, String productCategory, Double productPrice,
    LocalDate dateOfManufacture) {
    super();
    this.productId = productId;
    this.productName = productName;
    this.productCategory = productCategory;
    this.productPrice = productPrice;
    this.dateOfManufacture = dateOfManufacture;
}

public Integer getProductId() {
    return productId;
}

public void setProductId(Integer productId) {
    this.productId = productId;
}

public String getProductName() {
    return productName;
}

public void setProductName(String productName) {
    this.productName = productName;
}

public String getProductCategory() {
    return productCategory;
}

public void setProductCategory(String productCategory) {
    this.productCategory = productCategory;
}

public Double getProductPrice() {
    return productPrice;
}

public void setProductPrice(Double productPrice) {
    this.productPrice = productPrice;
}

public LocalDate getDateOfManufacture() {
    return dateOfManufacture;
}

public void setDateOfManufacture(LocalDate dateOfManufacture) {
    this.dateOfManufacture = dateOfManufacture;
}

public void printDetails() {

```

```

        System.out.println("ProductId= " + productId + "      " + "ProductName= " + productName +
            "      " + "ProductCategory= " + productCategory + "      " + "ProductPrice= " +
            "      " + "ManufacturingDate= " + dateOfManufacture);
    }

    @Override
    public String toString() {
        return "Product [productId=" + productId + ", productName=" + productName + ", productC
            + productCategory + ", productPrice=" + productPrice + ", dateOfManufacture=" +
    }

}

```

File: Demo10003_Streams_StreamCreations/com/accenture/ltt/sampleclasses/ProductUtility.java

```

package com.accenture.ltt.sampleclasses;

import java.time.LocalDate;
import java.util.ArrayList;
import java.util.List;

public class ProductUtility {
    public static List<Product> getProductList() {
        List<Product> products = new ArrayList<Product>();

        products.add(new Product(1, "HP Laptop", "N", 25000.0, LocalDate.of(2000, 2, 24)));
        products.add(new Product(2, "TV", "O", 30000.0, LocalDate.of(2021, 1, 1)));
        products.add(new Product(3, "Sony Handy Cam", "N", 28000.0, LocalDate.of(2020, 7, 7)));
        products.add(new Product(4, "Washing Machine", "O", 28000.0, LocalDate.of(2019, 11, 27)));
        products.add(new Product(5, "Ipod", "N", 90000.0, LocalDate.of(2000, 12, 12)));
        products.add(new Product(6, "IPhone", "N", 90000.0, LocalDate.of(2015, 6, 12)));
        products.add(new Product(7, "Ipad", "N", 25000.0, LocalDate.of(2015, 6, 12)));
        products.add(new Product(8, "A.C.", "O", 30000.0, LocalDate.of(2015, 6, 12)));
        products.add(new Product(9, "Fan", "N", 28000.0, LocalDate.now()));
        products.add(new Product(10, "Table", "O", 28000.0, LocalDate.now()));

        return products;
    }
}

```

File: Demo10003_Streams_StreamCreations/com/accenture/ltt/streamcreation/Tester1001ListCollection

```

package com.accenture.ltt.streamcreation;

import java.util.ArrayList;
import java.util.LinkedList;
import java.util.List;
import java.util.Vector;

import com.accenture.ltt.sampleclasses.Product;
import com.accenture.ltt.sampleclasses.ProductUtility;

/**
 * List creation

```

```

* @author deepali.shende
*
*/
public class Tester1001ListCollection {
    @SuppressWarnings("unused")
    public static void main(String[] args) {
        //ordered, duplicates are allowed
        List<Integer> productIds1 = new ArrayList<>();

        // ordered, duplicates are allowed. Thread safe
        List<Integer> productIds2 = new Vector<>();

        // ordered, duplicates are allowed. Doubly linkedlist
        List<Integer> productId3 = new LinkedList<>();
        productId3.add(101);
        productId3.add(102);
        productId3.add(103);
        productId3.add(104);
        productId3.add(101);
        productId3.forEach(productId -> System.out.println(productId));

        System.out.println("-----");

        List<Product> products = ProductUtility.getProductList();
        products.forEach(product -> System.out.println(product.getProductName()));
    }
}

```

File: Demo10003_Streams_StreamCreations/com/accenture/ltt/streamcreation/Tester1001Map.java

```

package com.accenture.ltt.streamcreation;

import java.time.LocalDate;
import java.util.HashMap;
import java.util.Map;
import java.util.TreeMap;

import com.accenture.ltt.sampleclasses.Product;

/**
 * Map creation
 * @author deepali.shende
 *
 */
public class Tester1001Map {
    public static void main(String[] args) {
        //unordered keys, unique keys
        //Map<Integer, String> products = new HashMap<Integer, String>();

        //ordered keys, unique keys
        //Map<Integer, String> products = new LinkedHashMap<Integer, String>();

        //sorted keys, unique keys
        Map<Integer, String> products = new TreeMap<Integer, String>();
    }
}

```

```

        products.put(1, "Fan");
        products.put(20, "Washing machine");
        products.put(3, "TV");

        products.forEach((id, name) -> System.out.println("ID = " + id + " Name = " + name));

        System.out.println("-----");

        // custom objects
        Map<Integer, Product> allProducts = new HashMap<Integer, Product>();
        allProducts.put(112233, new Product(1, "Fan", "N", 4000.0, LocalDate.now()));
        allProducts.put(445566, new Product(2, "TV", "N", 14000.0, LocalDate.now()));
        allProducts.put(778899, new Product(3, "Washing machine", "N", 24000.0, LocalDate.now()));
        allProducts.forEach( (orderNumber, product) ->
            System.out.println("Order no number = " + orderNumber + " -> Product"));
    }
}

```

File: Demo10003_Streams_StreamCreations/com/accenture/ltt/streamcreation/Tester1001SetCollection

```

package com.accenture.ltt.streamcreation;

import java.time.LocalDate;
import java.util.HashSet;
import java.util.LinkedHashSet;
import java.util.Set;
import java.util.TreeSet;

import com.accenture.ltt.sampleclasses.Product;

/**
 * Set creation
 * @author deepali.shende
 */
public class Tester1001SetCollection {
    public static void main(String[] args) {
        //unordered, unique elements
        Set<Integer> productIds1 = new HashSet<>();

        // ordered, unique elements
        Set<Integer> productIds2 = new LinkedHashSet<>();

        // sorted, unique elements
        Set<Integer> productIds3 = new TreeSet<>();
        productIds3.add(101);
        productIds3.add(106);
        productIds3.add(103);
        productIds3.add(102);
        productIds3.add(105);

        productIds3.forEach(productId -> System.out.println(productId));
    }
}

```

```

        System.out.println("-----");

        Set<Product> products = new HashSet<>();
        products.add(new Product(1, "Phone", "N", 20000.0, LocalDate.now()));
        products.add(new Product(2, "Fan", "N", 5000.0, LocalDate.now()));
        products.add(new Product(3, "Mouse", "N", 2000.0, LocalDate.now()));
        products.forEach(product -> System.out.println(product));
    }
}

```

File: Demo10003_Streams_StreamCreations/com/accenture/ltt/streamcreation/Tester1002StreamCreation

```

package com.accenture.ltt.streamcreation;

import java.util.Arrays;
import java.util.HashMap;
import java.util.HashSet;
import java.util.Iterator;
import java.util.List;
import java.util.Map;
import java.util.Map.Entry;
import java.util.Set;
import java.util.stream.Stream;

/**
 * Stream creation
 * @author deepali.shende
 *
 */

public class Tester1002StreamCreation {
    public static void main(String[] args) {
        // 1. creating a stream from a Collection implementation using the stream() method:
        createStreamfromCollection1();

        // 1.1. creating a stream from a Collection implementation using the stream() method:
        //createStreamfromCollection2();

        // 1.2. creating a stream from Map:
        //createStreamfromMap();

        // 2. creating a stream from individual values:
        //createStreamFromIndividualValues();

        // 3. creating a stream from an array using Stream.of():
        //createStreamFromArray1();

        // 4. creating a stream from an array using Arrays.stream():
        //createStreamFromArray2();

        // 5. creating a empty stream using Stream.empty():
        //createEmptyStream();

        // 6. creating a stream using Stream.Builder():
    }
}

```

```

        //createStreamUsingBuilder();

        // 7. creating a stream using Stream.iterate():
        //createStreamUsingIterator();

        // 8. creating a stream using Stream.generate()
        //createStreamUsingGenerate();
    }

//-----
// 1. creating a stream from a Collection implementation using the stream() method:
//-----
public static void createStreamfromCollection1() {
    List<String> productNames = Arrays.asList(new String[]{"TV", "Fan", "Mixer", "Washing m
    Stream<String> streamOfProductNames = productNames.stream();
    // forEach - is high order function/first class function. Consumer object is passed to
    streamOfProductNames.forEach(productName -> System.out.println(productName));
}

//-----
// 1.1. creating a stream from a Collection implementation using the stream() method:
//-----
public static void createStreamfromCollection2() {
    Set<Integer> productIds = new HashSet<>();
    productIds.add(112);
    productIds.add(113);
    productIds.add(114);
    productIds.add(115);
    // converting Set to Stream
    Stream<Integer> streamOfProductIds = productIds.stream();
    // displaying elements of Stream
    streamOfProductIds.forEach(productId -> System.out.println(productId));
}

//-----
// 1.2. creating a stream from Map:
//-----
public static void createStreamfromMap() {
    Map<Integer, String> products = new HashMap<Integer, String>();
    products.put(1, "TV");
    products.put(20, "Fan");
    products.put(3, "Mouse");

    Stream<Entry<Integer, String>> entrySet = products.entrySet().stream();
    entrySet.forEach(entry -> System.out.println(entry.getKey() + " -> " + entry.getValue()));
}

//-----
// 2. creating a stream from individual values:

```

```

//-----
public static void createStreamFromIndividualValues() {
    Stream<String> productNames = Stream.of("Laptop", "Camera", "TV", "Phone");
    productNames.forEach(product -> System.out.println(product));

    // Below line gives IllegalStateException as stream is already closed.
    //productNames.forEach(product -> System.out.println(product));
}

//-----
// 3. creating a stream from an array using Stream.of():
//-----
public static void createStreamFromArray1() {
    String[] productNames = new String[]{"Laptop", "Camera", "TV", "Phone"};
    Stream<String> streamOfProductNames = Stream.of(productNames);
    // access using iterator
    Iterator<String> it = streamOfProductNames.iterator();
    while (it.hasNext()) {
        System.out.print(it.next() + " ");
    }
}

//-----
// 4. creating a stream from an array using Arrays.stream():
//-----
public static void createStreamFromArray2() {
    String[] productNames = new String[]{"Laptop", "Camera", "TV", "Phone"};
    Stream<String> streamOfProductNames = Arrays.stream(productNames);
    streamOfProductNames.forEach(productName -> System.out.println(productName));
}

//-----
// 5. creating a empty stream using Stream.empty():
//-----
public static void createEmptyStream() {
    Stream<String> stream = Stream.empty();
    System.out.println(stream.count());    // 0
}

//-----
// 6. creating a stream using Stream.Builder():
//-----
public static void createStreamUsingBuilder() {
    Stream.Builder<String> productsBuilder = Stream.<String>builder().add("TV").add("Phone")
    productsBuilder.accept("IPhone");
    productsBuilder.accept("Headphone");
    Stream<String> allProducts = productsBuilder.build();
    System.out.println(allProducts.count());
}

```



```

//-----
// 7. creating a stream using Stream.iterate():
//-----
public static void createStreamUsingIterator() {
    // start - 1st value in stream
    // limit - no of elements in stream
    // n->n*n - lambda to generate next number in stream
    // infinite stream can be created without limit
    int start = 1;
    int limit = 10;
    Stream<Integer> generatedProductIds = Stream.iterate(start, n -> n+1)
                                                .limit(limit);
    generatedProductIds.forEach(System.out::println);
}

//-----
// 8. creating a stream using Stream.generate()
//-----
public static void createStreamUsingGenerate() {
    // infinite stream can be created without limit
    Stream.generate(Math::random)
        .limit(5)
        .forEach(System.out::println);
}
}

```

File: Demo10003_Streams_StreamCreations/com/accenture/ltt/streamcreation/Tester1003StreamCustomO

```

package com.accenture.ltt.streamcreation;

import java.util.List;
import java.util.stream.Stream;

import com.accenture.ltt.sampleclasses.Product;
import com.accenture.ltt.sampleclasses.ProductUtility;

/**
 * Stream creation of Custom objects
 * @author deepali.shende
 *
 */
public class Tester1003StreamCustomObjects {
    public static void main(String[] args) {
        List<Product> productList = ProductUtility.getProductList();
        Stream<Product> allProducts = productList.stream();
        allProducts.forEach(p -> p.printDetails());
    }
}

```

File: Demo10003_Streams_StreamCreations/com/accenture/ltt/streamcreation/Tester1004StreamPointTo

```

package com.accenture.ltt.streamcreation;

import java.util.stream.Stream;

/**
 * Points to remember
 * @author deepali.shende
 */
public class Tester1004StreamPointToRemember {

    public static void main(String[] args) {
        int[] productIds = {1, 2, 3, 4, 5};

        // When we create a stream from this array like this: Stream.of(nums), we are not creating a stream of
        // Integers (Stream<Integer>), but a stream of int arrays (Stream<int[]>). This means that the stream
        // having a stream with five elements, we have a stream of one element:
        System.out.println(Stream.of(productIds).count()); // It prints 1

        Integer[] productIds2 = {1, 2, 3, 4, 5};
        System.out.println(Stream.of(productIds2).count()); // It prints 5
    }
}

```

13.5.5 Stream Terminal Operations

File: Demo10004_Streams_TerminalOperations/com/accenture/ltt/sampleclasses/Customer.java

```

package com.accenture.ltt.sampleclasses;

public class Customer {
    private Integer customerId;
    private String customerName;
    private Integer age;

    public Customer(Integer customerId, String customerName, Integer age) {
        super();
        this.customerId = customerId;
        this.customerName = customerName;
        this.age = age;
    }

    public Integer getCustomerId() {
        return customerId;
    }

    public void setCustomerId(Integer customerId) {
        this.customerId = customerId;
    }

    public String getCustomerName() {
        return customerName;
    }
}

```

```

    public void setCustomerName(String customerName) {
        this.customerName = customerName;
    }

    public Integer getAge() {
        return age;
    }

    public void setAge(Integer age) {
        this.age = age;
    }

    @Override
    public String toString() {
        return "Customer [customerId=" + customerId + ", customerName=" + customerName + ", age=" + age + "]";
    }

    public void printDetails() {
        System.out.println("CustomerId= " + customerId + "      " + "CustomerName= " + customerName);
    }
}

```

File: Demo10004_Streams_Terminal0perations/com/accenture/ltt/sampleclasses/CustomerUtility.java

```

package com.accenture.ltt.sampleclasses;

import java.util.ArrayList;
import java.util.List;

public class CustomerUtility {
    public static List<Customer> getCustomerList() {
        List<Customer> customers = new ArrayList<>();

        customers.add(new Customer(1, "Sara", 45));
        customers.add(new Customer(2, "John", 23));
        customers.add(new Customer(3, "Tina", 56));
        customers.add(new Customer(4, "Jack", 54));
        customers.add(new Customer(12, "Jill", 34));
        customers.add(new Customer(5, "Soloman", 24));
        customers.add(new Customer(6, "Ziva", 56));
        customers.add(new Customer(7, "Rian", 32));
        customers.add(new Customer(8, "Rahyl", 74));
        customers.add(new Customer(9, "Adam", 27));
        customers.add(new Customer(10, "Eve", 63));

        return customers;
    }
}

```

File: Demo10004_Streams_Terminal0perations/com/accenture/ltt/sampleclasses/ProductUtility.java

```

package com.accenture.ltt.sampleclasses;

import java.time.LocalDate;

```

```

import java.util.ArrayList;
import java.util.List;

public class ProductUtility {
    public static List<Product> getProductList() {
        List<Product> products = new ArrayList<Product>();

        products.add(new Product(1, "HP Laptop", "N", 25000.0, LocalDate.of(2000, 2, 24)));
        products.add(new Product(2, "TV", "O", 30000.0, LocalDate.of(2021, 1, 1)));
        products.add(new Product(3, "Sony Handy Cam", "N", 28000.0, LocalDate.of(2020, 7, 7)));
        products.add(new Product(4, "Washing Machine", "O", 28000.0, LocalDate.of(2019, 11, 27)));
        products.add(new Product(5, "Ipod", "N", 90000.0, LocalDate.of(2000, 12, 12)));
        products.add(new Product(6, "IPhone", "N", 90000.0, LocalDate.of(2015, 6, 12)));
        products.add(new Product(7, "Ipad", "N", 25000.0, LocalDate.of(2015, 6, 12)));
        products.add(new Product(8, "A.C.", "O", 30000.0, LocalDate.of(2015, 6, 12)));
        products.add(new Product(9, "Fan", "N", 28000.0, LocalDate.now()));
        products.add(new Product(10, "Table", "O", 28000.0, LocalDate.now()));

        return products;
    }

    // get product prices
    public static List<Double> getProductPriceList() {
        List<Product> products = ProductUtility.getProductList();
        List<Double> productPrices = new ArrayList<>();

        for(int i=0; i<products.size(); i++) {
            productPrices.add(products.get(i).getProductPrice());
        }

        return productPrices;
    }
}

```

File: Demo10004_Streams_TerminalOperations/com/accenture/ltt/terminal/Tester1001ForEach.java

```

package com.accenture.ltt.terminal;

import java.util.Iterator;
import java.util.List;
import java.util.stream.Stream;

import com.accenture.ltt.sampleclasses.Product;
import com.accenture.ltt.sampleclasses.ProductUtility;

/**
 * forEach()
 * @author deepali.shende
 *
 */
public class Tester1001ForEach {
    public static void main(String[] args) {
        List<Product> products = ProductUtility.getProductList();
    }
}

```

```

        // makes code lengthy. More lines in byte code. Compilation takes time.
        //displayUsingIterator(products);

        // forEach method
        displayUsingForEachMethod(products);
    }

    //-----
    // Using iterator, makes code lengthy. More lines in byte code. Compilation takes time. More
    // Poor performance
    //-----
    public static void displayUsingIterator(List<Product> products) {
        Stream<Product> stream = products.stream();
        Iterator<Product> iterator = stream.iterator();
        while(iterator.hasNext()) {
            Product product = iterator.next();
            product.printDetails();
        }
    }

    //-----
    // Using forEach method
    //-----
    public static void displayUsingForEachMethod(List<Product> products) {
        // using Lambda expression
        Stream<Product> productStream1 = products.stream();
        productStream1.forEach(product -> product.printDetails());

        // Using method reference
        Stream<Product> productStream2 = products.stream();
        productStream2.forEach(System.out::println);
    }
}

```

File: Demo10004_Streams_Terminal0operations/com/accenture/ltt/terminal/Tester1002ToArray.java

```

package com.accenture.ltt.terminal;

import java.util.stream.Stream;

import com.accenture.ltt.sampleclasses.Product;
import com.accenture.ltt.sampleclasses.ProductUtility;

/**
 * Conversion of Stream to Array
 * @author deepali.shende
 *
 */
public class Tester1002ToArray {
    public static void main(String[] args) {
        Stream<Product> products1 = ProductUtility.getProductList().stream();
        // toArray converts to array of objects, not to specific type. Type casting would give
    }
}

```

```

Object[] allProducts = products1.toArray();
System.out.println("No of products = " + allProducts.length);

System.out.println("-----");

// A function passed to toArray() takes an integer, which is the size of the desired array
// and produces an array of the desired size.
Stream<Product> products2 = ProductUtility.getProductList().stream();
Product[] myProducts = products2.toArray(size -> new Product[size]);
System.out.println("No of products = " + myProducts.length);

System.out.println("-----");

//This can be expressed with an array constructor reference:
Stream<Product> products3 = ProductUtility.getProductList().stream();
Product[] myProducts1 = products3.toArray(Product[]::new);
System.out.println("No of products = " + myProducts1.length);
    }
}

```

File: Demo10004_Streams_TerminalOperations/com/accenture/ltt/terminal/Tester1003Collect.java

```

package com.accenture.ltt.terminal;

import java.util.ArrayDeque;
import java.util.Arrays;
import java.util.Deque;
import java.util.LinkedList;
import java.util.List;
import java.util.Set;
import java.util.Stack;
import java.util.stream.Collectors;
import java.util.stream.Stream;

import com.accenture.ltt.sampleclasses.Product;
import com.accenture.ltt.sampleclasses.ProductUtility;

/**
 * collect() of Stream
 * @author deepali.shende
 *
 */
public class Tester1003Collect {
    public static void main(String[] args) {
        //convertStreamToList();

        //convertStreamToSet();

        //convertStreamToCollection();

        //stringJoining();

        //streamCounting();
    }
}

```

```

        //streamCalculation1();

        streamCalculation2();

    }

    // -----
    // Convert stream to List.
    // -----
    private static void convertStreamToList() {
        Stream<Product> streamOfProducts = ProductUtility.getProductList().stream();
        List<Product> listOfProducts = streamOfProducts.collect(Collectors.toList());

        //Using lambda
        listOfProducts.forEach(product -> System.out.println(product));

        System.out.println("-----");

        //Using method reference
        listOfProducts.forEach(System.out::println);
    }

    // -----
    // Convert stream to Set.
    // -----
    private static void convertStreamToSet() {
        Stream<Integer> streamOfOrderIds = Stream.of(101, 201, 301, 401, 501);
        Set<Integer> setOfOrderIds = streamOfOrderIds.collect(Collectors.toSet());

        //Using lambda
        setOfOrderIds.forEach(orderId -> System.out.println(orderId));

        System.out.println("-----");

        //Using method reference
        setOfOrderIds.forEach(System.out::println);
    }

    // -----
    // Convert stream to different Collection objects.
    // -----
    private static void convertStreamToCollection() {
        LinkedList<Product> products = ProductUtility.getProductList().stream()
            .collect(Collectors.toCollection(LinkedList::new));
        products.forEach(System.out::println);

        System.out.println("-----");

        Stack<Product> newProducts = ProductUtility.getProductList().stream()
            .collect(Collectors.toCollection(Stack::new));
    }

```

```

newProducts.forEach(System.out::println);

System.out.println("-----");

Deque<Product> allProducts = ProductUtility.getProductList().stream()
    .collect(Collectors.toCollection(ArrayDeque::new));
allProducts.forEach(System.out::println);
}

// -----
// Join stream objects by some delimiter.
// -----
private static void stringJoining() {
    List<String> productNames = Arrays.asList("TV", "IPad", "IPhone", "Headphone");
    String allProductNames = productNames.stream().collect(Collectors.joining(", "));

    System.out.printf("Joined productNames: " + allProductNames);
}

// -----
// Count stream contents
// -----
private static void streamCounting() {
    long noOfProducts = ProductUtility.getProductList().stream().collect(Collectors.counting());
    System.out.println("No of products : " + noOfProducts);
}

// -----
// Calculate average of product prices
// -----
private static void streamCalculation1() {
    //Using lambda
    Stream<Product> streamOfProducts = ProductUtility.getProductList().stream();
    Double averagePriceOfTheProducts = streamOfProducts.collect(Collectors.averagingDouble());
    System.out.println("Average price of the products : Rs " + averagePriceOfTheProducts);

    System.out.println("-----");

    //Using method reference
    Stream<Product> streamOfProducts2 = ProductUtility.getProductList().stream();
    Double averagePriceOfTheProducts2 = streamOfProducts2.collect(Collectors.averagingDouble());
    System.out.println("Average price of the products : Rs " + averagePriceOfTheProducts2);
}

// -----
// Calculate sum of product prices.
// -----
private static void streamCalculation2() {
    //Using lambda
    Stream<Product> streamOfProducts = ProductUtility.getProductList().stream();

```



```

        Double totalProductPrice = streamOfProducts.collect(Collectors.summingDouble(product ->
        System.out.println("Total of the product's prices: Rs " + totalProductPrice));

        System.out.println("-----");

        //Using method reference
        Stream<Product> streamOfProducts1 = ProductUtility.getProductList().stream();
        Double totalProductPrice1 = streamOfProducts1.collect(Collectors.summingDouble(Product:
        System.out.println("Total of the product's prices: Rs " + totalProductPrice1);
    }
}

```

File: Demo10004_Streams_TerminalOperations/com/accenture/ltt/terminal/Tester1004MaxMinCount.java

```

package com.accenture.ltt.terminal;

import java.util.Comparator;
import java.util.List;
import java.util.Optional;
import java.util.stream.Collectors;

import com.accenture.ltt.sampleclasses.Product;
import com.accenture.ltt.sampleclasses.ProductUtility;

/**
 * aggregate functions in Stream
 * @author deepali.shende
 *
 */
public class Tester1004MaxMinCount {
    public static void main(String[] args) {
        //getMinProductPrice();

        //getMaxProductPrice();

        //getProductHavingMaxPrice();

        //getOldestProduct();

        getTotalProducts();
    }

    //-----
    // Get the min product price. Introduce Comparator in brief.
    //-----
    private static void getMinProductPrice() {
        List<Double> productPrices = ProductUtility.getProductPriceList();
        Double minPrice =
            productPrices.stream()
                .min(
                    (price1, price2) ->
                    {
                        int result = 0;
                        if(price1 > price2) {

```

```

        result = 1;
    }
    else if(price1 < price2) {
        result = -1;
    }
    return result;
}
)
)
    .orElse(0.0);
System.out.println("Min product price = " + minPrice);
}

//-----
// Get the max product price. Introduce Comparator in brief.
//-----
private static void getMaxProductPrice() {
    List<Double> productPrices = ProductUtility.getProductPriceList();
    Double maxPrice =
        productPrices.stream()
            .max(
                (price1, price2) ->
                {
                    int result = 0;
                    if(price1 > price2) {
                        result = 1;
                    }
                    else if(price1 < price2) {
                        result = -1;
                    }
                    return result;
                }
            )
            .orElse(0.0);
    System.out.println("Max product price = " + maxPrice);

    System.out.println("-----");

    Double maxPrice1 =
        productPrices.stream()
            .collect(Collectors.maxBy((price1, price2) ->
                {
                    int result = 0;
                    if(price1 > price2) {
                        result = 1;
                    }
                    else if(price1 < price2) {
                        result = -1;
                    }
                    return result;
                }
            )))
            .orElse(0.0);
    System.out.println("Max product price = " + maxPrice1);
}

```

```

//-----
// Get the product having maximum price. comparing method will be discussed in details with
//-----
private static void getProductHavingMaxPrice() {
    // Using Lambda
    Optional<Product> expensiveProduct = ProductUtility.getProductList().stream()
        .max(Comparator.comparing(product -> product.getProductPrice()));
    System.out.println("Max price of product : " + expensiveProduct.get());

    System.out.println("-----");

    // Using method reference
    Optional<Product> expensiveProduct1 = ProductUtility.getProductList().stream()
        .max(Comparator.comparing(Product::getProductPrice));

    System.out.println("Max price of product : " + expensiveProduct1.get());
}

//-----
// Get the oldest product. comparing method will be discussed in details with sorting.
//-----
private static void getOldestProduct() {
    // Using Lambda
    Optional<Product> oldProduct = ProductUtility.getProductList().stream()
        .min(Comparator.comparing(product -> product.getDateOfManufacture()));
    System.out.println("Old product : " + oldProduct.get());

    System.out.println("-----");

    // Using method reference
    Optional<Product> oldProduct1 = ProductUtility.getProductList().stream()
        .min(Comparator.comparing(Product::getDateOfManufacture));
    System.out.println("Old product : " + oldProduct1.get());
}

//-----
// Get total product count.
//-----
private static void getTotalProducts() {
    long totalProducts = ProductUtility.getProductList().stream().count();
    System.out.println("No of products : " + totalProducts);
}
}

```

File: Demo10004_Streams_TerminalOperations/com/accenture/ltt/terminal/Tester1005Summarizing.java

```
package com.accenture.ltt.terminal;
```

```
import java.util.DoubleSummaryStatistics;
import java.util.IntSummaryStatistics;
import java.util.stream.Collectors;
```

```

import java.util.stream.Stream;

import com.accenture.ltt.sampleclasses.Customer;
import com.accenture.ltt.sampleclasses.CustomerUtility;
import com.accenture.ltt.sampleclasses.Product;
import com.accenture.ltt.sampleclasses.ProductUtility;

/**
 * Summarizing info in Stream
 * @author deepali.shende
 *
 */
public class Tester1005Summarizing {
    public static void main(String[] args) {
        //summarizingInt();

        summarizingDouble();
    }

    //-----
    // Getting summary functions like min, max, average, count on customer age
    //-----
    private static void summarizingInt() {
        //Using lambda
        Stream<Customer> customers = CustomerUtility.getCustomerList().stream();
        IntSummaryStatistics intSummaryStatistics = customers.collect(Collectors.summarizingInt());
        System.out.println(intSummaryStatistics);

        System.out.println("-----");

        //Using method reference
        IntSummaryStatistics intSummaryStatistics1 = CustomerUtility.getCustomerList().stream().collect(Collectors.summarizingInt());
        System.out.println(intSummaryStatistics1);
    }

    //-----
    // Getting summary functions like min, max, average, count on product price
    //-----
    private static void summarizingDouble() {
        //Using lambda
        Stream<Product> products = ProductUtility.getProductList().stream();
        DoubleSummaryStatistics doubleSummaryStatistics = products.collect(Collectors.summarizingDouble());
        System.out.println(doubleSummaryStatistics);

        System.out.println("-----");

        //Using method reference
        DoubleSummaryStatistics doubleSummaryStatistics1 = ProductUtility.getProductList().stream().collect(Collectors.summarizingDouble());
        System.out.println(doubleSummaryStatistics1);
    }
}

```

13.5.6 Stream Non-Terminal (Intermediate) Operations

File: Demo10005_Streams_NonTerminalOperations/com/accenture/ltt/nonterminal/Tester1001Filter.java

```
package com.accenture.ltt.nonterminal;

import java.util.Set;
import java.util.stream.Collectors;

import com.accenture.ltt.sampleclasses.Product;
import com.accenture.ltt.sampleclasses.ProductUtility;

/**
 * Stream Filter
 * @author deepali.shende
 */
public class Tester1001Filter {
    public static void main(String[] args) {
        //Req 1 : get the products having price more than 30000
        req1();

        //Req 2 : get the products starting with I
        //req2();

        //Req 3 : get the products starting with I and price greater than 30000
        //req3();

        //Req 4 : get the products having manufacturing date before 2020
        //req4();

        //Req 5 : get the products having product category as 'N' in a Set
        //req5();
    }

    //-----
    // Req 1 : get the products having price greater than 30000
    //-----
    private static void req1() {
        // using lambda
        ProductUtility.getProductList()
            .stream()
            .filter(product -> product.getProductPrice() > 30000)
            .forEach(product -> System.out.println(product));

        System.out.println("-----");

        // using method reference
        ProductUtility.getProductList()
            .stream()
            .filter(Tester1001Filter::checkProductPrice)
            .forEach(product -> System.out.println(product));
    }

    public static boolean checkProductPrice(Product product) {
```

```

        return product.getProductPrice() > 30000;
    }

    // -----
    // Get the products starting with I
    // -----
    private static void req2() {
        // Using Lambda
        ProductUtility.getProductList()
            .stream()
            .filter(product -> product.getProductName().startsWith("I"))
            .forEach(product -> System.out.println(product));

        System.out.println("-----");

        // Using method reference
        ProductUtility.getProductList()
            .stream()
            .filter(Tester1001Filter::checkProductName)
            .forEach(System.out::println);
    }

    public static boolean checkProductName(Product product) {
        return product.getProductName().startsWith("I");
    }

    // -----
    // Get the products starting with I and price greater than 30000
    // -----
    private static void req3() {
        // Using Lambda
        ProductUtility.getProductList()
            .stream()
            .filter(product -> product.getProductName().startsWith("I") && product.ge

            .forEach(product -> System.out.println(product));

        System.out.println("-----");

        // Using Method reference
        ProductUtility.getProductList()
            .stream()
            .filter(Tester1001Filter::checkProductNameAndPrice)
            .forEach(System.out::println);

        System.out.println("-----");

        // Nesting of filter using Lambda
        ProductUtility.getProductList()
            .stream()
            .filter(product -> Tester1001Filter.checkProductName(product))
            .filter(product -> product.getProductPrice() > 30000)
            .forEach(product -> System.out.println(product));
    }

```

```

        System.out.println("-----");

        // Nesting of filter using method reference
        ProductUtility.getProductList()
            .stream()
            .filter(Tester1001Filter::checkProductName)
            .filter(Tester1001Filter::checkProductPrice)
            .forEach(System.out::println);
    }

    public static boolean checkProductNameAndPrice(Product product) {
        return product.getName().startsWith("I") && product.getPrice() > 30000;
    }

    // -----
    // Get the products having manufacturing date before 2020
    // -----
    private static void req4() {
        // Using Lambda
        ProductUtility.getProductList()
            .stream()
            .filter(product -> product.getDateOfManufacture().getYear() < 2020)
            .forEach(product -> System.out.println(product));

        System.out.println("-----");

        // Using method reference
        ProductUtility.getProductList()
            .stream()
            .filter(Tester1001Filter::checkProductManufacturingYear)
            .forEach(System.out::println);
    }

    public static boolean checkProductManufacturingYear(Product product) {
        return product.getDateOfManufacture().getYear() < 2020;
    }

    // -----
    // Get the products having product category as 'N' in a Set
    // -----
    private static void req5() {
        // Using Lambda
        Set<Product> products1 =
            ProductUtility.getProductList()
                .stream()
                .filter(product -> product.getProductCategory().equals("N"))
                .collect(Collectors.toSet());
        products1.forEach(product -> System.out.println(product));

        System.out.println("-----");
    }

```

```

        // Using method reference
        Set<Product> products2 =
            ProductUtility.getProductList()
                .stream()
                .filter(Tester1001Filter::filterCategories)
                .collect(Collectors.toSet());
        products2.forEach(System.out::println);
    }

    public static boolean filterCategories(Product product) {
        return product.getProductCategory().equals("N");
    }
}

```

File: Demo10005_Streams_NonTerminalOperations/com/accenture/ltt/nonterminal/Tester1002Map.java

```

package com.accenture.ltt.nonterminal;

import java.time.LocalDate;
import java.util.List;
import java.util.stream.Collectors;
import java.util.stream.DoubleStream;
import java.util.stream.Stream;

import com.accenture.ltt.sampleclasses.Product;
import com.accenture.ltt.sampleclasses.ProductUtility;

/**
 * map() of Stream
 * @author deepali.shende
 *
 */
public class Tester1002Map {
    public static void main(String[] args) {
        // Req 1 : get list of product prices
        //req1();

        // Req 2 : get the product names having price 90000
        // Sequence of filter and map can not be changed as both filter and map work on different streams
        //req2();

        // Req 3 : get the prices of all products having price >= 30000
        // Sequence of filter and map can be changed as both filter and map work on product price
        //req3();

        // Req 4 : get max price
        //req4();

        // Req 5 : get the list of years of the product manufacturing dates
        req5();
    }

    //-----

```



```

// Get list of product prices
//-----
private static void req1() {
    Stream<Product> products = ProductUtility.getProductList().stream();
    // using lambda
    List<Double> productPrices1 =
        products.map(product -> product.getProductPrice())
                .collect(Collectors.toList());
    productPrices1.forEach(price -> System.out.println(price));

    System.out.println("-----");

    // using method reference
    List<Double> productPrices2 = ProductUtility.getProductList()
                                .stream()
                                .map(Product::getProductPrice)
                                .collect(Collectors.toList());
    productPrices2.forEach(System.out::println);

    System.out.println("-----");

    // using mapToDouble
    DoubleStream prices = ProductUtility.getProductList()
                        .stream()
                        .mapToDouble(Product::getProductPrice);
    prices.forEach(System.out::println);
}

//-----
// Get the product names having price 90000.
// Sequence of filter and map can not be changed as both filter and map work on different t
//-----
private static void req2() {
    // Using Lambda
    ProductUtility.getProductList()
                .stream()
                .filter(product -> product.getProductPrice() == 90000)
                .map(product -> product.getName())
                .forEach(name -> System.out.println(name));

    System.out.println("-----");

    // Using method reference
    ProductUtility.getProductList()
                .stream()
                .filter(Tester1002Map::checkProductForPrice1)
                .map(Product::getName)
                .forEach(System.out::println);
}

public static boolean checkProductForPrice1(Product product) {
    return product.getProductPrice() == 90000;
}

```

```

}

//-----
// Get the prices of all products having price >= 30000.
// Sequence of filter and map can be changed as both filter and map work on product prices.
//-----
private static void req3() {
    // Using Lambda
    List<Double> productPrices1 =
        ProductUtility.getProductList()
            .stream()
            .filter(product -> product.getProductPrice() >= 30000)
            .map(product -> product.getProductPrice())
            .collect(Collectors.toList());
    productPrices1.forEach(price -> System.out.println(price));

    System.out.println("-----");

    // Using Lambda. Sequence of filter() and map() is changed
    List<Double> productPrices2 =
        ProductUtility.getProductList()
            .stream()
            .map(product -> product.getProductPrice())
            .filter(price -> price >= 30000)
            .collect(Collectors.toList());
    productPrices2.forEach(price -> System.out.println(price));

    System.out.println("-----");

    // Using method reference
    List<Double> productPrices3 =
        ProductUtility.getProductList()
            .stream()
            .filter(Tester1002Map::checkProductForPrice2)
            .map(Product::getProductPrice)
            .collect(Collectors.toList());
    productPrices3.forEach(System.out::println);
}

public static boolean checkProductForPrice2(Product product) {
    return product.getProductPrice() >= 30000;
}

//-----
// Get max price
//-----
private static void req4() {
    // First
    double maxPrice1 =
        ProductUtility.getProductList()
            .stream()

```

```

        .map(Product::getProductPrice)
        .max((price1, price2) -> {
            int res = 0;
            if(price1 < price2) {
                res = -1;
            } else {
                res = 1;
            }
            return res;
        })
        .orElse(0.0);
System.out.println("Maximum: " + maxPrice1);

System.out.println("-----");

// second
double maxPrice2 =
    ProductUtility.getProductList()
        .stream()
        .map(Product::getProductPrice)
        .collect(Collectors.maxBy((price1, price2) -> {
            int res = 0;
            if(price1 < price2) {
                res = -1;
            } else {
                res = 1;
            }
            return res;
        })))
        .orElse(0.0);
System.out.println("Maximum: " + maxPrice2);

System.out.println("-----");

// third
double maxPrice3 =
    ProductUtility.getProductList()
        .stream()
        .mapToDouble(Product::getProductPrice)
        .max()
        .orElse(0.0);
System.out.println("Maximum : " + maxPrice3);
}

//-----
// Get the list of years from the product manufacturing dates.
//-----
private static void req5() {
    // Using lambda
    Stream<Integer> yearList1 =
        ProductUtility.getProductList()
            .stream()

```

```

        .map(product -> product.getDateOfManufacture())
        .map(localDate -> localDate.getYear());
yearList1.forEach(year -> System.out.println(year));

System.out.println("-----");

// Using method reference
Stream<Integer> yearList2 =
    ProductUtility.getProductList()
        .stream()
        .map(Product::getDateOfManufacture)
        .map(LocalDate::getYear);
yearList2.forEach(System.out::println);
    }
}

```

File: Demo10005_Streams_NonTerminalOperations/com/accenture/ltt/nonterminal/Tester1003Peek.java

```

package com.accenture.ltt.nonterminal;

import java.util.stream.Collectors;

import com.accenture.ltt.sampleclasses.Product;
import com.accenture.ltt.sampleclasses.ProductUtility;

/**
 * peek() of Stream
 * @author deepali.shende
 *
 */
public class Tester1003Peek {
    public static void main(String[] args) {
        // using lambda
        ProductUtility.getProductList()
            .stream()
            .filter(product -> product.getProductName().length() > 5)
            .peek(product -> System.out.println("Filtered value: " + product.getProductName()))
            .map(product -> product.getProductName().toUpperCase())
            .peek(productName -> System.out.println("Mapped value: " + productName))
            .collect(Collectors.toList());

        System.out.println("-----");

        // using method reference
        ProductUtility.getProductList()
            .stream()
            .filter(Tester1003Peek::checkProductNameLength)
            .peek(Tester1003Peek::printProductOnNameLength)
            .map(Tester1003Peek::convertProductNameToUpperCase)
            .peek(Tester1003Peek::printProductNamesInUpperCase)
            .collect(Collectors.toList());
    }

    public static boolean checkProductNameLength(Product product) {

```

```

        return product.getProductName().length() > 5;
    }

    public static void printProductOnNameLength(Product product) {
        System.out.println("Filtered value: " + product.getProductName());
    }

    public static String convertProductNameToUpperCase(Product product) {
        return product.getProductName().toUpperCase();
    }

    public static void printProductNamesInUpperCase(String productName) {
        System.out.println("Mapped value: " + productName);
    }
}

/**
 * Instead of peek() in above example, we could have use forEach() also. As as forEach() is ter
 * we can not have any operation further.
 */

```

File: Demo10005_Streams_NonTerminalOperations/com/accenture/ltt/nonterminal/Tester1004Distinct.j

```

package com.accenture.ltt.nonterminal;

import java.util.List;
import java.util.stream.Collectors;

import com.accenture.ltt.sampleclasses.Customer;
import com.accenture.ltt.sampleclasses.CustomerUtility;
import com.accenture.ltt.sampleclasses.Product;
import com.accenture.ltt.sampleclasses.ProductUtility;

/**
 * distinct() of Stream
 * @author deepali.shende
 */
public class Tester1004Distinct {
    public static void main(String[] args) {
        // Req 1 : get distinct product categories
        //req1();

        // Req 2 : Get no of distinct customer names
        req2();
    }

    //-----
    // Get distinct product categories
    //-----
    private static void req1() {
        // using lambda
        List<String> distinctProductTypes1 =
            ProductUtility.getProductList()

```

```

        .stream()
        .map(product -> product.getProductCategory())
        .distinct()
        .collect(Collectors.toList());
distinctProductTypes1.forEach(category -> System.out.println(category));

System.out.println("-----");

// using method reference
List<String> distinctProductTypes2 =
    ProductUtility.getProductList()
        .stream()
        .map(Product::getProductCategory)
        .distinct()
        .collect(Collectors.toList());
distinctProductTypes2.forEach(System.out::println);
}

//-----
// Get no of distinct customer names.
//-----
private static void req2() {
    // using lambda
    long count1 =
        CustomerUtility.getCustomerList()
            .stream()
            .map(customer -> customer.getCustomerName())
            .distinct()
            .count();

    System.out.println("No of distinct customer namers = " + count1);

    // using method reference
    long count2 =
        CustomerUtility.getCustomerList()
            .stream()
            .map(Customer::getCustomerName)
            .distinct()
            .count();

    System.out.println("No of distinct customer namers = " + count2);
}
}

```

File: Demo10005_Streams_NonTerminalOperations/com/accenture/ltt/nonterminal/Tester1005Limit.java

```

package com.accenture.ltt.nonterminal;

import java.util.List;
import java.util.stream.Collectors;

import com.accenture.ltt.sampleclasses.Product;
import com.accenture.ltt.sampleclasses.ProductUtility;

/**

```

```

* limit() of Stream
* @author deepali.shende
*
*/
public class Tester1005Limit {
    public static void main(String[] args) {
        // Req 1 : get first 5 products
        req1();
    }

    //-----
    // Get first 5 products
    // Note: Passing negative value results in IllegalArgumentException.
    //-----
    private static void req1() {
        List<Product> products =
            ProductUtility.getProductList()
                .stream()
                .limit(5)
                .collect(Collectors.toList());
        products.forEach(System.out::println);
    }
}

```

File: Demo10005_Streams_NonTerminalOperations/com/accenture/ltt/nonterminal/Tester1006Skip.java

```

package com.accenture.ltt.nonterminal;

import java.util.List;
import java.util.stream.Collectors;

import com.accenture.ltt.sampleclasses.Customer;
import com.accenture.ltt.sampleclasses.CustomerUtility;

/**
 * skip() of Stream
 * @author deepali.shende
 *
 */
public class Tester1006Skip {
    public static void main(String[] args) {
        // Req 1 : skip first 3 Customers
        req1();
    }

    //-----
    // Skip first 3 and get the list of remaining Customers
    // Note: Passing negative value results in IllegalArgumentException
    //-----
    private static void req1() {
        List<Customer> customers =
            CustomerUtility.getCustomerList()

```

```

        .stream()
        .skip(3)
        .collect(Collectors.toList());
    customers.forEach(System.out::println);
}
}

```

File: Demo10005_Streams_NonTerminalOperations/com/accenture/ltt/sampleclasses/CustomerUtility.java

```

package com.accenture.ltt.sampleclasses;

import java.util.ArrayList;
import java.util.List;

public class CustomerUtility {
    public static List<Customer> getCustomerList() {
        List<Customer> customers = new ArrayList<>();

        customers.add(new Customer(1, "Sara", 45));
        customers.add(new Customer(2, "John", 23));
        customers.add(new Customer(3, "Rian", 56));
        customers.add(new Customer(4, "Jack", 54));
        customers.add(new Customer(12, "Jill", 34));
        customers.add(new Customer(5, "Soloman", 24));
        customers.add(new Customer(6, "Ziva", 56));
        customers.add(new Customer(7, "Rian", 32));
        customers.add(new Customer(8, "Sara", 74));
        customers.add(new Customer(9, "Adam", 27));
        customers.add(new Customer(10, "Eve", 63));

        return customers;
    }
}

```

File: Demo10005_Streams_NonTerminalOperations/com/accenture/ltt/sampleclasses/ProductUtility.java

```

package com.accenture.ltt.sampleclasses;

import java.time.LocalDate;
import java.util.ArrayList;
import java.util.List;

public class ProductUtility {
    public static List<Product> getProductList() {
        List<Product> products = new ArrayList<Product>();

        products.add(new Product(1, "HP Laptop", "N", 25000.0, LocalDate.of(2000, 2, 24)));
        products.add(new Product(2, "TV", "O", 30000.0, LocalDate.of(2021, 1, 1)));
        products.add(new Product(3, "Sony Handy Cam", "N", 28000.0, LocalDate.of(2020, 7, 7)));
        products.add(new Product(4, "Washing Machine", "O", 28000.0, LocalDate.of(2019, 11, 27)));
        products.add(new Product(5, "Ipod", "N", 90000.0, LocalDate.of(2000, 12, 12)));
        products.add(new Product(6, "IPhone", "N", 90000.0, LocalDate.of(2015, 6, 12)));
        products.add(new Product(7, "Ipad", "N", 25000.0, LocalDate.of(2015, 6, 12)));
        products.add(new Product(8, "A.C.", "O", 30000.0, LocalDate.of(2015, 6, 12)));
        products.add(new Product(9, "Fan", "N", 28000.0, LocalDate.now()));
    }
}

```



```

        products.add(new Product(10, "Table", "0", 28000.0, LocalDate.now()));

        return products;
    }
}

```

13.6 Viva Voce — Likely Questions & Answers

Q1. What is a functional interface? How does Java enforce that it has only one abstract method? A: An interface with exactly one abstract method (it may still have any number of default/ static methods, since those aren't abstract). The `@FunctionalInterface` annotation isn't mandatory, but if present, the compiler will raise an error the moment a second abstract method is added — a safety net against accidental design mistakes.

Q2. Why were default methods introduced in Java 8? A: To allow interfaces to evolve — adding a new method with a default body to an existing interface doesn't break any classes that already implement it, since they inherit the default implementation automatically instead of failing to compile from a missing method.

Q3. What are the four kinds of method references, with an example each? A: (1) Static: `ClassName::staticMethod`. (2) Instance method of a particular object: `instance::method`. (3) Instance method of an arbitrary object of a type: `ClassName::instanceMethod`. (4) Constructor reference: `ClassName::new`.

Q4. What does “effectively final” mean, and why must a lambda-captured local variable be effectively final? A: It means a local variable is never reassigned after its initial assignment, even though it isn't explicitly declared `final`. Lambdas capturing local variables require this because the lambda might execute later (even on a different thread), and Java needs a guaranteed-stable value at that point rather than a reference to a variable that could have since changed.

Q5. What is the difference between intermediate and terminal stream operations? A: Intermediate operations (`filter`, `map`, `sorted`, ...) return another `Stream` and can be chained; they don't execute anything by themselves. Terminal operations (`collect`, `forEach`, `count`, ...) trigger the entire pipeline to actually run and produce a final result.

Q6. Why are streams described as “lazy”, and what's the benefit? A: Intermediate operations are only descriptions of work to do — nothing executes until a terminal operation is called. This lets the JVM optimize the whole pipeline as a unit (e.g., short-circuiting early via `limit()`), avoiding materializing intermediate collections at every step.

Q7. Can you reuse the same `Stream` object for two different terminal operations? A: No — a stream can only be consumed once; invoking a second terminal operation on an already-consumed stream throws `IllegalStateException`. You must create a fresh stream from the source collection for each new pipeline.

Q8. Give an example of choosing the right standard functional interface for a scenario: “check if a number is even”. A: `Predicate<Integer>`, since it takes one input and returns a boolean: `Predicate<Integer> isEven = n -> n % 2 == 0;`.

Q9. What is the difference between `map()` and `filter()` in a stream pipeline? A: `filter()` takes a `Predicate` and keeps only elements matching a condition (same type in, same type out, possibly fewer elements). `map()` takes a `Function` and *transforms* each element into a new value (same number of elements, but potentially a different type).

Q10. Why can't Java have "true" multiple inheritance of implementation via interfaces even after default methods were added? A: Because if a class implements two interfaces with conflicting default methods of the same signature, Java forces the implementing class to explicitly resolve the conflict itself (overriding the method, optionally delegating via `InterfaceName.super.method()`) — it never silently picks a winner, avoiding the ambiguity that true multiple implementation inheritance would risk.

Chapter 14: Java 11 Features

Java 11 (2018) was the second Long-Term-Support (LTS) release after Java 8, consolidating several smaller JEPs (JDK Enhancement Proposals) delivered incrementally in the interim Java 9/10 releases, plus its own additions. Because many enterprises skip non-LTS releases entirely, Java 11 is where most teams’ “next” upgrade after Java 8 actually lands.

14.1 New String Methods

Java 11 added several convenience methods directly on `String`:

- **`isBlank()`** — returns `true` if the string is empty **or** contains only whitespace characters. (Contrast with `isEmpty()`, which *only* checks for zero length — a string of just spaces is `isEmpty() == false` but `isBlank() == true`, a frequently-tested distinction.)
- **`strip()` / `stripLeading()` / `stripTrailing()`** — Unicode-aware whitespace trimming, an improvement over the much older `trim()` method, which only correctly understood ASCII whitespace and could mishandle certain Unicode space characters.
- **`repeat(n)`** — returns a new string consisting of the original repeated `n` times concatenated together (e.g., `"ab".repeat(3) → "ababab"`).
- **`lines()`** — splits a multi-line string into a `Stream<String>`, one element per line, letting you process line-by-line using the Stream API from Chapter 13 rather than manually splitting on `\n`.

14.2 File Utility Enhancements

`java.nio.file.Files` gained simple one-liner helpers — `Files.writeString(path, content)` and `Files.readString(path)` — removing the need for manual `BufferedReader/BufferedWriter` boilerplate for the very common case of reading/writing an entire small text file in one shot.

14.3 The `var` Keyword (JEP 286, Java 10) and Local-Variable Type Inference in Lambdas (JEP 323)

`var` lets the compiler **infer** a local variable’s type from its initializer expression, reducing boilerplate while remaining **fully statically typed** — this is a critical distinction from dynamically-typed languages: the type is still fixed permanently at compile time based on the initializer, it is simply not *written out explicitly* by the developer:

```
var list = new ArrayList<String>(); // compiler infers the type as ArrayList<String>, fixed f
```

Java 11 extended this inference capability to lambda parameters: `(var x, var y) -> x + y`, whose main practical benefit is that it becomes possible to attach an annotation to an inferred lambda parameter (which bare, un-typed lambda parameters like `(x, y) -> ...` cannot support syntactically).

A common viva trap: `var` does **not** make Java dynamically typed — you cannot later assign a value of a different, incompatible type to a `var`-declared variable; the compiler still enforces the type it inferred at declaration time, exactly as if you had written that type explicitly.

14.4 `Predicate.not()` (a JEP-adjacent convenience addition)

`Predicate.not(somePredicate)` provides a clean, readable way to negate an existing predicate or method reference without manually wrapping it in a lambda — compare `x -> !somePredicate.test(x)` (verbose, easy to get subtly wrong with double-negatives) against `Predicate.not(somePredicate)` (reads naturally), which is especially valuable with method references like `Predicate.not(String::isBlank)` — writing `!(String::isBlank)` isn’t even

valid syntax, since you can't apply `!` directly to a method reference, which is precisely the gap `Predicate.not()` fills.

14.5 HTTP Client Standardization (JEP 321)

Java 11 promoted the previously *incubating* (experimental, Java 9+) `java.net.http.HttpClient` to a fully standard, stable API, providing a modern, fluent, builder-style way to make HTTP requests (supporting both HTTP/1.1 and HTTP/2, both synchronous and asynchronous calling styles) — effectively replacing the old, notoriously clunky and low-level `URLConnection` API that had been Java's only built-in HTTP client option for over two decades.

14.6 Epsilon Garbage Collector (JEP 318)

Epsilon is a deliberately **no-op** garbage collector — it happily allocates memory as requested but **never reclaims any of it**. Its purpose is emphatically **not** production use (any long-running application would eventually exhaust its heap and crash), but rather **performance testing and extremely short-lived jobs** where you can *guarantee* the heap will never fill up before the JVM exits anyway — letting engineers measure an application's raw *allocation pressure/rate* in complete isolation from any garbage-collection overhead, useful for micro-benchmarking memory behaviour.

14.7 Duration and Date/Time API Additions

`java.time.Duration` represents a **time-based** amount (hours/minutes/seconds/nanoseconds), complementing `java.time.Period` (a **date-based** amount — years/months/days) that was already part of the Java 8 `java.time` package. Additional Java 9-era Collection factory methods (`List.of()`, `Set.of()`, `Map.of()` — producing convenient, genuinely **immutable** collection literals in one line, throwing `UnsupportedOperationException` if any modification is attempted) and Optional enhancements (`ifPresentOrElse`, `or`, `stream()`) rounded out this era of releases and are included in the demo set below (bundled under the trainer's "Java Nine" folder, since Java 9's features are commonly taught alongside 11, as they collectively represent the post-Java-8, pre-Java-17 "in-between" releases most teams encounter together during an upgrade).

The demo files below are grouped as: new String functions, file utilities, `Predicate.not()`, var/local-variable inference, and the bundled Java-9-era feature set (collection factories, Optional enhancements, `takeWhile/dropWhile`, and date-range utilities).

14.7.1 New String Functions

File: `Demo_1001_String_Functions/com/accenture/ltt/zeleven/Tester1001.java`

```
package com.accenture.ltt.zeleven;

import java.util.Arrays;
import java.util.stream.Collectors;

public class Tester1001 {
    public static void main(String[] args) {
        var v = Arrays.asList("James", "Dan", "Eric", "Stuart", "Mario", "Cynthia", "Lucy");
        var list = v.stream().filter(name -> name.contains("a")).collect(Collectors.toList());
        list.forEach(System.out::print);
    }
}
```

```

}
// Run as Script Program
/**
 *
 * C:\Program Files\Java\jdk-11.0.4\bin>java zeleven\Tester1001.java
 * There is a demo placed in jdk under zeleven.
 */

```

File: Demo_1001_String_Functions/com/accenture/ltt/zeleven/Test1003_String_isBlank.java

```

package com.accenture.ltt.zeleven;

public class Tester1003_String_isBlank {

    public static void main(String[] args) {
        System.out.println(" ".isBlank()); //true

        String s = "MSD";
        System.out.println(s.isBlank()); //false
        String s1 = "";
        System.out.println(s1.isBlank()); //true

    }

}

//https://www.journaldev.com/24601/java-11-features#java-string-methods

```

File: Demo_1001_String_Functions/com/accenture/ltt/zeleven/Test1004_String_Functions.java

```

package com.accenture.ltt.zeleven;

import java.util.stream.Collectors;

public class Tester1004_String_Functions {
    public static void main(String[] args) throws Exception {

        String str = "This Lines Demo.\n Done by MSD\n new feature";
        System.out.println(str);
        var listString=str.lines().collect(Collectors.toList());
        System.out.println("List of values: "+listString);

    }

}

//Lines to split with respect to \n

```

File: Demo_1001_String_Functions/com/accenture/ltt/zeleven/Test1005_String_Strip.java

```

package com.accenture.ltt.zeleven;

public class Tester1005_String_Strip {
    public static void main(String[] args) {

        String name = " Mohpreet ";
        System.out.print("Start");
        System.out.print(name.strip());
    }
}

```

```

        System.out.println("End");

        System.out.print("Start");
        System.out.print(name.stripLeading());
        System.out.println("End");

        System.out.print("Start");
        System.out.print(name.stripTrailing());
        System.out.println("End");
    }
}

/**
 *strip(), stripLeading(), stripTrailing()

strip()  Removes the white space from both, beginning and the end of string.

But we already have trim(). Then what's the need of strip()?
strip() is Unicode-aware evolution of trim().
When trim() was introduced, Unicode wasn't evolved. Now, the new strip() removes all kinds of w
and trailing(check the method Character.isWhitespace(c) to know if a unicode is whitespace or n
 * */

```

File: Demo_1001_String_Functions/com/accenture/ltt/zeleven/Tester1006_String_repeat.java

```

package com.accenture.ltt.zeleven;

public class Tester1006_String_repeat {
    public static void main(String[] args) {

        String str = "MSD,".repeat(9);
        System.out.println(str);
    }
}

```

14.7.2 File Utility Additions

File: Demo_1002_Files/com/accenture/ltt/zeleven/File_Additions.java

```

package com.accenture.ltt.zeleven;

import java.io.File;
import java.nio.file.Files;
import java.nio.file.Path;

public class File_Additions {
    public static void main(String[] args) throws Exception {
        Path path = Files.writeString(new File("text").toPath(), "Writing to the file");
        System.out.println(path);
        String s = Files.readString(path);
    }
}

```

```

        System.out.println(s);
    }

}

/**
 *
 *
 * String readString(Path path)
 * String readString(Path path, Charset cs)
 * Path writeString(Path path, CharSequence cs, OpenOption options)
 * Path writeString(Path path, CharSequence cs, Charset cs, OpenOption options
 * */

```

14.7.3 Predicate.not()

File: Demo_1003_NotPredicate/com/accenture/ltt/zeleven/Tester1001_Not_Predicate.java

```

package com.accenture.ltt.zeleven;

import java.util.Optional;

public class Tester1001_Not_Predicate {
    public static void main(String[] args) {
        var opt =
            Optional.ofNullable(null);
        System.out.println(opt.isEmpty() == true); // isEmpty added
    }
}

```

File: Demo_1003_NotPredicate/com/accenture/ltt/zeleven/Tester1002_Not_Predicate.java

```

package com.accenture.ltt.zeleven;

import java.util.List;
import java.util.function.Predicate;
import java.util.stream.Collectors;

//Not Predicate...
public class Tester1002_Not_Predicate {
    public static void main(String[] args) {
        List<String> list = List.of("Jack", "Jim", "James", "Tim", "Dan", "Eric");
        List<String> filteredList = list.stream().filter(Predicate.not(x->x.length()>=4)).collect(Collectors.toList());
        System.out.println(filteredList);
    }
}

```

14.7.4 var and Local-Variable Type Inference

File: Demo_1004-variables/com/accenture/ltt/ten/Tester1002.java

```

package com.accenture.ltt.ten;

//Type inference of Local variables Java 10

```

```

public class Tester1002 {
    public static void main(String[] args) {
        var t = 10;
        var var =20; //var is not keyword

        //var empty = null;
        // we cannot have empty var declaration
    }
}

```

//Java is not a Dynamic Language, Type is still there, only now inferred by compiler

File: Demo_1004-variables/com/accenture/ltt/ten/Tester1003.java

```

package com.accenture.ltt.ten;

public class Tester1003 {
    public static void main(String[] args) {

        var p1 = s->s.length()>3;
        var p2 = (String s)->s.length()>3;

    }
}

```

//Lambdas must have an explicit target type

File: Demo_1004-variables/com/accenture/ltt/ten/Tester1004.java

```

package com.accenture.ltt.ten;

import java.util.ArrayList;
import java.util.List;

public class Tester1004 {
    public static void main(String[] args) {

        //This is List of Strings
        List<String> myList1 = new ArrayList<>();

        //This is List of the Objects
        var myList2 = new ArrayList<>();

        System.out.println(myList1);
        System.out.println(myList2);

    }
}

//Draw back of using var,
// datatype cannot be inferred for generic from left side

```


File: Demo_1004-variables/com/accenture/ltt/ten/Tester1005.java

```
package com.accenture.ltt.ten;

public class Tester1005 {
    public static void main(String[] args) {

    }

}

/**
public void aMethod(var input) {
...
}
Local Type Inference cannot be used with
Method parameters, Fields
Return types, Catch blocks*/
//Need to add point about Lambdas
// from Java 11 onwards it is allowed on parameters of Lambdas

/****
 *
 * Refer Links_Var_Java_11 file in project root
 *
 * */
```

File: Demo_1004-variables/com/accenture/ltt/ten/zRefTester1006.java

```
package com.accenture.ltt.ten;

public class zRefTester1006 {
    public static void main(String[] args) {
        var obj = new Object() {}; // child class of object

        obj = new Object(); // cannot point back to parent as data type is inferred in the p

    }

}
//Proof of static typing
```

File: Demo_1004-variables/com/accenture/ltt/ten/zRefTester1007.java

```
package com.accenture.ltt.ten;

public class zRefTester1007 {
    public static void main(String[] args) {
        var obj = new Object(); // Parent class object

        obj = new Object() {}; // as data type is inferred in the previous step as parent
                                // Parent reference can point to Child class reference
    }
}
```

```

    }
}

```

File: Demo_1004-variables/com/accenture/ltt/ten/zRefTester1008.java

```

package com.accenture.ltt.ten;

import java.util.List;

public class zRefTester1008 {
    public static void main(String[] args) {
        var list = List.of(1, 2.0, "3"); //Used to give error in java 10
        list.forEach(v-> System.out.println(v));
    }
}

```

14.7.5 Java 9-era Additions — Collection Factories, Optional, takeWhile/dropWhile, Date Ranges

File: Demo_1005-JavaNine/com/accenture/ltt/nine/Tester1001.java

```

package com.accenture.ltt.nine;

import java.util.List;

public class Tester1001 {

    public static void main(String[] args) {
        List<Integer> list = List.of(1,2,3,4,5,6,7,8);
        list.forEach(x->System.out.println(x));
    }

}

//Collection Factory method for immutable collections

/**
 *
 * List.of()
List.of(E e1)
List.of(E... elements)
List.of(E e1, E e2)
List.of(E e1, E e2, E e3)
List.of(E e1, E e2, E e3, E e4)
List.of(E e1, E e2, E e3, E e4, E e5)
List.of(E e1, E e2, E e3, E e4, E e5, E e6)
List.of(E e1, E e2, E e3, E e4, E e5, E e6, E e7)
List.of(E e1, E e2, E e3, E e4, E e5, E e6, E e7, E e8)
List.of(E e1, E e2, E e3, E e4, E e5, E e6, E e7, E e8, E e9)
List.of(E e1, E e2, E e3, E e4, E e5, E e6, E e7, E e8, E e9, E e10)

```

Intermediate array allocation

```
*
*
* */

/**
 *
 *
 * if you try to perform any of the below then you will get an error
 *
 * // all mutating methods throw UnsupportedOperationException
 * @Override public boolean add(E e) { throw uoe(); }
 * @Override public boolean addAll(Collection<? extends E> c) { throw uoe(); }
 * @Override public void clear() { throw uoe(); }
 * @Override public boolean remove(Object o) { throw uoe(); }
 * @Override public boolean removeAll(Collection<?> c) { throw uoe(); }
 * @Override public boolean removeIf(Predicate<? super E> filter) { throw uoe(); }
 * @Override public boolean retainAll(Collection<?> c) { throw uoe(); }
 * @Override public E remove(int index) { throw uoe(); }
 * @Override public void replaceAll(UnaryOperator<E> operator) { throw uoe(); }
 * @Override public E set(int index, E element) { throw uoe(); }
 * @Override public void sort(Comparator<? super E> c) { throw uoe(); }
 *
 *
 * */
```

File: Demo_1005-JavaNine/com/accenture/ltt/nine/Tester1002.java

```
package com.accenture.ltt.nine;

import java.util.Map;

public class Tester1002 {
    public static void main(String[] args) {
        Map<Object, Object> multipleEntryMap1 = Map.of(1, "1", 2, 1, 3, "three", 4, "1", 5, 1, 6, "1");
        Map<Object, Object> multipleEntryMap2 = Map.ofEntries(
            Map.entry(1, "One"),
            Map.entry(2, "Two"),
            Map.entry(3, "Three"),
            Map.entry(4, "Four")
        );
        multipleEntryMap2.forEach((x, y) -> System.out.println(x + " " + y));

        //case2: un even set of keys and values
        // Map.of("Key1", 1, "Key2", 2, "Key3"); // Won't compile as key and value are not the same
        // there should be even set of values

        //case3: duplicate keys
        //Map.of("a", 1, "a", 2); // IllegalArgumentException
    }
}
```

```

    }

}
/*
 *
 *
 * Map.of(K key, V value)
Map.of(K key1, V value1, K key2, V value2)
... Up to 10 key/values
Map.ofEntries(Map.Entry<K, V>... entries)
 * Iteration order not guaranteed
 * **/

/**
 *
 *
 * if you try to perform any of the below then you will get an error
 *
 *
 * static UnsupportedOperationException uoe() { return new UnsupportedOperationException(); }
 *
 * @Override public void clear() { throw uoe(); }
 * @Override public V compute(K key, BiFunction<? super K,? super V,? extends V> rf) { throw uoe(); }
 * @Override public V computeIfAbsent(K key, Function<? super K,? extends V> mf) { throw uoe(); }
 * @Override public V computeIfPresent(K key, BiFunction<? super K,? super V,? extends V> rf) { throw uoe(); }
 * @Override public V merge(K key, V value, BiFunction<? super V,? super V,? extends V> rf) { throw uoe(); }
 * @Override public V put(K key, V value) { throw uoe(); }
 * @Override public void putAll(Map<? extends K,? extends V> m) { throw uoe(); }
 * @Override public V putIfAbsent(K key, V value) { throw uoe(); }
 * @Override public V remove(Object key) { throw uoe(); }
 * @Override public boolean remove(Object key, Object value) { throw uoe(); }
 * @Override public V replace(K key, V value) { throw uoe(); }
 * @Override public boolean replace(K key, V oldValue, V newValue) { throw uoe(); }
 * @Override public void replaceAll(BiFunction<? super K,? super V,? extends V> f) { throw uoe(); }
 *
 * **/

```

File: Demo_1005-JavaNine/com/accenture/ltt/nine/Tester1003.java

```

package com.accenture.ltt.nine;

import java.util.List;
import java.util.Set;

public class Tester1003 {
    public static void main(String[] args) {
        List<Integer> ints = List.of(1, 2, 3);

        //ints.add(4); // UnsupportedOperationException, immutable! Unfortunately there's no way to make a mutable list

        System.out.println(ints.getClass()); // java.util.ImmutableCollections$ListN

        List.of(1).getClass(); // java.util.ImmutableCollections$List1 -> super-optimized version of List

        Set.of("first", "second");
    }
}

```

```

        //Set.of(null); // NullPointerException, same happens for List.of

        Set.of("a", "a"); // IllegalArgumentException: sets must consist of unique elements

    }
}

```

File: Demo_1005-JavaNine/com/accenture/ltt/nine/Tester1004.java

```

package com.accenture.ltt.nine;

public class Tester1004 {

    public static void main(String[] args) {

    }

}
/**
 *
 * Stream API improvements:
 *
 * Stream API Improvements
 * Stream<T> takeWhile(Predicate<? super T> predicate)
 * Added methods:
 * Stream<T> dropWhile(Predicate<? super T> predicate)
 * static Stream<T> ofNullable(T t)
 * static Stream<T> iterate(T seed,
 * Predicate<? super T> hasNext,
 * UnaryOperator<T> next)
 *
 *
 * */

```

File: Demo_1005-JavaNine/com/accenture/ltt/nine/Tester1005TakeDropWhile.java

```

package com.accenture.ltt.nine;

import java.util.stream.IntStream;

public class Tester1005TakeDropWhile {

    public static void main(String... args ) {

        System.out.println("Before takeWhile");
        IntStream.range(1, 100)
            .filter(i -> i < 4) // applied to all elements
            .forEach(System.out::print);

        System.out.println("\nWith takeWhile, only the first 4 elements are "
            + " evaluated against the predicate.");
        IntStream.range(1, 100)
            .takeWhile(i -> i < 4) // short-circuits on element '4'
            .forEach(System.out::print);
    }
}

```

```

        System.out.println("\nBefore dropWhile");
        IntStream.range(1, 10)
            .filter(i -> i >= 4)
            .forEach(System.out::print);

        System.out.println("\nWith dropWhile (only works if Stream is sorted!");
        IntStream.range(1, 10)
            .dropWhile(i -> i < 4)
            .forEach(System.out::print);
    }
}
//Use with ordered Streams

```

File: Demo_1005-JavaNine/com/accnture/ltt/nine/Tester1006OptionalOr.java

```

package com.accnture.ltt.nine;

import java.util.Optional;
import java.util.Set;

import com.accnture.ltt.nine.bean.Book;

public class Tester1006OptionalOr {
    public static void main(String... args) {
        Optional<Book> localFallback = Optional.of(Book.getBook());

        // Before Optional.or
        //we are trying to get books with offer
        // if not found then get from external
        // if not found from the external then get from the local
        // using get is bad as even local fallback could return null
        Book bestBookBefore = getBestOffer()
            .orElse(getExternalOffer().orElse(localFallback.get())); // .get() is BAD!

        Optional<Book> bestBook =
            getBestOffer()
                .or(() -> getExternalOffer())
                .or(() -> localFallback);

        System.out.println(bestBook);
    }

    static Optional<Book> getBestOffer() {
        return Optional.empty();
    }

    static Optional<Book> getExternalOffer() {
        return Optional.of(new Book("External Book", Set.of(), 11.99));
    }
}

```

```

/**new methods of Optional
 *
 * void ifPresentOrElse(Consumer<T> action,Runnable emptyAction)
 * Optional<T> or(Supplier<Optional<T>> supplier)
 * Stream<T> stream()
 * */

```

File: Demo_1005-JavaNine/com/accenture/ltt/nine/Tester1007OptionalIfPresentOrElse.java

```

package com.accenture.ltt.nine;

import java.util.Optional;

import com.accenture.ltt.nine.bean.Book;

public class Tester1007OptionalIfPresentOrElse {

    public static void main(String... args) {
        Optional<Book> full = Optional.of(Book.getBook());

        // Before ifPresentOrElse
        full.ifPresent(x->System.out.println("1-->" + x));

        if (full.isPresent()) {
            System.out.println("2-->" + full.get());
        } else {
            System.out.println("Nothing here");
        }

        //Now
        full.ifPresentOrElse(x->System.out.println("3-->" + x),
            () -> System.out.println("Nothing here!"));

        Optional.empty().ifPresentOrElse(x->System.out.println("4-->" + x),
            () -> System.out.println("Nothing here!"));

    }

}

```

File: Demo_1005-JavaNine/com/accenture/ltt/nine/Tester1009.java

```

package com.accenture.ltt.nine;

public class Tester1009 {
    public static void main(String[] args) {
        //String _ ="Jack";
        String name_ = "Jane";
    }
}

```

```

    }

}
/**
 * as of release 9, '_' is a
 * keyword, and may not be used as an
 * identifier
 *
 *
 * **/

```

File: Demo_1005-JavaNine/com/accenture/ltt/nine/Tester1010.java

```

package com.accenture.ltt.nine;

import java.io.FileInputStream;
import java.io.IOException;

public class Tester1010 {
    public static void main(String[] args) {

    }

    // 1 try with resources
    public void doWithFileJava3() throws IOException {
        try (FileInputStream fis = new FileInputStream("~/tmp/test")) {
            fis.read();
        } catch (Exception e) {
            System.out.println(e.getMessage());
        }
    }

    // 2 try with resources, new variable required here line 24
    public void doWithFileJava8(FileInputStream fis) throws IOException {
        try (FileInputStream fis2 = fis) {
            fis2.read();
        } catch (Exception e) {
            System.out.println(e.getMessage());
        }
    }

    // 3 try with resource, java 9
    public void doWithFileJava9(FileInputStream fis) throws IOException {
        try (fis) {
            fis.read();
        } catch (Exception e) {
            System.out.println(e.getMessage());
        }
    }
}

```

File: Demo_1005-JavaNine/com/accenture/ltt/nine/Tester1011.java


```

package com.accenture.ltt.nine;

import java.util.List;

public class Tester1011 {
    public static void main(String[] args) {
        List<Integer> list = List.of(1,2,3,4,1,1,1,1,1,1,1,1,1,1,1,1,1);
        List<Integer> list2=list;

        list.add(12);

        if(list==list2) {
            System.out.println("From here...");
        }

    }
}

```

File: Demo_1005-JavaNine/com/accenture/ltt/nine/bean/Book.java

```

package com.accenture.ltt.nine.bean;

import java.util.Set;
import java.util.stream.Stream;

public class Book {

    public final String title;
    public final Set<String> authors;
    public final double price;

    public Book(String title, Set<String> authors, double price) {
        this.title = title;
        this.authors = authors;
        this.price = price;
    }

    public String getTitle() {
        return title;
    }

    public Set<String> getAuthors() {
        return authors;
    }

    public double getPrice() {
        return price;
    }

    @Override
    public String toString() {
        return "Book{" +

```

```

        "title='" + title + '\'' +
        ", price=" + price +
        '}}';
    }

    public static Stream<Book> getBooks() {
        return Stream.of(new Book("Java 9 Modularity", Set.of("Sander Mak", "Paul Bakker"), 33.99),
            new Book("Designing Data-Intensive Applications", Set.of("Martin Klepmann", "Oscar Young"), 39.99),
            new Book("Java 8 Lambdas", Set.of("Richard Warburton"), 33.99));
    }

    public static Book getBook() {
        return getBooks().findFirst().get();
    }
}

```

File: Demo_1005-JavaNine/com/accnture/ltt/nine/zRefNewCollectors.java

```

package com.accenture.ltt.nine;

import static java.util.stream.Collectors.filtering;
import static java.util.stream.Collectors.flatMapping;
import static java.util.stream.Collectors.groupingBy;
import static java.util.stream.Collectors.toSet;

import java.util.Map;
import java.util.Set;
import java.util.stream.Stream;

import com.accenture.ltt.nine.bean.Book;

public class zRefNewCollectors {
    public static void main(String... args) {
        Stream<Book> books = Book.getBooks();

        // Collect all books costing more than 10, grouped by author(s)
        Map<Set<String>, Set<Book>> booksByAuthors =
            books.collect(
                groupingBy(Book::getAuthors,
                    filtering(b -> b.getPrice() > 10,
                        toSet())
                )
            );
        System.out.println(booksByAuthors);

        books = Book.getBooks();
        // Collect all authors that sell books at a given price
        Map<Double, Set<String>> authorsSellingForPrice =
            books.collect(
                groupingBy(Book::getPrice,
                    flatMapping(b -> b.getAuthors().stream(),
                        toSet())
                )
            );
    }
}

```

```

        System.out.println(authorsSellingForPrice);
    }
}

```

File: Demo_1005-JavaNine/com/accnture/ltt/nine/zRefTester1008OptionalStream.java

```

package com.accnture.ltt.nine;

import java.util.List;
import java.util.Optional;
import java.util.stream.Collectors;
import java.util.stream.Stream;

public class zRefTester1008OptionalStream {

    public static void main(String[] args) {

        // given
        Optional<String> value = Optional.of("a");

        // when
        List<String> collect = value.stream().map(String::toUpperCase).collect(Collectors.toList());

        System.out.println(collect);

        //Demo 2
        Stream<Optional<Integer>> optionals =
            Stream.of(Optional.of(1), Optional.empty(), Optional.of(2));

        Stream<Integer> ints = optionals.flatMap(x->x.stream());

        System.out.println(ints);

    }
}

//. The Java 9 introduces the stream() method on the Optional class that allows us to treat the
// Stream will operate on the underlying value/Datatype of Optional

//https://www.baeldung.com/java-9-optional

```

File: Demo_1005-JavaNine/com/accnture/ltt/nine/zRefTester1011DatesUntil.java

```

package com.accnture.ltt.nine;

import java.time.LocalDate;
import java.time.Period;
import java.time.Year;
import java.util.stream.Stream;

public class zRefTester1011DatesUntil {

```

```

public static void main(String... args) {
    LocalDate start = LocalDate.of(2017, 12, 1);
    Stream<LocalDate> dates = start.datesUntil(start.plusDays(7));
    dates.forEach(System.out::println);

    //Counting number of Leap years from a date till today
    LocalDate birthday = LocalDate.of(1983, 12, 6);

    long leapYears = birthday
        .datesUntil(LocalDate.now(), Period.ofYears(1))
        .map(d -> Year.of(d.getYear()))
        .filter(year->year.isLeap())
        .count();

    System.out.printf("%d leap years since Sander was born on %s", leapYears, birthday);
}
}

```

14.8 Viva Voce — Likely Questions & Answers

Q1. What is the difference between isEmpty() and isBlank() for Strings? A: isEmpty() returns true only when the string has zero length. isBlank() (Java 11+) returns true when the string is empty *or* consists entirely of whitespace characters — a string of just spaces is isEmpty() == false but isBlank() == true.

Q2. Does using var make Java dynamically typed? A: No. var only tells the *compiler* to infer the variable's type from its initializer at compile time — once inferred, that type is fixed permanently, exactly as if it had been written explicitly. You still cannot later assign an incompatible type to that variable.

Q3. What is the difference between Duration and Period in java.time? A: Duration represents a *time-based* amount (hours, minutes, seconds, nanoseconds) — suited to precise elapsed-time calculations. Period represents a *date-based* amount (years, months, days) — suited to calendar-oriented calculations that need to account for varying month lengths, leap years, etc.

Q4. What does Predicate.not() do, and why is it particularly useful with method references? A: It returns the logical negation of a given predicate. It's especially useful with method references (like String::isBlank) because you cannot apply the ! operator directly to a method reference — Predicate.not(String::isBlank) provides the clean way to express "is not blank".

Q5. What is the purpose of the Epsilon garbage collector, and why would you never use it in production? A: Epsilon allocates memory but never reclaims it — useful only for measuring an application's raw memory-allocation behaviour in isolation from real GC overhead during performance testing. In production, any long-running application would eventually exhaust its heap and crash, since nothing is ever freed.

Q6. What replaced HttpURLConnection as Java's standard HTTP client, and in which release did it become stable? A: java.net.http.HttpClient, which was incubating (experimental) starting Java 9 and became a fully standardized, stable API in Java 11 (JEP 321).

Q7. What guarantee do List.of(), Set.of(), and Map.of() provide that Arrays.asList()

does not? A: They produce genuinely **immutable** collections — any attempted modification (add, remove, set) throws `UnsupportedOperationException`. `Arrays.asList()`, by contrast, produces a *fixed-size* (but not fully immutable) list backed by an array, where `set()` is still permitted.

Chapter 15: Java 17 Features

Java 17 (2021) is the current Long-Term-Support release most enterprises (including this training’s tooling, e.g. SonarQube automation) standardize on — it’s a natural “current baseline” question in a viva about modern Java.

15.1 JDEPS — the Java Dependency Analyzer

jdeps is a JDK command-line tool that statically analyzes the dependencies of your compiled class files or JARs — revealing exactly which packages/modules a given piece of code actually depends on at the bytecode level (not just what’s imported in source, which can include unused imports). This is invaluable when migrating a legacy codebase to the Java Platform Module System (JPMS, introduced in Java 9), or simply auditing what a third-party legacy JAR genuinely needs at runtime before trusting it in a minimal, security-hardened deployment.

15.2 JEP, JSR, and JLS — How Java Itself Evolves

- **JSR (Java Specification Request)** — the *historic*, more formal proposal mechanism submitted to the Java Community Process (JCP) to add or change a Java specification; heavier-weight and largely superseded for OpenJDK’s own evolution by JEPs.
- **JEP (JDK Enhancement Proposal)** — the modern, lighter-weight process (used since Java 9) for proposing, describing, and tracking a *specific* feature through OpenJDK’s actual development — e.g., “JEP 361: Switch Expressions”. Each Java release is essentially defined by the bundle of JEPs it delivers.
- **JLS (Java Language Specification)** — the single authoritative, formal document defining Java’s syntax and semantics precisely, which every compliant JVM/compiler implementation (not just Oracle’s) must conform to — this is what makes “Java” a portable *specification*, not just whatever one particular vendor’s compiler happens to do.

15.3 Sealed Classes (JEP 409)

A sealed class or interface explicitly restricts **which specific other classes** are permitted to extend or implement it, declared via a `permits` clause:

```
public sealed interface Shape permits Circle, Square, Triangle {}
```

This gives the compiler **exhaustive, complete knowledge** of every possible concrete subtype at compile time. This is especially powerful in combination with pattern-matching switch (15.4) — the compiler can then verify a switch over a sealed type’s variants is truly exhaustive (every possible subtype is handled) and flag a compile error if a new subtype is ever added later without updating every relevant switch. This is generally useful for modeling any strictly closed, finite set of possibilities (e.g., a fixed product-category hierarchy, as shown in the demo’s `Product/ ElectronicsProducts/ BeautyProducts/ KidsProducts/ BabyProducts` hierarchy) where you deliberately *don’t* want arbitrary external subclasses being added later, unlike a normal open class hierarchy.

Permitted subclasses must be one of: `final` (cannot be extended further), `sealed` (continues restricting further down the hierarchy), or `non-sealed` (explicitly reopens that specific branch to unrestricted further extension) — every permitted subclass must pick exactly one of these three.

15.4 Pattern Matching for switch (Preview in Java 17, JEP 406)

Pattern matching lets a single switch branch simultaneously **test the runtime type of** and **bind a new variable to** an object in one step, eliminating the older, more verbose manual instanceof check + explicit cast boilerplate:

```
// Old style, pre-pattern-matching:
if (obj instanceof Integer) {
    Integer i = (Integer) obj;
    return "int: " + i;
}
// New style, pattern matching for switch:
String describe(Object obj) {
    return switch (obj) {
        case Integer i -> "int: " + i;
        case String s   -> "string of length " + s.length();
        default          -> "unknown";
    };
}
```

Combined with sealed classes (15.3), the compiler can verify such a switch is **exhaustive** across all permitted subtypes, entirely removing the need for a default branch in that specific case (and flagging an error at compile time if a subtype is left unhandled).

15.5 strictfp and Floating-Point Consistency

The strictfp keyword historically forced floating-point calculations to strictly follow the IEEE-754 standard identically across **all** platforms, guaranteeing bit-for-bit identical results everywhere — relevant for financial or scientific computation where certain platform-specific floating-point hardware extensions could otherwise cause tiny (but potentially consequential) discrepancies between machines. **Since Java 17, all floating-point operations are strictfp by default** — so the strictfp keyword itself is now effectively a no-op, retained purely for source compatibility with older code that still uses it.

15.6 Enhanced Pseudorandom Number Generators (JEP 356)

Java 17 introduced a new RandomGenerator interface and several additional concrete algorithm implementations, going beyond the single classic java.util.Random class that had existed largely unchanged for decades. These new generators offer better statistical properties for non-cryptographic use cases, and easier runtime **algorithm selection** and streaming of random values via RandomGenerator.of("algorithmName"), without being locked into one specific implementation choice at compile time.

The demo programs below cover: the sealed Product hierarchy, jdeps usage, both pattern-matching switch demos, the new random-number generator API, and a strictfp illustration.

File: Demo10001_Java17/com/accenture/ltt/sampleclasses/BabyProducts.java

```
package com.accenture.ltt.sampleclasses;

//non-sealed classes can be extended.
public class BabyProducts extends KidsProducts {
    @Override
    public String getProductDescription() {
        return "Its a baby's product";
    }
}
```

```
}
```

File: Demo10001_Java17/com/accenture/ltt/sampleclasses/BeautyProducts.java

```
package com.accenture.ltt.sampleclasses;

public sealed class BeautyProducts extends Product permits BeautyProductsForWomen {

    @Override
    public String getProductDescription() {
        return "Its a beauty product";
    }
}
```

File: Demo10001_Java17/com/accenture/ltt/sampleclasses/BeautyProductsForWomen.java

```
package com.accenture.ltt.sampleclasses;

public non-sealed class BeautyProductsForWomen extends BeautyProducts {
    @Override
    public String getProductDescription() {
        return "Its a beauty product for women";
    }
}
```

File: Demo10001_Java17/com/accenture/ltt/sampleclasses/ElectronicsProducts.java

```
package com.accenture.ltt.sampleclasses;

final public class ElectronicsProducts extends Product {

    @Override
    public String getProductDescription() {
        return "Its a electronic product";
    }
}
```

File: Demo10001_Java17/com/accenture/ltt/sampleclasses/KidsProducts.java

```
package com.accenture.ltt.sampleclasses;

public non-sealed class KidsProducts extends Product {

    @Override
    public String getProductDescription() {
        return "Its a kid's product";
    }
}
```

File: Demo10001_Java17/com/accenture/ltt/sampleclasses/Product.java

```
package com.accenture.ltt.sampleclasses;

public sealed abstract class Product permits ElectronicsProducts, KidsProducts, BeautyProducts
    abstract public String getProductDescription();
```



```

}

//Uncomment the below class declaration and observe the error.
/*class HomeAppliances extends Product {

    @Override
    public String getProductDescription() {
        return "My Product";
    }

}*/

```

File: Demo10001_Java17/com/accenture/ltt/ui/tester/java17/JdepsDemo.java

```

package com.accenture.ltt.ui.tester.java17;

import java.util.Date;
import java.util.ArrayList;
import java.util.*;

public class JdepsDemo {
    public static void main(String[] args) {
        Date now = new Date();
        ArrayList<String> names = new ArrayList<>();
        names.add("Java");
        names.add("Full Stack");

        System.out.println("Hello, time is: " + now);
        System.out.println("Learning: " + names);
    }
}

```

File: Demo10001_Java17/com/accenture/ltt/ui/tester/java17/PatternSwitchDemo.java

```

package com.accenture.ltt.ui.tester.java17;
public class PatternSwitchDemo {
    public static void main(String[] args) {
        // Creating an Object that holds a String value
        // We use Object type to show how pattern matching works with different possible types
        Object obj = "Java 17";

        /*
         * Pattern Matching in switch (Java 17 preview feature)
         * -----
         * - You can match both the type AND the value in a single switch case.
         * - The syntax 'case Type variable ->' checks if obj is an instance of 'Type'
         *   and automatically casts it to that type.
         */
        String result = switch (obj) {
            // If obj is a String, cast to 's' and convert it to uppercase
            case String s -> "It's a string: " + s.toUpperCase();

            // If obj is an Integer, cast to 'i' and multiply by 2
            case Integer i -> "Integer value: " + (i * 2);
        };
    }
}

```

```

        // If obj is neither String nor Integer, handle it in the default case
        default -> "Unknown type";
    };

    // Printing the result based on the matched case
    System.out.println(result);
}
}

```

File: Demo10001_Java17/com/accenture/ltt/ui/tester/java17/PatternSwitchDemo2.java

```

package com.accenture.ltt.ui.tester.java17;

public class PatternSwitchDemo2 {
    public static void main(String[] args) {
        // Creating an Object that holds a String value
        // We use Object type to show how pattern matching works with different possible types
        Object obj = "Java 17";

        /*
         * Pattern Matching in switch (Java 17 preview feature)
         * -----
         * - You can match both the type AND the value in a single switch case.
         * - The syntax 'case Type variable ->' checks if obj is an instance of 'Type'
         *   and automatically casts it to that type.
         */
        String result = switch (obj) {
            // If obj is a String, cast to 's' and convert it to uppercase
            case String s -> "It's a string: " + s.toUpperCase();

            // If obj is an Integer, cast to 'i' and multiply by 2
            case Integer i -> "Integer value: " + (i * 2);

            // If obj is neither String nor Integer, handle it in the default case
            default -> "Unknown type";
        };

        // Printing the result based on the matched case
        System.out.println(result);
    }
}

```

File: Demo10001_Java17/com/accenture/ltt/ui/tester/java17/RandomDemo.java

```

package com.accenture.ltt.ui.tester.java17;

import java.util.random.RandomGenerator;

public class RandomDemo {
    public static void main(String[] args) {
        // Create a PRNG using a specific algorithm
        RandomGenerator rng = RandomGenerator.of("L64X128MixRandom");

        // Generate random integers
    }
}

```

```

        System.out.println("Random int: " + rng.nextInt(100));
        System.out.println("Random double: " + rng.nextDouble());

        // Generate 5 random numbers as a stream
        rng.ints(5, 1, 10).forEach(System.out::println);
    }
}

```

File: Demo10001_Java17/com/accenture/ltt/ui/tester/java17/StrictFPDemo.java

```

package com.accenture.ltt.ui.tester.java17;

public class StrictFPDemo {
    public static void main(String[] args) {
        double result = 0.1 + 0.2;
        // Result will be consistent across all platforms
        System.out.println(result); // Prints 0.30000000000000004 (IEEE 754 standard)
    }
}

```

File: Demo10001_Java17/com/accenture/ltt/ui/tester/java17/Tester1001Sealed.java

```

package com.accenture.ltt.ui.tester.java17;

import com.accenture.ltt.sampleclasses.BabyProducts;
import com.accenture.ltt.sampleclasses.BeautyProducts;
import com.accenture.ltt.sampleclasses.ElectronicsProducts;
import com.accenture.ltt.sampleclasses.KidsProducts;
import com.accenture.ltt.sampleclasses.Product;

//Make sure JRE compliance pointing to JRE17.
// If not, enable preview feature from Eclipse -> Right click to Project -> Build path -> Confi
public class Tester1001Sealed {
    public static void main(String[] args) {
        Product product;
        product = new BabyProducts();
        //product = new ElectronicsProducts();
        //product = new KidsProducts();
        //product = new BeautyProducts();

        String productPescription = getDescription(product);
        System.out.println("Product description: " + productPescription);
    }

    private static String getDescription(Product product) {
        String description = null;
        if(product instanceof BabyProducts babyProduct) {
            description = babyProduct.getProductDescription();
        }
        else if(product instanceof BeautyProducts beautyProduct) {
            description = beautyProduct.getProductDescription();
        }
        else if(product instanceof ElectronicsProducts electronicsProduct) {
            description = electronicsProduct.getProductDescription();
        }
    }
}

```

```

        else if (product instanceof KidsProducts kidsProduct) {
            description = kidsProduct.getProductDescription();
        }
        else {
            description = "Invalid product";
        }

        return description;
    }
}

```

15.7 Viva Voce — Likely Questions & Answers

Q1. What problem do sealed classes solve, and how do they combine with pattern-matching switch? A: Sealed classes/interfaces restrict exactly which classes may extend/implement them via a `permits` clause, giving the compiler complete, exhaustive knowledge of every possible subtype. This lets a pattern-matching switch over that sealed type be verified as truly exhaustive at compile time, with no default branch needed and a compile error raised if a new permitted subtype is ever added without updating the switch.

Q2. What three options does a class permitted by a sealed class have for itself? A: It must be declared exactly one of: `final` (no further extension allowed), `sealed` (continues restricting further down the hierarchy with its own `permits` list), or `non-sealed` (explicitly reopens that branch to unrestricted further subclassing).

Q3. What is `jdeps` used for? A: A JDK command-line tool that statically analyzes the *actual bytecode-level* dependencies of compiled classes or JARs — showing exactly which packages/modules code truly depends on, useful for JPMS migration or auditing legacy JAR dependencies.

Q4. What is the difference between a JSR and a JEP? A: JSR (Java Specification Request) is the older, more formal Java Community Process mechanism for proposing specification changes. JEP (JDK Enhancement Proposal) is the lighter-weight process used since Java 9 to propose and track individual features through actual OpenJDK development — modern Java releases are essentially defined by their bundle of JEPs.

Q5. How does pattern matching for switch improve on the older `instanceof` + manual cast idiom? A: It combines the type check and the variable binding into a single step within the case label itself (case `Integer i -> ...`), eliminating the separate explicit cast statement previously required after an `instanceof` check, and reducing boilerplate/room for casting mistakes.

Q6. Is `strictfp` still meaningful to write in Java 17+ code? A: Not functionally — since Java 17, all floating-point operations already behave in the strict IEEE-754-compliant way by default, so the keyword is effectively a no-op, kept only for source compatibility with pre-existing code.

Q7. What is the JLS, and why does its existence matter for Java’s portability claims? A: The Java Language Specification is the single authoritative document defining Java’s syntax and semantics. Because any compliant JVM/compiler implementation (not only Oracle’s) must conform to it, it’s what actually makes “Java” a portable, vendor-neutral *specification* rather than just the behaviour of one specific tool.

Chapter 16: Unit Testing & Test-Driven Development (TDD)

16.1 What is Unit Testing?

A **unit test** verifies that one small, isolated piece of code (typically a single method, or a tightly related small group) behaves correctly, entirely independent of the rest of the system. Good unit tests share four properties, often remembered by the acronym **F.I.R.S.T**:

- **Fast** — runs in milliseconds; a suite of thousands of unit tests should still complete in seconds, so developers run them constantly without friction.
- **Isolated/Independent** — no shared mutable state between tests, no dependency on test *execution order*, no real database/network/filesystem calls (those belong to *integration* tests, a different, slower category entirely).
- **Repeatable** — produces the exact same result every single run, on any machine, regardless of time of day, network availability, or environment.
- **Self-checking / Self-validating** — the test itself determines pass/fail automatically via assertions; no human needs to manually eyeball console output to judge success.
- (*Timely, sometimes added as the “T”*) — ideally written close in time to (or, per TDD, even *before*) the production code it tests, while the intended behaviour is still fresh in the author’s mind.

16.2 Test-Driven Development — the Red-Green-Refactor Cycle

TDD deliberately inverts the usual order of writing code: instead of writing the implementation first and tests afterward (if at all — a pattern that often results in tests that merely *describe* whatever the code happens to already do, rather than what it *should* do), you write a **failing test first**, then just enough code to pass it, then clean up:

1. **RED** — Write a test for a behaviour that doesn’t exist yet in the code. Run it — it **must** fail. This step is not a formality: it *proves* the test is actually capable of failing and is therefore meaningfully exercising something real, rather than silently passing by accident (e.g., due to a typo in an assertion that always evaluates true).
2. **GREEN** — Write the **minimum** code necessary to make that one failing test pass — deliberately resisting the urge to over-engineer, add unrequested flexibility, or handle cases no test yet demands. This keeps the design driven strictly by actual, demonstrated need.
3. **REFACTOR** — With the safety net of a passing test suite now in place, clean up the implementation (remove duplication, improve naming, simplify convoluted logic, extract helper methods) — the *existing* passing tests act as a regression safety net, giving you confidence that this cleanup hasn’t silently broken any previously-working behaviour.

This tight, disciplined loop repeats for every new piece of behaviour, growing the implementation **incrementally**, always backed by an ever-growing suite of tests that documents exactly what the system is supposed to do.

16.3 Benefits of TDD

- Tests double as **living, always-accurate documentation** of what the code is supposed to do — unlike a separate design document, a test *cannot* silently go stale, because if the behaviour it describes ever changes, the test itself fails immediately and demands attention.
- Bugs are caught **immediately, at the smallest possible scope** (a single method), rather than surfacing much later during integration testing, QA, or (worst of all) in pro-

duction — where the same bug is dramatically more expensive and time-consuming to trace back to its root cause.

- TDD naturally encourages more **modular, loosely-coupled design**, because code that is tightly coupled to concrete dependencies (a real database, a real external API) is inherently hard to unit-test in isolation — the discipline of “make this testable first” tends to push developers toward better structural design as a side effect, not merely as an explicit goal.
- Provides psychological confidence to refactor fearlessly, since any regression is caught immediately by the existing suite rather than being discovered by a customer weeks later.

16.4 A Common Misconception, Worth Clarifying for Viva

TDD is **not** primarily “a testing technique” in the sense of finding bugs after the fact — it is better understood as a **design technique** that happens to produce a comprehensive automated test suite as a natural side effect. The tests aren’t the primary goal; disciplined, incrementally-driven design is.

The demo below implements a `DiscountClass` following exactly this Red-Green-Refactor rhythm — the accompanying `DiscountTestClass` is the test written *before* (or alongside, driving) the implementation.

File: 01_TDD_Demos/Demo_1001_ TDDCycle_RGR/com/accenture/ltt/DiscountClass.java

```
package com.accenture.ltt;

public class DiscountClass {

    public static double calculate(double price, double discountPercent) {
        // To Show the Phase - 1 Red Uncomment the below code
        return 0;

        //To Show the Phase - 2 Green Uncomment the below code
        //return price;

        // To Show the Phase - 3 Refactor Uncomment the below code
        //return findDiscount(price, discountPercent);
    }

    private static double findDiscount(double price, double discountPercent) {
        return price - (price * discountPercent / 100);
    }
}
```

File: 01_TDD_Demos/Demo_1001_ TDDCycle_RGR/com/accenture/ltt/DiscountTestClass.java

```
package com.accenture.ltt.test;

import static org.junit.jupiter.api.Assertions.assertEquals;

import org.junit.Test;

import com.accenture.ltt.DiscountClass;

public class DiscountTestClass {
```

```

@Test
public void testReturningSamePriceIfNoDiscount() {
    // To Show the Phase - 1 Red Uncomment the below code[Also for Phase - 2 Green]
    assertEquals(100, DiscountClass.calculate(100, 0));

    //To Show the Phase - 3 Refactor Uncomment the below code
    assertEquals(90, DiscountClass.calculate(100, 10)); // Test Case Passed for the Discount
    assertEquals(90, DiscountClass.calculate(100, 20)); // Test Case Passed for the Discount
}
}

```

16.5 Viva Voce — Likely Questions & Answers

Q1. What are the key properties of a good unit test (F.I.R.S.T.)? A: Fast, Isolated/Independent, Repeatable, Self-checking (and ideally Timely — written close to, or even before, the code it tests).

Q2. Describe the Red-Green-Refactor cycle. A: RED — write a failing test for behaviour that doesn't exist yet, and confirm it actually fails. GREEN — write the minimum code needed to make that test pass, without over-engineering. REFACTOR — clean up the implementation, relying on the now-passing test(s) as a safety net against regressions.

Q3. Why is it important that the test actually fails first, in the RED step, before you write any implementation? A: It proves the test is genuinely capable of detecting the absence (or brokenness) of the intended behaviour — a test that “passes” even before any implementation exists is not actually testing anything meaningful, often due to a flawed assertion.

Q4. Is TDD primarily a testing technique or a design technique? A: It's better understood as a design technique — writing tests first drives incremental, minimal, well-structured implementation and naturally discourages tight coupling to concrete dependencies (since tightly-coupled code is hard to test in isolation). The resulting comprehensive test suite is a valuable side effect, not the sole goal.

Q5. What's the risk of writing tests *after* the implementation, rather than following TDD? A: Tests written after the fact often end up simply describing whatever the code *already happens* to do (including any latent bugs), rather than rigorously specifying what it *should* do — reducing their value at catching genuine defects or unintended behaviour.

Chapter 17: JUnit 5

17.1 JUnit 5 Architecture

JUnit 5 is composed of three distinct sub-projects, a common area of confusion worth clarifying:

- **JUnit Platform** — the foundation that launches testing frameworks on the JVM; provides the API that build tools (Maven, Gradle) and IDEs use to discover and run tests, regardless of which actual testing framework/version wrote them.
- **JUnit Jupiter** — the *new* programming model and extension API that you actually write your tests against (@Test, Assertions, etc. — everything covered in this chapter).
- **JUnit Vintage** — a compatibility layer that runs older JUnit 3 and JUnit 4 tests *on* the new JUnit Platform, allowing a gradual, incremental migration of a large legacy test suite rather than requiring an all-at-once rewrite.

17.2 Assertions

Assertions are the actual pass/fail checks inside a test method, imported from `org.junit.jupiter.api.Assertions`:

- `assertEquals(expected, actual)` / `assertNotEquals(...)` — value equality checks.
- `assertTrue(condition)` / `assertFalse(condition)` — boolean condition checks.
- `assertNull(obj)` / `assertNotNull(obj)` — null checks.
- `assertThrows(ExceptionType.class, () -> riskyCode())` — verifies that executing the given code block **does** throw an exception of the specified type; central to properly testing exceptional/ error-handling paths rather than only “happy path” behaviour (and can capture the thrown exception object itself, to further assert on its message).
- `assertAll(() -> assert1, () -> assert2, ...)` — groups multiple related assertions so that **all** of them are actually executed and reported, even if an earlier one in the group fails — unlike writing separate sequential assertions, where the *first* failure would immediately halt the test method and hide whether any of the *subsequent* assertions would also have failed.

17.3 Assumptions

`Assumptions.assumeTrue(condition)` (and related methods like `assumeFalse`) causes a test to be **skipped entirely** (reported as neither pass nor fail) when a stated precondition isn’t met — useful for environment-dependent tests, e.g. only meaningfully running a test when a certain operating system, external tool, or configuration value is present, without polluting the test report with a false “failure” for an environment where the test was never applicable in the first place.

17.4 Lifecycle Annotations

Annotation	When it runs	Typical use
@BeforeAll	Once, before any test method in the class (must be static)	Expensive, class-wide one-time setup (e.g., starting a shared in-memory resource)
@BeforeEach	Before every individual test method	Resetting/creating fresh test data so tests don’t leak state into one another

Annotation	When it runs	Typical use
@Test	Marks a method as an actual, runnable test case	The test logic itself
@AfterEach	After every individual test method	Cleanup specific to that one test
@AfterAll (formerly @AfterClass in JUnit 4)	Once, after all tests in the class have finished	Releasing class-wide shared resources
@Disabled	Skips a test intentionally	Temporarily disabling a known-broken or not-yet-relevant test, ideally with a reason comment explaining why

Why @BeforeEach matters so much in practice: re-creating fresh test data before *every* test method (rather than sharing one mutable object across all tests in the class) is what keeps tests properly **isolated** from one another (recall the “Isolated” property from Chapter 16’s F.I.R.S.T. principles) — without it, one test’s side effects on shared state could silently influence whether a *later* test passes or fails, making failures far harder to diagnose and tests order-dependent.

17.5 Testing for Exceptional Conditions

Rather than wrapping code in a manual try/catch inside a test (verbose, and easy to get subtly wrong — e.g., forgetting to fail() if the expected exception never actually gets thrown), JUnit 5’s assertThrows() is the idiomatic, concise way to assert that a specific exception type is thrown by a given block of code — and it can also capture and further inspect the thrown exception’s message or other properties, all in one clean expression.

The demo set below is a full EmployeeService test suite (10 progressively built test classes, EmployeeServiceTest01 through EmployeeServiceTest10) exercising assertions, lifecycle annotations, and exception testing against a realistic Employee/EmployeeService business layer — showing concretely how a real test suite accumulates coverage incrementally, test-by-test, much like the TDD rhythm from Chapter 16.

File: Demo1001_Junit/com/accenture/ltt/businessbean/Employee.java

```
package com.accenture.ltt.businessbean;

public class Employee {
    private int employeeId;
    private String employeeName;
    private int salary;
    private int departmentCode;
    private String email;

    public Employee() {
        super();
    }

    public Employee(int employeeId, String employeeName, int salary, int departmentCode, String
        super();
        this.employeeId = employeeId;
        this.employeeName = employeeName;
    }
}
```

```

        this.salary = salary;
        this.departmentCode = departmentCode;
        this.email = email;
    }

    public int getEmployeeId() {
        return employeeId;
    }

    public void setEmployeeId(int employeeId) {
        this.employeeId = employeeId;
    }

    public String getEmployeeName() {
        return employeeName;
    }

    public void setEmployeeName(String employeeName) {
        this.employeeName = employeeName;
    }

    public int getSalary() {
        return salary;
    }

    public void setSalary(int salary) {
        this.salary = salary;
    }

    public int getDepartmentCode() {
        return departmentCode;
    }

    public void setDepartmentCode(int departmentCode) {
        this.departmentCode = departmentCode;
    }

    public String getEmail() {
        return email;
    }

    public void setEmail(String email) {
        this.email = email;
    }
}

```

File: Demo1001_Junit/com/accenture/ltt/exceptions/InvalidEmailAddressException.java

```

package com.accenture.ltt.exceptions;

public class InvalidEmailAddressException extends Exception{
    private static final long serialVersionUID = 1L;

```

```

    public InvalidEmailAddressException(String message) {
        super(message);
    }
}

```

File: Demo1001_Junit/com/accenture/ltt/service/EmployeeService.java

```

package com.accenture.ltt.service;

import com.accenture.ltt.buisnessbean.Employee;
import com.accenture.ltt.exceptions.InvalidEmailAddressException;

public class EmployeeService {

    public int computeSalary(Employee employee, int taxRate) {
        int finalSalary = 0;
        try {
            int salary = employee.getSalary();
            int taxOnSalary = salary / taxRate;
            finalSalary = salary - taxOnSalary;
        } catch (ArithmeticException | NullPointerException exception) {
            throw exception;
        }
        return finalSalary;
    }

    public int incrementSalary(Employee employee, int increment) {
        int newSalary = 0;
        try {
            newSalary = employee.getSalary() + increment;
        } catch (NullPointerException exception) {
            throw exception;
        }
        return newSalary;
    }

    public void validateEmailAddress(Employee employee) throws InvalidEmailAddressException{
        String email = employee.getEmail();
        if (! email.contains("@")) {
            throw new InvalidEmailAddressException("Employee's Email Address is invalid");
        }
        System.out.println(email);
    }

    public void getEmployeeDetails() {
        System.out.println("Fetching details ...");
        try {
            Thread.sleep(3000);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }
}

```

File: Demo1001_Junit/com/accenture/ltt/ui/UITester.java

```
package com.accenture.ltt.ui;

import java.util.Scanner;

import com.accenture.ltt.businessbean.Employee;
import com.accenture.ltt.service.EmployeeService;

public class UITester {
    public static void main(String[] args) {
        Employee employee = createEmployee();
        EmployeeService employeeService = createEmployeeService();

        try (Scanner scanner = new Scanner(System.in)) {
            int newSal = employeeService.incrementSalary(employee, 3000);
            employee.setSalary(newSal);
            System.out.println("Enter tax Rate ");
            int finalSalary = employeeService.computeSalary(employee, scanner.nextInt());
            System.out.println("Salary after tax reduction is " + finalSalary);
        } catch (ArithmeticException | NullPointerException exception) {
            System.out.println(exception.getMessage());
        }
    }

    public static Employee createEmployee() {
        // Employee emp = null;
        Employee emp = new Employee(1001, "John", 10000, 101, "john@accenture.com");
        return emp;
    }

    public static EmployeeService createEmployeeService() {
        return new EmployeeService();
    }
}
```

File: Demo1001_Junit/com/accenture/ltt/junit/EmployeeServiceTest01.java

```
package com.accenture.ltt.junit.test;

import static org.junit.jupiter.api.Assertions.assertEquals;

import org.junit.jupiter.api.AfterAll;
import org.junit.jupiter.api.BeforeAll;
import org.junit.jupiter.api.Test;

import com.accenture.ltt.businessbean.Employee;
import com.accenture.ltt.service.EmployeeService;

class EmployeeServiceTest01 {

    private static EmployeeService employeeService;

    @BeforeAll
    public static void setup() {
```

```

        System.out.println("Setting up EmployeeService");
        employeeService = new EmployeeService();
    }

    @AfterAll
    public static void tearDown() {
        System.out.println("Tearing Down EmployeeService");
        employeeService = null;
    }

    @Test
    public void testComputeSalary() {
        Employee emp = new Employee(1001, "John", 10000, 101, "john@accenture.com");
        assertEquals(9000, employeeService.computeSalary(emp, 10));
    }
}

```

File: Demo1001_Junit/com/accenture/ltt/junit/EmployeeServiceTest02.java

```

package com.accenture.ltt.junit.test;

import static org.junit.jupiter.api.Assertions.assertEquals;

import org.junit.jupiter.api.AfterAll;
import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.BeforeAll;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.TestInfo;

import com.accenture.ltt.businessbean.Employee;
import com.accenture.ltt.service.EmployeeService;

class EmployeeServiceTest02 {

    private static EmployeeService employeeService;

    @BeforeAll
    static void setup() {
        System.out.println("Setting up EmployeeService");
        employeeService = new EmployeeService();
    }

    @AfterAll
    static void tearDown() {
        System.out.println("Tearing Down EmployeeService");
        employeeService = null;
    }

    @BeforeEach
    void beforeTestMethod(TestInfo testInfo) {
        System.out.println("Start Executing Test..." + testInfo.getDisplayName());
    }
}

```

```

@AfterEach
void afterTestMethod(TestInfo testInfo) {
    System.out.println("Completed Test..." + testInfo.getDisplayName());
}

@Test
public void testIncrementSalary() {
    Employee emp = new Employee(1001, "John", 10000, 101, "john@accenture.com");
    assertEquals(11000, employeeService.incrementSalary(emp, 1000));
}

@Test
public void testComputeSalary() {
    Employee emp = new Employee(1001, "John", 10000, 101, "john@accenture.com");
    assertEquals(9000, employeeService.computeSalary(emp, 10));
}
}

```

File: Demo1001_Junit/com/accenture/ltt/junit/EmployeeServiceTest03.java

```

package com.accenture.ltt.junit.test;

import static org.junit.jupiter.api.Assertions.assertEquals;

import org.junit.jupiter.api.AfterAll;
import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.BeforeAll;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.TestInfo;
import org.junit.jupiter.api.TestInstance;
import org.junit.jupiter.api.TestInstance.Lifecycle;

import com.accenture.ltt.buisnessbean.Employee;
import com.accenture.ltt.service.EmployeeService;
@TestInstance(Lifecycle.PER_CLASS)
class EmployeeServiceTest03 {

    private EmployeeService employeeService;

    @BeforeAll
    public void setup() {
        System.out.println("Setting up EmployeeService");
        employeeService = new EmployeeService();
    }

    @AfterAll
    public void tearDwon() {
        System.out.println("Tearing Down EmployeeService");
        employeeService = null;
    }

    @BeforeEach
    public void startTest(TestInfo testInfo) {

```

```

        System.out.println("Start Executing Test..." + testInfo.getDisplayName());
    }

    @AfterEach
    public void endTest(TestInfo testInfo) {
        System.out.println("Completed Test..." + testInfo.getDisplayName());
    }

    @Test
    public void testIncrementSalary() {
        Employee emp = new Employee(1001, "John", 10000, 101, "john@accenture.com");
        assertEquals(11000, employeeService.incrementSalary(emp, 1000));
    }

    @Test
    public void testComputeSalary() {
        Employee emp = new Employee(1001, "John", 10000, 101, "john@accenture.com");
        assertEquals(9000, employeeService.computeSalary(emp, 10));
    }
}

```

File: Demo1001_Junit/com/accenture/ltt/junit/EmployeeServiceTest04.java

```

package com.accenture.ltt.junit.test;

import static org.junit.jupiter.api.Assertions.assertEquals;

import org.junit.jupiter.api.AfterAll;
import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.BeforeAll;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.TestInfo;

import com.accenture.ltt.buisnessbean.Employee;
import com.accenture.ltt.service.EmployeeService;

class EmployeeServiceTest04 {

    private static EmployeeService employeeService;

    @BeforeAll
    static void setup() {
        System.out.println("Setting up EmployeeService");
        employeeService = new EmployeeService();
    }

    @AfterAll
    static void tearDwon() {
        System.out.println("Tearing Down EmployeeService");
        employeeService = null;
    }
}

```

```

@BeforeEach
void startTest(TestInfo testInfo) {
    System.out.println("Start Executing Test..." + testInfo.getDisplayName());
}

@AfterEach
void endTest(TestInfo testInfo) {
    System.out.println("Completed Test..." + testInfo.getDisplayName());
}

@Test
@DisplayName("Increment Salary")
public void testIncrementSalary() {
    Employee emp = new Employee(1001, "John", 10000, 101, "john@accenture.com");
    assertEquals(11000, employeeService.incrementSalary(emp, 1000));
}

@Test
@DisplayName("Tax Calculation")
public void testComputeSalary() {
    Employee emp = new Employee(1001, "John", 10000, 101, "john@accenture.com");
    assertEquals(9000, employeeService.computeSalary(emp, 10));
}
}

```

File: Demo1001_Junit/com/accenture/ltt/junit/EmployeeServiceTest05.java

```

package com.accenture.ltt.junit.test;

import static org.junit.jupiter.api.Assertions.assertEquals;

import org.junit.jupiter.api.AfterAll;
import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.BeforeAll;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.RepeatedTest;
import org.junit.jupiter.api.TestInfo;

import com.accenture.ltt.buisnessbean.Employee;
import com.accenture.ltt.service.EmployeeService;

class EmployeeServiceTest05 {

    private static EmployeeService employeeService;

    @BeforeAll
    static void setup() {
        System.out.println("Setting up EmployeeService");
        employeeService = new EmployeeService();
    }

    @AfterAll
    static void tearDwon() {

```



```

        System.out.println("Tearing Down EmployeeService");
        employeeService = null;
    }

    @BeforeEach
    void startTest(TestInfo testInfo) {
        System.out.println("Start Executing Test..." + testInfo.getDisplayName());
    }

    @AfterEach
    void endTest(TestInfo testInfo) {
        System.out.println("Completed Test..." + testInfo.getDisplayName());
    }

    @RepeatedTest(value = 3)
    @DisplayName("Increment Salary")
    public void testIncrementSalary() {
        Employee emp = new Employee(1001, "John", 10000, 101, "john@accenture.com");
        assertEquals(11000, employeeService.incrementSalary(emp, 1000));
    }

    @RepeatedTest(value = 3, name = "{displayName} - repetition {currentRepetition} of {totalRepetitions}")
    @DisplayName("Tax Calculation")
    public void testComputeSalary() {
        Employee emp = new Employee(1001, "John", 10000, 101, "john@accenture.com");
        assertEquals(9000, employeeService.computeSalary(emp, 10));
    }
}

```

File: Demo1001_Junit/com/accenture/ltt/junit/EmployeeServiceTest06.java

```

package com.accenture.ltt.junit.test;

import static org.junit.jupiter.api.Assertions.assertEquals;

import org.junit.jupiter.api.AfterAll;
import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.BeforeAll;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Disabled;
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.TestInfo;

import com.accenture.ltt.buisnessbean.Employee;
import com.accenture.ltt.service.EmployeeService;

//@Disabled
class EmployeeServiceTest06 {

    private static EmployeeService employeeService;

    @BeforeAll
    static void setup() {

```

```

        System.out.println("Setting up EmployeeService");
        employeeService = new EmployeeService();
    }

    @AfterAll
    static void tearDwon() {
        System.out.println("Tearing Down EmployeeService");
        employeeService = null;
    }

    @BeforeEach
    void startTest(TestInfo testInfo) {
        System.out.println("Start Executing Test..." + testInfo.getDisplayName());
    }

    @AfterEach
    void endTest(TestInfo testInfo) {
        System.out.println("Completed Test..." + testInfo.getDisplayName());
    }

    @Disabled
    @Test
    @DisplayName("Increment Salary")
    public void testIncrementSalary() {
        Employee emp = new Employee(1001, "John", 10000, 101, "john@accenture.com");
        assertEquals(11000, employeeService.incrementSalary(emp, 1000));
    }

    @Test
    @DisplayName("Tax Calculation")
    public void testComputeSalary() {
        Employee emp = new Employee(1001, "John", 10000, 101, "john@accenture.com");
        assertEquals(9000, employeeService.computeSalary(emp, 10));
    }
}

```

File: Demo1001_Junit/com/accenture/ltt/junit/EmployeeServiceTest07.java

```

package com.accenture.ltt.junit.test;

import static org.junit.Assert.assertNotNull;
import static org.junit.Assert.assertTrue;
import static org.junit.jupiter.api.Assertions.assertEquals;

import java.util.ArrayList;
import java.util.List;

import org.junit.jupiter.api.AfterAll;
import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.BeforeAll;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.TestInfo;
import org.junit.jupiter.params.ParameterizedTest;

```

```

import org.junit.jupiter.params.provider.CsvFileSource;
import org.junit.jupiter.params.provider.CsvSource;
import org.junit.jupiter.params.provider.EnumSource;
import org.junit.jupiter.params.provider.MethodSource;
import org.junit.jupiter.params.provider.ValueSource;

import com.accenture.ltt.businessbean.Employee;
import com.accenture.ltt.service.EmployeeService;

class EmployeeServiceTest07 {

    private static EmployeeService employeeService;

    @BeforeAll
    static void setup() {
        System.out.println("Setting up EmployeeService");
        employeeService = new EmployeeService();
    }

    @AfterAll
    static void tearDown() {
        System.out.println("Tearing Down EmployeeService");
        employeeService = null;
    }

    @BeforeEach
    void startTest(TestInfo testInfo) {
        System.out.println("Start Executing Test..." + testInfo.getDisplayName());
    }

    @AfterEach
    void endTest(TestInfo testInfo) {
        System.out.println("Completed Test..." + testInfo.getDisplayName());
    }

    @ParameterizedTest
    @ValueSource(ints = {1000, 2000, 3000, 4000})
    @DisplayName("Increment Salary")
    public void testIncrementSalary1(int value) {
        Employee emp = new Employee(1001, "John", 10000, 101, "john@accenture.com");
        int expected = emp.getSalary() + value;
        assertEquals(expected, employeeService.incrementSalary(emp, value));
    }

    @ParameterizedTest
    @ValueSource(strings = {"John", "Jack", "Joe", "Jim"})
    @DisplayName("Employee Name")
    public void testEmployeeName(String value) {
        assertTrue(value.length() >= 3);
    }

    enum DepartmentName{
        LKM(101),

```

```

        HR(102),
        FINANCE(103);

        private int departmentCode;
        private DepartmentName(int departmentCode) {
            this.departmentCode = departmentCode;
        }
        public int getDepartmentCode() {
            return departmentCode;
        }
        public void setDepartmentCode(int departmentCode) {
            this.departmentCode = departmentCode;
        }
    }

    @ParameterizedTest
    @EnumSource(DepartmentName.class)
    @DisplayName("Enum Test")
    public void departmentNameTest(DepartmentName departmentName) {
        assertNotNull(departmentName);
        assertTrue(departmentName.getDepartmentCode() > 100);
        System.out.println(departmentName);
        System.out.println(departmentName.getDepartmentCode());
    }

    static List<Employee> getEmployeeListData(){
        List<Employee> empList = new ArrayList<Employee>();
        empList.add(new Employee(1001, "John", 10000, 101, "john@accenture.com"));
        empList.add(new Employee(1002, "Jack", 20000, 102, "jack@accenture.com"));
        empList.add(new Employee(1003, "Tim", 13000, 101, "tim@accenture.com"));
        return empList;
    }

    @ParameterizedTest
    @MethodSource("getEmployeeListData")
    @DisplayName("Employees")
    public void testEmployee(Employee value) {
        assertNotNull(value);
    }

    @ParameterizedTest
    @CsvSource(value = { "1001,John,10000,101,john@accenture.com", "1002,Jack,20000,102,jack@accenture.com" })
    @DisplayName("CSV Test")
    public void defaultDelimiterCsvTest(int empId, String empName, int salary, int departmentCode, String email) {
        assertTrue(empId > 1000);
        assertNotNull(empName);
        assertTrue(salary >= 10000);
        assertTrue(departmentCode > 100);
        assertNotNull(email);
    }

    @ParameterizedTest
    @CsvSource(value = { "1001-John-10000-101-john@accenture.com", "1002-Jack-20000-102-jack@accenture.com" })

```

```

        delimiter='-')
@DisplayName("CSV Test")
public void customDelimiterCsvTest(int empId, String empName, int salary, int departmentCode) {
    assertTrue(empId > 1000);
    assertNotNull(empName);
    assertTrue(salary >= 10000);
    assertTrue(departmentCode > 100);
    assertNotNull(email);
}

@ParameterizedTest
@CsvFileSource(resources = "/resources/employees.csv")
@DisplayName("CSV file test")
public void csvFileTest(int empId, String empName, int salary, int departmentCode, String email) {
    assertTrue(empId > 1000);
    assertNotNull(empName);
    assertTrue(salary >= 10000);
    assertTrue(departmentCode > 100);
    assertNotNull(email);
}
}

```

File: Demo1001_Junit/com/accenture/ltt/junit/EmployeeServiceTest08.java

```

package com.accenture.ltt.junit.test;

import static org.junit.Assert.assertNotNull;
import static org.junit.Assert.assertTrue;
import static org.junit.jupiter.api.Assertions.assertEquals;

import java.util.ArrayList;
import java.util.List;

import org.junit.jupiter.api.AfterAll;
import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.BeforeAll;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Tag;
import org.junit.jupiter.api.TestInfo;
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.CsvFileSource;
import org.junit.jupiter.params.provider.CsvSource;
import org.junit.jupiter.params.provider.EnumSource;
import org.junit.jupiter.params.provider.MethodSource;
import org.junit.jupiter.params.provider.ValueSource;

import com.accenture.ltt.businessbean.Employee;
import com.accenture.ltt.service.EmployeeService;

class EmployeeServiceTest08 {

    private static EmployeeService employeeService;
}

```

```

@BeforeAll
static void setup() {
    System.out.println("Setting up EmployeeService");
    employeeService = new EmployeeService();
}

@AfterAll
static void tearDwon() {
    System.out.println("Tearing Down EmployeeService");
    employeeService = null;
}

@BeforeEach
void startTest(TestInfo testInfo) {
    System.out.println("Start Executing Test..." + testInfo.getDisplayName());
}

@AfterEach
void endTest(TestInfo testInfo) {
    System.out.println("Completed Test..." + testInfo.getDisplayName());
}

@Tag("group1")
@ParameterizedTest
@ValueSource(ints = {1000, 2000, 3000, 4000})
@DisplayName("Increment Salary")
public void testIncrementSalary1(int value) {
    Employee emp = new Employee(1001, "John", 10000, 101, "john@accenture.com");
    int expected = emp.getSalary() + value;
    assertEquals(expected, employeeService.incrementSalary(emp, value));
}

@Tag("group1")
@ParameterizedTest
@ValueSource(strings = {"John", "Jack", "Joe", "Jim"})
@DisplayName("Employee Name")
public void testEmployeeName(String value) {
    assertTrue(value.length() >= 3);
}

enum DepartmentName{
    LKM(101),
    HR(102),
    FINANCE(103);

    private int departmentCode;
    private DepartmentName(int departmentCode) {
        this.departmentCode = departmentCode;
    }
    public int getDepartmentCode() {
        return departmentCode;
    }
    public void setDepartmentCode(int departmentCode) {

```

```

        this.departmentCode = departmentCode;
    }
}

@Tag("group2")
@ParameterizedTest
@EnumSource(DepartmentName.class)
@DisplayName("Enum Test")
public void departmentNameTest(DepartmentName departmentName) {
    assertNotNull(departmentName);
    assertTrue(departmentName.getDepartmentCode() > 100);
    System.out.println(departmentName);
    System.out.println(departmentName.getDepartmentCode());
}

static List<Employee> getEmployeeListData(){
    List<Employee> empList = new ArrayList<Employee>();
    empList.add(new Employee(1001, "John", 10000, 101, "john@accenture.com"));
    empList.add(new Employee(1002, "Jack", 20000, 102, "jack@accenture.com"));
    empList.add(new Employee(1003, "Tim", 13000, 101, "tim@accenture.com"));
    return empList;
}

@Tag("group1")
@ParameterizedTest
@MethodSource("getEmployeeListData")
@DisplayName("Employees")
public void testEmployee(Employee value) {
    assertNotNull(value);
}

@Tag("group2")
@ParameterizedTest
@CsvSource(value = { "1001 John 10000 101 john@accenture.com", "1002 Jack 20000 102 jack@accenture.com" })
@DisplayName("CSV Test")
public void simpleCsv(int empId, String empName, int salary, int departmentCode, String email) {
    assertTrue(empId > 1000);
    assertNotNull(empName);
    assertTrue(salary >= 10000);
    assertTrue(departmentCode > 100);
    assertNotNull(email);
}

@Tag("group2")
@ParameterizedTest
@CsvFileSource(resources = "/resources/employees.csv")
@DisplayName("CSV file test")
public void simpleCsvFile(int empId, String empName, int salary, int departmentCode, String email) {
    assertTrue(empId > 1000);
    assertNotNull(empName);
    assertTrue(salary >= 10000);
    assertTrue(departmentCode > 100);
    assertNotNull(email);
}

```

```
}  
}
```

File: Demo1001_Junit/com/accenture/ltt/junit/EmployeeServiceTest09.java

```
package com.accenture.ltt.junit.test;  
  
import java.time.Duration;  
  
import org.junit.jupiter.api.AfterAll;  
import org.junit.jupiter.api.AfterEach;  
import org.junit.jupiter.api.Assertions;  
import org.junit.jupiter.api.BeforeAll;  
import org.junit.jupiter.api.BeforeEach;  
import org.junit.jupiter.api.DisplayName;  
import org.junit.jupiter.api.Test;  
import org.junit.jupiter.api.TestInfo;  
  
import com.accenture.ltt.buisnessbean.Employee;  
import com.accenture.ltt.exceptions.InvalidEmailAddressException;  
import com.accenture.ltt.service.EmployeeService;  
  
class EmployeeServiceTest09 {  
  
    private static EmployeeService employeeService;  
  
    @BeforeAll  
    static void setup() {  
        System.out.println("Setting up EmployeeService");  
        employeeService = new EmployeeService();  
    }  
  
    @AfterAll  
    static void tearDwon() {  
        System.out.println("Tearing Down EmployeeService");  
        employeeService = null;  
    }  
  
    @BeforeEach  
    void startTest(TestInfo testInfo) {  
        System.out.println("Start Executing Test..." + testInfo.getDisplayName());  
    }  
  
    @AfterEach  
    void endTest(TestInfo testInfo) {  
        System.out.println("Completed Test..." + testInfo.getDisplayName());  
    }  
  
    @Test  
    @DisplayName("Assertions Test")  
    public void testAssertions() {  
        Employee emp = new Employee(1001, "John", 10000, 101, "john");  
        // Employee emp1 = emp;  
        // Assertions.assertSame(emp, emp1);  
    }  
}
```



```

//      Employee emp2 = new Employee(1001, "John", 10000, 101, "john");
//      Assertions.assertNotSame(emp, emp2);

//      Assertions.assertEquals(10000, employeeService.computeSalary(emp, 10));
//      Assertions.assertEquals(10000, employeeService.computeSalary(emp, 10), "values are not");

//      Assertions.assertTrue(emp.getSalary() > 10000);
//      Assertions.assertTrue(emp.getSalary() > 10000, "Salary is not greater than 10000");

//      Assertions.assertAll("employee",
//          ()->Assertions.assertEquals(9000, employeeService.computeSalary(emp, 10)),
//          ()->Assertions.assertEquals(12000, employeeService.incrementSalary(emp, 2000)))

//      Throwable exception1 = Assertions.assertThrows(ArithmeticException.class,
//          () -> employeeService.computeSalary(emp, 0));
//      Assertions.assertEquals("/ by zero", exception1.getMessage());

//      Throwable exception2 = Assertions.assertThrows(InvalidEmailAddressException.class,
//          () -> employeeService.validateEmailAddress(emp));
//      Assertions.assertEquals("Employee's Email Address is invalid", exception2.getMessage())

      Assertions.assertTimeout(Duration.ofSeconds(2), () -> employeeService.getEmployeeDetails(emp));
    }
}

```

File: Demo1001_Junit/com/accenture/ltt/junit/EmployeeServiceTest10.java

```

package com.accenture.ltt.junit.test;

import static org.junit.Assume.assumeFalse;
import static org.junit.Assume.assumeTrue;
import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assumptions.assumingThat;

import org.junit.jupiter.api.AfterAll;
import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.BeforeAll;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.TestInfo;

import com.accenture.ltt.businessbean.Employee;
import com.accenture.ltt.service.EmployeeService;

class EmployeeServiceTest10 {

    private static EmployeeService employeeService;

    @BeforeAll
    static void setup() {
        System.out.println("Setting up EmployeeService");
        employeeService = new EmployeeService();
    }
}

```

```

}

@AfterAll
static void tearDown() {
    System.out.println("Tearing Down EmployeeService");
    employeeService = null;
}

@BeforeEach
void startTest(TestInfo testInfo) {
    System.out.println("Start Executing Test..." + testInfo.getDisplayName());
}

@AfterEach
void endTest(TestInfo testInfo) {
    System.out.println("Completed Test..." + testInfo.getDisplayName());
}

@Test
@DisplayName("Increment Salary")
public void testIncrementSalary() {
    Employee emp = new Employee(1001, "John", 10000, 101, "john@accenture.com");
    assumeFalse(emp.getSalary() > 10000);
    assertEquals(11000, employeeService.incrementSalary(emp, 1000));
}

@Test
@DisplayName("Tax Calculation")
public void testComputeSalary() {
    Employee emp = new Employee(1001, "John", 10000, 101, "john@accenture.com");
    assumeTrue(emp.getSalary() > 10000);
    assertEquals(90000, employeeService.computeSalary(emp, 10));
}

@Test
@DisplayName("Assuming That")
public void testAssumingThat() {
    Employee emp = new Employee(1001, "John", 10000, 101, "john@accenture.com");
    assumingThat(emp.getSalary() > 10000, () -> {
        System.out.println("Increment Salary");
        employeeService.incrementSalary(emp, 1000);
    });
    assumingThat(emp.getSalary() >= 10000, () -> {
        System.out.println("Tax computation");
        employeeService.computeSalary(emp, 10);
    });
}
}

```

17.6 Viva Voce — Likely Questions & Answers

Q1. What are the three sub-projects that make up JUnit 5, and what does each do? A: JUnit Platform (the foundation that discovers/launches tests for build tools and IDEs), JUnit

Jupiter (the new programming model you write tests against — `@Test`, assertions, etc.), and JUnit Vintage (a compatibility layer that runs legacy JUnit 3/4 tests on the new platform).

Q2. What is the difference between `@BeforeEach` and `@BeforeAll`? A: `@BeforeEach` runs before *every individual* test method (commonly used to reset test data, keeping tests isolated from each other). `@BeforeAll` runs *once*, before *any* test method in the class, and must be static — used for expensive, class-wide one-time setup.

Q3. How do you properly test that a method throws an expected exception in JUnit 5? A: Using `assertThrows(ExceptionType.class, () -> methodCall())`, which both verifies the exception type was actually thrown and can capture the thrown exception object for further assertions (e.g., on its message) — far cleaner and less error-prone than a manual try/catch with a `fail()` call if no exception occurs.

Q4. What is the purpose of `assertAll()`, and how does it differ from writing several separate assertion statements in a row? A: `assertAll()` groups multiple assertions so that *all* of them execute and get reported, even if an earlier one fails. Separate sequential assertion statements would stop the test at the *first* failure, hiding whether any subsequent assertions would also have failed.

Q5. What does `Assumptions.assumeTrue()` do, and how does a skipped test differ from a failed one? A: It causes the test to be skipped entirely (not run at all) if the given condition is false — useful for environment-dependent tests. A skipped test is reported separately from a failure; it represents “this test wasn’t applicable here”, not “this test ran and found a problem”.

Q6. Why is resetting test data in `@BeforeEach` (rather than sharing one mutable object across all tests in a class) important for test quality? A: It keeps tests properly isolated — without a fresh setup each time, one test’s side effects on shared state could silently affect whether a later test passes or fails, making the test suite order-dependent and failures much harder to diagnose.

Q7. What does JUnit Vintage allow you to do? A: It lets existing JUnit 3 and JUnit 4 tests run *on* the newer JUnit Platform infrastructure unmodified, enabling teams to migrate a large legacy test suite to JUnit 5 gradually rather than all at once.

Chapter 18: Performance Monitoring

(No demo Java project was supplied for this topic — like Code Quality, this module is primarily a hands-on tool walkthrough. The theory below reflects the classroom content in more depth.)

18.1 Why Monitor Performance?

Functionally-correct code can still be a poor production citizen if it leaks memory, spikes CPU unpredictably, or responds too slowly under real-world load. Performance monitoring tools let you **observe** a running JVM's actual behaviour rather than guess at the cause of a problem, turning vague complaints like “the app feels slow” into concrete, measurable evidence — e.g., “this specific method consumes 40% of CPU time”, or “this particular object type is never released and its count grows unboundedly over 24 hours”. Diagnosing performance issues *without* this kind of tooling usually devolves into unproductive guesswork and premature “optimizations” that may not even target the real bottleneck.

18.2 VisualVM — Monitoring & Profiling

VisualVM is a free, GUI-based tool bundled with (or easily downloadable alongside) the JDK that attaches directly to a running JVM process and shows, live, in real time:

- **CPU usage and thread activity** over time, as a live graph.
- **Heap memory usage** — its growth pattern over time is often the very first hint of a memory leak: a healthy application's heap usage should rise and then fall back down repeatedly as garbage collection reclaims short-lived objects (a classic “sawtooth” pattern); a heap that only ever climbs, never meaningfully dropping back down even after GC runs, strongly suggests objects are being unintentionally retained forever.
- **Thread states** (Running, Waiting, Blocked, etc.) — useful for spotting **deadlocks** (two or more threads each waiting on a resource the other holds, neither able to proceed) or excessive lock contention/blocking that's silently throttling throughput.
- **Profiling** — sampling (periodically checking which method each thread is currently executing) or instrumenting (inserting timing code directly) which methods actually consume the most CPU time or allocate the most objects, so optimization effort is spent precisely where it matters, rather than on guesswork or “premature optimization” of code that was never actually a bottleneck in the first place.

18.3 Eclipse Memory Analyzer Tool (MAT)

MAT analyzes a **heap dump** — a complete snapshot of literally every object present in the JVM's memory at one specific point in time — entirely **offline** (after the fact, not while the application is still running), to answer questions like: what objects exist and how many of each, how much memory does each object (and everything it, in turn, keeps alive) actually **retain**, and — most critically for genuine leak diagnosis — what is the **GC root chain** (the exact reference path from a garbage-collection root, like a static field or an active thread, all the way down to the suspect object) that is keeping a particular object alive when it clearly *shouldn't* still be reachable. This “why is this object still alive” root-chain analysis is precisely what distinguishes MAT's offline heap-dump analysis from VisualVM's simpler live-monitoring view — MAT answers not just *that* a leak exists, but *exactly why*, down to the specific reference holding it.

18.4 Load Testing with JMeter

Apache JMeter simulates many concurrent (often virtual/simulated) users hitting an application simultaneously, to measure how the system behaves realistically **under load**: response

times, throughput (requests handled per second), and error rates, **as load progressively increases**. This kind of testing surfaces an entire category of problems that simply never appear when testing with a single developer manually clicking through the UI — for example, a database connection pool that functions perfectly fine at 10 concurrent users but becomes completely exhausted (causing requests to queue or fail) at 500 concurrent users; or a caching layer that helps enormously at low traffic but becomes a lock-contention bottleneck itself at high concurrency.

18.5 Code Profiling in Java

Beyond ad-hoc, manually-triggered VisualVM sampling sessions, dedicated commercial Java profilers (e.g., JProfiler, YourKit) or the JDK's own built-in **Java Flight Recorder (JFR)** can attach with much lower overhead and provide finer-grained, genuinely **production-safe continuous** profiling — capturing exactly which lines/methods dominate CPU time or allocate the most memory, over long running periods (hours, days) rather than only short, manually-initiated debugging sessions, without meaningfully impacting the monitored application's own performance while doing so.

18.6 Viva Voce — Likely Questions & Answers

Q1. What does a “sawtooth” heap-usage pattern in VisualVM typically indicate, versus a continuously climbing pattern? A: A sawtooth pattern (heap rising, then dropping sharply as garbage collection reclaims short-lived objects, repeatedly) is normal, healthy behaviour. A pattern where heap usage only ever climbs and never meaningfully drops back down, even after GC runs, is a strong indicator of a memory leak — objects are being unintentionally kept referenced (alive) forever.

Q2. What is a heap dump, and what tool is typically used to analyze one? A: A heap dump is a complete snapshot of every object present in JVM memory at one specific instant in time. Eclipse Memory Analyzer Tool (MAT) is the typical tool used to analyze such a dump *offline* (after capture), to find memory leaks and understand exactly what's keeping specific objects alive.

Q3. What specifically does MAT's “GC root chain” analysis tell you that a simple object count does not? A: It shows the exact reference path from a garbage-collection root (e.g., a static field, an active thread) all the way down to a suspect object — answering *why* that object is still reachable/alive, not merely *that* many instances of it currently exist.

Q4. What is JMeter used for, and what kinds of problems does load testing surface that manual, single-user testing cannot? A: JMeter simulates many concurrent users hitting an application to measure response time, throughput, and error rate under realistic load. It surfaces problems specific to concurrency and scale — like a connection pool or cache that works fine for one user but becomes a bottleneck or fails entirely once hundreds of users hit the system simultaneously.

Q5. What is the practical difference between VisualVM-style ad-hoc profiling and a continuous profiler like Java Flight Recorder (JFR)? A: VisualVM sampling is typically a manually-initiated, relatively short debugging session. JFR (and similar continuous profilers) are designed to run with very low overhead *continuously*, even in production, over long time periods (hours or days), capturing detailed CPU/allocation data without materially affecting the monitored application's own performance.

Q6. Why is guessing at performance bottlenecks without monitoring tools considered risky or wasteful? A: Without concrete measurement, “optimization” effort often targets code that was never actually the real bottleneck (a form of premature optimization) —

monitoring tools convert vague symptoms (“the app feels slow”) into precise, actionable evidence about which specific method, object type, or resource is actually responsible, ensuring effort is spent where it will genuinely help.

Java Full Stack Training

SOLID Principles to JPA with Hibernate ORM — Extended Theory, Concepts
& Demo Programs (Viva / Interview Preparation Edition)

Accenture QuantumCrew Batch — Day-wise Training Reference (Continuation Volume —

Chapter 19

SOLID Principles

19.1 Why Design Principles Matter

Working code is not the same thing as *well-designed* code. A class can compile, pass every test, and still be a liability six months later — because every new requirement forces you to reopen it, re-test everything that touches it, and risk breaking behaviour that used to work. SOLID is a set of five design guidelines, popularised by Robert C. Martin (“Uncle Bob”) in the early 2000s, that describe the properties of a codebase which stays easy to change as it grows. Each letter names one principle:

Letter	Principle	One-line idea
S	Single Responsibility Principle (SRP)	A class should have only one reason to change
O	Open/Closed Principle (OCP)	Open for extension, closed for modification
L	Liskov Substitution Principle (LSP)	Subtypes must be substitutable for their base types
I	Interface Segregation Principle (ISP)	Don’t force a class to depend on methods it doesn’t use
D	Dependency Inversion Principle (DIP)	Depend on abstractions, not concrete implementations

None of these are laws enforced by the compiler — they are design habits. Violating one won’t produce a compile error or even a runtime error immediately; the cost shows up later, as **rigidity** (a small change ripples everywhere), **fragility** (a change in one place breaks something unrelated), and **immobility** (a component can’t be reused because it’s tangled up with things it doesn’t need). Every demo in this chapter deliberately shows a “Problem” version that compiles and runs, followed by a “Solution” version, so the difference is about design quality, not correctness in the narrow sense.

19.2 Coupling and Cohesion — the Vocabulary Behind SOLID

Before looking at each principle individually, it helps to have two words that describe *why* a design feels good or bad in the first place — every one of the five SOLID principles is really just a specific, actionable way of improving one or both of these two properties.

Cohesion measures how closely the responsibilities *inside* a single class belong together. A class with **high cohesion** does one well-defined thing, and every field and method in it exists to support that one thing. A class with **low cohesion** is a grab-bag — it does several unrelated things simply because they happened to be convenient to put in the same place. The Employee class in the SRP problem demo below (calculating pay, generating reports, *and* saving to a database) is a textbook example of low cohesion — those three responsibilities have nothing to do with one another beyond both happening to involve an employee.

Coupling measures how much one class *depends on the internal details of* another class. **Tight coupling** means a change to one class is likely to force a change in another — for example, a class that directly instantiates another class with new SomeConcreteClass() is tightly coupled to that specific implementation. **Loose coupling** means classes interact only through stable, narrow contracts (interfaces), so one side can change its internal implementation freely without breaking the other side.

The goal of good object-oriented design, in one sentence, is: **maximise cohesion within each class, and minimise coupling between classes**. With that in mind, here is how each SOLID principle maps onto these two ideas — worth memorising as a quick mental model before studying the individual principles in depth:

Principle	Primarily improves
SRP	Cohesion — keeps each class focused on one responsibility
OCP	Coupling — new behaviour is added without touching (and re-coupling to) existing code
LSP	Coupling — client code stays safely coupled only to a base type's <i>contract</i> , not a specific subtype's quirks
ISP	Coupling — a class is only coupled to the methods it actually uses, not an entire fat interface
DIP	Coupling — high-level code is coupled to a stable abstraction, not a volatile concrete class

Beginners often ask “why does this matter if the code works?” — the honest answer is that tight coupling and low cohesion don't break anything *today*; they make the *next* change slower and riskier, which is exactly the kind of cost that's invisible in a

training exercise but very visible in a codebase that's been evolving in production for two years.

19.3 Single Responsibility Principle (SRP)

Definition: A class should have one, and only one, reason to change. “Responsibility” here means an axis of change — a category of requirement that could evolve independently of the others. If a class bundles two responsibilities, then a change driven by one requirement (say, a new report format) forces you to touch and re-test a class that also handles something completely unrelated (say, database persistence).

A very common violation is an entity-like class that has silently grown “manager” methods — calculation logic, reporting logic, and persistence logic — all inside itself, simply because it was convenient to add them there at the time.

Problem — everything crammed into one class:

File: Demo1001_Solid_Principles_SRP/src/com/accenture/ltt/SRPDemoProblem.java

```
package com.accenture.ltt;
//This Employee class is violating SRP
//Because it handles Salary Calculation, Reporting, and Database operations
class Employee {

    //Salary related responsibility
    public void calculatePay() {
        System.out.println("Calculating employee pay...");
    }
    //Reporting logic should not be in Employee class
    public void reportEmployee() {
        System.out.println("Generating employee report...");
    }
    //Database logic also should not be in Employee class
    public void saveToDatabase() {
        System.out.println("Saving employee details to database...");
    }
}

public class SRPDemoProblem {
    public static void main(String[] args) {
        Employee emp = new Employee();
        //Employee class doing too many task
        emp.calculatePay();
        emp.reportEmployee();
        emp.saveToDatabase();
    }
}
```

Why this is a problem: Employee now changes for three completely unrelated rea-

sons — a change to the pay-calculation formula, a change to the report layout, and a change in how records are persisted (say, moving from JDBC to JPA) all touch the *same* class. A developer working on the reporting format has to open a file that also contains salary math and SQL — and any mistake there risks breaking the other two responsibilities that they never intended to touch.

Solution — one responsibility per class:

File: Demo1001_Solid_Principles_SRP/src/com/accenture/ltt/solution/SRPDemoSolution.java

```
package com.accenture.ltt.solution;

//This class now only handles Salary related responsibility
class Employee {

    public void calculatePay() {
        System.out.println("Calculating employee pay...");
    }

}

//Separate class to handle reporting duty
class EmployeeReport {
    public void generateReport() {
        System.out.println("Generating employee report...");
    }
}

//Separate class for database operations
class EmployeeRepository {

    public void saveToDatabase() {
        System.out.println("Saving employee details to database...");
    }
}

public class SRPDemoSolution {

    public static void main(String[] args) {
        //Each class has only one responsibility
        Employee employee = new Employee();
        EmployeeReport report = new EmployeeReport();
        EmployeeRepository repository = new EmployeeRepository();

        employee.calculatePay();           // Handles only salary
        report.generateReport();           // Handles reporting
        repository.saveToDatabase();       // Handles DB tasks
    }
}
```

```
}
```

Now `Employee` changes only when payroll logic changes; `EmployeeReport` changes only when the reporting format changes; `EmployeeRepository` changes only when the persistence mechanism changes. This is exactly the DAO/Service/Entity layering you will see formalised in the JDBC and JPA chapters later in this volume — SRP is the underlying reason that architecture exists at all.

Key nuance for viva: SRP is not “one method per class” — a class can (and usually should) have several methods, as long as they all serve the *same* responsibility. `Employee.calculatePay()` could grow to include overtime and bonus calculation without violating SRP, because all of that is still “salary responsibility.”

19.4 Open/Closed Principle (OCP)

Definition: Software entities (classes, modules, functions) should be **open for extension** but **closed for modification**. In practice this means: when a new requirement arrives, you should be able to add new behaviour by *adding new code* (a new subclass, a new implementation), not by editing and re-testing code that already works.

The supplied demo illustrates OCP through a closely related lens — **encapsulation as the enabling mechanism for OCP**. When a class exposes its internal fields directly (`public double length`), any external code can set them to invalid values, and the validation/behaviour logic ends up scattered wherever the fields are used — which means every new usage site is a new place that can violate the invariant, and every fix has to be repeated at every call site instead of once, inside the class.

Problem — public fields, no encapsulation, no protection:

File: `Demo1002_Solid_Principles_OCP/src/com/accenture/ltt/OCPDemoProblem.java`

```
package com.accenture.ltt;

class Rectangle {
    public double length;
    public double width;
}

class Circle {
    public double radius;
}

// This violates Encapsulation – Object data is exposed directly.
public class OCPDemoProblem {

    public static void main(String[] args) {
        Rectangle r = new Rectangle();
        r.length = -10; // Accepted but invalid
        r.width = 5;
    }
}
```

```
Circle c = new Circle();
c.radius = 7;

System.out.println("Rectangle Area: " + (r.length * r.width));
System.out.println("Circle Area: " + (3.14 * c.radius * c.radius));
// Data is unprotected - anyone can change it
// No validation logic -> Incorrect results possible
// Violates Encapsulation principle of OOP
// Area calculation logic scattered -> Hard to maintain
}

}
```

Solution — encapsulated, validated, and extensible without modification:

File: Demo1002_Solid_Principles_OCP/src/com/accenture/ltt/solution/OCPDemoSolution.java

```
package com.accenture.ltt.solution;
//Encapsulation applied properly
class Circle {

    private double radius;

    //Validation ensures only valid values are set
    public Circle(double radius) {
        if(radius <= 0) {
            throw new IllegalArgumentException("Invalid radius!");
        }
        this.radius = radius;
    }
    //Behavior belongs inside the class
    public double calculateArea() {
        return Math.PI * radius * radius;
    }
}
//Encapsulation applied
class Rectangle {

    private double length;
    private double width;

    //Constructor enforces valid data at object creation
    public Rectangle(double length, double width) {
        if(length <= 0 || width <= 0) {
            throw new IllegalArgumentException("Invalid rectangle dimensions!");
        }
        this.length = length;
        this.width = width;
    }
}
```

```

    }
    //Logic encapsulated -> no duplication
    public double calculateArea() {
        return length * width;
    }
}

public class OCPDemoSolution {

    public static void main(String[] args) {

        //Objects created with valid dimensions only
        Rectangle rectangle = new Rectangle(10, 5);
        Circle circle = new Circle(7);

        System.out.println("Rectangle Area: " + rectangle.calculateArea());
        System.out.println("Circle Area: " + circle.calculateArea());
    }
}

```

Connecting this back to OCP directly: because each shape now exposes a common shape of behaviour (`calculateArea()`), a new shape — Triangle, Hexagon — can be **added** as a brand-new class implementing that same contract, without modifying `Rectangle`, `Circle`, or any code that already consumes them. The system is closed for modification (existing, tested classes are untouched) yet open for extension (new classes can be introduced freely). The classic textbook version of this demo uses a common `Shape` interface with an abstract `calculateArea()` method and a single loop that sums areas polymorphically — worth knowing for viva, since interviewers often ask you to draw exactly that UML.

19.5 Liskov Substitution Principle (LSP)

Definition: Objects of a superclass should be replaceable with objects of a subclass without altering the correctness of the program. In other words, if B extends A, any code written against A must keep working — with the same expectations, the same guarantees, no surprises — even when it is secretly handed a B instance. LSP is a stricter test than “does it compile” — it asks whether the subclass **honours the contract** the base class promised, not just its method signatures.

Problem — a subclass that breaks the parent’s promise:

File: `Demo1003_Solid_Principles_LSP/src/com/accenture/ltt/problem/LSPDemoProblem.java`

```

package com.accenture.ltt.problem;
//Base Class
abstract class Account {
    protected double amount;
}

```

```

    public void deposit(double amt) {
        amount += amt;
    }
    // Parent promises every account supports withdraw
    public abstract void withdraw(double amt);
}
//Working Child Class
class SavingsAccount extends Account {
    private static final double MIN_BALANCE = 1000;

    @Override
    public void withdraw(double amt) {
        if ((amount - amt) < MIN_BALANCE) {
            throw new RuntimeException("Below minimum balance! ");
        }
        amount -= amt;
    }
}
//Problematic Child Class

class FixedTermDepositAccount extends Account {

    @Override
    public void withdraw(double amt) {
        // Parent allowed withdrawal but child COMPLETELY rejects it
        throw new RuntimeException("Withdrawal not allowed before maturity! ");
    }
}
//What goes Wrong
public class LSPDemoProblem {

    public static void main(String[] args) {
        Account account = new FixedTermDepositAccount(); // Valid substitution

        account.deposit(10000);

        account.withdraw(2000);
        // Runtime failure!
        // Because FD Account does not allow withdrawal
    }
}

```

Account declares `withdraw()` as something every account can do. Substituting a `FixedTermDepositAccount` wherever an `Account` is expected is legal *by the type system*, but it silently breaks the contract at runtime — any code written generically against `Account` (e.g. a loop that withdraws from a list of accounts) will blow up the mo-

ment it encounters an FD account, even though nothing in the type signature warned it.

Solution — restructure the hierarchy so the contract only promises what every subtype can actually deliver:

File: Demo1003_Solid_Principles_LSP/src/com/accenture/ltt/solution/LSPDemoSolution.java

```
package com.accenture.ltt.solution;
//Base Class
abstract class Account {
    protected double amount;

    public void deposit(double amt) {
        amount += amt;
    }

    public double getBalance() {
        return amount;
    }
}

// Only accounts that allow withdrawal extend this class
abstract class WithdrawableAccount extends Account
{
    public abstract boolean withdraw(double amt);
}

class SavingsAccount extends WithdrawableAccount {
    private static final double MIN_BALANCE = 1000;

    @Override
    public boolean withdraw(double amt) {
        if ((amount - amt) >= MIN_BALANCE) {
            amount -= amt;
            return true;
        }
        return false;
    }
}

class FixedTermDepositAccount extends Account {
    // No withdraw method – now expectations are clear
}

public class LSPDemoSolution {

    public static void main(String[] args) {
        WithdrawableAccount sa = new SavingsAccount();
    }
}
```



```

        sa.deposit(5000);
        sa.withdraw(2000);
        System.out.println(sa.getBalance()); // Works

        Account fda = new FixedTermDepositAccount();
        fda.deposit(10000);
        // No withdrawal, no behavior conflict
        System.out.println(fda.getBalance());

    }
}

```

The fix does not “make withdraw() optional” or wrap it in a try/catch at every call site — it **removes the false promise from the base class**. Account now only promises what every account type can genuinely deliver (deposit, getBalance); the ability to withdraw is pulled into a separate WithdrawableAccount abstraction, so code that depends on withdrawal support declares that dependency explicitly (by coding against WithdrawableAccount, not Account), and the compiler — not a runtime exception — stops you from calling withdraw() on something that was never meant to support it.

Classic LSP viva example worth knowing: the Rectangle/Square problem. A Square that extends Rectangle and overrides setWidth()/setHeight() to always keep both sides equal *looks* like valid inheritance (“a square is a rectangle” is true geometrically), but it breaks any code that relies on setting width and height independently — proving that “is-a” in English does not automatically mean “is-a” in LSP terms.

19.6 Interface Segregation Principle (ISP)

Definition: No class should be forced to depend on (implement) methods it does not use. Large, “fat” interfaces that bundle unrelated behaviours force every implementer to provide dummy or exception-throwing implementations for the parts that don’t apply to them — which is a design smell, not a convenience.

Problem — one bloated interface forces irrelevant behaviour on every implementer:

File: Demo1004_Solid_Principles_ISP/src/com/accenture/ltt/problem/ISPDemoProblem.java

```

package com.accenture.ltt.problem;
//One Big Interface
interface Vehicle {

    void start();
    void stop();
    void fly();
}

```

```
// Airplane can fly - but okay here
class Airplane implements Vehicle {

    @Override
    public void start() {
        System.out.println("Airplane started");
    }

    @Override
    public void stop() {
        System.out.println("Airplane stopped");
    }

    @Override
    public void fly() {
        throw new UnsupportedOperationException("Airplane flying!!");
    }
}

//Car forced to implement fly() unnecessarily
class Car implements Vehicle {

    @Override
    public void start() {
        System.out.println("Car started");
    }

    @Override
    public void stop() {
        System.out.println("Car stopped");
    }

    @Override
    public void fly() {
        // Car cannot fly -> Wrong design!
        throw new UnsupportedOperationException("Car cannot fly!!");
    }
}

//This triggers exactly what ISP warns against:
//A class should not be forced to depend on methods it does not use

public class ISPDemoProblem {

    public static void main(String[] args) {
        Vehicle v = new Car(); // Valid substitution
        v.start();
    }
}
```

```
        v.fly(); // Runtime Exception -> Wrong interface design!
    }
}
```

Car is forced to provide a fly() method purely because the interface bundled it in — and that implementation can only ever throw an exception, which is itself a subtle LSP violation stacked on top of the ISP one (calling code that trusts the Vehicle contract can be handed a Car and call fly() on it, only to blow up at runtime).

Solution — split the fat interface into focused, role-specific interfaces:

File: Demo1004_Solid_Principles_ISP/src/com/accenture/ltt/solution/ISPDemoSolution.java

```
package com.accenture.ltt.solution;

//Split the big interface into small specific behavior interfaces
interface EngineVehicle {
    void start();
    void stop();
}

interface Flyable {
    void fly();
}

//Airplane implements both
class Airplane implements EngineVehicle, Flyable {

    @Override
    public void start() {
        System.out.println("Airplane started");
    }

    @Override
    public void stop() {
        System.out.println("Airplane stopped");
    }

    @Override
    public void fly() {
        System.out.println("Airplane flying");
    }
}

//Car now implements only what it needs
class Car implements EngineVehicle {
```

```
@Override
public void start() {
    System.out.println("Car started");
}

@Override
public void stop() {
    System.out.println("Car stopped");
}
}

public class ISPDemoSolution {

    public static void main(String[] args) {
        EngineVehicle car = new Car();
        car.start();
        car.stop();

        Flyable airplane = new Airplane();
        airplane.fly();

        EngineVehicle airplaneEngine = new Airplane();
        airplaneEngine.start(); // engine operations allowed
    }
}
```

Now Car only ever has to implement methods that are actually meaningful for a car; Airplane opts into Flyable because — and only because — it genuinely supports that behaviour. This is the same idea behind splitting a giant Repository interface into Readable/Writable, or a giant Employee interface into role-specific interfaces like Payable and Reportable — each client depends only on the slice of behaviour it actually needs.

19.7 Dependency Inversion Principle (DIP)

Definition: High-level modules should not depend on low-level modules; both should depend on abstractions. Abstractions should not depend on details; details should depend on abstractions. In plainer terms: the class that contains the important business logic (high-level) should never new up or hard-code a reference to a specific concrete implementation class (low-level) — it should depend on an interface, and have the concrete implementation *handed to it* from outside.

DIP is easy to confuse with simple “dependency” — every non-trivial class depends on something. DIP is specifically about **which direction the dependency points**: does the important, stable, business-rule class depend on volatile implementation

detail classes, or does everything depend on a shared, stable abstraction?

Problem — the high-level Project class is hard-wired to concrete developer classes:

File: Demo1005_Solid_Principles_DIP/src/com/accenture/ltt/problem/DIPDemoProblem.java

```
package com.accenture.ltt.problem;

//Low-level module
class BackEndDeveloper {
    public void writeJava() {
        System.out.println("Writing Java code...");
    }
}

//Low-level module
class FrontEndDeveloper {
    public void writeJavaScript() {
        System.out.println("Writing JavaScript code...");
    }
}

// High-level module directly depends on low-level modules
// Tight coupling - cannot add new developer types without modifying this class
class Project {

    private BackEndDeveloper backEndDeveloper = new BackEndDeveloper();
    private FrontEndDeveloper frontEndDeveloper = new FrontEndDeveloper();

    public void buildProject() {
        backEndDeveloper.writeJava();           // Strong dependency
        frontEndDeveloper.writeJavaScript();     // Strong dependency
    }
}

public class DIPDemoProblem {
    public static void main(String[] args) {
        Project project = new Project();
        project.buildProject();
    }
}
```

Project — the important, high-level coordinating class — directly instantiates two specific low-level classes and calls their specific method names. Adding a DatabaseDeveloper means opening and editing Project again (which also violates OCP), and Project cannot be unit-tested without dragging in real BackEndDeveloper/FrontEndDeveloper objects.

Solution — introduce a shared abstraction; Project depends only on that:

File: Demo1005_Solid_Principles_DIP/src/com/accenture/ltt/solution/DIPDemoSolution.java

```
package com.accenture.ltt.solution;

// Abstraction – Contract for all developer types
interface Developer {
    void develop();
}

// Low-level modules now depend on abstraction
class BackEndDeveloper implements Developer {
    public void develop() {
        System.out.println("Writing Java code...");
    }
}

class FrontEndDeveloper implements Developer {
    public void develop() {
        System.out.println("Writing JavaScript code...");
    }
}

//High-level module depends on abstraction, not concrete classes
class Project {

    private Developer[] developers;

    public Project(Developer[] developers) {
        this.developers = developers;
    }

    public void buildProject() {
        for (Developer dev : developers) {
            dev.develop();
        }
    }
}

public class DIPDemoSolution {
    public static void main(String[] args) {

        Developer[] team = {
            new BackEndDeveloper(),
            new FrontEndDeveloper()
        };

        Project project = new Project(team);
        project.buildProject(); // Works without knowing specific developer types
    }
}
```

```
}  
}
```

Notice the mechanism used to hand the dependency in: it is passed through the **constructor** (`new Project(team)`) rather than created inside `Project`. This is called **Dependency Injection**, and it is the standard *technique* used to satisfy the DIP *principle* — a distinction that is a favourite viva trap (see below). `Project` can now be given any array of `Developer` implementations — including a `MockDeveloper` for unit testing, or a `DatabaseDeveloper` added tomorrow — with zero changes to `Project` itself. This exact pattern (constructor-based dependency injection into a class that only knows about an interface) is precisely what Spring’s IoC container automates at scale, so a solid grip on this demo is directly transferable to Spring Boot’s `@Autowired` constructor injection later in a full-stack curriculum.

19.8 Practice Activities

1. Take the `SRPDemoProblem` class and refactor an `Invoice` class that currently calculates totals, formats a printable invoice, and saves itself to a file — split it into three SRP-compliant classes the way `Employee` was split in this chapter.
2. Extend the OCP Shape idea: add a `Triangle` class implementing the same shape contract used by `Rectangle`/`Circle`, and write a loop that computes the total area of a list of mixed shapes without any instanceof checks.
3. Identify an LSP violation in a `Bird` hierarchy where a `Penguin` subclass overrides `fly()` to throw an exception, and redesign the hierarchy the way `WithdrawableAccount` was introduced in this chapter.
4. Take the ISP Vehicle interface problem and, instead of `EngineVehicle`/`Flyable`, design segregated interfaces for a `MultiFunctionPrinter` scenario (`Printable`, `Scannable`, `Faxable`) so a basic printer only implements `Printable`.
5. Rewrite `DIPDemoProblem` for a `NotificationService` that currently hard-codes an `EmailSender` — introduce a `MessageSender` interface and inject the implementation via the constructor, then swap in an `SmsSender` without touching `NotificationService`.

19.9 Viva Voce — Likely Questions & Answers

Q1. What does SOLID stand for, and who introduced it? SOLID is an acronym for five object-oriented design principles — Single Responsibility, Open/Closed, Liskov Substitution, Interface Segregation, and Dependency Inversion — popularised by Robert C. Martin.

Q2. Is violating a SOLID principle a compile-time error? No. All five are design guidelines, not language rules. Violating them still compiles and often still runs correctly at first; the cost appears later as code that is harder to extend, more fragile to change, and harder to test in isolation.

Q3. What exactly does “responsibility” mean in SRP? A responsibility is an axis of change — a reason a class might need to be modified. If a class can be changed for

two unrelated reasons (e.g., a new report format vs. a new persistence mechanism), it has more than one responsibility and violates SRP.

Q4. Does SRP mean a class can only have one method? No — a class can have many methods as long as they all serve the same single responsibility. `Employee.calculatePay()` growing to handle bonus and overtime calculation is still SRP-compliant, because all of it is payroll responsibility.

Q5. How does encapsulation relate to the Open/Closed Principle? Encapsulating state (private fields with validated constructors/setters) keeps the “how” hidden inside the class, so behaviour can be extended by adding new classes that honour the same contract, without needing to modify or duplicate validation/calculation logic at every call site.

Q6. What is the practical test for whether a subclass satisfies LSP? Ask: can every piece of code written against the base class keep working, with the same expectations, if a subclass instance is substituted in? If the subclass throws where the base type promised success, weakens post-conditions, or strengthens pre-conditions, LSP is violated.

Q7. Why is the Square/Rectangle example a classic LSP violation even though “a square is a rectangle” is mathematically true? Because LSP is about behavioural substitutability, not geometric truth. A Square that forces width and height to always stay equal breaks the expectation (established by Rectangle) that width and height can be set independently — so code that resizes a Rectangle breaks when handed a Square.

Q8. What problem does ISP solve that SRP does not already cover? SRP is about a class having one reason to change. ISP is about an interface not forcing its implementers to depend on (and often stub out or throw exceptions from) methods that are irrelevant to them. A class can have a single responsibility and still be forced to implement unrelated methods if the interface it implements is too broad.

Q9. What is the difference between Dependency Inversion and Dependency Injection? Dependency Inversion is the *principle* — high-level modules should depend on abstractions, not concrete low-level classes. Dependency Injection is the *technique* commonly used to satisfy that principle — passing the concrete implementation in from outside (via constructor, setter, or a framework like Spring), rather than the class creating it internally with `new`.

Q10. In the DIP demo, why is the dependency passed through the constructor rather than created inside Project? So that Project never needs to know which concrete Developer implementations exist. This makes Project open to new developer types (OCP), testable with mock Developer implementations, and exactly mirrors how Spring’s IoC container injects beans into a class’s constructor.

Q11. Give a one-line real-world consequence for violating each SOLID principle. SRP violation -> a small change ripples into unrelated features. OCP violation -> adding a feature means editing tested code. LSP violation -> polymorphic code breaks unpredictably at runtime. ISP violation -> classes carry dead-weight methods and unnecessary exceptions. DIP violation -> code cannot be unit-tested or swapped without rewriting the high-level class.

Chapter 20

Design Patterns

20.1 What Are Design Patterns?

A design pattern is a **named, reusable solution to a recurring design problem** — not a finished piece of code you copy-paste, but a proven shape of a solution that you adapt to your situation. The value of learning patterns by name is communication: saying “let’s use a Strategy here” instantly conveys an entire design shape to another engineer who knows the pattern, the same way saying “binary search” conveys an algorithm without re-explaining it. The patterns catalogued by the “Gang of Four” (Erich Gamma, Richard Helm, Ralph Johnson, John Vlissides, in their 1994 book *Design Patterns: Elements of Reusable Object-Oriented Software*) are traditionally grouped into three families:

Family	Concerned with	Patterns in this chapter
Creational	<i>How</i> objects get created	Singleton, Factory Method, Builder, Prototype
Structural	How classes/objects are <i>composed</i> into larger structures	Adapter, Decorator, Facade
Behavioural	How objects <i>communicate</i> and distribute responsibility	Strategy, Command, Observer

This training’s demo set adds a fourth, more modern grouping specific to Java’s concurrency APIs — **Asynchronous patterns**, covered in §20.5, using `CompletableFuture`.

20.2 “Favour Composition Over Inheritance” — the Idea Behind Most Patterns

Before looking at individual patterns, it’s worth understanding one design preference that quietly shapes almost every pattern in this chapter: **composition is usually a safer tool than inheritance for reusing behaviour**.

Inheritance (“is-a”) lets a subclass reuse a parent’s code automatically, but it creates the *tightest* possible coupling between two classes — a subclass is exposed to every implementation detail of its parent, can only ever have one parent (in Java), and any change to the parent’s behaviour risks silently breaking every subclass (this is sometimes called the **fragile base class problem**). Chapter 19’s LSP discussion is really a special case of this: inheritance hierarchies are easy to build incorrectly, because “is-a” in plain English does not always mean “is-a” in a way that preserves behavioural correctness.

Composition (“has-a”) reuses behaviour by *holding a reference* to another object and delegating to it, rather than extending it. A class composed of other objects can change which object it holds at runtime, can hold several collaborators rather than being limited to one parent, and never risks breaking because of a change to a class it doesn’t directly control the internals of.

	Inheritance (“is-a”)	Composition (“has-a”)
Reuse mechanism	Extending a parent class	Holding a reference and delegating
Coupling	Tight — subclass depends on parent’s internals	Loose — depends only on the held object’s public contract
Flexibility	Fixed at compile time; one parent only	Can change at runtime; can hold multiple collaborators
Risk	Fragile base class problem; can silently violate LSP	Minimal — changes to the held object’s internals don’t leak out

Look back at the Decorator demo (§20.3.2) and the Strategy demo (§20.4.1) with this lens: both achieve flexible, swappable behaviour purely through composition (PizzaDecorator *holds* a Pizza; ShoppingCart *holds* a PaymentStrategy) rather than through subclassing. This is precisely why the Gang of Four’s own advice, and the guiding idea behind most of the patterns in this chapter, is: reach for inheritance only when there is a genuine, stable “is-a” relationship (as with CardPayment implements PaymentStrategy, where every card payment really is a kind of payment strategy), and reach for composition whenever you simply want to reuse or combine behaviour flexibly.

20.3 Creational Patterns

Creational patterns abstract away the details of *how* an object gets instantiated, so client code depends on *what* it needs, not on the concrete constructor calls needed to build it.

20.3.1 Singleton Pattern

Intent: Ensure a class has exactly one instance across the entire application, and provide a single, well-known global access point to it. Classic use cases: a database connection manager, a configuration/settings object, a logging service — anything where having two independent instances would be wasteful or would cause inconsistent state.

Mechanism: make the constructor private (so nobody outside the class can call new), keep a single static reference to the one instance, and expose a public static accessor method that creates the instance the first time it's asked for and returns the same one every time after.

File: Demo1001_CreationalDesignPatternsDemos/src/com/accenture/ltt/SingletonPatternDemo.

```
package com.accenture.ltt;
//Ensures only ONE instance of this class exists in entire application
// Provides a global point of access to that instance
class DatabaseConnection {

    // static instance - shared globally
    private static DatabaseConnection instance;

    // private constructor - Restricts creation from outside the class
    private DatabaseConnection() {}

    // Public method to return the same instance every time
    public static DatabaseConnection getInstance() {
        if (instance == null) {
            instance = new DatabaseConnection(); // Create only once
        }
        return instance;
    }

    public void connect() {
        System.out.println("Connected to DB...");
    }
}

public class SingletonPatternDemo {

    public static void main(String[] args) {

        // Both variables refer to EXACT same object
```

```

        DatabaseConnection db1 = DatabaseConnection.getInstance();
        DatabaseConnection db2 = DatabaseConnection.getInstance();

        System.out.println(db1 == db2); // true (same object)
        db1.connect();
    }
}

```

Because instance is only created the first time `getInstance()` is called, this is **lazy initialization**. Worth knowing for viva: this exact form is **not thread-safe** — two threads calling `getInstance()` at the same instant, before instance has been assigned, could both see null and both construct a separate object. The standard fixes are synchronizing the method (`public static synchronized DatabaseConnection getInstance()`), double-checked locking, or the simplest robust option in modern Java — an enum singleton or eager initialization (`private static final DatabaseConnection instance = new DatabaseConnection();`), which the JVM class-loading mechanism itself guarantees is thread-safe.

20.3.2 Builder Pattern

Intent: Construct a complex object step-by-step, avoiding a constructor with a huge number of parameters (the “telescoping constructor” problem, where you end up with overloaded constructors like `Pizza(base)`, `Pizza(base, cheese)`, `Pizza(base, cheese, toppings)`, and so on).

File: `Demo1001_CreationalDesignPatternsDemos/src/com/accenture/ltt/BuilderPatternDemo.java`

```

package com.accenture.ltt;
//Used to create complex objects step-by-step
//Avoids telescoping constructor (multiple parameter constructors)
class Pizza {
    private String base;
    private String cheese;
    private String toppings;

    // Private constructor - object built using Builder only
    private Pizza(Builder builder) {
        this.base = builder.base;
        this.cheese = builder.cheese;
        this.toppings = builder.toppings;
    }

    // Static inner Builder class
    static class Builder {
        private String base;
        private String cheese;
        private String toppings;
    }
}

```

```

        // Methods return Builder itself for chaining
        Builder setBase(String base) { this.base = base; return this; }
        Builder setCheese(String cheese) { this.cheese = cheese; return this; }
        Builder setToppings(String toppings) { this.toppings = toppings; return this; }

        // Final step: build the actual Pizza object
        Pizza build() { return new Pizza(this); }
    }
}

public class BuilderPatternDemo {
    public static void main(String[] args) {
        // Step-by-step construction
        Pizza pizza = new Pizza.Builder()
            .setBase("Thin Crust")
            .setCheese("Mozzarella")
            .setToppings("Mushrooms")
            .build();

        System.out.println("Pizza Ordered!");
    }
}

```

Each Builder method returns `this`, which is what enables the **fluent, chained** call style (`.setBase(...).setCheese(...).setToppings(...)`). The outer `Pizza` constructor is private, so a `Pizza` object can *only* be produced by going through the Builder — which also means you can freely add optional fields to the builder later without breaking any existing calling code, since callers only set the fields they care about.

20.3.3 Factory Method Pattern

Intent: Centralise object-creation logic behind a factory, so client code asks for “a payment method of this type” without knowing (or caring) which concrete class actually gets instantiated.

File: Demo1001_CreationalDesignPatternsDemos/src/com/accenture/ltt/FactoryMethodPatternD

```

package com.accenture.ltt;
//Helps in creating objects without exposing creation logic to the client
//Factory decides which object to create based on input
interface Payment {
    void pay(int amount);
}

class UpiPayment implements Payment {
    public void pay(int amount) {
        System.out.println("Paid via UPI: " + amount);
    }
}

```

```

}

class CardPayment implements Payment {
    public void pay(int amount) {
        System.out.println("Paid via Card: " + amount);
    }
}

//Factory class responsible for object creation logic
class PaymentFactory {
    public static Payment getPaymentMethod(String type) {
        if (type.equalsIgnoreCase("UPI")) return new UpiPayment();
        if (type.equalsIgnoreCase("CARD")) return new CardPayment();
        return null;
    }
}

public class FactoryMethodPatternDemo {
    // User doesn't know which class object is created internally
    public static void main(String[] args) {
        Payment payment = PaymentFactory.getPaymentMethod("CARD");
        payment.pay(500);
    }
}

```

The client (main) only ever talks to the Payment interface and a String type code — it never writes new CardPayment() itself. Adding a NetBankingPayment tomorrow means adding one if line inside PaymentFactory and one new class; it requires zero changes to any client code that already calls getPaymentMethod(...). Notice this pattern combines naturally with DIP from Chapter 19 — client code depends only on the Payment abstraction.

20.3.4 Prototype Pattern

Intent: Create new objects by *cloning* an existing, already-configured instance, instead of building one from scratch — valuable when object construction is expensive (e.g., involves a database read, heavy computation, or network access) and a copy is cheaper than re-creating it.

File: Demo1001_CreationalDesignPatternsDemos/src/com/accenture/ltt/PrototypePatternDemo.

```

package com.accenture.ltt;
//Create new objects by cloning an existing object
//Best when object creation is costly or time-consuming
class Document implements Cloneable {
    String content;

```

```

    Document(String content) {
        this.content = content;
    }
    // clone() returns a copy of current object
    public Document clone() throws CloneNotSupportedException {
        return (Document) super.clone();
    }
}

public class PrototypePatternDemo {
    public static void main(String[] args) throws Exception {
        Document doc1 = new Document("Original Content");

        // Create duplicate instead of re-creating from scratch
        Document doc2 = doc1.clone();

        System.out.println(doc2.content); // Same content cloned
    }
}

```

Cloneable is a marker interface — it carries no methods of its own, but signals to `Object.clone()` that cloning this class is permitted; without implementing `Cloneable`, calling `super.clone()` throws `CloneNotSupportedException` at runtime. Worth knowing for viva: `super.clone()` performs a **shallow copy** — if `Document` held a reference-type field like a `List<String> pages`, both the original and the clone would share the *same* list object after cloning, unless you explicitly deep-copy that field inside your own `clone()` override.

20.4 Structural Patterns

Structural patterns are about composing classes and objects into larger structures while keeping those structures flexible and efficient.

20.4.1 Adapter Pattern

Intent: Convert the interface of one class into another interface the client expects, allowing classes with incompatible interfaces to work together — without modifying either the client or the existing (adaptee) class.

File: Demo1002_StructuralDesignPatternDemos/src/com/accenture/ltt/AdapterPatternDemo.java

```

package com.accenture.ltt;
//Target interface expected by client
interface Charger {
    void charge();
}

//Adaptee class having different interface

```

```
class MicroUsbCharger {
    void plugMicroUsb() {
        System.out.println("Charging using Micro USB...");
    }
}

//Adapter makes Micro USB compatible with Charger interface
class UsbTypeCAdapter implements Charger {

    private MicroUsbCharger microUsb;

    public UsbTypeCAdapter(MicroUsbCharger microUsb) {
        this.microUsb = microUsb;
    }

    @Override
    public void charge() {
        // converting behavior of MicroUSB to USB-C
        microUsb.plugMicroUsb();
    }
}

public class AdapterPatternDemo {
    public static void main(String[] args) {
        MicroUsbCharger microUsb = new MicroUsbCharger();
        Charger charger = new UsbTypeCAdapter(microUsb);
        charger.charge(); // Works even though interfaces differ
    }
}
```

UsbTypeCAdapter implements the interface the client (Charger) expects, while internally holding a reference to and delegating to the incompatible MicroUsbCharger object. The old MicroUsbCharger class is never touched — this is the same “no change required in old classes” benefit that OCP asks for, achieved here through composition rather than inheritance. This pattern is exactly how many real-world integration layers work — e.g., wrapping a third-party payment gateway’s SDK behind your application’s own PaymentGateway interface, so switching vendors later means writing one new adapter, not rewriting the whole application.

20.4.2 Decorator Pattern

Intent: Attach additional responsibilities to an object dynamically, without altering its class or affecting other instances of the same class — an alternative to subclassing for extending behaviour.

File: Demo1002_StructuralDesignPatternDemos/src/com/accenture/ltt/DecoratorPatternDemo.j


```
package com.accenture.ltt;

//Component Interface
interface Pizza {
    String makePizza();
}

//Concrete Component
class BasicPizza implements Pizza {
    public String makePizza() {
        return "Pizza Base";
    }
}

//Base Decorator
class PizzaDecorator implements Pizza {
    protected Pizza pizza;

    public PizzaDecorator(Pizza pizza) {
        this.pizza = pizza;
    }

    public String makePizza() {
        return pizza.makePizza();
    }
}

//Concrete Decorators adding behavior
class CheeseDecorator extends PizzaDecorator {
    public CheeseDecorator(Pizza pizza) { super(pizza); }
    public String makePizza() {
        return super.makePizza() + " + Cheese";
    }
}

class OliveDecorator extends PizzaDecorator {
    public OliveDecorator(Pizza pizza) { super(pizza); }
    public String makePizza() {
        return super.makePizza() + " + Olives";
    }
}

public class DecoratorPatternDemo {
    public static void main(String[] args) {
        Pizza pizza = new OliveDecorator(new CheeseDecorator(new BasicPizza()));
        System.out.println(pizza.makePizza());
        // Pizza Base + Cheese + Olives
    }
}
```

```
}  
}
```

Each decorator **wraps** another Pizza, adds its own bit of behaviour, and delegates the rest to the object it wraps — so decorators can be **stacked in any combination and any order** at runtime (new CheeseDecorator(new OliveDecorator(new BasicPizza())) is equally valid and just changes the order the toppings are appended). This is precisely the design behind Java I/O's `BufferedReader(new InputStreamReader(new FileInputStream(...)))` chain — each wrapper adds one capability (buffering, character decoding) around the stream underneath it.

20.4.3 Facade Pattern

Intent: Provide a single, simplified, unified interface to a set of interfaces in a complex subsystem, so client code doesn't need to know about or coordinate every subsystem class individually.

File: Demo1002_StructuralDesignPatternDemos/src/com/accenture/ltd/FacadePatternDemo.java

```
package com.accenture.ltd;  
class KitchenService {  
    void prepareFood() {  
        System.out.println("Food prepared!");  
    }  
}  
  
class BillingService {  
    void generateBill() {  
        System.out.println("Bill generated!");  
    }  
}  
  
class DeliveryService {  
    void deliver() {  
        System.out.println("Food delivered!");  
    }  
}  
  
// Facade class that hides complexities  
class FoodOrderFacade {  
    private KitchenService kitchen = new KitchenService();  
    private BillingService billing = new BillingService();  
    private DeliveryService delivery = new DeliveryService();  
  
    public void placeOrder() {  
        kitchen.prepareFood();  
        billing.generateBill();  
        delivery.deliver();  
    }  
}
```

```
        System.out.println("Order completed!");
    }
}

public class FacadePatternDemo {
    public static void main(String[] args) {
        new FoodOrderFacade().placeOrder();
    }
}
```

The client calls exactly one method, `placeOrder()`, and never has to know that three separate subsystems exist behind it, in what order they must run, or how they communicate. Note the difference from Adapter: Adapter's job is to make an *incompatible* interface usable; Facade's job is to make a *complicated, multi-class* subsystem simple to use — the subsystem interfaces underneath a Facade are perfectly usable on their own, just inconvenient to coordinate one-by-one.

20.5 Behavioural Patterns

Behavioural patterns focus on how responsibility and communication are distributed between objects.

20.5.1 Strategy Pattern

Intent: Define a family of interchangeable algorithms, encapsulate each one, and make them swappable at runtime — replacing long if/else or switch chains that select behaviour with polymorphism.

File: Demo1003_StrategyPatternDemos/src/com/accenture/ltt/StrategyPatternDemo.java

```
package com.accenture.ltt;

//Problem solved: Avoid multiple if-else for different behaviors.
// Provides flexibility to change the algorithm at runtime.
//Strategy interface

interface PaymentStrategy {
    void pay(int amount);
}

//Concrete strategies
class UpiPayment implements PaymentStrategy {
    public void pay(int amount) {
        System.out.println("Paid " + amount + " using UPI");
    }
}

//Concrete Strategy
class CardPayment implements PaymentStrategy {
```

```

    public void pay(int amount) {
        System.out.println("Paid " + amount + " using Card");
    }
}

//Context class that uses a strategy
class ShoppingCart {
    private PaymentStrategy strategy;

    // Set Strategy at runtime
    public void setPaymentStrategy(PaymentStrategy strategy) {
        this.strategy = strategy;
    }

    public void checkout(int amount) {
        strategy.pay(amount); // Delegating behavior
    }
}

public class StrategyPatternDemo {
    public static void main(String[] args) {
        ShoppingCart cart = new ShoppingCart();

        cart.setPaymentStrategy(new UpiPayment());
        cart.checkout(500); // Paid using UPI

        cart.setPaymentStrategy(new CardPayment());
        cart.checkout(1000); // Paid using Card
    }
}

//Benefit: Avoids multiple if-else conditions -> flexible behavior change

```

ShoppingCart (the “context”) holds a reference to a PaymentStrategy and delegates to it, without any conditional logic of its own about *which* strategy is in play — the same checkout() call behaves differently purely because a different strategy object was set beforehand. Note the resemblance to the Factory Method demo in §20.2.3 — both use the Payment/PaymentStrategy abstraction — the distinction is that Factory Method is about *which object gets created*, while Strategy is about *which behaviour gets delegated to* an object that’s already been given to the context.

20.5.2 Command Pattern

Intent: Encapsulate a request (an action plus its receiver) as a stand-alone object, so that requests can be queued, logged, undone, or parameterised — decoupling the object that *invokes* an action from the object that actually *performs* it.

File: Demo1003_StrategyPatternDemos/src/com/accenture/ltt/CommandPatternDemo.java

```
package com.accenture.ltt;

//Command interface
interface Command {
    void execute();
}

//Receiver class performing actual action
class Light {
    void on() { System.out.println("Light ON"); }
    void off() { System.out.println("Light OFF"); }
}

//Concrete commands
class LightOnCommand implements Command {
    Light light;
    public LightOnCommand(Light light) { this.light = light; }
    public void execute() { light.on(); }
}

class LightOffCommand implements Command {
    Light light;
    public LightOffCommand(Light light) { this.light = light; }
    public void execute() { light.off(); }
}

//Invoker
class Remote {
    Command command;
    void setCommand(Command command) { this.command = command; }
    void pressButton() { command.execute(); }
}

public class CommandPatternDemo {
    public static void main(String[] args) {
        Light light = new Light();
        Remote remote = new Remote();

        remote.setCommand(new LightOnCommand(light));
        remote.pressButton(); // Light ON

        remote.setCommand(new LightOffCommand(light));
        remote.pressButton(); // Light OFF
    }
}

//Benefit: Undo, logging, macro commands become possible
```

Remote (the invoker) never calls `light.on()/light.off()` directly — it only knows how to call `execute()` on whatever Command object it currently holds. Because each request is now an object, not just a method call, it can be stored in a list (for undo/redo history or macro playback), passed around, or queued for later execution — none of which would be possible if Remote simply called `light.on()` inline.

20.5.3 Observer Pattern

Intent: Define a one-to-many dependency between objects so that when one object (the subject) changes state, all its dependents (observers) are notified and updated automatically — the foundation of publish/subscribe systems and, later, of reactive and event-driven architectures.

File: Demo1003_StrategyPatternDemos/src/com/accenture/ltt/ObserverPatternDemo.java

```
package com.accenture.ltt;

import java.util.ArrayList;
import java.util.List;

// Observer (Subscriber)
interface Subscriber {
    void notify(String video);
}

// Concrete Observers
class User implements Subscriber {
    private String name;

    public User(String name) { this.name = name; }

    public void notify(String video) {
        System.out.println(name + " notified about: " + video);
    }
}

// Subject (Publisher)
class YouTubeChannel {
    private List<Subscriber> subscribers = new ArrayList<>();

    public void subscribe(Subscriber subscriber) {
        subscribers.add(subscriber);
    }

    public void uploadVideo(String title) {
        System.out.println("New Video Uploaded: " + title);
        subscribers.forEach(sub -> sub.notify(title)); // Notify all
    }
}
```

```

}

public class ObserverPatternDemo {
    public static void main(String[] args) {
        YouTubeChannel channel = new YouTubeChannel();

        channel.subscribe(new User("Alice"));
        channel.subscribe(new User("Bob"));

        channel.uploadVideo("Strategy Pattern in Java");
    }
}
//Benefit: Automatic notifications -> loose coupling

```

YouTubeChannel (the subject) keeps a list of Subscriber observers and, on `uploadVideo()`, loops through and notifies every one of them — but it has no idea *what* each observer does with that notification, nor how many observers exist. New observer types can subscribe without YouTubeChannel ever being modified. This is the same underlying shape as Java’s own `java.util.Observer/Observable` (now deprecated) and, more importantly, is the conceptual root of listener-based GUI event handling and message-broker/event-bus architectures used across full-stack systems.

20.6 Asynchronous Design Patterns — `CompletableFuture`

Modern Java (from Java 8 onward) provides `java.util.concurrent.CompletableFuture` as a way to compose asynchronous, non-blocking pipelines of work — a pattern-level answer to “how do I run something in the background and react to its result without blocking the calling thread.”

20.6.1 Basic Async Pipeline — `supplyAsync` / `thenApply` / `thenAccept`

File: `Demo1004_AsynchronousDesignPatternsDemo/src/com/accenture/ltt/CompletableFutureDemo`

```

package com.accenture.ltt;

import java.util.concurrent.*;

public class CompletableFutureDemo {
    public static void main(String[] args) {
        // supplyAsync -> Runs in background thread without blocking main thread
        CompletableFuture.supplyAsync(() -> {
            System.out.println("Fetching data...");
            try { Thread.sleep(2000); } catch (Exception e) {}
            return 100; // Task returns value
        })
        // thenApply -> Used for processing returned result
        .thenApply(data -> {

```

```

        System.out.println("Processing data...");
        return data * 2; // Transform result
    })
    // thenAccept -> Consumes the final value without returning further value
    .thenAccept(result -> System.out.println(" Final Result: " + result));

    // Main thread not blocked - continues executing
    System.out.println("Main thread is free to do other work!");

    try { Thread.sleep(3000); } catch (Exception e) {}
}
}

```

`supplyAsync()` submits the given lambda to run on a background thread (by default, the common `ForkJoinPool`) and immediately returns a `CompletableFuture<Integer>` — a placeholder for a value that doesn't exist *yet*. `thenApply()` registers a transformation to run automatically once that value becomes available, itself returning a *new* `CompletableFuture` wrapping the transformed value — which is what allows chaining. `thenAccept()` is a **terminal** step: it consumes the final value (for a side effect like printing) but returns `CompletableFuture<Void>`, since there's nothing further to chain. Crucially, none of this blocks main — the line `System.out.println("Main thread is free to do other work!")` executes immediately, often printing *before* “Fetching data...” finishes its two-second sleep.

20.6.2 Combining Two Independent Async Tasks — `thenCombine`

File: `Demo1004_AsynchronousDesignPatternsDemo/src/com/accenture/ltt/ParallelTasksDemo.java`

```

package com.accenture.ltt;

import java.util.concurrent.*;

public class ParallelTasksDemo {
    static CompletableFuture<String> fetchWeather() {
        return CompletableFuture.supplyAsync(() -> {
            sleep(2000);
            return " Weather: 29°C";
        });
    }

    static CompletableFuture<String> fetchStock() {
        return CompletableFuture.supplyAsync(() -> {
            sleep(3000);
            return " Stock: Rs. 2150";
        });
    }

    public static void main(String[] args) {

```



```

    long start = System.currentTimeMillis();

    CompletableFuture<String> combined =
        fetchWeather().thenCombine(fetchStock(), (w, s) -> w + " | " + s);

    System.out.println("Main thread continues working...");
    System.out.println(" Result: " + combined.join());

    long end = System.currentTimeMillis();
    System.out.println(" Completed in " + (end - start) + "ms");
}

private static void sleep(long ms) {
    try { Thread.sleep(ms); } catch (Exception e) {}
}
}

```

`fetchWeather()` and `fetchStock()` both start immediately and run **concurrently** on separate background threads — one takes about 2 seconds, the other about 3. `thenCombine()` waits for *both* to complete and merges their two results with the supplied combining function. Because the two tasks overlap in time rather than running one after another, the total wall-clock time is roughly the *longer* of the two (about 3 seconds), not the *sum* of both (5 seconds) — the measurable payoff of asynchronous composition. `combined.join()` blocks the calling thread only at the very end, to retrieve the final combined value for printing — a normal and expected use of `join()/get()` once you actually need the result.

Worth knowing for viva: `get()` throws checked exceptions (`InterruptedException`, `ExecutionException`) and must be wrapped in a `try/catch` or declared in a `throws` clause; `join()` is functionally equivalent but throws only unchecked exceptions, which is why it's used more freely inside lambdas and stream pipelines.

20.7 Viva Voce — Likely Questions & Answers

Q1. What is a design pattern, and how is it different from an algorithm? A design pattern is a reusable, named solution shape to a recurring *design* problem — it describes relationships between classes/objects. An algorithm is a step-by-step procedure to solve a *computational* problem. Patterns are about structure and communication; algorithms are about computation.

Q2. What are the three classic categories of design patterns (Gang of Four), and where does Singleton fall? Creational, Structural, and Behavioural. Singleton is Creational — it's concerned with controlling how (and how many times) an object gets instantiated.

Q3. Why is the simple `if (instance == null)` Singleton implementation unsafe in a multithreaded environment? Two threads could both check `instance == null`, both see `true` before either one finishes constructing the object, and both end up

creating separate instances — breaking the “exactly one instance” guarantee. Fixes include synchronizing the accessor method, double-checked locking, or eager/static initialization.

Q4. What problem does the Builder pattern solve that a normal constructor doesn’t? It avoids the “telescoping constructor” problem — many overloaded constructors for every combination of optional parameters — by letting the caller set only the fields it cares about, in a readable, chained style, and only build the final object once everything needed is set.

Q5. How is Factory Method different from Builder? Factory Method decides *which class* to instantiate based on some input (e.g., a type code), typically returning a fully-formed object in one call. Builder constructs a *single, potentially complex* object step-by-step, useful when there are many optional parts to assemble.

Q6. What does “shallow copy” mean in the context of the Prototype pattern, and why does it matter? `Object.clone()`’s default behaviour copies primitive fields by value, but reference-type fields are copied by reference — so the clone and the original end up pointing to the *same* underlying object for any field that isn’t a primitive. If that shared object is later mutated through one reference, the change is visible through the other too — which is often not what was intended, and requires a deliberate deep copy to avoid.

Q7. What is the key difference between the Adapter and Facade patterns, since both “simplify” an interface? Adapter converts one *incompatible* interface into the interface a client expects, letting existing, otherwise-unusable code work with new client code. Facade doesn’t fix incompatibility — it wraps a *complex, multi-class* subsystem (each of which is already perfectly usable) behind one simplified entry point, purely to reduce coordination complexity for the client.

Q8. In the Decorator pattern demo, why can decorators be applied in any order or combination? Because every decorator (and the base component) implements the same interface (Pizza), and each decorator holds a reference to another Pizza it wraps and delegates to. Since the “wrapped” object could be the base component or another decorator, decorators compose freely and can be nested in any sequence.

Q9. How does Strategy differ from Factory Method, given both demos use a Payment type abstraction? Factory Method is about object *creation* — deciding which concrete class to instantiate. Strategy is about *behaviour delegation* — a context object (ShoppingCart) holds a reference to whatever strategy it’s been given and delegates the actual work to it, without concerning itself with how that object was created.

Q10. What does the Command pattern decouple, and what capability does that decoupling enable? It decouples the object that *invokes* an action (the Remote, the “invoker”) from the object that actually knows how to *perform* it (the Light, the “receiver”), by wrapping the request itself as an object. Because the request is now an object rather than an inline method call, it can be stored, queued, logged, undone, or grouped into macros.

Q11. What is the core relationship the Observer pattern models? A one-to-

many dependency: when a subject's state changes, every one of its registered observers is automatically notified — without the subject needing to know anything about what each observer specifically does with that notification.

Q12. In `CompletableFuture`, what is the difference between `thenApply()` and `thenAccept()`? `thenApply()` transforms the result and returns a new `CompletableFuture` carrying that transformed value, so further chaining is possible. `thenAccept()` is a terminal consumer — it uses the result (typically for a side effect) but returns `CompletableFuture<Void>`, since it produces no further value to chain onward.

Q13. Why does combining two independent async tasks with `thenCombine()` typically finish faster than running them sequentially? Because both tasks are submitted to run concurrently on separate threads as soon as they're created; `thenCombine()` simply waits for both to finish and merges their results. The total time is close to the *longer* of the two tasks' durations, not the sum of both, since the work overlaps rather than happening one after another.

Q14. What is the difference between `CompletableFuture.get()` and `.join()`? Both block the calling thread until the result is available. `get()` declares checked exceptions (`InterruptedException`, `ExecutionException`) that must be handled or declared. `join()` wraps the same failure conditions in an unchecked `CompletionException`, which is why it's more convenient to call inside lambdas and stream pipelines that can't declare checked exceptions.

Chapter 21

JDBC — Java Database Connectivity

21.1 What Is JDBC, and Why Do We Need It?

JDBC (Java Database Connectivity) is a standard Java API that lets Java code talk to a relational database — send SQL statements, and retrieve results — in a database-vendor-neutral way. Without JDBC, every database vendor (MySQL, Oracle, PostgreSQL, SQL Server...) would require its own proprietary Java API, and switching databases would mean rewriting all your data-access code. JDBC solves this the same way the JDBC/JVM split solves platform portability in Chapter 1: your application code is written entirely against the `java.sql/javax.sql` interfaces (`Connection`, `Statement`, `ResultSet`...); a vendor-supplied **JDBC driver** (a `.jar` you add to the classpath, e.g. `mysql-connector-j`) implements those interfaces for one specific database. Change the driver and the connection URL, and — in principle — the rest of your code barely changes.

21.2 Relational Databases in One Page — Tables, Keys, and Basic SQL

Everything from this point through the end of Chapter 24 assumes a working knowledge of relational database basics. If SQL and table design are already familiar, this section can be skipped — but for anyone coming to this chapter without a database background, the rest of this volume will be much easier to follow with these few ideas firmly in place first.

- **Table** — a named, structured collection of data, organised into rows and columns, similar in shape to a spreadsheet. The employee table used throughout this chapter’s demos stores one row per employee.
- **Row (a.k.a. record or tuple)** — a single entry in a table — one specific employee, in this case — with a value for every column defined on that table.
- **Column (a.k.a. field or attribute)** — a named, typed slot every row has a value for — `employeeId`, `employeeName`, `salary`, etc. Every value in a given column

shares the same data type across all rows.

- **Primary Key** — one column (or a combination of columns — see Chapter 24’s composite key discussion) whose value **uniquely identifies each row** in a table. No two rows may share the same primary key value, and a primary key column’s value cannot be left empty (NULL). `employeeId` is the primary key of the employee table in these demos.
- **Foreign Key** — a column in one table that holds the primary key value of a row in *another* table, creating a link between the two. This is the relational database’s native mechanism for expressing relationships — the same relationships Chapter 23 later shows how to express in JPA using `@OneToOne`, `@ManyToOne`, etc.
- **Schema** — the overall structure of a database: which tables exist, which columns each has, its data type, and the constraints (primary keys, foreign keys, uniqueness, not-null) enforced on it.

The four foundational SQL statements, worth being able to read comfortably before working through this chapter’s demos:

Statement	Purpose	Example
SELECT	Retrieve rows from one or more tables	SELECT * FROM employee WHERE salary > 50000
INSERT	Add a new row to a table	INSERT INTO employee (employeeId, employeeName) VALUES (1, 'Alice')
UPDATE	Modify existing rows that match a condition	UPDATE employee SET salary = 60000 WHERE employeeId = 1
DELETE	Remove rows that match a condition	DELETE FROM employee WHERE employeeId = 1

SELECT and the three others fall into two broader SQL categories worth knowing by name: **DQL** (Data Query Language — just SELECT, since it only *reads* data) and **DML** (Data Manipulation Language — INSERT/UPDATE/DELETE, since these *change* data). A third category, **DDL** (Data Definition Language — CREATE TABLE, ALTER TABLE, DROP TABLE), defines or changes the schema itself rather than the data inside it; `hibernate.hbm2ddl.auto` (seen later in Chapter 22) is Hibernate automatically generating and running DDL on your behalf. Every JDBC method in this chapter, and every JPQL query in Chapter 24, ultimately boils down to one of these categories of statement — JDBC just gives you a way to send that SQL text from Java and read back the results; JPA/Hibernate (Chapter 22 onward) generates that same SQL *for* you, from higher-level Java code.

21.3 The JDBC Workflow, Step by Step

Every JDBC-based interaction with a database follows the same five steps:

1. **Load/register the driver** (mostly automatic in modern JDBC 4.0+, but explicit `Class.forName(...)` is still seen in older code and training material).
2. **Open a Connection** to the database, via `DriverManager.getConnection(url, username, password)`.
3. **Create a Statement or PreparedStatement** object from that connection.
4. **Execute SQL** through that statement — `executeQuery()` for SELECT, `executeUpdate()` for INSERT/UPDATE/DELETE.
5. **Process the ResultSet** (if any), and **close** the connection/statement/result set to release database resources.

21.3.1 Establishing a Connection

File: Demo1000_JDBC_ConnectionToDB/src/com/accenture/ltt/utility/DBUtility.java

```
package com.accenture.ltt.utility;

import java.sql.Connection;
import java.sql.DatabaseMetaData;
import java.sql.DriverManager;
import java.sql.SQLException;

public class DBUtility {
    public static String url = "jdbc:mysql://localhost:3306/jdbcdemos";
    public static String username = "root";
    public static String password = "root";
    private static Connection connection = null;
    public static Connection getDBConnection() throws ClassNotFoundException, SQLException {
        Class.forName("com.mysql.cj.jdbc.Driver");
        System.out.println("Driver registered!!");
        connection = DriverManager.getConnection(url, username, password);
        System.out.println("Connection established!!");
        //To check JDBC Version
        DatabaseMetaData meta = connection.getMetaData();
        System.out.println("JDBC Version: "+meta.getJDBCMajorVersion());
        return connection;
    }

    public static void getDBDestroyConnection() throws SQLException {
        connection.close();
    }
}
```

The **JDBC URL** `jdbc:mysql://localhost:3306/jdbcdemos` has a fixed structure worth memorising for viva: `jdbc:<subprotocol>://<host>:<port>/<database-name>` — the subprotocol (mysql here) tells the `DriverManager` which registered driver should handle this connection. `Class.forName("com.mysql.cj.jdbc.Driver")` forces the JVM to load that driver class, whose static initializer registers itself with `DriverManager`;

from JDBC 4.0 onward (Java 6+), this step happens **automatically** via the META-INF/services mechanism as long as the driver jar is on the classpath, which is why you'll see it commented out in the later demos in this chapter — it's kept only as a fallback / for clarity in teaching. DatabaseMetaData gives you introspection into the database and driver — its JDBC version, product name, supported features — useful for diagnostics.

21.4 Statement-Based CRUD — and Why It's Dangerous

The Statement interface executes a SQL string exactly as given. The demo below builds the SQL by string concatenation.

File: Demo1001_JDBC_Statement_CRUD/src/com/accenture/ltt/dao/EmployeeDAOImpl.java

```
package com.accenture.ltt.dao;

import java.sql.Connection;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;
import java.util.Date;

import com.accenture.lkm.businessbean.EmployeeBean;
import com.accenture.lkm.utility.DBUtility;

public class EmployeeDAOImpl implements EmployeeDAO {

    private static Connection connection = null;
    private static Statement statement = null;

    public int insertEmployee(EmployeeBean bean) throws ClassNotFoundException, SQLException {
        try {
            int rowsUpdateCount = 0;
            int id = bean.getEmployeeId();
            String ename = bean.getEmployeeName();
            String role = bean.getRole();
            Date insertTime = new java.sql.Date(bean.getInsertTime().getTime());
            double salary = bean.getSalary();

            String insertdata = "insert into employee values('" + id + "','" + ename +
                + "','" + insertTime + "','" + salary + "')";
            connection = DBUtility.getDBConnection();
            statement = connection.createStatement();
            rowsUpdateCount = statement.executeUpdate(insertdata);
            if (rowsUpdateCount > 0) {
                System.out.println("Employee Registered Successfully: " + rowsUpdateCount);
            } else {
                System.out.println("00ps some thing went wrong!! we are on process..");
            }
        }
    }
}
```

```

    }
    return rowsUpdateCount;
} catch (Exception e) {
    throw e;
} finally {
    DBUtility.getDBConnectionClose();
}
}

@Override
public void readEmployee() throws ClassNotFoundException, SQLException {
    try {
        String retrievedata = "select * from employee";
        connection = DBUtility.getDBConnection();
        statement = connection.createStatement();
        ResultSet resultSet = statement.executeQuery(retrievedata);

        System.out.println("Retrieveing Query executed!!");
        while (resultSet.next()) {
            System.out.println("Employee ID: " + resultSet.getInt("employeeId") + "
                                + resultSet.getString("employeeName") + "\nRole: " + resultSet.
                                + "\nInsert Time: " + resultSet.getString("insertTime") + "\nSa
                                + resultSet.getDouble("salary"));
            System.out.println("=====");
        }
    } catch (Exception e) {
        throw e;
    } finally {
        DBUtility.getDBConnectionClose();
    }
}

@Override
public void updateEmployee(EmployeeBean bean) throws SQLException, ClassNotFoundException {
    try {
        int id = bean.getEmployeeId();
        double modifiedsalary = bean.getSalary();
        String updatedata = "update employee set salary=" + modifiedsalary + "where";
        connection = DBUtility.getDBConnection();
        statement = connection.createStatement();
        int rowsUpdateCount = statement.executeUpdate(updatedata);
        if (rowsUpdateCount > 0) {
            System.out.println("Employee Updated Successfully: " + rowsUpdateCount);
        } else {
            System.out.println("Ops some thing went wrong!! we are on process..");
        }
    } catch (Exception e) {

```



```

        throw e;
    } finally {
        DBUtility.getDBConnectionClose();
    }

}

@Override
public void deleteEmployee(EmployeeBean bean) throws ClassNotFoundException, SQLException {
    try {
        int id = bean.getEmployeeId();
        String deletedata = "DELETE from employee WHERE employeeId = '" + id + "'";
        connection = DBUtility.getDBConnection();
        statement = connection.createStatement();
        int deleteRowsUpdateCount = statement.executeUpdate(deletedata);
        if (deleteRowsUpdateCount > 0) {
            System.out.println("Employee deleted Successfully: " + deleteRowsUpdateCount);
        } else {
            System.out.println("Oops some thing went wrong!! we are on process..");
        }
    } catch (Exception e) {
        throw e;
    } finally {
        DBUtility.getDBConnectionClose();
    }
}
}

```

Notice the insertEmployee method builds its SQL by directly concatenating the employee's name and role into the query string: "...values('" + id + "', '" + ename + "' This is functionally correct for well-behaved input, but it is precisely the pattern that enables **SQL Injection** — if ename ever contained a value like '; DROP TABLE employee; --, that text becomes part of the executed SQL itself, not a literal data value, because Statement has no concept of “this part is data, this part is SQL syntax.” It also requires manual, error-prone type-to-string conversion (dates, numbers) and manual escaping of quote characters that appear inside string data.

21.5 PreparedStatement — Fixing SQL Injection and Type Safety

PreparedStatement pre-compiles a parameterised SQL template (using ? placeholders) at the database, then binds actual values to those placeholders through typed setter methods (setInt, setString, setDouble, setDate...). Because the SQL structure is fixed *before* any user-supplied value is attached, an attacker-supplied string

can never change the shape of the query — it can only ever be treated as a data value.

File: Demo1002_JDBC_PreparedStatement_CRUD/src/com/accenture/ltt/dao/EmployeeDAOImpl.java

```
package com.accenture.ltt.dao;

import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;

import com.accenture.lkm.businessbean.EmployeeBean;
import com.accenture.lkm.utility.DBUtility;

public class EmployeeDAOImpl implements EmployeeDAO {

    private static Connection connection = null;
    private static PreparedStatement pstatement = null;

    @Override
    public int insertEmployee(EmployeeBean bean) throws ClassNotFoundException, SQLException {
        try {
            int rowsInsertCount = 0;
            int id = bean.getEmployeeId();
            String ename = bean.getEmployeeName();
            String role = bean.getRole();
            java.sql.Date insertTime = new java.sql.Date(bean.getInsertTime().getTime());
            double salary = bean.getSalary();
            String insertdata = "insert into employee (employeeId, employeeName, role, salary, insertTime) values (" + id + ", " + ename + ", " + role + ", " + salary + ", " + insertTime + ")";
            connection = DBUtility.getDBConnection();
            PreparedStatement pstatement = connection.prepareStatement(insertdata);
            pstatement.setInt(1, id);
            pstatement.setString(2, ename);
            pstatement.setString(3, role);
            pstatement.setDate(4, insertTime);
            pstatement.setDouble(5, salary);
            rowsInsertCount = pstatement.executeUpdate();
            return rowsInsertCount;
        } catch (Exception e) {
            throw e;
        } finally {
            DBUtility.getDBCcloseConnection();
        }
    }

    @Override
    public void readEmployee() throws ClassNotFoundException, SQLException {
        try {
```

```

        String retrievedata = "select * from employee";
        connection = DBUtility.getConnection();
        PreparedStatement pstatement = connection.prepareStatement(retrievedata);
        ResultSet resultSet = pstatement.executeQuery();
        while (resultSet.next()) {
            System.out.println("Employee ID: " + resultSet.getInt("employeeId") + "
                                + resultSet.getString("employeeName") + "\nRole: " + resultSet.
                                + "\nInsert Time: " + resultSet.getString("insertTime") + "\nSa
                                + resultSet.getDouble("salary"));
            System.out.println("=====");
        }
    } catch (Exception e) {
        throw e;
    } finally {
        DBUtility.getDBCloseConnection();
    }
}

```

```

@Override
public void updateEmployee(EmployeeBean bean) throws SQLException, ClassNotFoundException
    try {
        int id = bean.getEmployeeId();
        double modifiedsalary = bean.getSalary();
        String updatedata = "update employee set salary=? where employeeId= ?";
        connection = DBUtility.getConnection();
        pstatement = connection.prepareStatement(updatedata);
        pstatement.setInt(2, id);
        pstatement.setDouble(1, modifiedsalary);
        boolean rowsUpdateCount = pstatement.execute();
        if (rowsUpdateCount != true) {
            System.out.println("Employee update successfully!!");
        } else {
            System.out.println("00ps some thing went wrong!! we are on process..");
        }
    } catch (Exception e) {
        throw e;
    } finally {
        DBUtility.getDBCloseConnection();
    }
}

```

```

@Override
public void deleteEmployee(EmployeeBean bean) throws ClassNotFoundException, SQLException
    try {
        int id = bean.getEmployeeId();
        String deletedata = "DELETE from employee WHERE employeeId = ?";
    }
}

```

```

        connection = DBUtility.getConnection();
        pstatement = connection.prepareStatement(deletedata);
        pstatement.setInt(1, id);
        int deletedRowsUpdateCount = pstatement.executeUpdate();
        if (deletedRowsUpdateCount > 0) {
            System.out.println("Employee deleted successfully!!" + deletedRowsUpdateCount);
        } else {
            System.out.println("Oops some thing went wrong!! we are on process..");
        }
    } catch (Exception e) {
        throw e;
    } finally {
        DBUtility.getConnectionCloseConnection();
    }
}
}

```

Two details worth pausing on for viva:

- **Placeholder numbering is 1-based, and matches left-to-right position in the SQL, not the order you call the setters.** In `updateEmployee`, the SQL is "update employee set salary=? where employeeId= ?", so placeholder 1 is the salary and 2 is the employee ID — the code correctly calls `pstatement.setInt(2, id)` and `pstatement.setDouble(1, modifiedsalary)`, even though `setInt` is written first in the source.
- **`execute()` vs `executeUpdate()` vs `executeQuery()`.** `executeQuery()` is for SELECT and returns a `ResultSet`. `executeUpdate()` is for INSERT/UPDATE/DELETE/DDI and returns an `int` — the number of rows affected. `execute()` is the general-purpose form that returns a `boolean` (true if the result is a `ResultSet`, false if it's an update count) — used when you genuinely don't know ahead of time what kind of statement is being run; the `updateEmployee` method above uses `execute()` but really should logically use `executeUpdate()`, since it always knows it's running an UPDATE — a subtlety worth flagging if asked to review this code in an interview.

21.6 Statement vs PreparedStatement — Summary Table

Aspect	Statement	PreparedStatement
SQL compilation	Compiled by the DB engine on every execution	Pre-compiled once; re-used with different parameter values
Parameter binding	Manual string concatenation	Typed <code>setXxx(index, value)</code> methods

Aspect	Statement	PreparedStatement
SQL Injection risk	High (raw string concatenation)	Effectively eliminated for parameterised values
Performance for repeated execution	Recompiled every time	Faster on repeated execution (cached execution plan)
Readability	Gets messy with many values	Clean, self-documenting placeholders
When to prefer	Simple, one-off DDL statements with no user input	Almost always, for any query involving variable/user-supplied data

21.7 Layered Architecture in the JDBC Demos

Even at the JDBC level, the demo code is already organised into the same layers you'll see formalised and reused throughout the JPA chapters:

- **businessbean** (EmployeeBean) — a plain data-holder (POJO) used to move data between layers.
- **dao** (EmployeeDAO / EmployeeDAOImpl) — the only layer allowed to touch JDBC/SQL directly; the interface decouples callers from the implementation (an application of DIP from Chapter 19).
- **service** (EmployeeService / EmployeeServiceImpl) — sits above DAO, exposes business operations, and is where cross-cutting concerns (validation, transactions spanning multiple DAO calls) would eventually live.
- **ui** (UITester) — the entry point exercising the whole stack.
- **utility** (DBUtility) — centralises connection creation/teardown so it isn't duplicated across every DAO method.

21.8 Viva Voce — Likely Questions & Answers

Q1. What is JDBC, and what problem does it solve? JDBC is a standard Java API for connecting to and executing SQL against relational databases in a vendor-neutral way. It solves the problem of needing a different, proprietary API for every database vendor — application code is written against JDBC's interfaces, and a vendor-supplied driver implements those interfaces for a specific database.

Q2. What are the five broad steps in any JDBC operation? Load/register the driver, open a connection, create a Statement/PreparedStatement, execute the SQL, and process the ResultSet before closing the connection/statement/result set.

Q3. What does the JDBC URL `jdbc:mysql://localhost:3306/jdbcdemos` mean, part by part? `jdbc` is the fixed API prefix, `mysql` is the sub-protocol identifying which driver should handle the connection, `localhost:3306` is the database server host and port, and `jdbcdemos` is the specific database/schema to connect to.

Q4. Is `Class.forName("com.mysql.cj.jdbc.Driver")` still necessary in modern JDBC? No, not strictly — from JDBC 4.0 (Java 6+), compliant drivers on the classpath are auto-registered via the META-INF/services mechanism, so `DriverManager.getConnection(...)` can locate the right driver without an explicit `Class.forName()` call. It's still seen in legacy code and early teaching examples for clarity.

Q5. What is the core security weakness of building SQL with `Statement` and string concatenation? Since the SQL text and the data values are concatenated into a single string before being sent to the database, there is no distinction between “this is SQL syntax” and “this is a data value.” A malicious or malformed input string can alter the actual structure of the executed query — this is SQL Injection.

Q6. How does `PreparedStatement` prevent SQL Injection? The SQL statement (with `?` placeholders) is compiled first, before any parameter values are known. Values are then bound afterward through typed setter methods, so the database always treats them strictly as literal data values — they cannot change the structure of the already-compiled query.

Q7. In `pstatement.setInt(2, id)`, what does the 2 refer to? It refers to the position of the `?` placeholder in the SQL string, counted left to right starting at 1 — not the order in which the setter methods are called in the Java code.

Q8. What is the difference between `executeQuery()`, `executeUpdate()`, and `execute()`? `executeQuery()` is used for SELECT statements and returns a `ResultSet`. `executeUpdate()` is used for INSERT/UPDATE/DELETE/DDI and returns an `int` row-count. `execute()` is the general form, returning a boolean indicating whether the result is a `ResultSet` (true) or an update count (false) — used when the statement type isn't known ahead of time.

Q9. Why does `PreparedStatement` typically perform better than `Statement` for repeated execution of the same query shape? The database can compile/cache an execution plan for the parameterised SQL once, and reuse that plan across multiple executions with different bound values, avoiding the cost of re-parsing and re-planning identical SQL text on every call.

Q10. Why is the connection closed in a `finally` block in every DAO method? So that the database connection (and its native/system resources) is released whether the operation succeeds or throws an exception — leaving connections open under error conditions would eventually exhaust the connection pool/limit and starve the application of the ability to connect at all.

Q11. What layers does the JDBC demo code already show, even before JPA is introduced? A businessbean (plain data object), a dao layer (the only layer touching JDBC directly, exposed through an interface), a service layer (business operations above the DAO), a ui layer (the caller/entry point), and a utility layer centralising connection management.

Q12. What is the difference between a primary key and a foreign key? A primary key uniquely identifies each row within its own table, and cannot be NULL or duplicated. A foreign key is a column in one table that stores the primary key value of a row in another table, creating a link between the two tables — the relational

mechanism behind everything Chapter 23 later expresses in JPA through `@OneToOne`, `@ManyToOne`, `@OneToMany`, and `@ManyToMany`.

Q13. What is the difference between DQL, DML, and DDL in SQL? DQL (Data Query Language) covers `SELECT` — reading data without changing it. DML (Data Manipulation Language) covers `INSERT/UPDATE/DELETE` — statements that change the data inside tables. DDL (Data Definition Language) covers `CREATE/ALTER/DROP` — statements that define or change the schema (tables and columns) itself, rather than the data inside it.

Chapter 22

Introduction to ORM & JPA — The Entity Lifecycle

22.1 The Object-Relational Impedance Mismatch

Object-oriented programs think in terms of objects, references, and inheritance; relational databases think in terms of tables, rows, foreign keys, and joins. The friction between these two worldviews is called the **object-relational impedance mismatch**, and it shows up in several concrete ways:

- An object graph is naturally traversed via references (`employee.getDepartment().getCity()`); a relational database has no notion of “reference” — only foreign keys and joins.
- Java supports inheritance (`Manager` extends `Employee`); a relational table has no native concept of inheritance.
- An object’s identity is its memory reference; a row’s identity is its primary key value — two different `Employee` objects with the same data are *not* `==` in Java, but two rows with the same primary key *are* the same row.

Chapter 21’s JDBC code shows this friction directly: every DAO method manually converts between an `EmployeeBean` (Java object) and SQL column values, and manually writes the SQL for every operation. As the number of entities and relationships grows, this hand-written mapping code becomes repetitive, error-prone, and expensive to maintain — which is exactly the gap ORM is built to close.

22.2 What Is ORM? What Is JPA? What Is Hibernate?

These three terms are frequently confused in interviews, so it’s worth being precise:

- **ORM (Object-Relational Mapping)** is a *technique/concept* — mapping Java objects to relational database tables automatically, so a developer works with objects and lets the framework generate the SQL.
- **JPA (Java Persistence API)** is a **specification** — a set of interfaces and annotations (`EntityManager`, `@Entity`, `@Id`, `@OneToMany`, `JPQL`...) defined by Jakarta EE (formerly Java EE) that describes *how* ORM should be exposed to Java devel-

opers. JPA itself contains **no implementation** — it’s a contract, the same way `List` is an interface with no behaviour of its own.

- **Hibernate** is the most widely used **implementation (provider)** of the JPA specification. When you write `import javax.persistence.Entity`, you’re coding against the JPA specification; Hibernate is the library doing the actual work of translating that into SQL, managing the persistence context, and talking to the JDBC driver underneath.

The practical implication: JPA code is meant to be provider-agnostic — in theory, you can switch from Hibernate to EclipseLink (another JPA provider) by changing one configuration line, without rewriting your entities or DAO code, precisely because you coded against the JPA interfaces, not against Hibernate-specific classes. This is DIP (Chapter 19) applied at the framework level.

Term	What it is	Analogy
ORM	A concept/technique	“Database access without writing SQL by hand”
JPA	A specification (interfaces + annotations)	Like the <code>java.sql</code> interfaces JDBC drivers implement
Hibernate	An implementation of JPA	Like a specific JDBC driver implementing <code>java.sql.Connection</code>

22.3 What Is a POJO, and Why Does It Matter for ORM?

Every `EmployeeBean` and every `@Entity`-annotated class seen throughout this training is an example of a **POJO — Plain Old Java Object**. The term (coined in the early 2000s, partly as a reaction against the heavier, more restrictive component models Java EE required at the time) simply means: an ordinary Java class that

- does **not** extend any special framework base class or implement any special framework interface just to gain basic functionality,
- has a simple structure of private fields with public getters/setters (and usually a no-argument constructor),
- carries **no built-in framework behaviour** of its own — it’s just a plain data-holder, with no knowledge of how it will be persisted, transmitted, or displayed.

Why this matters specifically for ORM: JPA’s design philosophy is that your domain classes should stay POJOs, decorated only with **annotations** (`@Entity`, `@Id`, `@OneToOne`, ...) rather than being forced to extend a framework-specific base class or implement a framework-specific interface. This is a deliberate, important design choice — it means an `EmployeeEntity` is still a perfectly ordinary Java object that can be constructed, tested, serialized, and reasoned about independently of JPA/Hibernate; the annotations are *metadata* that JPA reads (typically via reflection) to know how to map the object to a table, but they don’t change what kind of object it fundamentally is. This is also precisely why the `EmployeeBean` classes used throughout the JDBC and

JSQL “structured” DAO methods (Chapters 21 and 24) look nearly identical to the `@Entity` classes — both are POJOs; the entity simply has annotations added on top of the same basic POJO shape.

22.4 persistence.xml — Configuring a Persistence Unit

Every JPA application needs a `persistence.xml` file (placed at `META-INF/persistence.xml` on the classpath) that defines one or more **persistence units** — named configuration blocks describing which entities belong to this unit, which JPA provider to use, and how to connect to the database.

File: `Demo1003_1_JPA_ORM_EntityLifecycle_persist/src/META-INF/persistence.xml`

```
<persistence xmlns="http://xmlns.jcp.org/xml/ns/persistence"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/persistence
http://xmlns.jcp.org/xml/ns/persistence/persistence_2_1.xsd"
version="2.1">

  <persistence-unit name="unit1">
    <description> Hibernate JPA Configuration Example</description>
    <provider>org.hibernate.jpa.HibernatePersistenceProvider</provider>
    <class>com.accenture.ltt.entity.EmployeeEntity</class>

    <properties>
      <property name="javax.persistence.jdbc.driver" value="com.mysql.cj.jdbc.Driver" />
      <property name="javax.persistence.jdbc.url" value="jdbc:mysql://localhost:3306/jp" />
      <property name="javax.persistence.jdbc.user" value="root" />
      <property name="javax.persistence.jdbc.password" value="root" />
      <property name="hibernate.show_sql" value="true" />
      <property name="hibernate.hbm2ddl.auto" value="update" />
    </properties>

  </persistence-unit>

</persistence>
```

Key elements worth knowing individually for viva:

- **<persistence-unit name="unit1">** — the name used later in code: `Persistence.createEntityManagerFactory("unit1")`. An application can define multiple persistence units (e.g., pointing at different databases) and choose which one to use by name.
- **<provider>** — explicitly names Hibernate’s `HibernatePersistenceProvider` as the JPA implementation to use for this unit.
- **<class>** — registers which entity classes belong to this persistence unit (in larger, annotation-scanned setups this is sometimes omitted in favour of automatic scanning, but explicit listing is unambiguous and used throughout this training).

- **hibernate.show_sql** — when true, Hibernate prints every generated SQL statement to the console — invaluable for understanding exactly what SQL your JPA calls are producing underneath.
- **hibernate.hbm2ddl.auto** — controls whether/how Hibernate automatically manages the database schema. update means Hibernate will alter existing tables to match the entity mappings without dropping data — convenient for training/demo purposes, but a setting to be very cautious with (or avoid entirely) in production, where schema changes should go through reviewed migration scripts, not be auto-generated on application startup.

22.5 EntityManagerFactory and EntityManager

- **EntityManagerFactory** is a heavyweight, thread-safe object, built once per persistence unit for the lifetime of the application — it represents the configured connection to a specific database plus a specific set of mapped entity classes.
- **EntityManager** is a lightweight, **not thread-safe** object created *from* the factory, representing a single unit of work / conversation with the database — roughly analogous to a single JDBC Connection, and typically created, used, and closed within the scope of a single business operation or request.

File: Demo1003_1_JPA ORM_EntityLifecycle_persist/src/com/accenture/ltt/utility/JPAUtility

```
package com.accenture.ltt.utility;

import javax.persistence.EntityManagerFactory;
import javax.persistence.Persistence;

public class JPAUtility {

    private static EntityManagerFactory entityManagerFactory;

    private static EntityManagerFactory buildEntityManagerFactory() {
        try {
            entityManagerFactory = Persistence.createEntityManagerFactory("unit1");

            return entityManagerFactory;
        } catch (Throwable ex) {
            // Make sure you log the exception, as it might be swallowed
            System.err.println("Initialization of EntityManagerFactory failed." + ex);
            throw new RuntimeException(ex);
        }
    }

    public static EntityManagerFactory getEntityManagerFactory() {
        if (entityManagerFactory == null) entityManagerFactory = buildEntityManagerFactory();
    }
}
```

```

        return entityManagerFactory;
    }

    public static void closeEntityManagerFactory(){

        if(entityManagerFactory != null && entityManagerFactory.isOpen()==true ) entity
    }

}

```

JPAUtility is itself a **Singleton** (Chapter 20, §20.2.1) applied to the EntityManagerFactory — it's expensive to create, so the whole application shares exactly one instance, built lazily on first use and closed once, typically when the application shuts down. Every DAO method in this chapter calls JPAUtility.getEntityManagerFactory().createEntityManager() to get a fresh, short-lived EntityManager for that one operation.

22.6 Mapping a Class to a Table — @Entity, @Table, @Id, @GeneratedValue

A **basic entity** — the minimum needed for JPA to recognise a class as a persistable table mapping:

File: Demo1006_JPA_ORM_EntityLifeCycle_merge_successfulupdate/src/com/accenture/ltt/entity

```

package com.accenture.ltt.entity;

import javax.persistence.Entity;
import javax.persistence.Id;
import javax.persistence.Table;

@Entity
@Table(name = "Employee")
public class EmployeeEntity {

    @Id
    private Integer employeeId;
    private String employeeName;
    private Double salary;

    public Integer getEmployeeId() {
        return employeeId;
    }

    public void setEmployeeId(Integer employeeId) {
        this.employeeId = employeeId;
    }
}

```

```
public String getEmployeeName() {
    return employeeName;
}

public void setEmployeeName(String employeeName) {
    this.employeeName = employeeName;
}

public Double getSalary() {
    return salary;
}

public void setSalary(Double salary) {
    this.salary = salary;
}
}
```

- @Entity marks the class as a JPA-managed entity — it must have a no-argument constructor (implicitly provided here since none is declared) and must not be final, because Hibernate creates proxy subclasses for lazy-loading.
- @Table(name = "Employee") explicitly names the backing database table; if omitted, JPA defaults to using the entity's class name as the table name.
- @Id marks the primary-key field. Every entity must have exactly one identifier (or an @EmbeddedId/@IdClass composite key — see Chapter 24).
- Every other non-transient field is, by default, mapped to a column of the same name — this is why employeeName maps to a column literally called employeeName unless overridden.

22.6.1 Primary Key Generation Strategies — @GeneratedValue

Rather than the application assigning primary key values manually, @GeneratedValue delegates that responsibility to the database/JPA provider:

File: Demo1001_Employee_JPA_Strategy_autoandidentity/src/com/accenture/ltt/entity/Employ

```
package com.accenture.ltt.entity;

import java.io.Serializable;
import java.util.Date;

import javax.persistence.Column;
import javax.persistence.Entity;
import javax.persistence.GeneratedValue;
import javax.persistence.GenerationType;
import javax.persistence.Id;
import javax.persistence.Table;
```

```

/**
 * Entity implementation class for Entity: EmployeeEntity
 *
 */
@Entity
@Table(name = "Employee")
public class EmployeeEntity implements Serializable {

    private static final long serialVersionUID = 1L;

    public EmployeeEntity() {
        super();
    }

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer employeeId;
    @Column(name = "emp_name")
    private String employeeName;
    private String role;
    private Date insertTime;
    private Double Salary;

    //setters and getters goes here
    // ... (standard getters/setters omitted for brevity)
}

```

GenerationType has four possible values, and the difference between them is a very common interview question:

Strategy	How the ID is generated	Typical use
AUTO	JPA provider picks the most appropriate strategy for the underlying database automatically	Database-agnostic code; lets Hibernate decide
IDENTITY	Relies on the database's native auto-increment column (AUTO_INCREMENT in MySQL)	MySQL is common here; the ID is only known <i>after</i> the INSERT executes
SEQUENCE	Uses a database sequence object to pre-allocate IDs	Common with Oracle/PostgreSQL, which support sequences natively
TABLE	Simulates a sequence using a separate database table that tracks the next value	Portable across any database, but slower (extra table read/lock per ID)

`@Column(name = "emp_name")` shows how to override the default column name — here, the Java field `employeeName` maps to a database column literally named `emp_name`, which is useful when the database’s naming convention (`snake_case`, legacy names) doesn’t match Java’s `camelCase` convention.

22.7 Transactions and the ACID Properties

Every `entityManager.getTransaction().begin() / .commit()` pair seen throughout the rest of this chapter is opening and closing a **transaction** — a fundamental database concept that deserves a clear explanation before diving into the lifecycle demos, since transactions appear in almost every one of them.

A **transaction** is a group of one or more database operations that are treated as a single, indivisible unit of work: either **all** of them succeed and are saved permanently (committed), or, if anything goes wrong partway through, **none** of them take effect (rolled back) — the database is left exactly as if the transaction had never started. The classic illustration is a bank transfer: debiting one account and crediting another must both succeed together, or neither should happen — a transaction that debited the first account but crashed before crediting the second would leave the bank’s books permanently wrong.

Database textbooks describe the guarantees a properly-implemented transaction provides using the acronym **ACID**:

Letter	Property	Meaning
A	Atomicity	The transaction’s operations happen as one indivisible unit — all or nothing
C	Consistency	A transaction can only move the database from one valid state to another valid state, never leaving it in a state that violates its rules/constraints
I	Isolation	Concurrent transactions don’t see each other’s incomplete/uncommitted changes
D	Durability	Once a transaction is committed, its changes survive even a subsequent crash or power loss

In JPA terms, calling `entityManager.getTransaction().begin()` starts a new transaction; every `persist()`, a direct field mutation on a managed entity, `merge()`, or

`remove()` call made afterward becomes part of that same unit of work; calling `.commit()` durably saves all of it together, while calling `.rollback()` instead (used in error-handling paths not shown explicitly in this training’s demos, but standard practice) would discard every uncommitted change made since `begin()`. This is precisely why every lifecycle demo in this chapter wraps its `persist()/find()/merge()/remove()` calls in a `begin()/commit()` pair — without an active transaction, JPA has no unit of work to group the operation into, and data-modifying operations will fail with a `TransactionRequiredException`.

22.8 The Entity Lifecycle — Four States

This is one of the most heavily tested JPA concepts in interviews. Every entity instance is, at any given moment, in exactly one of four lifecycle states:

State	Meaning	How you get there
New / Transient	A plain Java object, created with <code>new</code> , not yet associated with any <code>EntityManager</code> or database row	<code>new EmployeeEntity()</code>
Managed / Persistent	Associated with an open persistence context (an <code>EntityManager</code>); any changes to it are tracked and will be synchronized to the database	After <code>persist()</code> , or after <code>find()</code> , or as the object returned by <code>merge()</code>
Detached	Was managed at some point, but its <code>EntityManager</code> has since been closed (or the entity was explicitly <code>detach()</code> -ed / the persistence context was <code>clear()</code> -ed) — changes are no longer tracked	After <code>entityManager.close()</code> or <code>entityManager.clear()/detach()</code>
Removed	Scheduled for deletion; still managed until the transaction commits, at which point the corresponding row is deleted	After <code>remove()</code> , before <code>commit</code>

The demo projects in this training deliberately isolate each transition into its own numbered demo so each one can be studied independently: `persist` (§22.7), `find` (§22.8), `merge` (§22.9), and `remove` (§22.10). Every one of these demos prints `entityManager.contains(entity)` at multiple points — `contains()` is literally the JPA API

for asking “is this specific object currently managed by this persistence context?” — which is the clearest possible way to observe the lifecycle transitions actually happening.

22.9 persist() — Making a Transient Object Managed

File: Demo1003_1_JPA_ORM_EntityLifecycle_persist/src/com/accenture/ltt/dao/EmployeeDAOIMP

```
package com.accenture.ltt.dao;

import javax.persistence.EntityManager;
import javax.persistence.EntityManagerFactory;

import com.accenture.ltt.businessbean.EmployeeBean;
import com.accenture.ltt.entity.EmployeeEntity;
import com.accenture.ltt.utility.JPAUtility;

public class EmployeeDAOIMPL implements EmployeeDAO {

    public Integer addEmployee(EmployeeBean employee) throws Exception {
        Integer employeeID = 0;
        EntityManager entityManager = null;
        try {
            EntityManagerFactory entityManagerFactory = JPAUtility.getEntityManagerFactory();
            entityManager = entityManagerFactory.createEntityManager();
            EmployeeEntity employeeEntity = new EmployeeEntity();
            employeeEntity.setEmployeeName(employee.getEmployeeName());
            employeeEntity.setRole(employee.getRole());
            employeeEntity.setSalary(employee.getSalary());
            employeeEntity.setInsertTime(employee.getInsertTime());
            System.out.println("EmployeeEntity before transaction: " + entityManager.contains(employeeEntity));
            entityManager.getTransaction().begin();
            System.out.println("EmployeeEntity after transaction begin, before persist: " + entityManager.contains(employeeEntity));
            entityManager.persist(employeeEntity);
            System.out.println("EmployeeEntity after transaction begin, after persist: " + entityManager.contains(employeeEntity));
            entityManager.getTransaction().commit();
            System.out.println("EmployeeEntity after transaction commit, after persist: " + entityManager.contains(employeeEntity));
            employeeID = employeeEntity.getEmployeeId();

            //clearing the entity manager...
            entityManager.clear();
            System.out.println("EmployeeEntity after clearing entity manager: " + entityManager.contains(employeeEntity));
        } catch (Exception e) {
            e.printStackTrace();
        } finally {
            if (entityManager != null) {
                entityManager.close();
            }
        }
    }
}
```

```

        } catch (Exception exception) {
            throw exception;
        } finally {
            if (entityManager != null) {
                entityManager.close();
            }
        }

        return employeeID;
    }
}

```

Tracing the `contains()` output across this method tells the whole story:

1. **Before transaction, before persist:** `false` — `employeeEntity` was just created with `new`; it is transient, unknown to the `EntityManager`.
2. **After transaction begin, before persist:** still `false` — beginning a transaction does not, by itself, attach any entity; it only opens a unit of work.
3. **After persist():** `true` — the object is now managed. Note this happens *before* `commit()` — `persist()` immediately attaches the object to the persistence context, though the actual INSERT SQL may be deferred until `commit` (or `flush`) depending on the generation strategy.
4. **After commit():** still `true` — the transaction ending does not, by itself, detach anything.
5. **After entityManager.clear():** `false` — `clear()` detaches *every* entity currently managed by this persistence context, which is why `contains()` flips back to `false`.

Important nuance for viva: with `GenerationType.IDENTITY`, the primary key is only known once the actual INSERT statement has executed against the database — which is why `persist()` with `IDENTITY` triggers an immediate INSERT rather than deferring it, unlike some other strategies that can batch inserts until `flush/commit`.

22.10 find() — Retrieving and Attaching an Entity

File: `Demo1004_JPA_ORM_EntityLifecycle_find/src/com/accenture/ltt/dao/EmployeeDAOIMPL.java`

```

package com.accenture.ltt.dao;

import javax.persistence.EntityManager;
import javax.persistence.EntityManagerFactory;

import com.accenture.ltt.businessbean.EmployeeBean;
import com.accenture.ltt.entity.EmployeeEntity;
import com.accenture.ltt.utility.JPAUtility;

public class EmployeeDAOIMPL implements EmployeeDAO {

```

```

@Override
public EmployeeBean findEmployeeById(int employeeId) throws Exception {
    EntityManager entityManager = null;
    EmployeeBean employeeBean = null;
    EmployeeEntity employeeEntity = null;
    try {
        EntityManagerFactory entityManagerFactory = JPAUtility.getEntityManagerFactory();
        entityManager = entityManagerFactory.createEntityManager();
        System.out.println("EmployeeEntity before transaction begin, before find: " + employeeEntity);
        entityManager.getTransaction().begin();
        System.out.println("EmployeeEntity after transaction begin, before find: " + employeeEntity);
        employeeEntity = (EmployeeEntity) entityManager.find(EmployeeEntity.class, employeeId);
        System.out.println("EmployeeEntity after transaction begin, after find: " + employeeEntity);
        entityManager.getTransaction().commit();
        System.out.println("EmployeeEntity after transaction commit, after find: " + employeeEntity);

        if(employeeEntity != null){
            employeeBean = new EmployeeBean();
            employeeBean.setEmployeeId(employeeEntity.getEmployeeId());
            employeeBean.setEmployeeName(employeeEntity.getEmployeeName());
            employeeBean.setInsertTime(employeeEntity.getInsertTime());
            employeeBean.setRole(employeeEntity.getRole());
            employeeBean.setSalary(employeeEntity.getSalary());
        }
        employeeId = employeeBean.getEmployeeId();
        entityManager.clear();
        System.out.println("EmployeeEntity after clearing entity manager: " + employeeEntity);
    } catch (Exception exception) {
        throw exception;
    } finally {
        if (entityManager != null) {
            entityManager.close();
        }
    }

    return employeeBean;
}
}

```

`entityManager.find(EmployeeEntity.class, employeeId)` is JPA's primary-key lookup method — pass the entity class and the ID value, and it returns the matching managed entity, or null if no row exists with that key (unlike JPQL queries, which throw `NoResultException` for no match — a frequently-asked distinction). Note that `find()` does **not** require an active transaction to *read* data — but this demo deliberately wraps it in a transaction anyway to show that `find()` still attaches the result to the persistence context regardless. The important observation, again via

`contains()`: the entity becomes managed the instant `find()` returns it (assuming a row was found), not only after the surrounding transaction commits.

Also worth noting as an interesting side detail visible in this exact source (a good “spot the bug” viva exercise): the DAO also converts the found entity into an `EmployeeBean` for the caller, then re-reads `employeeBean.getEmployeeId()` back into the *parameter* `employeeId` — a slightly redundant re-assignment, but harmless, since it’s not used again afterward. Reviewing real demo code like this for such subtleties is itself good interview practice.

22.11 `merge()` — Reattaching a Detached (or Transient) Entity

`merge()` is one of the most misunderstood JPA operations. It does **not** attach the object you pass in — instead, it **copies the state of the object you pass in onto a managed copy** (fetched or created fresh inside the persistence context) and **returns that managed copy**. The object you originally passed to `merge()` remains detached (or transient) — only the *returned* reference is managed.

File: Demo1006_JPA_ORM_EntityLifecycle_merge_successfulupdate/src/com/accenture/ltt/dao/E

```
package com.accenture.ltt.dao;

import javax.persistence.EntityManager;
import javax.persistence.EntityManagerFactory;

import com.accenture.ltt.businessbean.EmployeeBean;
import com.accenture.ltt.entity.EmployeeEntity;
import com.accenture.ltt.utility.JPAUtility;

public class EmployeeDAOIMPL implements EmployeeDAO {

    public Integer addEmployee(EmployeeBean employee) throws Exception {
        Integer employeeID = 0;
        EntityManager entityManager = null;
        try {
            EntityManagerFactory entityManagerFactory = JPAUtility.getEntityManagerFacto
            entityManager = entityManagerFactory.createEntityManager();

            EmployeeEntity employeeEntity1 = new EmployeeEntity();
            employeeEntity1.setEmployeeName(employee.getEmployeeName());
            employeeEntity1.setSalary(employee.getSalary());
            employeeEntity1.setEmployeeId(employee.getEmployeeId());
            entityManager.getTransaction().begin();
            EmployeeEntity employeeEntity2 = entityManager.merge(employeeEntity1);
            employeeEntity2.setEmployeeName("dummy"); // entity2 is the reference of ma
                                                    // update

            entityManager.getTransaction().commit();
```

```

        employeeID = employeeEntity2.getEmployeeId();
    } catch (Exception exception) {
        throw exception;
    } finally {
        if (entityManager != null) {
            entityManager.close();
        }
    }

    return employeeID;
}
}

```

Walking through this deliberately: `employeeEntity1` is built with `new` — it is transient (or, if it were built with an existing employee’s ID, would represent a *detached* object standing in for a row that already exists in the database). `entityManager.merge(employeeEntity1)` returns `employeeEntity2` — a **different object reference**, which is now the actual managed copy tracked by the persistence context. The comment in the source makes the critical point explicit: calling `employeeEntity2.setEmployeeName("dummy")` succeeds in updating the database (because `employeeEntity2` is managed), whereas calling that same setter on `employeeEntity1` afterward would have **no effect on the database at all**, because `employeeEntity1` was never the object being tracked.

This exact confusion — mutating the object you originally passed to `merge()`, expecting it to persist, when you should be mutating the *returned* object — is precisely what the paired “unsuccessful update” demo in the same folder (`Demo1006_JPA_ORM_EntityLifeCycle_merge_unsuccessfulupdate`) is built to illustrate: if you keep mutating `employeeEntity1` after calling `merge()`, those changes are invisible to JPA, since `employeeEntity1` remains detached/transient throughout.

22.12 `remove()` — Deleting a Managed Entity

File: `Demo1005_JPA_ORM_EntityLifeCycle_remove/src/com/accenture/ltt/dao/EmployeeDAOIMPL.j`

```

package com.accenture.ltt.dao;

import javax.persistence.EntityManager;
import javax.persistence.EntityManagerFactory;

import com.accenture.ltt.entity.EmployeeEntity;
import com.accenture.ltt.utility.JPAUtility;

public class EmployeeDAOIMPL implements EmployeeDAO {

    @Override

```

```

public void removeEmployeeById(int id) throws Exception {
    EntityManager entityManager = null;
    EmployeeEntity employeeEntity = null;
    try {
        EntityManagerFactory entityManagerFactory = JPAUtility.getEntityManagerFactory();
        entityManager = entityManagerFactory.createEntityManager();
        System.out.println("Employee entity before transaction begin, before find: "+id);
        entityManager.getTransaction().begin();
        System.out.println("Employee entity after transaction begin, before find: "+id);
        employeeEntity = entityManager.find(EmployeeEntity.class, id);
        System.out.println("Employee entity after transaction begin, after find: "+id);
        System.out.println("Employee entity after transaction begin, before removing the entity");
        if(id == employeeEntity.getEmployeeId()) {
            entityManager.remove(employeeEntity);
            System.out.println("Employee entity after transaction begin, after removing the entity");
            entityManager.getTransaction().commit();
            System.out.println("Employee entity after removing entity, transaction committed");
        } else {
            System.out.println("employee id doesn't exist!!");
        }
    } catch (Exception exception) {
        throw exception;
    } finally{
        if(entityManager!=null){
            entityManager.close();
        }
    }
}
}

```

Critical rule visible in this code and frequently tested: `remove()` can only be called on a **managed** entity — you cannot construct a new, transient object with just an ID and call `remove()` on it directly; JPA has no way to know that object corresponds to an existing row unless it's currently managed. That's exactly why this method always calls `entityManager.find(...)` **first**, to obtain a managed reference, and only then calls `entityManager.remove(employeeEntity)` on that managed reference. `contains()` stays true immediately after `remove()` — the entity is still tracked by the persistence context (in the “removed” state) until the transaction actually commits, at which point the corresponding DELETE is issued and the object falls out of management entirely.

22.13 Viva Voce — Likely Questions & Answers

Q1. What is the object-relational impedance mismatch? The mismatch between how object-oriented programs model data (objects, references, inheritance) and how relational databases model data (tables, rows, foreign keys, no native inheritance). ORM frameworks like JPA/Hibernate exist to bridge this gap automatically.

Q2. What is the difference between JPA and Hibernate? JPA is a specification — a set of interfaces and annotations with no implementation of its own. Hibernate is a concrete implementation (provider) of that specification. Code written against JPA's interfaces can, in principle, run against any JPA provider, including Hibernate, EclipseLink, or others.

Q3. What does `hibernate.hbm2ddl.auto=update` do, and why should it be used cautiously? It tells Hibernate to automatically alter the database schema at startup to match the entity mappings, without dropping existing data. It's convenient for development/training, but risky in production, since uncontrolled automatic schema changes bypass reviewed migration processes and can behave unpredictably on a live database.

Q4. What is the difference between `EntityManagerFactory` and `EntityManager`? `EntityManagerFactory` is a heavyweight, thread-safe object built once per persistence unit for the life of the application. `EntityManager` is a lightweight, non-thread-safe object created from the factory to represent one unit of work, typically created and closed within the scope of a single operation.

Q5. What are the four states in the JPA entity lifecycle? New/Transient (just created with `new`, unknown to any persistence context), Managed/Persistent (tracked by an open `EntityManager`), Detached (was managed, but its `EntityManager` closed or was explicitly cleared/detached), and Removed (scheduled for deletion, still managed until commit).

Q6. What does `entityManager.contains(entity)` tell you? Whether the given entity instance is currently managed (tracked) by that specific persistence context — the standard way to directly observe entity lifecycle state transitions.

Q7. Does `persist()` guarantee an immediate INSERT statement is sent to the database? Not necessarily — it depends on the ID generation strategy and whether/when the persistence context flushes. With `GenerationType.IDENTITY`, the INSERT typically must run immediately, because the primary key can only be known after the database generates it. With other strategies, JPA may defer/batch the actual SQL until flush or commit.

Q8. What is the difference between the four `GenerationType` strategies? `AUTO` lets the JPA provider choose the most appropriate strategy for the database; `IDENTITY` relies on a native auto-increment column, with the ID known only after INSERT; `SEQUENCE` uses a database sequence object to pre-allocate IDs; `TABLE` simulates a sequence using a dedicated table, which is portable but slower.

Q9. What is the single most common misunderstanding about `merge()`? That the object passed into `merge()` becomes managed. It does not — `merge()` copies that

object's state onto a separate managed copy and returns that copy; only the returned reference is managed and tracked for changes.

Q10. Why does `remove()` always need to be called on a managed entity, and how do the demos guarantee that? JPA has no way to correlate an arbitrary transient object with an existing database row purely from its fields, so `remove()` requires the entity to already be managed. The demo achieves this by always calling `entityManager.find(...)` first to obtain a managed reference, and only then passing that reference to `remove()`.

Q11. If `find()` returns null for a given primary key, what happens if a JPQL query (`createQuery(...).getSingleResult()`) finds no match instead? `find()` simply returns null. A JPQL query using `getSingleResult()` throws `javax.persistence.NoResultException` if no row matches — a common source of `NullPointerException` vs. unexpected exception confusion for developers switching between the two APIs.

Q12. Why is `EmployeeManagerFactory/JPAUtility` implemented as effectively a Singleton in this codebase? Building an `EntityManagerFactory` is an expensive operation (it parses `persistence.xml`, sets up connection pooling and provider internals), so the application builds it exactly once, lazily, and reuses that single instance for the lifetime of the application — the same reasoning behind the Singleton pattern in Chapter 20.

Q13. What is a POJO, and why does JPA rely on annotations rather than requiring entities to extend a framework base class? A POJO (Plain Old Java Object) is an ordinary Java class with no special framework base class or interface required just to function — simple fields with getters/setters. JPA maps POJOs to tables using annotations as metadata precisely so that entity classes remain ordinary, independently testable Java objects, rather than being tightly bound to the persistence framework's own class hierarchy.

Q14. What does the “A” in ACID stand for, and what does it guarantee? Atomicity — it guarantees that all the operations within a single transaction are treated as one indivisible unit: either every one of them succeeds and is saved, or none of them take effect at all, even if the application crashes partway through.

Q15. Why does every entity lifecycle demo in this chapter wrap its `persist()/merge()/remove()` calls in `entityManager.getTransaction().begin()/commit()`? Because any operation that modifies data requires an active transaction in JPA — without one, such a call fails with a `TransactionRequiredException`. The `begin()/commit()` pair defines the boundaries of the atomic unit of work that groups those changes together.

Chapter 23

JPA Entity Relationships

23.1 Mapping Relationships — Overview

Relational databases express relationships between tables through foreign keys. JPA lets you express the *same* relationships directly on your Java classes, using object references and collections, via four annotations that mirror the classic ER-modelling cardinalities:

Annotation	Cardinality	Example in this chapter
@OneToOne	One row relates to exactly one row in another table	One Employee has exactly one Asset
@ManyToOne	Many rows relate to one row in another table	Many Employees belong to one Department
@OneToMany	One row relates to many rows in another table	One Company has many Employees
@ManyToMany	Many rows relate to many rows, via a join table	Many Employees attend many Meetings

Every relationship demo in this chapter also uses **@JoinColumn** or **@JoinTable** to control exactly how the foreign key (or join table) is named, and **cascade** to control whether operations on the “owning” side automatically propagate to the related entity.

23.2 One-to-One (@OneToOne)

Scenario: each Employee has exactly one Asset (e.g., a laptop) issued to them.

File: Demo1007_1_JPA ORM_OneToOne/src/com/accenture/ltt/entity/EmployeeEntity.java

```
package com.accenture.ltt.entity;
```

```
import java.util.Date;
```

```
import javax.persistence.Column;
import javax.persistence.Entity;
import javax.persistence.GeneratedValue;
import javax.persistence.GenerationType;
import javax.persistence.Id;
import javax.persistence.JoinColumn;
import javax.persistence.OneToOne;
import javax.persistence.Table;
import javax.persistence.Temporal;
import javax.persistence.TemporalType;

@Entity
@Table(name = "employee")
public class EmployeeEntity {
    @Id
    @GeneratedValue(strategy= GenerationType.AUTO)
    private Integer employeeId;
    private String employeeName;
    private String role;

    @Temporal(TemporalType.DATE)
    @Column(name = "insert_time")
    private Date insertTime;

    private Double salary;

    @OneToOne
    @JoinColumn(name="assetId_fk", unique=true)
    private AssetEntity assetEntity;

    // ... getters and setters
}
```

File: Demo1007_1_JPA_ORM_OneToOne/src/com/accenture/ltt/entity/AssetEntity.java

```
package com.accenture.ltt.entity;

import javax.persistence.Column;
import javax.persistence.Entity;
import javax.persistence.GeneratedValue;
import javax.persistence.GenerationType;
import javax.persistence.Id;
import javax.persistence.Table;

@Entity
@Table(name = "assets")
public class AssetEntity {
```

```

@Id
@GeneratedValue(strategy = GenerationType.AUTO)
private int assetId;

@Column(name = "asset_name")
private String assetName;

@Column(name = "asset_brandname")
private String assetBrandName;

@Column(name = "validityyears")
private Integer validityYears;

// ... getters and setters
}

```

`@JoinColumn(name="assetId_fk", unique=true)` on the Employee side means the employee table itself gets a foreign-key column called `assetId_fk`, referencing the assets table's primary key — Employee is the **owning side** of this relationship (the side whose table holds the foreign key). Adding `unique=true` on that join column is what genuinely makes this a *one-to-one* relationship at the database level, rather than a many-to-one — it enforces that no two employees can point at the same asset row.

23.2.1 Cascading a One-to-One Relationship

Without cascading, saving an `EmployeeEntity` that references a brand-new, not-yet-persisted `AssetEntity` would fail — JPA would try to insert a foreign key pointing at an asset row that doesn't exist yet. `CascadeType` tells JPA to automatically propagate an operation performed on the owning entity to its related entity.

File: `Demo1007_2_JPA_ORM_OneToOne_persist_cascade/src/com/accenture/ltt/entity/EmployeeEntity.java` (relevant excerpt)

```

@OneToOne(cascade=CascadeType.PERSIST)
@JoinColumn(name="assetId_fk", unique=true)
private AssetEntity assetEntity;

```

With `cascade = CascadeType.PERSIST`, calling `entityManager.persist(employeeEntity)` automatically also persists the `AssetEntity` object referenced by `employeeEntity.getAssetEntity()`, as long as that `AssetEntity` was set on the employee *before* `persist()` was called — you no longer need to explicitly call `entityManager.persist(assetEntity)` yourself. `Demo1007_3_JPA_ORM_OneToOne_All_cascade` extends this further using `CascadeType.ALL`, which cascades *every* operation (`persist`, `merge`, `remove`, `refresh`, `detach`) rather than just `persist` — useful when the related entity's lifecycle is fully “owned” by the parent and should never exist independently of it.

23.3 Many-to-One (@ManyToOne)

Scenario: many employees belong to one department.

File: Demo1008_JPA ORM_ManyToOne/src/com/accenture/ltt/entity/EmployeeEntity.java

```
package com.accenture.ltt.entity;

import javax.persistence.CascadeType;
import javax.persistence.Entity;
import javax.persistence.GeneratedValue;
import javax.persistence.GenerationType;
import javax.persistence.Id;
import javax.persistence.JoinColumn;
import javax.persistence.ManyToOne;
import javax.persistence.Table;

@Entity
@Table(name = "employee")
public class EmployeeEntity {

    @Id
    @GeneratedValue(strategy= GenerationType.IDENTITY)
    private Integer employeeId;
    private String employeeName;
    private String role;
    @ManyToOne(cascade= CascadeType.PERSIST)
    @JoinColumn(name="departmentId_fk")
    private DepartmentEntity department;

    // ... getters and setters
}
```

File: Demo1008_JPA ORM_ManyToOne/src/com/accenture/ltt/entity/DepartmentEntity.java

```
package com.accenture.ltt.entity;

import javax.persistence.Entity;
import javax.persistence.GeneratedValue;
import javax.persistence.GenerationType;
import javax.persistence.Id;
import javax.persistence.Table;

@Entity
@Table(name="department")
public class DepartmentEntity {

    @Id
```

```
@GeneratedValue(strategy= GenerationType.IDENTITY)
private int departmentId;

private String departmentName;

private String city;

// ... getters and setters
}
```

@ManyToOne is nearly always the “owning side by default” in a bidirectional relationship, because the “many” side is naturally where the foreign key belongs in a relational schema — the employee table gets a departmentId_fk column, and many employee rows can legally share the same department ID. This is the mirror image of @OneToMany (§23.4) — in fact, the two are frequently modelled together on both sides of the same relationship for bidirectional navigation, though this training’s demos keep them deliberately separate and unidirectional, one demo per cardinality, to isolate each concept for study. There’s also a variant demo, Demo1008_JPA_ORM_ManyToOne_All_cascade_remove, which explores CascadeType.ALL on the @ManyToOne side — worth pausing on for viva, because cascading remove from the “many” side toward the “one” side is usually the *wrong* direction to cascade delete (deleting one employee should not delete the entire department they belonged to!) — a subtlety examiners like to probe.

23.4 One-to-Many (@OneToMany)

Scenario: one company has many employees.

File: Demo1009_JPA_ORM_OneToMany/src/com/accenture/ltt/entity/CompanyEntity.java

```
package com.accenture.ltt.entity;

import java.util.List;

import javax.persistence.CascadeType;
import javax.persistence.Entity;
import javax.persistence.GeneratedValue;
import javax.persistence.GenerationType;
import javax.persistence.Id;
import javax.persistence.JoinColumn;
import javax.persistence.JoinTable;
import javax.persistence.OneToMany;
import javax.persistence.Table;

@Entity
@Table(name = "company")
public class CompanyEntity {
    @Id
```

```

@GeneratedValue(strategy = GenerationType.IDENTITY)
private int companyId;
private String companyName;
@OneToMany(cascade = CascadeType.ALL)
@JoinTable(name = "employee_company",
            joinColumns = { @JoinColumn(name = "my_db_company_id") },
            inverseJoinColumns = { @JoinColumn(name = "my_db_employee_id", unique =
            )
private List<EmployeeEntity> employees;

// ... getters and setters
}

```

Two design choices here are worth explaining carefully:

- **Why a `List<EmployeeEntity>`?** The “many” side of a one-to-many relationship is naturally a collection — one company can have zero, one, or many employees, so the Java-side representation must be a collection type (`List`, `Set`, or `Collection`), never a single reference.
- **Why `@JoinTable` here instead of a plain `@JoinColumn` on the Employee side?** This demo models the relationship using a **separate join table** (`employee_company`), rather than putting a `companyId_fk` column directly on the employee table. This is an unusual (though valid) choice for a true one-to-many — normally the foreign key would live directly on the “many” (employee) side. Using a join table here means Hibernate manages an *extra* table purely to record the association, and the `inverseJoinColumns = { @JoinColumn(..., unique = true) }` constraint is precisely what keeps this a genuine one-to-many rather than accidentally becoming many-to-many — enforcing that a given employee row can appear in at most one company’s join-table entry.

`cascade = CascadeType.ALL` here means persisting/removing a `Company` automatically persists/removes its associated `Employee` records too — appropriate if employees are considered to exist only in the context of their company in this particular demo’s data model.

23.5 Many-to-Many (@ManyToMany)

Scenario: many employees attend many meetings, and many meetings have many attendees.

File: `Demo1010_JPA_ORM_ManyToMany/src/com/accenture/ltt/entity/EmployeeEntity.java`

```

package com.accenture.ltt.entity;

import java.util.List;

import javax.persistence.CascadeType;
import javax.persistence.Column;
import javax.persistence.Entity;

```

```
import javax.persistence.GeneratedValue;
import javax.persistence.GenerationType;
import javax.persistence.Id;
import javax.persistence.JoinColumn;
import javax.persistence.JoinTable;
import javax.persistence.ManyToMany;
import javax.persistence.Table;

@Entity
@Table(name = "employee")
public class EmployeeEntity {

    @Id
    @GeneratedValue(strategy= GenerationType.IDENTITY)
    private Integer employeeId;

    @Column(name = "name")
    private String employeeName;

    private String role;

    @ManyToMany(cascade=CascadeType.ALL)
    @JoinTable(name="employee_meeting",
                joinColumns={@JoinColumn(name="my_db_employee_id") },
                inverseJoinColumns={@JoinColumn(name="my_db_meeting_id")})
    private List<MeetingEntity> meetings;

    // ... getters and setters
}
```

File: Demo1010_JPA_ORM_ManyToMany/src/com/accenture/ltt/entity/MeetingEntity.java

```
package com.accenture.ltt.entity;

import javax.persistence.Entity;
import javax.persistence.GeneratedValue;
import javax.persistence.GenerationType;
import javax.persistence.Id;
import javax.persistence.Table;

@Entity
@Table(name="meetings")
public class MeetingEntity {
    @Id
    @GeneratedValue(strategy=GenerationType.IDENTITY)
    private int meetingId;
    private String timeSlot;
```

```
// ... getters and setters
}
```

A genuine many-to-many relationship **cannot** be represented with a foreign key on either side alone — no single column on employee could record “this employee attends these three meetings” while also letting meetings record “this meeting has these five attendees.” The only correct relational solution is a **join table** with two foreign keys, one referencing each side — here, `employee_meeting(my_db_employee_id, my_db_meeting_id)`. Notice this is the *same* `@JoinTable` mechanism used in the (unusual) one-to-many demo above — the difference is that in a true many-to-many, **neither** `inverseJoinColumn` column is marked unique, since both an employee can attend many meetings *and* a meeting can have many employees; adding `unique=true` to either side, as the one-to-many demo did, is exactly what would collapse this into a one-to-many or one-to-one instead.

23.6 Cascade Types and Fetch Types — Quick Reference

CascadeType values — what operations propagate from the owning entity to the related entity:

Value	Propagates
PERSIST	Saving the parent also saves the (not-yet-saved) child
MERGE	Merging the parent also merges the child
REMOVE	Removing the parent also removes the child
REFRESH	Refreshing the parent’s state from the DB also refreshes the child
DETACH	Detaching the parent also detaches the child
ALL	All of the above

FetchType values — *when* the related entity/collection is actually loaded from the database (not used explicitly in every demo above, since JPA applies sensible defaults, but essential to know for viva):

Value	Behaviour	JPA default for
EAGER	Related data is loaded immediately, together with the owning entity, in the same query (or an immediate follow-up query)	<code>@OneToOne</code> , <code>@ManyToOne</code>

Value	Behaviour	JPA default for
LAZY	Related data is loaded only the first time it's actually accessed (via a proxy)	@OneToMany, @ManyToMany

Why this default split exists: a @ManyToOne/@OneToOne reference is a single related object, cheap to fetch eagerly; a @OneToMany/@ManyToMany reference is a potentially large collection, so JPA defaults to lazy loading to avoid accidentally pulling in huge amounts of related data every time the owning entity is loaded. A very common runtime pitfall this leads to — `org.hibernate.LazyInitializationException` — happens when code tries to access a lazily-loaded collection *after* the EntityManager that loaded it has already been closed; there's no more active persistence context left to fetch the data on demand.

23.7 Viva Voce — Likely Questions & Answers

Q1. What is the “owning side” of a JPA relationship, and why does it matter?

The owning side is the entity whose table holds the foreign key (or, for many-to-many, whose mapping is treated as authoritative for maintaining the join table). It matters because JPA only persists relationship changes made through the owning side — changes made purely on the inverse/mappedBy side of a bidirectional relationship are not, by themselves, written to the database.

Q2. How do you enforce that a @OneToOne relationship is genuinely one-to-one at the database level, rather than accidentally many-to-one? By adding `unique = true` to the @JoinColumn on the owning side — this places a unique constraint on the foreign-key column, preventing two rows from referencing the same related row.

Q3. Why must the “many” side of a @OneToMany relationship be a collection type in Java? Because the cardinality on that side is inherently plural — one parent can be associated with zero, one, or many children — so a single object reference cannot represent it; List, Set, or Collection is required.

Q4. Why can't a many-to-many relationship be represented with a single foreign key on either table? Because a foreign key column can only reference one row per row of its own table. In a many-to-many relationship, both sides need to reference multiple rows on the other side simultaneously, which requires an intermediate join table holding one foreign key per side.

Q5. What does `cascade = CascadeType.PERSIST` actually do, concretely? When `persist()` is called on the owning (parent) entity, JPA automatically also calls `persist()` on any related entity referenced by that parent (as long as that related entity was set before the parent's `persist()` call) — you don't need to explicitly persist the child yourself.

Q6. Why might cascading REMOVE from a @ManyToOne side toward the “one” side (e.g., deleting an Employee cascading to delete the whole Department) be a

design mistake? Because it inverts the natural ownership direction — many employees belong to one department, but deleting a single employee should never delete the entire department (and, by extension, every other employee in it). Cascade-remove should generally flow from the “owning”/parent side down to genuinely dependent children, not upward toward a shared parent.

Q7. What is the default fetch type for @OneToMany/@ManyToMany, versus @OneToOne/@ManyToOne, and why does the default differ? @OneToMany/@ManyToMany default to LAZY (loaded only on first access), while @OneToOne/@ManyToOne default to EAGER (loaded immediately). The difference exists because “many” side collections can be arbitrarily large and expensive to fetch every time, while a single related object is comparatively cheap to load eagerly.

Q8. What causes LazyInitializationException, and how is it typically avoided?

It occurs when code tries to access a lazily-loaded association or collection after the EntityManager/persistence context that originally loaded the owning entity has already been closed — there is no active session left to fetch the data on demand. It’s typically avoided by accessing lazy associations while the persistence context is still open, or by using an explicit JPQL JOIN FETCH to eagerly load exactly the data needed for a specific query.

Q9. In the @ManyToMany demo, what would happen if unique = true were added to one side of the @JoinTable’s inverseJoinColumn? It would incorrectly restrict that side to appearing at most once across the join table — effectively collapsing the relationship into a one-to-many (or one-to-one, if applied to both sides) instead of a genuine many-to-many.

Chapter 24

JPQL, Named Queries & Composite Keys

24.1 JPQL vs SQL

JPQL (Java Persistence Query Language) looks like SQL but operates on a fundamentally different vocabulary: JPQL queries are written against **entity names and their field names**, not table names and column names. `SELECT e FROM EmployeeEntity e` refers to the *entity class* `EmployeeEntity` and returns full `EmployeeEntity` objects — not rows, not a `ResultSet` — because JPQL is, by design, object-oriented and database-agnostic; the JPA provider translates it into the actual SQL appropriate for whichever database is configured.

24.2 Basic JPQL — `createQuery()` and Two Equivalent Syntaxes

File: `Demo1012_1_JPA ORM JPQL_nostructure/src/com/accenture/ltt/dao/EmployeeDAOIMPL.java`

```
package com.accenture.ltt.dao;

import java.util.List;

import javax.persistence.EntityManager;
import javax.persistence.EntityManagerFactory;
import javax.persistence.Query;

import com.accenture.ltt.entity.EmployeeEntity;
import com.accenture.ltt.utility.JPAUtility;

public class EmployeeDAOIMPL implements EmployeeDAO {

    public void retrieveEmployeeDetails() throws Exception{
        EntityManager entityManager = null;
```

```
try {
    EntityManagerFactory entityManagerFactory = JPAUtility.getEntityManagerFactory();
    entityManager = entityManagerFactory.createEntityManager();
    System.out.println("Retrieve results using JPQL");
    Query query = entityManager.createQuery("SELECT e FROM EmployeeEntity e");
    List<EmployeeEntity> employeeList = query.getResultList();
    for(EmployeeEntity e: employeeList) {
        System.out.println(e.getEmployeeId()+" "+e.getEmployeeName()+" "+e.getEmployeeSalary());
    }
} catch (Exception exception) {
    throw exception;
}
finally{
    if(entityManager!=null){
        entityManager.close();
    }
}

}

public void retrieveEmployeeDetailsUsingHibernateProvider() throws Exception{
    EntityManager entityManager = null;
    try {
        EntityManagerFactory entityManagerFactory = JPAUtility.getEntityManagerFactory();
        entityManager = entityManagerFactory.createEntityManager();

        System.out.println("Retrieve results using JPQL");
        Query query = entityManager.createQuery("FROM EmployeeEntity");
        List<EmployeeEntity> employeeList = query.getResultList();
        for(EmployeeEntity e: employeeList) {
            System.out.println(e.getEmployeeId()+" "+e.getEmployeeName()+" "+e.getEmployeeSalary());
        }

        entityManager.clear();
    } catch (Exception exception) {
        throw exception;
    }
    finally{
        if(entityManager!=null){
            entityManager.close();
        }
    }
}

}
```

Two syntaxes are shown side by side deliberately: "SELECT e FROM EmployeeEntity e" is standard JPQL as defined by the JPA specification — portable across any JPA provider. "FROM EmployeeEntity" (without an explicit SELECT) is a shorthand accepted by **Hibernate specifically** as a convenience extension (inherited from Hibernate's own older HQL — Hibernate Query Language, which JPQL was itself modelled on) — it works correctly here because the persistence provider is Hibernate, but it is *not* guaranteed to work if the persistence provider were switched to a different JPA implementation. This is precisely why the method is named ...UsingHibernateProvider — the demo is deliberately marking that shorthand as provider-specific, not part of the portable JPA standard.

`query.getResultList()` always returns a `List` — even if zero or one row matches — which is why it never throws an exception for “no results found,” unlike `getSingleResult()` (used when you expect exactly one row), which throws `NoResultException` if nothing matches, or `NonUniqueResultException` if more than one row matches.

24.3 Returning Structured Data — Mapping Entities Back to DTOs/Beans

File: Demo1012_2_JPA ORM_JPQL_structure/src/com/accenture/ltt/dao/EmployeeDAOIMPL.java (key excerpt)

```
public List<EmployeeBean> retrieveEmployeeDetails() throws Exception{
    EntityManager entityManager = null;
    List<EmployeeBean> listEmployeeBean = null;
    try {
        EntityManagerFactory entityManagerFactory = JPAUtility.getEntityManagerFactory();
        entityManager = entityManagerFactory.createEntityManager();

        Query query = entityManager.createQuery("SELECT e FROM EmployeeEntity e");
        List<EmployeeEntity> employeeList = query.getResultList();
        listEmployeeBean = new ArrayList<EmployeeBean>();
        for(EmployeeEntity e: employeeList) {
            EmployeeBean employeeBean = new EmployeeBean();
            employeeBean.setEmployeeId(e.getEmployeeId());
            employeeBean.setEmployeeName(e.getEmployeeName());
            employeeBean.setInsertTime(e.getInsertTime());
            employeeBean.setRole(e.getRole());
            employeeBean.setSalary(e.getSalary());
            listEmployeeBean.add(employeeBean);
        }

    } catch (Exception exception) {
        throw exception;
    }
    finally{
        if(entityManager!=null){
```

```

        entityManager.close();
    }
    }
    return listEmployeeBean;
}

```

This demo, named `_structure` (as opposed to `_nostructure` in §24.2), makes an important architectural point: the DAO layer should not leak JPA entity objects (which are tied to a persistence context and become useless/detached once the `EntityManager` closes) out to the service/UI layers. Instead, each `EmployeeEntity` retrieved is explicitly copied, field by field, into a plain `EmployeeBean` — a detached, framework-independent data-holder that’s safe to pass around and use freely even after the `EntityManager` that originally produced the data has been closed. This is the same DTO (Data Transfer Object) discipline that shows up again and again across full-stack layers — controllers returning DTOs rather than raw entities, for exactly the same reason.

24.4 Selecting Specific Columns (Projections)

File: `Demo1012_3_JPA_ORM_JPQL_structure_SelectedColumns/src/com/accenture/ltt/dao/Employ`
(key excerpts)

```

public List<String> retrieveEmployeeNames() throws Exception{
    EntityManager entityManager = null;
    List<String> listOfNames=null;
    try {
        EntityManagerFactory entityManagerFactory = JPAUtility.getEntityManagerFactory();
        entityManager = entityManagerFactory.createEntityManager();

        Query query = entityManager.createQuery("SELECT e.employeeName FROM EmployeeEnt
        listOfNames = query.getResultList();

    } catch (Exception exception) {
        throw exception;
    }
    finally{
        if(entityManager!=null){
            entityManager.close();
        }
    }
    return listOfNames;
}

public List<EmployeeBean> retrieveEmployeeIdAndNameColumns() throws Exception{
    EntityManager entityManager = null;

```

```

List<EmployeeBean> listEmployeeBean = null;
try {
    EntityManagerFactory entityManagerFactory = JPAUtility.getEntityManagerFactory();
    entityManager = entityManagerFactory.createEntityManager();
    Query query = entityManager.createQuery("select k.employeeId, k.employeeName FROM EmployeeEntity k");
    List<Object> nameAndIdList = query.getResultList();
    listEmployeeBean = new ArrayList<EmployeeBean>();
    for(Object e: nameAndIdList) {
        Object[] obj = (Object[])e;
        EmployeeBean employeeBean = new EmployeeBean();
        employeeBean.setEmployeeId((Integer)obj[0]);
        employeeBean.setEmployeeName((String)obj[1]);
        listEmployeeBean.add(employeeBean);
    }
} catch (Exception exception) {
    throw exception;
}
// finally block omitted here for brevity
return listEmployeeBean;
}

```

This demonstrates the two shapes JPQL projections can take:

- **Selecting a single column** (SELECT e.employeeName FROM EmployeeEntity e) returns a List<String> directly — each element is simply the raw value of that one field.
- **Selecting multiple columns** (select k.employeeId, k.employeeName FROM EmployeeEntity k) returns a List<Object>, where **each element is actually an Object[]** — one array per matched row, positionally ordered exactly as written in the SELECT clause. The code must cast each element to Object[] and then cast each individual array slot to its actual expected type ((Integer)obj[0], (String)obj[1]) — a very common source of ClassCastException if the column order in the query doesn't match the casting order in the Java code, and a frequent interview “gotcha” question.

24.5 Parameter Passing — Positional and Named Parameters

File: Demo1012_4_JPA_ORM_JPQL_structure_Parameters_Passing/src/com/accenture/ltt/dao/EmployeeDao.java (excerpt)

```

public List<EmployeeBean> retrieveEmployeeDetailsWithinSalaryRange1(Double lowerBound, Double upperBound) {
    EntityManager entityManager = null;
    List<EmployeeBean> listEmployeeBean = null;
    try {
        EntityManagerFactory entityManagerFactory = JPAUtility.getEntityManagerFactory();
    }
}

```

```

        entityManager = entityManagerFactory.createEntityManager();

        Query query = entityManager.createQuery("SELECT e FROM EmployeeEntity e where e
        query.setParameter(1, lowerBound);
        query.setParameter(2, upperBound);
        List<EmployeeEntity> employeeList = query.getResultList();
        // ... map to EmployeeBean list, as in earlier demos
    } catch (Exception exception) {
        throw exception;
    }
    // finally block omitted here for brevity
    return listEmployeeBean;
}

```

JSQL supports two parameter-binding styles, both of which prevent SQL-injection-style issues the exact same way PreparedStatement did in Chapter 21 — the parameter value is bound separately from the query text, never concatenated into it:

- **Positional parameters** — ?1, ?2, ... in the query string, bound via query.setParameter(1, value). The numbering is explicit in the query text itself and must match exactly between query and binding calls.
- **Named parameters** — :paramName in the query string, bound via query.setParameter("paramName", value). Named parameters are generally considered more readable and less error-prone for queries with several parameters, since the binding call is self-documenting rather than a bare number.

Both styles appear across this training's demos — for example, the DML demo in §24.6 mixes an unusual positional index (?22) with a named parameter (:p1) in the very same query, specifically to show that both mechanisms are always available and can even coexist (though mixing styles in real production code is discouraged purely for readability).

24.6 JPQL Functions — String and Date/Time Functions

File: Demo1012_5_JPA ORM JPQL_structure_Functions/src/com/accenture/lit/dao/EmployeeDAOImpl (excerpt)

```

public List<Object> stringFunctionsDemo() throws Exception {
    EntityManager entityManager = null;
    List<Object> list = null;
    try {
        EntityManagerFactory entityManagerFactory = JPAUtility.getEntityManagerFactory();
        entityManager = entityManagerFactory.createEntityManager();

        Query query = entityManager.createQuery(
            "select k.employeeName,uPPER(k.employeeName),LOWER(k.employeeName),SUBSTRIN
        list = query.getResultList();
    }
}

```



```

    } catch (Exception exception) {
        throw exception;
    }
    // finally block omitted here for brevity
    return list;
}

public List<Object> dateFunctionsDemo() throws Exception{
    EntityManager entityManager = null;
    List<Object> list =null;
    try {
        EntityManagerFactory entityManagerFactory = JPAUtility.getEntityManagerFactory();
        entityManager = entityManagerFactory.createEntityManager();

        Query query = entityManager.createQuery(
            "select k.insertTime, YEAR(k.insertTime), mOnTh(k.insertTime), DAY(k.insertTime) from k");
        list = query.getResultList();

    } catch (Exception exception) {
        throw exception;
    }
    // finally block omitted here for brevity
    return list;
}

```

JPQL defines a standard set of built-in functions usable inside queries, most notably:

- **String functions:** UPPER(), LOWER(), SUBSTRING(string, start, length), LENGTH(), CONCAT(), TRIM().
- **Date functions:** CURRENT_DATE, CURRENT_TIME, CURRENT_TIMESTAMP, and the extraction functions YEAR(), MONTH(), DAY().

A detail worth noticing in the source (and a nice “did you actually read the code” viva check): the demo deliberately writes function names in mixed case — `uPpER(...)`, `mOnTh(...)` — to demonstrate that **JPQL keywords and built-in function names are case-insensitive** (only entity names and field names are case-sensitive, since they map directly to Java class/field identifiers).

24.6.1 Aggregate Functions and GROUP BY

Beyond the string and date functions shown in the demo above, JPQL also supports the standard SQL **aggregate functions** — used to compute a single summary value across many rows, rather than returning one value per row:

Function	Computes
COUNT(e)	The number of matching rows
SUM(e.salary)	The total of a numeric field across matching rows
AVG(e.salary)	The average of a numeric field across matching rows
MIN(e.salary)	The smallest value of a field across matching rows
MAX(e.salary)	The largest value of a field across matching rows

A query like `SELECT COUNT(e) FROM EmployeeEntity e WHERE e.role = 'Analyst'` returns a single Long — the count of analysts — rather than a list of `EmployeeEntity` objects. When an aggregate is combined with one or more plain (non-aggregated) fields in the same SELECT, JPQL requires a **GROUP BY** clause listing those plain fields, exactly as standard SQL does — for example, `SELECT e.role, AVG(e.salary) FROM EmployeeEntity e GROUP BY e.role` returns one row per distinct role, each paired with that role’s average salary. Filtering on an *aggregated* value (e.g., “only roles whose average salary exceeds 50000”) requires **HAVING** rather than **WHERE**, since **WHERE** filters individual rows *before* grouping happens, while **HAVING** filters the grouped results *after* aggregation — `SELECT e.role, AVG(e.salary) FROM EmployeeEntity e GROUP BY e.role HAVING AVG(e.salary) > 50000`. This WHERE-vs-HAVING distinction is one of the most commonly asked SQL/JPQL interview questions, precisely because the two clauses look similar but operate at different stages of query evaluation.

24.7 JPQL DML — Bulk Update and Delete Queries

File: Demo1013_JPA ORM_JPQL_DML_Queries/src/com/accenture/ltt/dao/EmployeeDAOIMPL.java

```
package com.accenture.ltt.dao;

import javax.persistence.EntityManager;
import javax.persistence.EntityManagerFactory;
import javax.persistence.Query;

import com.accenture.ltt.utility.JPAUtility;

public class EmployeeDAOIMPL implements EmployeeDAO {

    public Integer updateSalaryOfEmployeeWithName(Double salary,String employeeName) throws Exception {
        EntityManager entityManager = null;
        Integer res =null;
        try {
            EntityManagerFactory entityManagerFactory = JPAUtility.getEntityManagerFactory();
            entityManager = entityManagerFactory.createEntityManager();
        } catch (Exception e) {
            e.printStackTrace();
        }
        Query query = entityManager.createQuery("UPDATE EmployeeEntity e SET e.salary = :salary WHERE e.role = :role");
        query.setParameter("salary", salary);
        query.setParameter("role", employeeName);
        res = query.executeUpdate();
        return res;
    }
}
```

```

        Query query = entityManager.createQuery("update EmployeeEntity k set k.salary = :salary");
        query.setParameter(22, salary);
        query.setParameter("p1", employeeName);

        entityManager.getTransaction().begin();
        res = query.executeUpdate();
        entityManager.getTransaction().commit();

    } catch (Exception exception) {
        throw exception;
    }
    finally{
        if(entityManager!=null){
            entityManager.close();
        }
    }
    return res;
}

public Integer deleteEmployeeWithNameAs(String employeeName) throws Exception{
    EntityManager entityManager = null;
    Integer res =null;
    try {
        EntityManagerFactory entityManagerFactory = JPAUtility.getEntityManagerFactory();
        entityManager = entityManagerFactory.createEntityManager();
        Query query = entityManager.createQuery("delete from EmployeeEntity k where k.name = :name");
        query.setParameter("p2", employeeName);
        entityManager.getTransaction().begin();
        res = query.executeUpdate();
        entityManager.getTransaction().commit();
    } catch (Exception exception) {
        throw exception;
    }
    finally{
        if(entityManager!=null){
            entityManager.close();
        }
    }
    return res;
}
}

```

Two crucial differences from every earlier read-only JPQL example in this chapter:

1. **executeUpdate()**, not **getResultList()**. Bulk UPDATE/DELETE JPQL statements

are executed with `query.executeUpdate()`, which returns an `int` — the number of rows affected — exactly mirroring `Statement.executeUpdate()` from JDBC.

2. **A transaction is mandatory.** Unlike a plain `SELECT`, any JPQL statement that modifies data must be wrapped in `entityManager.getTransaction().begin()` / `.commit()`, or JPA will throw a `TransactionRequiredException`.

Important caveat worth flagging for viva: bulk JPQL `UPDATE/DELETE` statements operate **directly against the database**, bypassing the persistence context entirely — meaning any `EmployeeEntity` objects *already loaded and managed* in the current persistence context are **not automatically updated in memory** to reflect the bulk change, which can lead to stale in-memory data if you're not careful to clear/refresh the persistence context afterward.

24.8 Named Queries — `@NamedQuery` and `@NamedQueries`

A **named query** moves the JPQL string out of the DAO method body and onto the entity class itself, as a compile-time-checked, centrally defined, reusable query identified by a string name.

File: `Demo1014_JPA ORM JPQL_namedQuery/src/com/accenture/ltt/entity/EmployeeEntity.java` (excerpt)

```
@Entity
@Table(name = "Employee")
@NamedQuery(name="findAllEmployeeSalaryRangePositionalParam",query="SELECT e FROM Empl
public class EmployeeEntity {
    // ... fields, getters, setters
}
```

File: `Demo1014_JPA ORM JPQL_namedQuery/src/com/accenture/ltt/dao/EmployeeDAOIMPL.java` (excerpt)

```
Query query = entityManager.createNamedQuery("findAllEmployeeSalaryRangePositionalParam
query.setParameter(1, lowerBound);
query.setParameter(2, upperBound);
List<EmployeeEntity> employeeList = query.getResultList();
```

`entityManager.createNamedQuery(name)` looks the query up by name, rather than taking a raw JPQL string — the DAO code no longer needs to know or repeat the actual query text. When multiple named queries are needed on the same entity, `@NamedQueries` groups them:

File: `Demo1015_JPA ORM JPQL_namedQueries/src/com/accenture/ltt/entity/EmployeeEntity.java` (excerpt)

```
@Entity
@Table(name = "Employee")
@NamedQueries(
{
    @NamedQuery(name="findAllEmployeeSalaryRangePositionalParam",query="SELECT e FF
```

```

        @NamedQuery(name="findEmployeesByNameLikeorderBySalaryDesc", query="Select e from EmployeeEntity e where e.name like :name order by e.salary desc")
    })
    public class EmployeeEntity {
        // ...
    }

```

Why named queries are worth using at all — the reasons that come up most often in interviews:

- **Centralisation** — the query text lives in exactly one place (the entity), rather than being duplicated or slightly re-worded across multiple DAO methods.
- **Early validation** — Hibernate parses and validates every `@NamedQuery` at `EntityManagerFactory` creation time (i.e., at application startup), so a typo in the JPQL fails fast, at boot, rather than only when that specific DAO method finally gets called at runtime.
- **Reusability** — any DAO can call `createNamedQuery("findAllEmployeeSalaryRangePositional")` from anywhere, without needing to know or re-type the underlying JPQL.

Named queries can also be defined **externally**, in an `orm.xml` mapping file, rather than as annotations directly on the entity class — useful when the query needs to be changed without recompiling the entity, or when keeping the entity class free of query-text clutter is preferred (Demo1016_JPA_ORM_JPQL_namedQueries_ORM_XML in the supplied demo set explores exactly this variant).

24.9 Managed vs Non-Managed Relations in JPQL Joins

Two closely related demos — Demo1017_JPA_ORM_OneToOne_JPQL_nonmanagedrelation and Demo1018_JPA_ORM_OneToOne_JPQL_JPA_Managed_Relation — explore the distinction between writing a JPQL query that **navigates an actual mapped JPA association** (e.g., `SELECT e FROM EmployeeEntity e WHERE e.assetEntity.assetName = :name`, walking the `@OneToOne` relationship declared on the entity) versus a query that manually joins two otherwise-unrelated entities using explicit `JOIN` syntax without relying on an annotated association at all. The practical takeaway for viva: when a relationship is already properly mapped with `@OneToOne/@ManyToOne/etc.`, JPQL lets you navigate it with simple dot notation (`e.assetEntity.assetName`) directly in the query, and JPA/Hibernate generates the correct SQL join automatically — you do not need to hand-write a `JOIN` clause yourself unless the entities are not already related through a mapped association.

24.10 Composite Primary Keys — `@EmbeddedId`

Sometimes a table's natural primary key is made up of more than one column (e.g., an `Employee_Composite_Key` table keyed by both an employee ID *and* a department ID together). JPA supports this via `@EmbeddedId`, using a separate `@Embeddable` class to represent the composite key as a single Java object.

File: Demo1019_JPA_ORM_CompositeKey_EmbeddedId/src/com/accenture/ltt/entity/EmployeeCmpP

```
package com.accenture.ltt.entity;

import java.io.Serializable;
import java.util.Objects;

import javax.persistence.Embeddable;

@Embeddable
public class EmployeeCmpPK implements Serializable{

    private Integer id;

    private Integer departmentId;

    public Integer getId() {
        return id;
    }

    public void setId(Integer id) {
        this.id = id;
    }

    public Integer getDepartmentId() {
        return departmentId;
    }

    public void setDepartmentId(Integer departmentId) {
        this.departmentId = departmentId;
    }

    @Override
    public int hashCode() {
        return Objects.hash(departmentId, id);
    }

    @Override
    public boolean equals(Object obj) {
        if (this == obj)
            return true;
        if (obj == null)
            return false;
        if (getClass() != obj.getClass())
            return false;
        EmployeeCmpPK other = (EmployeeCmpPK) obj;
        return Objects.equals(departmentId, other.departmentId) && Objects.equals(id, other.id);
    }
}
```

```
}
```

File: Demo1019_JPA ORM_CompositeKey_EmbeddedId/src/com/accenture/ltt/entity/EmployeeEnti

```
package com.accenture.ltt.entity;

import java.util.Date;

import javax.persistence.EmbeddedId;
import javax.persistence.Entity;
import javax.persistence.Table;
import javax.persistence.Temporal;
import javax.persistence.TemporalType;

@Entity
@Table(name="Employee_Composite_Key")
public class EmployeeEntity {

    @EmbeddedId
    private EmployeeCmpPK employeeCmpPK;
    private String name;
    private String role;

    @Temporal(TemporalType.DATE)
    private Date insertTime;

    private Double salary;

    // ... getters and setters, including getEmployeeCmpPK()/setEmployeeCmpPK()
}
```

Three requirements are non-negotiable for a class used as an `@EmbeddedId`, and are exactly the ones satisfied here — worth reciting from memory for viva:

1. It must be annotated **@Embeddable**.
2. It must implement **Serializable** — JPA providers may need to serialise composite keys (e.g., for second-level caching or for use as Map keys internally).
3. It must correctly override **equals()** and **hashCode()**, based on *all* the fields that make up the key. This is not optional or stylistic — JPA/Hibernate relies on these methods to determine row identity/equality for a composite key, since there's no longer a single simple value (like a plain Integer `employeeId`) to compare directly; `Objects.hash(...)` and a field-by-field `equals()`, exactly as written above, is the standard, correct pattern.

The owning entity then declares the composite key field with **@EmbeddedId** (instead of **@Id**), typed as the `@Embeddable` class — `entityManager.find(EmployeeEntity.class, someEmployeeCmpPKInstance)` is how you'd look up a row by composite key, passing a fully-populated `EmployeeCmpPK` instance rather than a single scalar value.

24.11 Viva Voce — Likely Questions & Answers

Q1. What is the fundamental difference between JPQL and SQL? JPQL queries are written against entity classes and their field names, and return entity objects (or projections of specific fields) — it is object-oriented and provider/database-agnostic. SQL queries are written against table and column names and return raw rows; JPQL is translated into the appropriate SQL for the configured database by the JPA provider underneath.

Q2. Is `FROM EmployeeEntity` (without `SELECT`) standard, portable JPQL? No — it's a Hibernate-specific shorthand inherited from HQL. It works when Hibernate is the JPA provider but is not guaranteed to be portable to a different JPA implementation; standard portable JPQL requires an explicit `SELECT e FROM EmployeeEntity e`.

Q3. What is the difference between `getResultList()`, `getSingleResult()`, and `executeUpdate()`? `getResultList()` always returns a `List` (possibly empty) and never throws for zero matches. `getSingleResult()` expects exactly one row — it throws `NoResultException` for zero matches and `NonUniqueResultException` for more than one. `executeUpdate()` is used for bulk `UPDATE/DELETE` JPQL and returns an `int` row-count.

Q4. When a JPQL query selects multiple columns (e.g. `select k.employeeId, k.employeeName ...`), what type does `getResultList()` actually return? A `List<Object>`, where each element must be cast to `Object[]` — one array per matched row — with each array slot corresponding positionally to a selected column, in the order written in the query.

Q5. What is the difference between JPQL positional parameters and named parameters? Positional parameters use `?n` placeholders bound via `setParameter(n, value)`, matched purely by position/number in the query text. Named parameters use `:name` placeholders bound via `setParameter("name", value)`, which is generally more readable for queries with multiple parameters.

Q6. Why must bulk JPQL `UPDATE/DELETE` statements be wrapped in a transaction, unlike a `SELECT`? Because they modify data — JPA requires an active transaction for any data-modifying operation, and will throw `TransactionRequiredException` if `executeUpdate()` is called without one; a read-only `SELECT` has no such requirement.

Q7. Does a bulk JPQL `UPDATE/DELETE` automatically update entities already loaded into the current persistence context? No — bulk JPQL DML operates directly against the database, bypassing the persistence context. Any already-managed entity objects in memory are not automatically refreshed to reflect the bulk change, which can cause the in-memory state to become stale unless explicitly refreshed or the context is cleared.

Q8. What are the concrete benefits of using `@NamedQuery` instead of an inline JPQL string in the DAO? Centralising the query text in one place (on the entity) instead of duplicating it across DAO methods; early validation, since Hibernate parses and checks named queries when the `EntityManagerFactory` is built (at application startup) rather than only when the query is actually executed; and easy reuse from

any DAO by name, without needing to know the underlying JPQL text.

Q9. What three requirements must a class satisfy to be used as an `@EmbeddedId`? It must be annotated `@Embeddable`, implement `Serializable`, and correctly override `equals()/hashCode()` based on all fields that make up the composite key.

Q10. Why is overriding `equals()/hashCode()` non-negotiable for a composite key class, specifically? Because with a composite key there is no single simple identifier value to compare directly (unlike a plain `Integer` primary key) — JPA/Hibernate relies on `equals()/hashCode()` to determine whether two composite key instances (and, by extension, two entity rows) represent the same identity, for correctness in caching, collections, and persistence-context bookkeeping.

Q11. How would you look up a row by its composite key using `EntityManager.find()`? By passing a fully-populated instance of the `@Embeddable` key class as the second argument — e.g. `entityManager.find(EmployeeEntity.class, employeeCmpPKInstance)` — rather than a single scalar ID value.

Q12. Name the five standard aggregate functions available in JPQL. `COUNT()`, `SUM()`, `AVG()`, `MIN()`, and `MAX()` — each computing a single summary value across a set of matching rows, rather than returning one value per row.

Q13. What is the difference between `WHERE` and `HAVING` in a JPQL (or SQL) query that uses `GROUP BY`? `WHERE` filters individual rows *before* grouping/aggregation happens. `HAVING` filters the *grouped, aggregated* results *after* grouping happens — so a condition on an aggregate function (like “average salary greater than 50000”) must go in `HAVING`, since that value doesn’t exist yet at the point `WHERE` is evaluated.

Chapter 25

Full CRUD Case Study — A Layered JPA Application

25.1 Why This Chapter Exists

Chapters 22–24 studied each JPA operation in isolation — one demo per lifecycle transition, one demo per query style — deliberately, so each concept could be understood on its own without other moving parts adding noise. This chapter puts the pieces back together into a single, realistic, layered application: a **Customer** management system supporting full CRUD plus two more advanced query-style operations, structured exactly the way a real assignment (and a real production module) would be — entity -> dao -> service -> ui, wired together through a Factory.

This chapter is also structured deliberately as a **practice activity**: the actual training material hands trainees a project with the entity, interfaces, and *some* DAO methods already fully implemented (to serve as a worked example), and other methods deliberately left as `// Your Code goes here` stubs, to be completed by the trainee using exactly the techniques from Chapters 22–24. Both the worked examples and the stubs are reproduced below.

25.2 The Entity

File: Activity_1001_JPA_ORM_CRUD_To_Trainee/src/com/accenture/ltt/entity/CustomerEntity.

```
package com.accenture.ltt.entity;

import java.util.Date;

import javax.persistence.Entity;
import javax.persistence.GeneratedValue;
import javax.persistence.GenerationType;
import javax.persistence.Id;
import javax.persistence.Table;
import javax.persistence.Temporal;
```

```
import javax.persistence.TemporalType;

@Entity
@Table(name = "Customer")
public class CustomerEntity {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer customerId;
    private String customerName;
    @Temporal(TemporalType.DATE)
    private Date purchaseDate;
    private Double bill;

    public Integer getCustomerId() {
        return customerId;
    }

    public void setCustomerId(Integer customerId) {
        this.customerId = customerId;
    }

    public String getCustomerName() {
        return customerName;
    }

    public void setCustomerName(String customerName) {
        this.customerName = customerName;
    }

    public Date getPurchaseDate() {
        return purchaseDate;
    }

    public void setPurchaseDate(Date purchaseDate) {
        this.purchaseDate = purchaseDate;
    }

    public Double getBill() {
        return bill;
    }

    public void setBill(Double bill) {
        this.bill = bill;
    }
}
```

Nothing here is new — `@GeneratedValue(strategy = GenerationType.IDENTITY)` (Chapter 22, §22.5.1) and `@Temporal(TemporalType.DATE)` (used identically in the Entity Relationships and JPQL chapters) — the point of this chapter is *assembling* familiar pieces into a complete, working slice of an application, not introducing new annotations.

25.3 The Worked Example — `findCustomerById()` and `updateCustomerBillById()`

File: `Activity_1001_JPA_ORM_CRUD_To_Trainee/src/com/accenture/ltt/dao/CustomerDAOIMPL.java` (worked-example excerpt)

```
public CustomerBean findCustomerById(Integer customerId) throws Exception {
    CustomerBean customer = null;
    EntityManager entityManager = null;
    try {
        EntityManagerFactory entityManagerFactory = JPAUtility.getEntityManagerFactory();
        entityManager = entityManagerFactory.createEntityManager();

        CustomerEntity customerEntity = (CustomerEntity) entityManager.find(CustomerEntity.class, customerId);

        if (customerEntity != null) {
            customer = new CustomerBean();
            customer.setBill(customerEntity.getBill());
            customer.setCustomerId(customerEntity.getCustomerId());
            customer.setCustomerName(customerEntity.getCustomerName());
            customer.setPurchaseDate(customerEntity.getPurchaseDate());
        }

    } catch (Exception exception) {
        throw exception;
    } finally {
        if (entityManager != null) {
            entityManager.close();
        }
    }

    return customer;
}

public CustomerBean updateCustomerBillById(CustomerBean customerBean) throws Exception {
    CustomerBean customerRet = null;
    EntityManager entityManager = null;
    try {
        EntityManagerFactory entityManagerFactory = JPAUtility.getEntityManagerFactory();
        entityManager = entityManagerFactory.createEntityManager();
```

```

        CustomerEntity customerEntity = (CustomerEntity) entityManager.find(CustomerEnt

        if (customerEntity != null) {

            entityManager.getTransaction().begin();
            customerEntity.setBill(customerBean.getBill());
            entityManager.getTransaction().commit();
            customerRet = new CustomerBean();
            customerRet.setBill(customerEntity.getBill());
            customerRet.setCustomerId(customerEntity.getCustomerId());
            customerRet.setCustomerName(customerEntity.getCustomerName());
            customerRet.setPurchaseDate(customerRet.getPurchaseDate());
        }

    } catch (Exception exception) {

        throw exception;
    } finally {
        if (entityManager != null) {
            entityManager.close();
        }
    }

    return customerRet;
}

```

The `updateCustomerBillById()` method is worth pausing on, because it demonstrates a technique **not shown explicitly** in Chapter 22's `isolated merge()` demo: **dirty checking**. Notice there is no explicit `entityManager.merge(...)` call anywhere here. Instead, the method `find()`s the managed `CustomerEntity`, and simply calls `customerEntity.setBill(...)` on it **directly, while the transaction is open**. Because `customerEntity` is already managed (it came from `find()`), Hibernate automatically detects that one of its fields changed — this is called **dirty checking** — and generates the corresponding `UPDATE SQL` automatically at `commit()` time, with no explicit `save/update/merge` call needed at all. This is arguably the single most powerful (and most surprising to JDBC-trained developers) feature of JPA's managed-entity model: *for already-managed entities, simply mutating a field inside an active transaction is enough to persist the change*.

25.4 Practice Activity — Methods Left for the Trainee to Complete

The remaining DAO methods are deliberately left incomplete in the source training material, as an exercise. Each is reproduced below exactly as supplied, along with guidance on which technique from earlier chapters solves it.

25.4.1 addCustomer() — apply persist() (Chapter 22, §22.7)

```
public Integer addCustomer(CustomerBean customer) throws Exception {
    Integer customerID = 0;
    EntityManager entityManager = null;
    try {

        // Your Code goes here

    } catch (Exception exception) {

        throw exception;
    } finally {
        if (entityManager != null) {
            entityManager.close();
        }
    }

    return customerID;
}
```

Guidance: create an EntityManager from JPAUtility.getEntityManagerFactory(), build a new CustomerEntity from the incoming CustomerBean's fields, begin a transaction, call entityManager.persist(customerEntity), commit, and return customerEntity.getId() — following the exact same shape as the persist() demo in Chapter 22, §22.7.

25.4.2 deleteCustomerById() — apply find() then remove() (Chapter 22, §22.10)

```
public CustomerBean deleteCustomerById(Integer customerId) throws Exception {
    CustomerBean customerRet = null;
    EntityManager entityManager = null;
    try {

        // Your Code goes here

    } catch (Exception exception) {

        throw exception;
    } finally {
        if (entityManager != null) {
```

```

        entityManager.close();
    }
}

return customerRet;
}

```

Guidance: exactly the pattern from Chapter 22, §22.10 — `remove()` requires a *managed* entity, so this method must first `find()` the `CustomerEntity` by ID, copy its field values into a `CustomerBean` to return *before* removing it (since after removal there's nothing sensible left to read from), begin a transaction, call `entityManager.remove(customerEntity)`, and commit.

25.4.3 `getCustomersWithinDateRange()` — apply JPQL with parameter passing (Chapter 24, §24.5)

```

@Override
public List<CustomerBean> getCustomersWithinDateRange(Date lowerBound, Date upperBound) {
    List<CustomerBean> listCustomer = null;
    EntityManager entityManager = null;
    try {

        // Your Code goes here

    } catch (Exception exception) {

        throw exception;
    } finally {
        if (entityManager != null) {
            entityManager.close();
        }
    }

    return listCustomer;
}

```

Guidance: write a JPQL query with a `BETWEEN` (or two comparison) conditions on `purchaseDate`, bind `lowerBound/upperBound` as parameters (positional or named — Chapter 24, §24.5), call `getResultList()`, and map each returned `CustomerEntity` into a `CustomerBean`, exactly as the structured JPQL demo in Chapter 24, §24.3 did for `EmployeeEntity`.

25.4.4 updateCustomerBillByName() — apply a JPQL bulk UPDATE (Chapter 24, §24.7)

```
@Override
public Integer updateCustomerBillByName(String name, Double bill) throws Exception {
    Integer ret = null;
    EntityManager entityManager = null;
    try {

        // Your Code goes here

    } catch (Exception exception) {

        throw exception;
    } finally {
        if (entityManager != null) {
            entityManager.close();
        }
    }

    return ret;
}
```

Guidance: this is a direct application of the bulk-update JPQL technique from Chapter 24, §24.7 — `entityManager.createQuery("update CustomerEntity c set c.bill = :bill where c.customerName = :name")`, bind both parameters, wrap in a transaction, and call `executeUpdate()`, returning the row-count it produces.

25.5 The UI/Driver Layer

File: Activity_1001_JPA ORM_CRUD_To_Trainee/src/com/accenture/ltt/ui/UITester.java

```
package com.accenture.ltt.ui;

import java.util.Calendar;
import java.util.Date;
import java.util.List;

import com.accenture.ltt.businessbean.CustomerBean;
import com.accenture.ltt.service.CustomerService;
import com.accenture.ltt.utility.Factory;
import com.accenture.ltt.utility.JPAUtility;

public class UITester {

    public static void main(String[] args) {
        try {
```



```

        //addCustomer();
        //findCustomerById();
        //deleteEmployeeById();
        //updateCustomerBillById();
        //updateCustomerBillByName("James", 343.4);

        Calendar fromDate= Calendar.getInstance();
        fromDate.set(2016, 01, 01); //month starts from 0, 0-Jan, 1-Feb

        Calendar toDate= Calendar.getInstance();
        toDate.set(2016, 02, 31); //month starts from 0, 0-Jan, 1-Feb

        getCustomersWithinDateRange(fromDate.getTime(), toDate.getTime());

    } catch (Exception e) {
        System.out.println(e.getMessage());
    } finally {
        JPAUtility.closeEntityManagerFactory();
    }
}

static public void addCustomer(){
    CustomerBean bean = new CustomerBean();
    bean.setPurchaseDate(new Date());
    bean.setCustomerName("MSD1");
    bean.setBill(100000.0);
    CustomerService customerService = Factory.createCustomerService();
    try {
        int id = customerService.addCustomer(bean);
        System.out.println("Customer Registered Successfully: " + id);
    } catch (Exception e) {
        System.out.println(e.getMessage());
    }
}

// ... findCustomerById(), updateCustomerBillById(), deleteEmployeeById(),
//     getCustomersWithinDateRange(), updateCustomerBillByName() drivers follow
//     the same shape: obtain the service via Factory, call it, print the result.
}

```

Two points worth calling out about this driver class specifically:

- **Each CRUD operation is commented out except one at a time** — this is a deliberate testing discipline: run and verify `addCustomer()` in isolation, confirm the row exists, *then* comment it out and test `findCustomerById()`, and so on — rather than running every operation in one pass and being unable to tell which step produced which output or failure.

- **JPAUtility.closeEntityManagerFactory()** runs in the finally block of **main()** itself, not inside any individual DAO method — this is the appropriate place to close the (application-scoped) EntityManagerFactory, since it should stay open for the entire lifetime of the application and only close when the application is genuinely shutting down, unlike the per-operation EntityManager instances, which are correctly opened and closed inside each individual DAO method call.
- The Calendar usage for building fromDate/toDate includes the classic java.util.Calendar trap flagged as a comment directly in the source: **months are zero-indexed** (0 = January), so fromDate.set(2016, 01, 01) actually sets the date to **February** 1st, 2016, not January — a well-known Java date-API pitfall (one of the reasons java.time was introduced in Java 8, covered in the Core Java Module's Java 8 Features chapter).

25.6 Viva Voce — Likely Questions & Answers

Q1. What is “dirty checking” in JPA/Hibernate, and where does it appear in this chapter’s case study? Dirty checking is Hibernate’s mechanism of automatically detecting that a field on an already-managed entity has changed (compared to its state when it was loaded), and generating the corresponding UPDATE SQL automatically at flush/commit time — with no explicit merge() or update() call required. It appears in updateCustomerBillById(), where the entity is find()-ed (making it managed) and then its bill field is simply set directly inside an open transaction.

Q2. Why does updateCustomerBillById() not need to call entityManager.merge(...) explicitly? Because merge() is only necessary to reattach a *detached* (or transient) entity to a new persistence context. Here, customerEntity was obtained via find() in the *same* persistence context that’s still open when the field is mutated — it’s already managed, so Hibernate’s dirty checking picks up the change automatically without any explicit save call.

Q3. In deleteCustomerById(), why must the customer’s data be copied into a return-value bean before calling remove(), not after? Because once remove() is called (and especially after the transaction commits), the entity is no longer a valid, readable representation of an existing row — reading its fields afterward for a “this is what was deleted” confirmation would be reading from an entity in the removed state, which is a source of subtle bugs; it’s safer and clearer to snapshot the data first.

Q4. Why is JPAUtility.closeEntityManagerFactory() called in main()’s finally block rather than inside each DAO method? Because EntityManagerFactory is expensive to build and is meant to be shared and reused for the entire application’s lifetime (Chapter 22, §22.4) — closing it after every single DAO call would defeat that purpose and force it to be rebuilt on every operation. It should only be closed once, when the whole application is shutting down.

Q5. What common java.util.Calendar mistake does the UITester’s date-range setup illustrate, and why does it happen? Calendar.set(year, month, day) treats the month parameter as zero-indexed (0 = January, 1 = February, etc.), so

`fromDate.set(2016, 01, 01)` actually produces February 1st, 2016, not January 1st — a frequently-cited legacy API design flaw, and one of the motivations behind the `java.time` package introduced in Java 8.

Q6. Which technique from earlier chapters solves `getCustomersWithinDateRange()`? A JPQL query with a range condition on the `purchaseDate` field, using either positional or named parameter binding for the two date bounds (Chapter 24, §24.5), followed by mapping each result `CustomerEntity` into a `CustomerBean` for return (Chapter 24, §24.3).

Q7. Which technique from earlier chapters solves `updateCustomerBillByName()`, and why is it different from `updateCustomerBillById()`'s approach? It requires a JPQL bulk UPDATE statement (Chapter 24, §24.7) using `executeUpdate()`, because the update needs to apply based on a *name* match that could, in principle, affect multiple rows at once — unlike `updateCustomerBillById()`, which operates on one already-known, already-managed entity and can rely on simple dirty checking instead.

Chapter 26

MongoDB & NoSQL Fundamentals for Java Developers

26.1 SQL vs NoSQL — Why This Chapter Exists Alongside JPA

Every chapter so far in this volume has dealt with **relational** databases — fixed schemas, tables, rows, foreign keys, and SQL/JPQL. MongoDB represents a different family entirely: a **document-oriented NoSQL** database. Understanding when and why a team would reach for MongoDB instead of (or alongside) MySQL is itself a common interview question, so it's worth contrasting the two models directly:

Aspect	Relational (MySQL, via JDBC/JPA)	MongoDB (Document NoSQL)
Structure	Fixed schema — every row in a table has the same columns	Flexible schema — each document in a collection can have different fields
Unit of storage	A row, in a table	A document (essentially a JSON/BSON object), in a collection
Relationships	Foreign keys, joins (Chapter 23)	Typically embedding related data directly inside a document, or manual application-level references
Query language	SQL / JPQL	MongoDB's own query API (filters, Document objects)

Aspect	Relational (MySQL, via JDBC/JPA)	MongoDB (Document NoSQL)
Schema changes	Require ALTER TABLE (or <code>hibernate.hbm2ddl.auto</code>)	A new document can simply include a new field; no schema migration needed
Best suited for	Structured, relational data with strong consistency needs (financial records, inventory with strict referential integrity)	Rapidly evolving data shapes, large volumes of semi-structured data, high write throughput

Neither model is universally “better” — full-stack systems increasingly use **both**, choosing the right store for each part of the system (a common pattern called polyglot persistence): relational for the core transactional data, document stores for content that varies in shape or needs flexible, fast iteration.

26.2 ACID vs BASE, and the CAP Theorem

Chapter 22 introduced **ACID** (Atomicity, Consistency, Isolation, Durability) as the guarantee relational databases (and JPA transactions) provide. Many NoSQL databases, including MongoDB in certain configurations, deliberately relax some of these guarantees in exchange for better performance and availability at large scale — a trade-off usually summarised as **BASE**:

Letter	Property	Meaning
BA	Basically Available	The system guarantees availability of data, even in the presence of failures, rather than strict all-or-nothing correctness at every instant
S	Soft state	The state of the system may change over time, even without new input, as data propagates/settles across nodes
E	Eventual consistency	Given enough time without new updates, all copies of the data will eventually converge to the same value — but might briefly disagree right after a write

This BASE vs ACID trade-off is closely related to a well-known result in distributed systems called the **CAP theorem**, which states that a distributed data store can only guarantee **two** of the following three properties at the same time, not all three:

- **Consistency (C)** — every read receives the most recent write (or an error).
- **Availability (A)** — every request receives a (non-error) response, even if it might not be the most recent write.
- **Partition tolerance (P)** — the system keeps operating even if network communication between some of its nodes is lost.

Since network partitions are a fact of life in any real distributed system, partition tolerance is rarely optional in practice — which means the real-world choice most distributed databases face is between prioritising strict **consistency** or prioritising **availability** when a partition actually occurs. This is precisely why the choice between a relational database and a NoSQL store like MongoDB is not just about schema flexibility (§26.1) — it can also be a deliberate choice about which of these guarantees matters more for a specific part of a system (e.g., strict consistency for a bank balance vs. high availability for a social media “like” counter).

26.3 MongoDB Core Concepts

- **Database** — the top-level container, analogous to a relational schema/database.
- **Collection** — analogous to a table, but with no enforced schema; a collection simply groups related documents.
- **Document** — analogous to a row, but represented as a BSON (Binary JSON) object — essentially a set of key-value pairs, where values can themselves be nested objects or arrays, unlike a flat relational row.
- **_id field** — every document has a unique identifier field called `_id`, roughly analogous to a primary key, generated automatically by MongoDB if not supplied explicitly.

26.4 Connecting to MongoDB from Java

File: Demo1001_MongoDB_ConnectionToMongoDB/src/com/accenture/ltt/utility/DBUtility.java

```
package com.accenture.ltt.utility;

import com.mongodb.client.MongoClient;
import com.mongodb.client.MongoClients;
import com.mongodb.client.MongoDatabase;

public class DBUtility {

    private static MongoClient mongoClient = null;
    private static MongoDatabase database = null;

    private static final String CONNECTION_STRING = "mongodb://localhost:27017";
    private static final String DATABASE_NAME = "myDatabase"; // your MongoDB database
```

```
public static MongoDBDatabase getDBConnection() {
    if (mongoClient == null) {
        mongoClient = MongoClient.create(CONNECTION_STRING);
        System.out.println("MongoDB connection established.");
    }

    if (database == null) {
        database = mongoClient.getDatabase(DATABASE_NAME);
        System.out.println("Connected to database: " + DATABASE_NAME);
    }

    return database;
}

public static void getDBDestroyConnection() {
    if (mongoClient != null) {
        mongoClient.close();
        mongoClient = null;
        database = null;
        System.out.println("MongoDB connection closed.");
    }
}
}
```

File: Demo1001_MongoDB_ConnectionToMongoDB/src/com/accenture/ltt/ui/Tester.java

```
package com.accenture.ltt.ui;

import com.accenture.ltt.utility.DBUtility;
import com.mongodb.client.MongoDatabase;

public class Tester {

    public static MongoDBDatabase database = null;

    public static void main(String[] args) {

        try {
            // Establish MongoDB connection
            database = DBUtility.getDBConnection();

            // Test: Print database name
            System.out.println("Connected to MongoDB Database: " + database.getName());
        } catch (Exception e) {
            e.printStackTrace();
        } finally {
```

```

        // Close MongoDB connection
        if (database != null) {
            DBUtility.getDBDestroyConnection();
        }
    }
}

```

Compare this connection pattern directly against the JDBC DBUtility from Chapter 21 — the shape is deliberately identical (a lazily-initialised, cached connection object exposed through static methods), even though the underlying types are entirely different (MongoClient/MongoDatabase from the MongoDB Java driver instead of `java.sql.Connection`). `MongoClients.create(CONNECTION_STRING)` establishes the client using MongoDB’s own connection-string URI format (`mongodb://host:port`) — analogous in spirit to a JDBC URL, but with its own distinct scheme and syntax — and `mongoClient.getDatabase(DATABASE_NAME)` selects (and implicitly creates, on first write, if it doesn’t already exist) the target database.

26.5 CRUD with the MongoDB Java Driver

File: `Demo1002_MongoDB_CrudApplication/src/com/accenture/ltt/dao/EmployeeDAOImpl.java`

```

package com.accenture.ltt.dao;

import java.util.ArrayList;
import java.util.List;

import com.accenture.ltt.businessbean.EmployeeBean;
import com.accenture.ltt.utility.DBUtility;
import com.mongodb.client.MongoCollection;
import com.mongodb.client.MongoDatabase;
import com.mongodb.client.model.Filters;
import com.mongodb.client.model.Updates;

import org.bson.Document;

public class EmployeeDAOImpl implements EmployeeDAO {

    private MongoCollection<Document> collection;

    public EmployeeDAOImpl() {
        MongoDatabase database = DBUtility.getDBConnection();
        collection = database.getCollection("employees");
    }

    @Override

```



```
public int insertEmployee(EmployeeBean bean) {
    collection.insertOne(convertToDocument(bean));
    return 1; // Success
}

@Override
public List<EmployeeBean> readEmployee() {
    List<EmployeeBean> list = new ArrayList<>();
    for (Document doc : collection.find()) {
        list.add(convertToBean(doc));
    }
    return list;
}

@Override
public void updateEmployee(EmployeeBean bean) {
    // Only update fields that are not null
    List<org.bson.conversions.Bson> updates = new ArrayList<>();

    if (bean.getEmployeeName() != null) {
        updates.add(Updates.set("employeeName", bean.getEmployeeName()));
    }
    if (bean.getRole() != null) {
        updates.add(Updates.set("role", bean.getRole()));
    }
    if (bean.getSalary() != null) {
        updates.add(Updates.set("salary", bean.getSalary()));
    }

    if (!updates.isEmpty()) {
        collection.updateOne(Filters.eq("employeeID", bean.getEmployeeID()),
            Updates.combine(updates));
    }
}

@Override
public void deleteEmployee(EmployeeBean bean) {
    collection.deleteOne(Filters.eq("employeeID", bean.getEmployeeID()));
}

// Convert EmployeeBean -> MongoDB Document
private Document convertToDocument(EmployeeBean emp) {
    Document doc = new Document("employeeID", emp.getEmployeeID())
        .append("employeeName", emp.getEmployeeName())
        .append("role", emp.getRole())
        .append("salary", emp.getSalary())
        .append("insertTime", emp.getInsertTime());
}
```

```

        return doc;
    }

    // Convert MongoDB Document -> EmployeeBean
    private EmployeeBean convertToBean(Document doc) {
        EmployeeBean emp = new EmployeeBean();
        emp.setEmployeeID(doc.getInteger("employeeID"));
        emp.setEmployeeName(doc.getString("employeeName"));
        emp.setRole(doc.getString("role"));
        emp.setSalary(doc.getDouble("salary"));
        emp.setInsertTime(doc.getDate("insertTime"));
        return emp;
    }
}

```

Walking through each operation and the API concepts it introduces:

- **Create — `collection.insertOne(document)`.** A Document is built directly as a chain of key-value pairs (`new Document("employeeID", ...).append("employeeName", ...)`), rather than mapped from annotations on a class the way `@Entity` fields are — there is no equivalent of JPA's declarative entity mapping in this low-level driver API; the mapping between `EmployeeBean` and `Document` is done entirely by hand in `convertToDocument()/convertToBean()`.
- **Read — `collection.find()`.** Called with no arguments, this returns *every* document in the collection, iterated directly as a `for (Document doc : collection.find())` loop — the MongoDB driver's `find()` returns an iterable cursor. A filtered read would instead pass a `Filters.eq(...)/Filters.gt(...)`-style query object, exactly as `Filters.eq("employeeID", ...)` is used for update/delete below.
- **Update — `collection.updateOne(filter, update)`.** Two separate objects are involved: a **filter** (`Filters.eq("employeeID", bean.getEmployeeID())`), identifying *which* document(s) to update, and an **update specification** (`Updates.combine(updates)`, itself built from a list of `Updates.set(field, value)` calls), describing *what* to change. Notice the deliberate null-check before adding each `Updates.set(...)` — this allows a partial update (only the fields actually present on the incoming bean get modified), which has no direct equivalent in a standard SQL `UPDATE` statement without dynamically building the SQL by hand; MongoDB's `Updates.set()` model makes partial, field-level updates natural and explicit.
- **Delete — `collection.deleteOne(filter)`.** Takes only a filter — no separate "value" argument is needed, since deletion doesn't set new field values.

26.6 Mapping Between Java Objects and MongoDB Documents — A Manual Process

A subtle but important point for viva: **this driver-level MongoDB code performs no automatic object-to-document mapping** — `convertToDocument()` and `convertToBean()` are hand-written, field-by-field conversions, exactly analogous to the manual `EmployeeBean <-> ResultSet` mapping seen in the JDBC chapter (Chapter 21), *not* to JPA's declarative, annotation-driven entity mapping (Chapter 22). This is a deliberate architectural choice in this training's demos, to show the MongoDB Java driver's native API directly; in a larger, real-world Spring-based application, a higher-level abstraction (Spring Data MongoDB, which *does* provide annotation-based document mapping similar in spirit to JPA's `@Entity`) would typically replace this hand-written conversion code — worth mentioning if asked to compare “JPA-style” development against this lower-level driver usage in an interview.

26.7 Viva Voce — Likely Questions & Answers

Q1. What is the fundamental structural difference between a relational database and MongoDB? A relational database enforces a fixed schema — every row in a table has the same set of columns. MongoDB is schema-flexible — each document within a collection can have a different set of fields, and documents can contain nested objects and arrays directly, rather than requiring separate joined tables.

Q2. What are the MongoDB equivalents of a relational database, table, and row? Database maps to database, collection maps to table, and document maps to row — though a document is a full BSON (Binary JSON) object rather than a flat set of columns, and can contain nested structures.

Q3. What does the `_id` field represent in a MongoDB document? A unique identifier for that document, roughly analogous to a primary key in a relational table — MongoDB generates one automatically if the application doesn't supply its own value for it.

Q4. In the MongoDB Java driver, what two things does `collection.updateOne(filter, update)` need, and why are they separate? A filter (identifying which document(s) to target, e.g. `Filters.eq("employeeID", id)`) and an update specification (describing what changes to apply, e.g. `Updates.set("salary", newSalary)`). They're kept separate because “which documents” and “what changes” are conceptually distinct operations, and this separation is what allows the same filter/update building blocks to be reused and combined flexibly across different queries.

Q5. How does the demo's `updateEmployee()` method perform a partial update, and why is that harder to express in plain SQL? It builds a list of `Updates.set(field, value)` calls only for the fields on the incoming bean that are non-null, then combines them with `Updates.combine(...)` — so only the fields actually provided get modified. In SQL, a single `UPDATE` statement's `SET` clause is fixed at the time the SQL is written, so a similarly flexible partial update would require dynamically constructing the SQL string based on which fields are present, rather than

simply omitting unwanted `Updates.set()` calls.

Q6. Does the MongoDB Java driver used in these demos provide automatic object-to-document mapping, the way JPA provides automatic object-to-table mapping? No — in these low-level driver demos, the mapping between `EmployeeBean` and `Document` is done entirely by hand, in `convertToDocument()/convertToBean()`. Higher-level libraries such as Spring Data MongoDB do provide declarative, annotation-based document mapping analogous to JPA's `@Entity`, but that is a separate abstraction layer on top of the raw driver API shown here.

Q7. Why might a full-stack application use both a relational database and MongoDB together, rather than choosing only one? Because different parts of an application often have different data-shape and consistency needs — structured, strongly related, transactional data (e.g., financial records, inventory with strict referential integrity) is naturally suited to a relational database with JPA/Hibernate, while rapidly evolving or loosely structured data (e.g., activity logs, product catalogs with varying attributes, content with flexible schemas) is often a better fit for a document store like MongoDB. Using the data store best suited to each specific need, rather than forcing one model to fit everything, is sometimes called polyglot persistence.

Q8. What does BASE stand for, and how does it differ philosophically from ACID? Basically Available, Soft state, Eventual consistency. Where ACID guarantees strict, immediate correctness after every transaction, BASE deliberately favours availability and performance, accepting that different copies of the data might briefly disagree and only converge to the same value “eventually,” rather than instantly.

Q9. According to the CAP theorem, why can a distributed database not guarantee Consistency, Availability, and Partition tolerance all at once? Because when a network partition actually occurs (nodes can't communicate), the system must choose between responding immediately with potentially stale data (favouring Availability) or refusing to respond until it can guarantee the most up-to-date value (favouring Consistency) — it cannot do both simultaneously during that partition. Since partitions are unavoidable in real distributed systems, Partition tolerance is rarely optional, making the practical trade-off one between Consistency and Availability.

Java Full Stack Training

Introduction to Maven through Spring Batch

Extended Theory, Concepts & Demo Programs

(Viva / Interview Preparation Edition)

Accenture QuantumCrew Batch — Day-wise Training Reference
(Continuation Volume II)

Covers Weeks 4–5 of the Day-wise Training Plan
Chapters 27–35 — Instructor Demos, Practice Activities & Assignments

Chapter 27

Introduction to Maven

27.1 The Problem Maven Solves

Every one of the demo projects used from Chapter 19 onward — SOLID demos, JDBC demos, JPA demos — has quietly been a **Maven project**. Before looking at what Maven *is*, it helps to see the problem it exists to solve, because every one of its features is a direct answer to a pain point that Java projects had before it existed.

Imagine building a Spring application by hand, without any build tool:

- You need the Spring JAR files, but Spring itself needs other JARs to work (commons-logging, spring-core needs spring-jcl, and so on). This is called **transitive dependency** — a dependency of a dependency — and by hand you would have to hunt down every one of these, and every version has to be compatible with every other version.
- You need a consistent, repeatable way to compile hundreds of `.java` files in the right order, run tests, and package the result into a `.jar` or `.war` — doing this by hand with `javac` commands does not scale past a handful of files.
- Every developer on the team, and the CI server, must all reproduce the *exact same* build with the *exact same* versions of every library — otherwise you get “works on my machine”.
- A new project should not need its folder structure invented from scratch each time — source under one folder, tests under another, resources under a third.

Maven is a build automation and project management tool for Java. It solves all four problems at once: it downloads dependencies (and their transitive dependencies) automatically from a shared repository, it defines a standard, predictable lifecycle for compiling/testing/packaging a project, and it enforces a standard directory layout so that any Maven project looks structurally the same as any other.

Why This Matters Before Spring

Every Spring demo project you will see for the rest of this volume opens with a `pom.xml` file. Understanding what that file means — and what happens when you run `mvn install` against it — is a prerequisite for understanding how Spring itself gets onto the classpath in the first place.

27.2 What Is Build Automation?

Build automation is the practice of scripting every repeatable step needed to turn source code into a running, deployable artifact: compiling, running tests, resolving dependencies, packaging,

and sometimes deploying. The goal is that the build is a single command (`mvn package`, for example) rather than a checklist a human has to remember and execute correctly every time. This matters for three reasons:

1. **Repeatability** — the same command produces the same artifact regardless of who runs it or on which machine.
2. **Speed** — a human typing individual `javac` commands for a multi-module project is slow and error-prone; a tool that already knows the dependency graph is fast and correct.
3. **Integration with CI/CD** — a build that is one command is trivially pluggable into Jenkins, GitLab CI, or any other pipeline, which is exactly what later chapters in this training (CI/CD with Jenkins) depend on.

Maven is one build-automation tool for the JVM ecosystem; Gradle and Ant are the other two you are likely to encounter, but Maven's declarative, convention-over-configuration style is what this course standardises on.

27.3 Maven as Build Automation — the Core Idea

Where a tool like Ant requires you to *script* every step (“first do this, then do that”), Maven is **declarative**: you describe *what* your project is — its identity, its dependencies, its packaging type — in an XML file called the **Project Object Model**, or `pom.xml`, and Maven already knows *how* to compile, test, and package it, because it follows a standard, predefined lifecycle. This is Maven's version of **convention over configuration**: if you follow Maven's expected folder layout and naming, you get a full build for free; you only add configuration where your project genuinely deviates from the convention.

27.3.1 The Standard Directory Layout

Every Maven project you have opened in this course — from the very first SOLID demo to the Spring Batch demos — follows the same skeleton:

```
1 my-project/  
2   pom.xml  
3   src/  
4     main/  
5       java/      <- your application source (.java)  
6       resources/ <- config files, XML, properties bundled onto the classpath  
7     test/  
8       java/      <- JUnit / test source  
9       resources/ <- test-only config files  
10    target/      <- generated by the build: .class files, the final jar/war
```

Because every Maven project looks like this, any developer — or any IDE, or any CI server — can open an unfamiliar Maven project and immediately know where the source is, where the tests are, and where the build output will land. This is exactly what you have seen in every demo project of this course: `src/main/java/com/accenture/ltt/...` for the actual classes, and (from the Spring Test chapter onward) `src/test/java/com/accenture/ltt/test/...` for the JUnit tests.

27.4 Maven Plugins, Goals, and MOJOs

Maven itself does almost nothing directly — compiling, testing, and packaging are all delegated to **plugins**. A plugin is a collection of related **goals**; a goal is a single, specific task (for example, `compiler:compile` compiles your `.java` files, `surefire:test` runs your JUnit tests,

`jar:jar` packages the compiled classes into a `.jar`). Internally, each goal is implemented by a Java class called a **MOJO** — short for “Maven Plain Old Java Object” (a deliberate pun on “Mojo”, i.e. a Maven goal). You will rarely write a MOJO yourself as a beginner, but it helps to know that when you type `mvn compile`, Maven is really invoking the `compile` goal of the `maven-compiler-plugin`, which is backed by a MOJO class that does the actual work of shelling out to `javac`.

Term	Meaning
Plugin	A packaged collection of related goals (e.g. <code>maven-compiler-plugin</code> , <code>maven-surefire-plugin</code>)
Goal	One specific unit of work exposed by a plugin (e.g. <code>compile</code> , <code>test</code> , <code>jar</code>)
MOJO	The Java class that implements a single goal’s logic
Phase	A named point in the build lifecycle to which one or more goals are bound (e.g. the <code>compile</code> phase runs the compiler plugin’s <code>compile</code> goal)

27.5 Maven Build Phases and the Default Lifecycle

Maven organises work into a **lifecycle**, which is an ordered sequence of **phases**. Running any phase automatically runs every phase before it, in order. This is the single most important thing to internalise about Maven: you almost never invoke a goal directly; you invoke a *phase*, and Maven walks the ladder up to that phase for you.

The default lifecycle’s phases, in order, are:

Phase	What Happens
<code>validate</code>	Checks the project structure and <code>pom.xml</code> are correct before doing any real work
<code>compile</code>	Compiles <code>src/main/java</code> into <code>.class</code> files under <code>target/classes</code>
<code>test</code>	Compiles <code>src/test/java</code> and runs the unit tests (via the Surefire plugin)
<code>package</code>	Bundles the compiled code into its distributable format — a <code>.jar</code> or <code>.war</code> , as declared by <code><packaging></code> in the <code>pom.xml</code>
<code>verify</code>	Runs any additional checks on the package to confirm quality criteria are met
<code>install</code>	Copies the package into your local repository (<code>~/.m2/repository</code>) so other local projects can depend on it
<code>deploy</code>	Copies the final package to a remote repository for sharing with other developers/projects

So when you run `mvn install` (the command you have been running throughout this course to build every demo project), Maven silently walks through `validate` → `compile` → `test` → `package` → `verify` → `install` in that order, and only then stops. If any phase fails — most commonly, a failing test during the `test` phase — the whole build stops there and later phases never run. This is precisely why a broken unit test can block a Spring Boot module from ever being packaged.

27.5.1 The Surefire Plugin — Running Tests as Part of the Build

The **Surefire plugin** is the plugin bound to the `test` phase; its job is to discover every test class under `src/test/java` (by naming convention, classes named `*Test`, `Test*`, or `*TestCase`) and run them, reporting pass/fail for each. This is what makes “run `mvn install` and let it fail if a test fails” possible without a human ever manually invoking JUnit. This is also the plugin that quietly runs every one of the `@Test`-annotated methods you will write in Chapter 30 (Spring Testing) whenever you build the project.

27.6 The pom.xml — the Project Object Model

Every Maven project’s identity, dependencies, and build configuration live in one file at its root: `pom.xml`. Below is the actual `pom.xml` from the very first Spring Core demo of this course, `10001_Springs_Loosely_Coupled_Example`:

```

1  <?xml version="1.0" encoding="UTF-8"?>
2  <project xmlns="http://maven.apache.org/POM/4.0.0"
3      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4      xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
5          https://maven.apache.org/xsd/maven-4.0.0.xsd">
6      <modelVersion>4.0.0</modelVersion>
7      <parent>
8          <groupId>org.springframework.boot</groupId>
9          <artifactId>spring-boot-starter-parent</artifactId>
10         <version>2.6.6</version>
11         <relativePath/> <!-- lookup parent from repository -->
12     </parent>
13     <groupId>com.accenture.ltt</groupId>
14     <artifactId>10001_Springs_Loosely_Coupled_Example</artifactId>
15     <version>0.0.1-SNAPSHOT</version>
16     <properties>
17         <java.version>1.8</java.version>
18     </properties>
19     <dependencies>
20         <dependency>
21             <groupId>org.springframework.boot</groupId>
22             <artifactId>spring-boot-starter</artifactId>
23         </dependency>
24     </dependencies>
25
26     <build>
27         <plugins>
28             <plugin>
29                 <groupId>org.springframework.boot</groupId>
30                 <artifactId>spring-boot-maven-plugin</artifactId>
31             </plugin>
32         </plugins>
33     </build>
34 </project>

```

Reading this file line by line, in terms of what each block means:

- `groupId`, `artifactId`, `version` — together these three form the project’s **coordinates**, its unique fingerprint in any Maven repository. `groupId` is normally the reversed organisation domain (`com.accenture.ltt`); `artifactId` is the project’s own name; `version` tracks its release, with `-SNAPSHOT` meaning “still under active development, not a final release”.
- `<parent>` — a `pom.xml` can inherit configuration from a **parent POM**. Here, the parent is `spring-boot-starter-parent`, which is what silently pins compatible versions for every Spring Boot-related dependency you add below, so you never have to specify a version number for `spring-boot-starter` yourself — the parent already decided it.
- `<dependencies>` — the list of libraries this project needs. Each `<dependency>` names a `groupId/artifactId` (and, without a parent pinning it, a `version`) that Maven will fetch — along with everything *that* library needs (its transitive dependencies) — from the Maven repository.
- `<build><plugins>` — extra plugins beyond Maven’s defaults. Here, `spring-boot-maven-plugin` is what allows the project to be packaged as an executable, self-contained “fat jar” and run with `java -jar`.

27.6.1 Where Do the Dependencies Actually Come From? The Repository Model

When Maven sees a `<dependency>` it does not already have, it looks in, in order:

1. The **local repository** — `~/.m2/repository` on your own machine, where every artifact you have ever built or downloaded is cached.
2. A **remote repository** — by default, **Maven Central**, the public repository that hosts virtually every open-source Java library (Spring, Hibernate, JUnit, and so on). Once downloaded, the artifact is cached in the local repository so it is never re-downloaded for that exact version again.

This is why the very first time you build a fresh project it can be slow (everything is being downloaded), while every build after that is fast (everything is already cached locally).

27.7 Maven Goals vs Phases — a Practical Distinction

A beginner's common confusion is treating “goal” and “phase” as the same thing. They are related but distinct:

- A **phase** is a named milestone in the lifecycle (`compile`, `test`, `package`, ...).
- A **goal** is the actual unit of work a plugin performs, and every phase has one or more goals *bound* to it by convention (the `compile` phase is bound to the compiler plugin's `compile` goal; the `test` phase is bound to Surefire's `test` goal).

You can invoke a bare goal directly (`mvn compiler:compile`) without walking the whole lifecycle, but in day-to-day work you almost always invoke a phase (`mvn compile`, `mvn test`, `mvn package`, `mvn install`) and let Maven's lifecycle machinery decide which goals actually need to run.

Viva Voce — Likely Questions & Answers

Q: What problem does Maven solve that plain `javac` does not?

A: Maven automatically resolves and downloads a project's dependencies (and their transitive dependencies) from a shared repository, enforces a standard project layout, and defines a repeatable lifecycle for compiling, testing, and packaging — none of which `javac` does on its own.

Q: What is the `pom.xml`, and what are Maven coordinates?

A: The `pom.xml` (Project Object Model) is the XML file describing a project's identity, dependencies, and build configuration. Its coordinates are the triple `groupId`, `artifactId`, and `version`, which together uniquely identify the project (or any dependency) inside a Maven repository.

Q: What is a transitive dependency?

A: A dependency of a dependency. If your project depends on library A, and A itself depends on library B, then B is pulled in automatically as a transitive dependency — you never have to declare it yourself.

Q: What is the difference between a Maven phase and a Maven goal?

A: A phase is a named step in the build lifecycle (e.g. `compile`, `test`, `package`); a goal is the actual unit of work performed by a plugin (e.g. the compiler plugin's `compile` goal). Phases have goals bound to them by convention.

Q: What is the default Maven lifecycle, in order?

A: `validate` → `compile` → `test` → `package` → `verify` → `install` → `deploy`. Running any phase runs every phase before it automatically.

Q: What is the Surefire plugin responsible for?

A: It is the plugin bound to the `test` phase; it discovers and runs unit test classes under `src/test/java` by naming convention and reports pass/fail, failing the build if any test fails.

Q: What is a MOJO?

A: The Java class inside a Maven plugin that implements the actual logic of a single goal — short for “Maven Plain Old Java Object”.

Q: What is the difference between the local repository and Maven Central?

A: The local repository (`~/.m2/repository`) is a per-machine cache of every artifact you have built or downloaded; Maven Central is the public, shared remote repository Maven falls back to the first time a dependency isn’t already cached locally.

Q: What does a `-SNAPSHOT` version suffix mean?

A: It marks the artifact as an in-progress, non-final build — it can change at any time — as opposed to a released, immutable version number.

Q: Why does the `pom.xml` in this course’s demos declare a `<parent>` of `spring-boot-starter-parent` rather than specifying every dependency’s version explicitly?

A: The parent POM centrally pins compatible versions for the whole family of Spring Boot starter dependencies, so child projects only need to declare *which* starters they want, not which exact version of each — avoiding version-mismatch problems across dozens of transitive Spring/Hibernate/Jackson jars.

Chapter 28

Spring Core — Inversion of Control & Dependency Injection

28.1 The Problem: Tight Coupling

The very first demo of the Spring Core module in this course, `10001.SpringsLooselyCoupledExample`, exists to make one point vividly clear before a single line of Spring code is shown: what does it cost a program *not* to use a framework like Spring?

Consider an `Employee` class that needs an `Address`. The naive way to give it one is for `Employee` to create its own `Address` object internally, typically inside its constructor:

```
1 public class Employee {
2     private Address address;
3     private int employeeId;
4
5     public Employee() {
6         // Employee creates its own dependency -- tight coupling
7         this.address = new Address("HSR Layout", "Sector1", "Bangalore", "Karnataka");
8     }
9     // ...
10 }
```

This works, but `Employee` is now **tightly coupled** to one particular `Address` implementation. If tomorrow you need an `Employee` with a different address, or a mock `Address` for a unit test, you cannot supply one — `Employee` always builds its own. Every class that creates its own dependencies internally has this same problem: it cannot be reused in a different context, and it cannot be tested in isolation, because you can never substitute what it depends on.

28.2 Loose Coupling and Dependency Injection

The fix is to **invert** who is responsible for creating the dependency: instead of `Employee` creating its own `Address`, something *external* creates the `Address` and hands it (“injects” it) into the `Employee`. This is **Dependency Injection (DI)** — a class simply declares what it needs (usually via a constructor parameter or a setter), and an outside party supplies it.

```
1 public class Employee {
2     private Address address;
3     private int employeeId;
4     private double salary;
5     private String employeeName;
6
7     // The Address is *given* to Employee -- not created by it
8     public Employee(Address address, int employeeId, double salary, String employeeName) {
9     }
```

```

9         this.address = address;
10        this.employeeId = employeeId;
11        this.salary = salary;
12        this.employeeName = employeeName;
13    }
14 }

```

This is **loose coupling**: `Employee` depends only on the *shape* of `Address` (its public interface), not on how or where that particular `Address` object was built. You can now hand `Employee` any `Address` you like — a real one, a test double, a differently-configured one — without changing `Employee`'s code at all.

28.3 Inversion of Control (IoC) — the Bigger Idea Behind DI

Dependency Injection is one specific technique that implements a larger design principle called **Inversion of Control**. In ordinary procedural code, *your* code is in control: your `main` method decides when to create an object, in what order, and hands objects to each other explicitly. Under IoC, that responsibility is handed over (“inverted”) to a **container** — a piece of framework code that reads a configuration (XML or annotations) describing what objects (“beans”) exist and how they relate, and then creates and wires all of them for you, in the correct order, before your own code ever runs.

The One-Sentence Version

IoC means: *you no longer call ‘new’ to build the objects your application needs* — the Spring container builds them and hands them to you, fully wired, on request.

In Spring, this container is the **ApplicationContext**. Look at the driver class from the very first DI demo of this course:

```

1 package com.accenture.ltt;
2
3 import org.springframework.context.ApplicationContext;
4 import org.springframework.context.support.ClassPathXmlApplicationContext;
5
6 import com.accenture.ltt.bean.Employee;
7
8 public class Application {
9
10     public static void main(String[] args) {
11         ApplicationContext applicationContext =
12             new ClassPathXmlApplicationContext("com/accenture/ltt/resources/
13                 my_springbean.xml");
14         Employee employee = (Employee) applicationContext.getBean("empObject");
15         System.out.println("\n\nOutput is :");
16         employee.display();
17     }
18 }

```

Notice that `Application` never writes `new Employee(...)` or `new Address(...)`. It asks the **ApplicationContext** for a bean named `"empObject"`, and the container hands back a fully-constructed, fully-wired `Employee` — `Address` and all. The container read the wiring instructions from `my_springbean.xml`:

```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <beans xmlns="http://www.springframework.org/schema/beans"
3     xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4     xsi:schemaLocation="http://www.springframework.org/schema/beans
5         http://www.springframework.org/schema/beans/spring-beans.xsd">
6
7     <bean id="addressObject" class="com.accenture.ltt.bean.Address">
8         <property name="addressLine1" value="HSR Layout, Sector1"></property>
9         <property name="addressLine2" value="Bangalore, Karnatka"></property>

```

```
10     </bean>
11
12     <bean id="empObject" class="com.accenture.ltt.bean.Employee">
13         <constructor-arg ref="addressObject" />
14         <constructor-arg value="1001" />
15         <constructor-arg value="200000" />
16         <constructor-arg value="MSD" />
17     </bean>
18
19 </beans>
```

Two beans are declared: `addressObject` (an `Address`, built with setter injection via `<property>` tags) and `empObject` (an `Employee`, built with constructor injection via `<constructor-arg>` tags, one of which, `ref="addressObject"`, wires in the *other* bean by reference). This is the **IoC container** in action: it read this file, instantiated `addressObject` first (because `empObject` depends on it), then instantiated `empObject` by passing `addressObject` into its constructor — exactly as if you had written `new Employee(new Address(...), 1001, 200000, "MSD")` yourself, except Spring did the wiring, driven entirely by declarative configuration rather than by Java code.

28.4 Setter Injection vs Constructor Injection

Spring supports two principal styles of Dependency Injection, and this course's demos deliberately show both.

28.4.1 Setter Injection

With **setter injection**, the container creates the bean using its no-argument constructor, and then calls setter methods to populate its dependencies, driven by `<property>` tags:

```
1 <bean id="addressObject" class="com.accenture.ltt.bean.Address">
2     <property name="addressLine1" value="HSR Layout, Sector1"></property>
3     <property name="addressLine2" value="Bangalore, Karnatka"></property>
4 </bean>
```

For this to work, `Address` must expose a no-arg constructor and a `setAddressLine1(...)/setAddressLine2(...)` pair of setters — Spring matches `property name="addressLine1"` to `setAddressLine1(...)` by JavaBean naming convention.

28.4.2 Constructor Injection

With **constructor injection**, the container builds the object by calling a constructor with arguments, driven by `<constructor-arg>` tags, as already shown for `empObject` above. Constructor arguments can be matched three different ways when there is ambiguity between overloaded constructors:

- **By type** — `<constructor-arg type="int" value="1001"/>`
- **By index** — `<constructor-arg index="0" ref="addressObject"/>`
- **By name** — `<constructor-arg name="employeeId" value="1001"/>` (requires debug symbols or `-parameters` compiler flag so Spring can read parameter names)

Aspect	Setter Injection	Constructor Injection
Object state	Can be left partially constructed (some setters never called)	Guarantees the object is fully initialised the moment it exists
Mutability	Dependencies can be changed after construction	Dependencies are typically final and immutable
Circular dependencies	Can resolve some circular bean references	Cannot resolve circular constructor dependencies — Spring throws BeanCurrentlyInCreationException
When preferred	Optional dependencies	Mandatory dependencies (the modern, generally-preferred style)

28.5 Mixed Injection, Namespace Shortcuts, and Nested Values

The demos in this course also cover a few practical variations worth knowing about:

- **Mixed injection** — a single bean can use constructor injection for mandatory dependencies and setter injection for optional ones (seen in `10002.SpringsDI.Intro.D.Mixed`).
- **The `c:` and `p:` namespaces** — Spring offers a terser XML shortcut, writing constructor args as `c:employeeId-ref="1001"` directly as an attribute on the `<bean>` tag instead of nested `<constructor-arg>` elements (seen in `10002.SpringsDI.Intro.A.setters.Using_P` for properties and `10002.SpringsDI.Intro.Constructor_injection.C.Namespace` for constructor args).
- **Nested `<value>/<bean>` tags** — instead of `value="..."` as an attribute, you can nest a `<value>...</value>` element as the property's body, which matters when the value itself contains characters awkward for an XML attribute (seen in `10002.SpringsDI.Intro.E.Mixed.setter.nested`).

28.6 Constructor Argument Matching in Full — Index, Name, and Type

Three further demos (`10002.SpringsDI.Intro.Constructor_injection.index`, `name`, and `type`) drill into exactly how Spring resolves *which* `<constructor-arg>` matches *which* constructor parameter when a class has more than one constructor, or when argument order alone would be ambiguous (e.g. two `String` parameters in a row):

```

1 <!-- Matching by index -->
2 <bean id="empObject" class="com.accenture.ltt.bean.Employee">
3   <constructor-arg index="0" ref="addressObject" />
4   <constructor-arg index="1" value="1001" />
5 </bean>
6
7 <!-- Matching by explicit type -->
8 <bean id="empObject" class="com.accenture.ltt.bean.Employee">
9   <constructor-arg type="com.accenture.ltt.bean.Address" ref="addressObject" />
10  <constructor-arg type="int" value="1001" />
11 </bean>
12
13 <!-- Matching by parameter name (requires compiled-in parameter names) -->
14 <bean id="empObject" class="com.accenture.ltt.bean.Employee">
15   <constructor-arg name="address" ref="addressObject" />
16   <constructor-arg name="employeeId" value="1001" />
17 </bean>

```


index is the most explicit and unambiguous (“this is argument position zero, this is argument position one”); **type** disambiguates by declared Java type, useful when two overloaded constructors exist with different type signatures; **name** is the most readable but requires the class to have been compiled with debug/parameter-name information available (the `-parameters` compiler flag, or `-g` debug symbols), since plain `.class` files do not retain constructor parameter names by default.

28.7 Inner Beans and Collections as Dependencies

Two further DI patterns from the demos:

- **Inner beans** (`10006.SpringsInnerBean`) — when a dependency (like `Address`) is used by exactly one other bean and never needs to be referenced anywhere else, you can declare it *inline*, nested directly inside the `<property>` or `<constructor-arg>` tag that needs it, instead of giving it its own top-level `id` and wiring it with `ref`. This keeps configuration local and signals “this object has exactly one owner”.
- **Collections as dependencies** (`10008.SpringsCollectionsInjected`) — properties of type `List`, `Set`, `Map`, or `Properties` can be populated directly from XML using `<list>`, `<set>`, `<map>`, and `<props>` elements, letting the container inject entire collections of values or bean references in one go.

Viva Voce — Likely Questions & Answers

Q: What is tight coupling, and why is it a problem?

A: Tight coupling is when a class creates its own dependencies internally (e.g. `new Address(...)` inside `Employee`’s constructor). It is a problem because the class can never be given a different implementation of that dependency, which makes it hard to reuse or to unit-test in isolation.

Q: What is Dependency Injection?

A: A design pattern where a class declares what it depends on (through constructor parameters or setters) and an external party supplies (“injects”) those dependencies, rather than the class creating them itself.

Q: What is Inversion of Control, and how does it relate to Dependency Injection?

A: IoC is the broader principle that control over object creation and wiring is handed over from application code to a container/framework. Dependency Injection is the specific technique Spring uses to implement IoC — the container creates the beans and injects their dependencies according to configuration.

Q: What is the Spring `ApplicationContext`?

A: The Spring IoC container — the object that reads bean configuration (XML, annotations, or Java config), creates and wires all the beans it declares, manages their lifecycle, and hands out fully-constructed beans on request via `getBean(...)`.

Q: What is the difference between setter injection and constructor injection?

A: Setter injection builds the bean with a no-arg constructor and then calls setters to populate dependencies, allowing partially-constructed or later-modified objects. Constructor injection passes dependencies directly into the constructor, guaranteeing the object is fully and immutably initialised the moment it exists.

Q: Can constructor injection resolve a circular dependency between two beans?

A: No — if bean A's constructor needs bean B, and bean B's constructor needs bean A, Spring cannot decide which to build first and throws a `BeanCurrentlyInCreationException`. Setter injection can sometimes work around this because the beans can be instantiated first (via their no-arg constructors) and wired to each other afterward.

Q: What is an inner bean, and when would you use one?

A: A bean declared inline inside the `<property>` or `<constructor-arg>` of the one bean that uses it, rather than being given its own top-level id. Use it when a dependency has exactly one owner and never needs to be referenced from anywhere else in the configuration.

Q: How does Spring decide which constructor argument matches which position when there is ambiguity?

A: Through explicit `type`, `index`, or `name` attributes on `<constructor-arg>`, which disambiguate which parameter position or name a given argument value is intended for.

Chapter 29

Spring Bean Configuration — Scopes, Lifecycle, Autowiring & Annotations

29.1 Bean Scopes

By default, every bean the Spring container manages is a **singleton** — exactly one instance is created for the entire container, and every call to `getBean(...)` (or every place that bean is injected) returns the *same* object. This surprises many beginners coming from plain Java, where **new** always creates a fresh object. Spring's default is different, and deliberate: most Spring-managed objects (services, DAOs, repositories) are stateless and safely shared, so creating a fresh one per request would waste memory for no benefit.

The demo `10007_SpringsBeanScopes` exists specifically to show what changes when you override this default:

```
1 <bean id="empObject" class="com.accenture.ltt.bean.Employee" scope="prototype">
2     ...
3 </bean>
```

Scope	Behaviour
singleton (default)	Exactly one shared instance per Spring container, for the container's entire lifetime
prototype	A brand-new instance is created every single time the bean is requested
request	(Web-aware contexts only) One instance per HTTP request
session	(Web-aware contexts only) One instance per HTTP session

How to Prove Scope With Your Own Eyes

Call `applicationContext.getBean("empObject")` twice and compare the two references with `==`. Under **singleton** scope they are the same object (`==` is true); under **prototype** scope they are two different objects (`==` is false), even though both came from identical configuration.

29.2 Bean Lifecycle — Init and Destroy Methods

A Spring bean does not simply spring into existence fully-formed the instant its constructor returns; the container walks it through a defined lifecycle, and you can hook into two specific points of that lifecycle: just *after* all dependencies have been injected (an **init method**), and just *before* the container discards the bean (a **destroy method**). This matters whenever a

bean needs to do setup that depends on its injected properties already being set (opening a connection, warming a cache) or cleanup before it disappears (closing a connection, flushing a buffer) — neither of which belongs inside the constructor, because the constructor runs *before* dependency injection completes.

```
1 <bean id="empObject" class="com.accenture.ltt.bean.Employee"
2     init-method="init" destroy-method="cleanup">
3     ...
4 </bean>
```

```
1 public class Employee {
2     public void init() {
3         System.out.println("Employee bean fully wired -- running init logic");
4     }
5     public void cleanup() {
6         System.out.println("Container is shutting down -- running destroy logic");
7     }
8 }
```

Destroy methods only fire for **singleton**-scoped beans when the container itself is closed (e.g. `((ConfigurableApplicationContext) ctx).close()`) — Spring does not track **prototype** beans after handing them out, so it cannot call a destroy method on them; the calling code owns their cleanup instead.

29.2.1 BeanPostProcessor and BeanFactoryPostProcessor

Beyond per-bean init/destroy hooks, the demos 10029_BeanPostProcessor and 10030/10031_SpringsInitAnd introduce two extension points that act on *every* bean in the container, not just one:

- **BeanPostProcessor** — a special bean whose `postProcessBeforeInitialization(...)` and `postProcessAfterInitialization(...)` methods are called for *every other bean*, immediately before and after its own init method runs. This is how Spring itself implements features like `@Autowired` processing and AOP proxy creation internally.
- **BeanFactoryPostProcessor** — runs even earlier, before any bean is actually instantiated, and can modify bean *definitions* (metadata) themselves — for example, `PropertyPlaceholderConfigurer` (see §4.4 below) is implemented as a **BeanFactoryPostProcessor** that rewrites `${...}` placeholders in bean definitions before any bean using them is created.

The full, precise ordering the container follows for a single bean is:

1. **BeanFactoryPostProcessors** run (act on bean *definitions*, container-wide, before instantiation)
2. Constructor is called → object exists
3. Dependencies are injected (setters/fields)
4. **BeanPostProcessor.postProcessBeforeInitialization(...)**
5. The bean's own `init-method` runs
6. **BeanPostProcessor.postProcessAfterInitialization(...)**
7. Bean is ready and handed out to callers
8. ... (container runs) ...
9. Container shuts down → `destroy-method` runs (singleton scope only)

29.3 Autowiring

Autowiring lets the container resolve a bean's dependencies automatically, without a `<property>` or `<constructor-arg>` explicitly wiring every collaborator by name. This course's demos deliberately walk through every autowiring mode and every failure mode it can produce, because interview questions on this topic are extremely common.

29.3.1 Autowire by Name

`10011.SpringsAutoWiring_ByName` wires a property by matching the **property name** to a bean id of the same name:

```
1 <bean id="empObject" class="com.accenture.ltt.bean.Employee" autowire="byName" />
2 <bean id="address" class="com.accenture.ltt.bean.Address">...</bean>
```

Here, if `Employee` has a property called `address` (i.e. a `setAddress(...)` method), Spring looks for a bean literally named `"address"` and injects it. If no bean with that exact name exists, the property is simply left unset (no error) — `byName` fails silently.

29.3.2 Autowire by Type

`10012.SpringsAutoWiring_ByType` instead matches purely by the **Java type** of the property, ignoring bean id entirely. This is more convenient, but introduces a new failure mode explored in `10013.SpringsAutoWiring_ByType_Problem_solved_by_using_primary_key`: if *two* beans of the same type exist in the container, Spring cannot decide which one to inject and throws a `NoUniqueBeanDefinitionException`. Two fixes exist:

- Mark one of the candidate beans as `primary="true"` — it becomes the default choice whenever there is ambiguity.
- Use `@Qualifier` (annotation style, covered below) to say explicitly *which* bean of that type is wanted at each injection point.

29.3.3 Autowire by Constructor

`10013.SpringsAutoWiring_ZByConstructor` autowires by matching constructor *parameter types* the same way `byType` matches property types, but against the class's constructor signature rather than its setters — effectively constructor injection with the argument values resolved automatically instead of declared with `<constructor-arg>`.

29.3.4 The Qualifier Problem, in Full

The most commonly asked interview question on this topic is exactly the scenario in `10022.SpringsAutoWiring` and `10022.SpringsAutoWiring_ByType_Problem_Qualifier`: two beans of type `Address` exist (say, `homeAddress` and `officeAddress`), and `Employee` autowires an `Address` by type. Spring has no way to know which one is intended, and refuses to guess:

```
1 @Autowired
2 @Qualifier("officeAddress")
3 private Address address;
```

`@Qualifier` breaks the tie explicitly, naming exactly which bean of the ambiguous type should be injected at this particular point.

29.3.5 Autowiring via a Static Factory Method

10014_Springs_XML_AutoWire_VIA_StaticMethod shows that a bean does not have to be created by calling its constructor at all — Spring can instead call a static factory method that returns an already-built instance:

```
1 <bean id="addressObject" class="com.accenture.ltt.bean.AddressFactory"
2     factory-method="createAddress" />
```

This is essential whenever the object you want as a bean comes from a library you don't control (e.g. it can only be obtained via a static factory or builder method), or whenever construction logic is more elaborate than a constructor call.

29.4 Annotation-Based Configuration

XML configuration is verbose. From 10015_Springs_Annotation onward, the demos switch to annotation-driven configuration, which is what almost all modern Spring (and every Spring Boot) application actually uses day to day.

Annotation	Purpose
@Component	Marks a plain class as a Spring-managed bean, to be auto-detected during component scanning
@Repository, @Service, @Controller	Specialisations of @Component that also carry semantic meaning for their architectural layer (and, for @Repository, automatic exception translation)
@Autowired	Requests dependency injection at a field, constructor, or setter, resolved by type (falling back to name/qualifier on ambiguity)
@Qualifier	Disambiguates <i>which</i> bean of a given type @Autowired should choose
@Value	Injects a literal, an EL expression, or a value read from a properties file into a field
@Configuration	Marks a class as a source of bean definitions (Java-based configuration, replacing XML)
@Bean	Inside an @Configuration class, marks a method whose return value becomes a container-managed bean

Component scanning is what makes @Component classes get picked up in the first place — either declared in XML,

```
1 <context:component-scan base-package="com.accenture.ltt" />
```

or, in pure Java configuration, via @ComponentScan("com.accenture.ltt") on an @Configuration class.

29.4.1 Method Invocation and EL Operators via Annotations

10016_Springs_Annotation_MethodInvocation and 10017_Springs_Annotation_EL_OPERATORS show that @Value is not limited to plain literals — it accepts full **Spring Expression Language (SpEL)** expressions, including calling methods on other beans and doing arithmetic/-logic at wiring time:

```
1 @Value("#{addressBean.addressLine1}")
2 private String employeeAddressLine1;
3
```

```

4 @Value("#{10 * 5}")
5 private int computedValue;

```

29.4.2 Collections via Annotations

10018.Springs_Annotations_CollectionsInjected mirrors the XML-based collection injection from Chapter 28, but with annotations — `@Autowired` on a `List<SomeType>` field injects *every* bean of that type currently in the container, in one shot, which is a common technique for plugin-style architectures (e.g. a list of all `Validator` beans).

29.4.3 Static and Non-Static Autowiring Nuances

10021.Springs_Annotation_Autowiring_static, `_static_setter`, and `_static_var_non_static_setter` explore a subtle gotcha: `@Autowired` cannot be applied meaningfully to a `static` field directly, because static fields belong to the class, not to any particular bean instance, and the Spring container has no notion of injecting into a class itself. The demos show the workaround — injecting via a normal (non-static) setter, and assigning the value to the static field *from inside* that setter.

29.5 Reading Values from a Properties File

Hardcoding configuration values (database URLs, timeouts, feature flags) directly into bean definitions is inflexible — switching environments (dev/test/prod) would mean editing and re-compiling. 10023.SpringDI_Intro_XML_PropertyPlaceholderConfigurer shows the fix: externalise those values into a `.properties` file, and let Spring substitute placeholders at container start-up.

```

1 <context:property-placeholder location="classpath:application.properties" />
2
3 <bean id="empObject" class="com.accenture.ltt.bean.Employee">
4   <property name="salary" value="${employee.salary}" />
5 </bean>

```

with `application.properties` containing:

```

1 employee.salary=200000

```

Internally, `<context:property-placeholder>` registers a `PropertySourcesPlaceholderConfigurer` (the modern successor to the older `PropertyPlaceholderConfigurer`), which — as a `BeanFactoryPostProcessor` — rewrites every `${...}` placeholder in every bean *definition* with the matching value from the properties file, before any bean is actually instantiated. The equivalent in annotation-driven Java configuration is:

```

1 @Configuration
2 @PropertySource("classpath:application.properties")
3 public class MyConfiguration {
4
5     @Value("${employee.salary}")
6     private double salary;
7 }

```

29.6 Java-Based Configuration — @Configuration and @Bean

10023.SpringDI_Intro_Java_Config_v demonstrates replacing XML entirely with a plain Java class:

```

1 @Configuration
2 public class MyConfiguration {
3
4     @Bean
5     public Address addressObject() {
6         Address address = new Address();
7         address.setAddressLine1("HSR Layout, Sector1");
8         address.setAddressLine2("Bangalore, Karnatka");
9         return address;
10    }
11
12    @Bean
13    public Employee empObject() {
14        return new Employee(addressObject(), 1001, 200000, "MSD");
15    }
16 }

```

Loaded not with `ClassPathXmlApplicationContext` but with:

```

1 ApplicationContext ctx = new AnnotationConfigApplicationContext(MyConfiguration.class);

```

Java configuration has one advantage XML cannot offer: it is checked by the compiler. A typo in a bean class name in XML only fails at runtime, when the container tries to load it; the equivalent mistake in `@Configuration` Java code fails to compile.

29.7 Spring Profiles — Environment-Specific Beans

10015.Springs.Annotation--Profiles shows **Spring Profiles**, which let you register more than one candidate bean for the same role and choose, at start-up, which set is actually active — typically because development, testing, and production each need a differently-configured bean (a different database, a different mail server) without maintaining separate copies of the whole configuration:

```

1 @Configuration
2 public class MyConfiguration {
3
4     @Bean
5     @Profile("dev")
6     public Employee devEmployee() { ... }
7
8     @Bean
9     @Profile("prod")
10    public Employee prodEmployee() { ... }
11 }

```

Which profile is active is controlled externally — typically the `spring.profiles.active` system property or environment variable — so switching environments never requires touching the configuration class itself, only which profile name is supplied at start-up.

29.8 Reading From Multiple Properties Files at Once

10023.SpringDI.Intro.XML.PropertyPlaceholderConfigurer3.XML extends the single-properties-file pattern from earlier in this chapter to show that `PropertySourcesPlaceholderConfigurer` can be pointed at *several* properties files simultaneously, which is common when a base configuration file is shared across environments and a second, smaller file overrides just a few environment-specific values:

```

1 <context:component-scan base-package="com.accenture.ltt"/>
2
3 <bean id="propConfig"
4     class="org.springframework.context.support.PropertySourcesPlaceholderConfigurer">

```

```

5     <property name="locations">
6         <list>
7             <value>classpath:config.properties</value>
8             <value>classpath:msd_config.properties</value>
9         </list>
10    </property>
11 </bean>
12
13 <bean id="employee" class="com.accenture.ltt.bean.Employee">
14     <property name="employeeId" value="${EmployeeId}" />
15     <property name="employeeName" value="${EmployeeName}" />
16     <property name="salary" value="${Salary}" />
17     <property name="address" ref="address" />
18     <property name="test" value="${TestProp}" />
19 </bean>

```

Every `${...}` placeholder is resolved by searching across *all* configured locations, in the order they are listed — so a value present in both files resolves to whichever file is listed later, letting the second file act as an override layer over the first.

29.9 Ordering Multiple BeanPostProcessors

10031 `SpringInitAndDestroyMethods.BeanPostProcessor.BeanFactoryPostProcessor.Ordered` addresses a subtlety that only shows up once more than one `BeanPostProcessor` exists in the same container: by default, the order in which multiple `BeanPostProcessors` run against a given bean is not guaranteed to match the order you declared them in. Implementing the `Ordered` interface (or using `@Order`) on each `BeanPostProcessor` lets you pin down exactly which one runs first:

```

1 @Component
2 public class LoggingPostProcessor implements BeanPostProcessor, Ordered {
3     @Override
4     public int getOrder() {
5         return 1; // lower values run first
6     }
7     // postProcessBeforeInitialization / postProcessAfterInitialization ...
8 }

```

This matters whenever one post-processor's work is a prerequisite for another's — for example, a post-processor that decrypts a configuration value must run *before* one that validates that same value.

29.10 Lazy Initialization

By default, every `singleton` bean is created eagerly, at container start-up, whether or not anything ever actually uses it — this is deliberate, so that configuration mistakes surface immediately at start-up rather than mysteriously, later, the first time a rarely-used bean is finally requested. 10027 `SpringDI.Lazy` and 10028 `SpringDI.Lazy.Java.Config` show how to override this for beans that are genuinely expensive to create and rarely needed:

```

1 <bean id="expensiveReportGenerator" class="..." lazy-init="true" />

```

```

1 @Bean
2 @Lazy
3 public ReportGenerator reportGenerator() { ... }

```

A lazily-initialised bean is only created the first time it is actually requested (either directly via `getBean(...)`, or because some eagerly-created bean depends on it).

Viva Voce — Likely Questions & Answers

Q: What is the default scope of a Spring bean, and what does it mean?

A: `singleton` — exactly one instance is created per Spring container, and every request for that bean (directly or via injection) returns the same shared object.

Q: What is the difference between singleton and prototype scope?

A: A `singleton` bean is created once and shared for every request; a `prototype` bean is instantiated fresh every single time it is requested from the container.

Q: Does Spring call a bean's destroy method for prototype-scoped beans?

A: No. The container hands out a prototype bean and stops tracking it; it never calls its destroy method. Any cleanup for a prototype bean is the calling code's own responsibility.

Q: What is the order in which init-method and dependency injection happen?

A: The constructor runs first, then dependencies are injected (via setters or fields), and only after that does the configured init-method run — because init logic frequently depends on those injected values already being present.

Q: What is the difference between autowiring byName and byType?

A: `byName` matches a property to a bean whose id is identical to the property name; `byType` matches purely on the Java type of the property, ignoring the bean's id, but fails with an ambiguity error if more than one bean of that type exists.

Q: What happens if two beans of the same type exist and you autowire byType?

A: Spring throws `NoUniqueBeanDefinitionException` because it cannot decide which of the candidate beans to inject; the fix is to mark one bean `primary`, or use `@Qualifier` to name the exact bean wanted.

Q: What is the difference between a BeanPostProcessor and a BeanFactory-PostProcessor?

A: A `BeanFactoryPostProcessor` runs before any bean is instantiated and can modify bean *definitions*; a `BeanPostProcessor` runs once per bean, immediately before and after that bean's own init method, and can modify (or wrap/proxy) the actual bean *instance*.

Q: What does @Qualifier do, and when is it needed?

A: It disambiguates which specific bean of a given type should be injected at an `@Autowired` injection point, and is needed whenever more than one bean of the same type exists in the container.

Q: What is component scanning?

A: The mechanism by which Spring automatically discovers classes annotated `@Component` (or its specialisations `@Service`, `@Repository`, `@Controller`) under a configured base package and registers them as beans, without each one needing an explicit XML `<bean>` entry.

Q: Why can't @Autowired be applied directly and meaningfully to a static field?

A: Because a static field belongs to the class itself, not to any one bean instance, and Spring's dependency injection operates on bean instances — the workaround is to inject via a normal instance setter and assign the static field from inside it.

Q: What is lazy initialization, and why isn't it Spring's default?

A: A lazily-initialised bean is only created the first time it is actually requested, rather than eagerly at container start-up. It is not the default because eager initialisation surfaces configuration errors immediately at start-up, rather than mysteriously later when a rarely-used bean is finally requested.

Q: How does `PropertySourcesPlaceholderConfigurer` work internally?

A: As a `BeanFactoryPostProcessor`, it runs before any bean is instantiated and rewrites every `${...}` placeholder found in bean *definitions* with the corresponding value from the configured properties file, so that by the time beans are actually created, no placeholders remain unresolved.

Chapter 30

Spring Testing

30.1 Why Testing Spring-Managed Code Is Different

An ordinary JUnit test instantiates the class under test directly with `new`. That does not work for a class whose whole reason for existing is that *it never calls new on its own dependencies* — a Spring-managed bean expects the container to hand it its collaborators. Testing such a bean in isolation, without ever starting the real Spring container, would mean manually constructing it and injecting mocks by hand every time, which duplicates the very wiring logic that already lives in your XML/annotations. **Spring-Test** exists to solve exactly this: it lets a JUnit test start a real (or realistically close) Spring `ApplicationContext`, get real, fully-wired beans out of it, and assert against their actual behaviour — combining the confidence of an integration test with the convenience of running inside JUnit.

30.1.1 The “Need for Testing” Demo

The very first Spring Test demo of this course, `Demo_10001.SpringTest_Need_Testing_DI.XML`, deliberately shows the *naive*, painful way first:

```
1 public class TestEmployeeClass {
2     @Test
3     public void testEmployee() {
4         ApplicationContext ctx =
5             new ClassPathXmlApplicationContext("com/accenture/ltt/resources/
6                 my_springbean.xml");
7         Employee employee = (Employee) ctx.getBean("empObject");
8         Assertions.assertTrue(employee != null);
9     }
10 }
```

This technically works, but every single `@Test` method that needs the context would have to start its own `ApplicationContext` from scratch — slow, and wasteful, because nothing about the container changes between tests in the same class.

30.2 SpringExtension and @ContextConfiguration

The very next demo, `Demo_10002.SpringTest_Testing_DI.XML`, replaces that manual bootstrapping with Spring’s actual JUnit 5 integration:

```
1 package com.accenture.ltt.test;
2
3 import org.junit.jupiter.api.Assertions;
4 import org.junit.jupiter.api.Test;
5 import org.junit.jupiter.api.extension.ExtendWith;
6 import org.springframework.beans.factory.annotation.Autowired;
```

```

7 import org.springframework.test.context.ContextConfiguration;
8 import org.springframework.test.context.junit.jupiter.SpringExtension;
9
10 import com.accenture.ltt.bean.Employee;
11
12 @ExtendWith(SpringExtension.class)
13 @ContextConfiguration(locations="/com/accenture/ltt/resources/my_springbean.xml")
14 public class TestEmployeeClass {
15
16     @Autowired
17     private Employee employee;
18
19     @Test
20     public void testEmployee(){
21         Assertions.assertTrue(employee!=null);
22     }
23
24     @Test
25     public void testEmployeeSalary(){
26         Assertions.assertTrue(employee.getSalary()==200000);
27     }
28 }

```

Two annotations do all the work:

- `@ExtendWith(SpringExtension.class)` — this is JUnit 5's extension mechanism; it plugs Spring's test infrastructure into the JUnit lifecycle, so that JUnit itself becomes responsible for starting an `ApplicationContext` before running any test method in the class.
- `@ContextConfiguration(locations = "...")` — tells `SpringExtension` *which* configuration to load the context from — here, the same `my_springbean.xml` used by the production code. (For annotation/Java config, this instead takes a `classes = MyConfiguration.class` attribute.)

Once these are in place, `@Autowired` works *inside the test class itself* — the test class becomes, in effect, one more bean the container wires up, exactly like `empObject` in the previous chapters. This is the key conceptual leap: your test class is not calling Spring; Spring is populating your test class.

30.2.1 Context Caching

Crucially, Spring's test framework does not start a fresh `ApplicationContext` for every single `@Test` method, or even for every test class. Spring **caches** the `ApplicationContext` keyed by its configuration (the exact set of XML locations / config classes / profiles used to build it), and *reuses* that same cached context across every test method — and every test *class* — that asks for an identical configuration. This is why `testEmployee()` and `testEmployeeSalary()` above, run in the same class, only pay the cost of starting the container once.

Why Context Caching Matters

Starting a full Spring container — resolving all bean definitions, wiring every dependency, running every init-method — is comparatively expensive. A test suite with hundreds of `@Test` methods spread across many classes would be intolerably slow if each one restarted the container from scratch. Context caching is what makes Spring integration testing practical at scale.

30.3 @DirtiesContext — Forcing the Cache to Reload

Context caching is a huge performance win, but it has one sharp edge: if a test method *mutates* shared state inside the cached context (e.g. it changes a singleton bean's internal field, or writes

rows into a shared in-memory database), every *other* test that reuses that same cached context now sees a container that is no longer in its original, clean state — and the next test can fail (or worse, silently produce wrong results) for a reason that has nothing to do with its own logic.

Demo_10002.SpringTest.Testing_DI_XML.DirtiesContext demonstrates the fix:

```
1 @ExtendWith(SpringExtension.class)
2 @ContextConfiguration(locations="/com/accenture/ltt/resources/my_springbean.xml")
3 public class TestEmployeeClass {
4
5     @Autowired
6     private Employee employee;
7
8     @Test
9     @org.springframework.test.annotation.DirtiesContext
10    public void testEmployeeSalaryMutation(){
11        employee.setSalary(500000); // mutates the shared singleton bean
12        Assertions.assertTrue(employee.getSalary()==500000);
13    }
14 }
```

@DirtiesContext tells Spring's test framework: "after this test method (or class) runs, throw away the cached ApplicationContext, because I have left it in a modified state — the *next* test that asks for this configuration must get a brand new, clean container, not the dirtied one." It trades away some of context caching's speed benefit exactly where correctness needs it, and only there — every test *not* marked @DirtiesContext keeps enjoying the cached, fast context.

30.4 Testing With Java Configuration

Demo_10003.SpringTest.Testing_DI_Java_Config shows the identical pattern, but pointed at an @Configuration class instead of an XML file:

```
1 @ExtendWith(SpringExtension.class)
2 @ContextConfiguration(classes = MyConfiguration.class)
3 public class TestEmployeeClass {
4
5     @Autowired
6     private Employee employee;
7
8     @Test
9     public void testEmployee(){
10        Assertions.assertTrue(employee!=null);
11    }
12 }
```

The only change from the XML version is `classes = MyConfiguration.class` in place of `locations = "..."` — everything else (the extension, the caching behaviour, the ability to @Autowired fields directly into the test) is identical, because both are just different ways of telling SpringExtension how to build the same kind of ApplicationContext.

30.5 Testing With Spring Profiles

Demo_10004.SpringTest.Testing_Profile shows that a test's context configuration can also select which **Spring profile** is active, which matters when a bean definition varies between environments (e.g. a test-only in-memory Address, versus a production one):

```
1 @ExtendWith(SpringExtension.class)
2 @ContextConfiguration(classes = MyConfiguration.class)
3 @ActiveProfiles("test")
4 public class TestEmployeeClass {
5     @Autowired
6     private Employee employee;
```

```
7 // ...
8 }
```

with the configuration class declaring profile-specific beans:

```
1 @Configuration
2 public class MyConfiguration {
3     @Bean
4     @Profile("test")
5     public Address testAddress() { ... }
6
7     @Bean
8     @Profile("prod")
9     public Address prodAddress() { ... }
10 }
```

`@ActiveProfiles("test")` on the test class ensures that only the `@Profile("test")` beans are considered when the context is built for this test run, so the test exercises test-appropriate wiring without touching production configuration at all.

30.6 Standards-Based DI Annotations — JSR-330 and JSR-250

Everything shown so far uses Spring’s own annotations (`@Autowired`, `@Component`, `@Qualifier`). Because dependency injection became common enough across the Java ecosystem to standardise, two JSRs (Java Specification Requests) define vendor-neutral equivalents that Spring also understands directly, which matters if code needs to remain portable to a different DI container.

30.6.1 JSR-330 — `@Named` and `@Inject`

Demo_10005_SpringDI_Intro_Named_JS330_Annotation_Field_Level (from the Spring Test demos) shows the standard `javax.inject` annotations used exactly where `@Component`/`@Autowired` would otherwise appear:

```
1 import javax.inject.Inject;
2 import javax.inject.Named;
3
4 @Named // JSR-330 equivalent of @Component
5 public class Employee {
6
7     @Inject // JSR-330 equivalent of @Autowired
8     private Address address;
9
10    @Value("1234")
11    private Integer employeeId;
12    // ...
13 }
```

`@Named` registers the class as a bean the same way `@Component` does, and `@Inject` requests dependency injection the same way `@Autowired` does. The two families are largely interchangeable in a Spring application, with one practical difference worth knowing: `@Autowired` has a `required` attribute (`@Autowired(required = false)`) for declaring an optional dependency directly, whereas plain `@Inject` has no such attribute — optionality under JSR-330 is instead expressed via a `Provider<T>` wrapper or, more simply in Spring code, by falling back to `@Autowired` for that specific case.

30.6.2 JSR-250 — `@Resource`

Demo_10008_SpringDI_JS250_Resource_Annotation shows `javax.annotation.Resource`, from the older JSR-250 (“Common Annotations for the Java Platform”):

```

1 import javax.annotation.Resource;
2
3 @Component
4 public class Employee {
5
6     @Resource
7     private Address address;
8
9     @Resource
10    public void setAddress(Address address) {
11        this.address = address;
12    }
13    // ...
14 }

```

The key conceptual difference from `@Autowired`/`@Inject` is the **default resolution order**: `@Resource` resolves **by name first** (matching the field or setter name to a bean id), falling back to type-based matching only if no bean of that exact name exists — the reverse of `@Autowired`'s by-type-first, disambiguate-by-qualifier-second behaviour. This makes `@Resource` closer in spirit to the XML `autowire="byName"` mode from Chapter 29, expressed as a field-level annotation instead.

Annotation	Source	Default Resolution
<code>@Autowired</code>	Spring	By type, first; disambiguated by <code>@Qualifier</code> on conflict
<code>@Inject</code>	JSR-330 (<code>javax.inject</code>)	By type, first — functionally aligned with <code>@Autowired</code>
<code>@Resource</code>	JSR-250 (<code>javax.annotation</code>)	By name, first; falls back to type if no name match

Viva Voce — Likely Questions & Answers

Q: Why can't you simply test a Spring-managed bean the way you'd test a plain Java class, with 'new'?

A: Because a Spring-managed bean is designed to receive its dependencies from the container rather than creating them itself; instantiating it with `new` bypasses dependency injection entirely and leaves its collaborators unset.

Q: What does `@ExtendWith(SpringExtension.class)` do?

A: It plugs Spring's test infrastructure into the JUnit 5 extension model, making JUnit responsible for starting (or reusing a cached) Spring `ApplicationContext` before running the test class's methods, and for injecting dependencies into the test class itself.

Q: What does `@ContextConfiguration` specify?

A: Which configuration source — an XML file (locations) or an `@Configuration` class (classes) — should be used to build the `ApplicationContext` for this test.

Q: What is Spring's test context caching, and why does it matter?

A: Spring caches the `ApplicationContext` keyed by its exact configuration and reuses the same cached instance across every test method/class that requests an identical configuration, because building a full container repeatedly for every test would be prohibitively slow.

Q: What problem does `@DirtiesContext` solve?

A: It solves the problem of a test method mutating shared state inside a cached `ApplicationContext` (e.g. changing a singleton bean's field), which would otherwise silently leak into and affect every subsequent test reusing that same cached context. It forces Spring to discard the cached context after the annotated test runs and rebuild a fresh one for what-

ever runs next.

Q: Does using `@DirtiesContext` on every test method improve or hurt overall test suite performance?

A: It hurts performance if overused, since it defeats context caching for every test it is applied to, forcing a fresh container build each time; it should be applied narrowly, only to the specific tests that actually mutate shared container state.

Q: How do you point a Spring test at Java-based configuration instead of XML?

A: Use `@ContextConfiguration(classes = MyConfiguration.class)` instead of `@ContextConfiguration(locations = "...")`; everything else about the test (autowiring, caching) behaves identically.

Q: What does `@ActiveProfiles` do in a Spring test class?

A: It specifies which Spring profile(s) should be active while building the test's `ApplicationContext`, so that only `@Profile`-matching bean definitions are considered — letting a test exercise environment-specific wiring (e.g. test doubles) without touching production bean definitions.

Q: What is the difference between Spring's `@Autowired` and the JSR-330 standard `@Inject`?

A: They are functionally very similar — both resolve by type first — but `@Autowired` additionally supports a `required` attribute for declaring an optional dependency directly, which plain `@Inject` does not offer.

Q: How does `@Resource` (JSR-250) differ from `@Autowired`/`@Inject` in its default resolution strategy?

A: `@Resource` resolves by bean name first, falling back to type-based matching only if no bean of that exact name exists — the reverse of `@Autowired`'s by-type-first, qualifier-to-disambiguate approach.

Chapter 31

Spring Data JPA & Transaction Management

31.1 Where This Chapter Picks Up

Chapter 22–25 of the previous volume covered raw JPA: an `EntityManagerFactory`, an `EntityManager`, and hand-written `persist()/find()/merge()/remove()` calls inside your own DAO classes. This chapter asks the next logical question: *if Spring already knows how to manage beans, transactions, and configuration for you, why should you keep hand-writing a DAO layer that mostly repeats the same boilerplate for every entity?* The answer is **Spring Data JPA** — and, along the way, **Spring’s transaction management**, which removes the need to manually begin/commit/rollback transactions the way plain JPA required.

31.2 Spring + JPA Integration — Before Spring Data

Before reaching Spring *Data* JPA proper, the demos first show what it takes to get an *ordinary* JPA `EntityManager` itself placed under Spring’s IoC container, so a DAO can simply have it `@Autowired` in rather than obtaining it manually. `Demo-10001.Spring-With-JPA-Integration.Xml` is a full, layered application in the familiar style from the previous volume’s Chapter 25 case study:

```
1 public interface EmployeeDAO {
2     EmployeeBean addEmployee(EmployeeBean bean);
3     List<EmployeeBean> getAllEmployeeDetails();
4     // ...
5 }
```

```
1 @Repository
2 public class EmployeeDAOImpl implements EmployeeDAO {
3
4     @PersistenceContext
5     private EntityManager entityManager;
6
7     @Override
8     public EmployeeBean addEmployee(EmployeeBean bean) {
9         EmployeeEntity entity = convertBeanToEntity(bean);
10        entityManager.persist(entity);
11        return convertEntityToBean(entity);
12    }
13    // ...
14 }
```

The key difference from the raw-JPA style of the previous volume is `@PersistenceContext`: rather than the DAO calling `Persistence.createEntityManagerFactory(...)` and build-

ing its own `EntityManager`, Spring *injects* a container-managed `EntityManager` directly into the field, the same way `@Autowired` injects any other bean. The corresponding XML wiring (Demo_10001) declares the `LocalContainerEntityManagerFactoryBean` that produces this `EntityManager`, referencing the same `persistence.xml` concepts from the previous volume; Demo_10002.Spring_With_JPA_Inte shows the same wiring expressed as Java `@Configuration`:

```

1 @Configuration
2 @EnableTransactionManagement
3 public class SpringDBConfig {
4
5     @Bean
6     public LocalContainerEntityManagerFactoryBean entityManagerFactory(DataSource
7         dataSource) {
8         LocalContainerEntityManagerFactoryBean emf = new
9             LocalContainerEntityManagerFactoryBean();
10        emf.setDataSource(dataSource);
11        emf.setPackagesToScan("com.accenture.ltt.entity");
12        emf.setJpaVendorAdapter(new HibernateJpaVendorAdapter());
13        return emf;
14    }
15
16    @Bean
17    public PlatformTransactionManager transactionManager(EntityManagerFactory emf) {
18        return new JpaTransactionManager(emf);
19    }
20 }

```

31.3 Spring's Declarative Transaction Management

In plain JDBC/JPA, a transaction is managed by hand: `begin` → `do work` → `commit`, with a `catch` block calling `rollback` on failure. This is repetitive and easy to get subtly wrong (forgetting a rollback path, or committing after a partial failure). Spring's **declarative transaction management** removes this boilerplate entirely: you *declare*, via a single annotation, that a method should run inside a transaction, and Spring wraps it in a transactional proxy that begins the transaction before the method runs and commits (or rolls back, on an uncaught exception) after it returns.

Demo_10003.Spring_With_JPA_Integration_with_TransactionManager_Xml and Demo_10004...Java show the annotation:

```

1 @Service
2 public class EmployeeServiceImpl implements EmployeeService {
3
4     @Autowired
5     private EmployeeDAO employeeDAO;
6
7     @Override
8     @Transactional
9     public EmployeeBean addEmployee(EmployeeBean bean) {
10        return employeeDAO.addEmployee(bean);
11    }
12 }

```

For `@Transactional` to have any effect, the container needs `@EnableTransactionManagement` (Java config) or `<tx:annotation-driven/>` (XML) declared once, plus a configured `PlatformTransactionManager` bean (here, a `JpaTransactionManager` wrapping the `EntityManagerFactory`) that actually knows how to begin/commit/roll back against the underlying persistence provider.

31.3.1 Where `@Transactional` Should Live — Service Layer, Not DAO

A recurring design point across these demos: `@Transactional` is placed on the **service layer** method, not the DAO method. This matters because a single business operation (“transfer

money between two accounts”, “place an order and decrement stock”) often calls *several* DAO methods that must all succeed or all fail together. If `@Transactional` were placed on each individual DAO method instead, each DAO call would get its own separate transaction, and a failure in the second DAO call could not roll back work already committed by the first. Putting `@Transactional` on the service method that calls both DAO methods ensures they share one transaction boundary.

31.3.2 Types of Context and Transaction Propagation

Demo_10005_Spring_With_JPA_Integration_with_TM_Types_of_Context.Xml explores **propagation** — what should happen when a `@Transactional` method is called from *inside* another method that is already running in a transaction:

Propagation	Behaviour
REQUIRED (default)	Join the existing transaction if one is already active; otherwise start a new one
REQUIRES_NEW	Always suspend any existing transaction and start a brand-new, independent one
NESTED	Start a nested transaction (a savepoint) inside the existing one, which can be rolled back independently of the outer transaction
MANDATORY	Requires an already-active transaction to exist; throws an exception if there isn't one
SUPPORTS	Joins a transaction if one exists, but runs non-transactionally if none does
NOT_SUPPORTED / NEVER	Suspends/forbids running inside a transaction at all

```
1 @Transactional(propagation = Propagation.REQUIRES_NEW)
2 public void auditLog(String message) { ... }
```

31.3.3 Read-Only Transactions

Demo_10006_Spring_With_JPA_Integration_with_TM_ReadOnly shows `@Transactional(readOnly = true)` for query-only service methods:

```
1 @Transactional(readOnly = true)
2 public List<EmployeeBean> getAllEmployeeDetails() { ... }
```

Marking a transaction `readOnly` is a hint to the underlying persistence provider (Hibernate) and the database driver that no writes are expected in this transaction — some providers use this to skip dirty-checking overhead on managed entities, and it also documents intent clearly to anyone reading the code.

31.3.4 Transaction Timeout

Demo_10007_Spring_With_JPA_Integration_with_TM_TimeOut shows `@Transactional(timeout = 5)` (in seconds): if the annotated method's transaction has not committed within that window, Spring forces a rollback and throws a `TransactionTimedOutException` — a safety net against a transaction that hangs (e.g. waiting on a lock) indefinitely.

31.4 Spring Data JPA — Repositories That Write Themselves

Even with `@PersistenceContext` injected cleanly, the DAO code in the previous section still hand-writes every CRUD method's body. **Spring Data JPA** removes this last layer of boilerplate: you declare an *interface* describing what operations you need, and Spring generates a

working implementation of that interface at runtime — you never write an implementing class at all.

31.4.1 CrudRepository and JpaRepository

```
1 public interface EmployeeRepository extends JpaRepository<EmployeeEntity, Integer> {
2     // that's it -- save(), findById(), findAll(), deleteById(), count() all exist
   already
3 }
```

Interface	Provides
CrudRepository<T, ID>	Basic CRUD: save, findById, findAll, deleteById, count, existsById
PagingAndSortingRepository<T, ID>	Everything CrudRepository has, plus findAll(Pageable) and findAll(Sort)
JpaRepository<T, ID>	Everything above, plus JPA-specific extras: batch operations, flush(), and returning List instead of Iterable from findAll()

The two generic type parameters are the entity type (`EmployeeEntity`) and the type of its primary key (`Integer`). Spring Data JPA inspects this interface at start-up and, using a dynamic proxy, generates an object that implements every one of these methods against the underlying `EntityManager` — the same way `ClassPathXmlApplicationContext` generated an `Employee` bean from configuration rather than your code, Spring Data generates a whole *repository implementation* from an interface signature.

31.4.2 The Repository/DAO-Wrapper Pattern

`Demo_20005_SpringJPAData_Custom_Repository` shows the actual repository interface style used through this course — built on the lower-level `@RepositoryDefinition` annotation rather than extending `JpaRepository` directly, which lets the interface expose *only* the specific methods it needs, rather than the entire CRUD surface:

```
1 package com.accenture.ltt.dao;
2
3 import org.springframework.data.repository.RepositoryDefinition;
4 import org.springframework.transaction.annotation.Transactional;
5
6 import com.accenture.ltt.entity.EmployeeEntity;
7
8 @RepositoryDefinition(idClass = Integer.class, domainClass = EmployeeEntity.class)
9 @Transactional(value = "txManager")
10 public interface EmployeeDAO {
11     EmployeeEntity save(EmployeeEntity en);
12     Iterable<EmployeeEntity> findAll();
13     // only the methods actually needed are declared -- nothing else is exposed
14 }
```

A thin **wrapper** class then sits between this repository and the rest of the application, converting between the JPA `@Entity` type and the plain bean type the service/UI layer works with (keeping persistence concerns out of the business layer, exactly as in the layered architecture from the previous volume):

```
1 @Repository
2 public class EmployeeDAOWrapper {
3
4     @Autowired
5     private EmployeeDAO employeeDAO;
6
7     public EmployeeBean addEmployee(EmployeeBean bean) throws Exception {
8         EmployeeEntity employeeEntity = convertEmployeeBeanToEntity(bean);
```

```

9      EmployeeEntity saved = employeeDAO.save(employeeEntity);
10     return convertEmployeeEntityToBean(saved);
11 }
12
13 public List<EmployeeBean> getAllEmployeeDetails() throws Exception {
14     List<EmployeeBean> result = new ArrayList<>();
15     for (EmployeeEntity entity : employeeDAO.findAll()) {
16         result.add(convertEmployeeEntityToBean(entity));
17     }
18     return result;
19 }
20
21 public static EmployeeBean convertEmployeeEntityToBean(EmployeeEntity entity) {
22     EmployeeBean employee = new EmployeeBean();
23     BeanUtils.copyProperties(entity, employee);
24     return employee;
25 }
26
27 public static EmployeeEntity convertEmployeeBeanToEntity(EmployeeBean bean) {
28     EmployeeEntity entity = new EmployeeEntity();
29     BeanUtils.copyProperties(bean, entity);
30     return entity;
31 }
32 }

```

Note `BeanUtils.copyProperties(...)` — a small Spring utility that copies same-named properties from one object to another via reflection, which avoids writing a tedious field-by-field mapper method by hand for every bean/entity pair.

31.4.3 Query Method Approach — Deriving Queries From Method Names

Spring Data JPA can generate an entire query just from a method's *name*, following a documented keyword grammar:

```

1 public interface EmployeeDAO extends JpaRepository<EmployeeEntity, Integer> {
2     List<EmployeeEntity> findByDepartmentCode(String departmentCode);
3     List<EmployeeEntity> findByEmployeeNameAndDepartmentCode(String name, String
4         deptCode);
5     List<EmployeeEntity> findBySalaryGreaterThan(double salary);
6     long countByDepartmentCode(String departmentCode);
7 }

```

Spring parses `findByDepartmentCode` into “find all `EmployeeEntity` where `departmentCode` equals the given parameter” purely from the method's name — `And/Or`, comparison keywords (`GreaterThan`, `Between`, `Like`), and result-shaping prefixes (`findBy`, `countBy`, `existsBy`) are all recognised without a single line of query code.

31.4.4 @Query — Explicit JPQL/SQL When the Method Name Isn't Enough

Once a query's logic becomes more elaborate than the naming convention can express cleanly, `@Query` lets you write the JPQL (or native SQL) directly:

```

1 public interface EmployeeDAO extends JpaRepository<EmployeeEntity, Integer> {
2
3     @Query("SELECT e FROM EmployeeEntity e WHERE e.salary > :minSalary")
4     List<EmployeeEntity> findHighEarnings(@Param("minSalary") double minSalary);
5
6     @Query("SELECT e FROM EmployeeEntity e WHERE e.departmentCode = ?1")
7     List<EmployeeEntity> findByDept(String deptCode);
8 }

```

Note the two ways parameters are bound, mirroring the JPQL parameter-passing styles from the previous volume's Chapter 24: **named parameters** (`:minSalary`, matched via `@Param("minSalary")` on the method argument) and **positional parameters** (`?1`, matched by argument order).

Viva Voce — Likely Questions & Answers

Q: What does `@PersistenceContext` do, and how is it different from creating an `EntityManagerFactory` manually?

A: It has Spring inject an already-configured, container-managed `EntityManager` directly into a field, instead of the class having to call `Persistence.createEntityManagerFactory(...)` and manage the `EntityManager`'s lifecycle itself.

Q: What problem does `@Transactional` solve?

A: It removes the boilerplate of manually beginning, committing, and rolling back a transaction; Spring wraps the annotated method in a transactional proxy that begins a transaction before the method executes and commits it after (or rolls it back on an uncaught exception).

Q: Why is `@Transactional` typically placed on service-layer methods rather than DAO methods?

A: Because a single business operation often calls multiple DAO methods that need to succeed or fail together as one unit; placing `@Transactional` on the service method that calls them ensures they share one transaction boundary, whereas annotating each DAO method separately would give each its own independent transaction.

Q: What is transaction propagation, and what does `REQUIRES_NEW` do differently from the default `REQUIRED`?

A: Propagation defines how a `@Transactional` method behaves when called from inside a method that is already running in a transaction. `REQUIRED` (the default) joins the existing transaction if one is active; `REQUIRES_NEW` always suspends any existing transaction and starts an independent new one, so its own commit/rollback doesn't depend on the caller's transaction outcome.

Q: What does `@Transactional(readOnly = true)` do?

A: It hints to the persistence provider and database driver that the annotated method's transaction performs no writes, which can let Hibernate skip dirty-checking overhead, and documents the method's intent as query-only.

Q: What is Spring Data JPA, and what problem does it solve beyond plain JPA?

A: A layer on top of JPA that lets you declare a repository as an interface (extending `CrudRepository`/`JpaRepository`, or using `@RepositoryDefinition`) and have Spring generate a working implementation at runtime, eliminating the need to hand-write DAO implementation classes for standard CRUD operations.

Q: What is the difference between `CrudRepository` and `JpaRepository`?

A: `CrudRepository` provides basic CRUD operations; `JpaRepository` extends `PagingAndSortingRepository` (which adds pagination/sorting) and adds further JPA-specific conveniences such as batch operations and returning `List` instead of `Iterable` from `findAll()`.

Q: What is the Query Method approach in Spring Data JPA?

A: Deriving a repository method's query automatically from its method name, following a keyword grammar (e.g. `findByDepartmentCode`, `findBySalaryGreaterThan`), without writing any explicit query.

Q: When would you use `@Query` instead of the Query Method naming convention?

A: When the required query's logic is too complex to express cleanly via method-name

derivation (e.g. joins, aggregate functions, or highly specific filtering), `@Query` lets you write the JPQL or native SQL explicitly while still returning results through the repository interface.

Q: What is the difference between named and positional parameters in a Spring Data `@Query`?

A: Named parameters use a `:name` placeholder matched to a method argument annotated `@Param("name")`; positional parameters use a `?N` placeholder matched purely by the method argument's position (1-indexed).

Q: Why does the custom repository pattern in this course wrap the Spring Data interface with a separate wrapper class rather than exposing the repository directly to the service layer?

A: To keep the persistence-specific entity type from leaking into the business/service layer; the wrapper converts between the JPA entity and the plain bean type used elsewhere, and can also narrow the exposed operations to only those actually needed rather than the repository's entire CRUD surface.

Chapter 32

Spring MVC

32.1 From Servlets to Spring MVC

The very first demo of this module, `Demo_10000_ServletMVC`, deliberately starts *without* Spring at all — a plain Java Servlet handling an HTTP request directly — for the same pedagogical reason Chapter 28 started with a tightly-coupled `Employee`: to make the pain visible before the framework removes it. A raw Servlet mixes together request parsing, business logic, and response-writing all inside one `doGet/doPost` method, and every new URL needs a brand-new Servlet class registered by hand in `web.xml`. **Spring MVC** organises this properly using the **Model-View-Controller** pattern: a **Controller** handles the incoming request and decides what should happen, a **Model** carries data, and a **View** (a JSP, in these demos) renders the response — and Spring's `DispatcherServlet` sits in front of all of it, routing every incoming request to the right controller method automatically.

32.2 The DispatcherServlet — Spring MVC's Front Controller

Every Spring MVC application has exactly one central Servlet, the `DispatcherServlet`, registered once in `web.xml` (or, in newer Servlet 3.0+ style, via a `WebApplicationInitializer`), which intercepts *every* matching HTTP request and dispatches it to the correct `@Controller` method based on its own configured mapping rules. This is the **Front Controller** design pattern: one single entry point through which all requests flow, rather than one Servlet class per URL.

`Demo_10002_Spring_MVC` shows the first working controller:

```
1 package com.accenture.ltt.web.controller;
2
3 import org.springframework.beans.factory.annotation.Autowired;
4 import org.springframework.stereotype.Controller;
5 import org.springframework.web.bind.annotation.RequestMapping;
6 import org.springframework.web.bind.annotation.RequestMethod;
7 import org.springframework.web.bind.annotation.RequestParam;
8 import org.springframework.web.servlet.ModelAndView;
9
10 import com.accenture.ltt.business.bean.LoginBean;
11 import com.accenture.ltt.service.LoginService;
12
13 @Controller
14 public class LoginController {
15
16     @Autowired
17     private LoginService loginService;
18
19     @RequestMapping(value = "/validateLogin.html", method = RequestMethod.POST)
20     public ModelAndView validateLogin(@RequestParam("uName") String userName,
```



```

21                                     @RequestParam("pwd") String password) {
22
23         LoginBean loginBean = new LoginBean();
24         loginBean.setPassword(password);
25         loginBean.setUserName(userName);
26
27         String returnValue = loginService.validateLogin(loginBean);
28
29         ModelAndView modelAndView = new ModelAndView();
30
31         if (returnValue.equals("success")) {
32             modelAndView.setViewName("success.jsp");
33             modelAndView.addObject("message", "Welcome: " + userName);
34         } else {
35             modelAndView.setViewName("failure.jsp");
36             modelAndView.addObject("errorMessage", "Please Login again with valid
37                                     credentials");
38         }
39         return modelAndView;
40     }

```

Walking through what happens on a real request to `/validateLogin.html`:

1. The browser POSTs a login form to `/validateLogin.html`.
2. The **DispatcherServlet** receives it (because it is registered against that URL pattern), and consults the **HandlerMapping** to find which **@Controller** method is registered for that exact path and HTTP method — here, `LoginController.validateLogin(...)`, because of `@RequestMapping(value = "/validateLogin.html", method = RequestMethod.POST)`.
3. `@RequestParam("uName")` and `@RequestParam("pwd")` extract the two form fields directly as method parameters — Spring handles the request-parsing so the controller method never touches `HttpServletRequest` directly.
4. The controller (this is the **Controller** part of MVC) delegates the actual business decision to `loginService`, which the container has already `@Autowired` in — this is *business logic*, so it does not belong inside the controller itself.
5. The controller builds a **ModelAndView**: the **Model** half (`addObject(...)`) carries data the view will need to render (the welcome message, or the error), and the **View** half (`setViewName(...)`) names which JSP should render the response.
6. Spring's **ViewResolver** translates the logical view name (`"success.jsp"`) into an actual JSP resource path, and the `DispatcherServlet` forwards the request there for rendering, with the `Model`'s attributes available to the JSP as request attributes.

32.3 Context Configuration — XML and Java

`Demo_10003.Spring_MVC.ContextConfiguration` shows the XML wiring behind the `DispatcherServlet`: a `dispatcher-servlet.xml` declaring `<mvc:annotation-driven/>` (so `@RequestMapping/@Request` and friends are recognised) and a `<context:component-scan>` over the controller package. `Demo_10006.Spring_MVC.Java_Conf_AbsAnotConfigDispServ` and `Demo_10006.Spring_MVC.Java_Conf_WebApp` show the modern, XML-free equivalent using `AbstractAnnotationConfigDispatcherServletInitializer`, a class Spring itself calls at container start-up (via `ServletContainerInitializer`, part of the Servlet 3.0 spec) instead of parsing `web.xml` at all.

32.4 Root Context vs Child (Servlet) Context

A subtlety this course deliberately calls out (`Demo_10004_Spring_RootContext_And_ChildContext`) is that a typical Spring MVC web application actually has **two** Spring containers layered on top of each other, not one:

Context	Contains
Root (Parent) <code>ApplicationContext</code>	Application-wide beans not tied to the web layer: services, DAOs/repositories, the transaction manager, the <code>EntityManagerFactory</code> . Loaded once by a <code>ContextLoaderListener</code> .
Servlet (Child) <code>WebApplicationContext</code>	Web-layer beans only: <code>@Controllers</code> , <code>ViewResolvers</code> , interceptors, form validators. Loaded per- <code>DispatcherServlet</code> (there can, in principle, be more than one <code>DispatcherServlet</code> , each with its own child context, e.g. for a REST API vs an admin UI).

The child context can see and inject beans from the root/parent context (a controller can `@Autowired` a service bean defined only in the root context), but the reverse is not true — the root context has no visibility into web-layer beans defined only in the child. This separation matters because it lets the same service/DAO layer (root context) be shared by, in principle, multiple different web front-ends (multiple child contexts), and keeps web-specific concerns out of the application's core business wiring.

32.5 View Resolvers

`Demo_10005_Spring_MVC_ViewResolver_Internal` focuses on the **`InternalResourceViewResolver`**, the component that turns a logical view name returned by a controller ("success") into an actual resource path (`/WEB-INF/views/success.jsp`):

```
1 <bean class="org.springframework.web.servlet.view.InternalResourceViewResolver">
2     <property name="prefix" value="/WEB-INF/views/" />
3     <property name="suffix" value=".jsp" />
4 </bean>
```

Because the controller only ever returns the logical name "success" (not the full path), the actual JSP location can be reorganised later purely by changing the resolver's `prefix/suffix` configuration, without touching a single controller.

32.6 Form Handling and Validation

Spring MVC's `<form:form>` JSP tags bind an HTML form directly to a Java bean (a **command object**, sometimes called a **form-backing bean**), so a controller method receives a fully-populated object instead of individually extracting each `@RequestParam`:

```
1 @Controller
2 public class EmployeeFormController {
3
4     @RequestMapping(value = "/addEmployee", method = RequestMethod.POST)
5     public ModelAndView submitEmployee(@ModelAttribute("employeeForm") @Valid
6         EmployeeFormBean formBean,
7         BindingResult bindingResult) {
8         if (bindingResult.hasErrors()) {
```

```

8         ModelAndView mv = new ModelAndView("employeeForm");
9         return mv;
10    }
11    // proceed -- formBean is fully populated and already validated
12    ModelAndView mv = new ModelAndView("employeeSaved");
13    mv.addObject("employee", formBean);
14    return mv;
15 }
16 }

```

32.6.1 Standard Bean Validation Annotations

jakarta.validation (formerly javax.validation) annotations declare validation rules directly on the bean's fields, and `@Valid` on the controller parameter tells Spring to run them automatically before the method body executes:

Annotation	Rule Enforced
<code>@NotNull</code> / <code>@NotEmpty</code> / <code>@NotBlank</code>	Value must be present (and, for the latter two, non-empty / non-whitespace-only)
<code>@Size(min=, max=)</code>	String/collection length must fall within bounds
<code>@Min</code> / <code>@Max</code>	Numeric value must fall within bounds
<code>@Email</code>	Value must be a syntactically valid email address
<code>@Pattern(regexp=)</code>	Value must match a given regular expression

32.6.2 Custom Validators

When a validation rule cannot be expressed by a standard annotation (e.g. “employee ID must not already exist in the database”), Spring MVC lets you implement the `Validator` interface directly:

```

1 @Component
2 public class EmployeeFormValidator implements Validator {
3
4     @Override
5     public boolean supports(Class<?> clazz) {
6         return EmployeeFormBean.class.equals(clazz);
7     }
8
9     @Override
10    public void validate(Object target, Errors errors) {
11        EmployeeFormBean bean = (EmployeeFormBean) target;
12        if (bean.getSalary() < 0) {
13            errors.rejectValue("salary", "salary.negative", "Salary cannot be negative");
14        }
15    }
16 }

```

registered against a controller with `@InitBinder`, so it runs alongside (or instead of) the standard annotation-driven rules for that specific form.

32.7 The `<form:form>` Tag Library and UI Components

Demo_10007_Spring_MVC_SpringForm introduces Spring's own JSP tag library for building HTML forms bound directly to a command object, rather than writing plain `<input>` tags and reading each field back with `@RequestParam` one at a time:

```

1 @Controller
2 public class LoginController {
3
4     @Autowired

```

```

5     LoginService loginService;
6
7     @RequestMapping(value = "/loadLogin.html", method = RequestMethod.GET)
8     public ModelAndView loadLoginPage() {
9         // the empty command object is placed into the Model up front,
10        // so the JSP form has something to bind its fields to
11        return new ModelAndView("login", "loginBean", new LoginBean());
12    }
13
14    @RequestMapping(value = "/validateLogin.html", method = RequestMethod.POST)
15    public ModelAndView validateLogin(@ModelAttribute("loginBean") LoginBean paramBean,
16                                     HttpServletRequest request) {
17        String returnVal = loginService.validateLogin(paramBean);
18        ModelAndView modelAndView = new ModelAndView();
19        if (returnVal.equals("success")) {
20            modelAndView.setViewName("success");
21            modelAndView.addObject("message", "Welcome: " + paramBean.getUserName());
22        } else {
23            modelAndView.setViewName("failure");
24            modelAndView.addObject("errorMessage", "Please Login again with valid
25                                credentials");
26        }
27        return modelAndView;
28    }

```

The corresponding JSP uses `<form:form modelAttribute="loginBean">` together with `<form:input path="userName"/>` and `<form:password path="password"/>`: each path attribute binds that specific input directly to the named property of the command object placed in the Model by `loadLoginPage()` — Spring both *renders* the form pre-populated from the bean's current values and *parses* the submission back into a fresh instance of that same bean type, without any manual parameter-by-parameter extraction.

Demo_10008.Spring_MVC_UIComponents.Static and Demo_10009..._Dynamic extend this to the richer form tags — `<form:select>`, `<form:checkbox>`, `<form:radiobutton>` — distinguishing between **static** options (a fixed, hardcoded list of dropdown values written directly into the JSP) and **dynamic** options (a Map or List of choices supplied from the Model by the controller, typically because those choices come from a database and can change over time).

32.8 Model vs Request Scope — Where Data Actually Lives

Demo_10010.Spring_MVC.SpringModel.Vs.RequestScope makes a distinction that trips up many beginners: adding an attribute to the **Model** (via `ModelAndView.addObject(...)` or a `ModelMap/Model` parameter) and adding one directly to the **HttpServletRequest** both end up readable by the JSP using the same `${...}` EL syntax — because Spring MVC copies every Model attribute onto the request as a request attribute before forwarding to the view. The difference only becomes visible once you also inject `HttpServletRequest` directly into the controller and compare:

```

1  @RequestMapping(value = "/confirm.html", method = RequestMethod.GET)
2  public ModelAndView confirmDetails(@ModelAttribute("employeeBean") EmployeeBean
3                                     employeeBean,
4                                     ModelMap map, HttpServletRequest req, HttpSession
5                                     sess) {
6      System.out.println("Model Data: " + map.get("employeeBean"));
7      System.out.println("Request: " + req.getAttribute("employeeBean"));
8      System.out.println("Session: " + sess.getAttribute("employeeBean"));
9      ModelAndView modelAndView = new ModelAndView();
10     modelAndView.setViewName("ConfirmDetails");
11     modelAndView.addObject("message", "Welcome to confirm Details: " + employeeBean.
12                             getName());
13     return modelAndView;
14 }

```

The Model is scoped to exactly *one* request-response cycle and is the idiomatic, framework-provided way to pass data to a view; reading `HttpServletRequest/HttpSession` directly (as this demo does, deliberately, for illustration) bypasses that abstraction and ties the controller to the Servlet API. `Demo_10017_Spring_MVC_Need_Of_Session` and `Demo_10018_...Session_Vs_Req_Vs_SpringMoc` extend this into a genuine multi-request flow — a registration form (request 1) followed by a confirmation page (request 2) — to show *why* the Model alone is not enough for that scenario: the Model dies at the end of request 1, so any data that must survive into request 2 (the submitted registration details, to be shown back for confirmation) has to be placed into the **HttpSession** instead, which persists across multiple requests from the same client until it expires or is invalidated.

32.9 Custom Validation via a Constraint Annotation

Chapter 32's earlier discussion of custom validators used the `Validator` interface directly. `Demo_10012_Spring_MVC_Custom_Validations` shows the alternative, more reusable approach: writing your *own* constraint annotation, backed by a `ConstraintValidator` implementation, so the rule can be applied declaratively on a bean field exactly like a built-in `@NotNull` or `@Size`:

```

1 // The annotation itself
2 @Documented
3 @Constraint(validatedBy = EmployeeNameValidator.class)
4 @Target({ ElementType.METHOD, ElementType.FIELD })
5 @Retention(RetentionPolicy.RUNTIME)
6 public @interface EmployeeNameValidatorVal {
7     String message() default "Employee name must have 2 or 3 words";
8     Class<?>[] groups() default {};
9     Class<? extends Payload>[] payload() default {};
10 }

1 // The validation logic itself
2 package com.accenture.ltt.validator;
3
4 import javax.validation.ConstraintValidator;
5 import javax.validation.ConstraintValidatorContext;
6
7 public class EmployeeNameValidator implements ConstraintValidator<
8     EmployeeNameValidatorVal, String> {
9     @Override
10     public boolean isValid(String employeeName, ConstraintValidatorContext arg1) {
11         if (employeeName == null) {
12             return false;
13         }
14         int wordCount = employeeName.split(" ").length;
15         return wordCount == 2 || wordCount == 3;
16     }
17 }

```

Once defined this way, `@EmployeeNameValidatorVal` can be placed directly on the `EmployeeBean.employeeName` field, and `@Valid` on the controller parameter runs it automatically alongside every standard `jakarta.validation` constraint — this is precisely how the standard annotations (`@NotNull`, `@Size`, and so on) are themselves implemented internally, so writing your own is following the exact same extension point the framework itself uses. `Demo_10012_Spring_MVC_Custom_Validations_Using_Init` shows the alternative registration route for a plain `Validator`-interface-based validator (from earlier in this chapter): rather than `@Valid` picking it up automatically, it is registered explicitly per-controller via `@InitBinder`, which is useful when the same command object needs *different* validation rules depending on which controller is handling it.

32.10 Exception Handling in Spring MVC

Demo_10014_Spring_MVC_ExceptionHandlers_Need first shows the problem without any handling at all: an unchecked business exception thrown from deep inside a service call (e.g. `InvalidCountryNameAndAgeCombinationException`) propagates all the way up through the controller and out to the Servlet container, which shows the client a raw, unstyled HTTP 500 error page — not something a real application can show a user.

32.10.1 @ExceptionHandler — Scoped to One Controller

Demo_10015_Spring_MVC_ExceptionHandlers shows the fix, scoped to a single controller:

```

1 @Controller
2 public class EmployeeController {
3     // ... normal request-handling methods ...
4
5     @ExceptionHandler(value = InvalidCountryNameAndAgeCombinationException.class)
6     public ModelAndView handleInvalidCombination(
7         InvalidCountryNameAndAgeCombinationException ex) {
8         ModelAndView mv = new ModelAndView();
9         mv.setViewName("ExceptionHandlerPage");
10        mv.addObject("message", ex.getMessage());
11        return mv;
12    }
13 }

```

Any exception of the declared type, thrown from *any* request-handling method inside this same controller class, is caught here instead of propagating further, and rendered as a proper, styled view.

32.10.2 @ControllerAdvice — Global, Application-Wide Exception Handling

The limitation of the previous approach is exactly that it is scoped to one controller — duplicating identical `@ExceptionHandler` methods into every controller class in an application is exactly the kind of repetition Spring is meant to remove. Demo_10016_Spring_MVC_ExceptionHandlers_ControllerAdvice shows the application-wide fix:

```

1 @ControllerAdvice
2 public class GlobalExceptionHandlerAdvice {
3
4     @ExceptionHandler(value = InvalidCountryNameAndAgeCombinationException.class)
5     public ModelAndView handleInvalidCountryNameAndAgeCombinationException(
6         InvalidCountryNameAndAgeCombinationException exception) {
7         ModelAndView modelAndView = new ModelAndView();
8         modelAndView.setViewName("ExceptionHandlerPage");
9         modelAndView.addObject("message", exception.getMessage());
10        modelAndView.addObject("exception", exception);
11        return modelAndView;
12    }
13
14     @ExceptionHandler(value = Exception.class)
15     public ModelAndView handleAllExceptions(Exception exception) {
16         ModelAndView modelAndView = new ModelAndView();
17         modelAndView.setViewName("GeneralizedExceptionHandlerPage");
18         modelAndView.addObject("message", exception.getMessage());
19         modelAndView.addObject("exception", exception);
20        return modelAndView;
21    }
22 }

```

`@ControllerAdvice` marks this class as a cross-cutting component whose `@ExceptionHandler` methods apply to *every* `@Controller` in the application, not just one — and, exactly like a `catch` block hierarchy, Spring picks the *most specific* matching handler for a given exception: the narrowly-typed `handleInvalidCountryNameAndAgeCombinationException` method

catches that one specific exception type, while `handleAllExceptions(Exception exception)` acts as a catch-all fallback for anything not matched by a more specific handler elsewhere.

32.11 Spring MVC Integration With Spring Data JPA

The final piece brings everything from Chapters 28–31 together in one request-handling pipeline: a `@Controller` method receives the request, delegates to a `@Service` method (running inside a `@Transactional` boundary), which in turn calls a Spring Data JPA repository to persist or retrieve entities — and the result flows back up through the `ModelAndView` to the JSP view. This is exactly the same layered structure (`Controller` → `Service` → `Repository/DAO` → `Entity`) that the JPA case study built in the previous volume, with the web layer now sitting on top of it rather than a plain `main()` method driving it.

Viva Voce — Likely Questions & Answers

Q: What design pattern does the Spring MVC `DispatcherServlet` implement, and why does it matter?

A: The Front Controller pattern — a single Servlet receives every incoming request and dispatches it to the appropriate handler, rather than each URL needing its own separately-registered Servlet. This centralises cross-cutting concerns (routing, view resolution, exception handling) in one place.

Q: What is the difference between the root `ApplicationContext` and the `DispatcherServlet`'s (child) `WebApplicationContext`?

A: The root context holds application-wide, non-web beans (services, DAOs, transaction managers) and is loaded once via a `ContextLoaderListener`; the child context holds web-layer beans (controllers, view resolvers) and is loaded per `DispatcherServlet`. The child context can see beans from the root context, but not vice versa.

Q: What is a `ModelAndView`, and what are its two responsibilities?

A: An object a controller method returns that carries both the Model (data the view needs to render, via `addObject`) and the View (the logical name of the view that should render the response).

Q: What does a `ViewResolver` do?

A: It translates the logical view name returned by a controller (e.g. "success") into an actual resource location (e.g. `/WEB-INF/views/success.jsp`), decoupling controller code from the physical layout of view files.

Q: What is the difference between `@RequestParam` and a command/form-backing object bound via `@ModelAttribute`?

A: `@RequestParam` extracts a single named request parameter directly as a method argument; `@ModelAttribute` binds an entire form submission to a Java bean's properties in one step, letting a controller method work with a fully-populated object rather than extracting each field individually.

Q: How does Spring MVC perform standard bean validation on a submitted form, and how do you detect validation failures?

A: By annotating the command object's fields with `jakarta.validation` constraints (e.g. `@NotNull`, `@Size`) and marking the controller method's parameter `@Valid`; a `BindingResult` parameter immediately after it captures any validation errors, which the controller checks via `bindingResult.hasErrors()` before proceeding.

Q: When would you write a custom Validator instead of relying only on standard bean validation annotations?

A: When a validation rule cannot be expressed declaratively by a standard constraint annotation — for example, a rule that depends on checking against database state, or a rule that spans multiple fields with custom logic.

Q: Why is business logic delegated to a service bean from the controller, rather than written inline inside the controller method?

A: To keep the controller focused purely on request/response handling (parsing input, choosing a view), while business rules and transactional boundaries live in the service layer — the same separation of concerns that motivated the layered DAO/service architecture in the JPA chapters.

Q: What does the path attribute of a <form:input> tag actually do?

A: It binds that specific form field to the named property of the command object supplied via `modelAttribute` on the enclosing `<form:form>`, so Spring both pre-populates the rendered input from the bean's current value and parses the submitted value straight back into that same property on form submission.

Q: Why does adding data to the Model and adding it directly to the `HttpServletRequest` look the same from a JSP's perspective?

A: Because Spring MVC copies every Model attribute onto the request as a request attribute before forwarding to the view, so both are readable via identical EL syntax in the JSP — the Model is simply the framework-idiomatic way of getting there, without the controller needing to touch the Servlet API directly.

Q: Why can't the Model alone carry data across a multi-step flow such as a registration form followed by a confirmation page?

A: Because the Model's scope is exactly one request-response cycle; it does not survive into a second, separate request. Data that must persist across multiple requests from the same client (such as submitted form details awaiting confirmation) needs to be stored in the `HttpSession` instead.

Q: How do you create a reusable custom validation rule that can be applied declaratively, the same way `@NotNull` or `@Size` can?

A: By defining your own annotation meta-annotated with `@Constraint(validatedBy = ...)`, and implementing the validation logic in a class that implements `ConstraintValidator<YourAnnotation, FieldType>`; once defined, the annotation can be placed on any bean field and is picked up automatically by `@Valid` alongside the standard constraints.

Q: What is the difference between using `@ExceptionHandler` directly inside a `@Controller` versus inside a class annotated `@ControllerAdvice`?

A: An `@ExceptionHandler` method inside a `@Controller` only catches exceptions thrown by request-handling methods within that same controller class; the same method inside a class annotated `@ControllerAdvice` applies globally, across every controller in the application.

Q: If both a specific and a generic `@ExceptionHandler` method exist in the same `@ControllerAdvice` class, which one runs for a given exception?

A: Spring selects the most specific matching handler for the actual exception type thrown, falling back to a more generic handler (e.g. one declared for `Exception.class`) only when no more specific match exists — the same specificity-first resolution a Java `catch` block hierarchy would apply.

Chapter 33

Spring REST

33.1 Why REST, When Spring MVC Already Exists

Everything built in Chapter 32 assumes the client is a *browser*, expecting an HTML page back. But a mobile app, a React front-end (covered later in this course), or another microservice calling this application does not want an HTML page — it wants **data**, typically as **JSON**, that it can parse and use itself. **REST** (Representational State Transfer) is an architectural style for exposing exactly that: resources (like “an employee”, “a car”) addressed by URLs, manipulated through standard HTTP methods, and represented as a portable data format rather than a rendered view. Spring REST is not a separate framework from Spring MVC — it is the *same* `DispatcherServlet`, `HandlerMapping`, and controller machinery from Chapter 32, with a small number of additional annotations that change what a controller method *returns*: data, not a view name.

33.2 @RestController and Mapping HTTP Verbs to CRUD

Demo_10029_Spring_MVC_REST_CRUD_MAP_Xml is a complete REST CRUD controller for an in-memory employee store:

```
1 package com.accenture.ltt.web.controller;
2
3 import java.util.ArrayList;
4 import java.util.List;
5
6 import org.springframework.beans.factory.annotation.Autowired;
7 import org.springframework.http.HttpStatus;
8 import org.springframework.http.MediaType;
9 import org.springframework.http.ResponseEntity;
10 import org.springframework.web.bind.annotation.PathVariable;
11 import org.springframework.web.bind.annotation.RequestBody;
12 import org.springframework.web.bind.annotation.RequestMapping;
13 import org.springframework.web.bind.annotation.RequestMethod;
14 import org.springframework.web.bind.annotation.RestController;
15
16 import com.accenture.ltt.bussiness.bean.EmployeeBean;
17 import com.accenture.ltt.service.EmployeeService;
18
19 @RestController
20 public class EmployeeController {
21
22     @Autowired
23     private EmployeeService employeeService;
24
25     @RequestMapping(value = "emp/controller/getDetails", method = RequestMethod.GET,
26                     produces = MediaType.APPLICATION_JSON_VALUE)
27     public ResponseEntity<List<EmployeeBean>> getEmployeeDetails() {
```

```

28     List<EmployeeBean> listEmployee = new ArrayList<>(employeeService.
29         getAllEmployee());
30     return new ResponseEntity<>(listEmployee, HttpStatus.OK);
31 }
32 @RequestMapping(value = "emp/controller/getDetailsById/{id}", method =
33     RequestMethod.GET,
34     produces = MediaType.APPLICATION_JSON_VALUE)
35 public ResponseEntity<EmployeeBean> getEmployeeDetailByEmployeeId(@PathVariable("id"
36     ") int myId) {
37     EmployeeBean employee = employeeService.getEmployeeDetailsById(myId);
38     if (employee != null) {
39         return new ResponseEntity<>(employee, HttpStatus.OK);
40     } else {
41         return new ResponseEntity<>(HttpStatus.NOT_FOUND);
42     }
43 }
44 @RequestMapping(value = "/emp/controller/addEmp", method = RequestMethod.POST,
45     consumes = MediaType.APPLICATION_JSON_VALUE, produces = MediaType.
46     TEXT_HTML_VALUE)
47 public ResponseEntity<String> addEmployee(@RequestBody EmployeeBean employee) {
48     int count = employeeService.addEmployee(employee);
49     return new ResponseEntity<>("Employee added successfully with id:" + count,
50         HttpStatus.CREATED);
51 }
52 @RequestMapping(value = "/emp/controller/updateEmp", method = RequestMethod.PUT,
53     consumes = MediaType.APPLICATION_JSON_VALUE, produces = MediaType.
54     APPLICATION_JSON_VALUE)
55 public ResponseEntity<EmployeeBean> updateEmployee(@RequestBody EmployeeBean
56     employee) {
57     if (employeeService.getEmployeeDetailsById(employee.getEmployeeId()) == null) {
58         return new ResponseEntity<>((EmployeeBean) null, HttpStatus.
59             INTERNAL_SERVER_ERROR);
60     }
61     EmployeeBean updated = employeeService.updateEmployee(employee);
62     return new ResponseEntity<>(updated, HttpStatus.OK);
63 }
64 @RequestMapping(value = "/emp/controller/deleteEmp/{id}", method = RequestMethod.
65     DELETE,
66     produces = MediaType.APPLICATION_JSON_VALUE)
67 public ResponseEntity<EmployeeBean> deleteEmployee(@PathVariable("id") int myId) {
68     if (employeeService.getEmployeeDetailsById(myId) == null) {
69         return new ResponseEntity<>((EmployeeBean) null, HttpStatus.
70             INTERNAL_SERVER_ERROR);
71     }
72     EmployeeBean employee = employeeService.removeEmployee(myId);
73     return new ResponseEntity<>(employee, HttpStatus.OK);
74 }
75 }

```

33.2.1 @RestController vs @Controller

`@RestController` is a convenience meta-annotation equal to `@Controller` plus `@ResponseBody` applied to every method in the class. `@ResponseBody` is the crucial piece: it tells Spring *not* to treat a method's return value as a logical view name to resolve, but to instead write it directly into the HTTP response body — serialised, typically to JSON, by a registered `HttpMessageConverter` (Jackson, by default, if it is on the classpath). This is the single conceptual difference between everything in Chapter 32 and everything in this chapter: whether the return value means “go render this view” or “send this object back as data”.

33.2.2 Mapping HTTP Verbs to CRUD Semantics

REST conventionally maps each CRUD operation onto the HTTP method whose semantics match it:

HTTP Method	CRUD Operation	Notes
GET	Read	Safe (no side effects) and idempotent (repeating it changes nothing further)
POST	Create	Not idempotent — calling it twice typically creates two resources
PUT	Update (full replace)	Idempotent — calling it twice with the same body leaves the same end state
DELETE	Delete	Idempotent — deleting an already-deleted resource still ends with it gone

Note that the demo controller above uses the older, single `@RequestMapping(method = RequestMethod.GET/...)` style; modern Spring code more commonly uses the shorthand annotations `@GetMapping`, `@PostMapping`, `@PutMapping`, and `@DeleteMapping`, which are exactly equivalent but more concise and self-documenting.

33.2.3 @PathVariable vs @RequestParam vs @RequestBody

Annotation	Extracts
@PathVariable	A segment of the URL path itself, e.g. the <code>{id}</code> in <code>/getDetailsById/{id}</code> — used to identify <i>which</i> resource
@RequestParam	A query-string parameter (<code>?name=value</code>) or form field — used for optional filters, pagination, etc.
@RequestBody	The entire HTTP request body, deserialised (typically from JSON) into a Java object — used to receive the full payload for a POST/PUT

33.2.4 ResponseEntity and HttpStatus

`ResponseEntity<T>` lets a controller method control the *entire* HTTP response — status code, headers, and body — rather than only the body. This is why the demo above returns `HttpStatus.NOT_FOUND` (404) when a lookup fails, `HttpStatus.CREATED` (201) after a successful POST, and `HttpStatus.OK` (200) for successful reads/updates: a REST client is expected to make decisions based on the numeric status code, not just by inspecting the body, so choosing the semantically correct one at each branch matters.

33.3 @ResponseBody Before @RestController — and Producing More Than One Format

It helps to see `@ResponseBody` used explicitly, on a plain `@Controller`, before accepting `@RestController` as a convenient shorthand for it. `Demo.10027.Spring.MVC.Controller.WS.XML` does exactly that, and additionally shows that the *same* underlying data can be exposed in more than one representation, chosen purely by the `produces` attribute of `@RequestMapping`:

```

1 @Controller
2 public class EmployeeController {
3
4     @Autowired
5     private EmployeeServiceImpl employeeService;
6
7     @RequestMapping(value = "emp/controller/json/getDetails", method = RequestMethod.
8         GET,
9         produces = MediaType.APPLICATION_JSON_VALUE)
10    @ResponseBody
11    public List<EmployeeBean> getEmployeeDetails() {
12        return employeeService.getAllEmployees();
13    }
14 }

```

```

14     @RequestMapping(value = "emp/controller/xml/getDetails", method = RequestMethod.GET
15         ,
16         produces = MediaType.APPLICATION_XML_VALUE)
17     @ResponseBody
18     public EmployeeListWrapper getEmployeeDetails2() {
19         List<EmployeeBean> listEmployee = employeeService.getAllEmployees();
20         EmployeeListWrapper employeeListWrapper = new EmployeeListWrapper();
21         employeeListWrapper.setListEmp(listEmployee);
22         return employeeListWrapper;
23     }

```

Two things are worth noticing here. First, `@ResponseBody` is applied *per method* on a plain `@Controller` — exactly the piece that `@RestController` later applies automatically to *every* method in the class, so nothing conceptually changes between this style and the `@RestController` style used earlier in this chapter. Second, the XML-producing method wraps the `List<EmployeeBean>` inside an `EmployeeListWrapper` object rather than returning the bare list directly — this is necessary because JAXB (the default XML marshaller Spring uses) needs a single root element to serialise to valid XML, and a bare Java `List` has no natural single root tag the way a JSON array does. `Demo_10028_Spring_MVC_Controller_WS_Java` repeats this exact pair of endpoints configured via Java-based configuration instead of XML, and `Demo_10030_Spring_MVC_REST_CRUD_MAP_JavaConfig` does the same for the full CRUD controller from earlier in this chapter — in every case, only *how the container is wired* changes; the controller code and its annotations are identical either way.

33.4 Consuming a REST Service — RestTemplate

Everything above is the *server* side — exposing a REST API. The client side — one Java application calling *another* application's REST API — uses Spring's **RestTemplate**, exactly as shown in `Demo10033_Spring_MVC_REST_TEMPLATE`:

```

1  RestTemplate restTemplate = new RestTemplate();
2
3  // GET a single resource
4  EmployeeBean employee = restTemplate.getForObject(
5      "http://localhost:8080/emp/controller/getDetailsById/{id}",
6      EmployeeBean.class, 1001);
7
8  // GET a list of resources
9  EmployeeBean[] employees = restTemplate.getForObject(
10     "http://localhost:8080/emp/controller/getDetails", EmployeeBean[].class);
11
12 // POST a new resource
13 ResponseEntity<String> response = restTemplate.postForEntity(
14     "http://localhost:8080/emp/controller/addEmp", newEmployee, String.class);
15
16 // PUT to update, DELETE to remove
17 restTemplate.put("http://localhost:8080/emp/controller/updateEmp", updatedEmployee);
18 restTemplate.delete("http://localhost:8080/emp/controller/deleteEmp/{id}", 1001);

```

RestTemplate handles the mechanics of the HTTP call (opening the connection, serialising the request body to JSON, deserialising the JSON response back into a Java object) so that calling a REST endpoint from Java code reads almost like calling a local method. `getForObject` is used for simple GETs where you only need the body; `exchange(...)` (not shown above) is the more general form used when you also need control over request headers or the full `ResponseEntity` of the response.

33.5 Spring MVC REST Integrated With Spring Data JPA

Demo_10031_Spring_MVC_REST_CRUD_JPA_Data_XML and Demo_10032_..._JavaConfig bring Chapter 31's Spring Data JPA repositories directly under a REST controller, completing the same layered pipeline as Chapter 32's MVC integration, except the controller now returns `ResponseEntity<EmployeeBean>` (JSON) instead of a `ModelAndView` (a rendered JSP):

```

1 @RestController
2 @RequestMapping("/emp")
3 public class EmployeeController {
4
5     @Autowired
6     private EmployeeService employeeService;
7
8     @GetMapping(value =("/{id}", produces = MediaType.APPLICATION_JSON_VALUE)
9     public ResponseEntity<EmployeeBean> getById(@PathVariable("id") int id) {
10         EmployeeBean employee = employeeService.getEmployeeDetailsById(id);
11         return employee != null
12             ? new ResponseEntity<>(employee, HttpStatus.OK)
13             : new ResponseEntity<>(HttpStatus.NOT_FOUND);
14     }
15 }

```

where `employeeService` internally delegates to the Spring Data JPA repository (or the `EmployeeDAOWrapper` from Chapter 31) exactly as before — the REST controller is simply a thinner, JSON-producing replacement for the JSP-rendering controller from Chapter 32, sitting on top of the exact same service and persistence layers.

Viva Voce — Likely Questions & Answers

Q: What is the key difference between `@Controller` and `@RestController`?

A: `@RestController` is equivalent to `@Controller` plus `@ResponseBody` on every method, meaning a method's return value is written directly into the HTTP response body (typically as JSON) rather than being resolved as a logical view name.

Q: What does `@ResponseBody` actually change about how a controller method's return value is handled?

A: Instead of Spring treating the returned value as a view name to hand to a `ViewResolver`, it passes the value to a registered `HttpMessageConverter` (by default, Jackson) which serialises it directly into the response body.

Q: Why is PUT considered idempotent but POST is not?

A: Calling PUT repeatedly with the same request body leaves the resource in the same end state each time (a full replace); calling POST repeatedly typically creates a new resource each time, so the end state differs after each call.

Q: What is the difference between `@PathVariable` and `@RequestParam`?

A: `@PathVariable` extracts a value embedded directly in the URL path (e.g. the resource's identifier), while `@RequestParam` extracts a query-string parameter or form field — typically used for optional, non-identifying data like filters or pagination.

Q: What does `ResponseEntity` let a controller method do that a plain return type cannot?

A: It lets the method explicitly control the full HTTP response — status code and headers, in addition to the body — rather than always returning the default 200 OK with just the body.

Q: What is `RestTemplate` used for?

A: It is Spring's client-side helper for calling another application's REST endpoints from

Java code, handling the HTTP request/response mechanics (serialising the outgoing body, deserialising the incoming response) so the calling code can work with the request/response as ordinary Java objects.

Q: If a REST controller and an MVC controller from the previous chapter both call the same service layer, what actually changes between them?

A: Essentially nothing about the service or persistence layers changes — only the controller's return type and annotations change, from a ModelAndView naming a JSP view (rendered HTML) to a ResponseEntity carrying a serialisable object (returned as JSON).

Q: What does @RestController save you from writing compared to a plain @Controller?

A: It saves you from adding @ResponseBody to every single request-handling method individually — @RestController applies it automatically across the whole class.

Q: Why might a controller method returning XML wrap a List<EmployeeBean> inside a container object (like EmployeeListWrapper) instead of returning the bare list directly?

A: Because the default JAXB-based XML marshaller requires a single root element to produce valid XML, and a bare Java List has no natural single root tag the way a JSON array does; wrapping it in an object gives JAXB that root element to serialise around.

Q: How can the exact same underlying data be exposed as both JSON and XML from the same service, without duplicating the business logic?

A: By declaring two separate @RequestMapping-annotated methods with different produces media types (APPLICATION_JSON_VALUE vs APPLICATION_XML_VALUE), both delegating to the identical underlying service call — only the final serialisation format differs, chosen by which endpoint URL and Accept header the client uses.

Chapter 34

Spring Batch

34.1 The Problem Spring Batch Solves

Everything covered so far — Spring MVC, Spring REST — is built around handling one HTTP request at a time, quickly, for a human or another program waiting on the other end. Some real-world work does not fit that shape at all: reading a CSV file with a million payroll rows and loading them into a database, generating a nightly summary report, or migrating data between two systems. This kind of work is called **batch processing**: large volumes of data, processed without a human waiting interactively, usually on a schedule, and — critically — work that must be resumable if it fails halfway through, because re-running a million-row job from row one after a crash at row 900,000 wastes enormous time. Hand-rolling this yourself means writing your own chunking logic, your own progress-tracking, and your own restart logic, all of which is exactly the kind of undifferentiated boilerplate a framework should remove. **Spring Batch** is that framework.

34.2 Core Vocabulary — Job and Step

Spring Batch organises all work around two central concepts:

- A **Job** is the entire batch process from start to finish — e.g. “load today’s employee CSV into the database”. A Job is composed of one or more Steps, run in a defined order.
- A **Step** is one independent, self-contained phase of a Job — e.g. “read employees from the CSV, transform them, write them to the database” might be a single Step, and a second Step might then “generate a summary report from what was just loaded”.

Each Step, in the most common style (**chunk-oriented processing**), is itself built from exactly three collaborating pieces:

Component	Responsibility
ItemReader	Reads one item at a time from the data source (a file, a database, a queue) until there is nothing left to read
ItemProcessor	Transforms, validates, or filters a single item before it is written (optional — a Step can skip this and write items unchanged)
ItemWriter	Writes out a chunk (a configured batch size) of processed items at once, in a single transaction

Why “Chunk-Oriented” Rather Than Row-by-Row

Committing to the database after every single row would be extremely slow at scale, and committing only once at the very end would mean a crash near the finish line loses *all* progress. Chunk-oriented processing reads and processes items one at a time, but only commits a whole **chunk** (e.g. every 10 items) as one transaction — balancing throughput against how much work is lost if something fails partway.

34.3 A Complete Job, End to End — Java Configuration

The Spring Batch demos of this course build the same job — read employee records from a CSV, transform them, and write them out as XML — first in XML configuration and then, more instructively, entirely in Java. Here is the actual `SpringBatchConfig` from the course:

```

1 package com.accenture.ltt.config;
2
3 import javax.sql.DataSource;
4
5 import org.springframework.batch.core.Job;
6 import org.springframework.batch.core.Step;
7 import org.springframework.batch.core.configuration.annotation.EnableBatchProcessing;
8 import org.springframework.batch.core.configuration.annotation.JobBuilderFactory;
9 import org.springframework.batch.core.configuration.annotation.StepBuilderFactory;
10 import org.springframework.batch.core.launch.JobLauncher;
11 import org.springframework.batch.core.launch.support.SimpleJobLauncher;
12 import org.springframework.batch.core.repository.JobRepository;
13 import org.springframework.batch.core.repository.support.JobRepositoryFactoryBean;
14 import org.springframework.batch.item.ItemProcessor;
15 import org.springframework.batch.item.ItemReader;
16 import org.springframework.batch.item.file.FlatFileItemReader;
17 import org.springframework.batch.item.file.mapping.DefaultLineMapper;
18 import org.springframework.batch.item.file.transform.DelimitedLineTokenizer;
19 import org.springframework.batch.item.xml.StaxEventItemWriter;
20 import org.springframework.beans.factory.annotation.Autowired;
21 import org.springframework.beans.factory.annotation.Qualifier;
22 import org.springframework.context.annotation.Bean;
23 import org.springframework.context.annotation.Configuration;
24 import org.springframework.core.io.ClassPathResource;
25 import org.springframework.core.io.FileSystemResource;
26 import org.springframework.jdbc.datasource.DataSourceTransactionManager;
27 import org.springframework.jdbc.datasource.DriverManagerDataSource;
28 import org.springframework.jdbc.datasource.init.DataSourceInitializer;
29 import org.springframework.jdbc.datasource.init.ResourceDatabasePopulator;
30 import org.springframework.oxm.jaxb.Jaxb2Marshaller;
31 import org.springframework.transaction.PlatformTransactionManager;
32
33 import com.accenture.ltt.dto.Employee;
34 import com.accenture.ltt.mapper.CustomRecordFieldSetMapper;
35 import com.accenture.ltt.transformer.CustomItemProcessor;
36
37 @Configuration
38 @EnableBatchProcessing
39 public class SpringBatchConfig {
40
41     @Autowired
42     private StepBuilderFactory stepBuilderFactory;
43
44     @Autowired
45     private JobBuilderFactory jobBuilderFactory;
46
47     @Bean
48     public DriverManagerDataSource dataSource() {
49         DriverManagerDataSource driverSource = new DriverManagerDataSource();
50         driverSource.setDriverClassName("com.mysql.cj.jdbc.Driver");
51         driverSource.setUrl("jdbc:mysql://localhost:3306/spring_batch_demos");
52         driverSource.setUsername("root");
53         driverSource.setPassword("root");
54         return driverSource;

```



```

55     }
56
57     @Bean
58     public ResourceDatabasePopulator databasePopulator() {
59         ResourceDatabasePopulator populator = new ResourceDatabasePopulator();
60         populator.addScript(new ClassPathResource("org/springframework/batch/core/
61             schema-drop-mysql.sql"));
62         populator.addScript(new ClassPathResource("org/springframework/batch/core/
63             schema-mysql.sql"));
64         return populator;
65     }
66
67     @Bean
68     public DataSourceInitializer dataSourceInitializer(DataSource dataSource) {
69         DataSourceInitializer initializer = new DataSourceInitializer();
70         initializer.setDataSource(dataSource);
71         initializer.setDatabasePopulator(databasePopulator());
72         return initializer;
73     }
74
75     @Bean(name = "jobRepository")
76     public JobRepository jobRepository(DataSource dataSource,
77         PlatformTransactionManager transactionManager)
78         throws Exception {
79         JobRepositoryFactoryBean jobRepoFactory = new JobRepositoryFactoryBean();
80         jobRepoFactory.setDataSource(dataSource);
81         jobRepoFactory.setTransactionManager(transactionManager);
82         jobRepoFactory.setDatabaseType("mysql");
83         return jobRepoFactory.getObject();
84     }
85
86     @Bean
87     public PlatformTransactionManager transactionManager(DataSource dataSource) {
88         return new DataSourceTransactionManager(dataSource);
89     }
90
91     @Bean(name = "reader1")
92     public FlatFileItemReader<Employee> itemReader() {
93         FlatFileItemReader<Employee> builder = new FlatFileItemReader<>();
94         builder.setResource(new ClassPathResource("employee.csv"));
95         DefaultLineMapper<Employee> lineMapper = new DefaultLineMapper<>();
96         DelimitedLineTokenizer lineTokenizer = new DelimitedLineTokenizer();
97         lineTokenizer.setDelimiter(",");
98         lineTokenizer.setNames("employeeId", "employeeName", "designation",
99             "dateOfJoining", "departmentCode", "salary");
100         lineMapper.setLineTokenizer(lineTokenizer);
101         lineMapper.setFieldSetMapper(new CustomRecordFieldSetMapper());
102         builder.setLineMapper(lineMapper);
103         return builder;
104     }
105
106     @Bean(name = "processor1")
107     public CustomItemProcessor itemProcessor() {
108         return new CustomItemProcessor();
109     }
110
111     @Bean(name = "writer1")
112     public StaxEventItemWriter<Employee> itemWriter() {
113         StaxEventItemWriter<Employee> writer = new StaxEventItemWriter<>();
114         writer.setResource(new FileSystemResource("xml/output.xml"));
115         writer.setMarshaller(recordMarshaller());
116         writer.setRootTagName("employees");
117         return writer;
118     }
119
120     public Jaxb2Marshaller recordMarshaller() {
121         Jaxb2Marshaller marshaller = new Jaxb2Marshaller();
122         marshaller.setClassesToBeBound(Employee.class);
123         return marshaller;
124     }
125
126     @Bean
127     public Step step1(@Qualifier("reader1") ItemReader<Employee> itemReader,

```

```

125         @Qualifier("writer1") StaxEventItemWriter<Employee> itemWriter,
126         @Qualifier("processor1") ItemProcessor<Employee, Employee>
            itemProcessor) {
127     return this.stepBuilderFactory.get("step1")
128         .<Employee, Employee>chunk(10)
129         .reader(itemReader)
130         .processor(itemProcessor())
131         .writer(itemWriter)
132         .build();
133 }
134
135 @Bean(name = "firstBatchJob")
136 public Job firstBatchJob(Step step1) {
137     return jobBuilderFactory.get("firstBatchJob").start(step1).build();
138 }
139
140 @Bean
141 public JobLauncher jobLauncher(JobRepository jobRepository) throws Exception {
142     SimpleJobLauncher simpleJobLauncher = new SimpleJobLauncher();
143     simpleJobLauncher.setJobRepository(jobRepository);
144     return simpleJobLauncher;
145 }
146 }

```

34.3.1 Reading This Configuration Piece by Piece

- **@EnableBatchProcessing** — turns on Spring Batch's own infrastructure beans (JobBuilderFactory, StepBuilderFactory, and the default JobRepository wiring), the same way **@EnableTransactionManagement** in Chapter 31 turned on transactional-proxy support.
- **dataSource() and the schema scripts** — Spring Batch tracks every Job's execution history (when it ran, which step it reached, whether it succeeded) in its own set of database tables. `schema-drop-mysql.sql/schema-mysql.sql` are Spring Batch's *own* bundled scripts that create those bookkeeping tables — this is infrastructure for tracking the batch process itself, entirely separate from whatever business tables the job's own reader/writer touch.
- **JobRepository** — the component that actually persists that execution history (Job instances, Job executions, Step executions) to the tables above. This is what makes a Job **restartable**: if it fails partway, the JobRepository knows exactly which step, and which chunk, it reached.
- **reader1 (a FlatFileItemReader)** — reads `employee.csv` line by line. A `DelimitedLineTokenizer` splits each line on `,` into named fields (`employeeId`, `employeeName`, ...), and a `CustomRecordFieldSetMapper` (an application-supplied class implementing `FieldSetMapper<Employee>`) converts each tokenized line into an actual `Employee` object.
- **processor1 (a CustomItemProcessor)** — the application-supplied transformation step, run once per item, between reading and writing (for example, normalising a field, or computing a derived value).
- **writer1 (a StaxEventItemWriter)** — writes the processed `Employee` objects out as XML, using a `Jaxb2Marshaller` to convert each Java object into its XML representation, wrapped in a root `<employees>` tag.
- **step1** — assembled with `stepBuilderFactory.get("step1").chunk(10)...`: the `chunk(10)` call is exactly the chunk size discussed above — ten `Employee` records are read and processed before the writer is invoked once with all ten, as a single transaction.
- **firstBatchJob** — the Job itself, built with `jobBuilderFactory.get("firstBatchJob").start(step1).build()` — a Job with exactly one Step.

- **JobLauncher** — the component actually invoked (typically from a small **UITester**/driver class with a main method, or a Servlet endpoint, or a scheduled trigger) to *start* the Job: `jobLauncher.run(firstBatchJob, jobParameters)`.

34.4 The Equivalent XML Configuration

The same Job, expressed in the older XML style also covered in this course (`spring-batch-config.xml`), makes the `<job>/<step>/<chunk>` structure even more explicit, since Spring Batch's own XML namespace mirrors the Job/Step/chunk vocabulary directly as XML elements:

```

1 <beans:beans
2   xmlns="http://www.springframework.org/schema/batch"
3   xmlns:beans="http://www.springframework.org/schema/beans"
4   xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
5   xsi:schemaLocation="http://www.springframework.org/schema/beans
6     https://www.springframework.org/schema/beans/spring-beans.xsd
7     http://www.springframework.org/schema/batch
8     https://www.springframework.org/schema/batch/spring-batch.xsd">
9
10  <beans:import resource="spring-config.xml" />
11
12  <beans:bean id="itemReader" class="org.springframework.batch.item.file.
13    FlatFileItemReader">
14    <beans:property name="resource" value="classpath:employee.csv" />
15    <beans:property name="lineMapper">
16      <beans:bean class="org.springframework.batch.item.file.mapping.
17        DefaultLineMapper">
18        <beans:property name="lineTokenizer">
19          <beans:bean class="org.springframework.batch.item.file.transform.
20            DelimitedLineTokenizer">
21            <beans:property name="names"
22              value="employeeId,employeeName,designation,dateOfJoining,
23                departmentCode,salary" />
24          </beans:bean>
25        </beans:property>
26        <beans:property name="fieldSetMapper">
27          <beans:bean class="com.accenture.ltt.mapper.
28            CustomRecordFieldSetMapper" />
29        </beans:property>
30      </beans:bean>
31    </beans:property>
32  </beans:bean>
33
34  <beans:bean id="itemProcessor" class="com.accenture.ltt.transformer.
35    CustomItemProcessor" />
36
37  <beans:bean id="itemWriter" class="org.springframework.batch.item.xml.
38    StaxEventItemWriter">
39    <beans:property name="resource" value="file:xml/output.xml" />
40    <beans:property name="marshaller" ref="recordMarshaller" />
41    <beans:property name="rootTagName" value="employees" />
42  </beans:bean>
43
44  <beans:bean id="recordMarshaller" class="org.springframework.xml.jaxb.
45    Jaxb2Marshaller">
46    <beans:property name="classesToBeBound">
47      <beans:list>
48        <beans:value>com.accenture.ltt.dto.Employee</beans:value>
49      </beans:list>
50    </beans:property>
51  </beans:bean>
52
53  <job id="firstBatchJob">
54    <step id="step1">
55      <tasklet>
56        <chunk reader="itemReader" writer="itemWriter" processor="itemProcessor"
57          commit-interval="10" />
58      </tasklet>
59    </step>
60  </job>

```

```

52
53 </beans:beans>

```

Notice `commit-interval="10"` in the XML version — it is exactly the same concept as `.chunk(10)` in the Java configuration, just expressed as an XML attribute instead of a builder method call.

34.5 Configuring Multiple Steps and Step Elements

A realistic Job is rarely a single Step. Once a Job needs to, say, first load raw records, then validate them, then generate a summary — each of those becomes its own Step, and the Job declares the order (and, optionally, conditional transitions) between them:

```

1 @Bean
2 public Job multiStepBatchJob(Step loadStep, Step validateStep, Step summaryStep) {
3     return jobBuilderFactory.get("multiStepBatchJob")
4         .start(loadStep)
5         .next(validateStep)
6         .next(summaryStep)
7         .build();
8 }

```

Each Step here is built exactly like `step1` above — its own reader/processor/writer trio, its own chunk size — and `.next(...)` defines what runs after the previous Step *completes successfully*. Spring Batch also supports conditional flow (`.on("FAILED").to(errorHandlingStep)`) for Jobs that need to branch based on how a Step actually finished, rather than always following a strictly linear sequence.

34.6 Job Parameters and Late Binding

A batch Job is very often the *same* Job definition run repeatedly with different inputs — “load *today’s* file”, not always literally the same file. **Job Parameters** are how a specific value is supplied at launch time, without hardcoding it into the Job’s configuration:

```

1 JobParameters jobParameters = new JobParametersBuilder()
2     .addString("inputFile", "employee_20260728.csv")
3     .addLong("runId", System.currentTimeMillis())
4     .toJobParameters();
5
6 jobLauncher.run(firstBatchJob, jobParameters);

```

Two details matter here:

- **Job Parameters identify a specific run.** Spring Batch’s `JobRepository` treats a Job together with a particular set of parameters as one unique **JobInstance**. Running the same Job definition with the *same* parameters again, once that instance already completed successfully, is rejected (`JobInstanceAlreadyCompleteException`) — this is precisely what makes Spring Batch’s restart behaviour safe: adding a fresh, distinguishing parameter (like a timestamp `runId`) is a common way to deliberately force a genuinely new run when that is what’s actually wanted.
- **Late binding** lets a Step-scoped bean (like the `FlatFileItemReader` above) reference a Job Parameter directly inside its own configuration, resolved only when the Step actually starts running — not when the application context is first built:

```

1 @Bean
2 @StepScope
3 public FlatFileItemReader<Employee> itemReader(
4     @Value("#{jobParameters['inputFile']}") String inputFile) {
5     FlatFileItemReader<Employee> reader = new FlatFileItemReader<>();

```

```

6     reader.setResource(new ClassPathResource(inputFile));
7     // ...
8     return reader;
9 }

```

`@StepScope` is what makes this possible: it tells Spring not to create this bean once, eagerly, at container start-up (when no Job Parameters exist yet), but instead to create a fresh instance of it *each time the Step runs*, at which point the actual Job Parameters for that specific run are already known and can be substituted into the `#{jobParameters['inputFile']}` SpEL expression.

34.7 The Full Demo Progression — From a Single Step to a Production-Shaped Job

The Spring Batch demos of this course are deliberately arranged as a staircase, each one adding exactly one new capability on top of the last: Demo_1.1_JDBC_db_metadata → 1.2_SQLite → 1.3..._Read_from_db → 1.4..._Multiple_Steps → 1.5..._JobListener → 1.6..._Step_Chunk_Listener → 1.7..._Multiple_Listeners → 1.8..._next_tag → 1.9..._end_tag → 2.0..._stop_tag / fail_tag → 2.1..._split_tag → 2.2..._decider_job_params → 2.3..._lateBinding → 2.4..._Concurrency. Each “A”-suffixed demo (1.1A, 1.2A, ...) is the identical Job expressed in Java `@Configuration` instead of XML — the same pairing pattern used throughout this course wherever both styles are taught side by side.

34.8 Listeners — Observing (Without Altering) a Running Job

A **listener** lets code run at specific lifecycle points of a Job or Step, without becoming part of the read/process/write pipeline itself — useful for logging, auditing, metrics, or alerting, cleanly separated from the actual data-processing logic.

34.8.1 JobExecutionListener

Demo_1.5_SQLite_Read_from_db_Multiple_Steps_JobListener shows the coarsest-grained listener, which observes the Job as a whole:

```

1 package com.accenture.ltt.job.configuration;
2
3 import java.util.Date;
4 import org.springframework.batch.core.BatchStatus;
5 import org.springframework.batch.core.JobExecution;
6 import org.springframework.batch.core.JobExecutionListener;
7 import org.springframework.batch.core.JobInstance;
8
9 public class MSDJobConfigListener implements JobExecutionListener {
10
11     @Override
12     public void beforeJob(JobExecution jobExecution) {
13         JobInstance j = jobExecution.getJobInstance();
14         System.out.println("[MSDCustom]: Job started [" + new Date() + "]: "
15             + j.getJobName() + ", instance Id: " + j.getInstanceId());
16     }
17
18     @Override
19     public void afterJob(JobExecution jobExecution) {
20         JobInstance j = jobExecution.getJobInstance();
21         if (jobExecution.getStatus() == BatchStatus.COMPLETED) {
22             System.out.println("[MSDCustom]: Job Completed Successfully: [" + new Date() +
23                 "]: "
24                 + j.getJobName() + ", instance Id: " + j.getInstanceId());
25         } else if (jobExecution.getStatus() == BatchStatus.FAILED) {
26             System.out.println("Job Failed [" + new Date() + "]: "

```

```

26         + j.getJobName() + ", instance Id: " + j.getInstanceId());
27     }
28 }
29 }

```

`beforeJob(...)` fires exactly once, right before the first Step starts; `afterJob(...)` fires exactly once, after the last Step finishes, and can inspect `jobExecution.getStatus()` to branch behaviour on whether the Job actually succeeded (`BatchStatus.COMPLETED`) or failed (`BatchStatus.FAILED`). Registering it is a one-line addition to the Job builder: `jobBuilderFactory.get("first").listener(new MSDJobConfigListener()).start(step1).build()`.

34.8.2 Step, Chunk, Read, Process, and Write Listeners

`Demo_1.6...Step_Chunk_Listener` and `Demo_1.7...Multiple_Step_Multiple_Listeners` extend the same idea down to finer granularity — a Job can have a listener, and *each Step within it* can independently have its own listeners at multiple levels:

Listener Interface	Observes
<code>JobExecutionListener</code>	Before/after the entire Job
<code>StepExecutionListener</code>	Before/after one specific Step
<code>ChunkListener</code>	Before/after each individual chunk commit within a Step
<code>ItemReadListener</code>	Before/after/on-error around each individual item <i>read</i>
<code>ItemProcessListener</code>	Before/after/on-error around each individual item <i>processed</i>
<code>ItemWriteListener</code>	Before/after/on-error around each chunk <i>written</i>

The demo's own listener classes — `MSDStepListener`, `MSDChunkListener`, `MSDReadListener`, `MSDProcessListener`, `MSDWriteListener` — each implement exactly one of these interfaces, and are attached to the Step builder independently:

```

1 Step step1 = stepBuilderFactory.get("step1")
2     .<Employee, Employee>chunk(10)
3     .reader(itemReader)
4     .processor(itemProcessor)
5     .writer(itemWriter)
6     .listener(new MSDStepListener())
7     .listener(new MSDChunkListener())
8     .listener((ItemReadListener<Employee>) new MSDReadListener())
9     .listener((ItemProcessListener<Employee, Employee>) new MSDProcessListener())
10    .listener((ItemWriteListener<Employee>) new MSDWriteListener())
11    .build();

```

This fine-grained visibility — knowing exactly which item failed to read, or which chunk failed to write, rather than only that “the Job failed” — is what makes production batch jobs debuggable at scale.

34.9 Controlling Flow Between Steps — next, end, stop, fail, and split

A real Job is rarely one Step, or even a strictly linear sequence of Steps that always all run. Spring Batch's flow-control elements let a Job's execution branch based on how each Step actually finished — its **exit status** (`COMPLETED`, `FAILED`, or a custom status set explicitly).

Element	Effect
next	(Demo_1.8) Explicitly names which Step runs after the current one on a given exit status, rather than relying on pure declaration order
end	(Demo_1.9) Ends the Job successfully at this point, marking it COMPLETED , even if further steps are declared elsewhere in the flow
stop	(Demo_2.0A) Ends the Job with status STOPPED — distinct from failure, and specifically restartable : a stopped Job can later be resumed via restartable(true) from exactly where it stopped
fail	(Demo_2.0) Explicitly marks the Job FAILED on a given condition, rather than only failing implicitly when a Step throws an uncaught exception
split	(Demo_2.1) Runs two or more independent Step <i>flows</i> concurrently, in parallel threads, rather than one strictly after another

```

1 @Bean
2 public Job flowControlledJob(Step step1, Step step2, Step errorStep) {
3     return jobBuilderFactory.get("flowControlledJob")
4         .start(step1)
5         .on("FAILED").to(errorStep)
6         .from(step1).on("*").to(step2)
7         .end()
8         .build();
9 }

```

Here, `.on("FAILED").to(errorStep)` is exactly the **next**-style branching from Demo_1.8 expressed in the Java builder API: if `step1` exits with status **FAILED**, control moves to `errorStep` instead of `step2`; `.on("*")` matches any other exit status and proceeds normally to `step2`.

Demo_2.1.SQLite_Read_From_db.Multiple_Steps.split_tag shows **split** specifically for running genuinely independent work concurrently — for example, one flow validating incoming records while a completely unrelated flow generates an unrelated report, neither depending on the other's output:

```

1 @Bean
2 public Job splitJob(Flow flow1, Flow flow2) {
3     return jobBuilderFactory.get("splitJob")
4         .start(flow1)
5         .split(new SimpleAsyncTaskExecutor())
6         .add(flow2)
7         .end()
8         .build();
9 }

```

34.10 JobExecutionDecider — Branching on More Than Just Exit Status

Sometimes the decision about which Step should run next depends on something more than a Step's own exit status — for example, on a Job Parameter supplied at launch time. Demo_2.2A.SQLite_Read_From_db.Multiple_Steps.split_tag shows a **JobExecutionDecider**, a dedicated component whose only job is to make exactly that kind of routing decision:

```

1 package com.accenture.ltt.job.ex.decider;
2
3 import org.springframework.batch.core.JobExecution;
4 import org.springframework.batch.core.JobParameters;
5 import org.springframework.batch.core.StepExecution;
6 import org.springframework.batch.core.job.flow.FlowExecutionStatus;
7 import org.springframework.batch.core.job.flow.JobExecutionDecider;
8
9 public class MSDDecider implements JobExecutionDecider {
10
11     @Override

```



```

12     public FlowExecutionStatus decide(JobExecution jobExecution, StepExecution
13         stepExecution) {
14         JobParameters jp = jobExecution.getJobParameters();
15         String user = jp.getString("username");
16         Long id = jp.getLong("id");
17
18         if (user.equals("ADMIN") && id.equals(11498686L)) {
19             return FlowExecutionStatus.COMPLETED;
20         } else {
21             return FlowExecutionStatus.FAILED;
22         }
23     }

```

Wired into the Job flow, the decider's returned `FlowExecutionStatus` is treated exactly like a Step's own exit status for routing purposes:

```

1  @Bean
2  public Job deciderJob(Step step1, Step adminOnlyStep, Step rejectedStep) {
3      return jobBuilderFactory.get("deciderJob")
4          .start(step1)
5          .next(new MSDDecider())
6              .on("COMPLETED").to(adminOnlyStep)
7              .from(new MSDDecider()).on("FAILED").to(rejectedStep)
8          .end()
9          .build();
10 }

```

This separates *routing logic* (“should this Job proceed to the admin-only step?”) cleanly from *processing logic* (what each Step's reader/processor/writer actually does) — the decider's only responsibility is to answer a routing question, nothing else. `Demo_2.2B...decider_job_params_validator` additionally shows a `JobParametersValidator`, which rejects a Job launch outright (before any Step runs at all) if the supplied Job Parameters don't meet required criteria (e.g. a mandatory `username` parameter is missing), rather than letting the Job start and only failing later, mid-flow, inside the decider.

34.11 Late Binding and Concurrency, Together

`Demo_2.3...lateBinding` combines the Job Parameter late-binding pattern from earlier in this chapter with a decider, so the actual routing decision depends on a value only known at launch time; `Demo_2.4...Concurrency` goes one step further and runs multiple chunks of the *same* Step concurrently, via a configured `TaskExecutor`:

```

1  @Bean
2  public Step concurrentStep(TaskExecutor taskExecutor) {
3      return stepBuilderFactory.get("concurrentStep")
4          .<Employee, Employee>chunk(10)
5          .reader(itemReader())
6          .processor(itemProcessor())
7          .writer(itemWriter())
8          .taskExecutor(taskExecutor)
9          .throttleLimit(4)
10         .build();
11 }

```

`taskExecutor(...)` tells the Step to process multiple chunks in parallel threads rather than strictly one after another, and `throttleLimit(4)` caps how many chunks may be in flight concurrently at once — balancing throughput against resource usage (database connections, memory) the same way the chunk size itself balances commit frequency against transaction overhead.

Viva Voce — Likely Questions & Answers

Q: What problem does Spring Batch solve that Spring MVC/REST do not address?

A: Processing large volumes of data in bulk, without an interactive client waiting on a response, in a way that can track progress and safely resume from a checkpoint if it fails partway through — rather than every request being handled quickly and independently.

Q: What is the difference between a Job and a Step in Spring Batch?

A: A Job is the entire batch process end to end; a Step is one independent, self-contained phase of that process. A Job is composed of one or more Steps run in a defined sequence.

Q: What are the three components of chunk-oriented Step processing, and what does each do?

A: ItemReader reads one item at a time from the data source; ItemProcessor (optional) transforms or validates each item; ItemWriter writes out a whole chunk of processed items together, in one transaction.

Q: Why does Spring Batch commit in chunks rather than after every single item or only once at the very end?

A: Committing after every item is too slow at scale; committing only once at the end means a failure near completion loses all progress. Chunk-oriented commits balance throughput against how much work is lost if the job fails partway through.

Q: What is the JobRepository, and why does it matter for restartability?

A: It is the component that persists a Job's execution history — which Job instances have run, which Step they reached, and whether they succeeded — to Spring Batch's own database tables. This history is what lets a failed Job be resumed from where it left off rather than restarted from the beginning.

Q: What is a JobInstance, and how are Job Parameters related to it?

A: A JobInstance is a Job definition combined with one specific, unique set of Job Parameters. Spring Batch refuses to re-run a JobInstance that has already completed successfully with the same parameters, which is why a genuinely new run typically needs at least one distinguishing parameter (e.g. a timestamp).

Q: What is late binding in the context of Spring Batch, and why is @StepScope needed for it?

A: Late binding defers resolving a configuration value (e.g. an input file name) until the Step actually starts executing, at which point the real Job Parameters for that run are known. @StepScope is required because it tells Spring to create a fresh bean instance per Step execution rather than once, eagerly, at application start-up before any Job Parameters exist.

Q: In the demo's chunk-oriented step, what does .chunk(10) (or commit-interval="10" in XML) actually control?

A: It sets the chunk size — the number of items read and processed before the writer is invoked once with the entire batch, committed as a single transaction.

Q: What is the purpose of the Spring Batch schema scripts (schema-drop-mysql.sql / schema-mysql.sql) used in the demo's DataSourceInitializer?

A: They create Spring Batch's own bookkeeping tables (tracking Job instances, executions, and step executions), which are entirely separate from whatever business data tables the Job's reader/writer actually read from or write to.

Q: How would you configure a Job made of three sequential Steps rather than one?

A: By building each Step independently (its own reader/processor/writer) and chaining them in the Job definition with `.start(step1).next(step2).next(step3).build()`, so each Step runs only after the previous one completes successfully.

Q: What is the difference between a JobExecutionListener and a StepExecutionListener?

A: A JobExecutionListener's before/after methods fire exactly once each, around the entire Job; a StepExecutionListener's before/after methods fire around one specific Step, letting you observe or react to that Step's own outcome independently of the overall Job.

Q: Name the finer-grained listener interfaces available below the Step level, and what each observes.

A: ChunkListener observes before/after each chunk commit; ItemReadListener observes around each individual item read; ItemProcessListener observes around each individual item processed; ItemWriteListener observes around each chunk written.

Q: What is the difference between a Step ending with stop versus fail?

A: `stop` ends the Job with status STOPPED, which is explicitly restartable — a stopped Job can later resume from where it stopped; `fail` marks the Job FAILED explicitly, which is Spring Batch's ordinary failure status, distinct from a deliberate, resumable pause.

Q: What does the split element let a Job do that a purely linear sequence of next steps cannot?

A: It runs two or more independent Step flows concurrently, in parallel threads, rather than strictly one after another — appropriate when those flows have no dependency on each other's output.

Q: What is a JobExecutionDecider, and how is its return value used in the Job flow?

A: A dedicated component that inspects the JobExecution/StepExecution (for example, to read a Job Parameter) and returns a FlowExecutionStatus; that returned status is then used for routing exactly like a Step's own exit status would be, letting the Job branch on more than what a Step itself reports.

Q: What is a JobParametersValidator, and when does it run relative to the Job's Steps?

A: A component that validates the supplied Job Parameters before any Step is allowed to run, rejecting the launch outright if required parameters are missing or invalid — catching a configuration mistake immediately rather than letting the Job start and fail partway through.

Q: What do `taskExecutor(...)` and `throttleLimit(...)` control on a Step builder?

A: `taskExecutor(...)` makes the Step process multiple chunks concurrently in separate threads instead of strictly sequentially; `throttleLimit(...)` caps how many chunks may be processed concurrently at once, balancing throughput against resource usage.

Chapter 35

Practice Activities & Assignments

35.1 How the Practice Material Is Organised

Alongside the instructor-led demos covered in Chapters 27–34, the course provides two further kinds of hands-on material, and it is worth being clear about the difference between them, because they are used differently:

- **Activities** (e.g. `Activity_10001_SpringTest_Testing_DI_XML_To_Trainee`, `Activity_10001_Spring_MVC`) are *mostly-complete* projects handed to the trainee, with a small number of specific methods deliberately left unimplemented (returning `null`, or simply empty) for the trainee to complete during the session, applying a technique that was just demonstrated moments earlier.
- **Assignments** (e.g. `Assignment_10001_1/2_Springs_Core_Participant`, `Assignment_10004_1/2_SpringR`, the `SpringORM_DataJPA_MicroserviceAssignment` pair) are typically issued *after* a topic is fully taught, as independent practice; several are structured as a numbered checklist of “POA” (Point of Activity) instructions embedded directly as code comments, walking the trainee through building an entire feature end-to-end, one annotation or method at a time.

This chapter walks through a representative sample of both, in the same theory-plus-code style as the rest of this volume, so a trainee revisiting this material later has a worked explanation of not just *what* the demos showed, but what was actually being asked of them in the practice exercises that followed.

35.2 Activity: Completing a Spring Test Suite (XML-Based DI)

`Activity_10001_SpringTest_Testing_DI_XML_To_Trainee` hands the trainee a working `Employee/Address` XML-configured project (structurally identical to the demos in Chapter 30) along with three test classes to complete: `TestAddressClass`, `TestEmployeeClass`, and `TestEmployeeCollections`. The exercise deliberately gives the trainee only the `@ExtendWith/@ContextConfiguration` scaffolding and asks them to write the actual `@Test` methods themselves — for example, verifying that `Employee`’s injected `Address` collaborator is not null, and that a collection-typed property (from the collections-injection pattern in Chapter 28) contains exactly the expected number of elements, in the expected order. This directly exercises the context-caching behaviour from Chapter 30: because all three test classes point at the identical `my_springbean.xml` configuration, Spring reuses one single cached `ApplicationContext` across all of them, which is worth confirming by adding a temporary `System.out.println` inside a bean’s constructor and observing that it only prints once across the whole test run, not once per test class.

35.3 Activity: Completing a Spring MVC + JPA Controller

Activity_10001_Spring_MVC_With_JPA_Integration.Xml_To_Trainee extends a fully-working layered application (Controller → Service → DAO → Entity, exactly the shape built in Chapters 31–32) with a second controller, `EmployeeControllerAssignment`, whose methods are deliberately left as stubs:

```

1 @Controller
2 @SuppressWarnings("unused")
3 public class EmployeeControllerAssignment {
4
5     @Autowired
6     private EmployeeService employeeService;
7
8     @InitBinder
9     public void initBinder(WebDataBinder binder) {
10         SimpleDateFormat sdf = new SimpleDateFormat("dd-MMM-yyyy");
11         binder.registerCustomEditor(Date.class, new CustomDateEditor(sdf, true));
12     }
13
14     @RequestMapping(value = "/LoadGetDetailsInDateRange", method = RequestMethod.GET)
15     public ModelAndView getEmployeesWithinDateRangePage() throws Exception {
16         return null; // <-- trainee completes this
17     }
18
19     @RequestMapping(value = "/GetDataWithinDateRange", method = RequestMethod.GET)
20     public ModelAndView getEmployeesDataInDateRange(@RequestParam("fromDate") Date
21                                                     startDate,
22                                                     @RequestParam("toDate") Date
23                                                     endDate) throws Exception {
24         return null; // <-- trainee completes this
25     }
26
27     @RequestMapping(value = "/LoadDeleteEmployeeByName", method = RequestMethod.GET)
28     public ModelAndView getDeleteEmployeesByNamePage() throws Exception {
29         return null; // <-- trainee completes this
30     }
31
32     @RequestMapping(value = "/DeleteEmployeeByName", method = RequestMethod.POST)
33     public ModelAndView deleteEmployeesByName(@ModelAttribute("employeeBean")
34                                               EmployeeBean bean) throws Exception {
35         return null; // <-- trainee completes this
36     }
37
38     @ExceptionHandler(value = Exception.class)
39     public ModelAndView handleAllExceptions(Exception exception) {
40         ModelAndView modelAndView = new ModelAndView();
41         modelAndView.setViewName("GeneralizedExceptionHandlerPage");
42         modelAndView.addObject("message", exception.getMessage());
43         modelAndView.addObject("exception", exception);
44         return modelAndView;
45     }
46 }

```

Three separate techniques from earlier chapters have to come together to complete this stub correctly:

- **The `@InitBinder` already provided** registers a `CustomDateEditor` so that `@RequestParam("fromDate") Date startDate` can bind a plain `dd-MMM-yyyy` form string directly to a `java.util.Date` parameter — without it, Spring has no idea how to convert the incoming text into a `Date` and the request fails to bind. Completing `getEmployeesDataInDateRange(...)` means calling into the service/DAO layer with a JPQL query using **parameter passing** (Chapter 31, and Chapter 24 of the previous volume) filtered BETWEEN `startDate` AND `endDate`, and returning a `ModelAndView` that lists the results.
- `getEmployeesWithinDateRangePage()` and `getDeleteEmployeesByNamePage()` are simple “load the form” handlers, exactly like `loadLoginPage()` from Chapter 32’s Spring Form

section — they should return a `ModelAndView` naming the appropriate JSP and placing an empty command object into the Model for the form to bind against.

- `deleteEmployeesByName(...)` needs a JPQL **bulk delete** (or a lookup-then-remove() sequence, per the entity lifecycle from the previous volume’s Chapter 22) invoked through the service layer, wrapped in a `@Transactional` boundary as covered in Chapter 31 — since a delete is a write operation, it cannot use a read-only transaction.

The provided `@ExceptionHandler` at the bottom is a safety net (Chapter 32’s exception-handling section) that catches anything the trainee’s own implementation might throw while it is still being completed, so the application fails gracefully with a generic error page rather than an unhandled server error.

35.4 Assignment: Spring Core — Constructor Injection with an Optional Dependency

`Assignment_10001_1/2.Springs_Core.Participant` builds a `Customer/Product/Address` bean graph using the exact DI techniques from Chapter 28, but with one deliberately tricky twist embedded directly as a code comment (a “POA”, or Point of Activity) inside the `Customer` bean itself:

```

1 public class Customer {
2     private Integer customerId;
3     private String customerType;
4     private String customerName;
5     private Address address;
6
7     /*
8      * POA 9: Annotate with required annotation to inject product object
9      * and it should be marked as optional dependency injection
10    */
11    private Product product;
12    // constructors, getters, setters ...
13 }

```

The exercise is testing whether the trainee remembers that `@Autowired` — and only `@Autowired`, not the JSR-330 `@Inject` covered in Chapter 30 — supports an explicit `required` attribute for exactly this situation:

```

1 @Autowired(required = false)
2 private Product product;

```

Without `required = false`, Spring would throw a `NoSuchBeanDefinitionException` at container start-up if no `Product` bean happens to be configured; with it, the container simply leaves `product` as null when no matching bean exists, and proceeds normally. The accompanying `CustomerController` exercises the `@Qualifier` technique from Chapter 29 to select a specific `Customer` bean by name when more than one `Customer` bean is configured in the container:

```

1 @RestController
2 public class CustomerController {
3
4     @Autowired
5     @Qualifier("customer1")
6     private Customer customer;
7
8     @GetMapping(value = "customer/controller/getCustomer", produces = MediaType.
9         APPLICATION_JSON_VALUE)
10    public ResponseEntity<Customer> getCustomer() {
11        return new ResponseEntity<Customer>(customer, HttpStatus.OK);
12    }
13 }

```

35.5 Assignment: Building a REST API End to End (POA-Style)

Assignment_10004_1/2.SpringRest_Participant is the most instructive of the assignment set precisely because its controller is handed to the trainee almost entirely as a checklist of comments rather than working code, forcing every technique from Chapters 33 (REST) and 32 (validation) to be applied from scratch:

```

1  /*
2  * POA 3: Write the respective annotation to mark the class as a Controller class
3  *       The annotation should also make this class as a Spring Managed Bean
4  *       The annotation should also make sure that all occurrences of
5  *       @GetMapping/@PostMapping/@PostMapping methods assume @ResponseBody semantics
6  *       by default.
7  */
8  public class CustomerController {
9
10     /*
11     * POA 4: Write the respective annotation to inject the CustomerService
12     *       implementation class object
13     */
14     private CustomerService customerService;
15
16     /*
17     * POA 5: Create a method for GET customer/controller/getCustomerByCustomerType/
18     *       Platinum
19     *       Return JSON of CustomerBean list with HTTP OK if found, else HTTP
20     *       NOT_FOUND.
21     *       Hint: invoke customerService.findByCustomerType(String customerType)
22     */
23
24     /*
25     * POA 6: Create a method for GET customer/controller/findByBillAmountBetween
26     *       /34000--85000
27     *       Return JSON of CustomerBean list within the given bill amount range, else
28     *       NOT_FOUND.
29     *       Hint: invoke customerService.findByBillAmountBetween(double min, double
30     *       max)
31     */
32
33     /*
34     * POA 7: Create a method for POST customer/controller/addCustomer
35     *       Accept and deserialize JSON to CustomerBean; validate it, returning
36     *       validation
37     *       errors with HTTP BAD_REQUEST on failure; otherwise save and return a
38     *       success message.
39     *       Hint: invoke customerService.save(CustomerBean customerBean)
40     */
41
42     /*
43     * POA 8: Create a method for PUT customer/controller/updateCustomer/1001
44     *       Check whether the customer exists (customerService.findById(...)); if so,
45     *       update
46     *       and return HTTP OK, else return HTTP NOT_FOUND with an explanatory
47     *       message.
48     */
49 }

```

Mapping each POA back to the exact technique it is testing:

- **POA 3** is asking for `@RestController` specifically — not `@Controller` alone — because the requirement explicitly names “`@ResponseBody` semantics by default” for every mapped method, which is precisely the shorthand Chapter 33 introduced.
- **POA 4** is asking for `@Autowired` on the `customerService` field, the ordinary DI pattern from Chapter 28 applied at the service-layer boundary.
- **POA 5 and 6** both call for `@GetMapping` with `@PathVariable` to extract the URL seg-

ments (Platinum, and the two bounds 34000/85000), each delegating to a Query-Method-derived (Chapter 31) or explicit `@Query` repository method, then wrapping the result in a `ResponseEntity` with the correct status exactly as the REST CRUD demo in Chapter 33 does for a failed lookup.

- **POA 7** is a `@PostMapping` accepting `@RequestBody @Valid CustomerBean`, with a `BindingResult` parameter checked for validation errors before proceeding (Chapter 32's form-validation pattern, applied here to a JSON payload instead of an HTML form) — validation constraints on a REST payload work identically to validation on an MVC command object, since both go through the same underlying `jakarta.validation` machinery.
- **POA 8** is a `@PutMapping` with `@PathVariable` for the id and `@RequestBody` for the updated data, following the same existence-check-then-update pattern as the REST CRUD demo's `updateEmployee(...)` method in Chapter 33.

The second half of this assignment pairing, `Assignment_10004_2`, goes one step further and introduces a **custom constraint annotation** for validating a `customerType` field against a fixed set of allowed values — built with exactly the `@Constraint(validatedBy = ...)` pattern from Chapter 32:

```

1 package org.accenture.ltt.validator;
2
3 import java.lang.annotation.*;
4 import javax.validation.Constraint;
5 import javax.validation.Payload;
6
7 @Documented
8 @Constraint(validatedBy = CustomerTypeValidatorImpl.class)
9 @Target({ ElementType.METHOD, ElementType.FIELD })
10 @Retention(RetentionPolicy.RUNTIME)
11 public @interface CustomerTypeValidator {
12     String message() default "";
13     Class<?>[] groups() default {};
14     Class<? extends Payload>[] payload() default {};
15 }

```

with `CustomerTypeValidatorImpl` (implementing `ConstraintValidator<CustomerTypeValidator, String>`, following the same shape as `EmployeeNameValidator` in Chapter 32) checking that the submitted `customerType` is one of a fixed, allowed set (e.g. "Platinum", "Gold", "Silver") before the `CustomerBean` is accepted by `addCustomer(...)`.

35.6 Assignment: Spring ORM / Data JPA Microservice Practice

The `SpringORM_DataJPA_MicroserviceAssignment` pair (`Assignment_10002_1/2.SpringORM_Faculty/Participant` and `Assignment_10003_1/2.SpringORM/SpringDataJPA_Participant`) walks the exact same `Customer` domain through the full progression of Chapter 31, in four graduated stages:

1. A plain `@PersistenceContext`-based DAO (`Assignment_10002_1`), identical in shape to `Demo_10001.Spring-With-JPA.Integration.Xml` from Chapter 31, with `CustomerEntity` in place of `EmployeeEntity`.
2. A layer requiring the trainee to add `@Transactional` at the service layer themselves (`Assignment_10002_2`, another POA-commented stub — `CustomerServiceImpl` is handed over with `@Service` and `@Autowired` deliberately omitted, exactly like the REST assignment above), matching Chapter 31's transaction-management demos — including its own `InvalidUpdateOperationException` to signal a business-rule violation (e.g. attempting to update a customer record that does not exist). This exception is worth pausing on precisely because it *extends* `Exception`

rather than `RuntimeException` — a **checked** exception — which means Spring’s *default* `@Transactional` rollback rule (roll back only on unchecked exceptions) would *not* roll back a transaction when this particular exception is thrown; completing this assignment correctly requires explicitly widening the rollback rule with `@Transactional(rollbackFor = InvalidUpdateOperationException.class)`, a detail easy to miss if you assume every custom business exception behaves like the unchecked ones shown in Chapter 31’s demos.

3. A migration from the hand-written DAO to a Spring Data JPA repository interface (`Assignment_10003_1/2`) following the `@RepositoryDefinition` plus DAO-wrapper pattern from Chapter 31’s `EmployeeDAOWrapper` example — here renamed `CustomerDAOWrapper`, converting between `CustomerEntity` and `CustomerBean` with the same `BeanUtils.copyProperties(...)` technique.

Working through this progression by hand — rather than only reading it — is the single most effective way to internalise *why* each layer in Chapter 31 exists: attempting stage 3 without having actually built stage 1 makes the DAO-wrapper’s conversion methods look like unnecessary ceremony, when in fact they are precisely what keeps the JPA `@Entity` type from leaking out of the persistence layer and into the controller.

Viva Voce — Likely Questions & Answers

Q: What is the practical difference between an “Activity” and an “Assignment” in this course’s practice material?

A: An Activity is a mostly-complete project with a small number of specific methods left as stubs for the trainee to fill in during the session, immediately after a technique is demonstrated; an Assignment is typically fuller independent practice, often structured as a numbered checklist of instructions (POAs) guiding the trainee through building a feature from a much larger starting point.

Q: In the Spring Core assignment, why does the Product field on Customer need `@Autowired(required = false)` specifically, rather than the JSR-330 `@Inject`?

A: Because Spring’s own `@Autowired` annotation exposes a `required` attribute for declaring an optional dependency directly; the standard `@Inject` annotation has no equivalent attribute, so expressing “inject this if available, otherwise leave it null” requires Spring’s own annotation rather than the JSR-330 standard one.

Q: Why does completing `getEmployeesDataInDateRange(...)` in the MVC+JPA activity require the `@InitBinder` method that is already provided?

A: Because `@RequestParam("fromDate") Date startDate` needs Spring to know how to convert the incoming request’s plain text date string into a `java.util.Date`; the `@InitBinder` method registers a `CustomDateEditor` with the expected date format specifically so that conversion succeeds.

Q: In the REST assignment’s POA 3, why does the requirement specifically call for `@RestController` rather than `@Controller`?

A: Because the requirement states that every mapped method should assume `@ResponseBody` semantics by default, which is exactly what `@RestController` provides as a shorthand over `@Controller` plus manually adding `@ResponseBody` to every method.

Q: Why does a custom `@Constraint`-based validator (like `CustomerTypeValidator`) work identically whether it’s applied to an MVC form-backing bean or a REST `@RequestBody` payload?

A: Because both paths run through the same underlying `jakarta.validation` machinery triggered by `@Valid` on the controller method’s parameter; the validation constraint itself has no awareness of whether the bean it’s validating arrived via an HTML form submission

or a deserialised JSON request body.

Q: In the Spring ORM microservice assignment, will a plain `@Transactional` method automatically roll back when it throws `InvalidUpdateOperationException`?

A: Not by default. `InvalidUpdateOperationException` extends `Exception` (a checked exception), and Spring's default `@Transactional` rollback rule only rolls back automatically for unchecked (`RuntimeException`-derived) exceptions. To roll back on this specific checked exception, the method must be annotated `@Transactional(rollbackFor = InvalidUpdateOperationException.class)` to explicitly widen the rollback rule.

Java Full Stack Training

Domain-Driven Design through Spring Security

Extended Theory, Concepts & Demo Programs

(Viva / Interview Preparation Edition)

Accenture QuantumCrew Batch — Day-wise Training Reference
(Continuation Volume III)

Covers Weeks 6–7 of the Day-wise Training Plan
Chapters 36–42 — Instructor Demos, Practice Activities & Assignments

Chapter 36

Domain-Driven Design (DDD)

36.1 The Problem DDD Solves

Every layered application built so far in this course — Controller, Service, DAO/Repository, Entity — answers the question “how is the code organised?”. Domain-Driven Design answers a different, earlier question: “does the code actually reflect how the business itself thinks about the problem?”. A payroll system’s code should read like a payroll expert would describe payroll, not like a generic collection of tables and CRUD endpoints. When code and business understanding drift apart, every new feature becomes harder to reason about, because developers and business experts are effectively speaking two different languages about the same system.

Domain-Driven Design, a term and methodology introduced by Eric Evans, is an approach to building software where the **domain** — the specific business problem the software exists to solve (banking, e-commerce, insurance claims, and so on) — is placed at the centre of the design, and the code’s structure is deliberately shaped to mirror the concepts, rules, and language that domain experts already use.

Why This Matters Before Spring Boot and Microservices

The rest of this volume is about *how* to build, deploy, and secure services. DDD is about deciding *what* a service’s boundaries and internal model should even be in the first place — which is exactly the decision that has to be made correctly before “build a microservice for it” becomes a meaningful next step. Chapter 38’s microservice patterns lean directly on DDD’s notion of a bounded context to decide where one service ends and another begins.

36.2 Strategic Design — Ubiquitous Language and Bounded Context

DDD splits into two levels: **strategic design**, which is about the big picture (how the whole system is divided into meaningful parts, and how those parts relate), and **tactical design**, which is about how code *within* one of those parts is actually structured (the topic of the rest of this chapter).

36.2.1 Ubiquitous Language

The **Ubiquitous Language** is a shared vocabulary, built jointly by developers and domain experts, that is used consistently everywhere — in conversations, in documentation, and critically, *in the code itself* (class names, method names, variable names). If a bank’s domain experts say “withdraw”, the code should have a method literally called `withdraw(...)`, not

`processDebitTransaction(...)` or `updateBalance(-amount)`. The Bank demo later in this chapter names its own method `withdraw(Money amount)` for exactly this reason — not as a stylistic choice, but as a direct, deliberate translation of the Ubiquitous Language into code.

36.2.2 Bounded Context

A **Bounded Context** is an explicit boundary within which a particular model, and the Ubiquitous Language that describes it, applies consistently and unambiguously. The same word can mean different things in different parts of a large business — “Customer” in a Sales context might mean a prospective lead with contact details, while “Customer” in a Billing context might mean an account with a payment history and outstanding balance. Rather than forcing one giant, compromise definition of “Customer” to serve every part of the business (which inevitably serves none of them well), DDD says: draw an explicit boundary around each context, let “Customer” mean exactly what that context needs it to mean inside that boundary, and manage the *relationships* between contexts deliberately (via a discipline called **Context Mapping**) rather than accidentally.

Bounded Contexts and Microservices

It is not a coincidence that a well-designed microservice, as covered in Chapter 38, very often maps almost one-to-one onto a Bounded Context. A microservice that spans two different Bounded Contexts tends to accumulate exactly the kind of conflicting, compromise domain model that Bounded Contexts exist to prevent.

36.3 Tactical Design — The Building Blocks

Tactical design is the set of patterns DDD offers for structuring the code *inside* one Bounded Context. Every one of the following patterns appears, by name, in this course’s two DDD demos (Demo1001.BankDDD and Demo1002.ECommerceDDD), each in its own dedicated package (`entity`, `value`, `repository`, `factory`, `event`, `application/service`) — the package structure itself is the tactical design made visible.

36.3.1 Entities — Objects Defined by Identity

An **Entity** is an object defined by a persistent **identity** that continues to matter even as its other attributes change over time. Two `BankAccount` objects with the exact same owner name and balance are still two *different* accounts if they have different account IDs — identity, not attribute values, is what makes an Entity the entity it is.

```
1 public class BankAccount {
2     private String id;           // identity -- this is what makes two accounts "the
    same" or "different"
3     private String ownerName;
4     private Money balance;
5
6     public BankAccount(String ownerName, Money openingBalance) {
7         this.id = UUID.randomUUID().toString();
8         this.ownerName = ownerName;
9         this.balance = openingBalance;
10    }
11
12    public void deposit(Money amount) {
13        this.balance = this.balance.add(amount);
14        System.out.println("Deposited: " + amount + " | New Balance: " + balance);
15    }
16
17    public void withdraw(Money amount) {
18        if (amount.getAmount() > balance.getAmount()) {
19            throw new IllegalArgumentException("Insufficient balance!");
20        }
21    }
22 }
```

```

20     }
21     this.balance = this.balance.subtract(amount);
22     System.out.println("Withdrew: " + amount + " | New Balance: " + balance);
23 }
24 // getId(), getOwnerName(), getBalance(), toString() ...
25 }

```

Notice that `withdraw(...)` contains an actual business rule (insufficient balance cannot be withdrawn) *inside the Entity itself*, rather than leaving that rule to be re-implemented (and potentially re-implemented inconsistently) by every caller. This is a deliberately different habit from the plain JPA `@Entity` classes seen earlier in this course, which were largely just data holders with getters and setters — a DDD Entity is expected to **protect its own invariants** (its own business rules) rather than trusting external code to keep it in a valid state.

36.3.2 Value Objects — Objects Defined by Their Values

A **Value Object** is the mirror image of an Entity: it has no identity of its own, and two Value Objects with the same attribute values *are*, for every practical purpose, the same value. **Money** is the textbook example, and it appears, independently implemented, in both of this chapter's demos:

```

1 public class Money {
2     private final double amount;
3
4     public Money(double amount) {
5         if (amount < 0) throw new IllegalArgumentException("Amount cannot be negative");
6         ;
7         this.amount = amount;
8     }
9
10    public Money add(Money other) {
11        return new Money(this.amount + other.amount);
12    }
13
14    public Money subtract(Money other) {
15        return new Money(this.amount - other.amount);
16    }
17
18    @Override
19    public boolean equals(Object o) {
20        if (this == o) return true;
21        if (!(o instanceof Money)) return false;
22        Money money = (Money) o;
23        return Double.compare(money.amount, amount) == 0;
24    }
25
26    @Override
27    public int hashCode() { return Objects.hash(amount); }
28 }

```

Three details are worth calling out explicitly, because each is a deliberate DDD habit, not an accident:

- **Immutability** — `amount` is `final`, and `add(...)/subtract(...)` both *return a new Money* rather than mutating `this`. A Value Object that could change in place would risk two different parts of the code silently sharing (and corrupting) what looks like one immutable value.
- **Self-validation** — the constructor itself rejects a negative amount, so it is simply impossible to construct an invalid **Money** anywhere in the system; validity is guaranteed by construction, not by hoping every caller remembers to check.
- **Value-based equality** — `equals(...)/hashCode()` compare the underlying `amount`, not

object reference, so two separately-constructed `Money` objects holding 500 are equal, consistent with a Value Object having no identity of its own.

The E-Commerce demo's `Money` makes the same choices with a `BigDecimal` instead of a `double` (a detail worth noting on its own: DDD demos deliberately avoid floating-point binary types for real money, because binary floating point cannot represent many decimal fractions exactly, which is unacceptable for a value that must add up correctly to the last unit of currency) and additionally overrides `toString()` to render with a currency symbol — keeping the formatting concern inside the Value Object that owns the concept, rather than scattered across every place that happens to print a price.

36.3.3 Identifier Value Objects

The E-Commerce demo goes one step further and wraps even simple identifiers in their own Value Objects, rather than passing raw `Strings` or `UUIDs` around:

```
1 public final class OrderId {
2     private final String id;
3     private OrderId(String id) { this.id = id; }
4     public static OrderId generate() { return new OrderId(UUID.randomUUID().toString())
5         ; }
6     @Override public String toString() { return id; }
7 }
```

`ProductId` is built identically. This might look like unnecessary ceremony for what is ultimately just a wrapped string, but it buys real type safety: a method signature `findById(OrderId id)` makes it *impossible* to accidentally pass a `ProductId` where an `OrderId` was expected — a mistake the compiler would happily allow if both were just plain `Strings`, since a `String` carries no information about which kind of identifier it's supposed to be.

36.3.4 Aggregates and Aggregate Roots

An **Aggregate** is a cluster of related Entities and Value Objects that are treated as one single, consistent unit for the purposes of data changes — and every Aggregate has exactly one **Aggregate Root**, the single Entity through which all access to the rest of the cluster must go. In the E-Commerce demo, `Order` is the Aggregate Root of an aggregate that also includes its `Product` line items:

```
1 public class Order {
2     private final OrderId id;
3     private final List<Product> products = new ArrayList<>();
4     private Money total = new Money(new java.math.BigDecimal("0"));
5
6     public Order(OrderId id) { this.id = id; }
7
8     public void addProduct(Product product) {
9         products.add(product);
10        total = total.add(product.getPrice());
11    }
12
13    public Money getTotal() { return total; }
14    public OrderId getId() { return id; }
15 }
```

Nothing outside `Order` ever adds a `Product` directly to the `products` list or recalculates `total` by hand — `addProduct(...)` is the *only* door in, and it keeps `total` correctly in sync with the line items as an invariant the Aggregate Root itself guarantees. This is precisely why the Repository pattern below only ever exposes `save(Order)/findById(OrderId)` — an Aggregate is always loaded and saved as one whole unit through its root, never in fragments.

36.3.5 Repository — Persistence Behind a Domain-Shaped Interface

The **Repository** pattern here plays a very similar role to the Spring Data JPA repositories from the previous volume, but framed in DDD terms: it gives the domain model a collection-like interface for retrieving and storing Aggregates, while hiding *how* that storage actually happens behind that interface.

```
1 public interface BankAccountRepository {
2     void save(BankAccount account);
3     BankAccount findById(String id);
4     List<BankAccount> findAll();
5 }

1 public class InMemoryBankAccountRepository implements BankAccountRepository {
2     private final Map<String, BankAccount> store = new HashMap<>();
3
4     @Override
5     public void save(BankAccount account) { store.put(account.getId(), account); }
6
7     @Override
8     public BankAccount findById(String id) { return store.get(id); }
9
10    @Override
11    public List<BankAccount> findAll() { return new ArrayList<>(store.values()); }
12 }
```

The interface (`BankAccountRepository`) lives in a `repository` package alongside the domain model; its implementation (`InMemoryBankAccountRepository`) lives in a separate `infrastructure` package. This separation is the DDD name for exactly the same dependency-inversion habit that motivated declaring a plain DAO *interface* back in Chapter 31's Spring Data JPA chapter: the domain layer depends only on the abstraction, and knows nothing about whether the real implementation is an in-memory map (as in this teaching demo), a relational database via JPA, or anything else.

36.3.6 Factory — Encapsulating Complex Construction

A **Factory** takes responsibility for constructing a valid Aggregate (or Entity) when that construction is non-trivial enough that scattering `new Order(...)` calls throughout the codebase would risk inconsistently-built objects:

```
1 public class OrderFactory {
2     public static Order createNewOrder() {
3         return new Order(OrderId.generate());
4     }
5 }
```

Even in this simple demo (where a Factory doing anything more than one line might look like overkill), the value of the pattern is clear: every part of the application that needs a brand-new `Order` calls `OrderFactory.createNewOrder()` rather than reaching for `new Order(OrderId.generate())` directly, so if the rules for what counts as a properly-initialised new order ever grow more elaborate (a default status, a creation timestamp, an associated empty shopping cart), there is exactly one place to change.

36.3.7 Domain Events — Recording Things That Happened

A **Domain Event** is an object representing something meaningful that happened in the domain, that other parts of the system might care to react to — a fact about the past, not a request to do something. The Bank demo's `TransactionEvent` is a direct example:

```
1 public class TransactionEvent {
2     private final String type;
```

```

3     private final Money amount;
4     private final BankAccount account;
5     private final LocalDateTime occurredOn;
6
7     public TransactionEvent(String type, Money amount, BankAccount account) {
8         this.type = type;
9         this.amount = amount;
10        this.account = account;
11        this.occurredOn = LocalDateTime.now();
12    }
13    // getters, toString() ...
14 }

```

Every field here is `final`, and `occurredOn` is stamped once, at construction — exactly like a Value Object, a Domain Event describes an immutable fact and is never edited after it is created (you cannot “undo” something that already happened; you can only, at most, record a second event that reverses its effect). In a full production system, a Domain Event like this would typically be published onto a message broker or an in-process event bus so that other parts of the system (an audit log, a fraud-detection service, a notification service) can react to it *without* the `BankService` that raised the event needing to know anything about who is listening — a clean decoupling point that becomes especially valuable once a monolith is later split into the kind of independent microservices covered in Chapter 38.

36.3.8 Application Service — Orchestrating a Use Case

An **Application Service** (sometimes just called a Service, as in the E-Commerce demo’s `PaymentService`) coordinates a single use case: it talks to Repositories to load Aggregates, calls methods on the domain model to actually change something, and returns a result — but it deliberately contains *no business rules of its own*. Every business rule belongs inside an Entity, Value Object, or a dedicated Domain Service instead.

```

1 public class BankService {
2     private final BankAccountRepository repository;
3
4     public BankService(BankAccountRepository repository) {
5         this.repository = repository;
6     }
7
8     public TransactionEvent openAccount(BankAccount account) {
9         repository.save(account);
10        System.out.println("Account opened: " + account);
11        return new TransactionEvent("ACCOUNT_OPENED", account.getBalance(), account);
12    }
13
14    public TransactionEvent deposit(BankAccount account, Money amount) {
15        account.deposit(amount);           // the actual rule lives inside BankAccount,
16        not here
17        repository.save(account);
18        return new TransactionEvent("DEPOSIT", amount, account);
19    }
20
21    public TransactionEvent withdraw(BankAccount account, Money amount) {
22        account.withdraw(amount);          // ditto -- including the insufficient-
23        balance check
24        repository.save(account);
25        return new TransactionEvent("WITHDRAWAL", amount, account);
26    }
27 }

```

Notice that `deposit(...)` does not itself compute a new balance — it delegates to `account.deposit(amount)`, which is where that logic actually lives (Section 36.4 above). `BankService` is thin *on purpose*: it is the seam between the outside world (in the demo, a plain `main` method; in a full application, a Spring `@RestController`) and the domain model, and keeping it thin is what stops business logic from leaking out into the layer that should only be orchestrating it.

36.4 Seeing All the Pieces Work Together

The Bank demo’s driver class exercises every pattern above in one linear script, which is worth reading start to finish precisely because nothing here is Spring-specific — this is DDD’s tactical design entirely on its own, before any framework is involved:

```
1 public class BankApp {  
2     public static void main(String[] args) {  
3         BankService service = new BankService(new InMemoryBankAccountRepository());  
4  
5         BankAccount account = new BankAccount("Harsha", new Money(500));  
6         TransactionEvent openEvent = service.openAccount(account);  
7         System.out.println(openEvent);  
8  
9         TransactionEvent depositEvent = service.deposit(account, new Money(200));  
10        System.out.println(depositEvent);  
11  
12        TransactionEvent withdrawEvent = service.withdraw(account, new Money(100));  
13        System.out.println(withdrawEvent);  
14  
15        System.out.println("Final Account: " + account);  
16    }  
17 }
```

Reading this against the vocabulary of this chapter: **BankAccount** is the Entity/Aggregate Root; **Money** is the Value Object protecting the notion of an amount; **InMemoryBankAccountRepository** is the Repository implementation, injected into **BankService** (plain constructor injection, exactly as in Chapter 28, with no Spring container involved at all here); **BankService** is the Application Service orchestrating three use cases; and each use case returns a **TransactionEvent** Domain Event recording what just happened. The E-Commerce demo’s **ECommerceApp** follows the identical shape with **OrderFactory** additionally in play for constructing the Aggregate Root.

36.5 Context Mapping — How Bounded Contexts Actually Relate

Section 36.2.2 introduced the Bounded Context and asserted that the relationships *between* contexts should be managed deliberately rather than accidentally. **Context Mapping** is the discipline that names and structures exactly what those relationships can look like, and it matters because two teams owning two different Bounded Contexts almost always have *some* dependency on each other’s model, even if each context’s internal model stays cleanly separate. Recognising which of the following patterns actually describes a given relationship is what turns a vague “these two services need to talk to each other” into a concrete, honest picture of who depends on whom, and how much power each side has to demand changes from the other.

Context Mapping Pattern	What It Describes
Shared Kernel	Two teams deliberately share a small, jointly-owned subset of their models (e.g. a common <code>Money Value Object</code>) — powerful for consistency, but it requires close, continuous coordination between both teams, since a change to the shared part affects both sides at once.
Customer/Supplier	One context (the supplier) provides something another context (the customer) depends on, and the supplier team explicitly takes the customer team's needs into account when planning changes — an acknowledged, negotiated upstream/downstream relationship.
Conformist	A downstream context has no real influence over an upstream context's model (perhaps it belongs to a different organisation, or a legacy system nobody can change) and simply <i>conforms</i> to whatever model the upstream side provides, translating nothing.
Anti-Corruption Layer (ACL)	Instead of conforming, the downstream context builds a deliberate translation layer that converts the upstream system's model into its <i>own</i> clean, locally-meaningful model at the boundary — protecting the downstream context's own design from being distorted by an external system's inconsistencies or awkward legacy shape.
Open Host Service	An upstream context publishes a well-defined, standardised protocol or API (rather than a bespoke integration per consumer) that any number of downstream contexts can integrate against uniformly.

Why the Anti-Corruption Layer Matters for This Course's Microservices

Every `RestTemplate`/Feign client built in Chapters 37–39 that converts a raw `LinkedHashMap` or a remote DTO into a locally-meaningful `Employee` object before the rest of the consuming application ever touches it is, in miniature, already practising an Anti-Corruption Layer: the consumer's own domain model never has to know or care about the exact shape the producer happens to expose — a boundary conversion step absorbs that difference once, at the edge, rather than letting the producer's shape leak inward and quietly become the consumer's own model by default.

Choosing the wrong pattern for a real relationship — most commonly, silently becoming a **Conformist** to an upstream system by accident, simply because nobody built a translation layer, rather than as a deliberate choice — is one of the most common ways a supposedly well-bounded microservice architecture ends up quietly re-coupled anyway: the downstream service's own internal model starts mirroring the upstream system's quirks and legacy naming, and the clean boundary Chapter 36 opened with erodes over time.

It is worth being explicit about how the patterns in this chapter map onto everything already built in this course, because the mapping is close but not identical:

DDD Concept	Closest Analogue Already Seen in This Course
Entity	A JPA <code>@Entity</code> class — but a DDD Entity additionally enforces its own business rules in its own methods, rather than being a plain data holder
Value Object	Not directly present in the earlier JPA chapters (which modelled everything as <code>@Entity</code> rows); closest in spirit to an immutable Java class with value-based <code>equals()</code> / <code>hashCode()</code>
Repository	A Spring Data JPA repository interface (Chapter 31) — the same dependency-inversion idea, expressed independently of any framework here
Application Service	A Spring <code>@Service</code> class (Chapter 29 onward) — again, the same orchestration role, without a Spring container in this chapter's plain-Java demos
Domain Event	Conceptually similar to the <code>TransactionEvent</code> pattern used again, almost by name, in the Spring Batch listener demos of the previous volume — an immutable record of something that already happened

Recognising this mapping is the point: DDD is not a competing technology to Spring, but a way of thinking about where business logic should live *before* it gets distributed across Spring's Controller/Service/Repository layers — so that, once Spring Boot enters the picture in the next chapter, the `@Service` and `@Repository` beans have a genuinely well-designed domain model underneath them, rather than becoming a thin label wrapped around what is really still just a database table.

Viva Voce — Likely Questions & Answers

Q: What is the core problem Domain-Driven Design tries to solve?

A: It addresses the gap between how a business actually thinks and talks about its problem domain, and how that domain ends up represented in code — ensuring the software's structure and vocabulary genuinely reflect business understanding, rather than drifting into a generic, business-agnostic data model.

Q: What is the Ubiquitous Language, and why must it appear in the code itself, not just in documentation?

A: It is a shared vocabulary built jointly by developers and domain experts and used consistently everywhere. It must appear in the code (class and method names) because if the code uses different terms than the business does, developers and domain experts are effectively no longer talking about the same model, defeating the language's purpose.

Q: What is a Bounded Context, and why can the same term (e.g. "Customer") legitimately mean different things in different parts of a large system?

A: A Bounded Context is an explicit boundary within which one particular model and vocabulary apply consistently. The same real-world term can carry different meaning and different data in different business contexts (e.g. Sales vs Billing); rather than forcing one compromise definition everywhere, DDD scopes each definition to its own Bounded Context and manages relationships between contexts deliberately.

Q: What is the fundamental difference between an Entity and a Value Object?

A: An Entity is defined by a persistent identity that matters independently of its current attribute values (two Entities with identical attributes but different identities are still different); a Value Object has no identity of its own and is defined entirely by its attribute values (two Value Objects with identical values are considered the same value).

Q: Why are Value Objects typically implemented as immutable, with operations

like `add()` returning a new instance rather than mutating the existing one?

A: Because a Value Object represents a value, not a thing with a life story; if it could change in place, two parts of the code that appear to share the same value could end up silently seeing different values as one of them mutates it, which contradicts the idea of it being a fixed value in the first place.

Q: What is an Aggregate Root, and what rule does it enforce?

A: The single Entity through which all access to the rest of its Aggregate's Entities and Value Objects must go. It enforces that the Aggregate is always modified as one consistent unit — for example, `Order.addProduct(...)` is the only way to add a line item, which lets `Order` keep its own total invariant correctly in sync rather than trusting external code to update both the item list and the total separately.

Q: Why might a DDD codebase wrap even a simple identifier like an order ID in its own Value Object (e.g. `OrderId`) instead of just using a `String`?

A: Doing so provides compile-time type safety: a method expecting an `OrderId` cannot accidentally be passed a `ProductId` or any other unrelated `String`, an error the compiler would not catch if both were represented as plain `Strings`.

Q: What is the difference between a Repository and a Factory in DDD?

A: A Repository is responsible for retrieving and persisting already-existing Aggregates, giving the domain model a collection-like interface over storage; a Factory is responsible for constructing new, valid instances of an Aggregate or Entity in the first place, particularly when that construction involves more than a trivial constructor call.

Q: Why does a Domain Event's data (e.g. `TransactionEvent`) get created once and never modified afterward?

A: Because a Domain Event represents a fact about something that has already happened; like history itself, it cannot be edited after the fact — only a new, subsequent event can be raised to record a correction or reversal.

Q: Why should an Application Service (like `BankService`) avoid containing business rules itself?

A: Because its role is to orchestrate a use case — loading Aggregates via a Repository and invoking their own methods — while the actual business rules (such as insufficient-balance validation) belong inside the Entity or Value Object that owns that invariant; keeping the Application Service thin prevents business logic from leaking out of the domain model into the orchestration layer.

Q: Why does a well-designed microservice architecture (covered in the next chapters) tend to align closely with DDD's Bounded Contexts?

A: Because a microservice that spans two different Bounded Contexts inherits the same problem Bounded Contexts are meant to solve — conflicting or compromise definitions of the same term forced to coexist — whereas a microservice scoped to a single Bounded Context can maintain one consistent model and Ubiquitous Language throughout.

Q: What is the difference between the Conformist and Anti-Corruption Layer context mapping patterns?

A: A Conformist downstream context has no real influence over an upstream context's model and simply adopts it as-is, translating nothing; an Anti-Corruption Layer instead builds a deliberate translation boundary that converts the upstream model into the downstream context's own clean, locally-meaningful model, protecting its design from being distorted by the external system's shape.

Q: Why is silently becoming a Conformist to an upstream system often an accident rather than a deliberate choice, and why does that matter?

A: It typically happens simply because no one built a translation layer at the integration boundary, not because a team consciously decided conforming was the right relationship; over time the downstream service's own model starts mirroring the upstream system's quirks and legacy naming, quietly eroding the clean separation Bounded Contexts were meant to establish.

Chapter 37

Spring Boot Fundamentals & Microservices

37.1 Monolithic vs Microservice Architecture

Every application built across the previous two volumes of this course — however many layers it had internally — was ultimately deployed as *one single unit*: one WAR or JAR, one running process, one database. This is a **monolithic architecture**. It is not a mistake or an outdated approach; for small-to-medium applications, it is often the simplest, fastest way to build and deploy something that works, and every demo in this course has deliberately used it to keep the focus on one concept at a time.

A monolith starts to strain as a system grows, in a few specific, recognisable ways:

- **Scaling is all-or-nothing.** If only the reporting module is under heavy load, the entire application — including parts that are barely used — still has to be scaled (more instances of the whole thing) to handle it, wasting resources.
- **Deployments are all-or-nothing.** A one-line fix to an unrelated module still requires rebuilding, retesting, and redeploying the entire application.
- **Team boundaries blur.** Multiple teams working in one shared codebase and one shared deployment unit inevitably step on each other's changes and release schedules.
- **Technology choice is locked in.** The whole application shares one language, one framework version, one set of library versions — a genuinely better tool for one specific piece of the system cannot be adopted without touching everything else.

A **microservice architecture** addresses this by splitting one large application into a set of small, independently deployable services, each responsible for one well-defined piece of business capability, each with its own codebase, its own deployment lifecycle, and often its own database — communicating with each other over the network (typically HTTP/REST, as built throughout this course, or messaging). Chapter 36's Bounded Context is precisely the tool used to decide *where* those service boundaries should actually fall: a microservice that respects a Bounded Context boundary tends to stay cohesive and independently meaningful; one that ignores it tends to need constant, tightly-coupled coordination with whatever other service ended up holding the other half of that same context's model.

Microservices are not free of cost, however — they trade the problems above for a new set of problems, which is exactly why Chapters 38 and 39 exist: now that there are *many* independently-deployed services instead of one, something has to handle how they **find each other** (service discovery), what happens when **one of them is slow or down** (resilience patterns), and how

anyone can **observe** what is actually happening across all of them at once (monitoring and distributed tracing).

37.2 Why Spring Boot

Every Spring application built in the previous two volumes needed a fair amount of manual configuration to get off the ground: an explicit `DispatcherServlet` registration, an explicit `DataSource` bean, an explicit `EntityManagerFactory` bean, an explicit embedded-or-external servlet container decision. None of that configuration is wrong, but nearly all of it is *the same, boilerplate configuration* that almost every Spring web application needs. **Spring Boot** exists to remove exactly that repetition, through two central ideas:

- **Auto-configuration** — Spring Boot inspects what is on the classpath (is Spring MVC present? is a JDBC driver present? is Spring Data JPA present?) and automatically registers and wires up the beans a typical application using those libraries would need, *unless* the developer has already defined their own bean of that type, in which case Boot's default backs off and the developer's bean is used instead.
- **An embedded server** — rather than building a WAR and deploying it into an externally-installed Tomcat (the traditional deployment model), a Spring Boot application *contains its own* embedded servlet container (Tomcat by default, or Jetty/Undertow as swapped-in alternatives) and can be run directly as an executable JAR with `java -jar`, with nothing else to install first.

37.3 How Auto-Configuration Actually Decides What to Register

Section 37.2 described auto-configuration at the level of “Spring Boot inspects the classpath and wires up sensible defaults.” It is worth understanding *how* that inspection actually happens, because the mechanism is a small, learnable set of conditional annotations, not a mysterious black box.

Every auto-configuration class is an ordinary `@Configuration` class, discovered at start-up via an entry in a special file on the classpath (historically `spring.factories`; in current Spring Boot versions, `META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports`) — this is how Spring Boot knows *which* auto-configuration classes even exist to consider, without a developer ever manually registering them. Each such class then decides, for itself, whether it should actually contribute any beans, using a family of **conditional annotations**:

Conditional Annotation	Registers Its Beans Only If...
<code>@ConditionalOnClass</code>	A specific class is present on the classpath (e.g. JPA auto-configuration only activates if <code>EntityManager</code> is actually on the classpath at all)
<code>@ConditionalOnMissingBean</code>	No bean of that type has <i>already</i> been defined by the developer's own configuration — this is the exact mechanism behind “Boot's default backs off if you supply your own bean” from Section 37.2
<code>@ConditionalOnProperty</code>	A specific property is set (and, optionally, set to a specific value) in the application's configuration
<code>@ConditionalOnWebApplication</code>	The application is a web application (as opposed to a plain command-line application with no embedded server at all)

Reading these against the demos already built in this chapter makes the mechanism concrete: the embedded-server auto-configuration behind Section 37.3.2’s Tomcat-to-Jetty swap is guarded by `@ConditionalOnClass` checks against Tomcat’s and Jetty’s respective server classes — excluding the Tomcat starter dependency (and therefore its classes) from the classpath is precisely what makes that `@ConditionalOnClass` check for Tomcat fail and back off, letting the Jetty auto-configuration’s own equivalent check succeed instead. Similarly, the JPA/DataSource auto-configuration behind Section 37.6’s `application.properties`-only database setup is guarded by `@ConditionalOnClass` against a JDBC driver and `@ConditionalOnMissingBean` against `DataSource` itself — which is exactly why a developer’s own hand-written `@Bean public DataSource dataSource() {...}`, exactly as built by hand in the previous volume’s Chapter 31, would silently and correctly *replace* Boot’s auto-configured one, with no explicit “turn off auto-configuration” step needed at all.

37.3.1 Starters — Curated Dependency Bundles, Not Auto-Configuration Themselves

It is worth being precise about a common point of confusion: a Spring Boot **starter** (`spring-boot-starter-web`, `spring-boot-starter-data-jpa`, and so on, used throughout every `pom.xml` in this course) is *not itself* auto-configuration — a starter is simply a curated Maven dependency bundle, pulling in a consistent, tested-together set of transitive dependencies for one particular concern (Spring MVC plus an embedded Tomcat plus Jackson, for `spring-boot-starter-web`). Auto-configuration is the *separate* mechanism, described above, that then *reacts* to whichever of those dependencies actually end up on the classpath. Adding `spring-boot-starter-web` does two genuinely different things at once, worth keeping conceptually distinct: it pulls in the libraries (the starter’s job), and it thereby causes several `@ConditionalOnClass`-guarded auto-configuration classes to activate because their required classes are now present (auto-configuration’s job) — the starter is the cause, auto-configuration reacting to it is the effect, and neither one is the other.

Every Spring Boot application in this course’s demos begins with the identical three lines, and it is worth understanding exactly what each part of them does, because everything else in this chapter builds on top of this:

```
1 package com.accenture.ltt;
2
3 import org.springframework.boot.SpringApplication;
4 import org.springframework.boot.autoconfigure.SpringBootApplication;
5
6 @SpringBootApplication
7 public class Application {
8     public static void main(String[] args) {
9         SpringApplication.run(Application.class, args);
10    }
11 }
12
13 // @SpringBootApplication
14 // @configuration
15 // @componentscan
16 // @Enableautoconfiguration
```

The commented-out lines at the bottom of the demo file are the whole point: `@SpringBootApplication` is a **single, convenience meta-annotation that bundles together three separate annotations**:

Bundled Annotation	Role
@Configuration	Marks the class as a source of bean definitions, exactly as in Chapter 29’s Java-config demos
@ComponentScan	Enables component scanning (Chapter 29) starting from this class’s own package downward, so every @Component/@Service/@Repository/@Controller under it is auto-detected
@EnableAutoConfiguration	Turns on Spring Boot’s auto-configuration machinery itself — the classpath-inspection behaviour described above

`SpringApplication.run(Application.class, args)` is what actually bootstraps the whole process: it creates the `ApplicationContext`, refreshes it (running every `@Configuration` class, every auto-configuration class, every component-scanned bean), and — because Spring MVC is on the classpath in a typical web project — starts the embedded servlet container listening for requests, all in one call.

37.3.2 Project Structure and the Banner

Every Spring Boot project in this course’s demos follows the identical Maven layout from Chapter 27 (`src/main/java`, `src/main/resources`, and so on), with one small addition worth knowing: `application.properties` (or, as shown later in this chapter, `application.yml`) sits directly under `src/main/resources` and is picked up automatically at start-up without any explicit `@PropertySource` needed. `Demo10014.Spring_Boot_Project_Structure_Banner_Mode_Off` shows one of Spring Boot’s smaller but very visible defaults: the ASCII-art Spring Boot banner printed to the console at start-up, which can be turned off (useful for quieter logs in a production or CI environment):

```
1 SpringApplication app = new SpringApplication(Application.class);
2 app.setBannerMode(Banner.Mode.OFF);
3 app.run(args);
```

or, more concisely, via the fluent `SpringApplicationBuilder`, or simply by setting `spring.main.banner-mode` in `application.properties` — three different ways to reach the identical outcome, echoing the same “more than one valid path to the same configuration” theme already seen throughout this course.

37.3.3 Choosing the Embedded Server — Tomcat vs Jetty

Spring Boot defaults to an embedded **Tomcat** server, pulled in transitively by `spring-boot-starter-web`. `Demo10015.Spring_Boot_Jetty_Server_Configuration` shows how to swap that default out for **Jetty** instead, entirely through Maven dependency management — no Java code changes at all:

```
1 <dependency>
2   <groupId>org.springframework.boot</groupId>
3   <artifactId>spring-boot-starter-web</artifactId>
4   <exclusions>
5     <exclusion>
6       <groupId>org.springframework.boot</groupId>
7       <artifactId>spring-boot-starter-tomcat</artifactId>
8     </exclusion>
9   </exclusions>
10 </dependency>
11 <dependency>
12   <groupId>org.springframework.boot</groupId>
13   <artifactId>spring-boot-starter-jetty</artifactId>
14 </dependency>
```

This is auto-configuration’s classpath-inspection behaviour made concrete: Spring Boot’s embedded-server auto-configuration checks the classpath for *which* servlet-container starter is present and configures accordingly, so *excluding* Tomcat’s transitive dependency and *including* Jetty’s is the entire change needed — exactly the “developer’s own choice silently overrides Boot’s default” principle from Section 37.2 applied to infrastructure, not just to beans.

37.4 Content Negotiation

A single REST endpoint can, in principle, serve more than one representation format (JSON, XML) to different clients, as already shown by the JAXB-wrapped XML example in the previous volume’s REST chapter. **Content negotiation** is the mechanism Spring uses to decide, for a given incoming request, which representation the client actually wants. This course’s three content-negotiation demos each drive that decision by a different signal:

Strategy	How the Client Signals Its Preference
Extension-based (Demo10019-1)	The URL itself ends in an extension, e.g. <code>/emp/controller/getDetails.json</code> vs <code>.xml</code>
Parameter-based (Demo10019-2)	A query parameter, e.g. <code>/emp/controller/getDetails?format=json</code>
Header-based (Demo10019-3)	The standard HTTP Accept request header (the technically “correct”, REST-idiomatic approach)

```

1 @Configuration
2 public class SpringConfig implements WebMvcConfigurer {
3     @SuppressWarnings("deprecation")
4     public void configureContentNegotiation(ContentNegotiationConfigurer configurer) {
5         configurer.favorPathExtension(true)
6                     .favorParameter(false)
7                     .ignoreAcceptHeader(true)
8                     .defaultContentType(MediaType.APPLICATION_JSON);
9     }
10 }
```

This is the extension-based configuration: `favorPathExtension(true)` makes the URL’s own extension take priority, `favorParameter(false)` disables the query-parameter strategy for this configuration, `ignoreAcceptHeader(true)` tells Spring to disregard the Accept header even if the client sends one, and `defaultContentType(...)` sets what to fall back to if none of the above resolves a format at all. The parameter-based and header-based demos flip exactly these same four settings to make their respective signal the one that actually decides.

37.5 Spring Data JPA in a Spring Boot Project

Demo10020.Spring_Boot.Spring_JPA_Repository repeats the exact `@RepositoryDefinition`-plus-DAO-wrapper pattern from the previous volume’s Chapter 31, but now inside a Spring Boot project where the `DataSource` and `EntityManagerFactory` beans are no longer hand-configured at all — Spring Boot’s JPA auto-configuration builds them automatically purely from a handful of properties:

```

1 server.port = 8090
2
3 # Database
4 spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
5 spring.datasource.url=jdbc:mysql://localhost:3306/springbootdb
6 spring.datasource.username=root
7 spring.datasource.password=root
8
9 spring.jpa.hibernate.ddl-auto = update
10 spring.jpa.show-sql = true
```

```

11 spring.jpa.hibernate.naming.physical-strategy=org.hibernate.boot.model.naming.
    PhysicalNamingStrategyStandardImpl
12 spring.jpa.properties.hibernate.dialect = org.hibernate.dialect.MySQL8Dialect

```

Compare this to the previous volume’s `SpringDBConfig @Configuration` class, which built a `LocalContainerEntityManagerFactoryBean` and a `JpaTransactionManager` by hand, field by field — every one of those beans still exists in a Spring Boot application, but Boot’s auto-configuration builds them silently from properties like the ones above, and only if the developer’s own `@Configuration` class doesn’t already supply one itself.

37.5.1 The Physical Naming Strategy

`Demo10021.Spring_Boot.Spring_JPA_Repository_NamingStrategy` highlights a detail easy to be surprised by: by default, modern Hibernate (via Spring Boot’s auto-configuration) converts Java’s camelCase entity and field names into `snake_case` table and column names automatically (`employeeName` becomes a column named `employee_name`). Setting `spring.jpa.hibernate.naming.physical` to `PhysicalNamingStrategyStandardImpl` (as above) disables that automatic conversion and instead uses the identifiers exactly as written in the `@Entity/@Column` annotations — worth knowing explicitly, because “why did Hibernate create a table with different column names than what I wrote in my entity?” is one of the most common points of confusion for a beginner moving from raw JPA (previous volume) into a Spring Boot project.

37.6 Hibernate Bean Validation in Spring Boot

`Demo10016.Spring_Boot.Hibernate_Validation` applies the standard `jakarta.validation`/Hibernate Validator annotations from the previous volume’s Chapter 32 directly to a Spring Boot REST payload bean:

```

1 public class Employee {
2     @NotEmpty(message="Employee Name is required Field")
3     // cannot use @NotNull for String, since a missing form field still arrives as ""
4     private String employeeName;
5
6     private Integer employeeId;
7
8     @Range(min=10, max=100000, message="Salary should be in Range of 1000 and 100000")
9     private double salary;
10
11     @NotNull(message="Department is required Field")
12     private Integer departmentCode;
13     // ...
14 }

```

The comment on `@NotEmpty` is worth pausing on, because it is a genuinely common mistake: `@NotNull` only rejects a *missing* value; an empty string `""` still satisfies `@NotNull`, since `""` is not `null`. `@NotEmpty` (and, more strictly still, `@NotBlank`, which additionally rejects a whitespace-only string) is what is actually needed to guarantee a `String` field has meaningful content.

37.6.1 Externalising Validation Messages, Including Locale-Specific Ones

`Demo10017-1.Spring_Boot.Hibernate_Validation_Properties_File` and `Demo10017-2...LocaleSpecific` move validation failure messages out of the annotation itself and into an externalised properties file, following the exact key-naming convention Hibernate Validator expects (`ConstraintAnnotation.beanName`

```

1 NotEmpty.employee.employeeName=EmployeeName should not be null
2 Range.employee.salary=Employee salary should be in range 1000 and 100000
3 NotEmpty.employee.dept=Employee Department cannot be null
4 EmployeeNameValidator.employee.employeeName=Employee name should contain firstName[,
    middleName],lastName

```

Once this file is named `ValidationMessages.properties` and placed on the classpath, Hibernate Validator picks it up automatically as the default message bundle for every constraint annotation in the application — and, in the locale-specific variant, additional files named `ValidationMessages_fr.properties`, `ValidationMessages_de.properties`, and so on let the exact same validation rule report its failure message in whichever language matches the request's `Locale`, without a single line of Java code changing.

37.6.2 A Fully Custom Validation Constraint, End to End

`Demo10018.Spring_Boot_Hibernate_Custom_Validation_Properties_File` shows the same custom-`@Constraint` pattern from the previous volume's Spring MVC chapter, this time deliberately paired with an externalised message:

```
1 @Documented
2 @Constraint(validatedBy = EmployeeFieldValidator.class)
3 @Target({ ElementType.FIELD })
4 @Retention(RetentionPolicy.RUNTIME)
5 public @interface EmployeeFieldValidationAnnotation {
6     String message() default "Value should have 2 or 3 words";
7     Class<?>[] groups() default {};
8     Class<? extends Payload>[] payload() default {};
9 }
```

```
1 public class EmployeeFieldValidator
2     implements ConstraintValidator<EmployeeFieldValidationAnnotation, String> {
3
4     @Override
5     public void initialize(EmployeeFieldValidationAnnotation arg0) { }
6
7     @Override
8     public boolean isValid(String string, ConstraintValidatorContext arg1) {
9         if (string == null) { return true; } // let @NotNull/@NotEmpty own the null/
10            blank case
11         int wordCount = string.split(" ").length;
12         return wordCount == 2 || wordCount == 3;
13     }
14 }
```

Notice `isValid(...)` deliberately returns `true` for a `null` value rather than `false`. This is a standard, easy-to-miss Bean Validation convention: a single field is very often annotated with *several* constraints at once (`@NotEmpty` and this custom word-count rule), and each individual constraint is expected to validate only its *own* concern — `null`/blank-ness is `@NotEmpty`'s job, and word-count is this validator's job. If this validator also failed on `null`, a missing value would incorrectly trigger *two* separate validation error messages for what is really one underlying problem.

37.7 Spring Data JPA Queries and Their Externalization

`Demo10022.Spring_Boot_Spring_JPA_Queries` revisits the Query Method and `@Query` techniques from the previous volume's Chapter 31 inside a Spring Boot project; `Demo10023.Spring_Boot_Spring_JP` then takes the *named-query* style of `@Query` one step further, moving even the JPQL text itself out of the Java interface entirely:

```
1 @SuppressWarnings("rawtypes")
2 @RepositoryDefinition(idClass=Integer.class, domainClass=EmployeeEntity.class)
3 @Transactional
4 public interface EmployeeDAO {
5
6     @Query(name="EmployeeDAO.getAllEmployeesBySalary")
7     List<EmployeeEntity> getAllEmployeesBySalary(Double salary);
8
9     @Query(name="EmployeeDAO.getDeptCodesAndCountOfEmployee")
```

```

10 List getDeptCodesAndCountOfEmployee();
11 }

```

with the actual JPQL living in META-INF/jpa-named-queries.properties:

```

1 EmployeeDAO.getAllEmployeesBySalary=from EmployeeEntity k where k.salary>=?1
2 EmployeeDAO.getDeptCodesAndCountOfEmployee=select count(k),k.departmentCode from
  EmployeeEntity k group by k.departmentCode
3
4 #interfaceName.methodName is used as key -- best practice

```

Here, `@Query(name = "...")` refers to a query *by name* rather than embedding JPQL text directly in the annotation, and Hibernate resolves that name against entries in `jpa-named-queries.properties` at start-up. The practical benefit mirrors exactly why the previous volume externalised bean configuration into `.properties` files in the first place: a DBA or a senior developer can tune or fix a query's JPQL without touching (or recompiling) a single line of Java code, and the naming convention shown in the file's own comment — `interfaceName.methodName` — keeps a large set of externalised queries unambiguous as the application grows.

37.8 Caching Query Results with Spring Cache

Repeatedly hitting the database for data that rarely changes (a lookup table, a frequently-viewed record) is wasted work. `Demo10024.Spring_Boot.Spring_JPA.Caching` introduces Spring's caching abstraction, and `Demo10025-1/Demo10025-2` swap in two different real cache providers (EhCache and Redis respectively) underneath the identical annotations:

```

1 @Service
2 public class EmployeeServiceImpl {
3
4     @Autowired
5     private EmployeeDAO employeeDAO;
6
7     // Cache the return value under key = employeeId, in a cache region named "
8     // empCacheSpace".
9     // "unless" skips caching if the result would be null -- no point caching "not
10    found".
11    @Cacheable(value="empCacheSpace", key="#employeeId", unless="#result==null")
12    public Optional<EmployeeBean> getEmployeeDetailByEmployeeId(int employeeId) {
13        System.out.println("I am inside the getEmployeeDetailByEmployeeId() method");
14        return employeeDAO.findById(employeeId).map(employeeEntity -> {
15            EmployeeBean employeeBean = new EmployeeBean();
16            BeanUtils.copyProperties(employeeEntity, employeeBean);
17            return employeeBean;
18        });
19    }
20
21    // @CachePut always runs the method AND updates the cache with its result --
22    // used for writes, where the cache must reflect what was actually just saved.
23    @CachePut(value="empCacheSpace", key="#result.employeeId")
24    public EmployeeBean addEmployee(EmployeeBean employee) {
25        EmployeeEntity employeeEntity = new EmployeeEntity();
26        BeanUtils.copyProperties(employee, employeeEntity);
27        EmployeeEntity emp = employeeDAO.save(employeeEntity);
28        BeanUtils.copyProperties(emp, employee);
29        return employee;
30    }
31
32    // @CacheEvict removes an entry once the underlying data it represents is gone.
33    @CacheEvict(value="empCacheSpace", key="#employeeId")
34    public Optional<EmployeeBean> deleteEmployee(int employeeId) {
35        return employeeDAO.findById(employeeId).map(employeeEntity -> {
36            employeeDAO.delete(employeeEntity);
37            EmployeeBean employeeBean = new EmployeeBean();
38            BeanUtils.copyProperties(employeeEntity, employeeBean);
39            return employeeBean;
40        });
41    }
42
43 }

```

```

38     });
39 }
40
41 // Purge the entire cache region in one call.
42 @CacheEvict(value="empCacheSpace", allEntries=true)
43 public void evictAll() { }
44 }

```

The three annotations divide cleanly along the three things that can happen to a cache entry:

Annotation	Behaviour
@Cacheable	Read-through: on a cache hit, the method body does not run at all, and the cached value is returned directly; on a miss, the method runs and its result is stored for next time
@CachePut	Always runs the method, and <i>always</i> updates the cache with the fresh result — appropriate for writes (create/update), where skipping the method would skip the actual database change
@CacheEvict	Removes one entry (by key) or the whole cache region (allEntries=true) — appropriate whenever the underlying data the cache was representing has just been deleted or otherwise invalidated

The key SpEL expressions are worth reading closely: `key="#employeeId"` refers to the method's own *parameter* named `employeeId`; `key="#result.employeeId"` refers to a property of the method's *return value* instead, used on `addEmployee(...)` specifically because the newly-generated `employeeId` is only known *after* the save happens, not from an incoming parameter. Demo10025-1 and Demo10025-2 change nothing about this service class at all — only the underlying `CacheManager` bean auto-configured by Spring Boot changes, from a simple in-memory `ConcurrentMapCacheManager` default, to **EhCache** (an embedded, JVM-local cache with more configurable eviction policies), to **Redis** (an external, shared cache server, which matters once more than one instance of this same service is running, as in a scaled-out microservice deployment — an in-JVM cache would otherwise leave every instance with its own inconsistent cache).

37.9 Profiles and Conditional Class Loading

Chapter 29 of the previous volume introduced Spring Profiles for selecting between alternative `@Bean` definitions. Demo10026_Spring_Boot_Profiles_Class_Loading shows the same mechanism used slightly differently — to decide whether an entire supporting *class* gets loaded into the container at all, based on which profile is active:

```

1 @Profile("Logging_Profile")
2 @Component
3 public class EmployeeServiceImplLogs {
4     static private Logger logger = LoggerFactory.getLogger("EmployeeServiceImplLogs");
5     static {
6         logger.info("*****")
7         ;
8         logger.info("Class Loading Custom Log[Class loaded: [EmployeeServiceImplLogs]")
9         ;
10        logger.info("*****")
11        ;
12    }
13 }

```

This bean — and its static initializer block, which only ever runs if the class is actually loaded by the JVM — exists purely to make an otherwise invisible fact visible: with `Logging_Profile` inactive, this class is never even instantiated, and the log lines in its static block never print; activate the profile (via `spring.profiles.active=Logging_Profile`, an environment variable, or a

command-line flag) and it loads, logging its own presence as proof. Demo10027..._Programmatic shows the equivalent decision made in Java code instead of purely via `@Profile`, by querying the active `Environment` directly (`environment.acceptsProfiles(...)`) — useful when the loading decision needs logic more elaborate than “is this one profile active or not”. Demo10028 and Demo10029 extend the same idea to **profile-specific property files** (`application-dev.properties`, `application-prod.properties`, automatically layered over the base `application.properties` depending on the active profile) combined with **customizers** — beans implementing Spring Boot’s `WebServerFactoryCustomizer` or similar customizer interfaces to programmatically adjust auto-configured infrastructure (like the embedded server’s port) per-profile, rather than only per-property.

37.10 ApplicationRunner and CommandLineRunner

Sometimes an application needs to run a piece of logic exactly once, right after start-up completes, before it starts serving any requests — warming a cache, validating that a required external system is reachable, seeding reference data. Demo10030_Spring_Boot_Application_CommandLine_Runners shows Spring Boot’s two purpose-built interfaces for exactly this:

```

1 @Component
2 public class CommandLineRunnerBean implements CommandLineRunner {
3     private static final Logger logger = LoggerFactory.getLogger(CommandLineRunnerBean.class);
4
5     public void run(String... args) {
6         String strArgs = Arrays.stream(args).collect(Collectors.joining("|"));
7         logger.info("*****Command Line Runner
8             1*****");
9         logger.info("*****Application started with arguments:" + strArgs + "*****");
10        logger.info("*****");
11    }
12 }
```

```

1 @Component
2 public class ApplicationRunnerBean implements ApplicationRunner {
3     private static final Logger logger = LoggerFactory.getLogger(ApplicationRunnerBean.class);
4
5     @Override
6     public void run(ApplicationArguments arg0) throws Exception {
7         String strArgs = Arrays.stream(arg0.getSourceArgs()).collect(Collectors.joining("|"));
8         logger.info("*****Application Runner 1*****");
9         logger.info("Application started with arguments:" + strArgs);
10        logger.info("*****");
11    }
12 }
```

Both interfaces expose a single `run(...)` method, invoked automatically, exactly once, immediately after `SpringApplication.run(...)` finishes starting the container — the only real difference between them is *how command-line arguments are handed over*: `CommandLineRunner.run(String... args)` receives the raw array of command-line arguments exactly as passed to `main(String[] args)`, while `ApplicationRunner.run(ApplicationArguments args)` receives them wrapped in Spring’s own `ApplicationArguments` type, which additionally distinguishes **option arguments** (`--server.port=8081`) from plain non-option arguments and lets you query them by name rather than parsing the raw array yourself. If more than one runner bean exists (as in Demo10031..._Changed_Ordered, which defines a second runner of each kind), they can be given an explicit execution order with `@Order` (or by implementing `Ordered`) — the exact same ordering mechanism the previous volume introduced for multiple `BeanPostProcessors`, applied here to control which runner’s start-up logic executes first.

37.11 Testing a Spring Boot Application

Demo10032.Spring_Boot.Spring_JPA.Repository_Service_Test adapts the Spring Testing techniques from the previous volume's Chapter 30 to a Spring Boot project, using Spring Boot's own dedicated test annotation in place of a bare `@ContextConfiguration`:

```

1  @RunWith(SpringJUnit4ClassRunner.class)
2  @SpringBootTest(classes = Application.class)
3  @Transactional
4  public class EmployeeServiceTest {
5      private Logger logger = LoggerFactory.getLogger(EmployeeServiceTest.class);
6
7      @Autowired
8      private EmployeeServiceImpl empServiceIMPL;
9
10     @Before
11     public void testMethodBeforeInterceptor() {
12         logger.info("Starting a new Test Method..");
13     }
14
15     @After
16     public void testMethodAfterInterceptor() {
17         logger.info("End the Test Method..");
18     }
19
20     @Test
21     public void testFindAll() {
22         Collection<Employee> empList = empServiceIMPL.getEmployeeDetails();
23         Assert.assertTrue(empList != null);
24     }
25
26     @Test
27     public void testUpdateInvalid() {
28         Employee employee = new Employee("MSD", 10007, 2345.0, 123);
29         Employee empUpdated = empServiceIMPL.updateEmployee(employee);
30         Assert.assertTrue(empUpdated == null);
31     }
32 }

```

`@SpringBootTest` replaces the plain `@ContextConfiguration` used in the previous volume; it bootstraps the *entire* Spring Boot application context the same way `SpringApplication.run(...)` would (auto-configuration included), which is exactly what makes it suitable for a genuine **integration test** — one that exercises the real, fully-wired service and DAO layers together, as opposed to a narrower **unit test** that isolates one class with mocked collaborators. `@Transactional` on the test class means every `@Test` method runs inside its own transaction that is, by default, rolled back automatically once the test method finishes — so `testUpdateInvalid()` can freely attempt an update against the real database and trust that no actual change survives past the test, exactly the same rollback-by-default behaviour introduced for test transactions in the previous volume's discussion of `@DirtiesContext`-adjacent concerns, but achieved here through the transactional test infrastructure itself rather than context caching.

Demo10033...Controller_Integration_Test moves one layer up the stack and tests the `@RestController` itself (via `MockMvc` or a real HTTP client against a running embedded server), verifying the full request-to-response path including HTTP status codes and JSON serialisation — the REST equivalent of an integration test. Demo10034...Controller_Unit_Test narrows the scope back down again, testing the controller class in isolation with its service dependency mocked out (typically with `Mockito`), so the test only verifies the controller's own request-handling logic without needing a real database or a real service implementation behind it at all.

Unit Test vs Integration Test — the Practical Difference

A **unit test** isolates one class, replacing its collaborators with mocks/stubs, so a failure points unambiguously at that one class. An **integration test** wires up real collaborators (a real service, a real repository, a real — if test-configured — database) to verify that the pieces actually work correctly *together*, which a passing set of unit tests alone cannot guarantee, since two individually-correct classes can still be wired together incorrectly.

37.12 YAML Configuration and Relaxed Binding

Every properties file shown so far in this course has used the flat **key=value** `.properties` format. `Demo10035.Spring.Boot.Project.Structure.YML.Config` shows Spring Boot's equally well-supported alternative, **YAML** (`application.yml`), which represents the exact same hierarchical configuration more compactly by using indentation instead of repeated dotted key prefixes:

```

1 server:
2   port: 8090
3
4 data:
5   description: Data from YAML File
6   employeeList:
7     - employeeName: Jack
8       salary: 4500
9       employeeId: 1001
10      departmentCode: 102
11
12     - employeeName: Justin
13       salary: 3500
14       employeeId: 1002
15       departmentCode: 101
16
17     - employeeName: Eric
18       salary: 9500
19       employeeId: 1003
20       departmentCode: 102

```

Note how `data.employeeList` in YAML represents an entire *list of objects* far more readably than the equivalent flattened `.properties` keys (`data.employeeList[0].employeeName=Jack`, `data.employeeList[0].salary=4500`, and so on) would. `Demo10036.Spring.Boot.Spring.JPA.YML.Config` repeats the earlier JPA/DataSource properties in this same YAML format to confirm Spring Boot treats both formats identically as configuration sources.

To actually bind a structured block like `data` above into a Java object, Spring Boot's `@ConfigurationProperties` is used instead of individual `@Value` injections:

```

1 @Component
2 @ConfigurationProperties("data")
3 public class EmployeeListReader {
4     private String description;
5     private List<Employee> employeeList;
6     // getters and setters ...
7
8     @Override
9     public String toString() {
10         return "EmployeeListReader [description=" + description
11             + ", employeeList=" + employeeList + "]";
12     }
13 }

```

`@ConfigurationProperties("data")` tells Spring Boot to bind every property nested under the `data` prefix in `application.yml` onto this bean's matching fields — `description` to

`data.description`, and the entire `employeeList` array of maps to a genuine `List<Employee>`, including nested object binding for each list element's own fields, all without a single explicit `@Value` annotation anywhere. This binding is also **relaxed**: Spring Boot tolerates several equivalent naming conventions between the YAML/properties key and the Java field (`employee-name`, `employeeName`, and `EMPLOYEE_NAME` in an environment variable would all bind to a Java field named `employeeName`), which matters because different configuration sources (a YAML file, a Linux environment variable, a command-line argument) each have their own idiomatic casing convention, and relaxed binding lets all three feed the identical Java property without translation.

37.13 Deploying a Spring Boot Application as a WAR

Every demo so far has assumed the default, simplest Spring Boot deployment model: an executable JAR with an embedded server, run directly with `java -jar`. `Demo10037.SpringBoot.War.File.Deploy` shows that Spring Boot can still, when genuinely needed (for example, deploying into an organisation's existing, externally-managed Tomcat installation rather than each application bringing its own embedded server), be packaged as a traditional WAR instead:

```
1 public class Application extends SpringBootServletInitializer {
2
3     @Override
4     protected SpringApplicationBuilder configure(SpringApplicationBuilder application)
5     {
6         return application.sources(Application.class);
7     }
8
9     public static void main(String[] args) {
10         SpringApplication.run(Application.class, args);
11     }
12 }
```

Extending `SpringBootServletInitializer` and overriding `configure(...)` is what allows the exact same application class to be started two different ways: `main(...)` still works for local development with the embedded server, while an external servlet container (a traditional Tomcat installation, in this deployment model) can instead discover and bootstrap the application itself through the standard Servlet 3.0 `ServletContainerInitializer` mechanism when the WAR is dropped into that container's deployment folder. The accompanying `pom.xml` change needed is equally small: `<packaging>war</packaging>` in place of the default `jar`, plus marking the embedded-Tomcat starter dependency as `<scope>provided</scope>` (present at compile time, but deliberately *not* bundled into the final WAR, since the external container already provides its own).

37.14 Consuming REST Services from a Spring Boot Application — RestTemplate

`Demo10038.SpringBoot.REST.TEMPLATE` repeats the `RestTemplate` client pattern from the previous volume's REST chapter, now calling a Spring Boot-hosted REST API specifically:

```
1 public class Tester1001 {
2     public static final String REST_SERVICE_URI = "http://localhost:8090/emp/controller/";
3
4     private static void listAllEmployees() {
5         RestTemplate restTemplate = new RestTemplate();
6         List<LinkedHashMap<String, Object>> employeeMap =
7             restTemplate.getForObject(REST_SERVICE_URI + "getDetails", List.class);
8
9         ObjectMapper mapper = new ObjectMapper();
10     }
```

```

10     List<Employee> list = new ArrayList<Employee>();
11     if (employeeMap != null) {
12         for (LinkedHashMap<String, Object> map : employeeMap) {
13             Employee emp = mapper.convertValue(map, Employee.class);
14             list.add(emp);
15         }
16         System.out.println("Employee Details are: " + list);
17     }
18 }
19 // getEmployee(), createEmployee(), updateEmployee(), deleteEmployee() follow the
    // same
20 // GET/POST/PUT/DELETE pattern already covered in the previous volume's REST
    // chapter
21 }

```

The one detail worth adding to what the previous volume already covered: `restTemplate.getForObject(..., List.class)` deserialises a JSON array only as far as a raw `List` of generic `LinkedHashMaps`, *not* directly as a `List<Employee>` — Java's generic type erasure means Jackson has no way to know, from `List.class` alone, what element type to build. The Jackson `ObjectMapper`'s `convertValue(map, Employee.class)` call is the manual second step that converts each raw map into an actual `Employee`, one element at a time. (A cleaner alternative — `RestTemplate.exchange(...)` with a `ParameterizedTypeReference<List<Employee>>` — exists precisely to avoid this two-step dance, by preserving the full generic type information that a bare `.class` literal cannot carry.)

37.15 Spring Boot with Spring Batch

`Demo10039.Spring.Boot.With.Spring.Batch` brings the previous volume's Spring Batch chapter into a Spring Boot project. The Job/Step/reader/processor/writer configuration itself is unchanged from before; what changes is how the Job is triggered:

```

1 @SpringBootApplication
2 @EnableBatchProcessing // enables batch configuration
3 @ImportResource(locations = "classpath:spring-batch-config.xml")
4 public class SpringBatchApplication {
5     public static void main(String[] args) {
6         SpringApplication.run(SpringBatchApplication.class, args);
7     }
8 }

```

`@ImportResource` is the bridge between Spring Boot's Java/annotation-first world and the previous volume's XML-based `spring-batch-config.xml` — rather than rewriting that entire Job configuration in Java, it is imported wholesale into the Boot application's context exactly as written. A `@RestController` then exposes an HTTP endpoint that triggers the Job on demand, rather than only via a command-line main method:

```

1 @RestController
2 public class JobLauncherController {
3
4     @Autowired
5     CustomJobLauncher jobLauncher;
6
7     @Autowired
8     Job job;
9
10    @Autowired
11    RepeatTemplate repeatTemplate;
12
13    @RequestMapping("/launchjob")
14    public String launchJobController() {
15        try {
16            SimpleCompletionPolicy completionPolicy = new SimpleCompletionPolicy();
17            completionPolicy.setChunkSize(3);

```

```

18         repeatTemplate.setCompletionPolicy(completionPolicy);
19         repeatTemplate.iterate(new MyRepeatCallback(jobLauncher));
20     } catch (Exception e) {
21         e.printStackTrace();
22     }
23     return "Done!";
24 }
25 }

```

This demo additionally introduces a `RepeatTemplate` with a `SimpleCompletionPolicy` — a Spring Batch construct for repeating a block of work (here, launching the Job via `MyRepeatCallback`) a controlled number of times per invocation (`setChunkSize(3)`), independent of and layered on top of the chunk-oriented step processing covered in the previous volume; it is a general-purpose repeat/retry building block that Spring Batch itself is partly built on, made directly usable from application code. Turning a batch Job from something only launchable at the command line into something launchable over HTTP is precisely the shift that matters for microservices: a batch process that used to run as a standalone script can now be triggered on demand (or wired into a scheduler) as one endpoint of a larger, always-running service.

37.16 Aspect-Oriented Programming (AOP) in Spring Boot

Every layer built so far in this course — logging, transaction demarcation, security checks — has a recurring shape: a piece of behaviour that is not really part of any one class’s core business logic, but needs to run consistently *around* many different methods across many different classes (log every repository call; wrap every service method in a transaction). Writing that behaviour by hand, repeated inside every single method, is exactly the kind of repetition Spring exists to remove. **Aspect-Oriented Programming (AOP)** is the general programming paradigm for extracting this kind of **cross-cutting concern** into one place and having it apply automatically, without touching the methods it applies to.

37.16.1 Core AOP Vocabulary

Term	Meaning
Aspect	A module that encapsulates a cross-cutting concern (e.g. logging) — in Spring, a plain class annotated <code>@Aspect</code>
Join Point	A specific point during program execution where an aspect <i>could</i> apply — in Spring AOP, always a method execution
Pointcut	An expression that <i>selects</i> which join points a particular piece of advice actually applies to (e.g. “every method in this one repository interface”)
Advice	The actual code that runs at a matched join point, and <i>when</i> relative to it (before, after, or wrapping around the method call)
Weaving	The process of actually linking an aspect into the target code at the matched join points — Spring AOP does this at runtime, via dynamically generated proxies

37.16.2 @Around Advice — Wrapping a Method Call

`Demo10040.SpringBoot.AOP.Logging` shows the most powerful advice type, `@Around`, which wraps the *entire* target method call, giving the aspect control over what happens both before and after it, and even whether it runs at all:

```

1 @Aspect
2 @Component
3 public class LoggingAspect {
4

```

```

5     private final Logger logger = LoggerFactory.getLogger(LoggingAspect.class);
6
7     @Around("execution(* com.accenture.ltt.repository.UserRepository.*(..))")
8     public Object logRepositoryMethods(ProceedingJoinPoint pjp) throws Throwable {
9         logger.info("Entering repository method: {} with args {}", pjp.getSignature(),
10             pjp.getArgs());
11         Object result = pjp.proceed(); // actually invoke the real, wrapped method
12         logger.info("Exiting repository method: {} with result {}", pjp.getSignature(),
13             result);
14         return result;
15     }
16 }

```

The string `"execution(* com.accenture.ltt.repository.UserRepository.*(..))"` is the **pointcut expression**: `execution(...)` matches method execution join points, the first `*` means “any return type”, `UserRepository.*` means “any method name declared on `UserRepository`”, and `(..)` means “any number and type of arguments”. `pjp.proceed()` is what actually calls through to the real `UserRepository` method being advised — omitting that call would mean the underlying method never runs at all, which is exactly the extra power `@Around` has over the simpler `@Before`/`@After` advice types (which cannot skip or replace the underlying call, only run code immediately before or after it unconditionally).

37.16.3 Named Pointcuts — Reusing a Selection Across Multiple Advice

Demo10041.SpringBoot_AOP_Poinctut shows the same aspect refactored to separate the *pointcut* (which join points) from the *advice* (what to do at them), by naming the pointcut expression once and referencing it by name:

```

1  @Aspect
2  @Component
3  public class LoggingAspect {
4
5      private final Logger logger = LoggerFactory.getLogger(LoggingAspect.class);
6
7      @Pointcut("execution(* com.accenture.ltt.repository.UserRepository.*(..))")
8      private void repositoryMethods() {
9          // Pointcut signature -- no implementation needed, this method body never
10             actually runs
11      }
12
13      @Around("repositoryMethods()")
14      public Object logRepositoryMethods(ProceedingJoinPoint pjp) throws Throwable {
15          logger.info("Entering repository method: {} with args {}", pjp.getSignature(),
16              pjp.getArgs());
17          Object result = pjp.proceed();
18          logger.info("Exiting repository method: {} with result {}", pjp.getSignature(),
19              result);
20          return result;
21      }
22  }

```

This refactoring matters the moment more than one piece of advice needs to apply to the *same* set of join points — for example, both a logging aspect and a separate performance-timing aspect both wanting to advise every `UserRepository` method. Without a named pointcut, the identical `execution(...)` expression string would need to be copy-pasted into both aspects, and a later change to which methods should be advised would require updating every copy consistently; with a named pointcut, `repositoryMethods()` is defined exactly once and referenced by name from as many pieces of advice as needed.

Viva Voce — Likely Questions & Answers

Q: What is the core difference between a monolithic and a microservice architecture?

A: A monolith deploys the entire application as one single unit sharing one codebase, one deployment lifecycle, and typically one database; a microservice architecture splits the application into multiple small, independently deployable services, each owning one well-defined piece of business capability and communicating over the network.

Q: What two central ideas does Spring Boot contribute on top of the plain Spring Framework?

A: Auto-configuration (automatically registering and wiring the beans a typical application needs, based on what's on the classpath, unless the developer already supplied their own) and an embedded server (bundling a servlet container inside the application itself so it can run as a single executable JAR without an externally installed server).

Q: What three annotations does `@SpringBootApplication` bundle together?

A: `@Configuration` (marks the class as a bean-definition source), `@ComponentScan` (enables component scanning from this class's package downward), and `@EnableAutoConfiguration` (turns on Spring Boot's classpath-based auto-configuration).

Q: What does `@ConditionalOnMissingBean` do, and which behaviour described earlier does it directly explain?

A: It causes an auto-configuration class's bean definition to be skipped if a bean of that same type has already been defined elsewhere (typically by the developer's own configuration). It is the exact mechanism behind Spring Boot auto-configuration "backing off" whenever a developer supplies their own bean of a given type.

Q: What is the difference between a Spring Boot starter and auto-configuration?

A: A starter is a curated Maven dependency bundle that pulls in a consistent set of libraries for one concern; auto-configuration is a separate mechanism that reacts to whichever classes actually end up on the classpath (often, but not only, because of a starter), using conditional annotations like `@ConditionalOnClass` to decide whether to register its beans.

Q: How does excluding the Tomcat starter and adding the Jetty starter (Section 37.3.2) actually cause Spring Boot to switch embedded servers, at the mechanism level?

A: The embedded-server auto-configuration classes are guarded by `@ConditionalOnClass` checks against each server's own classes; removing Tomcat's transitive dependency makes its `@ConditionalOnClass` check fail so its auto-configuration backs off, while adding Jetty's dependency makes Jetty's own equivalent check succeed instead.

Q: What is the practical difference between `@NotNull` and `@NotEmpty` on a String field, and why does it matter?

A: `@NotNull` only rejects a missing (null) value; an empty string "" still passes `@NotNull` since it isn't null. `@NotEmpty` additionally rejects an empty string, which matters because a submitted form field that is simply left blank typically arrives as "", not null.

Q: Why does a custom `ConstraintValidator`'s `isValid(...)` method typically return true for a null input rather than false?

A: Because a single field is often validated by multiple constraints at once, each responsible for only its own concern; null/blank-ness is expected to be handled by a dedicated constraint like `@NotEmpty`, so a different validator returning false on null would produce a redundant, duplicate error message for the same underlying problem.

Q: What is the difference between @Cacheable and @CachePut?

A: @Cacheable is read-through: on a cache hit it skips the method body entirely and returns the cached value; on a miss it runs the method and caches the result. @CachePut always runs the method and always updates the cache with the fresh result, which is why it's used for writes rather than reads.

Q: What is the key difference between ApplicationRunner and CommandLineRunner?

A: Both run exactly once, immediately after the Spring Boot application starts. CommandLineRunner receives the raw String array of command-line arguments; ApplicationRunner receives them wrapped in Spring's ApplicationArguments type, which additionally distinguishes named option arguments from plain arguments.

Q: What does @SpringBootTest do that a plain @ContextConfiguration from the previous volume does not?

A: It bootstraps the entire Spring Boot application context the same way SpringApplication.run(...) would, including auto-configuration, making it suitable for genuine integration tests of the fully-wired application rather than a narrowly scoped test context.

Q: What is the practical advantage of YAML configuration over the flat .properties format for something like a list of objects?

A: YAML represents nested and list-shaped configuration far more readably through indentation, whereas the equivalent .properties format would require verbose, repeated, indexed dotted keys for every field of every list element.

Q: What does "relaxed binding" mean in the context of @ConfigurationProperties?

A: Spring Boot tolerates multiple equivalent naming conventions (e.g. employee-name, employeeName, EMPLOYEE_NAME) across different configuration sources (YAML files, environment variables, command-line arguments) and binds them all to the same Java field, since each source has its own idiomatic casing convention.

Q: Why might a Spring Boot application still need to be packaged as a WAR rather than an executable JAR?

A: When it needs to be deployed into an organisation's existing, externally-managed servlet container (such as a shared Tomcat installation) rather than bringing its own embedded server, typically for operational or infrastructure standardisation reasons.

Q: What is the role of pjp.proceed() inside an @Around advice method, and what happens if it's omitted?

A: It actually invokes the real, wrapped method being advised. If omitted, the underlying method never runs at all — the advice has effectively replaced it rather than wrapped it, which is the extra control @Around has over simpler advice types like @Before/@After.

Q: Why would you refactor a pointcut expression into a separate, named @Pointcut method rather than repeating the expression string inline in each @Around/@Before annotation?

A: So the same set of join points can be reused consistently across multiple pieces of advice (e.g. both a logging aspect and a timing aspect) by referencing the pointcut by name, avoiding duplicated expression strings that would all need updating consistently if the selection criteria ever changed.

Chapter 38

Spring Cloud — Microservice Patterns

38.1 The Challenges a Monolith Never Had

Section 37.1 listed why a monolith becomes limiting as a system grows. Splitting it into microservices solves those problems, but immediately introduces a new set of problems that a monolith, by virtue of being one single process, never had to face at all:

- **How does one service find another?** In a monolith, one module calling another is just a Java method call, resolved at compile time. Once `EmployeeConsumer` and `EmployeeProducer` are two separate, independently-deployed processes, `EmployeeConsumer` needs to know *where* (which host, which port) `EmployeeProducer` is currently running — and that location can change every time `EmployeeProducer` is redeployed, restarted, or scaled to multiple instances.
- **What happens when a service is slow or unreachable?** In a monolith, a failing module usually means the whole process crashes or throws visibly. In a distributed system, a slow or down downstream service can cause calling threads to pile up waiting for a response that never comes, and that pile-up can itself cascade into *other* services that depend on the caller — an entire system brought down by one failing dependency.
- **How is load spread across multiple instances of the same service?** If `EmployeeProducer` is scaled to three running instances for capacity, something needs to decide, for each individual call, *which* of the three instances actually receives it.

Spring Cloud is a collection of libraries, built on top of Spring Boot, that provide ready-made solutions to exactly these distributed-systems problems, so that each individual microservice team does not need to solve service discovery, resilience, and load balancing from scratch.

38.2 Further Microservice Design Patterns

Eureka, client-side load balancing, and Resilience4j (covered in full, with working code, later in this chapter) are three specific, concrete answers to the challenges named above. They sit alongside a handful of further, equally standard microservice patterns that this course's demos do not build directly, but which are worth knowing by name and purpose, since they very often come up in the same breath as the patterns this chapter does cover:

Pattern	Purpose
API Gateway	A single, well-known entry point that sits in front of many internal microservices, handling cross-cutting concerns (routing, authentication, rate limiting) once, centrally, rather than duplicating that logic inside every individual service. A client talks only to the gateway; it never needs to know how many internal services exist behind it or where each one lives.
Backend for Frontend (BFF)	A variation on the API Gateway idea where, rather than one single generic gateway, each distinct kind of client (a mobile app, a web app, a third-party partner integration) gets its <i>own</i> tailored gateway/aggregation layer, shaped around that specific client's needs, rather than forcing every client to consume one one-size-fits-all API.
Saga	A pattern for maintaining data consistency across a business transaction that spans <i>multiple</i> independent services (and therefore multiple independent databases), since a traditional single-database ACID transaction is no longer available once the data is split across services. A Saga breaks the transaction into a sequence of local transactions, each with its own defined compensating action to undo it if a later step in the sequence fails — accepting eventual consistency in exchange for not needing a single, all-encompassing distributed transaction.
Strangler Fig	A migration strategy for incrementally moving a monolith to microservices without a risky “big bang” rewrite: new functionality is built as microservices from the start, and pieces of the old monolith are peeled off into their own services one at a time, with a routing layer directing traffic to whichever — old monolith or new service — currently owns a given piece of functionality, until eventually nothing routes to the original monolith at all.
Database per Service	Each microservice owns its own private database, never accessed directly by any other service — a direct consequence of the Bounded Context boundary from Chapter 36: if two services shared one database, that database's schema would itself become a second, hidden, badly-managed integration point bypassing whatever clean API boundary the services otherwise present to each other.

Why “Database per Service” Matters, Even Though Every Demo in This Course Shares One

Every layered demo across this entire course — for simplicity, and to keep the focus on one concept at a time — has used a single shared database. In a genuine microservice deployment, letting two independently-deployed services both read and write the same tables reintroduces exactly the tight coupling microservices are meant to remove: neither service can change its own table structure without risking breaking the other, which is precisely the kind of hidden, undeclared dependency a well-drawn Bounded Context (Chapter 36) is supposed to prevent. Recognising this trade-off is more important than memorising the pattern's name — it is the direct, practical reason a real microservice architecture needs the Saga pattern above at all: without a single shared database, a business transaction

spanning services can no longer rely on one database's transaction manager to keep it all consistent.

38.3 Service Discovery with Netflix Eureka

Service discovery solves the “where is it right now” problem: rather than every consumer hardcoding the host and port of every producer it depends on (which breaks the moment that producer moves, scales, or is redeployed to a new machine), each service instance **registers itself** with a central **service registry** on start-up, announcing “I am an instance of `cst-employee-producer`, reachable at this host and port”; any consumer that needs to call that service asks the registry, at call time, for the current list of instances, rather than hardcoding an address anywhere.

Eureka (originally built by Netflix, now part of Spring Cloud Netflix) is the service registry used throughout this course's demos. Setting one up is a single annotation:

```
1 @SpringBootApplication
2 @EnableEurekaServer
3 public class EurekaServer {
4     public static void main(String[] args) {
5         SpringApplication.run(EurekaServer.class, args);
6     }
7 }
```

```
1 server.port = 7090
2 eureka.client.serviceUrl.defaultZone:http://localhost:7090/eureka/
3 # The registry itself does not need to register with -- or discover services from --
   itself
4 eureka.client.register-with-eureka=false
5 eureka.client.fetch-registry=false
```

`eureka.client.register-with-eureka=false` and `eureka.client.fetch-registry=false` are worth pausing on: the Eureka *server* project still has the Eureka *client* auto-configuration on its classpath (it's the same Spring Cloud Netflix starter), so without these two properties the server would attempt to register itself as a client of itself, which is meaningless — these two lines explicitly opt the registry out of that default client behaviour.

38.3.1 Registering a Producer as a Discoverable Client

Any service that should be discoverable simply adds `@EnableDiscoveryClient` and points at the registry's URL:

```
1 @SpringBootApplication
2 @EnableDiscoveryClient // part of the Spring Cloud Commons project
3 public class EurekaProducer {
4     public static void main(String[] args) {
5         SpringApplication.run(EurekaProducer.class, args);
6     }
7 }
```

```
1 server.port=7091
2 eureka.client.serviceUrl.defaultZone=http://localhost:7090/eureka
```

`@EnableDiscoveryClient` is a deliberately *vendor-neutral* annotation from Spring Cloud Commons (rather than Eureka-specific `@EnableEurekaClient`) — the same annotation works whether the actual registry underneath happens to be Eureka, Consul, or another supported implementation, so switching registry technology later does not require changing this annotation in every single microservice.

38.3.2 Looking a Service Up by Name — DiscoveryClient

Demo1003.Employee_Consumer_Eureka_Demo shows a consumer resolving a producer's actual address at call time, by name, through Spring's `DiscoveryClient` abstraction:

```

1  @RestController
2  public class ConsumerControllerClient {
3
4      @Autowired
5      private DiscoveryClient discoveryClient;
6
7      @RequestMapping(value = "emp/controller/getDetailsClient", method = RequestMethod.
8          GET,
9          produces = MediaType.APPLICATION_JSON_VALUE)
10     public ResponseEntity<Collection<Employee>> getEmployee() {
11         List<ServiceInstance> instances = discoveryClient.getInstances("cst-employee-
12             producer");
13         ServiceInstance serviceInstance = instances.get(0); // just the first instance
14             found
15         String baseUrl = serviceInstance.getUri().toString();
16         baseUrl = baseUrl + "/emp/controller/getDetails";
17
18         RestTemplate restTemplate = new RestTemplate();
19         ResponseEntity<List> employeeMapRespEntity =
20             restTemplate.exchange(baseUrl, HttpMethod.GET, null, List.class);
21         // ... convert the raw List<LinkedHashMap> into List<Employee>, exactly as in
22             Chapter 37 ...
23         return new ResponseEntity<Collection<Employee>>(list, employeeMapRespEntity.
24             getStatusCode());
25     }
26 }

```

`discoveryClient.getInstances("cst-employee-producer")` is the entire point of service discovery made concrete: rather than a hardcoded `http://localhost:7091/...` baked into the consumer's code, the consumer asks Eureka, by the producer's registered *name*, for whatever instances currently exist — if the producer moves to a different host, restarts on a different port, or is scaled up to several instances, this consumer's code needs no change at all, because it never hardcoded a specific address anywhere. Note also that this demo picks `instances.get(0)` — always the *first* instance found — which is a deliberately naive placeholder for the next section's actual concern: with more than one instance available, *which* one should be picked for a given call?

38.4 Client-Side Load Balancing

Picking `instances.get(0)` every time means every single call goes to the exact same instance, leaving any other running instances of that same service completely idle — defeating the entire purpose of having scaled the service out in the first place. **Client-side load balancing** moves the responsibility for spreading calls across multiple available instances into the calling application itself, rather than requiring a separate, centrally-managed load balancer sitting in front of the service.

Demo1005.Employee_Consumer_LoadBalancer_Demo shows Spring Cloud LoadBalancer used in the simplest possible way — by annotating the `RestTemplate` bean itself:

```

1  @SpringBootApplication
2  @EnableDiscoveryClient
3  public class LoadBalancerConsumer {
4      public static void main(String[] args) {
5          SpringApplication.run(LoadBalancerConsumer.class, args);
6      }
7
8      @LoadBalanced
9      @Bean
10     public RestTemplate restTemplate() {

```

```

11         return new RestTemplate();
12     }
13 }

1 @RestController
2 public class ConsumerControllerClient1 {
3
4     @Autowired
5     private RestTemplate restTemplate;
6
7     @RequestMapping(path = "/consumer1/getDetails")
8     public ResponseEntity<String> getEmployee() {
9         String baseUrl = "http://cst-employee-producer"; // a logical service name,
10            not a real host!
11         baseUrl = baseUrl + "/emp/controller/getDetails";
12         return restTemplate.exchange(baseUrl, HttpMethod.GET, null, String.class);
13     }
14 }

```

The single `@LoadBalanced` annotation on the `RestTemplate` `@Bean` definition changes everything about how that bean behaves: instead of treating `"http://cst-employee-producer"` as a literal, resolvable hostname (which it is not — no DNS entry for it exists), the `@LoadBalanced RestTemplate` intercepts the call, treats `cst-employee-producer` as a *service name*, asks the discovery client (Eureka, in this course) for every currently-registered instance of that name, and *itself* picks one — by default, in a simple round-robin fashion, spreading successive calls evenly across every available instance — before actually issuing the HTTP request to that chosen instance's real host and port. Compare this directly against the previous section's `DiscoveryClient`-based demo: that consumer manually looked up instances and manually built the real URL; this consumer writes a URL that looks almost like a hardcoded hostname, but the load-balancing machinery underneath resolves it dynamically and spreads the load, entirely hidden behind the familiar `RestTemplate` API.

38.5 Resilience with Resilience4j — the Circuit Breaker Pattern

Service discovery and load balancing solve *finding* a healthy instance to call. Neither one solves what happens when a call to an instance that *looks* healthy nonetheless starts failing or timing out repeatedly — a database the producer depends on going down, for example. Naively retrying a failing call, over and over, against a producer that is already struggling only makes that producer's problem worse, and ties up the consumer's own threads waiting on calls that are unlikely to succeed. The **Circuit Breaker** pattern addresses this directly, borrowing its name and its core idea from an electrical circuit breaker: after a configured rate of failures, the breaker **trips open**, and further calls are rejected *immediately*, without even attempting to reach the failing service, giving it room to recover; after a cooldown period, the breaker moves to a cautious **half-open** state, letting a small number of test calls through to check whether the underlying problem has actually been resolved, before either closing again (resuming normal calls) or tripping back open.

38.5.1 Producing a Deliberately Unreliable Endpoint

`Demo1006.Employee_Producer_Resilience4J.Demo` builds a producer endpoint that deliberately fails intermittently, purely so the circuit breaker on the consumer side has something real to react to:

```

1 @RestController
2 public class EmployeeController {
3     private static int count = 0;

```

```

4     private static boolean flag = true;
5
6     @RequestMapping(value = "emp/controller/getDetails", method = RequestMethod.GET,
7                     produces = MediaType.APPLICATION_JSON_VALUE)
8     public ResponseEntity<Collection<Employee>> getEmployeeDetails3() throws Exception
9     {
10         Collection<Employee> listEmployee = mapOfEmployeeess.values();
11         if (flag) {
12             count++;
13             if (count % 2 == 0) {
14                 return new ResponseEntity<Collection<Employee>>(HttpStatus.
15                     INTERNAL_SERVER_ERROR);
16             }
17         }
18         return new ResponseEntity<Collection<Employee>>(listEmployee, HttpStatus.OK);
19     }
20
21     @RequestMapping(value = "emp/controller/setFlag/{value}", method = RequestMethod.
22         GET)
23     public ResponseEntity<String> setFlag(@PathVariable("value") Boolean value) {
24         flag = value;
25         return new ResponseEntity<String>("Flag set to " + value + "...", HttpStatus.OK
26     );
27     }
28 }

```

Every second call (`count % 2 == 0`) deliberately returns 500 Internal Server Error — a controllable, on-demand failure the consumer-side circuit breaker in the next section is configured to actually watch for and react to.

38.5.2 Wrapping a Call with @CircuitBreaker and a Fallback

Demo1007_Employee_Consumer_Resilience4J_Demo wraps the call to that unreliable producer:

```

1  @RestController
2  public class ConsumerControllerClient2 {
3
4      private static final String RESILIENCE4J_INSTANCE_NAME = "jasKeyCompute";
5      private ResponseEntity<String> response = null;
6
7      @RequestMapping(value = "/to-read2", method = RequestMethod.GET, produces =
8          MediaType.APPLICATION_JSON_VALUE)
9      @CircuitBreaker(name = RESILIENCE4J_INSTANCE_NAME, fallbackMethod = "
10          subscribesFallbackMethod")
11      public ResponseEntity<String> getEmployee() throws RestClientException, IOException
12      {
13          System.out.println("Consumer calling Producer app");
14          String baseUrl = "http://localhost:7091/emp/controller/getDetails";
15          RestTemplate restTemplate = new RestTemplate();
16          response = restTemplate.exchange(baseUrl, HttpMethod.GET, null, String.class);
17          return new ResponseEntity<String>(response.getBody(), HttpStatus.OK);
18      }
19
20      public ResponseEntity<String> subscribesFallbackMethod(InternalServerError e) {
21          System.out.println("\tProducer app responded with InternalServerError,
22              generating cached response ");
23          return new ResponseEntity<String>(response.getBody(), HttpStatus.OK);
24      }
25
26      public ResponseEntity<String> subscribesFallbackMethod(CallNotPermittedException e)
27      {
28          System.out.println("Circuit is in open state, Consumer not calling Producer app
29              , generating cached response ");
30          return new ResponseEntity<String>(response.getBody(), HttpStatus.OK);
31      }
32  }

```

with the breaker's behaviour tuned entirely through configuration, no further code needed:

```

1 server.port = 7093
2 resilience4j.circuitbreaker.instances.jasKeyCompute.
  automaticTransitionFromOpenToHalfOpenEnabled=true
3 resilience4j.circuitbreaker.instances.jasKeyCompute.failureRateThreshold=50
4 resilience4j.circuitbreaker.instances.jasKeyCompute.minimumNumberOfCalls=6
5 resilience4j.circuitbreaker.instances.jasKeyCompute.waitDurationInOpenState=60s
6 resilience4j.circuitbreaker.instances.jasKeyCompute.
  permittedNumberOfCallsInHalfOpenState=4
7 resilience4j.circuitbreaker.instances.jasKeyCompute.slidingWindowType=COUNT_BASED

```

Reading the two `subscribesFallbackMethod` overloads against the properties above ties the whole pattern together:

- `minimumNumberOfCalls=6` and `failureRateThreshold=50` together mean: once at least 6 calls have been made, if 50% or more of them failed, the breaker trips **open**.
- While the breaker is **open**, every further call is rejected *immediately*, without even attempting to reach `http://localhost:7091/...` — this immediate rejection is exactly what surfaces as a `CallNotPermittedException`, caught by the second fallback overload (“Circuit is in open state ...”).
- When an individual call *is* actually attempted and the producer itself returns a 500 (as it does on alternating calls in this demo), that surfaces instead as an `InternalServerError`, caught by the first fallback overload.
- `waitDurationInOpenState=60s` and `automaticTransitionFromOpenToHalfOpenEnabled=true` together mean: after 60 seconds in the open state, the breaker automatically transitions to **half-open** and, per `permittedNumberOfCallsInHalfOpenState=4`, allows exactly 4 test calls through to check whether the producer has actually recovered, before deciding whether to close the circuit again or trip back open.

Both fallback methods return `response.getBody()` — the *last successfully cached* response — rather than propagating the failure to whatever called this consumer endpoint in the first place. This is the circuit breaker pattern’s real payoff: a downstream failure degrades gracefully (a slightly stale cached answer) instead of cascading upward as a hard failure that the caller’s caller, and the caller’s caller’s caller, would otherwise all have to handle too.

38.5.3 Observing State Transitions

`Resilience4JStateNotifier` shows how to make the breaker’s otherwise-invisible internal state machine observable, simply by subscribing to its event publisher on start-up:

```

1 @Component
2 public class Resilience4JStateNotifier {
3     private static Logger LOG = LoggerFactory.getLogger(Resilience4JStateNotifier.class);
4
5     @Autowired
6     private CircuitBreakerRegistry circuitBreakerRegistry;
7
8     @PostConstruct
9     private void postConstruct() {
10         circuitBreakerRegistry.getAllCircuitBreakers().forEach(circuitBreaker ->
11             circuitBreaker.getEventPublisher().onStateTransition(event -> LOG.info(
12                 event.toString()))
13         );
14     }
15 }

```

`@PostConstruct` (a standard Java lifecycle annotation, conceptually the annotation-based equivalent of the XML `init-method` from the earlier volume’s Spring Core chapter) runs this method once, immediately after the bean is fully constructed and its dependencies injected — here, used

to register a listener against *every* circuit breaker the application has defined, logging each time one transitions between closed, open, and half-open, which is exactly the kind of operational visibility that matters once this pattern is running in a real, unattended production environment rather than a classroom demo.

Viva Voce — Likely Questions & Answers

Q: What new problems does splitting a monolith into microservices introduce that a monolith never had?

A: Service discovery (finding where a needed service instance is currently running), resilience (handling a downstream service that is slow, failing, or unreachable without cascading that failure upward), and load balancing (spreading calls across multiple instances of the same service).

Q: What is the purpose of an API Gateway, and how does a Backend for Frontend refine that idea?

A: An API Gateway is a single entry point in front of many internal microservices, centralising cross-cutting concerns like routing and authentication rather than duplicating them in every service. A Backend for Frontend refines this by giving each distinct kind of client (mobile, web, partner integrations) its own tailored gateway shaped around that client's specific needs, instead of one generic gateway serving every client identically.

Q: Why does splitting data across a Database-per-Service architecture require a pattern like Saga for multi-service business transactions?

A: Because a traditional single-database ACID transaction is no longer available once related data lives in separate, independently-owned databases; Saga instead breaks the overall transaction into a sequence of local transactions, each with its own compensating action to undo it if a later step fails, trading strict consistency for eventual consistency.

Q: What problem does the Strangler Fig pattern solve when migrating a monolith to microservices?

A: It allows an incremental migration rather than a risky all-at-once rewrite: new functionality is built as microservices from the start, existing monolith functionality is peeled off into its own services gradually, and a routing layer directs traffic to whichever currently owns a given piece of functionality until the original monolith is eventually retired entirely.

Q: Why does letting two independently-deployed microservices share one database reintroduce the coupling microservices are meant to remove?

A: Because the shared database's schema itself becomes a hidden, undeclared integration point between the two services – neither can change its own table structure without risking breaking the other – which defeats the clean, independent boundary a well-drawn Bounded Context is meant to establish.

Q: What is the role of a service registry like Eureka?

A: It lets each service instance register itself on start-up, announcing its logical service name and current network location, so that any consumer can look up the current set of instances for a given service name at call time, rather than hardcoding a fixed address that would break whenever that service moves, restarts, or scales.

Q: Why does the Eureka server itself set `eureka.client.register-with-eureka=false` and `eureka.client.fetch-registry=false`?

A: Because the Eureka server project still includes the Eureka client auto-configuration on its classpath; without these properties it would attempt to register itself as a client of itself and fetch its own registry, which is meaningless for the registry itself.

Q: Why is `@EnableDiscoveryClient` used instead of the Eureka-specific `@EnableEurekaClient` in these demos?

A: `@EnableDiscoveryClient` is a vendor-neutral annotation from Spring Cloud Commons that works regardless of which specific service registry implementation is used underneath, so a later switch away from Eureka would not require changing this annotation across every microservice.

Q: What problem does client-side load balancing solve that plain service discovery alone does not?

A: Plain service discovery only returns the list of available instances; without load balancing, a naive consumer might always call the same instance (e.g. always the first one returned), leaving other scaled-out instances idle. Load balancing actively distributes calls across the available instances.

Q: What does the `@LoadBalanced` annotation actually change about a `RestTemplate` bean?

A: It makes the `RestTemplate` treat the hostname portion of a URL as a logical service name rather than a literal, DNS-resolvable hostname, resolving it at call time via the discovery client to a list of real instances and choosing one (by default, round-robin) before issuing the actual HTTP request.

Q: What is the Circuit Breaker pattern, and what three states does it move between?

A: A resilience pattern where repeated failures calling a dependency cause further calls to be rejected immediately rather than repeatedly attempted, giving the failing dependency room to recover. It moves between CLOSED (calls proceed normally), OPEN (calls are rejected immediately without attempting the real call), and HALF-OPEN (a limited number of test calls are allowed through to check for recovery).

Q: In the Resilience4j demo, what is the difference between the two fallback method overloads, one accepting `InternalServerError` and one accepting `CallNotPermittedException`?

A: The `InternalServerError` overload handles the case where the actual call was attempted and the producer itself returned a failure response; the `CallNotPermittedException` overload handles the case where the circuit was already open and the call was rejected immediately without ever reaching the producer at all.

Q: What do the properties `minimumNumberOfCalls` and `failureRateThreshold` control together?

A: They define when the circuit breaker trips open: once at least `minimumNumberOfCalls` calls have been recorded, if the proportion of failures among them reaches or exceeds `failureRateThreshold` percent, the breaker transitions to the open state.

Q: Why do the fallback methods in this demo return a previously cached response rather than propagating the failure upward?

A: To degrade gracefully – returning a slightly stale but still useful answer rather than letting a downstream failure cascade as a hard error up through every caller that depends on this consumer, which is the core benefit the circuit breaker pattern is meant to provide.

Chapter 39

Microservice Monitoring & API Communication

39.1 Why Monitoring and Communication Deserve Their Own Chapter

Chapter 38 solved *finding* a service instance and *surviving* its failures. This chapter addresses two further practical needs that only become pressing once a system is actually made of many independent services: **how do you see what is going on** inside a running service, and across a whole chain of services handling one logical request, without attaching a debugger to a production process; and **how do services describe and call each other’s APIs** in a less error-prone, less boilerplate-heavy way than hand-writing every `RestTemplate` call, as done throughout this course so far.

39.2 The Three Pillars of Observability

Before looking at Actuator, Feign, Swagger, and Sleuth/Zipkin as four separate tools, it helps to see that they collectively answer the three standard questions that make up **observability** — the broader discipline of understanding a running system’s internal state purely from the external signals it produces, without needing to attach a debugger or guess. Observability is conventionally described as resting on three distinct kinds of signal, each answering a different question about a running system:

Pillar	Question It Answers	Tool in This Chapter
Logs	What specific events happened, in what order, with what detail?	Every <code>Logger/LoggerFactory</code> call already used throughout this course’s demos, automatically tagged with trace/span IDs once Sleuth is added
Metrics	What do the numbers look like in aggregate, over time — request rates, error rates, latency percentiles, memory usage?	Actuator’s <code>/actuator/metrics</code> endpoint
Traces	For one specific request, what path did it take through the system, and where did the time actually go?	Sleuth (generating and propagating trace/span IDs) plus Zipkin (visualising the resulting timeline)

Each pillar alone tells an incomplete story. Logs alone, spread across several independent microservices with no shared identifier, cannot answer “which of these log lines, across all these

different services, belong to the one slow request a user just complained about” — exactly the gap trace propagation closes. Metrics alone can tell you *that* the 95th-percentile latency for the checkout endpoint spiked at 2pm, but not *why*, or which specific downstream call was responsible — a trace for one of the slow requests during that window is what actually points at the cause. And a trace for one single request, however detailed, cannot tell you whether that request was a one-off anomaly or represents a system-wide trend — which is exactly what an aggregate metric is for. This is precisely why a real production microservice deployment reaches for all three together rather than treating any one of them as sufficient on its own, and why this chapter’s four tools — each addressing one pillar, or the connective tissue between them — form one coherent picture rather than four unrelated features.

Spring Boot Actuator adds a set of ready-made, production-grade HTTP endpoints to any Spring Boot application, exposing operational information about the running process — its health, its configuration, its metrics — without a single line of custom code beyond adding the starter dependency and a small amount of configuration.

Demo1012.SpringBoot.Actuator.Demo enables it with:

```
1 management.endpoint.health.show-details=ALWAYS
2 management.endpoints.web.exposure.include=*
3 management.info.env.enabled=true
4
5 info.app.name=SpringBoot Actuator Demo
6 info.app.description=An Application to demonstrate the usage of Actuator
7 info.app.version=1.0
8 info.app.java.source=1.8
9 info.app.java.target=1.8
10
11 management.endpoint.shutdown.enabled=true
```

`management.endpoints.web.exposure.include=*` exposes every available Actuator endpoint over HTTP (by default, Spring Boot only exposes the minimal `/actuator/health` endpoint, deliberately, since some Actuator endpoints reveal internal details that should not be public in a real production deployment without further access control — Chapter 41’s Spring Security chapter is precisely how a real deployment would lock these endpoints down rather than exposing them wide open as this teaching demo does).

Endpoint	Reports
<code>/actuator/health</code>	Whether the application (and its key dependencies, like the database) is up; <code>show-details=ALWAYS</code> includes the full breakdown rather than just an overall UP/DOWN
<code>/actuator/info</code>	Arbitrary custom information — here, the <code>info.app.*</code> properties above, surfaced verbatim
<code>/actuator/metrics</code>	Runtime metrics — JVM memory usage, HTTP request counts and latencies, and more
<code>/actuator/env</code>	The application’s fully resolved Environment — every property source and its current values
<code>/actuator/shutdown</code>	(Only if explicitly enabled, as above) Gracefully shuts the application down — a genuinely dangerous endpoint to expose without authentication, included here purely to illustrate that Actuator endpoints can do more than just report information

39.2.1 Customising Actuator’s Base Path and Endpoint Names

Demo1013.SpringBoot.Actuator.Customized.Demo shows that Actuator’s defaults — the `/actuator` base path, and each endpoint’s default name — are themselves just configuration, not fixed behaviour:

```

1 management.endpoints.web.base-path=/appActuator
2 management.endpoints.web.path-mapping.health=appHealth
3 management.endpoint.metrics.enabled=false

```

After this change, the health endpoint moves from `/actuator/health` to `/appActuator/appHealth`, and the metrics endpoint is disabled entirely. This matters in a real deployment for two practical reasons: an organisation may want operational endpoints under a consistent, custom path across all of its services (rather than every team's services all defaulting to `/actuator`), and disabling specific endpoints (like `metrics` here) is a simple way to reduce the surface area of what is exposed, alongside the access-control approach Chapter 41 covers.

39.3 OpenFeign — Declarative REST Clients

Every REST client written so far in this course — across both volumes — has used `RestTemplate`: build a URL string, call `exchange(...)` or `getForObject(...)`, manually convert the response. This works, but it is boilerplate that scales linearly with the number of remote calls a consumer needs to make. **OpenFeign** (integrated into Spring Cloud as `spring-cloud-openfeign`) replaces all of that boilerplate with a plain Java *interface*: you declare what the remote API looks like, annotated exactly like a Spring MVC controller, and Spring generates a working REST client implementation of that interface at runtime — the same “declare an interface, let the framework generate a working implementation” pattern already seen for Spring Data JPA repositories in the previous volume, now applied to *consuming* an API instead of a database.

`Demo1008.Feign.Producer.Demo` is a plain producer (nothing new here — the same kind of `@RestController` built repeatedly throughout this course); `Demo1009.Feign.Consumer1.Demo` shows the client side that actually matters for this section:

```

1 @FeignClient(name = "employee", url = "http://localhost:7091/")
2 public interface MyFeignClient {
3
4     @RequestMapping(value = "/emp/controller/getDetails", method = RequestMethod.GET,
5                     produces = MediaType.APPLICATION_JSON_VALUE)
6     public ResponseEntity<List<Employee>> findAll();
7
8     @RequestMapping(value = "/emp/controller/getDetailsById/{id}", method =
9                     RequestMethod.GET,
10                    produces = MediaType.APPLICATION_JSON_VALUE)
11    public ResponseEntity<Employee> findById(@PathVariable("id") Integer
12        employeeId);
13
14    @RequestMapping(value = "/emp/controller/addEmp", method = RequestMethod.POST,
15                    consumes = MediaType.APPLICATION_JSON_VALUE, produces = MediaType.
16                        TEXT_HTML_VALUE)
17    public ResponseEntity<String> addEmployee(@RequestBody Employee employee);
18
19    @RequestMapping(value = "/emp/controller/updateEmp", method = RequestMethod.PUT,
20                    consumes = MediaType.APPLICATION_JSON_VALUE, produces = MediaType.
21                        APPLICATION_JSON_VALUE)
22    public ResponseEntity<Employee> updateEmployee(@RequestBody Employee employee);
23
24    @RequestMapping(value = "/emp/controller/deleteEmp/{id}", method = RequestMethod.
25                    DELETE,
26                    produces = MediaType.APPLICATION_JSON_VALUE)
27    public ResponseEntity<Employee> deleteEmployee(@PathVariable("id") int myId);
28 }

```

```

1 @SpringBootApplication
2 @EnableFeignClients
3 public class FeignConsumer1 {
4     public static void main(String[] args) {
5         SpringApplication.run(FeignConsumer1.class, args);
6     }
7 }

```

```
7 }
```

Notice what is entirely *absent* compared to every `RestTemplate`-based consumer in this course: there is no manual URL string concatenation, no manual `RestTemplate.exchange(...)` call, and — crucially — no manual Jackson conversion step. `ResponseEntity<List<Employee>>` as a return type works correctly here because Feign, unlike a bare `RestTemplate.getForObject(..., List.class)` call from Chapter 37, is generating a real proxy implementation of this specific interface method and *does* retain the full generic type information (`List<Employee>`, not just `List`) needed to deserialise correctly in one step. `@EnableFeignClients` on the application class is what triggers Spring to scan for `@FeignClient`-annotated interfaces and generate their proxy implementations, exactly as `@EnableBatchProcessing` and `@EnableDiscoveryClient` each turn on their own respective piece of auto-configuration elsewhere in this course. `Demo1009_Feign_Consumer2.Demo` repeats this identical interface and wiring under a second consumer instance (`FeignConsumer2`), simply to demonstrate that more than one independent consumer application can each hold its own `@FeignClient` pointed at the same producer.

39.3.1 Feign Combined with Service Discovery

`Demo1010_Eureka_Feign_Producer_Demo` and `Demo1011_Eureka_Feign_Consumer_Demo` combine this chapter's two ideas so far. The producer changes only by becoming Eureka-discoverable, exactly as in Chapter 38:

```
1 @SpringBootApplication
2 @EnableDiscoveryClient
3 public class FeignProducer {
4     public static void main(String[] args) {
5         SpringApplication.run(FeignProducer.class, args);
6     }
7 }
```

and the consumer application enables *both* discovery and Feign together on the same class:

```
1 @SpringBootApplication
2 @EnableDiscoveryClient
3 @EnableFeignClients
4 public class FeignConsumer {
5     public static void main(String[] args) {
6         SpringApplication.run(FeignConsumer.class, args);
7     }
8 }
```

With both annotations present, the natural next step — left as an exercise in this particular demo's own `client` package — is to rewrite `MyFeignClient` from Section 39.3 with *no* `url` attribute at all, just a logical, Eureka-registered service name: `@FeignClient(name = "cst-employee-producer")`. With `url` omitted, Feign, exactly like the `@LoadBalanced RestTemplate` from Chapter 38, resolves that name via the discovery client and load-balances across whatever instances are currently registered — meaning a Feign client gets service discovery and load balancing *automatically*, as a natural consequence of Feign's own Spring Cloud integration, with `@EnableDiscoveryClient` and `@EnableFeignClients` together being the only two annotations that actually need to be present for it.

39.3.2 Feign Error Handling

`Demo10005_Spring_security_FEIGN_Client`'s accompanying `FeignErrorDecoder` shows that a non-2xx response from a Feign-called service does not have to surface only as a generic exception — implementing Feign's `ErrorDecoder` interface lets an application translate a specific

HTTP status code returned by the remote service into a specific, meaningful application exception, giving calling code the same ability to distinguish “not found” from “server error” from “unauthorized” that a `RestTemplate`-based consumer gets from inspecting a `ResponseEntity`’s status directly.

39.4 API Documentation with Swagger

As a system grows into many independently-owned microservices, keeping every team’s understanding of every other team’s REST API accurate becomes its own challenge — a hand-maintained document describing request/response shapes goes stale the moment the code changes and nobody remembers to update it. **Swagger** (the tooling ecosystem built around the **OpenAPI Specification**) solves this by generating interactive, always-up-to-date API documentation directly from the running application’s own annotated controllers.

`Demo10020.Rest_Controller_Full_Deployment_Swagger` pulls in Swagger via two Maven dependencies:

```
1 <dependency>
2   <groupId>io.springfox</groupId>
3   <artifactId>springfox-boot-starter</artifactId>
4   <version>3.0.0</version>
5 </dependency>
6 <dependency>
7   <groupId>io.springfox</groupId>
8   <artifactId>springfox-swagger-ui</artifactId>
9   <version>3.0.0</version>
10</dependency>
```

With these on the classpath (and, in older Springfox versions, a small `@EnableSwagger2` configuration bean describing which packages to scan), Swagger inspects every `@RestController` already in the application — the exact same `@GetMapping`/`@PostMapping`/`@RequestBody`/`@PathVariable` annotations covered in the previous volume’s REST chapter — and automatically generates a browsable `/swagger-ui.html` page listing every endpoint, its HTTP method, its expected request body shape, its possible response codes, and even a “try it out” form to fire a real test request directly from the browser. Because this documentation is generated *from* the controller’s own annotations rather than hand-written separately, it can never drift out of sync with the actual code the way a manually maintained document can — if a developer adds a new `@PostMapping` endpoint or changes a `@RequestBody` type, the very next time the application starts, Swagger’s documentation reflects that change automatically.

39.5 Distributed Tracing with Sleuth and Zipkin

A single logical request in a microservice architecture very often does not stay inside one service — a request might enter through `EmployeeConsumer`, which calls `EmployeeMediator`, which itself calls `EmployeeProducer`. If something goes wrong or runs slowly somewhere in that chain, grepping through three separate services’ independent log files, each with no way to tell which log lines across all three even belong to the *same* originating request, is close to impossible at any real scale. **Distributed tracing** solves this by tagging every request, as it enters the system, with a unique **trace ID** that is then propagated along with the request through *every* service it touches, so that all the log lines and timing data generated anywhere in that chain, for that one logical request, can be correlated together afterward.

Spring Cloud Sleuth is the library that automatically generates and propagates this trace ID (plus a separate **span ID** for each individual hop within the trace) through outgoing calls (including, automatically, calls made via `RestTemplate` or Feign, both already covered in this

course), and injects both IDs directly into the application's own log output. `Demo1015_Employee_Producer_Sleuth`, `Demo1016_Employee_Mediator_Sleuth_Zipkin`, and `Demo1017_Employee_Consumer_Sleuth_Zipkin` form exactly the three-service chain described above, each configured with only:

```
1 server.port = 8090
2 spring.application.name = producer
```

(with `mediator` and `consumer` as the respective `spring.application.name` values in the other two services). Once Sleuth is on the classpath, every log line automatically gains a prefix like `[producer,4d6a1c...,8f21ab...]` — application name, trace ID, span ID — with the *same trace ID* appearing in the consumer's, the mediator's, and the producer's logs for calls that are all really part of one single originating request, purely because Sleuth propagates that trace ID automatically along every outgoing HTTP call in the chain.

Zipkin is the companion tool that actually *collects* and *visualises* this tracing data: each Sleuth-instrumented service reports its trace/span timing information to a running Zipkin server, and Zipkin's own UI then lets a developer search by trace ID and see a full timeline — which service handled which span, how long each hop took, and where in the chain the most time was actually spent — turning what would otherwise be a manual, cross-service log correlation exercise into a single visual timeline.

Why Monitoring and Tracing Matter More as Microservices Grow

A single monolith needs relatively little of this machinery, because one process's own logs and one debugger session already tell most of the story. The moment a request's handling is spread across several independently-deployed services — exactly the trade this course's Chapter 37 introduced — Actuator, Feign, Swagger, and Sleuth/Zipkin stop being optional conveniences and become the only practical way to understand what a running system is actually doing.

Viva Voce — Likely Questions & Answers

Q: What are the three pillars of observability, and what question does each answer?

A: Logs answer what specific events happened and in what order; metrics answer what the aggregate numbers look like over time (rates, latencies, error counts); traces answer what path one specific request took through the system and where its time was actually spent.

Q: Why is no single observability pillar sufficient on its own in a microservice architecture?

A: Logs alone cannot correlate related events across independent services without a shared identifier; metrics alone can show that something degraded but not which specific request or downstream call caused it; and a single trace cannot reveal whether what it shows is a one-off anomaly or part of a broader system-wide trend, which only aggregate metrics can establish.

Q: What does Spring Boot Actuator provide, and why does Spring Boot not expose every Actuator endpoint publicly by default?

A: Actuator adds ready-made operational HTTP endpoints (health, info, metrics, environment, and more) to a Spring Boot application with minimal configuration. Not every endpoint is exposed by default because some reveal internal details (or, like shutdown, can even alter the running application) that should not be publicly reachable without proper access control in a real deployment.

Q: What does `management.endpoint.health.show-details=ALWAYS` change about

the health endpoint?

A: It makes the health endpoint report a full breakdown of each individual health indicator (e.g. database connectivity) rather than only a single overall UP/DOWN status.

Q: What core problem does OpenFeign solve compared to writing REST clients with RestTemplate?

A: It removes the boilerplate of manually building request URLs, invoking `exchange()/getForObject()`, and manually converting the response, by letting the developer declare the remote API as an annotated interface and having Spring generate a working client implementation at runtime – while also correctly retaining generic type information (e.g. `List<Employee>`) that a bare RestTemplate call using a raw `.class` literal cannot.

Q: How does a Feign client obtain service discovery and load balancing “for free”?

A: By naming a logical, registry-registered service name in `@FeignClient` (rather than a literal URL), Feign’s own Spring Cloud integration resolves that name via the discovery client and load-balances across the currently registered instances automatically, the same underlying mechanism as a `@LoadBalanced RestTemplate`.

Q: Why does Swagger-generated API documentation stay accurate over time, unlike a manually written API document?

A: Because it is generated directly from the application’s own live `@RestController` annotations rather than maintained as a separate document; any change to the actual endpoints is automatically reflected the next time the application starts, with no separate manual update step to forget.

Q: What problem does distributed tracing (Sleuth) solve that individual per-service logging cannot?

A: It correlates all the log lines and timing data generated across multiple services that are all really handling one single originating request, by tagging that request with a shared trace ID (and per-hop span IDs) that is automatically propagated through every downstream call in the chain.

Q: What is the difference between a trace ID and a span ID in distributed tracing?

A: A trace ID identifies one entire logical request as it flows through every service it touches; a span ID identifies one individual hop or unit of work within that overall trace, so a single trace can contain multiple spans, one for each service or operation involved.

Q: What role does Zipkin play alongside Sleuth?

A: Sleuth generates and propagates the trace/span IDs and tags log output with them; Zipkin is the separate server that collects this tracing data from every instrumented service and provides a visual timeline UI for inspecting a given trace’s full path and timing across all the services it touched.

Chapter 40

Secure Coding Practices (OWASP)

40.1 Why Secure Coding Is a Separate Topic From Spring Security

The next chapter, Spring Security, is about *authentication and authorization* — deciding *who* is allowed to call an endpoint at all. This chapter is about something that matters even for an endpoint that *is* correctly locked down to the right authenticated user: whether the code behind that endpoint handles the data it receives and returns *safely*. A perfectly authenticated request can still trigger a SQL injection, leak an internal stack trace, or accept malformed data that corrupts application state, if the code behind the endpoint was not written with those risks in mind. Secure coding practice is what closes that gap, and it applies *regardless* of whether Spring Security is present at all.

OWASP (the Open Web Application Security Project) maintains the most widely referenced, community-driven catalogue of common web application security risks — the **OWASP Top 10** — and this chapter’s demos are built directly around several of its most consistently ranked entries: injection, security misconfiguration, and broken access control chief among them.

40.2 Demo 1: Input Validation as the First Line of Defence

Demo1001.SpringBoot.SecureCodingValidation is built to show that Bean Validation — already covered purely as a *correctness* tool in the previous volume and in Chapter 37 — is equally a *security* tool, because unvalidated input is the raw material almost every other injection or corruption vulnerability is built from.

```
1 public class CreateEmployeeRequest {
2
3     @NotBlank(message = "employee name is required")
4     @Size(min = 2, max = 100, message = "employee name must be 2-100 characters")
5     private String employeeName;
6
7     @NotNull(message = "employee ID is required")
8     @Positive(message = "employee ID must be positive")
9     private Integer employeeId;
10
11     @NotNull(message = "salary is required")
12     @Positive(message = "salary must be positive")
13     private double salary;
14
15     @NotNull(message = "department code is required")
16     @Positive(message = "department code must be positive")
17     private Integer departmentCode;
18     // getters and setters ...
19 }
```


Notice this demo introduces a dedicated **DTO** (`CreateEmployeeRequest`, “Data Transfer Object”) specifically for the incoming request payload, entirely separate from the JPA `@Entity` class used for persistence:

```

1 @Entity
2 @Table(name = "employees")
3 public class Employee {
4     @Id
5     @Column(name = "employee_id", nullable = false)
6     private Integer employeeId;
7
8     @Column(name = "employee_name", nullable = false)
9     private String employeeName;
10
11     @Column(nullable = false)
12     private double salary;
13
14     @Column(name = "department_code", nullable = false)
15     private Integer departmentCode;
16     // ...
17 }

```

This separation is itself a secure-coding practice, not just a layering preference: binding a request directly to a JPA `@Entity` risks **mass assignment** — a client sending extra, unexpected JSON fields that happen to match internal entity properties never meant to be externally settable (an auto-generated ID, an internal audit flag, another user’s foreign key) and having them silently bound and persisted. A dedicated request DTO, containing *only* the fields a client is actually meant to supply, closes that door structurally: a field that does not exist on `CreateEmployeeRequest` simply cannot be set by any request body, no matter what extra JSON the client sends.

`@Positive` deserves a specific mention here, distinct from its purely-correctness role in Chapter 37: an `employeeId` or `departmentCode` that is not validated to be positive could otherwise be supplied as a negative number or zero, values that might coincide with special, unintended meanings elsewhere in the system (a sentinel value, an “all records” wildcard in a poorly-written query) — validating the *business-meaningful range* of a value, not just its presence, is exactly the kind of defence-in-depth secure coding asks for.

40.3 Handling Validation Failures Without Leaking Internals

`GlobalExceptionHandler` in this same demo applies the `@ControllerAdvice` pattern from the previous volume’s Spring MVC chapter, specifically to keep validation failure responses informative to a legitimate client *without* leaking anything else:

```

1 @ControllerAdvice
2 public class GlobalExceptionHandler {
3
4     @ExceptionHandler(MethodArgumentNotValidException.class)
5     public ResponseEntity<Object> handleMethodArgumentNotValid(
6         MethodArgumentNotValidException ex) {
7         Map<String, Object> body = new HashMap<>();
8         body.put("errors", ex.getBindingResult().getFieldErrors().stream()
9             .collect(Collectors.toMap(err -> err.getField(), err -> err.
10                 getDefaultMessage(), (a, b) -> a)));
11         return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(body);
12     }
13
14     @ExceptionHandler(ConstraintViolationException.class)
15     public ResponseEntity<Object> handleConstraintViolation(
16         ConstraintViolationException ex) {
17         Map<String, Object> body = new HashMap<>();
18         body.put("errors", ex.getConstraintViolations().stream()
19             .collect(Collectors.toMap(v -> v.getPropertyPath().toString(), v -> v.
20                 getMessage(), (a, b) -> a)));
21     }
22 }

```

```

17     return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(body);
18 }
19 }

```

The secure-coding discipline here is what is *deliberately not* in this response: no stack trace, no internal class name, no SQL fragment, no server file path — only the field name and the human-readable validation message defined on the DTO’s own annotations. This matters because a verbose default error page (or, worse, a raw uncaught exception’s stack trace serialised straight into a JSON response) hands an attacker a free reconnaissance tool: the exact framework version, internal package names, sometimes even fragments of a query, all of which make a targeted attack meaningfully easier to construct. **Security misconfiguration**, of exactly this “the framework’s own default, overly-detailed error page is still switched on in production” variety, is a perennial entry on the OWASP Top 10 precisely because it is so easy to leave unaddressed by accident.

40.4 Secure Database Access — Preventing SQL Injection

Injection — classically SQL injection, but the same underlying idea applies to any place untrusted input is concatenated directly into a command interpreted by another system — has topped or near-topped the OWASP Top 10 for over a decade, and remains one of the most damaging vulnerability classes precisely because it is also one of the easiest to prevent *if* the danger is understood. The vulnerability arises whenever user-supplied input is concatenated directly into a query string, letting an attacker’s input change the *structure* of the query itself rather than merely supplying a value within it — for example, a naively built query like:

```

1 // NEVER DO THIS -- shown here only to name the danger, not to demonstrate working code
2 String query = "SELECT * FROM employees WHERE employee_name = '" + userSuppliedName + "
    ',';

```

would let an attacker supply `employee_name` as something like `' OR '1'='1`, turning the intended “find one specific employee” query into one that matches — and returns — every row in the table, or worse.

`EmployeeRepository` in this same demo shows exactly why every Spring Data JPA technique already taught across this course is, by construction, immune to this class of attack, provided the parameter-binding style shown is actually used:

```

1 @Repository
2 public interface EmployeeRepository extends JpaRepository<Employee, Integer> {
3
4     // Derived query method -- safe parameter binding
5     Employee findById(Integer employeeId);
6
7     // Explicit JPQL using a named parameter -- safe
8     @Query("SELECT e FROM Employee e WHERE e.employeeId = :employeeId")
9     Employee findByIdUsingQuery(@Param("employeeId") Integer employeeId);
10
11     // Native query example -- still safe, because it still uses parameter binding
12     @Query(value = "SELECT * FROM employees WHERE employee_id = :employeeId",
13           nativeQuery = true)
14     Employee findByIdNative(@Param("employeeId") Integer employeeId);
15 }

```

Every one of these three methods — the derived Query Method, the named-parameter JPQL `@Query`, and even the *native* SQL `@Query` — is safe from injection, and for the identical underlying reason: in every case, the parameter’s *value* is handed to the JDBC driver *separately* from the query’s *structure* (as a bound parameter, resolved by the driver itself at execution time),

never spliced into the query text as a string before it is parsed. This is precisely why the previous volume’s Chapter 31 consistently used named (`:employeeId`) or positional (`?1`) parameter placeholders in every `@Query` example rather than string concatenation — what looked there like a stylistic convention is, from a secure-coding standpoint, the entire defence against SQL injection. The *only* way to reintroduce the vulnerability into this same repository would be to abandon parameter binding and manually concatenate a value into the native query string instead — which is exactly why `nativeQuery = true` deserves extra scrutiny in any code review: it is the one place in a Spring Data JPA codebase where a careless developer, reaching for “just write it as SQL”, is most likely to reintroduce string concatenation out of old habit.

The General Principle Behind Parameterised Queries

The defence against injection is never “sanitise the input by removing dangerous characters” as a first-line strategy — character blocklists are notoriously easy to bypass and easy to get subtly wrong. The reliable defence is structural: keep the query’s *structure* and the query’s *data* in two entirely separate channels from the very start, so that no value supplied as data can ever be interpreted as part of the query’s structure, no matter what characters it contains.

40.5 Output Encoding — Preventing Cross-Site Scripting (XSS)

Injection, covered above, is about untrusted input corrupting a *query*. **Cross-Site Scripting (XSS)** is the same underlying idea — untrusted input changing the structure of something it should only ever be *data* within — applied to a *web page* instead of a database query. If a value a user submitted (a comment, a display name, a search term) is later rendered back into an HTML page *without being escaped*, and that value happens to contain something like `<script>...</script>`, the browser rendering that page cannot tell the difference between markup the application intended and markup an attacker smuggled in through user input — it simply executes whatever script tag it finds, in the context of the victim’s own logged-in session, on the victim’s own browser.

XSS Variant	How the Malicious Script Reaches the Victim
Stored (Persistent) XSS	The malicious input is saved by the application (e.g. as a comment or profile field) and later rendered, unescaped, to <i>every</i> user who views that stored content — the most dangerous variant, since one successful injection can affect every subsequent visitor
Reflected XSS	The malicious input is part of the request itself (commonly a query parameter) and is reflected back, unescaped, in that same response — typically requires tricking a specific victim into clicking a crafted link
DOM-based XSS	The vulnerability exists entirely in client-side JavaScript that unsafely writes untrusted data (e.g. part of the URL) directly into the page’s DOM, without the raw HTML response from the server ever containing the malicious payload at all

The defence is the mirror image of parameterised queries from the previous section, applied at the point content is rendered rather than the point it is queried: **output encoding** (also called escaping) — converting characters that are structurally significant in HTML (`<`, `>`, `&`, `"`, `'`) into their harmless textual equivalents (`<`, `>`, and so on) at the moment untrusted data is written into a page, so that whatever a user submitted is guaranteed to be treated as inert, displayable *text*, never as executable markup. Modern templating engines — including the JSP tag libraries used throughout the earlier volume’s Spring MVC chapter, and virtually

every modern front-end templating system — escape output by default precisely because this class of vulnerability is so easy to introduce accidentally otherwise; the dangerous, deliberate exception in most such systems is an explicit “render raw/unescaped HTML” directive, which should be reserved only for content the application itself fully controls and trusts, never for anything a user supplied.

Injection and XSS Are the Same Lesson, Applied Twice

Both vulnerabilities have the identical underlying shape: untrusted data was allowed to be interpreted as *structure* (a query’s syntax; a page’s markup) rather than being strictly confined to *content*. Parameterised queries keep a database query’s structure and data in separate channels; output encoding keeps a web page’s markup and data in separate channels. Recognising this shared shape is more valuable than memorising the two defences as unrelated facts, because the same underlying discipline — never let untrusted input be interpreted as anything other than inert data — generalises to every other injection-family vulnerability an application might encounter beyond just these two.

40.6 Secure API and Web Practices — the Case for Authentication

Demo1002.SpringBoot.SecureCoding.Authentication.Need exists to make a single point unmissable, by *not* fixing the problem it demonstrates:

```

1  @RestController
2  public class EmployeeController {
3
4      @Autowired
5      private EmployeeServiceImpl employeeService;
6
7      private void logInsecureAccess(String endpoint) {
8          System.out.println("[INSECURE DEMO] Unauthenticated access allowed to endpoint: "
9              + endpoint + ". This is unsafe in production.");
10     }
11
12     @RequestMapping(value = "emp/controller/getDetails", method = RequestMethod.GET,
13         produces = MediaType.APPLICATION_JSON_VALUE)
14     public ResponseEntity<Collection<Employee>> getEmployeeDetails() {
15         logInsecureAccess("GET /emp/controller/getDetails");
16         Collection<Employee> listEmployee = employeeService.getEmployeeDetails();
17         return new ResponseEntity<Collection<Employee>>(listEmployee, HttpStatus.OK);
18     }
19
20     @RequestMapping(value = "/emp/controller/addEmp", method = RequestMethod.POST,
21         consumes = MediaType.APPLICATION_JSON_VALUE, produces = MediaType.TEXT_HTML_VALUE)
22     public ResponseEntity<String> addEmployee(@RequestBody Employee employee) {
23         logInsecureAccess("POST /emp/controller/addEmp");
24         int id = employeeService.addEmployee(employee);
25         return new ResponseEntity<String>("Employee added successfully with id:" + id,
26             HttpStatus.CREATED);
27     }
28     // updateEmployee(...), deleteEmployee(...) follow the identical pattern
29 }

```

Every single endpoint here — including `addEmp`, a data-mutating `POST` — is reachable by *anyone* who can send an HTTP request to this server, with no login, no token, no credential of any kind. `logInsecureAccess(...)`’s own message spells out the point in the code itself: this is **broken access control** in its most literal form — not a flaw in *which* roles can do *what* (the more subtle version of the problem, addressed by Spring Security’s expression-based authorization in the next chapter), but the far more basic absence of *any* access control whatsoever. This

demo is deliberately paired with `Demo1003_Rest_JPA_Repository_Security` (covered in full in the next chapter), which takes this identical, unprotected controller and locks it down — the “before” and “after” of this chapter and the next existing side by side on purpose.

40.7 The OWASP Top 10, at a Glance

The demos in this chapter deliberately touch only a handful of the OWASP Top 10’s categories in working code, but it is worth naming the fuller list, since interview questions on secure coding very often ask for it directly, and each entry maps onto a concrete habit already built or about to be built in this course:

OWASP Category (representative)	Addressed in This Course By
Broken Access Control	Spring Security’s expression-based authorization (next chapter)
Cryptographic Failures	<code>BCryptPasswordEncoder</code> for password storage (next chapter)
Injection	Parameterised Spring Data JPA queries (this chapter)
Injection — Cross-Site Scripting (XSS)	Output encoding at render time; default auto-escaping in JSP tag libraries (this chapter)
Insecure Design	Bounded contexts and a deliberately-modelled domain (Chapter 36)
Security Misconfiguration	Not leaking stack traces/internal details via <code>@ControllerAdvice</code> (this chapter); restricting exposed Actuator endpoints (Chapter 39)
Vulnerable and Outdated Components	Maven dependency management and version pinning via a parent POM (the earlier volume’s Chapter 27)
Identification and Authentication Failures	Basic Authentication and a proper <code>AuthenticationEntryPoint</code> (next chapter)
Software and Data Integrity Failures	Validated, DTO-bound request payloads rather than binding directly to persistence entities (this chapter)

Viva Voce — Likely Questions & Answers

Q: Why does secure coding matter as a topic separate from Spring Security?

A: Spring Security governs who is allowed to call an endpoint at all; secure coding practice governs whether the code behind that endpoint handles data safely once a call is permitted. A perfectly authenticated endpoint can still be vulnerable to injection, information leakage, or data corruption if the underlying code was not written defensively.

Q: What is Cross-Site Scripting (XSS), and how is it structurally similar to SQL injection?

A: XSS occurs when untrusted user input is rendered into an HTML page without being escaped, letting a browser interpret it as executable markup rather than inert text. It shares the same underlying shape as SQL injection: untrusted data being allowed to change the structure of something (a page’s markup, a query’s syntax) that it should only ever appear within as data.

Q: What is the difference between Stored and Reflected XSS?

A: Stored XSS persists the malicious input in the application (e.g. a saved comment) so it affects every subsequent user who views that content; Reflected XSS is carried entirely within a single request (typically a query parameter) and reflected back unescaped in that same response, generally requiring a victim to be tricked into clicking a specially crafted link.

Q: What is output encoding, and how does it defend against XSS?

A: Output encoding converts characters that are structurally significant in HTML (such as `<` and `>`) into harmless textual equivalents at the point untrusted data is rendered into a page, ensuring that data is always treated as inert displayable text rather than executable markup, regardless of what characters it contains.

Q: Why does the secure coding demo bind incoming requests to a dedicated DTO (CreateEmployeeRequest) rather than directly to the JPA @Entity class?

A: To prevent mass assignment – a client sending extra JSON fields that happen to match internal entity properties never meant to be externally settable. A DTO containing only the fields a client should legitimately supply makes it structurally impossible for unexpected fields to be bound and persisted.

Q: Why is @Positive validation on an identifier or code field considered a security practice, not just a correctness check?

A: Because an unvalidated negative or zero value could coincide with unintended special meanings elsewhere in the system (a sentinel value or wildcard in a poorly written query), so validating the business-meaningful range of a value is a defence-in-depth measure, not merely a data-quality one.

Q: Why does the GlobalExceptionHandler in the secure coding demo return only field names and validation messages, deliberately omitting stack traces or internal details?

A: Because a verbose, detailed error response (a raw stack trace, internal class or package names, fragments of a query) hands an attacker free reconnaissance information about the system's internals, which is exactly the kind of security misconfiguration the OWASP Top 10 warns against.

Q: What is the structural reason parameterised queries (named parameters, positional parameters, or Spring Data derived query methods) are immune to SQL injection?

A: Because the query's value and the query's structure are kept in two entirely separate channels – the parameter value is bound and passed to the JDBC driver separately from the already-parsed query text – so no value supplied as data can ever be interpreted as part of the query's structure, regardless of what characters it contains.

Q: Why is a native query (nativeQuery = true) worth extra scrutiny in a code review, even though it can be written safely?

A: Because it is the one place in a Spring Data JPA codebase where a developer, reaching for "just write it as raw SQL", is most likely to fall back into manually concatenating a value into the query string out of old habit, reintroducing the exact vulnerability that named or positional parameter binding otherwise structurally prevents.

Q: Why is "sanitising" input by stripping dangerous characters considered a weaker defence against injection than parameterised queries?

A: Because character blocklists are notoriously easy to bypass or to implement subtly incorrectly, whereas parameterised queries prevent the vulnerability structurally, by design,

regardless of what characters the input happens to contain.

Q: In the Authentication_Need demo, what specific OWASP category does an endpoint reachable by anyone, with no login or token required, exemplify?

A: Broken Access Control in its most basic form – not a flaw in which roles can perform which actions, but the complete absence of any access control at all on the endpoint.

Chapter 41

Spring Security

41.1 The Need for Security

Demo1002.SpringBoot.SecureCoding.Authentication.Need from the previous chapter ended with every endpoint of an `EmployeeController` reachable by anyone, with no credential of any kind. **Spring Security** is the framework-level answer to exactly that gap: a comprehensive, configurable layer that intercepts every incoming request *before* it reaches a controller, and decides two separate questions in sequence:

- **Authentication** — *who* is making this request? (Is this really `msd_admin`, and can they prove it?)
- **Authorization** — now that we know who they are, *are they allowed* to do the specific thing this request is asking for? (Is `msd_admin` allowed to delete an employee? Is `msd_user` allowed to only read, not write?)

These are deliberately two separate questions, answered in that order: a request always has to be authenticated (or explicitly permitted as anonymous) *before* authorization can even be evaluated, since “are they allowed to do this” is meaningless without first establishing who “they” are.

41.2 How a Request Actually Reaches a Controller Under Spring Security

Every request touched by `@EnableWebSecurity` passes through a chain of **Servlet Filters** before it ever reaches Spring MVC’s own `DispatcherServlet` (the Front Controller from the earlier volume’s Chapter 32). Spring Security registers itself into the Servlet container’s own filter chain through a single bridging filter, `DelegatingFilterProxy`, which in turn delegates to Spring Security’s own internal `FilterChainProxy` — and it is *that* proxy which actually runs a whole ordered sequence of Spring-managed security filters, each with one specific, narrow job:

Filter (representative, in typical order)	Responsibility
SecurityContextPersistenceFilter	Loads any existing security context (e.g. from a session) at the start of the request, and clears it at the end
UsernamePasswordAuthenticationFilter / BasicAuthenticationFilter	Actually attempts authentication for this specific request, based on whichever mechanism is configured (a submitted

Reading the demo's own `SecurityFilterChain` @Bean against this table makes it concrete: `.httpBasic()` is what registers `BasicAuthenticationFilter` into this chain in the first place; `.authenticationEntryPoint(entryPoint)` is what tells `ExceptionTranslationFilter` which `AuthenticationEntryPoint` to hand off to on failure (Section 41.2.3's `MyBasicAuthenticationEntryPoint`); and every `.antMatchers(...).access(...)` rule is configuration that `FilterSecurityInterceptor/AuthorizationManager` actually evaluates, request by request, at the very end of the chain — *after* authentication has already succeeded or failed earlier in the same chain. Only once a request has passed *every* filter in this chain does it finally reach the `DispatcherServlet`, and from there, the target `@RestController` method exactly as built throughout the rest of this course.

41.2.1 CSRF — What It Is, and Why a Stateless REST API Disables It

`http.csrf().disable()` in the demo's configuration is worth explaining rather than treating as boilerplate. **Cross-Site Request Forgery (CSRF)** is an attack where a malicious website, visited by a victim who is *already* authenticated to a target site via a session cookie, silently triggers a request to that target site (for example, an auto-submitting hidden form) — and because browsers automatically attach a site's cookies to any request sent to that site, regardless of which page triggered the request, the target site's server has no way to tell that this particular request wasn't something the legitimate, logged-in user actually intended. Spring Security's default CSRF protection defends against exactly this by requiring a special, unpredictable **CSRF token** to accompany any state-changing request, a token that a malicious third-party site has no way to obtain or forge.

CSRF protection is specifically a defence against *cookie/session-based* authentication being silently exploited by a browser's automatic cookie-attaching behaviour. This chapter's demos, as Section 41.2.3 already established, use `SessionCreationPolicy.STATELESS` together with HTTP Basic Authentication — there is no session cookie for a browser to automatically attach on the victim's behalf in the first place, since every single request must independently carry its own `Authorization` header, which a malicious third-party page has no way to silently attach either. This is precisely why `.csrf().disable()` is a defensible choice *here specifically*, rather than a security shortcut taken for convenience: CSRF protection exists to guard a vulnerability that a genuinely stateless, header-authenticated API does not have in the first place. Disabling it on an application that *does* still rely on session cookies for authentication, purely to make testing more convenient, would be a real and different mistake — the lesson is not “CSRF protection is safe to disable”, but specifically “CSRF protection defends against a session-cookie-specific attack, so its relevance depends entirely on which authentication mechanism is actually in use.”

41.2.2 Beyond Basic Authentication — Token-Based Authentication in Production

Every demo in this chapter uses HTTP Basic Authentication, which is an excellent teaching vehicle — it makes the `Authorization` header, the encoding, and the stateless philosophy all directly visible in a single, simple header value — but a production system rarely uses *raw* Basic Authentication for its actual client-facing API, for a fairly direct reason: it requires the raw username and password to be resent, in reversible Base64, on *every single request*, indefinitely, which maximises how often (and how many places) the actual long-lived credential is transmitted and potentially exposed. Production systems instead very commonly authenticate once (still typically via a username/password check, functionally similar to what this chapter's `AuthenticationProvider` already does internally) and issue the client a short-lived, purpose-built **token** — very often a **JSON Web Token (JWT)** — to present on every subsequent request *instead of* the raw password. A JWT is a compact, digitally-signed (not encrypted, but tamper-evident) block of claims — typically including the authenticated username and

granted roles, plus an expiry time — that a server can verify *without* needing to look anything up in a database or session store on every request, since the signature alone proves the token hasn't been altered since the authentication server issued it. This preserves exactly the same statelessness this chapter's demos already established as valuable for a scaled-out microservice (Section 41.2.3), while removing the specific downside of resending a long-lived raw credential on every call — a JWT can simply be left to expire and be re-issued, without the user's actual password ever needing to be involved again until it does.

41.3 Basic Authentication and the Security Filter Chain

Demo10001_Rest_JPA_Repository_Security (and its near-identical sibling in the Secure Coding demos, Demo1003_Rest_JPA_Repository_Security) secures the exact same `EmployeeController` left wide open in the previous chapter, using **HTTP Basic Authentication** — the simplest standard authentication scheme, where the client sends a username and password, Base64-encoded (not encrypted — Base64 is trivially reversible, which is precisely why Basic Authentication is only considered acceptably secure when the entire connection itself is also protected by HTTPS/TLS), in an `Authorization` request header on every single request.

```

1  @Configuration
2  @EnableWebSecurity
3  public class SpringWebSecurityConfig {
4
5      @Autowired
6      private MyBasicAuthenticationEntryPoint entryPoint;
7
8      @Bean
9      public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
10         http.csrf().disable();
11
12         http.authorizeRequests()
13             .antMatchers("/emp/controller/addEmp**").access("hasRole('MSD_ADMIN')")
14             .antMatchers("/emp/controller/updateEmp**").access("hasRole('MSD_ADMIN'
15                 ')")
16             .antMatchers("/emp/controller/deleteEmp/**").access("hasRole('MSD_ADMIN'
17                 ')")
18             .antMatchers("/emp/controller/getDetails**").access("hasAnyRole('MSD_ADMIN', 'MSD_DBA', 'MSD_USER'
19                 ')")
20             .antMatchers("/emp/controller/getDetailsById/**").access("hasRole('MSD_ADMIN') or hasRole('MSD_DBA'
21                 ')")
22             .and().sessionManagement().sessionCreationPolicy(SessionCreationPolicy.STATELESS)
23             .and().httpBasic().realmName(MyBasicAuthenticationEntryPoint.REALM)
24             .authenticationEntryPoint(entryPoint);
25         return http.build();
26     }
27
28     @Autowired
29     public void configureGlobal(AuthenticationManagerBuilder auth) throws Exception {
30         auth.inMemoryAuthentication()
31             .passwordEncoder(new BCryptPasswordEncoder())
32             .withUser("msd_user").password(new BCryptPasswordEncoder().encode("msd_user"))
33             .roles("MSD_USER").and()
34             .withUser("msd_admin").password(new BCryptPasswordEncoder().encode("msd_admin"))
35             .roles("MSD_ADMIN").and()
36             .withUser("msd_dba").password(new BCryptPasswordEncoder().encode("msd_dba"))
37             .roles("MSD_DBA");
38     }
39 }

```

`@EnableWebSecurity` is this chapter's own version of the “one annotation turns on a whole subsystem” pattern seen repeatedly throughout this course (`@EnableBatchProcessing`, `@EnableDiscoveryClient`, `@EnableFeignClients`) — it activates Spring Security's web-request filtering machinery, and the `SecurityFilterChain` `@Bean` is where that machinery is actually configured.

41.3.1 Reading the `authorizeRequests()` Rules

Every `.antMatchers(...).access(...)` line is one authorization rule, matched *in the order they are declared* — the first matching pattern for a given request wins, which is why the most specific patterns generally need to come before more general ones. `antMatchers("/emp/controller/addEmp**")` matches the URL pattern; `.access("hasRole('MSD_ADMIN')")` is a **Spring Security Expression Language (SpEL for security)** expression evaluated against the authenticated user's granted authorities. `hasRole(...)` checks for exactly one specific role; `hasAnyRole(...)` checks whether the user holds *any one* of several listed roles; and a full boolean expression like `hasRole('MSD_ADMIN')` or `hasRole('MSD_DBA')` composes these checks exactly as any other boolean logic would, letting authorization rules be as fine-grained as the business actually requires — notice that the demo deliberately gives *different* roles different levels of access to different endpoints (only `MSD_ADMIN` can write; `MSD_ADMIN` or `MSD_DBA` can look up one specific employee; all three roles can list every employee), which is authorization doing real, meaningful work rather than a single all-or-nothing gate.

41.3.2 Password Encoding — Never Storing Plaintext

`BCryptPasswordEncoder` appears twice: once as the configured `passwordEncoder(...)` Spring Security itself will use to check a submitted password against a stored one, and once used directly to *encode* each in-memory user's password before it is stored. **BCrypt** is a purpose-built password-hashing algorithm — not a general-purpose hash like MD5 or SHA-256 — specifically designed to be *slow* and to incorporate an automatically-generated random **salt** into every hash it produces. Both properties matter directly for security: a fast hash function (like a bare SHA-256 of a password) lets an attacker who obtains the hash try billions of guesses per second; BCrypt's deliberate slowness makes that same brute-force attack orders of magnitude more expensive. The random salt means two users who happen to choose the identical password still end up with two completely different stored hashes, which defeats a precomputed **rainbow table** attack (a lookup table mapping common password hashes back to their plaintext) that would otherwise instantly reveal any password matching a pre-hashed common value.

A Password Is Never Stored, and Never Decrypted

Spring Security never has to “decrypt” a stored password to check a login attempt, because BCrypt is a one-way hash, not reversible encryption. Instead, `passwordEncoder.matches(rawPasswordSubmitted, storedHash)` re-hashes the submitted password (reusing the salt embedded in the stored hash) and compares the two hashes. A system's own developers, and anyone who later steals its database, should never be able to recover a user's actual plaintext password from what is stored — only confirm whether a given guess matches.

41.3.3 Session Management — Why REST APIs Go Stateless

`.sessionManagement().sessionCreationPolicy(SessionCreationPolicy.STATELESS)` tells Spring Security not to create or rely on an `HttpSession` at all for authentication state. This is a deliberate, characteristic choice for a REST API (as opposed to a traditional server-rendered web application, which typically *does* rely on a session cookie to remember a logged-in user between requests): every single request to a stateless REST API must re-authenticate itself independently — which Basic Authentication naturally does anyway, since the **Authorization** header is sent on *every* request by design. Statelessness also matters directly for the scaled-out, multi-instance microservices built in Chapter 38: if authentication state lived in a server-side session, a load-balanced request landing on a *different* instance than the one that created that session would find the new instance has no idea who the user is, unless that session state were

somehow shared across every instance — an entirely avoidable complication that a stateless authentication scheme like Basic Authentication (or, in a fuller production system, a signed token such as a JWT) sidesteps by design.

41.3.4 A Proper 401 Response — the `AuthenticationEntryPoint`

```

1 @Component
2 public class MyBasicAuthenticationEntryPoint extends BasicAuthenticationEntryPoint {
3
4     public static final String REALM = "MYREALM1";
5
6     @Override
7     public void afterPropertiesSet() {
8         setRealmName(REALM);
9         super.afterPropertiesSet();
10    }
11
12    @Override
13    public void commence(HttpServletRequest request, HttpServletResponse response,
14        AuthenticationException authException) throws IOException {
15        response.addHeader("WWW-Authenticate", "Basic realm=" + getRealmName() + "");
16        response.setStatus(HttpServletResponse.SC_UNAUTHORIZED);
17        PrintWriter writer = response.getWriter();
18        writer.println("Client Message - HTTP Status 401 - " + authException.getMessage());
19    }
20 }

```

An `AuthenticationEntryPoint` is the component Spring Security invokes specifically when an *unauthenticated* request reaches a protected endpoint — `commence(...)` is what actually builds the 401 Unauthorized response the client receives. Without a custom one, Spring Security's own default behaviour is used instead, which is functionally similar but far less informative; writing a custom `AuthenticationEntryPoint`, exactly like writing a custom `GlobalExceptionHandler` in the previous chapter, is how an application controls precisely what an unauthenticated caller is told (and, just as importantly, what it is *not* told) rather than leaving that response to a generic framework default. The `WWW-Authenticate: Basic realm=...` response header is itself part of the HTTP Basic Authentication standard — it is what causes a browser hitting this endpoint directly to pop up its own native username/password prompt.

41.4 Authenticating Against a Database — a Real Authentication Provider

The in-memory `auth.inMemoryAuthentication()` configuration above is fine for a demo, but a real application's users obviously live in a database, not hardcoded Java code. `Demo10002_Rest_JPA_Repository` shows exactly that upgrade, changing only the `configureGlobal(...)` method:

```

1 @Configuration
2 @EnableWebSecurity
3 @PropertySource("classpath:com/accenture/lkm/resources/cst_conn.properties")
4 public class SpringWebSecurityConfig {
5
6     @Value("${cst_db_driver}")
7     private String driverName;
8     @Value("${cst_db_url_sec}")
9     private String securityUrl;
10    @Value("${cst_user}")
11    private String userName;
12    @Value("${cst_password}")
13    private String password;
14
15    public DriverManagerDataSource getSecurityDataSource() {
16        DriverManagerDataSource dataSource = new DriverManagerDataSource();
17        dataSource.setDriverClassName(driverName);
18    }
19 }

```

```

18     dataSource.setUrl(securityUrl);
19     dataSource.setUsername(userName);
20     dataSource.setPassword(password);
21     return dataSource;
22 }
23
24 @Autowired
25 public void configureGlobal(AuthenticationManagerBuilder auth) throws Exception {
26     auth.jdbcAuthentication()
27         .dataSource(getSecurityDataSource())
28         .usersByUsernameQuery("select username, password, enabled from msd_users
29                               where username=?")
30         .authoritiesByUsernameQuery("select username, role from msd_user_roles
31                                    where username=?");
32 }
33
34 @Bean
35 public static PropertySourcesPlaceholderConfigurer propertyConfigInDev() {
36     return new PropertySourcesPlaceholderConfigurer();
37 }

```

`auth.jdbcAuthentication()` is Spring Security's built-in **JDBC-based AuthenticationProvider** — rather than an application writing its own custom logic to check a submitted password against a database row, this provider is told, via two plain SQL queries, exactly *where* to find a user's stored (already-BCrypt-hashed) password and enabled status, and exactly *where* to find that same user's granted roles, and it handles the actual authentication decision internally using those two queries. Notice the property values themselves (the JDBC driver class, the security database's URL, its own credentials) are pulled from an externalised `cst.conn.properties` file via `@PropertySource` and `@Value` — the same externalisation discipline from the earlier volume's Spring Core chapter, applied here to what are, appropriately, some of the most sensitive configuration values in the entire application. `PropertySourcesPlaceholderConfigurer` registered explicitly as a `static @Bean` is a small but easy-to-miss detail: a `BeanFactoryPostProcessor` (which is what this ultimately is, per the earlier volume's Spring Core chapter) must be instantiated *very* early in the container's startup sequence — before most other beans are even processed — which is exactly why Spring requires this particular kind of bean method to be declared `static`.

Same AuthenticationManagerBuilder, Different Provider

Notice that switching from in-memory to database-backed authentication changed *only* `configureGlobal(...)`. Every other piece of this security configuration — the `authorizeRequests()` rules, the `AuthenticationEntryPoint`, the stateless session policy — is completely unchanged, because all of it operates purely in terms of the *authenticated user and their granted roles*, regardless of which underlying `AuthenticationProvider` actually established who that user is. This is dependency inversion again, in essentially the same shape seen throughout this entire course: the authorization logic depends only on the abstraction (an authenticated `Authentication` object with granted authorities), not on which concrete mechanism produced it.

41.5 Consuming a Secured REST API

Every client covered so far in this course — `RestTemplate` in the previous volume, Feign in Chapter 39 — was calling an *open* endpoint. Once the endpoint itself is secured, as in this chapter, the client must supply valid credentials on every request, or every call fails with `401 Unauthorized`.

41.5.1 RestTemplate Against a Secured Endpoint

Demo10004.Spring_security_Rest_Template shows exactly how those credentials are attached, using the raw Authorization header this chapter's server side expects:

```

1 private static HttpHeaders getHeaders() {
2     String username = "db_dba";
3     String password = "db_dba";
4     byte[] encodedBytes = Base64Utils.encode((username + ":" + password).getBytes());
5     String authHeader = "Basic " + new String(encodedBytes);
6
7     HttpHeaders headers = new HttpHeaders();
8     headers.add("Authorization", authHeader);
9     headers.setAccept(Arrays.asList(MediaType.APPLICATION_JSON, MediaType.TEXT_HTML));
10    return headers;
11 }
12
13 public static HttpEntity<String> getSecurityEntity() {
14     return new HttpEntity<String>(getHeaders());
15 }
16
17 private void getEmployee() {
18     RestTemplate restTemplate = new RestTemplate();
19     restTemplate.setErrorHandler(new MyErrorHandler());
20     ResponseEntity<Employee> emp = restTemplate.exchange(
21         REST_SERVICE_URI + "getDetailsById/1009", HttpMethod.GET, getSecurityEntity(),
22         Employee.class);
23     System.out.println(emp.getBody());
24 }

```

This is the exact same Base64-encoding step (username:password, Base64-encoded, prefixed with "Basic ") that the server side's MyBasicAuthenticationEntryPoint expects to find, wrapped into an HttpHeaders object and attached as an HttpEntity passed into RestTemplate.exchange(...) — this is why exchange(...), rather than the simpler getForObject(...) used for open endpoints in earlier chapters, is required here: getForObject(...) has no parameter for attaching custom request headers at all, while exchange(...) accepts a full HttpEntity carrying whatever headers a given request needs. restTemplate.setErrorHandler(new MyErrorHandler()) additionally installs a custom ResponseErrorHandler, so that a 401 (or any other non-2xx) response from the secured server can be handled deliberately by the client, rather than only surfacing as a generic runtime exception.

41.5.2 Feign Against a Secured Endpoint

Demo10005.Spring_security_FEIGN_Client shows the identical requirement — attaching credentials to every call — expressed through Feign's declarative style instead:

```

1 @FeignClient(name = "employee", url = "http://localhost:8095/emp/controller")
2 public interface MyFeignClient {
3
4     @RequestMapping(value = "/getDetailsById/{id}", method = RequestMethod.GET)
5     ResponseEntity<Employee> findByIdByEmployeeId(@RequestHeader("Authorization") String
6         header,
7
8         @PathVariable("id") Integer employeeId);
9
10    // ...
11 }

```

```

1 @SpringBootApplication
2 @EnableFeignClients
3 public class FeignConsumer {
4
5     @Autowired
6     private MyFeignClient feignClient;
7
8     private String getAuthorizationHeaders() {
9         String username = "db_user";
10        String password = "db_user";

```



```

11     byte[] encodedBytes = Base64Utils.encode((username + ":" + password).getBytes()
12     );
13     return "Basic " + new String(encodedBytes);
14 }
15 public void getEmployee() {
16     System.out.println(feignClient.findByEmployeeId(getAuthorizationHeaders(),
17     1001).getBody());
18 }

```

Rather than an `HttpHeaders` object built and attached at the point of the call (as `RestTemplate` required), `@RequestHeader("Authorization") String header` declares, directly on the Feign interface’s own method signature, that this call always needs an `Authorization` header value supplied by the caller — the same declarative, “describe the contract, let the framework handle the mechanics” style Feign brought to REST calls generally in Chapter 39, now extended naturally to authentication headers as well.

Viva Voce — Likely Questions & Answers

Q: What is the difference between authentication and authorization, and why must authentication happen first?

A: Authentication establishes who is making a request; authorization decides whether that already-identified caller is permitted to do the specific thing being requested. Authorization is meaningless without first knowing who the caller is, so authentication must always be resolved first.

Q: What role does `DelegatingFilterProxy` play in Spring Security?

A: It is the bridging Servlet Filter that plugs Spring Security into the Servlet container’s own filter chain, delegating to Spring Security’s internal `FilterChainProxy`, which then runs the actual ordered sequence of Spring-managed security filters against every incoming request.

Q: Which filter in the security chain is actually responsible for enforcing the `authorizeRequests()/hasRole(...)` rules, and when does it run relative to authentication?

A: `FilterSecurityInterceptor` (or, in current Spring Security, `AuthorizationFilter`) enforces those rules, and it runs as the final filter in the chain, after authentication has already succeeded or failed earlier in the same chain.

Q: What attack does CSRF protection defend against, and why does it specifically target session-cookie-based authentication?

A: Cross-Site Request Forgery exploits the fact that browsers automatically attach a site’s cookies to any request sent to that site regardless of which page triggered it, letting a malicious site silently trigger authenticated actions on a victim’s behalf. It specifically targets cookie/session-based authentication because that automatic cookie attachment is exactly the browser behaviour the attack relies on.

Q: Why is disabling CSRF protection considered defensible for the stateless, Basic-Authentication-based APIs in this chapter’s demos, specifically?

A: Because there is no session cookie for a browser to automatically attach on a victim’s behalf in the first place — every request must independently carry its own `Authorization` header, which a malicious third-party page has no way to silently attach. CSRF protection defends against a vulnerability this particular authentication scheme does not have.

Q: Why do production systems commonly prefer a token like a JWT over raw HTTP Basic Authentication for ongoing API access?

A: Basic Authentication requires resending the raw username and password, in reversible Base64, on every single request indefinitely. A JWT is issued once after an initial authentication and then presented on subsequent requests instead of the raw password, limiting how often the actual long-lived credential is transmitted, while a server can still verify the token's signature without a database or session lookup on every request.

Q: Why is HTTP Basic Authentication only considered acceptably secure when the connection is also protected by HTTPS/TLS?

A: Because Basic Authentication only Base64-encodes the username and password, which is trivially reversible, not encrypted; without TLS protecting the connection itself, anyone able to observe the network traffic could recover the plaintext credentials directly from the encoded header.

Q: Why does antMatchers() rule ordering matter in a Spring Security configuration?

A: Because the first matching pattern for a given request is the one applied; a more general pattern declared before a more specific one could unintentionally match and take effect first, so more specific rules generally need to be declared ahead of more general ones.

Q: Why is BCrypt used for password storage instead of a general-purpose hash function like SHA-256?

A: BCrypt is deliberately slow, which makes brute-force guessing attacks against a stolen password hash far more expensive; it also incorporates an automatically generated random salt into every hash, so identical passwords produce different stored hashes, defeating precomputed rainbow-table attacks that a fast, unsalted hash would remain vulnerable to.

Q: Does Spring Security ever "decrypt" a stored password to check a login attempt?

A: No – BCrypt is a one-way hash, not reversible encryption. Spring Security instead re-hashes the submitted password (using the salt embedded in the already-stored hash) and compares the two resulting hashes for a match, never recovering the original plaintext password from storage.

Q: Why do REST APIs typically use SessionCreationPolicy.STATELESS rather than relying on an HttpSession for authentication?

A: Because a stateless scheme requires no server-side session state to be shared across instances, which matters directly for load-balanced, scaled-out microservices – a session created on one instance would otherwise be invisible to a different instance that a later request from the same client happens to land on.

Q: What is the role of a custom AuthenticationEntryPoint like MyBasicAuthenticationEntryPoint?

A: It is invoked specifically when an unauthenticated request reaches a protected endpoint, and its commence() method builds the actual 401 Unauthorized response returned to the client – allowing the application to control exactly what an unauthenticated caller is told, rather than relying on Spring Security's generic default response.

Q: What changes, and what stays the same, when switching a Spring Security configuration from in-memory authentication to JDBC/database-backed authentication?

A: Only the configureGlobal(...) method's AuthenticationManagerBuilder configuration changes, replacing inMemoryAuthentication() with jdbcAuthentication() pointed at a DataSource and two SQL queries. Every other part of the configuration – the authorizeRequests()

authorization rules, the `AuthenticationEntryPoint`, the stateless session policy – remains unchanged, since they all operate purely on the resulting authenticated user and roles, regardless of which provider produced them.

Q: Why must a client calling a secured REST endpoint with `RestTemplate` use `exchange(...)` rather than `getForObject(...)`?

A: Because `getForObject(...)` provides no way to attach custom request headers, while `exchange(...)` accepts a full `HttpEntity` that can carry the `Authorization` header (and any other headers) the secured endpoint requires.

Q: How does a Feign client attach authentication credentials to a secured call, compared to a `RestTemplate`-based client?

A: Rather than building an `HttpHeaders` object at the point of the call, a Feign client declares the requirement directly on the interface method's own signature via a parameter annotated `@RequestHeader("Authorization")`, letting the caller supply the header value as a plain method argument while Feign handles attaching it to the actual HTTP request.

Chapter 42

Practice Activities & Assignments

42.1 What This Chapter Covers

Following the same convention established in the previous volume, this chapter walks through the hands-on practice material supplied alongside this volume's instructor-led demos: `Spring_Boot_Assignment` and its companion `Unit-25SpringBootTestActivity2`, two graduated stages of the same POA (Point of Activity) checklist-style exercise, built around an `Order` domain rather than the `Employee/Customer` domains used in the previous volume's assignments — giving a trainee a fresh domain to apply the identical Spring Boot REST techniques to, rather than simply repeating a domain they may already have memorised the shape of.

42.2 Assignment Stage 1 — A Single Endpoint

`Assignment_10005_1.SpringBootTestParticipant` hands over a fully-working `OrderBean`, `OrderEntity`, `OrderDAO/OrderDAOWrapper`, and `OrderService/OrderServiceImpl` — exactly the layered shape built repeatedly across both volumes of this course — leaving only the controller itself as a POA checklist:

```
1  /*
2  * POA 5: Write the respective annotation to mark the class as a Controller class
3  *       The annotation should also make this class as a Spring Managed Bean
4  *       The annotation should also make sure that all the occurrences of
5  *       @GetMapping/@PostMapping/@PostMapping methods assume @ResponseBody semantics
        by default.
6  */
7  public class OrderController {
8
9      /*
10     * POA 6: Write the respective annotation to inject the CustomerService
11     *         implementation class object
12     */
13     private OrderService orderService;
14
15     /*
16     * POA 7: Create a Method that executes for the URL order/controller/addOrder with
17     *         POST HTTP VERB
18     * It should accept the JSON in the Request body and de-serialize it to OrderBean
19     * It should validate the OrderBean and if in case of any validation errors then
20     *         return
21     * complete error messages with HTTP status BAD_REQUEST
22     * else
23     * It should invoke the appropriate method from OrderService class to save the
24     *         OrderBean
25     * and return the Message "Order Added Successfully with OrderId :<<orderId>>"
26     *
27     * Hint : Invoke the orderService.save(OrderBean customerBean) method to save the
28     *         object
29     */
30 }
```

26 }

This is a deliberately smaller version of the previous volume’s `Assignment_10004` REST assignment, isolating exactly one technique cluster at a time: **POA 5** is asking specifically for `@RestController` (named explicitly by the requirement’s own wording, “`@ResponseBody` semantics by default” — the identical phrasing, and the identical underlying reasoning, as the earlier volume’s Chapter 33); **POA 6** is plain constructor-or-field `@Autowired` injection of the service layer; and **POA 7** exercises the full request-validation-response cycle in one method — `@RequestBody @Valid OrderBean`, a `BindingResult` checked for errors before proceeding, and a `ResponseEntity` carrying either the validation errors (`BAD_REQUEST`) or a success message (`CREATED`) — exactly the pattern this volume’s Chapter 40 additionally reframed as a *secure* coding habit, not merely a correctness one: validating the incoming `OrderBean` here is simultaneously satisfying this assignment’s functional requirement and guarding against the same unvalidated-input risk Chapter 40 named directly.

42.3 Assignment Stage 2 — A Full CRUD Controller Plus a Custom Validator

`Assignment_10005_2.SpringBoot.Participant` (delivered as `Unit-25SpringBootActivity2`) extends the identical `Order` domain to a complete CRUD surface, and adds one further technique — a custom validation constraint — into the mix:

```

1  /*
2  * POA 12: Write the respective annotation to mark the class as a Controller class ...
3  */
4  public class OrderController {
5
6      /* POA 13: Write the respective annotation to inject the CustomerService
7       * implementation class object */
8      private OrderService orderService;
9
10     /*
11      * POA 14: Create a Method that executes for the URL order/controller/addOrder with
12      * POST HTTP VERB
13      * ... (identical shape to Stage 1's POA 7)
14      */
15     /*
16      * POA 15: Create a Method that executes for the URL
17      * order/controller/getOrdersInBillingRange/56000--90000 with GET HTTP VERB
18      * It should invoke the appropriate method from OrderService class to get a List of
19      * Orders
20      * within given billRange(56000--90000)
21      * If the List returned is not null return the same with HTTP status OK
22      * else
23      * method should return HTTP status as NOT_FOUND
24      * Hint : Invoke the orderService.getOrdersInBillingRange(double minBillAmount,
25      * double maxBillAmount)
26      */
27     /*
28      * POA 16: Create a Method that executes for the URL order/controller/
29      * updateOrderStatus with PUT HTTP VERB
30      * It should accept the JSON in the Request body and de-serialize it to orderBean
31      * It should also validate the OrderBean and if in case of any validation errors
32      * then return
33      * complete error messages with HTTP status BAD_REQUEST
34      * else
35      * It should invoke the appropriate method from OrderService class to update the
36      * Orderstatus
37      * If the method invoked from OrderService class returns true then return message
38      * as
39      * "Order status updated succesfully" with HTTP status OK

```

```

35     * else
36     * return HTTP status INTERNAL_SERVER_ERROR
37     *
38     * Hint : Invoke the orderService.updateOrderStatus(OrderBean orderBean) to update
           the order status
39     */
40
41     /*
42     * POA 17: Create a Method that executes for the URL order/controller/deleteOrder
           /1001 with DELETE HTTP VERB
43     * It should invoke the appropriate method from OrderService class to delete the
           order with the given orderId
44     * If deletion is successful returns the orderBean JSON with HTTP status as OK
45     * else
46     * return HTTP status INTERNAL_SERVER_ERROR
47     *
48     * Hint : Invoke the orderService.deleteOrder(Integer orderId) to delete the order
           */
49     */
50 }

```

Mapping each new POA back to a specific technique already built across this course:

- **POA 15** is a `@GetMapping` with two `@PathVariable`-extracted bounds, delegating to a Query-Method-derived or explicit `@Query` repository lookup (the previous volume’s Chapter 31), wrapped in the same not-found-vs-OK `ResponseEntity` branching used throughout every REST CRUD demo in this course.
- **POA 16** is a `@PutMapping` that combines *two* things this assignment deliberately forces together: request-body validation (as in POA 14) *and* a boolean-returning service method whose `true/false` result — not an exception — is what the controller branches HTTP status on. This is a subtly different error-signalling convention from the `InvalidUpdateOperationException` pattern seen in the previous volume’s Spring ORM microservice assignment, and worth noticing precisely *because* it’s different: here, “the order didn’t exist to update” is modelled as `updateOrderStatus(...)` simply returning `false`, not as a thrown exception — a perfectly valid, arguably simpler alternative design, and recognising when a service method should signal failure by return value versus by exception is itself a design judgement this assignment is quietly asking the trainee to notice.
- **POA 17** is a `@DeleteMapping` with a `@PathVariable` id, following the identical existence-check-then-act shape as every other REST CRUD demo’s delete endpoint in this course.

42.3.1 The Custom OrderStatusValidator

This stage’s DTO additionally introduces its own custom constraint, built with the exact `@Constraint(validatedBy = ...)` pattern from the previous volume’s Chapter 32 and this volume’s Chapter 37:

```

1 package org.accenture.lkm.validator;
2
3 @Documented
4 @Constraint(validatedBy = OrderStatusValidatorImpl.class) // Related Validator
5 @Target({ ElementType.METHOD, ElementType.FIELD, ElementType.PARAMETER })
6 @Retention(RetentionPolicy.RUNTIME)
7 public @interface OrderStatusValidator {
8     String message() default ""; // can be applied on string fields
9     Class<?>[] groups() default {};
10    Class<? extends Payload>[] payload() default {};
11 }

```

This annotation’s `@Target` is worth comparing carefully against every earlier custom validator built in this course: those were all restricted to `ElementType.FIELD` (and sometimes `METHOD`) only. This one additionally allows `ElementType.PARAMETER` — meaning `@OrderStatusValidator`

can be applied directly to a *method parameter*, not only to a bean's field, which matters if a trainee wants to validate a raw `String orderStatus` argument passed directly into a service method signature, rather than only a field nested inside a larger request DTO. Implementing `OrderStatusValidatorImpl` correctly means checking the submitted status string against whatever fixed, business-meaningful set of order statuses the domain actually defines (e.g. "PLACED", "SHIPPED", "DELIVERED", "CANCELLED") — the same fixed-set validation idea as the previous volume's `CustomerTypeValidator` assignment, now applied to an order's lifecycle status instead of a customer's tier.

Viva Voce — Likely Questions & Answers

Q: In Assignment Stage 1's POA 5, why does the requirement explicitly mention that `GetMapping/PostMapping/PostMapping` methods should "assume `@ResponseBody` semantics by default"?

A: Because this phrasing is specifically describing what `@RestController` provides – it is equivalent to `@Controller` plus `@ResponseBody` applied automatically to every mapped method in the class, which is exactly the annotation this POA is asking the trainee to supply.

Q: Why does validating the incoming `OrderBean` in POA 7/POA 14 serve both a functional and a security purpose at once?

A: Functionally, it ensures the controller only proceeds with a well-formed `OrderBean` and returns a clear `BAD_REQUEST` with details otherwise; from a secure-coding standpoint (this volume's Chapter 40), it also guards against exactly the kind of unvalidated, potentially malformed or malicious input that later causes downstream problems if it reaches the persistence layer unchecked.

Q: In POA 16, how does this assignment signal that an update failed (e.g. the order didn't exist), compared to the `InvalidUpdateOperationException` pattern from the previous volume's Spring ORM assignment?

A: Here, the service method itself returns a boolean, and the controller branches HTTP status (`OK` vs `INTERNAL_SERVER_ERROR`) directly on that return value, rather than the service throwing a dedicated exception that the controller would need to catch – a different, equally valid convention for signalling the same kind of failure.

Q: What does adding `ElementType.PARAMETER` to a custom constraint annotation's `@Target` allow that a `FIELD`-only target does not?

A: It allows the annotation to be applied directly to a method parameter (for example, a plain `String` argument passed into a service method), rather than being restricted to validating a field nested inside a larger request DTO object.

Q: Why might a custom constraint like `OrderStatusValidator` be preferable to simply hardcoding an if-else chain checking the status string inside the service method?

A: Because expressing it as a reusable, declarative constraint lets it be applied consistently anywhere an order status needs validating (a DTO field, a service parameter, or elsewhere) via a single annotation, the same way a built-in constraint like `@NotBlank` is applied, rather than duplicating equivalent validation logic by hand at every call site that needs it.

Front-End Foundations to Integration with Spring Boot API

HTML5, CSS3, JavaScript and XML/XSLT through
React (Components, Hooks) to a Spring Boot REST Backend

Extended Theory, Concepts & Demo Programs
(Viva / Interview Preparation Edition)

Accenture QuantumCrew Batch — Day-wise Training Reference

(Front-End Foundations Volume)

Covers the Front-End Track of the Day-wise Training Plan
Chapters 1–10 — Instructor Demos, Practice Activities & Assignments

Chapter 1

HTML5 Foundations — Structuring the Web Page

1.1 Why Front-End Training Starts With HTML

Every one of the React components built later in this volume ultimately renders down to plain HTML elements sitting in a browser's DOM — a `<Component/>` is, at the end of the day, a convenient, reusable way of generating `<div>`s, `<table>`s, and `<input>`s. Before any of that convenience is worth having, a trainee needs to be fluent in what those underlying elements actually are, and what job each one does. HTML (HyperText Markup Language) is not a programming language — it carries no logic of its own — it is a *markup* language: a vocabulary of tags that describes the *structure and meaning* of a document's content (this is a heading, this is a table, this is a form field), leaving *how it looks* to CSS (Chapter 2) and *how it behaves* to JavaScript (Chapter 3). Keeping these three concerns separate — structure, presentation, behaviour — is the single most important habit this chapter tries to instil, because every framework covered later in this volume (React included) is still, underneath, built on exactly this same three-way separation.

1.2 The Core Tags Every Page Is Built From

Before reading `HTMLDemo.html` line by line, it helps to have the small, common vocabulary of tags it draws on named explicitly, since these same tags recur in every demo for the rest of this volume:

Tag	Purpose
<code><h1></code> through <code><h6></code>	Six levels of heading, <code><h1></code> the most important on a page down to <code><h6></code> the least — chosen for <i>meaning</i> (this is the page's main heading), not merely for how large the text happens to render.
<code><p></code>	A paragraph of body text.
<code></code>	A hyperlink; <code>href</code> holds the destination URL.
<code></code>	An embedded image; <code>alt</code> supplies alternative text read by a screen reader or shown if the image fails to load — never purely decorative, and expected on every meaningful image.
<code>//</code>	An unordered (bulleted) or ordered (numbered) list, each item wrapped in its own <code></code> .
<code><div></code>	A generic, block-level container with no inherent meaning of its own — used purely to group other elements for styling or scripting purposes.
<code></code>	A generic, <i>inline</i> container, the inline counterpart to <code><div></code> — used to wrap a small run of text inside a larger block purely to style or script that one run.

Tag	Purpose
<code><table>/<thead>/<tr>/<tbody>/</table></code>	Tabular data, already introduced through the registration form's own layout above.

`<div>` and `<span>` deserve a specific mention because they are the two tags with no semantic meaning of their own, and are therefore the two tags a React component (Chapter 5 onward) renders far more often than any other — `<div className="App">` in the very first React demo this volume looks at is exactly this same generic `<div>`, simply written inside JSX instead of a plain `.html` file.

1.3 Semantic HTML5 — Naming a Section by What It *Is*

Older HTML relied almost entirely on generic `<div>`s, distinguished only by an `id` or `class`, to lay out an entire page — a `<div id="header">`, a `<div class="nav">`, and so on. HTML5 added a set of **semantic elements** that name a section of a page by what it *is*, the same way `<h1>` names a heading by what it is rather than merely how it looks:

Element	Represents
<code><header></code>	Introductory content for a page or a section — typically a logo, a title, or navigation.
<code><nav></code>	A block of navigation links.
<code><main></code>	The one, primary content of the page, excluding anything repeated across pages (headers, footers, navigation).
<code><section></code>	A thematically-related grouping of content, generally with its own heading.
<code><article></code>	A self-contained piece of content that would still make sense distributed or syndicated on its own (a blog post, a news story).
<code><aside></code>	Content only tangentially related to the main content around it (a sidebar, a pull-quote).
<code><footer></code>	Closing content for a page or section — typically copyright information, links, or contact details.

Using `<nav>` instead of `<div class="nav">` costs nothing in how the page looks — CSS still styles it identically either way — but it hands a screen reader, a search engine, or any other program reading the page a direct, reliable way to identify what that section of the page *is*, without having to guess from a `class` name that could, in principle, say anything at all. This is the same underlying idea Chapter 1's very first section named for `<h1>` and `<table>`, simply extended to page-level structure.

1.4 HTML5's Newer Input Types

Chapter 1's registration form uses only the oldest, most general `<input type="text">`. HTML5 added a range of more specific input types that let the *browser itself* enforce a basic shape for the data and offer the user a more appropriate control, without any JavaScript validation (Chapter 3) being written at all:

type Value	Behaviour
email	Expects an email-shaped value; a browser will refuse to submit the form if the value does not contain an @ and a domain part.
number	Restricts input to numeric values, and typically renders small increment/decrement arrows alongside the field.
date	Presents a native date picker and stores the value in a predictable YYYY-MM-DD format.
range	Renders a slider between a min and max, rather than a text box.
color	Renders a native colour-picker control.

Every one of these newer types still carries the identical `id/ name/value` vocabulary already covered above — they are a richer set of *controls*, not a new mechanism, which is precisely why Chapter 6's React `<input>` elements can freely use `type="email"` or `type="number"` without needing any React-specific handling beyond the `onChange` pattern already established there. `required`, `placeholder` (greyed-out hint text shown until the user types something), `minlength/maxlength`, and `disabled` are further attributes, usable on any of these input types, that let a page enforce or hint at basic constraints natively, again before a single line of JavaScript is written.

1.5 Anatomy of an HTML Document

The course's own `HTMLDemo.html` demo is a compact, complete example worth reading start to finish, because every one of its lines demonstrates a distinct HTML concept:

```

1  <!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
2    "http://www.w3.org/TR/html4/loose.dtd">
3  <html>
4  <head>
5  <meta http-equiv="Content-Type" content="text/html; charset=ISO-8859-1">
6  <title>ADF 2.0 Java</title>
7
8  <!-- Linking to the external CSS file -->
9  <link rel="stylesheet" type="text/css" href="CssDemo.css">
10
11 <!-- Linking to the external JavaScript file -->
12 <script language="javascript" src="DemoJs.js"></script>
13
14 </head>
15 <body>
16   <center>
17     <h1>ADF Java School</h1>
18     <hr>
19     <form>
20       <table border=1>
21         <thead>
22           <b>Student Registration Form</b>
23         </thead>
24         <tr>
25           <td>Enter Student Id</td>
26           <td><input type="text" id="sid" name="studentId" /></td>
27         </tr>
28         <tr>
29           <td>Enter Student Name</td>
30           <td><input type="text" id="sname" name="studentName" /></td>
31         </tr>
32         <tr>
33           <td>Enter Course Id</td>
34           <td><input type="text" id="cid" name="courseId" /></td>
35         </tr>

```

```
36         <tr>
37             <td><input id="mybtn" type="submit" value="Add Student"
38                 onclick="addStudent()"></td>
39         </tr>
40     </table>
41     <br />
42 </form>
43 </center>
44 </body>
45 </html>
```

Listing 1.1: HTMLDemo.html — Student Registration Form

Reading this top to bottom against the vocabulary of this chapter:

- **<!DOCTYPE>** — the very first line of any HTML document, telling the browser which version/flavour of HTML the rest of the document is written in, so the browser knows which rendering rules to apply. A missing or malformed **<!DOCTYPE>** is one of the most common reasons a page renders subtly differently across browsers (*quirks mode* versus *standards mode*).
- **<head> vs <body>** — **<head>** holds metadata *about* the page (its **<title>**, its character encoding, its linked stylesheets and scripts) that is never itself rendered as visible content; **<body>** holds everything the user actually sees.
- **<link> and <script>** — these two tags are the seam between this chapter and the next two: **<link rel="stylesheet">** pulls in the external CSS file covered in Chapter 2, and **<script src="...">** pulls in the external JavaScript file covered in Chapter 3. Notice that the demo's own comments call out that these two tags only have a visible effect *once* the corresponding CSS/JS file has actually been written — exactly the incremental, one-concept-at-a-time teaching structure this whole training programme follows.
- **Semantic structure vs generic containers** — **<h1>** is used for the page's main heading (not just because it happens to render large and bold, but because it tells a browser, a search engine, or a screen reader that this text *is* the page's primary heading); **<table>** is used because the registration form's fields genuinely are tabular, row-and-column data, which is exactly what **<table>** is for.
- **Forms and form controls** — **<form>** wraps every field that should be submitted together as one unit; **<input type="text">** accepts free text; **<input type="submit">** is the control that actually triggers submission (here, additionally wired to **onclick="addStudent()"**, a first, very direct glimpse of Chapter 3's event handling, before any of Chapter 3's own vocabulary has been introduced).

1.5.1 Why id and name Are Both Present, and Why They Differ

Every **<input>** in the demo carries both an **id** and a **name** attribute, and it is worth being precise about why both exist rather than treating them as interchangeable:

- **id** is a unique handle used by *CSS* (to target one specific element with a selector like **#mybtn**, as Chapter 2 shows) and by *JavaScript* (**document.getElementById(...)**, as Chapter 3 shows) to find one specific element in the DOM. An **id** must be unique across the entire document.
- **name** is the key under which a form field's value is actually submitted to the server (as part of the request body or query string). It does not need to be unique in the same way **id** does — several radio buttons that belong to the same logical choice, for instance, deliberately share one **name** so the browser treats them as one mutually-exclusive group.

1.6 HTML Form Controls Used Throughout This Course

The `HTMLTryIt.html` companion demo leaves the actual registration form as an exercise (“*Create a registration form, which will accept the employee id, name and department id of an employee*”), which is precisely the shape every later React form in Chapter 6 re-implements as a controlled component — the HTML form controls below are the exact vocabulary that Chapter 6’s `<input>`, `<select>`, and `<textarea>` JSX elements are built out of, so it is worth having this table as a running reference.

Control	Purpose
<code><input type="text"></code>	A single-line free-text field.
<code><input type="password"></code>	A single-line field whose characters are masked on screen.
<code><input type="checkbox"></code>	An independent on/off toggle; any number of checkboxes in a group can be checked at once.
<code><input type="radio"></code>	A mutually-exclusive choice; every radio button that shares a <code>name</code> belongs to the same group, and checking one unchecks the rest of that group.
<code><select></code> / <code><option></code>	A drop-down list of choices.
<code><textarea></code>	A multi-line free-text field.
<code><input type="submit"></code>	The control that triggers the enclosing <code><form></code> ’s submission.

1.7 The Same Vocabulary, Reused in React

Nothing in this section is React-specific, and that is deliberately the point: Chapter 6’s `EmployeeForm` component uses exactly this same set of controls — `<input>`, `<select>`, `<textarea>`, radio groups sharing a `name` — wrapped inside JSX instead of raw HTML. Recognising that `<input type="radio" name="gender" value="M">` means the identical thing whether it sits in a `.html` file or inside a React component’s `render()` method is what makes the jump into Chapter 5 far smaller than it first appears: JSX is HTML’s own vocabulary, embedded inside JavaScript, not a new markup language to learn from scratch.

Viva Voce — Likely Questions & Answers

Q: What is the difference between `<div>` and `<span>`?

A: Both are generic containers with no semantic meaning of their own. `<div>` is a block-level element, starting on its own new line and taking up the full available width by default; `<span>` is an inline element, sitting within the flow of surrounding text and taking up only as much width as its own content needs.

Q: Why would a developer use `<nav>` instead of `<div class="nav">`?

A: Both can be styled identically with CSS, but `<nav>` is a semantic element that directly tells a screen reader, a search engine, or any other program reading the page that this section *is* navigation, without relying on a `class` name (which could say anything, or nothing reliable at all) as the only signal of what the section represents.

Q: What advantage does `<input type="email">` have over a plain `<input type="text">` for the same field?

A: The browser itself can refuse to submit the form unless the entered value is shaped like a valid email address, giving basic validation for free, before any JavaScript (Chapter 3) or

React `validate()` method (Chapter 6) is written at all.

Q: What is the difference between HTML being a markup language and JavaScript being a programming language?

A: HTML describes the structure and meaning of a document's content (headings, tables, form fields) but carries no logic of its own — it cannot make a decision or repeat an action. JavaScript is a full programming language that can branch, loop, and react to events, and is what actually changes what a page does, as opposed to what a page contains.

Q: Why does keeping structure (HTML), presentation (CSS) and behaviour (JavaScript) separate matter, even once React is introduced?

A: Separating these concerns keeps each layer independently understandable and maintainable — a designer can change CSS without touching markup, and a developer can change behaviour without touching either. React does not abandon this separation; it re-packages structure and behaviour together inside one component for the sake of reusability, but the underlying HTML vocabulary a component renders, and the underlying DOM events it listens to, are unchanged.

Q: What is the practical difference between an element's id and its name attribute?

A: `id` is a unique handle used by CSS and JavaScript to target one specific element in the DOM. `name` is the key under which a form field's value is submitted to the server, and is deliberately shared across every control in a single radio-button group so the browser treats them as one mutually exclusive choice.

Q: Why must every radio button in the same logical choice share an identical name attribute?

A: The shared `name` is what tells the browser that a set of radio buttons belong to the same group, so that selecting one automatically deselects every other button sharing that name — without a shared `name`, each radio button would behave as an independent toggle rather than a mutually exclusive choice.

Q: What does the `<!DOCTYPE>` declaration actually do, and what can go wrong if it is missing?

A: It tells the browser which HTML specification the rest of the document conforms to, so the browser applies the matching rendering rules. Omitting or malforming it can push a browser into “quirks mode”, where legacy, inconsistent rendering rules apply instead of modern standards-based ones, which is a common source of a page looking different across browsers.

Chapter 2

CSS3 — Styling, the Box Model and Selectors

2.1 Anatomy of a CSS Rule

Every CSS rule, however simple or elaborate, has exactly the same two-part shape: a **selector** (Section 2.2) naming *what* to style, followed by a **declaration block** in curly braces naming *how*:

```
1 selector {  
2     property: value;  
3     property: value;  
4 }
```

Listing 2.1: The shape of a single CSS rule

Each individual `property: value;` pair inside the block is a **declaration**; a semicolon separates one declaration from the next, and the final declaration's trailing semicolon, while technically optional, is conventionally kept so that a later declaration can be appended without first having to add one.

2.2 Separating Presentation From Structure

Chapter 1 deliberately left every visual decision — colour, spacing, font — out of `HTMLDemo.html` itself. CSS (Cascading Style Sheets) is where those decisions actually live, and keeping them in a separate file (linked in with `<link rel="stylesheet">`) rather than scattered across every tag's own attributes is what lets one stylesheet consistently re-style an entire site, and lets a designer change the look of a page without a developer having to touch its markup at all.

2.3 Selectors — Targeting the Right Elements

The course's own `CssDemo.css` demonstrates the three most common ways to select which elements a rule applies to:

```
1 body {  
2     background-color: #d2b48c;  
3     margin-left: 20%;  
4     margin-right: 20%;  
5     border: 2px dotted gray;  
6     padding: 10px 10px 10px 10px;  
7     font-family: sans-serif;  
8 }  
9  
10 h1 {  
11     font-style: italic;  
12 }  
13  
14 #mybtn {  
15     color: white;  
16     font-weight: bold;
```

```

17 background-color: brown;
18 }

```

Listing 2.2: CssDemo.css — Element, Type and ID Selectors

Selector	What It Targets
<code>body</code>	A <i>type selector</i> — every <code><body></code> element in the document (in practice, exactly one).
<code>h1</code>	A type selector matching every <code><h1></code> on the page, however many there are.
<code>#mybtn</code>	An <i>ID selector</i> , matching only the single element whose <code>id</code> attribute equals <code>mybtn</code> — exactly the submit button from Chapter 1's registration form. Because <code>id</code> values are unique within a document, an ID selector always targets at most one element.

A fourth selector type, the *class selector* (`.someClassName`), is introduced in Chapter 6 once React components start needing to apply the same style to many different elements at once (`className="App"`) — the distinction matters: use an ID selector for the one element that is genuinely unique on the page, and a class selector for a style that is meant to be reused across several elements.

2.4 The CSS Box Model

`margin-left: 20%; margin-right: 20%; border: 2px dotted gray; padding: 10px 10px 10px 10px;` in the demo above exercises every layer of the CSS box model in one declaration block, and understanding this model is the single most useful piece of CSS theory a beginner can learn, because every layout bug eventually traces back to a misunderstanding of one of these four layers:

Layer (outside in)	What It Controls
Margin	Transparent space <i>outside</i> the element's border, used to push the element away from its neighbours.
Border	A visible (or invisible) line drawn around the padding and content, as set by <code>border</code> above (<code>2px dotted gray</code> — width, style, colour, in that order).
Padding	Space <i>inside</i> the border, between the border and the element's own content — pushes the content away from its own edge without affecting neighbouring elements.
Content	The actual text or child elements the box contains, whose size is otherwise governed by <code>width/height</code> or, absent those, by the content itself.

2.5 Three Ways to Attach CSS to a Page

The React styling demos in Chapter 6 revisit this exact decision — where should a style actually live? — so it is worth naming all three options together here, before any React-specific wrapping is added on top:

Approach	How It Works
External stylesheet	A separate <code>.css</code> file linked in with <code><link rel="stylesheet" href="..."></code> , exactly as <code>CssDemo.css</code> is linked into <code>HTMLDemo.html</code> . Reusable across many pages; the approach used throughout this chapter.
Internal stylesheet	A <code><style></code> block placed directly inside a single page's own <code><head></code> — scoped to that one page only.
Inline style	A <code>style="..."</code> attribute set directly on one element, overriding any external or internal rule for that property on that one element specifically. Highest specificity, but the least reusable, and the hardest to maintain at scale — Chapter 6 revisits exactly this trade-off when a React component's <code>style</code> prop is compared against an external <code>.css</code> import.

2.6 Units of Measurement

`CssDemo.css`'s own `margin-left: 20%`; uses a percentage; `border: 2px dotted gray`; uses pixels. CSS supports several further units, each suited to a different kind of sizing decision:

Unit	Meaning
<code>px</code>	An absolute, fixed pixel size — does not scale if the user changes their browser's default font size.
<code>%</code>	Relative to the corresponding size of the element's own parent — <code>margin-left: 20%</code> above means 20% of the <code><body></code> 's own containing width.
<code>em</code>	Relative to the current element's own font size — <code>2em</code> means twice whatever font size already applies at that point in the page.
<code>rem</code>	Relative to the <i>root</i> (<code><html></code>) element's font size, regardless of any nested element's own font size — a common choice for consistent spacing that does not compound unpredictably the way nested <code>em</code> values can.
<code>vh / vw</code>	Relative to 1% of the browser's current viewport height / width — useful for sizing something relative to the visible screen itself, rather than to any particular element.

2.7 The display Property — Block, Inline and Inline-Block

Every HTML element has a default `display` value that governs how it lays out relative to its neighbours, and this default is exactly what separates `<div>` from `<span>` in Chapter 1:

display Value	Behaviour
<code>block</code>	Starts on its own new line and expands to fill the available width of its parent — <code><div></code> , <code><h1></code> – <code><h6></code> , and <code><p></code> are all block by default.

display Value	Behaviour
<code>inline</code>	Sits within the surrounding flow of text, only as wide as its own content, and cannot have an explicit <code>width/height</code> or <code>top/bottom margin</code> applied — <code></code> and <code><a></code> are inline by default.
<code>inline-block</code>	Sits within the surrounding flow like an inline element, but, unlike a plain inline element, <i>can</i> accept an explicit <code>width/height</code> and vertical margin.
<code>none</code>	Removes the element from the page's layout entirely, as though it were not in the document at all — distinct from <code>visibility:hidden</code> , which hides the element visually but still leaves its space reserved in the layout.

2.8 Positioning

The `position` property changes how an element is placed relative to its normal location in the page's flow:

Value	Behaviour
<code>static</code>	The default — the element sits exactly where the normal document flow would place it; <code>top/left/right/ bottom</code> have no effect.
<code>relative</code>	The element still occupies its normal spot in the flow, but <code>top/left/right/bottom</code> can then nudge it away from that spot, visually, without affecting where neighbouring elements are laid out.
<code>absolute</code>	The element is removed from the normal flow entirely (leaving no gap behind) and positioned relative to its nearest <i>ancestor</i> that is itself <code>relative</code> , <code>absolute</code> , or <code>fixed</code> (or, absent one, relative to the page itself).
<code>fixed</code>	Positioned relative to the browser's own viewport, and stays in that same screen position even as the page is scrolled.
<code>sticky</code>	Behaves as <code>relative</code> until the page is scrolled past a given threshold, at which point it behaves as <code>fixed</code> — the mechanism behind a heading that scrolls normally until it reaches the top of the screen, then sticks there.

2.9 Flexbox — Laying Out a Row or Column of Elements

Modern CSS layouts very commonly reach for the **Flexbox** model rather than juggling `float` or manual `position` calculations by hand. Setting `display: flex;` on a container turns every one of its direct children into a *flex item*, laid out along one axis (a row, by default) and governed by a small, powerful set of further properties on the container itself:

```

1 .toolbar {
2   display: flex;
3   justify-content: space-between; /* spacing along the main axis (the row) */
4   align-items: center;           /* alignment along the cross axis (vertical) */
5   gap: 10px;                    /* space between each flex item */
6 }

```

Listing 2.3: A simple Flexbox container

`justify-content` controls how children are spread out along the row (`flex-start`, `center`, `space-between`, and more); `align-items` controls their alignment on the perpendicular axis. Flexbox is worth knowing by name specifically because Chapter 6’s React styling demos, and any real-world component library, lean on it heavily for exactly the kind of layout a hand-rolled `<table>`-based layout (as used throughout this volume’s own simpler demos) becomes unwieldy for once a page grows more visually elaborate.

2.10 Pseudo-Classes and Pseudo-Elements

A **pseudo-class** selects an element based on a *state* it is currently in, rather than anything present in the markup itself:

Pseudo-Class	Matches
<code>:hover</code>	An element the mouse pointer is currently over.
<code>:focus</code>	An element (typically a form control) that currently has keyboard focus.
<code>:first-child</code> / <code>:last-child</code>	An element that is the first / last child of its parent.
<code>:nth-child(n)</code>	An element at a specific position among its siblings.

A **pseudo-element**, written with a double colon, targets a specific sub-part of an element rather than a state — `::before` and `::after` insert generated content immediately before or after an element’s own actual content, most often used together with the `content` property to add a decorative icon or label purely through CSS, with no corresponding tag in the HTML itself.

2.11 The Cascade — Why the Same Property Can Be Set Twice

CSS’s own name comes from the *cascade*: when more than one rule targets the same element and sets the same property, a well-defined set of rules — source order, selector specificity (an ID beats a class, which beats a type selector), and the `!important` escape hatch — decides which one actually wins, rather than the browser simply applying whichever rule was written last. This is precisely why `#mybtn`’s rules in the demo above win out over any more general `body` or type-selector rule that might otherwise have touched the same button: an ID selector is more specific than a type selector, so it takes precedence regardless of which one appears first in the file.

2.11.1 Calculating Specificity

When two rules genuinely conflict, specificity is calculated, conceptually, as a three-part score — counted from most to least powerful:

Selector Component	Specificity Weight
Inline <code>style="..."</code> attribute	Highest — beats any selector written in a stylesheet, however specific.
ID selectors (<code>#mybtn</code>)	Next highest — one point at this level per ID used in the selector.

Selector Component	Specificity Weight
Class selectors, attribute selectors, pseudo-classes (<code>.someClass</code> , <code>:hover</code>)	Next — one point at this level per class/attribute/pseudo-class used.
Type selectors, pseudo-elements (<code>h1</code> , <code>::before</code>)	Lowest — one point at this level per element type used.

Two selectors are compared level by level, starting from the highest: any ID advantage wins outright, regardless of how many classes or type selectors the less-specific rule piles on afterward. Only when two selectors tie at every level does the *later*-declared rule in the stylesheet actually win — source order is the final tiebreaker, not the first thing considered.

Viva Voce — Likely Questions & Answers

Q: Why is it good practice to keep CSS in a separate, external file rather than styling every tag with its own inline style attribute?

A: An external stylesheet lets one set of rules consistently style every matching element across an entire page (or site), and lets presentation be changed without touching the underlying markup at all — exactly the structure/presentation separation Chapter 1 introduced. Inline styles, by contrast, have to be repeated on every element individually and mix presentation back into the markup.

Q: What is the difference between a type selector, a class selector and an ID selector?

A: A type selector (`h1`) matches every element of that tag name on the page; a class selector (`.someClass`) matches every element carrying that class, however many there are; an ID selector (`#mybtn`) matches at most one element, since id values must be unique within a document.

Q: List the four layers of the CSS box model, from the content outward.

A: Content (the element’s own text or children), padding (space between the content and the border), border (the visible or invisible line around the padding), and margin (transparent space outside the border separating the element from its neighbours).

Q: What is the practical difference between padding and margin?

A: Padding is space *inside* an element’s border, pushing its own content away from its edge and expanding the element’s visible background along with it; margin is space *outside* the border, pushing the element away from neighbouring elements without affecting the element’s own visible background.

Q: Why does an ID selector like `#mybtn` override a more general type selector like `body` for the same property?

A: CSS resolves conflicting rules by specificity before considering source order: an ID selector is defined to be more specific than a type selector, so its declaration wins regardless of which rule was written first in the file — this specificity ordering is what the term “cascade” in Cascading Style Sheets refers to.

Q: What is the difference between `rem` and `em` as CSS units?

A: `em` is relative to the current element’s own font size, so nested elements using `em` can compound unpredictably; `rem` is always relative to the root `<html>` element’s font size,

regardless of how deeply nested the element using it is, giving more predictable, consistent sizing.

Q: What is the practical difference between `display:none` and `visibility:hidden`?

A: `display:none` removes the element from the page's layout entirely, as if it were not in the document at all, so no space is reserved for it. `visibility:hidden` hides the element visually but still reserves its normal space in the layout, leaving a gap where it would have appeared.

Q: What does setting `position:absolute` on an element actually position it relative to?

A: Its nearest ancestor that itself has `position` set to `relative`, `absolute`, or `fixed` — or, if no such ancestor exists, relative to the page itself. The element is also removed from the normal document flow, leaving no gap behind at its original location.

Q: What does setting `display:flex` on a container do to its direct children?

A: It turns every direct child into a flex item laid out along one axis (a row, by default), whose spacing and alignment can then be controlled with the container's own `justify-content` and `align-items` properties, without needing manual `float` or `position` calculations.

Q: How is CSS specificity resolved when two conflicting rules both apply to the same element?

A: Specificity is compared level by level, from highest to lowest: inline styles beat any ID selector, an ID selector beats any class/attribute/pseudo-class selector, and those beat a plain type selector, regardless of how many lower-level selectors are stacked on the losing side. Only when two rules are equally specific at every level does the rule declared later in the source win.

Chapter 3

JavaScript Essentials — the DOM and Event Handling

3.1 What JavaScript Adds That HTML and CSS Cannot

HTML describes structure; CSS describes appearance; neither can make a decision, repeat an action, or react to something the user does. JavaScript is the programming language that runs inside the browser and can do exactly that — and, critically for everything that follows in this volume, JavaScript can also *read and change* the HTML a page is built from, *after* the page has already loaded, through a live, in-memory representation of that page called the **DOM** (the Document Object Model). Every one of Chapter 1’s tags becomes one DOM node once the browser parses the page; JavaScript’s job, in this chapter, is to reach into that tree of nodes and read or modify it.

3.2 Variables — var, let and const

JavaScript has three keywords for declaring a variable, and the choice between them matters:

Keyword	Behaviour
<code>var</code>	The original keyword — scoped to the enclosing <i>function</i> , not the enclosing block, and can be re-declared. Legacy code (including this course’s own <code>DemoJs.js</code>) still uses it, but modern code generally avoids it in favour of the two keywords below.
<code>let</code>	Scoped to the enclosing <i>block</i> (any <code>{...}</code> , not just a function) — a variable declared with <code>let</code> inside an <code>if</code> or a <code>for</code> loop is not visible outside it. Can be reassigned, but not re-declared in the same scope.
<code>const</code>	Identical block scoping to <code>let</code> , but cannot be reassigned after its initial value is set — the default choice for any value that is not meant to change, and the keyword this course’s React demos use for every destructured <code>[state, setState]</code> pair from <code>useState</code> (Chapter 9), since the <i>pair itself</i> is never reassigned, even though the state value it currently points at changes.

3.3 Data Types

JavaScript is *dynamically typed* — a variable’s type is determined by whatever value currently sits in it, and can change over its lifetime. Its built-in types split into two groups:

Category	Types
Primitive	string , number (JavaScript has no separate integer type), boolean , undefined (a declared variable with no value assigned yet), null (an explicit “no value”), and symbol .
Reference	object (including plain object literals, arrays, and functions, all of which are technically objects under the hood).

A primitive is compared and copied *by value* — two separately-created strings holding “abc” are simply equal. An object is compared and copied *by reference* — two separately-created objects holding identical data are *not* equal unless they are literally the same object in memory, which is exactly why Chapter 9’s `useState` bug fix has to explicitly spread a previous object (`{...newObj, salary:...}`) into a genuinely *new* object, rather than being able to rely on any kind of automatic, value-based comparison.

3.4 Operators

Operator	Meaning
<code>+</code> <code>-</code> <code>*</code> <code>/</code> <code>%</code>	Arithmetic — addition, subtraction, multiplication, division, and remainder (modulo).
<code>==</code> vs <code>===</code>	<code>==</code> compares two values <i>after</i> converting them to a common type if they differ (<code>"5" == 5</code> is <code>true</code>); <code>===</code> (<i>strict equality</i>) compares both value and type with no conversion (<code>"5" === 5</code> is <code>false</code>). Modern JavaScript style, including every comparison in this course’s own demos, prefers <code>===</code> specifically to avoid the surprising, implicit conversions <code>==</code> can produce.
<code>&&</code> <code> </code> <code>!</code>	Logical AND, OR, and NOT, used to combine or invert boolean conditions.
<code>?:</code> (ternary)	A compact <code>if/else</code> expressed as one expression — <code>condition ? valueIfTrue : valueIfFalse</code> — already used repeatedly for conditional rendering in this volume’s own React JSX (Chapter 5 onward), precisely because a full <code>if</code> statement cannot be embedded directly inside a JSX <code>{...}</code> expression.

3.5 Functions — Three Ways to Write One

```

1 // 1) Function declaration -- hoisted, so it can be called before this line runs
2 function addStudent() {
3   alert("Registering...");
4 }
5
6 // 2) Function expression -- a function stored in a variable, NOT hoisted
7 const addStudent = function() {
8   alert("Registering...");
9 };
10
11 // 3) Arrow function -- the shorthand syntax this course's React demos use almost
12 //   exclusively for event handlers and Hook callbacks
13 const addStudent = () => {
14   alert("Registering...");
15 };

```

Listing 3.1: Function declaration, function expression, and arrow function

Beyond being shorter to write, an arrow function does not create its own **this** — it uses **this** from whatever scope it was *defined* in. This is precisely why every class-component event handler in Chapter 5 onward (`incrementHandlerCount = ()=>{...}`) is written as an arrow function assigned to a class field: it lets **this** inside the handler correctly refer to the component instance, without the extra, easy-to-forget `.bind(this)` step older React code needed when handlers were written as ordinary methods instead.

3.6 Arrays and Their Most-Used Methods

An array is JavaScript’s ordered list type. Alongside indexing (`arr[0]`) and `.length`, a handful of array methods recur constantly throughout this volume’s own React chapters:

Method	Behaviour
<code>.map(fn)</code>	Returns a <i>new</i> array, the same length as the original, with fn applied to every element — the exact method Chapter 10’s <code>GetAllComponent</code> uses to turn a JSON array of employees into an array of JSX <code><tr></code> elements, one per employee.
<code>.filter(fn)</code>	Returns a new array containing only the elements for which fn returns true .
<code>.forEach(fn)</code>	Calls fn once per element purely for a side effect (such as logging); unlike <code>.map</code> , it returns undefined , not a new array.
<code>.reduce(fn, initial)</code>	Collapses an entire array down to one single accumulated value, applying fn to an accumulator and each element in turn.

Every one of `.map`, `.filter` and `.reduce` returns a *new* array (or value) and leaves the original array completely untouched — this *non-mutating* habit is precisely why these three methods, rather than a hand-written `for` loop that pushes onto the existing array, are the idiomatic choice inside a React component: React’s own state and props are expected to be treated as read-only (Chapter 5), and methods that naturally produce a new array rather than mutating an existing one make that expectation easy to honour by default.

3.7 Objects and Object Literals

An object is JavaScript’s key/value container — exactly the shape every piece of React **state** used in this volume takes:

```

1  const employee = {
2      employeeName: 'Harsha',
3      salary: 45000
4  };
5
6  console.log(employee.employeeName); // dot notation
7  console.log(employee['salary']);   // bracket notation -- required when the
8                                     // key itself is a variable, not a fixed
9                                     // literal name (exactly Chapter 6's
10                                    // {[name]: value} computed key)
11  employee.departmentCode = 1001;    // adding a new property

```

Listing 3.2: An object literal, and reading/writing its properties

3.8 Conditionals and Loops

```
1 if (eid.length === 0) {
2   alert("Employee id is required");
3 } else if (ename.length === 0) {
4   alert("Employee name is required");
5 } else {
6   // proceed
7 }
8
9 switch (action) {
10  case "Increment": /* ... */ break;
11  case "Decrement": /* ... */ break;
12  default: /* ... */
13 }
14
15 for (let i = 0; i < employeeList.length; i++) {
16   console.log(employeeList[i]);
17 }
```

Listing 3.3: if/else, switch, and for

`switch` is worth naming specifically because Chapter 9’s `useReducer` reducer functions are, structurally, nothing more than a `switch` statement over an action’s type, returning a different new state for each `case`. A plain `for` loop over an array, by contrast, is rarely seen inside this volume’s own React components — `.map(...)` above is almost always preferred there instead, precisely because it produces a new array directly usable as JSX output, rather than requiring a separately declared, manually-populated results array alongside the loop.

3.9 Template Literals

Backtick-delimited strings allow a variable or expression to be embedded directly inside a string, using `$...`, instead of being concatenated with `+`:

```
1 // Without template literals:
2 alert("Registering " + document.getElementById("sname").value + " ...");
3
4 // With a template literal:
5 alert(`Registering ${document.getElementById("sname").value} ...`);
```

Listing 3.4: Template literals

3.10 JSON — JavaScript’s Own Data Interchange Format

JSON (JavaScript Object Notation) is a text format for representing structured data — objects and arrays of primitive values — that looks almost identical to a JavaScript object literal, but is technically a separate, language-independent *format*, not JavaScript code itself. `JSON.stringify(obj)` converts a JavaScript object into its JSON text representation (for sending over a network, as Chapter 10’s `axios` calls do automatically); `JSON.parse(text)` performs the reverse conversion, turning JSON text received from a server back into a real JavaScript object. JSON is, in effect, the modern, simpler cousin of the XML format Chapter 4 covers: both describe structured data independently of any one programming language, but JSON’s grammar is deliberately smaller (no separate DTD/XSD concept, no attributes distinct from elements, no XSLT-style transformation language) and maps far more directly onto JavaScript’s own object and array literals, which is precisely why every REST payload exchanged with the Spring Boot backend in Chapter 10 is JSON rather than XML.

3.11 Finding an Element — `document.getElementById`

The course’s own `DemoJs.js` is the working half of the `addStudent()` handler wired up on Chapter 1’s submit button (`onclick="addStudent()"`):

```
1 function addStudent(){
2   alert("Registering "+document.getElementById("sname").value+" ...")
3 }
```

Listing 3.5: `DemoJs.js`

`document` is JavaScript’s own handle onto the entire loaded page; `document.getElementById("sname")` walks the DOM tree and returns the one element whose `id` is `sname` — exactly the reason, from Chapter 1, that a form field’s `id` needs to be unique: this lookup would be ambiguous otherwise. `.value` then reads that specific `<input>`’s current, live content — whatever the user has actually typed into it, not whatever was originally written in the HTML source. This read-the-live-DOM-value pattern is the single most common thing a beginner’s first JavaScript program does, and it is worth recognising it precisely because Chapter 6 replaces it with something functionally similar but structurally very different: a React *controlled component*, where the input’s current value is tracked in React’s own state rather than read out of the DOM on demand.

3.12 Events — Reacting to What the User Does

`onclick="addStudent()"` is an **event handler** — code that runs not immediately, but only when a specific thing happens (here, a click). A page that only ran code once, top to bottom, at load time, could never respond to a user typing, clicking, or submitting a form; the event model is what makes a page interactive rather than merely displayed.

Common Event	Fires When...
<code>onclick</code>	The user clicks an element (a button, a link, and so on).
<code>onchange</code>	A form control’s value has changed <i>and</i> the control has lost focus (or, for a <code><select></code> , immediately on selection).
<code>onsubmit</code>	A <code><form></code> is submitted — commonly used, as in Chapter 6, together with “prevent the default full-page reload” so the submission can be handled entirely in JavaScript instead.
<code>onblur</code> / <code>onfocus</code>	An element loses / gains keyboard focus.

3.13 Form Validation — The `TryIt.js` Exercise

The course’s own `TryIt.js` deliberately leaves the actual validation logic as an exercise, but its comments already name the exact technique expected:

```
1 function validateForm() {
2   var eid = document.getElementById("eid").value;
3   var ename = document.getElementById("ename").value;
4
5   /* Topic : Javascript
6    * Instructions:
7    *   Ensure that the employee id and name are not empty.
8    *
9    * Hint:
10   * Use the length property of the variable
11   *
12   */
13
14 }
```

Listing 3.6: TryIt.js — Validation Exercise

A completed version of this handler would check `eid.length===0` and `ename.length===0`, and either block the submission (returning `false` from a form's `onsubmit`, or explicitly calling `event.preventDefault()`) or allow it to proceed. This is worth pausing on precisely because it is the exact same validation shape Chapter 6's React `validate()` method implements later — `this.state.password.length===0` — only there, the value being checked comes from React's own state rather than a fresh `document.getElementById(...).value` lookup. Recognising that the underlying validation *logic* (checking a string's `.length`, checking for a required substring) is unchanged between plain JavaScript and React is what makes Chapter 6 an extension of this chapter's ideas, not a replacement for them.

3.14 Why Client-Side Validation Is Never the Whole Story

It is worth naming a point this chapter's demo does not itself make explicit, but which becomes directly relevant once Chapter 10 introduces a real backend: any validation performed in the browser (as `validateForm()` does) can be bypassed entirely by a request that does not go through the browser's UI at all — a request crafted directly against the server's API. Client-side validation exists purely to give a legitimate user fast, immediate feedback without a round trip to the server; it is never, on its own, a security boundary. The server-side validation covered in this programme's back-end training (Bean Validation annotations on a Spring Boot request DTO) is what actually protects the application's data, and the two are complementary, not redundant: fast feedback in the browser, and a non-negotiable, structural check on the server behind it.

Viva Voce — Likely Questions & Answers

Q: What is the DOM, and why does JavaScript need it?

A: The DOM (Document Object Model) is a live, in-memory, tree-shaped representation of a loaded HTML page that JavaScript can read and modify at runtime. Without it, JavaScript would have no way to change what the user actually sees after the page has finished loading — the DOM is the bridge between JavaScript code and the rendered page.

Q: What does `document.getElementById("sname").value` actually return, and why must `sname` be unique in the page?

A: It returns the current, live content of the one `<input>` element whose `id` attribute equals `sname` — whatever the user has actually typed, not the HTML source's original value. Because `getElementById` returns a single element, `id` values must be unique in the document, or the lookup would be ambiguous about which element to return.

Q: What is an event handler, and why is it necessary for an interactive page?

A: An event handler is code that runs only when a specific user action occurs (a click, a change, a submission), rather than running once at page load. Without event handlers, a page could display content but could never respond to anything the user actually does with it.

Q: Why is client-side form validation, on its own, never sufficient to protect an application's data?

A: Client-side validation only runs inside the browser's own UI flow, and can be bypassed entirely by a request sent directly to the server without going through that UI at all. It exists to give a legitimate user fast feedback without a round trip to the server, while genuine

data protection has to be enforced again, independently, on the server itself.

Q: What is the conceptual link between `TryIt.js`'s planned `eid.length===0` check and the `validate()` method built later in React (Chapter 6)?

A: Both check the same underlying condition — whether a required field's string value is empty — using the same `.length` property. The only structural difference is where the value being checked comes from: a fresh DOM lookup via `document.getElementById(...).value` here, versus a value already held in a React component's own state in Chapter 6.

Q: Why does modern JavaScript style prefer `let/const` over `var`?

A: `var` is scoped to the enclosing function, so a variable declared inside a block such as an `if` or a `for` loop still leaks out into the surrounding function, which can cause confusing, hard-to-trace bugs. `let` and `const` are properly scoped to the enclosing block itself.

Q: What is the difference between `==` and `===` in JavaScript?

A: `==` converts both operands to a common type before comparing them if they are not already the same type, which can produce surprising results (`"5" == 5` is `true`); `===` compares both value and type with no conversion at all (`"5" === 5` is `false`), and is the preferred, safer default.

Q: Why is an arrow function the typical choice for a React class component's event handlers, rather than an ordinary method?

A: An arrow function does not create its own `this` — it uses `this` from the scope it was defined in. Assigning a handler as an arrow-function class field means `this` inside the handler correctly refers to the component instance automatically, without the manual `.bind(this)` step an ordinary method would otherwise require.

Q: Why is `.map(...)` generally preferred over a hand-written `for` loop when building a list of JSX elements in React?

A: `.map(...)` returns a brand-new array without mutating the original, matching React's expectation that props and state are treated as read-only, and its return value can be used directly as JSX output, rather than requiring a separately declared, manually-populated array built up inside a loop.

Q: What is JSON, and how does `JSON.stringify/JSON.parse` relate to it?

A: JSON (JavaScript Object Notation) is a language-independent text format for representing structured data, closely resembling a JavaScript object literal. `JSON.stringify(obj)` converts a JavaScript object into its JSON text form for sending over a network; `JSON.parse(text)` performs the reverse, turning received JSON text back into a JavaScript object.

Chapter 4

XML & XSLT — Structured Data and Its Transformation

4.1 Why a Front-End Track Covers XML At All

Chapter 10 of this volume has React talk to a Spring Boot backend entirely in JSON. It is still worth understanding XML directly, for two reasons this chapter’s own demos make concrete: first, a great deal of real-world enterprise data (configuration files, SOAP-era web services, document interchange formats) is still XML, so being able to read and validate it by eye remains a practical skill; second, XML’s strict, self-describing structure — and the very specific ways it can be malformed — is what makes an entire class of parsing and transformation bugs visible for the first time, in a way that JSON’s much simpler grammar tends to hide.

4.2 Well-Formed XML — The Non-Negotiable Rules

Zoo.xml in the course’s own demo project is a small, well-formed document:

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <?xml-stylesheet type="text/xsl" href="ZooTransform.xsl"?>
3 <zoo>
4   <animal>
5     <class>Monkey</class>
6     <quantity>20</quantity>
7   </animal>
8   <animal>
9     <class>Elephant</class>
10    <quantity>6</quantity>
11  </animal>
12  <animal>
13    <class>Tiger</class>
14    <quantity>3</quantity>
15  </animal>
16 </zoo>
```

Listing 4.1: Zoo.xml

Every well-formed XML document must obey a small, strict set of rules — rules that HTML (as seen in Chapter 1) is historically far more forgiving about, which is precisely why the two are worth contrasting directly:

Rule	What It Requires
Single root element	Exactly one top-level element (<zoo> here) must contain everything else — a document with two top-level siblings is not well-formed.
Every tag must be closed	<animal> must be matched by </animal> — there is no XML equivalent of an HTML tag that browsers tolerate leaving unclosed.

Rule	What It Requires
Proper nesting	Tags must close in the reverse order they opened (<code><a></code> , never <code><a></code>).
Case sensitivity	<code><Class></code> and <code><class></code> are two different, unrelated tag names — unlike HTML, XML tag names are case-sensitive.
Quoted attribute values	An attribute's value must be wrapped in a single consistent pair of quotes, e.g. <code>eventname="HolidayLights"</code> .

4.2.1 ZooDemo.xml and EventsTryIt.xml — Spotting Malformed XML

The course's own `ZooDemo.xml` and `EventsTryIt.xml` demos are deliberately *not* well-formed, and are used as a hands-on “spot the error” exercise:

```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <zoo>
3   <animal>
4     <class>Monkey<quantity>20</quantity></class>
5   </animal>
6   <animal>
7     <cla ss>Elephant</cla ss><quantity>6</quantity>
8   </animal>
9   <animal>
10    <class>Tiger<quantity>3</quantity>
11  </animal>
12 </zoo>

```

Listing 4.2: ZooDemo.xml — Deliberately Malformed

Reading this against the rules table above catches every one of its errors directly: `<quantity>` opens *inside* `<class>` but the closing tags cross rather than nest properly (improper nesting); `<cla ss>` contains a space, which is not a legal tag name at all; and the final `<animal>` block never closes its own `<class>` tag before the `</animal>` that follows it. `EventsTryIt.xml` repeats the same lesson with a richer set of errors — two top-level root elements (`<newcodington>` and, separately, `<events>`), an unclosed `<duration>3` tag, a tag name written without any surrounding quotes at all (`eventname=Klenz for Kids`), and an attribute name whose case does not match its own closing tag's case (`<schedule>...</SCHEDULE>`) — each one a direct, concrete illustration of one rule from the table above being broken.

4.3 Elements vs Attributes — Two Ways to Represent the Same Fact

`Zoo.xml` represents an animal's class and quantity as two child *elements*; the same information could equally be represented as *attributes* on the `<animal>` tag itself:

```

1 <!-- As child elements (Zoo.xml's own approach) -->
2 <animal>
3   <class>Monkey</class>
4   <quantity>20</quantity>
5 </animal>
6
7 <!-- As attributes instead -->
8 <animal class="Monkey" quantity="20" />

```

Listing 4.3: The same data, as elements versus as attributes

Neither form is objectively “more correct” XML — both are equally well-formed — but the choice carries a practical trade-off worth naming: an attribute can only ever hold one single,

simple text value, while an element can itself contain further nested child elements, mixed text, or repeat multiple times. Data that is genuinely simple and singular for a given item (an `id`, a `quantity`) is commonly modelled as an attribute; data that is itself structured, repeatable, or that a stylesheet might need to select and iterate over independently (as `ZooTransform.xml`'s own `<xsl:for-each select="zoo/animal">` does) is more naturally modelled as an element, exactly as `Zoo.xml` itself does.

4.4 XML Namespaces

Because different organisations, and different parts of even one large document, can each define a tag called, say, `<title>` with an entirely different meaning, XML provides **namespaces** to disambiguate which `<title>` is meant. `ZooTransform.xml`'s own root element already uses one: `xmlns:xsl="http://www.w3.org/1999/XSL/Transform"` declares that every tag prefixed `xsl:` in this document (`<xsl:template>`, `<xsl:for-each>`, `<xsl:value-of>`) belongs to that specific namespace, and should be interpreted as an XSLT instruction rather than as an arbitrary, document-specific tag the way `<zoo>` or `<animal>` are. The URL used as a namespace's identifier is not actually fetched by the processor — it functions purely as a long, deliberately-unique string unlikely to collide with any other organisation's own namespace identifier, not as an address to be visited.

4.5 DTD and XSD — Validating Structure, Not Just Well-Formedness

Being *well-formed* (Section 4.2) only means an XML document obeys XML's own generic syntax rules — properly nested, properly closed, and so on. It says nothing about whether *this particular* document actually contains the elements a given application expects, in the order and cardinality that application requires. A **DTD** (Document Type Definition) or its more modern, more expressive replacement, an **XSD** (XML Schema Definition), is a separate document that defines exactly that: which elements and attributes are allowed, in what order, how many times, and, for an XSD specifically, with what data type. A document that is well-formed but does not conform to its own DTD/XSD is described as *invalid*, a distinct and stricter failure than being merely *not well-formed*:

Check	Answers
Well-formedness	Does this document obey XML's own generic syntax rules at all (Section 4.2)? Every XML parser enforces this, unconditionally.
Validity (against a DTD/XSD)	Does this <i>specific</i> document additionally contain the right elements, attributes, order and data types a particular schema requires? Only enforced if the document is explicitly validated against a named DTD or XSD.

An XSD in particular can constrain a `<quantity>` element to be an integer, or require that every `<animal>` element contain exactly one `<class>` and one `<quantity>` child, in that order — the XML world's own, earlier counterpart to the Bean Validation annotations (`@NotBlank`, `@Positive`) this programme's back-end track applies to a JSON request body instead.

4.6 XSLT — Transforming XML Into Something Else

A well-formed XML document like `Zoo.xml` is easy for a program to parse, but it is not, by itself, meant to be read directly by a human in a browser. **XSLT** (Extensible Stylesheet Language Transformations) is a special-purpose language — itself written in XML — for describing how to transform one XML document into another format entirely, most commonly HTML for direct display. `Zoo.xml`'s own second processing instruction, `<?xml-stylesheet type="text/xsl" href="ZooTransform.xsl"?>`, is what tells a browser (or an XSLT processor) which transformation to apply:

```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <xsl:stylesheet version="1.0"
3   xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
4   <xsl:template match="/">
5     <html>
6       <body>
7         <h2>My Animal Collection</h2>
8         <table border="1">
9           <tr bgcolor="#9acd32">
10            <th>Animal</th>
11            <th>Quantity</th>
12          </tr>
13          <xsl:for-each select="zoo/animal">
14            <tr>
15              <td>
16                <xsl:value-of select="class" />
17              </td>
18              <td>
19                <xsl:value-of select="quantity" />
20              </td>
21            </tr>
22          </xsl:for-each>
23        </table>
24      </body>
25    </html>
26  </xsl:template>
27 </xsl:stylesheet>

```

Listing 4.4: ZooTransform.xsl

Reading this template against `Zoo.xml` makes the transformation concrete:

- `<xsl:template match="/">` — the transformation's entry point, matched against the root of the source document (`/`); everything inside this template is the literal HTML the processor should output, mixed with XSLT instructions that pull data out of the source XML.
- `<xsl:for-each select="zoo/animal">` — iterates once for every `<animal>` element found under `<zoo>` in the source document, emitting one `<tr>` per iteration — directly comparable, as a concept, to the `.map(...)` loop Chapter 10's `React GetAllComponent` uses to turn a JSON array of employees into one `<tr>` per employee.
- `<xsl:value-of select="class"/>` — extracts and writes out the text content of the current `<animal>`'s own `<class>` child element, relative to wherever the enclosing `<xsl:for-each>` currently is.

The output of applying this stylesheet to `Zoo.xml` is a plain HTML table with three data rows (Monkey/20, Elephant/6, Tiger/3) under an `Animal/Quantity` header — the source data and its final, human-readable presentation are kept in two entirely separate documents, exactly mirroring Chapter 1 and Chapter 2's insistence on keeping structure and presentation apart, only here applied to *data* and its presentation rather than to a page's own markup and its own styling.

Viva Voce — Likely Questions & Answers

Q: What does it mean for an XML document to be “well-formed”, and why does this matter more for XML than for HTML?

A: A well-formed XML document has exactly one root element, every tag properly closed, tags properly nested, case-sensitive matching between opening and closing tags, and quoted attribute values. It matters more for XML than HTML because XML parsers are strict by design and will reject a malformed document outright, whereas browsers historically tolerate a wide range of malformed HTML and simply do their best to render it anyway.

Q: Why is `<class>Monkey<quantity>20</quantity></class>` in `ZooDemo.xml` invalid?

A: Because `<quantity>` opens inside `<class>` but the closing tags do not mirror that nesting in reverse order — proper XML nesting requires that a tag opened most recently must also be the next one closed, and this fragment breaks that rule.

Q: Why is `<schedule>900 to 1800</SCHEDULE>` in `EventsTryIt.xml` invalid, even though HTML would tolerate a similar mismatch?

A: XML tag names are case-sensitive, so `<schedule>` and `</SCHEDULE>` are treated as two entirely different, unrelated tag names rather than an opening and closing pair of the same tag — this is exactly the kind of leniency HTML historically allows that XML strictly does not.

Q: What is XSLT, and what problem does it solve?

A: XSLT is a language, itself written in XML, for describing how to transform one XML document into another format — most commonly HTML for direct display in a browser. It solves the problem of keeping a document’s raw, structured data separate from how that data is ultimately presented to a human reader.

Q: What does `<xsl:for-each select="zoo/animal">` do in `ZooTransform.xsl`, and what is a conceptually similar operation used later in React?

A: It iterates once for every `<animal>` element found under `<zoo>` in the source XML, emitting the enclosed markup (one `<tr>`) for each one. This is conceptually the same operation as the `.map(...)` array method React’s `GetAllComponent` (Chapter 10) uses to turn a JSON array of employees into one table row per employee.

Q: What is the practical difference between representing a piece of data as an XML attribute versus as a child element?

A: An attribute can only hold one single, simple text value, while an element can contain further nested children, mixed text, or repeat multiple times. Simple, singular data is commonly modelled as an attribute; structured or repeatable data, or data a stylesheet needs to select and iterate over independently, is more naturally modelled as an element.

Q: What problem do XML namespaces solve?

A: They disambiguate tags of the same name that mean different things in different contexts, by prefixing each tag with a namespace identifier (`xmlns:xsl="..."`) declared once, so a processor can tell, for instance, that `<xsl:template>` is an XSLT instruction rather than an application-specific tag.

Q: What is the difference between an XML document being “well-formed” and being “valid”?

A: Well-formedness means the document obeys XML’s own generic syntax rules (properly nested and closed tags, one root element, and so on), and is checked unconditionally by any XML parser. Validity means the document additionally conforms to a specific DTD or XSD

— containing the right elements, attributes, order, and data types that schema requires — and is only checked if the document is explicitly validated against that schema.

Chapter 5

React Fundamentals — JSX, Components and Props

5.1 Why React, After HTML, CSS and JavaScript

Chapters 1–3 built one static page by hand: markup, a stylesheet, and a handful of `document.getElementById()` calls wiring behaviour onto specific elements. That approach works for one small form, but it does not scale: as a page grows to dozens of interdependent, repeating pieces of UI (a row per employee, a field per form control, a card per product), manually keeping every DOM lookup and every event handler in sync with the page’s actual state becomes its own, ever-growing source of bugs. **React** is a JavaScript library for building user interfaces out of small, reusable, composable **components** — each one responsible for its own little piece of the page, each one describing *what the UI should look like for a given set of data*, and leaving the actual, tedious work of comparing that description against the real DOM and only changing what actually changed to React itself.

5.2 Declarative vs Imperative UI Programming

Chapter 3’s plain JavaScript was written **imperatively**: it gave the browser a step-by-step recipe of *how* to change the page — find this specific element, read its value, pop up an alert. React instead encourages a **declarative** style: a component’s `render()` (or function body) simply describes *what* the UI should look like for the data it currently has, as a plain description, and leaves the step-by-step work of actually figuring out which real DOM nodes need to change to React itself. Neither style is “wrong” — React’s own internals are still, underneath, full of imperative DOM manipulation — but writing components declaratively is what keeps a component’s own code focused purely on *what* the UI should be for a given state, rather than accumulating an ever-growing, hand-maintained sequence of DOM-mutation steps as an application’s UI grows more complex.

5.3 The Virtual DOM and Reconciliation

Directly manipulating the real browser DOM is comparatively slow, and a naive approach that re-build an entire page’s real DOM from scratch on every single state change would not scale. React instead keeps its own lightweight, in-memory description of what the UI should currently look like — the **Virtual DOM** — and, whenever a component’s state or props change, builds a brand-new Virtual DOM tree describing the updated UI, then compares that new tree against the previous one in a process called **reconciliation**. Only the specific, minimal set of real DOM nodes that actually differ between the two trees are then touched — an unrelated sibling component that did not change is left completely alone, even though it was technically part of the same overall re-render. This is precisely why a class component’s `render()` method (Chapters 5–8) and a function component’s own body are expected to be pure, side-effect-free descriptions of the UI: React may call them again, speculatively, purely to compute the next

Virtual DOM tree to compare, and a `render()` that itself mutated state or triggered a network call would run that side effect far more often, and far less predictably, than the developer intended.

5.4 Setting Up and Running a React Project

Every demo in this volume shares the identical project shape, generated by the standard React tooling (`create-react-app`) rather than assembled by hand:

File / Folder	Role
<code>package.json</code>	Lists the project's dependencies (<code>react</code> , <code>react-dom</code> , <code>axios</code> , and so on) and the <code>npm</code> scripts used to run it (<code>npm start</code> to launch a local development server, <code>npm run build</code> to produce an optimised production bundle).
<code>public/index.html</code>	The one, real HTML page the whole application is ultimately injected into — containing little more than an empty <code><div id="root"></div></code> for React to take over.
<code>src/index.js</code>	The application's true entry point — it renders the top-level <code><App/></code> component into that <code><div id="root"></code> , and is the one place in the entire project that talks to the real DOM directly, via <code>ReactDOM.render(<App/>, document.getElementById('root'))</code> (or, in newer React versions, <code>createRoot(...).render(<App/>)</code>).
<code>src/App.js</code>	The top-level component every other component in this volume's demos is, directly or indirectly, rendered underneath.

`npm` (Node Package Manager) is what actually downloads `package.json`'s listed dependencies into a local `node_modules` folder, and what runs the scripts `package.json` defines — the JavaScript-ecosystem equivalent of the Java track's own Maven, resolving a project's dependencies from a central registry rather than requiring every library to be downloaded and wired in by hand.

5.5 Unidirectional Data Flow

Every demo in this volume follows one consistent rule: data flows *down* a component tree, from parent to child, exclusively through props (Section 5.3); a child never directly reaches up and modifies a value owned by its own parent. When a child genuinely needs to cause a change in data its parent owns, the parent instead passes the child a *function* as a prop (commonly named something like `onChange` or `onSubmit`), and the child calls that function — handing control back up to the parent, which decides for itself how to actually update its own state in response. Data flowing one way, down, and requests to change that data flowing back up only as explicit function calls, is called **unidirectional data flow**, and it is what keeps a large component tree's data changes traceable: at any point, a change to a given piece of state can only have originated from the one component that actually owns that state (or from a function that component itself explicitly handed to a descendant), never from an arbitrary descendant reaching in and mutating it directly.

5.6 JSX — HTML’s Vocabulary, Inside JavaScript

The course’s own `demo1001-helloworld` project’s `App.js` is the smallest possible React component:

```
1 import logo from './logo.svg';
2 import './App.css';
3
4 function App() {
5   return (
6     <div className="App">
7       <header className="App-header">
8         <img src={logo} className="App-logo" alt="logo" />
9         <p>
10           Edit <code>src/App.js</code> and save to reload.
11         </p>
12         <a className="App-link" href="https://reactjs.org"
13           target="_blank" rel="noopener noreferrer">
14           Learn React
15         </a>
16       </header>
17     </div>
18   );
19 }
20
21 export default App;
```

Listing 5.1: `demo1001-helloworld/src/App.js`

The markup inside `return (...)` is **JSX** — a syntax extension to JavaScript that looks like HTML but is not actually HTML at all: it is compiled (by Babel, as the reference back-end training’s own JSX-adjacent note on Spring’s transpilation pipeline echoes) into ordinary JavaScript function calls that build up a description of the UI, before it is ever handed to the browser. Two details in this very first demo are worth calling out precisely because they are easy to trip over:

- **className, not class** — `class` is a reserved word in JavaScript, so JSX uses `className` wherever HTML would use `class`, and this is what a CSS class selector (Chapter 2) still ultimately targets once the JSX is rendered to real DOM.
- **{...} embeds a JavaScript expression** — `src= {logo}` is not the literal string `"logo"`; the curly braces tell JSX to evaluate `logo` (the imported image file) as a JavaScript expression and use its resulting value as the attribute’s value. Anything that is a valid JavaScript *expression* (a variable, a function call, a ternary) can go inside `{...}`; a full statement (an `if`, a `for` loop) cannot.

5.6.1 The Single-Root-Element Rule

Every one of this component’s returned elements is wrapped inside one single `<div>`. This is not a stylistic habit — JSX requires a component’s `return` to describe exactly one root element, because under the hood `return (...)` can only hand back one single value, not several sibling elements at once. (Modern React also allows a component to return an empty `<>...</>` *Fragment* wrapper for exactly this purpose, when an actual extra wrapping `<div>` in the rendered DOM is not wanted.)

5.6.2 Comments Inside JSX

An ordinary JavaScript `//` comment does not work directly inside JSX markup, because JSX is not simply parsed as more JavaScript at that point — a comment written inside a JSX tree must itself be wrapped as a JavaScript expression, exactly like any other embedded value: `{/*`

a comment `*/`. This exact pattern already appears in `demo1002-helloworld`'s own `App.js`, immediately above each usage of `<ModifiedCustomMessage>`.

5.7 Props — Passing Data Into a Component

A component that always renders identical, hardcoded content is of limited use. `demo1002-helloworld` introduces **props** — the mechanism by which a parent passes data down into a child component, exactly the way an HTML attribute passes a value into a built-in tag:

```

1 import React from 'react'
2
3 /* This is a Functional Component of React */
4 /* Exporting the Component so that it can be used on the view */
5 /* This accepts props, which refers to the attributes/child elements
6    passed from wherever the functional component is used */
7 export function ModifiedCustomMessage(props) {
8
9     // Note that all the elements to be returned are wrapped in one <div>,
10    // because multiple sibling elements cannot be returned independently.
11    return (
12        <div>
13            <center>
14                <h2>Welcome {props.customAttribute}</h2>
15                <h3>Passed in Child Elements are {props.children}</h3>
16            </center>
17        </div>
18    )
19 }
```

Listing 5.2: `demo1002-helloworld/src/Component/ModifiedCustomMessage.js`

```

1 import { ModifiedCustomMessage } from './Component/ModifiedCustomMessage';
2 function App() {
3     return (
4         <div className="App">
5             <h1>Demo for Custom Default Message</h1>
6
7             /* Using the Component with an attribute AND child elements */
8             <ModifiedCustomMessage customAttribute="From Tag1">
9                 <h3>Parent1: Child1</h3>
10                <h3>Parent1: Child2</h3>
11                <h3>Parent1: Child3</h3>
12            </ModifiedCustomMessage>
13            <hr/>
14            /* Using the Component with an attribute and NO child elements */
15            <ModifiedCustomMessage customAttribute="From Tag2 No Child Element"/>
16        </div>
17    );
18 }
```

Listing 5.3: `demo1002-helloworld/src/App.js` — Using the Component

Every attribute written on `<ModifiedCustomMessage customAttribute= "From Tag1">` arrives inside the component as one property on a single `props` object — `props.customAttribute`, here — and whatever is written *between* that component's opening and closing tags arrives as one further, automatically-provided special prop, `props.children`. This is precisely why `<ModifiedCustomMessage customAttribute="..." />` (a self-closing tag with no children at all) still works correctly: `props.children` is simply undefined in that case, and the component's own JSX (`{props.children}`) renders nothing extra for it.

5.7.1 Props Are Read-Only

A component must never modify its own `props` — `props` flow strictly one way, from parent to child, and a child that tried to reassign `props.customAttribute` would be silently fighting

React's own rendering model rather than actually changing what the parent sees. (Section 5.5's `CounterParameterized` demo deliberately leaves a commented-out line making exactly this point: `{this.props.message=""}` is left in the file, with a comment inviting the trainee to uncomment it and observe that a prop is read-only.) Data that a component genuinely needs to change over time belongs in **state** instead — introduced next.

5.8 Destructuring Props — A Cleaner Reading Style

`demo1013-functional-components-introduction` shows the identical component written two ways, to make an ES6 JavaScript feature directly useful in React code:

```

1 // Order of destructuring does not matter.
2 // Each destructured variable name must match the actual prop's name.
3 export function DisplayEmployeeDetailsDestructingProps(
4   {salary,departmentCode,employeeId,employeeName}) {
5   return (
6     <div>
7       <center>
8         <h2>Employee Details - De-structuring Props</h2>
9         <table>
10          <tr><th>Employee Id</th><td>{employeeId}</td></tr>
11          <tr><th>Employee Name</th><td>{employeeName}</td></tr>
12          <tr><th>Salary</th><td>{salary}</td></tr>
13          <tr><th>Department Code</th><td>{departmentCode}</td></tr>
14        </table>
15      </center>
16    </div>
17  )
18 }
```

Listing 5.4: `DisplayEmployeeDetailsDestructingProps.js`

Compared against the equivalent `DisplayEmployeeDetails(props)` written with `props.employeeId`, `props.salary` and so on repeated at every usage, destructuring the incoming `props` object directly in the function's own parameter list pulls each named property out into its own local variable up front, once, so the rest of the function body can refer to `employeeId` directly rather than `props.employeeId` every time — a small but very common React idiom, worth recognising on sight.

5.9 Class Components — State That a Component Owns Itself

Every component seen so far renders the same output for the same props, forever — it has no memory of its own. A **class component** additionally owns **state**: data that belongs to the component itself, that can change over time (typically in response to a user action), and whose change automatically triggers React to re-render that component with the new data. `demo1003-class-components`'s `Counter` is the simplest possible example:

```

1 import React, { Component } from 'react'
2
3 export class Counter extends Component {
4   constructor(props) {
5     super(props)
6     this.state = {
7       counterValue:0
8     }
9   }
10
11   incrementHandlerCount = ()=>{
12     this.setState((prevState)=>({
13       counterValue:prevState.counterValue+1
14     })),
15     ()=>console.log("Value Changed: "+this.state.counterValue)
```

```

16         )
17     }
18
19     render() {
20         return (
21             <div>
22                 <h2>Increment Counter Example</h2>
23                 <h2>Counter Value is: {this.state.counterValue}</h2>
24                 <button onClick={this.incrementHandlerCount}>Increment Counter</button>
25             </div>
26         )
27     }
28 }

```

Listing 5.5: demo1003-class-components/src/Components/Counter.js

- **constructor(props) and super(props)** — a class component's constructor must call **super(props)** before doing anything else, so that React's own **Component** base class can correctly wire up **this.props**; **this.state** is then initialised as a plain object, here holding one field, **counterValue**.
- **this.setState(...)** is the *only* correct way to change a class component's state — directly assigning **this.state.counterValue = 1** would silently fail to trigger a re-render, because React has no way of knowing the value changed. Passing a *function* of the previous state (**(prevState)=>({...})**), rather than a literal new object, is itself a deliberate habit: React may batch several **setState** calls together, so reading the guaranteed-current **prevState** is safer than reading **this.state** directly, which could still reflect a stale value at the moment the update function runs.
- **The optional second argument to setState** is a callback run only *after* the state has actually been updated and the component re-rendered — necessary because **setState** itself is asynchronous, so **console.log(this.state.counterValue)** immediately after calling **setState** (without this callback) could still print the old value.
- **onClick=this.incrementHandlerCount** passes a *reference* to the handler function, not the *result of calling* it — **onClick={this.incrementHandlerCount()}** (with the trailing parentheses) would call the handler immediately, once, during rendering, rather than waiting for an actual click.

5.10 Composition — Building Bigger Components From Smaller Ones

CounterParameterized and **MessageDecorator** together show how components compose, and how a prop's value can drive which *other* component gets rendered:

```

1  render() {
2      return (
3          <div>
4              {this.state.child==="true"
5                  ?<MessageDecorator messageDec={this.state.headingMessage}></MessageDecorator>
6                  :<h2>{this.state.headingMessage}</h2>}
7              <h2>Counter Value is: {this.state.counterValue}</h2>
8              <button onClick={this.incrementHandlerCount}>Increment Counter</button>
9          </div>
10     )
11 }
12
13 // MessageDecorator.js
14 export class MessageDecorator extends Component {
15     render() {
16         return (
17             <div>
18                 <h2 style={{fontStyle:"italic", color:"green"}}>{this.props.messageDec}</h2>

```

```

19         </div>
20     )
21 }
22 }

```

Listing 5.6: CounterParameterized.js (render method) and MessageDecorator.js

The ternary `this.state.child=== "true" ? <MessageDecorator .../> : <h2>... </h2>` is **conditional rendering** — choosing which JSX to render based on a runtime value, exactly the way an `if/else` would choose which branch of ordinary code to run, only expressed as a single JSX expression because a full `if` statement cannot be embedded directly inside `{...}`. `MessageDecorator` itself demonstrates the **inline style** approach to CSS (Chapter 2’s third option) inside JSX specifically: `style= {{fontStyle:"italic", color:"green"}}` — note the double curly braces: the outer pair is JSX’s “this is a JavaScript expression” syntax, and the inner pair is an ordinary JavaScript object literal whose keys are camelCased CSS property names (`fontStyle`, not `font-style`) rather than the hyphenated names CSS itself uses.

Viva Voce — Likely Questions & Answers

Q: What is JSX, and is it valid HTML?

A: JSX is a syntax extension to JavaScript that looks like HTML but is not HTML at all — it is compiled by a tool such as Babel into ordinary JavaScript function calls that build up a description of the UI before that description is ever turned into real DOM elements.

Q: Why does JSX use `className` instead of `class`?

A: Because `class` is a reserved keyword in JavaScript (used to declare a class), JSX uses `className` for the equivalent HTML attribute so there is no collision with JavaScript’s own syntax.

Q: Why must a component’s return statement describe exactly one root JSX element?

A: Because `return` can only hand back a single value, and several sibling JSX elements written side by side without a common wrapper are not a single value. Wrapping them in one `<div>` (or an empty `<>...</>` Fragment, when no extra DOM element is wanted) satisfies this requirement.

Q: What is `props.children`, and where does its value come from?

A: `props.children` is an automatically-provided prop holding whatever JSX was written between a component’s own opening and closing tags at the point it was used. A self-closing usage of the component (`<Comp/>`) simply leaves `props.children` as `undefined`.

Q: Why are props described as read-only, and where should data that a component needs to change over time actually live instead?

A: Props flow strictly one way, from parent to child, and React’s rendering model assumes a child never mutates what it was handed — a child that tried to reassign a prop would not correctly propagate that change back to the parent or trigger a proper re-render. Data a component itself needs to change over time belongs in that component’s own state instead.

Q: Why is `this.setState(...)` the only correct way to update a class component’s state, rather than assigning `this.state.field = newValue` directly?

A: A direct assignment changes the underlying object but gives React no signal that anything happened, so React has no reason to re-render the component and the UI would silently fall out of sync with the actual data. `setState` is the API React relies on to know a re-render is needed.

Q: Why does Counter's `setState` call pass an updater function of `prevState` rather than a plain new state object?

A: Because React may batch multiple `setState` calls together, `this.state` at the moment the update runs is not guaranteed to reflect every prior pending update. Passing a function that receives the guaranteed-current `prevState` avoids computing the new value from a value that might already be stale.

Q: Why does `onClick=this.incrementHandlerCount` omit the trailing parentheses that would otherwise call the function?

A: `onClick` expects a *reference* to a function to call later, when the click actually happens. Writing `onClick={this.incrementHandlerCount()}` would instead call the function immediately, once, while the component is being rendered, and pass its (likely *undefined*) return value to `onClick` instead of the function itself.

Q: What is the difference between writing UI code imperatively and declaratively, and which style does React encourage?

A: Imperative code gives a step-by-step recipe for how to change the page (find this element, change this property); declarative code simply describes what the UI should look like for the current data, leaving the actual DOM changes to the framework. React encourages the declarative style, keeping a component's own code focused on what the UI should be rather than a hand-maintained sequence of DOM mutations.

Q: What is the Virtual DOM, and why does React use one instead of updating the real DOM directly on every state change?

A: The Virtual DOM is React's own lightweight, in-memory description of what the UI should currently look like. On a state or props change, React builds a new Virtual DOM tree and compares it against the previous one (reconciliation), updating only the specific real DOM nodes that actually differ — far cheaper than rebuilding the entire real DOM from scratch on every change.

Q: Why must a component's `render()` method (or a function component's own body) avoid side effects such as directly triggering a network call?

A: Because React may call it again speculatively, purely to compute the next Virtual DOM tree for comparison during reconciliation, a `render()` that performed a side effect directly would trigger that side effect more often, and less predictably, than intended.

Q: What is unidirectional data flow, and why does a child component never directly modify data owned by its parent?

A: Data flows one way, down a component tree, through props; when a child needs to cause a change, its parent instead hands it a function prop (such as `onChange`) to call, and the parent decides how to update its own state in response. This keeps every change to a piece of state traceable to the one component that actually owns it, rather than allowing an arbitrary descendant to mutate it directly.

Chapter 6

React Forms, Controlled Components & Styling

6.1 Controlled Components — Letting React Own the Form’s Data

Chapter 3’s plain-JavaScript form read an `<input>`’s value out of the DOM *on demand*, at the moment it was needed (`document.getElementById("sname").value`). React forms almost always take the opposite approach: the input’s value is stored in the component’s own **state** at all times, and the `<input>` element is given that state value as its **value** attribute, with an **onChange** handler that keeps state in sync with every keystroke. An `<input>` whose displayed value is driven entirely by React state, rather than by the browser’s own internal, independent input state, is called a **controlled component** — and it is the dominant, idiomatic way to build a form in React, because it means the component’s state is always the single source of truth for what the form currently contains, rather than that truth being split between React’s state and the DOM’s own hidden internal state.

6.2 React’s Event System — SyntheticEvent

Every **event** object handed to a React event handler in this chapter (`event.target.value`, `event.target.checked`) is not, strictly speaking, the browser’s own native event object — it is a **SyntheticEvent**, a thin, cross-browser wrapper React creates around the native event, exposing the same familiar properties (`target`, `preventDefault()`, and so on) with identical behaviour regardless of which underlying browser is actually running the page. This matters purely as a compatibility detail — older browsers historically disagreed on an event’s exact API, and **SyntheticEvent** is what lets a React event handler be written once and behave identically everywhere, without the developer needing to special-case any particular browser. It is also worth knowing that React attaches its event handling once, at the root of the application, rather than wiring up a separate native listener on every single DOM element a component happens to render — a performance detail invisible from inside any individual component, but part of why React can efficiently support applications with very large numbers of interactive elements on one page.

6.3 A Form With One Handler Per Field

`demo1004-formcomponents`’s first version, `EmployeeForm`, wires up every distinct kind of form control with its own dedicated change handler:

```
1 this.state = {
2   employeeName:'', password:'', salary:'', departmentCode:'',
3   gender:'', address:'', mailingList:false,
4   checkAsset1:'', checkAsset2:'', checkAsset3:'', assetList:[]
5 }
6
```

```

7  handleTextEmployeeNameChange=(event)=> {
8      this.setState({ employeeName: event.target.value },
9          ()=>console.log('Changed Value is :'+this.state.employeeName));
10 }
11 handleDropDownDepartmentChange=(event)=>{
12     this.setState({ departmentCode: event.target.value });
13 }
14 handleRadioGenderChange=(event)=>{
15     this.setState({ gender: event.target.value });
16 }
17 handleCheckBoxMailingListChange=(event)=> {
18     this.setState({ mailingList: event.target.checked });
19 }
20
21 // In render():
22 <input type="text" value={this.state.employeeName}
23     onChange={this.handleTextEmployeeNameChange}/>
24 <select value={this.state.departmentCode} onChange={this.handleDropDownDepartmentChange}>
25     <option value="0"--Select--</option>
26     <option value="1001">Java</option>
27     <option value="1002">Dotnet</option>
28 </select>
29 <input type="radio" name="gender" value="M"
30     checked={this.state.gender === "M"} onChange={this.handleRadioGenderChange} />Male
31 <input type="checkbox" checked={this.state.mailingList}
32     onChange={this.handleCheckBoxMailingListChange}/>

```

Listing 6.1: EmployeeForm.js — excerpt

Every distinct control type needs a slightly different way of reading its current value out of the native `event`: a text input's or a select's value comes from `event.target.value`; a checkbox's on/off state comes from `event.target.checked` instead, since a checkbox's own `value` attribute is a separate, fixed thing from whether it is actually ticked. A radio button is marked `checked` by comparing the *group's shared* state value against this particular button's own value (`this.state.gender === "M"`), which is exactly why every radio button in the group must share one name (Chapter 1) and be driven from one shared piece of state, rather than each button owning its own independent flag.

6.4 A Single, Reusable Change Handler

`EmployeeFormCommenHandler` (the demo's own, intentionally-informal name for a *common* handler) replaces every one of the field-specific handlers above with exactly one function, using a JavaScript feature earlier chapters have not yet needed:

```

1  handlerCommenValuesUpdate=(event)=>{
2      let name = event.target.name;
3      let value = event.target.value;
4      if(event.target.type !== "checkbox"){
5          // name is used to target the key on our state object with the same
6          // name, using bracket syntax: {[name]: value}
7          this.setState({[name]: value},
8              ()=>console.log("Other logs Field: "+name+", Changed Value: "+value));
9      } else {
10         if(event.target.name !== "mailingList"){ // case of the "assets" checkboxes
11             let dummyVar="";
12             if(event.target.checked){
13                 dummyVar = event.target.value
14             }
15             this.setState({[name]: dummyVar},
16                 ()=>{
17                     this.setAssetList(name,dummyVar)
18                     console.log("Checkbox log, Field: "+name+", Changed Value: "+dummyVar)}
19                 );
20         } else {
21             this.setState({[name]: event.target.checked});
22         }

```

```

23   }
24 }
25
26 // In render(), every control now carries a matching "name" attribute and points
27 // at the SAME handler:
28 <input name="employeeName" type="text" value={this.state.employeeName}
29       onChange={this.handlerCommenValuesUpdate}/>
30 <input name="salary" type="text" value={this.state.salary}
31       onChange={this.handlerCommenValuesUpdate}/>

```

Listing 6.2: EmployeeFormCommenHandler.js — the common handler

The technique that makes this collapse to one handler possible is the **computed property name**, `{[name]: value}` — an ES6 JavaScript feature that lets an object literal's key itself be the *result of an expression* (here, whatever string `name` currently holds) rather than a fixed, literal key written directly in the source. Because every input in the form now carries a `name` attribute that matches its own key in `this.state` (`name="employeeName"` paired with `this.state.employeeName`), one single handler can correctly update *whichever* field actually changed, purely by reading `event.target.name` at runtime — exactly the same underlying idea, and the same practical payoff (one handler instead of many near-identical ones), that this programme's back-end training reaches for whenever it externalises a repeated piece of configuration into one place instead of duplicating it per call site.

6.5 Client-Side Validation, Revisited From React's Own State

`demo1005-formcomponents-validations` adds a dedicated `...Error` field to state for every validated input, and a `validate()` method that checks every field's current state value before allowing submission:

```

1  handleSubmit =(event)=>{
2    event.preventDefault() // prevents the full-page reload a native form submit
3                             // would otherwise trigger
4    if(this.validate()){
5      alert(`${this.state.employeeName}- ${this.state.password}- ...`);
6      this.setState(initialState) // reset the form on success
7    }
8  }
9
10 validate = ()=>{
11   let employeeNameError='';
12   let passwordError='';
13   // ...
14   if(!this.state.employeeName.includes("@")){
15     employeeNameError="Employee Name should contain @";
16   }
17   if(this.state.password.length===0){
18     passwordError="Password is Mandatory";
19   }
20   if(this.state.address.length<=10){
21     addressError="Address is Mandatory and should have minimum 10 characters";
22   }
23   if(employeeNameError || passwordError || /* ...other errors... */){
24     // When the key name to set and the variable to read from share
25     // the same identifier, {employeeNameError} is short for
26     // {employeeNameError: employeeNameError} -- another ES6 feature.
27     this.setState({employeeNameError,passwordError, /* ... */})
28     return false;
29   }
30   return true
31 }
32
33 // In render(), each field's error is displayed directly beneath its control:
34 <input name="employeeName" type="text" value={this.state.employeeName}
35       onChange={this.handlerCommenValuesUpdate}/>
36 <div style={{fontSize:10,color:"red",fontWeight:"bold",fontStyle:"italic"}}>
37   {this.state.employeeNameError}

```

```
38 </div>
```

Listing 6.3: EmployeeFormCommenHandler.js (validations) — excerpt

Every one of these checks is exactly the same `.length/.includes` logic Chapter 3’s plain-JavaScript `validateForm()` exercise was pointed towards — the only structural change is that the value being validated is read from `this.state` rather than from a fresh DOM lookup, and the resulting error message is itself stored back into state so that JSX can conditionally render it (`{this.state.employeeNameError}` renders nothing at all when the string is empty). `event.preventDefault()` is the same native DOM API Chapter 3 named, stopping the browser’s own default full-page-reload form submission so the whole submit-and-validate cycle can be handled by React instead.

6.6 Three Ways to Style a React Component

`demo1006-formcomponents-css` rebuilds the identical validated form three times, once per CSS-attachment strategy named back in Chapter 2, so the trade-offs can be compared directly inside React specifically:

```
1 // 1) INLINE STYLE -- a JavaScript object literal passed to the "style" prop.
2 //   Scoped to exactly this one element; cannot use pseudo-classes like :hover.
3 const divErrorMessageStyle = {
4   fontSize:10, color:"red", fontWeight:"bold", fontStyle:"italic"
5 }
6 <div style={divErrorMessageStyle}>{this.state.employeeNameError}</div>
7
8 // 2) EXTERNAL STYLESHEET -- an ordinary .css file, imported for its side effect
9 //   of being added to the page; classes are applied via className.
10 import "../Message.css";
11 <div className="divErrorMessageStyle">{this.state.employeeNameError}</div>
12
13 /* Message.css */
14 .divErrorMessageStyle {
15   font-size:10px; color:red; font-weight:bold; font-style:italic
16 }
17
18 // 3) CSS MODULE -- EmployeeForm.module.css, imported directly (its selectors
19 //   apply globally to any element they match on the page, exactly like an
20 //   ordinary external stylesheet, in this course’s specific demo setup):
21 import "../EmployeeForm.module.css";
22 /* EmployeeForm.module.css */
23 table, th, td { border: 1px solid black; }
```

Listing 6.4: Three styling approaches, side by side

Approach	When It Is the Right Choice
Inline style object	A one-off style that depends on a runtime value already held in state or props (a colour computed from data, for instance) — but it cannot express pseudo-classes (<code>:hover</code>) or media queries, and does not benefit from being reused across components.
External stylesheet	A style meant to be shared, unchanged, across many elements or many components — exactly Chapter 2’s original recommendation, now applied inside a component-based project.
CSS Modules	An external stylesheet whose class names are scoped locally to the importing component by the build tool, avoiding one component’s class name accidentally colliding with another’s elsewhere in a large application.

Viva Voce — Likely Questions & Answers

Q: What is a `SyntheticEvent`, and why does React use one instead of handing an event handler the browser's raw native event directly?

A: `SyntheticEvent` is a thin, cross-browser wrapper React creates around the native event, exposing the same properties and methods (`target`, `preventDefault()`) with identical behaviour regardless of which browser is running the page — letting an event handler be written once without special-casing any particular browser's own native event API.

Q: What is a controlled component, and why is it the dominant pattern for React forms?

A: A controlled component is a form element (an `<input>`, `<select>`, etc.) whose displayed value is driven entirely by a React component's own state, kept in sync via an `onChange` handler. It is dominant because it makes the component's state the single, authoritative source of truth for what the form contains, rather than splitting that truth between React's state and the DOM's own hidden internal input state.

Q: Why does a checkbox's change handler read `event.target.checked` while a text input's reads `event.target.value`?

A: A checkbox's own `value` attribute is a fixed, separate thing from whether it is currently ticked, so its on/off state must be read from the dedicated `checked` property instead. A text input has no such separate concept — its current content *is* its `value`.

Q: What is the computed-property-name technique (`{[name]: value}`) used in `handlerCommonValuesUpdate`, and what problem does it solve?

A: It is an ES6 JavaScript feature letting an object literal's key be the result of an expression rather than a fixed identifier. It lets one single change handler correctly update whichever field actually fired the event, by reading that field's own `name` attribute at runtime and using it as the state key to update, rather than needing one hand-written handler per field.

Q: Why must every radio button in a group be driven from one shared state value, rather than each carrying its own local checked flag?

A: Only one radio button in a group can be selected at a time, and a single shared state value (compared against each button's own `value` via `checked={this.state.gender===this.value}`) is what naturally enforces that mutual exclusivity — independent per-button flags could drift out of sync and allow more than one button to appear checked simultaneously.

Q: Why does `handleSubmit` call `event.preventDefault()` before proceeding?

A: Without it, a native HTML form submission triggers a full-page reload/navigation by default, which would discard the entire React application's in-memory state. `preventDefault()` stops that native browser behaviour so the submission can be handled entirely by React's own `validate()/setState` logic instead.

Q: What is the practical trade-off between an inline style object and an external stylesheet in a React component?

A: An inline style object can easily incorporate a runtime value already sitting in state or props, but cannot express pseudo-classes or media queries and is not reusable elsewhere. An external stylesheet is reusable and can express the full range of CSS, but cannot directly reference a component's own JavaScript values.

Chapter 7

Error Boundaries and the Context API

7.1 The Need for Error Boundaries

demo1007-error-boundries-need deliberately builds a component that can throw, so that the *consequence* of an uncaught error in React is directly visible before the fix for it is introduced:

```
1 constructor(props) {
2   super(props)
3   let defText=this.props.defaultText;
4   if(defText===null ||defText.length===0){
5     throw new Error("Please give valid for default text");
6   }
7   this.state = { employeeId:0, employeeData:this.props.defaultText, errorMessage:"" }
8 }
```

Listing 7.1: SearchEmployeeById.js — excerpt

```
1 <SearchEmployeeById defaultText=""></SearchEmployeeById>
```

Listing 7.2: App.js — the component is used with an empty defaultText

Passing an empty string deliberately triggers the constructor's own `throw`. Without anything further in place, an error thrown during rendering (or, as here, during construction) does not stay neatly confined to the one component that threw it: by default, React unmounts the *entire* component tree it was rendering, and the whole page goes blank — one broken child can take down every sibling on the page along with it, including components that themselves had nothing wrong with them at all. This is precisely the gap an **Error Boundary** exists to close.

7.2 CustomErrorBoundary — Containing a Failure

demo1008-error-boundries introduces exactly that boundary:

```
1 export class CustomErrorBoundary extends Component {
2   constructor(props) {
3     super(props)
4     this.state = { errorFlag:false, message: "" }
5   }
6   componentDidCatch(error,info){
7     console.log("[Custom Error Boundary, componentDidCatch()]: Logging Error..")
8     console.log(error);
9     console.log(info)
10  }
11  static getDerivedStateFromError(error){
12    return { errorFlag:true, message:error.message }
13  }
14  render() {
15    if(this.state.errorFlag){
16      return( <h3>Some error happened, {this.state.message}</h3> )
17    } else {
18      return ( <div>{this.props.children}</div> )
19    }
20  }
21 }
```

Listing 7.3: CustomErrorBoundary.js


```

1 <CustomErrorBoundary>
2   <SearchEmployeeById defaultText=""></SearchEmployeeById>
3 </CustomErrorBoundary>
4 <EmployeeFormCommenHandlerInLineV2></EmployeeFormCommenHandlerInLineV2>

```

Listing 7.4: App.js — wrapping only the component that might fail

An Error Boundary is an ordinary class component that becomes one specifically by implementing one or both of two special lifecycle methods:

Method	Role
<code>static getDerivedStateFromError(error)</code>	Called during the <i>render</i> phase after a descendant has thrown an error. It must return an object used to update the boundary's own state (here, flipping <code>errorFlag</code> to <code>true</code>), so the boundary's next render can show a fallback UI instead of the broken subtree. Being <code>static</code> , it must not, and cannot, cause further side effects (like logging) itself.
<code>componentDidCatch(error, info)</code>	Called during the <i>commit</i> phase, <i>after</i> the fallback UI has already been committed — this is where side effects like logging the error (here, to the console; in a production system, typically to a monitoring service) belong, precisely because it is not called during rendering itself.

Wrapping only `<SearchEmployeeById>` — and deliberately *not* the sibling `<EmployeeFormCommenHandlerInLineV2>` — in `<CustomErrorBoundary>` is itself the point being demonstrated: an error inside the boundary's children is now contained to that boundary's own fallback UI, while every component outside the boundary keeps rendering and working normally, exactly the isolation a single global, uncontained failure did not provide in the previous section's demo.

An Error Boundary only catches errors thrown while React is rendering, in lifecycle methods, or in constructors of the components below it in the tree. It does *not* catch errors inside event handlers (those are ordinary JavaScript `try/catch` territory, exactly as `SearchEmployeeById`'s own `getEmployeeDetails` handler already demonstrates), inside asynchronous code such as a `setTimeout` callback or a Promise's `.then/.catch`, or during server-side rendering.

7.3 The Context API — Avoiding Prop Drilling

Every prop seen so far has flowed from a parent directly to its immediate child. `demo1009-react-context` shows what happens once a value is needed by a component nested *several* levels deep, and what React offers instead of threading that value through every intermediate component's own props purely so it can be passed further down:

```

1 import React from 'react'
2
3 // Step 1: create a Context object
4 const context = React.createContext();
5
6 // context.Provider / context.Consumer are exported for use anywhere in the app
7 const CustomReactContextProvider = context.Provider
8 const CustomReactContextConsumer = context.Consumer
9
10 export {CustomReactContextProvider, CustomReactContextConsumer}

```

Listing 7.5: `reactContextProvider.js`


```

1 // App.js -- the Provider sits at the top of the tree and supplies one value
2 <CustomReactContextProvider value="Demo Value">
3   <ComponentA></ComponentA>
4 </CustomReactContextProvider>
5
6 // ComponentA -> ComponentB -> ComponentC -- neither A nor B ever mentions the
7 // context value at all; they simply render their own child.
8 export class ComponentA extends Component {
9   render() { return ( <div><h1>From ComponentA</h1><ComponentB></ComponentB></div> ) }
10 }
11
12 // ComponentC, three levels down, consumes the value directly:
13 <CustomReactContextConsumer>
14   {
15     (contextValue)=>{
16       return <React.Fragment>
17         Hello from Child "C"    {contextValue?contextValue:"Context Value is Empty"}
18       </React.Fragment>
19     }
20   }
21 </CustomReactContextConsumer>
22 <ComponentD></ComponentD>

```

Listing 7.6: App.js – ComponentB.js – ComponentC.js – ComponentD.js – ComponentE.js

App → ComponentA → ComponentB → ComponentC → ComponentD → ComponentE is a five-level-deep chain, and the course's own demo deliberately has ComponentC, ComponentD and ComponentE each independently consume the identical "Demo Value" — without ComponentA or ComponentB ever needing to know that value exists at all, let alone having to accept and re-forward it as a prop purely on its way further down. Manually forwarding a prop through every intermediate component that does not itself use it, purely so a distant descendant can eventually receive it, is called **prop drilling** — and it is precisely the maintenance burden the Context API exists to remove: `React.createContext()` creates one shared channel; a `<Provider value="...">` anywhere near the top of a subtree supplies the value; and any `<Consumer>` anywhere within that subtree, however deeply nested, can read it directly, with every component in between remaining completely unaware the context exists.

Viva Voce — Likely Questions & Answers

Q: What happens to a React application, by default, when a component throws an error during rendering with no Error Boundary in place?

A: React unmounts the entire component tree it was in the middle of rendering, so one broken component can take down every sibling and ancestor along with it, typically leaving the whole page blank rather than failing gracefully in just the one broken part of the UI.

Q: What are the two lifecycle methods that make a class component an Error Boundary, and what is each one responsible for?

A: `static getDerivedStateFromError(error)`, called during the render phase, returns updated state used to render a fallback UI. `componentDidCatch (error, info)`, called during the commit phase after the fallback UI is already showing, is where side effects like logging the error belong.

Q: Why does the demo wrap only `<SearchEmployeeById>` in the Error Boundary, and not every component on the page?

A: So that a failure inside `SearchEmployeeById` is contained to that boundary's own fallback UI specifically, while every component outside the boundary continues rendering and working normally — an Error Boundary's protection only extends to the subtree actually wrapped inside it.

Q: Does an Error Boundary catch an error thrown inside an onClick event handler?

A: No. Error Boundaries only catch errors thrown during rendering, in lifecycle methods, or in constructors of components below them in the tree. Errors thrown inside event handlers must be handled with an ordinary JavaScript `try/catch`, exactly as `SearchEmployeeById`'s own `getEmployeeDetails` handler does.

Q: What is “prop drilling”, and what problem does the Context API solve?

A: Prop drilling is manually forwarding a prop's value through every intermediate component in a chain purely so a distant descendant can eventually receive it, even though the intermediate components never use that value themselves. The Context API solves this by letting a `Provider` supply a value once, near the top of a subtree, that any `Consumer` within that subtree can read directly, however deeply nested, without any component in between needing to know the value exists.

Q: In the Context demo, why do ComponentA and ComponentB never mention CustomReactContextConsumer at all?

A: Because they do not themselves need the context's value — only `ComponentC`, `ComponentD` and `ComponentE` do. Since context is read directly by whichever descendant needs it, rather than being passed prop by prop through every level, the components that have no use for the value simply never reference it.

Chapter 8

The Component Lifecycle — Class Components in Depth

8.1 Why a Component's Life Has Named Phases

A class component is not simply created once and left alone — it is constructed, rendered, possibly updated many times in response to new props or state, and eventually removed from the page entirely. React exposes each of these moments as a named **lifecycle method** a class component can optionally implement, giving a component a well-defined place to run logic tied specifically to *when*, in that life, something is happening — fetching data once the component is actually on the page, deciding whether an update is even worth re-rendering, or cleaning up a subscription before the component disappears.

8.2 Mounting — A Component's First Appearance

`demo1010-life-cycle-methods` builds a parent/child pair, each logging every lifecycle method it implements, specifically so the *order* those methods fire in — both within one component, and between a parent and its child — becomes directly observable in the browser console:

```
1 export class LifecycleA extends Component {
2   constructor(props) {
3     super(props)
4     this.state = { name: 'Parent State' }
5     console.log("From LifecycleA constructor")
6   }
7
8   static getDerivedStateFromProps(props, state){
9     console.log("From LifecycleA getDerivedStateFromProps()")
10    return null; // returning null means: do not change state from this hook
11  }
12
13  componentDidMount(){
14    console.log("LifecycleA component did mount")
15  }
16
17  render() {
18    console.log("From LifecycleA render()")
19    return ( <div><h3>LifecycleA</h3><LifecycleB></LifecycleB></div> )
20  }
21 }
```

Listing 8.1: LifecycleA.js (parent) and LifecycleB.js (child) — mounting

Running this demo prints, in order: LifecycleA constructor → LifecycleA getDerivedStateFromProps() → LifecycleA render() → LifecycleB constructor → LifecycleB getDerivedStateFromProps() → LifecycleB render() → LifecycleB component did mount → LifecycleA component did mount.

Mounting Method	Fires
<code>constructor(props)</code>	First — used to initialise state and, if needed, bind event-handler methods (not required here, since every handler in this course’s demos is already written as an arrow function, which binds <code>this</code> automatically).
<code>static getDerivedStateFromProps(props, state)</code>	Immediately before every render (mounting <i>and</i> every subsequent <code>render()</code>) — must be declared <code>static</code> or it is silently ignored; returns either an object to merge into state, or <code>null</code> to leave state unchanged, as every one of this course’s demos deliberately does.
<code>render()</code>	Returns the JSX describing what should appear on screen — must be a pure function of <code>props</code> and <code>state</code> , with no side effects of its own.
<code>componentDidMount()</code>	Fires once, immediately after the component (and its own DOM nodes) have actually been inserted into the page — the correct place for one-time work that needs a real, mounted DOM node to exist, such as an initial data fetch (Chapter 10) or setting up a subscription.

The specific ordering the console output reveals — every child finishes mounting (including its own `componentDidMount`) *before* its parent’s own `componentDidMount` fires — matters directly the first time a developer needs to reach into a child’s rendered DOM node from a parent’s `componentDidMount`: by the time the parent’s own `componentDidMount` runs, every child is already guaranteed to be fully mounted.

8.3 Updating — What Happens When Props or State Change

`demo1011-life-cycle-methods-update` adds a button that changes `LifeCycleA`’s own state, and adds the further lifecycle methods that only fire during an *update*, never during the initial mount:

```

1  changeStateHandler=()=>{
2    this.setState(
3      (prevState)=>({ name: prevState.name+"_Ch" })
4    )
5  }
6
7  // Update method
8  shouldComponentUpdate(nextProps,nextState){
9    console.log("LifeCycleA  shouldComponentUpdate");
10   return true;
11 }
12
13 getSnapshotBeforeUpdate(prevProp,prevState){
14   console.log("LifeCycleA  getSnapshotBeforeUpdate");
15   return null;
16 }
17
18 componentDidUpdate(prevProp,prevState,snapshot){
19   console.log("LifeCycleA  componentDidUpdate: "+snapshot)
20 }
```

Listing 8.2: `LifeCycleA.js` — update-only lifecycle methods

Update Method	Role
<code>shouldComponentUpdate(nextProps, nextState)</code>	Called <i>before</i> re-rendering; returning false lets a component skip an unnecessary re-render entirely (a manual performance optimisation) — the demo always returns true , meaning “always proceed with the update”, but the hook exists specifically so that decision can be made conditionally.
<code>getSnapshotBeforeUpdate(prevProps, prevState)</code>	Called <i>before</i> the freshly-rendered output is actually committed to the DOM, while the <i>previous</i> DOM is still visible — useful for capturing something about the DOM as it currently stands (such as a scroll position) before it changes; its return value is passed as the third argument to <code>componentDidUpdate</code> .
<code>componentDidUpdate(prevProps, prevState, snapshot)</code>	Fires <i>immediately</i> after an update has actually been committed to the DOM — the update-phase counterpart to <code>componentDidMount</code> , and a common place to react to a prop/state change by, for example, re-fetching data.

Clicking the button in this demo triggers, in order: `LifeCycleA getDerivedStateFromProps()` → `LifeCycleA shouldComponentUpdate` → `LifeCycleA render()` → (the same four-step sequence repeats for `LifeCycleB`, since a parent re-rendering by default re-renders its children too) → `LifeCycleB getSnapshotBeforeUpdate` → `LifeCycleA getSnapshotBeforeUpdate` → `LifeCycleB componentDidUpdate` → `LifeCycleA componentDidUpdate` — notice that `getSnapshotBeforeUpdate` and `componentDidUpdate` both fire child-before-parent, the identical ordering `componentDidMount` followed during the earlier mounting phase.

8.4 Unmounting

Although neither course demo builds an example that removes a component from the page, it is worth naming the third phase directly, since it is the natural counterpart to `componentDidMount`: `componentWillUnmount()` fires immediately *before* a component is removed from the DOM, and is where any subscription, timer, or event listener set up in `componentDidMount` must be torn down — failing to do so is a common source of a *memory leak*, where a removed component’s own timer or subscription keeps running and holding a reference to state that can never be garbage-collected, and that may even try to call `setState` on a component that no longer exists.

Viva Voce — Likely Questions & Answers

Q: What are the three broad phases of a class component’s lifecycle?

A: Mounting (the component is being created and inserted into the DOM for the first time), updating (the component’s props or state have changed and it is re-rendering), and unmounting (the component is being removed from the DOM).

Q: Why must `getDerivedStateFromProps` be declared with the static keyword?

A: React specifically looks for a **static** method of that exact name on the component’s class; a non-static version is simply never called as part of the lifecycle at all, even though it would compile without error.

Q: In the mounting demo’s console output, why does `LifeCycleB`’s own `componentDidMount` fire before `LifeCycleA`’s?

A: Because a parent's own `componentDidMount` only fires once every child it rendered has itself finished mounting — children always complete their own mounting sequence, `componentDidMount` included, before their parent's `componentDidMount` runs.

Q: What does returning false from `shouldComponentUpdate` actually do?

A: It tells React to skip re-rendering this component (and, by default, its subtree) for this particular update entirely — a manual performance optimisation for cases where a developer can determine, from the incoming `nextProps/nextState`, that nothing the component actually renders would change.

Q: What is the relationship between `getSnapshotBeforeUpdate`'s return value and `componentDidUpdate`?

A: Whatever `getSnapshotBeforeUpdate` returns is passed as the third argument (`snapshot`) to the subsequent `componentDidUpdate` call, giving `componentDidUpdate` access to information about the DOM as it stood *immediately before* the update was committed, which would otherwise already be gone by the time `componentDidUpdate` itself runs.

Q: What is `componentWillUnmount` used for, and what can go wrong if it is omitted?

A: It is the correct place to tear down anything set up in `componentDidMount` — a timer, a subscription, an event listener — immediately before the component is removed from the DOM. Omitting this cleanup is a common cause of a memory leak, where the removed component's own timer or subscription keeps running, potentially even attempting to call `setState` on a component instance that no longer exists.

Chapter 9

Modern React with Hooks — `useState`, `useEffect`, `useReducer`

9.1 Function Components, Revisited

Chapter 5 already introduced the function component as the simpler alternative to a class component — `demo1013-functional-components-introduction`’s `DisplayEmployeeDetails` took `props` and returned JSX, with no `state`, no `render()` method, and no lifecycle methods at all. For a long time, that was the ceiling of what a function component could do: any component that needed its own state, or needed to run code tied to mounting or updating, had to be written as a class, exactly as every component in Chapters 5 through 8 was. **Hooks** remove that ceiling: they are plain functions, each with a name starting `use...`, that let a function component opt into state, lifecycle-style effects, and more, without ever being converted into a class.

9.2 The Rules of Hooks

Every Hook in this chapter must be used according to two strict rules, both enforced in practice by a standard linting plugin most React project setups (including `create-react-app`, Section 5.4) enable automatically:

- **Only call Hooks at the top level.** Never call a Hook inside a loop, a condition, or a nested function — every Hook call in a component must run, in the exact same order, on every single render. React identifies which piece of state or effect a given `useState/useEffect` call corresponds to purely by the *order* it was called in during that render, not by any name; conditionally skipping a Hook call on one render but not another would silently shift every subsequent Hook’s identity out of sync with its previous render.
- **Only call Hooks from React functions.** A Hook may only be called directly from the body of a function component, or from another Hook (a *custom Hook*, covered below) — never from a plain, ordinary JavaScript function, and never from a class component (which uses the entirely separate lifecycle-method mechanism of Chapter 8 instead).

9.3 `useState` — State Inside a Function Component

`demo1014-use-state-hook` shows `useState` used correctly, and, first, used in a way that looks correct but is not:

```
1 export function EmployeeWithBugs() {
2   const initialObj={employeeName:'',salary:'',departmentCode:''}
3   const [newObj,setCommenHandler] = useState(initialObj)
4
5   return (
6     // ...
7     <input type="text" value={newObj.employeeName}
8       onChange={event=>setCommenHandler({employeeName:event.target.value})}/>
9     <input type="text" value={newObj.salary}
10      onChange={event=>setCommenHandler({salary:event.target.value})}/>
```

```

11      // ...
12    )
13  }

```

Listing 9.1: EmployeeWithBugs.js — a subtle bug

```

1  <input type="text" value={newObj.employeeName}
2    onChange={event=>setCommenHandler({...newObj,employeeName:event.target.value})}/>
3  <input type="text" value={newObj.salary}
4    onChange={event=>setCommenHandler({...newObj,salary:event.target.value})}/>

```

Listing 9.2: EmployeeWithBugsFixed.js — the fix

Typing into the *buggy* version's Employee Name field, and then into its Salary field, silently *loses* whatever was typed into Employee Name: unlike a class component's `this.setState(...)`, which automatically **merges** the object passed to it into the existing state, `useState`'s own up-dater function **replaces** the entire state value outright. `setCommenHandler({salary:event.target.value})` therefore does not update *just* salary — it replaces the whole state object with a brand-new one that has *only* a salary field, discarding `employeeName` and `departmentCode` entirely. The fix, `setCommenHandler({...newObj, salary:event.target.value})`, uses the JavaScript **spread operator** to first copy every existing field out of `newObj`, and only then overwrite the one field that actually changed — `demo1014`'s own `SpreadOperator.js` demo isolates exactly this one operator, independently of `useState`, specifically so its behaviour (`{...obj1, employeeName: 'FinalChanged'}` keeps every field of `obj1` except the one explicitly overridden afterward) is understood on its own before being relied on here.

This is the single most important practical difference between class-component state and Hook state to internalise: `this.setState({salary:...})` on a class component *merges* the given object into existing state automatically; `useState`'s setter function *replaces* state outright, so updating one field of an object held in a single `useState` call always requires spreading the previous value first.

9.4 useEffect — Running Code at the Right Moment

Hooks need their own way to express what class components expressed with `componentDidMount` / `componentDidUpdate` / `componentWillUnmount` (Chapter 8). `useEffect` is that single, unified mechanism, and its behaviour is controlled entirely by its optional second argument — the **dependency array** — as `demo1015-use-effect-hook` shows side by side:

```

1  // Runs after EVERY render -- mounting AND every subsequent state/prop update.
2  // (No dependency array argument at all.)
3  useEffect(()=>{
4    console.log("Invoking Mounting and update Effect");
5  })
6
7  // Runs ONCE, only after the initial mount -- never again on later updates.
8  // (An empty dependency array.)
9  useEffect(()=>{
10   console.log("Invoking Mounting Effect");
11 }, [])

```

Listing 9.3: EmployeeWithBugsFixed1.js — three different `useEffect` calls

```

1  // Runs on mount, AND again every time newEmployeeObj specifically changes --
2  // but NOT when newAssetObj changes.
3  useEffect(()=>{
4    console.log("Invoking Mounting and update Effect For Employee");
5  }, [newEmployeeObj])
6
7  useEffect(()=>{

```



```

8   console.log("Invoking Mounting and update Effect For Asset");
9 },[newAssetObj])

```

Listing 9.4: EmployeeWithBugsFixed2.js — effects scoped to specific state

Dependency Array	Effect Runs
Omitted entirely	After every single render — mount and every update, for any reason.
[] (empty array)	Only once, after the initial mount — the Hook equivalent of <code>componentDidMount</code> , and, when the effect's own function returns a cleanup function, that cleanup's equivalent of <code>componentWillUnmount</code> .
[someValue]	After the initial mount, and again every time <code>someValue</code> specifically changes between renders — letting an effect react to exactly one piece of state or props, rather than to any change whatsoever.

Getting the dependency array wrong is one of the most common React bugs in practice: an effect that reads a piece of state or props but omits it from the dependency array can silently keep using a stale, captured value from whichever render it was originally defined in, while an effect with an empty array that was actually meant to react to a changing prop simply never re-runs at all.

9.5 useReducer — State Transitions as an Explicit Function

For a single, simple value, `useState` is usually the more direct choice. Once a component's next state genuinely depends on *which action* occurred, not just on a new value being supplied directly, `useReducer` expresses that decision far more clearly. `demo10030-hooks-use-reducer`'s `Counter` is the simplest possible reducer:

```

1  // Step 2: the initial state
2  const initialState=0;
3
4  // Step 3: a reducer function -- (currentState, action) => newState
5  const reducerfunc =(currentState,action)=>{
6      var retValue = initialState;
7      switch(action){
8          case "Increment":  retValue=currentState+1; break;
9          case "Decrement":  retValue=currentState-1; break;
10         case "Reset":      retValue=initialState;   break;
11     }
12     return retValue
13 }
14
15 // Step 1: wiring the reducer up
16 const [newStateCount,actionDispatcher]=useReducer(reducerfunc,initialState)
17
18 // Step 4: reading the state, and dispatching actions from event handlers
19 <h3>Count is {newStateCount}</h3>
20 <button onClick={()=>actionDispatcher('Increment')}>Increment</button>
21 <button onClick={()=>actionDispatcher('Decrement')}>Decrement</button>
22 <button onClick={()=>actionDispatcher('Reset')}>Reset</button>

```

Listing 9.5: Counter.js — the four-step useReducer pattern

`useReducer(reducerfunc, initialState)` returns exactly two things, in a fixed order: the *current* state (`newStateCount`), and a **dispatch function** (`actionDispatcher`) used to request a state change. Crucially, a component never computes its own next state directly — every

event handler simply *dispatches an action* (here, a plain string describing what happened: 'Increment'), and the one, centralised `reducerfunc` is the only place that actually decides what the next state should be for that action. `CounterStateObjAndActionObj.js` extends the identical pattern to a richer, object-shaped action:

```

1  const reducerfunc =(currentState,action)=>{
2    var retValue = currentState
3    switch(action.actionType){
4      case "Increment":
5        retValue={firstCounter: currentState.firstCounter+action.value}; break;
6      case "Decrement":
7        retValue={firstCounter: currentState.firstCounter-action.value}; break;
8      case "Reset":
9        retValue=initialState; break;
10   }
11   return retValue
12 }
13 // ...
14 <button onClick={()=>actionDispatcher({actionType:'Increment', value:5})}>Increment 5</button>

```

Listing 9.6: `CounterStateObjAndActionObj.js` — excerpt

Dispatching `{actionType:'Increment', value:5}` rather than a bare string lets one "Increment" case carry additional data (*how much* to increment by) alongside *what* happened — the same underlying shape Chapter 10's data-fetching reducer reuses, dispatching `{type:'FETCH_SUCCESS', payload:response.data}` once an HTTP call actually completes.

9.6 useRef — A Value That Persists Without Triggering a Re-render

`useState` is the right tool whenever a changing value should cause the UI to re-render. Occasionally a component needs to hold onto a value across renders — a direct handle onto a real DOM node, or a plain mutable variable — *without* that value's own change ever causing a re-render by itself. `useRef` is built for exactly this:

```

1  function SearchBox() {
2    const inputRef = useRef(null);
3
4    useEffect(()=>{
5      inputRef.current.focus(); // runs once, after mount, moving keyboard
6                                // focus straight into the input
7    },[]);
8
9    return <input ref={inputRef} type="text" />
10 }

```

Listing 9.7: `useRef` – focusing a DOM input directly

`useRef(initialValue)` returns a single, plain object of the shape `{current: initialValue}`, and that same object is returned again, completely unchanged, on every subsequent render — reading or writing `ref.current` never itself triggers a re-render, unlike updating state would. Attaching a ref to a JSX element's own `ref` attribute (as above) is what gives a function component direct access to that element's real, underlying DOM node — the Hook-based equivalent of a class component's `React.createRef()`, and one of the few places a modern, Hook-based component still deliberately reaches past the declarative model of Section 5.2 to touch the real DOM directly.

9.7 useMemo and useCallback — Avoiding Unnecessary Recomputation

Because a function component's entire body re-runs on every render, any expensive calculation written directly in that body, or any function newly created inside it, is recomputed or recreated every single time — even when the values it actually depends on have not changed at all since the previous render.

```

1 // useMemo -- recompute the value only when employeeList actually changes
2 const totalSalary = useMemo(()=>{
3   return employeeList.reduce((sum,emp)=>sum+emp.salary, 0);
4 },[employeeList]);
5
6 // useCallback -- keep the SAME function reference across renders, as long as
7 // employeeId has not changed, rather than creating a brand-new function every render
8 const handleDelete = useCallback(()=>{
9   axios.delete("http://localhost:8095/emp/controller/deleteEmp/"+employeeId);
10 },[employeeId]);

```

Listing 9.8: useMemo and useCallback

`useMemo(fn, deps)` re-runs `fn` and caches its return value only when one of the values in `deps` has changed since the previous render — otherwise it simply hands back the previously-computed value. `useCallback(fn, deps)` is the same underlying memoisation idea applied specifically to a *function itself* rather than to a computed value — useful in particular when that function is passed down as a prop to a child wrapped in `React.memo`, since a genuinely stable function reference lets that child correctly skip a re-render it would otherwise wrongly perform every time its parent re-rendered and handed it what looked like a brand-new function prop.

9.8 Custom Hooks — Extracting Reusable Stateful Logic

Once the same combination of Hooks starts recurring across more than one component — exactly the `useState+useEffect+axios` combination repeated across `GetAllEmployeeDetails` and its siblings in Chapter 10 — that combination can be extracted into a **custom Hook**: an ordinary JavaScript function, named by convention starting with `use`, that itself calls other Hooks internally:

```

1 function useEmployeeList(url) {
2   const [data,setData] = useState([]);
3   const [errorMessage,setErrorMessage] = useState("");
4
5   useEffect(()=>{
6     axios.get(url)
7       .then(response=>setData(response.data))
8       .catch(error=>setErrorMessage(error.message));
9   },[url]);
10
11   return { data, errorMessage };
12 }
13
14 // Used inside any function component, exactly like a built-in Hook:
15 function GetAllEmployeeDetails() {
16   const { data, errorMessage } = useEmployeeList("http://localhost:8095/emp/controller/getDetails");
17   // ... render data / errorMessage ...
18 }

```

Listing 9.9: A custom Hook wrapping the fetch-employees pattern

A custom Hook shares *logic* between components — the fetching, the loading/error handling — without sharing *state*: every component that calls `useEmployeeList(...)` gets its own,

entirely independent `data/errorMessage` state, exactly as if that component had written out the `useState/useEffect` calls itself. This is the Hook-based answer to a problem class components had no clean solution for at all — reusing stateful behaviour across otherwise-unrelated components without resorting to inheritance or the more awkward, older higher-order-component/render-props patterns.

Viva Voce — Likely Questions & Answers

Q: What is a Hook, and what capability do Hooks give a function component that it did not have before?

A: A Hook is a plain function, conventionally named starting with `use`, that lets a function component opt into capabilities — state, lifecycle-style effects, context, and more — that were previously only available inside a class component, without ever converting that function component into a class.

Q: Why does `setCommenHandler({salary:event.target.value})` silently erase `employeeName` and `departmentCode` in `EmployeeWithBugs`?

A: Because `useState`'s setter function replaces the entire state value with whatever is passed to it, rather than merging it into the existing state the way a class component's `this.setState(...)` does. Passing an object with only one field therefore discards every other field that was previously held in state.

Q: What does the spread operator do in `{...newObj, salary:event.target.value}`, and why does it fix the bug above?

A: It first copies every existing property from `newObj` into a new object literal, and only then applies the explicit `salary` override that follows. Because every other field is copied forward unchanged, only `salary` actually changes, leaving the rest of the state intact.

Q: What is the effect of passing an empty array, `[]`, as `useEffect`'s second argument?

A: The effect runs exactly once, immediately after the component's initial mount, and never again on any subsequent re-render — the Hook-based equivalent of `componentDidMount`.

Q: Why does `demo1015` use two separate `useEffect` calls — one depending on `[newEmployeeObj]` and one on `[newAssetObj]` — instead of one effect covering both?

A: So that each effect only re-runs when the specific piece of state it actually cares about changes; a single combined effect (or one with an empty dependency array) would either re-run for changes it should ignore, or fail to re-run for changes it should react to.

Q: What two values does `useReducer(reducerFunc, initialState)` return, and in what order?

A: It returns an array of exactly two elements, conventionally destructured as `[state, dispatch]`: the current state value first, and a dispatch function used to request a state transition second.

Q: In `useReducer`, why does an event handler dispatch an action rather than compute and set the next state directly?

A: Centralising every state transition inside one reducer function keeps the *decision* of what the next state should be in a single, predictable, easily-tested place, rather than scattering that decision logic across every individual event handler that happens to trigger a change.

Q: Why must Hooks always be called at the top level of a component, never inside a loop or a condition?

A: React identifies which state or effect a given Hook call corresponds to purely by the

order it was called in during a render, not by name. Conditionally skipping a Hook call on one render but not another would shift every subsequent Hook's identity out of sync with the previous render, causing state to be attached to the wrong Hook call.

Q: How does `useRef` differ from `useState`, given that both persist a value across renders?

A: Updating a `useState` value triggers a re-render; changing `ref.current` on a value returned by `useRef` does not trigger a re-render at all. `useRef` is used for values a component needs to remember without that change, by itself, needing to be reflected in the UI — such as a direct handle onto a real DOM node.

Q: What problem does `useMemo` solve?

A: Since a function component's entire body re-runs on every render, any expensive calculation written directly in that body would otherwise be recomputed on every single render, even when the values it depends on have not changed. `useMemo` re-runs that calculation, and caches its result, only when one of its listed dependencies has actually changed.

Q: What is a custom Hook, and what does it actually share between the components that use it?

A: A custom Hook is an ordinary function, named starting with `use`, that itself calls other Hooks internally to package up a recurring combination of stateful logic. It shares *logic* between components, not *state* — every component calling the same custom Hook gets its own, independent copy of whatever state that Hook manages internally.

Chapter 10

Integration with a Spring Boot REST API — Axios, CRUD and Data-Fetching Patterns

10.1 Closing the Loop With the Back-End Track

Every component built so far in this volume has worked with data that was either hard-coded or typed directly by a user into the browser. This chapter closes the loop with this programme's Java back-end track: every demo below talks to a real, running Spring Boot `EmployeeController`, exposed exactly as the reference back-end training built it (GET `/emp/controller/getDetailsById/{id}`, POST `.../addEmp`, PUT `.../updateEmp`, DELETE `.../deleteEmp/{id}`, running on `localhost:8095`) — this is the front-end half of exactly the same client/server relationship Chapter 37 of the Java track's own reference material built with `RestTemplate`, only now consumed from a browser-based React application instead of another Spring Boot service.

10.2 REST Fundamentals — Resources, HTTP Verbs and Status Codes

Every call this chapter makes talks to a **REST** (Representational State Transfer) API — an architectural style, not a specific technology, in which data is organised into **resources** (here, an employee, identified by a URL such as `/emp/controller/getDetailsById/1009`), and a fixed, small set of standard HTTP verbs express what should happen to that resource, rather than the URL itself encoding the action:

Verb	Meaning
GET	Retrieve a resource (or a collection of resources) — must never itself change anything on the server.
POST	Create a new resource — <code>axios.post(url, this.state)</code> in this chapter.
PUT	Replace/update an existing resource in full.
DELETE	Remove a resource.

Every response also carries an HTTP **status code** — a three-digit number summarising the outcome, grouped by its first digit:

Range	Meaning
2xx	Success — 200 OK for a successful GET/ PUT, 201 Created for a successful POST.

Range	Meaning
4xx	The <i>client's</i> request was itself the problem — 400 Bad Request (malformed or failed-validation input), 401 Unauthorized (missing or invalid credentials), 404 Not Found (no resource exists at that URL).
5xx	The <i>server</i> failed to handle an otherwise valid request — 500 Internal Server Error .

`axios` uses this status code directly to decide which of its own two callbacks to invoke: any `2xx` response resolves the Promise, running `.then(...)`; any other status code (or a request that never reached the server at all, such as a network failure) rejects it, running `.catch (...)` instead — exactly the branching every component in this chapter relies on to update either its success state or its `errorMessage` state.

10.3 axios — A Promise-Based HTTP Client

Every demo in this chapter uses `axios`, a third-party HTTP client library, rather than the browser's native `fetch` API, because `axios` automatically parses a JSON response body and exposes a slightly more convenient, uniformly-shaped API across every HTTP verb (`axios.get(...)`, `axios.post(...)`, `axios.put(...)`, `axios.delete(...)`). Every one of these calls is **asynchronous** and returns a **Promise** — an object representing a value that is not available yet, but will be, at some point in the future, once the network request actually completes. `.then(response => {...})` registers the code to run *if* that Promise succeeds; `.catch(error => {...})` registers the code to run *if* it fails — and, critically, neither callback runs immediately: the component's `render()` (or function body) continues and finishes *before* the network response has actually arrived, which is precisely why every piece of data coming back from an `axios` call must land in **state**, so that its eventual arrival can trigger a fresh re-render rather than being lost.

10.4 GET — Fetching a List, From a Class Component

`demo1012-http-client's` `GetAllComponent` is the simplest complete example — a button-triggered fetch of every employee:

```

1 import axios from 'axios'
2
3 export class GetAllComponent extends Component {
4   constructor(props) {
5     super(props)
6     this.state = { list: [], errorMessage: "" }
7   }
8
9   fetchResult={()=>{
10     // Placing a request to the Spring Boot app for every employee's details
11     axios.get("http://localhost:8095/emp/controller/getDetails")
12       .then(
13         response=> {
14           this.setState({list: response.data})
15         }
16       ).catch(error=>{
17         this.setState({errorMessage:error.message})
18       });
19   }
20
21   render() {
22     return (
23       <div>
24         <button onClick={this.fetchResult}>Get Data</button>

```

```

25      {
26          this.state.list.length?
27              <table>
28                  <tr><th>Employee Id</th><th>Employee Name</th>
29                  <th>Salary</th><th>Department Code</th></tr>
30                  {this.state.list.map((emp,idx)=>
31                      // A dynamically-generated, non-editable list of
32                      // child elements MUST carry a unique "key" prop.
33                      <tr key={idx+100}>
34                          <td>{emp.employeeId}</td>
35                          <td>{emp.employeeName}</td>
36                          <td>{emp.salary}</td>
37                          <td>{emp.departmentCode}</td>
38                      </tr>
39                  )}
40              </table>
41              :<h3>{this.state.errorMessage}</h3>
42          }
43      </div>
44  )
45  }
46  }

```

Listing 10.1: GetAllComponent.js

`response.data` is where `axios` places the already-JSON-parsed response body — a plain JavaScript array of employee objects here, deserialised automatically from whatever the Spring Boot controller's own `ResponseEntity<Collection<Employee>>` serialised it as. `.map((emp,idx)=> <tr key={idx+100}>...)` is the same iteration idea Chapter 4's `<xsl:for-each>` demonstrated for XML, applied here to a JSON array instead — one `<tr>` rendered per element. The `key` prop deserves its own explanation: whenever React renders a list of elements generated programmatically (rather than written out by hand, one JSX tag per item), each sibling needs a stable, unique `key` so React can efficiently tell which items changed, were added, or were removed between renders, rather than re-rendering the entire list from scratch on every update.

10.5 GET by Id, POST, PUT and DELETE — The Full CRUD Surface

The remaining components in `demo1012-http-client` exercise every other HTTP verb the Java track's own `EmployeeController` exposes, each following the identical `axios → .then → setState / .catch → setState` shape:

```

1  // PostForm.js -- creating a new Employee
2  submitHandler = event=>{
3      event.preventDefault();
4      axios.post("http://localhost:8095/emp/controller/addEmp", this.state)
5      .then(
6          response=> { this.setState({message: response.data}); this.setState({errorMessage: ""}) }
7      ).catch(error=>{
8          this.setState({errorMessage: error.message}); this.setState({message: ""})
9      });
10 }
11
12 // DeleteEmpComponent.js -- deleting by id
13 delEmp=(event)=>{
14     event.preventDefault()
15     axios.delete("http://localhost:8095/emp/controller/deleteEmp/"+this.state.employeeId)
16     .then(
17         response=> {
18             this.setState({employee: response.data});
19             this.setState({successMsg: "Record Deleted successfully"})
20         }
21     ).catch(error=>{
22         this.setState((previous)=>({errorMessage: error.message + " : " + previous.errorMessage}))
23     });

```


24 }

Listing 10.2: PostForm.js — `axios.post`, and DeleteEmpComponent.js — `axios.delete`

Two details are worth calling out precisely because they generalise to every `axios` call in this chapter:

- `axios.post(url, this.state)` — the second argument to `axios.post/axios.put` is the request body, sent exactly as the current component state, and serialised by `axios` to JSON automatically — this is the client-side mirror of the Spring Boot side's own `@RequestBody` annotation, which deserialises that same JSON back into a Java object.
- **Both success *and* failure paths update state** — every handler resets whichever message field does *not* apply on this particular outcome (`this.setState({errorMessage:""})` on success, `this.setState({message:""})` on failure), so that JSX driven by a ternary against those two fields never displays a stale message left over from a previous, different outcome.

`UpdateComponent.js` chains two separate requests together in sequence — an `axios.get` (fired from the Employee Id field's own `onBlur`, to pre-populate the form once an id has been entered) followed, on the form's own submission, by an `axios.put` carrying the edited fields back to the server — the same `.then/.catch` shape used twice, once per independent network call.

10.6 GET, POST and Data Fetching From Function Components, With Hooks

`demo1016-http-client-with-hooks` rebuilds this same functionality as function components, and is worth reading directly against the class-based version above, since every structural piece maps onto a Hook from Chapter 9:

```

1 export function GetAllEmployeeDetails() {
2   const [newEmployeeList,setEmployeeListHandler] = useState(null);
3   const [newErrorMessage,setErrorMessageHandler] = useState("");
4
5   useEffect(()=>{
6     axios.get("http://localhost:8095/emp/controller/getDetails")
7       .then(
8         response=> { setEmployeeListHandler(response.data) }
9       ).catch(error=>{
10         setErrorMessageHandler(error.message)
11       });
12   },[]) // effect works only on Component Mounting
13
14   return (
15     <div>
16       <h4>{newEmployeeList?
17         <table>{/* ...one <tr> per employee, via .map(...), as before... */}</table>
18       :newErrorMessage}</h4>
19     </div>
20   )
21 }
```

Listing 10.3: GetAllEmployeeDetails.js

Where `GetAllComponent` (the class version) fetched data *on demand*, triggered by a button's `onClick`, this Hook-based version fetches once, automatically, the moment the component mounts — because the `axios` call now lives inside a `useEffect` with an empty dependency array (Chapter 9), which is precisely the Hook equivalent of firing a request from `componentDidMount`.

```

1 export function CreateEmployee() {
2   const [newEmployeeObjForInput,setEmployeeObjForInputHandler] = useState(initialEmployeeObjForInput)
3   ;
4   const [newEmployeeObjForRequest,setEmployeeObjForRequestHandler] = useState("DefVal");
```

```

4   const [newOutputData,setOutputDataRequestHandler] = useState(null);
5   const [newErrorMessage,setErrorMessageHandler] = useState("");
6
7   // The submit button copies the input state into a SECOND, dedicated state,
8   // purely to give useEffect something distinct to watch for.
9   function submitHandler(){
10      setEmployeeObjForRequestHandler(newEmployeeObjForInput)
11   }
12
13   useEffect(()=>{
14      if(newEmployeeObjForRequest!=="DefVal"){ // skip the axios call on mount
15         axios.post("http://localhost:8095/emp/controller/addEmp", newEmployeeObjForRequest)
16            .then(
17               response=> {
18                  setErrorMessageHandler("")
19                  setOutputDataRequestHandler(response.data)
20               }
21            ).catch(error=>{
22               setOutputDataRequestHandler(null)
23               setErrorMessageHandler(error.message)
24            });
25      }
26      },[newEmployeeObjForRequest]) // only re-runs when THIS state actually changes
27      // ...
28 }

```

Listing 10.4: CreateEmployee.js — a two-state pattern for a Hook-driven POST

The deliberate split between `newEmployeeObjForInput` (updated on every keystroke, driving the controlled form fields) and a *second*, `newEmployeeObjForRequest` state (updated only once, when the button is actually clicked) is the Hook-based way of expressing “run this network call only when the user explicitly submits, not on every keystroke”: since a `useEffect`’s dependency array is what controls when it re-runs (Chapter 9), the effect is deliberately made to depend on the *second* state rather than the first, and `submitHandler` exists purely to update that second state at exactly the right moment. The “DefVal” sentinel value, checked inside the effect, additionally guards against the `axios.post` firing accidentally on the component’s very first mount (when `useEffect` always runs once regardless), before the user has clicked anything at all.

10.7 useReducer for Fetch/Success/Error State

`demo10031-fetching-data-use-reducer` revisits the plain `GetAllEmployeeDetails` fetch once more, first with three separate `useState` calls, and then consolidated into the single-reducer pattern Chapter 9 introduced in the abstract:

```

1   const initialState={ loading:true, errorMessage:'', data:[] }
2
3   const reducerFunction = (state,action)=>{
4      switch(action.type){
5         case 'FETCH_SUCCESS':
6            return { loading:false, data:action.payload, errorMessage:'' }
7         case 'FETCH_ERROR':
8            return { loading:false, data:[], errorMessage:action.errorMessage }
9         default:
10            return state
11      }
12   }
13
14   const [newState,actionDispatcher] =useReducer(reducerFunction,initialState);
15
16   useEffect(()=>{
17      axios.get("http://localhost:8095/emp/controller/getDetails")
18         .then(
19            response=> {
20               actionDispatcher({ type:'FETCH_SUCCESS', payload:response.data })
21            }

```

```

22         ).catch(error=>{
23             actionDispatcher({ type:'FETCH_ERROR', errorMessage:error.message })
24         })
25     },[])
26
27     return (
28         <div>
29             {newState.loading? 'Loading Data...':
30                 newState.data.length
31                 ?newState.data.map((emp,idx)=><div key={idx+100}>{emp.employeeName}</div>)
32                 : "No Data found"
33             }
34             <h4>{newState.errorMessage}</h4>
35         </div>
36     )

```

Listing 10.5: DataFetchingTwo.js — one reducer covering loading, success and error

Compared against `DataFetchingOne.js`'s three independent `useState` calls (`loading`, `errorMessage`, `data`), this reducer-based version guarantees the three related pieces of state can never drift out of sync with one another — every transition (`FETCH_SUCCESS`, `FETCH_ERROR`) updates `loading`, `data` and `errorMessage` together, as one atomic object, in exactly one place, rather than relying on three separate `setLoading/setData/ setErrorMessage` calls each being remembered and called correctly, every single time, at every call site that fetches data.

This is precisely the same shape the Java back-end track's own Resilience4j circuit-breaker demo reaches for on the server side: a fixed, small set of named outcomes (there, `CLOSED/OPEN/HALF_OPEN`; here, `loading/success/error`), each transition handled in exactly one place, rather than scattered flags that have to be kept consistent by hand at every call site.

10.8 CORS — Why a Browser Can Block a Perfectly Valid Request

Every `axios` call in this chapter runs from a React development server on one origin (typically `http://localhost:3000`) against a Spring Boot API on a different origin (`http://localhost:8095`) — different port numbers count as different **origins**, exactly as a different domain would. Browsers enforce the **Same-Origin Policy** by default: JavaScript running on one origin is blocked from reading the response of a request made to a different origin, purely as a security measure, regardless of whether the request itself would otherwise have succeeded. **CORS** (Cross-Origin Resource Sharing) is the mechanism that relaxes this restriction deliberately: the *server* being called includes a response header (`Access-Control-Allow-Origin`) explicitly naming which origins are permitted to read its responses, and the browser only allows the calling JavaScript to see the response if the calling origin is one of those permitted. A Spring Boot controller enables this, most simply, with an `@CrossOrigin` annotation (or an application-wide CORS configuration) naming the React development server's own origin — without it, every `axios` call in this chapter would fail in the browser's console with a CORS error, even though the exact same request sent directly (from a tool that is not itself a browser enforcing this policy) would succeed perfectly well. This is a browser-enforced restriction on *reading a cross-origin response*, not a general restriction on servers talking to each other — it is precisely why this same concern never arose anywhere in this programme's back-end track, where one Spring Boot service calling another (via `RestTemplate` or `Feign`) is not subject to a browser's Same-Origin Policy at all.

Viva Voce — Likely Questions & Answers

Q: Why does every axios call in this chapter store its result in state, rather than simply using the response directly where the call was made?

A: Every axios call is asynchronous and returns a Promise, so the surrounding function's own execution continues and finishes before the network response has actually arrived. Only a state update can correctly trigger a re-render once that response eventually does arrive; a value that had been used directly at the call site, without going through state, would already be stale or simply unavailable by the time execution reaches it.

Q: What does response.data contain after a successful axios.get call?

A: The response body, already parsed from JSON into a plain JavaScript value (an object or an array, depending on what the server returned) — axios performs this JSON parsing automatically, unlike the browser's native fetch API, which requires an explicit extra .json() step.

Q: Why must every element in a list rendered via .map(...) carry a unique key prop?

A: So React can efficiently match each rendered element against its previous render, correctly identifying which items were added, removed, or reordered, rather than being forced to re-render the entire list from scratch on every update.

Q: In CreateEmployee.js, why does the component maintain two separate pieces of state – one for the form's input, and a second purely to trigger the axios.post call?

A: Because the useEffect that performs the network call is scoped, via its dependency array, to react only to changes in the second state. Keeping the form's live input in a separate first state means every keystroke does not, by itself, trigger a network call; only the deliberate act of updating the second state (from the submit button's handler) does.

Q: What does the "DefVal" sentinel value guard against in CreateEmployee.js's effect?

A: It prevents the axios.post call from firing accidentally on the component's very first mount, since a useEffect with a dependency array still always runs once immediately after mounting regardless of whether its dependency has genuinely changed yet.

Q: What advantage does consolidating loading, data and errorMessage into one useReducer state have over three independent useState calls?

A: It guarantees the three related pieces of state are always updated together, as one atomic transition, in exactly one place (the reducer function) — removing the risk that a call site correctly updates data but forgets to also reset loading or errorMessage, which three independent setState calls would otherwise leave possible.

Q: What is the client-side equivalent of the Spring Boot back end's @RequestBody annotation, in these demos?

A: Passing the data to be sent as the second argument to axios.post/ axios.put (e.g. axios.post(url, this.state)). axios serialises that JavaScript object to a JSON request body automatically, which @RequestBody on the Spring Boot side then deserialises back into a Java object.

Q: What is the difference between the four core HTTP verbs used throughout this chapter?

A: GET retrieves a resource without changing anything; POST creates a new resource; PUT replaces or updates an existing resource in full; DELETE removes a resource.

Q: How does axios decide whether to run `.then(...)` or `.catch(...)` for a given call?

A: It uses the HTTP status code of the response: any 2xx status resolves the Promise, running `.then(...)`; any other status code, or a request that never reached the server at all, rejects it, running `.catch(...)` instead.

Q: What is CORS, and why does an axios call from a React app on localhost:3000 to a Spring Boot API on localhost:8095 need it?

A: Different port numbers count as different origins, and a browser's Same-Origin Policy by default blocks JavaScript from reading a response from a different origin. CORS lets the *server* explicitly permit specific origins to read its responses via an `Access-Control-Allow-Origin` header; without it configured on the Spring Boot side, the browser blocks the response even though the request itself succeeds.

Q: Is CORS a restriction on the server sending a request, or on the browser reading a response?

A: It is specifically a browser-enforced restriction on *reading a cross-origin response*. It has no bearing on server-to-server calls (such as one Spring Boot service calling another via RestTemplate or Feign), since those are not subject to a browser's Same-Origin Policy at all.

Chapter 11

Practice Activities & Assignments

11.1 What This Chapter Covers

Following the same convention as the Java back-end reference's own Chapter 42, this chapter walks through the hands-on *See It / Try It* exercises supplied alongside this volume's instructor-led demos — the front-end track's own POA (Point of Activity) checklists, built directly into the `HTML_CSS_JS_Demos` and `XML&XSLT_Demos` projects as `TODO` comments, plus a consolidated capstone assignment that carries the Order domain from the back-end track's own Chapter 42 assignment into a React front end, closing the loop this volume's Chapter 10 opened.

11.2 Activity 1 — HTML: The Employee Registration Form

`HTMLTryIt.html` hands over a fully-linked page shell (already wired to `CssTryIt.css` and `TryIt.js`) and leaves the actual form as a single, open-ended `TODO`:

```
1 <body>
2   <center>
3     <h1>Employee Registration System</h1>
4     <hr>
5
6     <!-- TODO - Create a registration form,
7          which will accept the employee id, name and department id of an employee -->
8
9   </center>
10 </body>
```

Listing 11.1: `HTMLTryIt.html` — the activity as supplied

POA 1: Build a `<form>` containing three labelled `<input>` fields — `id="eid"` for the employee id, `id="ename"` for the employee name, and a third field for the department id — laid out in a `<table>`, exactly following the structural pattern already demonstrated in `HTMLDemo.html` (Chapter 1). **POA 2:** Add a submit control whose `onclick` invokes `validateForm()` (rather than submitting immediately), so that Activity 3's JavaScript actually has a chance to run before any submission takes effect.

11.3 Activity 2 — CSS: Styling the Registration Page

`CssTryIt.css` is supplied as a comment-only checklist, deliberately empty of any actual rules:

```
1 /*
2   Topic : CSS
3
4   Instructions:
5
6   1. Set the background color, text color, margin-left, margin-right,
7      border and font-family for body
8   2. Set the font-style and color for heading tag
9   3. Set the color, background-color and font-style for the button
10  */
```

Listing 11.2: `CssTryIt.css` — the activity as supplied

POA 3: Write a body rule covering every property named in instruction 1, directly modelled on `CssDemo.css`'s own body rule from Chapter 2. **POA 4:** Write an `h1` rule for instruction 2. **POA 5:** Write an ID selector rule for the submit button added in Activity 1, covering `color`, `background-color` and `font-style`, exactly mirroring `CssDemo.css`'s own `#mybtn` rule.

11.4 Activity 3 — JavaScript: Validating the Form

`TryIt.js` supplies the function signature and the DOM lookups, leaving only the actual validation logic as the exercise (already read in full in Chapter 3):

```
1 function validateForm() {
2     var eid = document.getElementById("eid").value;
3     var ename = document.getElementById("ename").value;
4
5     /* Instructions: Ensure that the employee id and name are not empty.
6        Hint: Use the length property of the variable */
7 }
```

Listing 11.3: `TryIt.js` — the activity as supplied

POA 6: Complete `validateForm()` so that it checks `eid.length===0` and `ename.length===0`, alerts a clear message naming whichever field is missing, and returns `false` to block submission in that case (or otherwise allows the form to proceed) — exactly the shape Chapter 6's `React validate()` method later re-implements against component state instead of a live DOM lookup.

11.5 Activity 4 — XML: Correcting Malformed Documents

Chapter 4's `ZooDemo.xml` and `EventsTryIt.xml` are themselves the activity: each is supplied deliberately malformed, and the exercise is to locate and correct every violation of the well-formedness rules from Section 4.2.

POA 7: Rewrite `ZooDemo.xml` so that every `<animal>` block properly nests and closes `<class>` and `<quantity>`, and so that `<cla ss>` (an illegal tag name containing a space) is corrected to `<class>`. **POA 8:** Rewrite `EventsTryIt.xml` so that the document has exactly one root element (removing or properly nesting the stray `<newcodington>` element), every `<duration>` value is closed, every attribute value is quoted, and `</SCHEDULE>` is corrected to match its own opening tag's case, `</schedule>`. **POA 9:** Once corrected, write an XSLT stylesheet for the events document, directly modelled on `ZooTransform.xsl`, that renders one HTML table row per `<event>`, showing its `eventname` attribute and its `description` child element.

11.6 Capstone Assignment — A React Front End for the Order Domain

The back-end track's own Chapter 42 hands a trainee a complete, working `Order`-domain REST API (`OrderBean`, `OrderService`, and a `POST /order/controller/addOrder / GET /order/controller/getOrdersInBillingRange/{min}--{max} / PUT /order/controller/updateOrderStatus / DELETE /order/controller/deleteOrder/{id}` controller), built specifically so a fresh domain — distinct from the `Employee` domain every demo in this volume has used — is available to practise against. This capstone asks a trainee to build the React front end for that exact API, applying every technique from Chapters 5 through 10 to a domain they have not yet seen wired up in React:

- **POA 10:** A `CreateOrder` component, modelled directly on Chapter 10's `CreateEmployee`, using a controlled form (Chapter 6) to collect an order's fields and an `axios.post` to `/order/controller/addOrder` to submit it.

- **POA 11:** A `GetOrdersInBillingRange` component, modelled on `GetAllEmployeeDetails`, that accepts a minimum and maximum bill amount through two controlled `<input>`s and renders the matching list of orders in a table, one row per order via `.map(...)`, each row carrying a unique `key`.
- **POA 12:** An `UpdateOrderStatus` component, modelled on `UpdateComponent`, that fetches an order by id on blur, pre-populates a form with its current status, validates that a status was actually selected before allowing submission (Chapter 6), and issues the `axios.put` on submit.
- **POA 13:** Wrap at least the `UpdateOrderStatus` component in a `CustomErrorBoundary` (Chapter 7), and deliberately trigger its fallback UI by supplying an invalid order id, to confirm the boundary correctly contains the failure without taking down the sibling components on the page.
- **POA 14:** Re-implement at least one of the components above twice — once as a class component using `this.state`, and once as a function component using `useState/useEffect` — specifically to compare the two approaches directly, the same side-by-side comparison this volume's own Chapter 10 draws between `demo1012` and `demo1016`.

Closing Note

This volume has carried a trainee from the plain building blocks a browser actually renders — HTML's structure, CSS's presentation, JavaScript's DOM and events, and XML's own strict, structured alternative — through React's own way of packaging structure and behaviour together into small, reusable components: first as class components with their own state and a well-defined lifecycle, then guarded against failure with Error Boundaries and freed from prop drilling with the Context API, and finally rebuilt on modern function components and Hooks. It closed exactly where the back-end track's own reference material closes its distributed-systems arc — talking, for real, to a running Spring Boot REST service — only from the other side of the wire: a browser, not another microservice, issuing the request. Read together with the back-end track's own volumes, this collection now spans the full arc of the training programme from a single static HTML page through to a React front end backed by a securely deployed, independently scalable Spring Boot microservice architecture — worth revisiting as a whole precisely because the same handful of underlying habits — a single source of truth for a piece of data, a clean separation between what a layer contains and how it is presented, a validated request handled the same way whether it originates from a browser or another service — recur at every layer of that architecture, front end included.

Java Full Stack Training

Cloud Computing Essentials

Cloud Compute Essentials & Cloud Storage / Networking Basics

Extended Theory, Concepts & Hands-On AWS Activities

(Viva / Interview Preparation Edition)

Accenture QuantumCrew Batch — Day-wise Training Reference
(Continuation Volume IV)

Covers Week 7 of the Day-wise Training Plan (Units 33–34)
Chapters 43–44 — AWS Theory Integrated with Hands-On Activities

Chapter 43

Cloud Compute Essentials

43.1 Introduction to Cloud Computing

Every application built across the previous three volumes of this course was, implicitly, assumed to run *somewhere* — on a server, with a CPU, memory, disk, and a network connection. **Cloud computing** is the delivery of exactly that kind of computing infrastructure — servers, storage, databases, networking, and increasingly, entire managed platforms — as an on-demand service over the internet, rather than infrastructure an organisation buys, owns, and physically maintains itself.

The U.S. National Institute of Standards and Technology (NIST) definition of cloud computing, still the most widely cited, names five essential characteristics that distinguish genuine cloud computing from merely renting a server:

Characteristic	What It Means
On-demand self-service	A consumer can provision computing resources (an EC2 instance, a database) automatically, without requiring human interaction with the provider — exactly what happens when the AWS Console’s Launch Instance button provisions a running server in minutes
Broad network access	Resources are available over the network and accessed through standard mechanisms, usable from laptops, mobile devices, and other services alike
Resource pooling	The provider’s physical resources serve multiple customers using a multi-tenant model, with resources dynamically assigned and reassigned according to demand — the customer generally has no control over, or knowledge of, the exact physical location of the resources they are using
Rapid elasticity	Capacity can scale outward and inward quickly, sometimes automatically, to match demand — to the consumer, the capacity available often appears unlimited
Measured service	Resource usage is monitored, controlled, and reported, providing transparency for both the provider and the consumer — this is what underlies AWS’s pay-as-you-go billing, down to the second of EC2 usage or the gigabyte of S3 storage

Why This Matters Before Touching the AWS Console

Every activity in this chapter — launching an EC2 instance, creating an ECS cluster, deploying a Lambda function — is a concrete instance of these five characteristics. Recognising *which* characteristic a given AWS feature is expressing (elasticity, measured service, self-service) is what turns “clicking through a console wizard” into an actual understanding of why cloud computing is architected the way it is.

43.1.1 The Traditional Alternative — On-Premises Infrastructure

Before cloud computing, an organisation needing a new application server had to forecast demand months in advance, purchase physical hardware, wait for it to be delivered and racked in a data centre, install an operating system, and only then begin actually running the application — a process that could take weeks. If the forecast was wrong, the organisation was stuck either with idle, wasted capacity (over-provisioned) or an outage-prone, overloaded system (under-provisioned), and either mistake was expensive and slow to correct, since buying more hardware or returning purchased hardware both take real time. Cloud computing collapses this entire provisioning cycle down to the minutes it took to launch the EC2 instance in this chapter’s own activity — the underlying economic shift, from a large upfront **capital expenditure** (buying hardware) to an ongoing, usage-based **operating expenditure** (paying only for what is actually used, as billed by AWS), is one of cloud computing’s most consequential practical effects on how organisations budget for infrastructure at all.

43.2 Cloud Compute Resources — Virtual Machines, Containers, and Serverless

Cloud providers offer compute capacity at several different levels of abstraction, and this chapter’s five hands-on activities — EC2, ECS, EKS, Lambda, and the DynamoDB/Lambda integration — are, not coincidentally, arranged to walk through exactly this spectrum, from the least abstracted (a virtual machine you fully manage) to the most abstracted (a function you simply upload).

Model	What You Manage	What the Provider Manages
Virtual Machine (EC2)	The guest operating system, all installed software, patching, scaling decisions	The physical host, hypervisor, and the underlying virtualisation layer
Container (ECS/EKS)	Your application and its dependencies, packaged into an image; how many container instances run	The underlying host OS and (with Fargate, covered below) the entire server fleet the containers run on
Serverless / Function-as-a-Service (Lambda)	Only your application code itself, packaged as a function	Absolutely everything else — the server, the OS, the runtime, scaling, and patching

43.2.1 Virtual Machines

A **virtual machine (VM)** is a software-based emulation of a physical computer, created by a **hypervisor** that partitions one physical server's CPU, memory, and storage among several independent, isolated virtual machines, each running its own complete operating system as if it had the entire physical machine to itself. This is the oldest and most flexible cloud compute model — Amazon EC2, covered in depth later in this chapter, is AWS's virtual machine offering — and it gives the most control (you choose the OS, install anything you like, configure networking directly) at the cost of also taking on the most operational responsibility (patching the OS, managing security updates, deciding how to scale).

43.2.2 Containers

A **container** packages an application together with everything it needs to run — its code, runtime, system libraries, and configuration — into a single, portable unit, but, unlike a virtual machine, a container does *not* bundle its own full operating system. Instead, multiple containers on the same host share that host's OS kernel, each isolated from the others (in terms of filesystem, process space, and network) by OS-level features rather than full hardware virtualisation. This makes containers dramatically lighter-weight than VMs — they start in seconds rather than minutes, and many more containers can run on the same physical hardware than equivalent VMs could — at the cost of a slightly less complete isolation boundary than a true VM provides (all containers on one host ultimately share that host's kernel). **Docker**, covered in its own section later in this chapter, is the tool this course uses to build and run containers; **Amazon ECS** and **Amazon EKS**, both covered in dedicated activities in this chapter, are AWS's two services for running many containers reliably, at scale, across a fleet of underlying hosts.

43.2.3 Serverless Computing

Serverless computing removes server management from the picture entirely — not because there is literally no server involved (there always is, somewhere), but because the cloud provider handles provisioning, scaling, and patching that server completely invisibly, and the consumer's only remaining responsibility is the application code itself. **AWS Lambda**, covered later in this chapter, is AWS's premier serverless compute service, specifically in the **Function-as-a-Service (FaaS)** style: you upload a function, AWS runs it in response to triggering events, and you are billed only for the actual compute time your function consumes while running — there is no charge at all for idle capacity, because there is no persistently-running server to be idle in the first place.

The Trade-off Across the Spectrum

Moving from VMs toward serverless trades *control* for *convenience*: a VM gives you the most control over the exact environment your application runs in, at the cost of managing that entire environment yourself; a serverless function gives you almost no control over the environment (you cannot even choose exactly which underlying server your code runs on) but removes essentially all of the operational burden of managing it. Neither end of the spectrum is universally “better” — the right choice depends on how much control a given workload genuinely needs versus how much operational overhead the team is willing to take on.

43.3 Cloud Deployment Models — Public, Private, Hybrid, and Multi-Cloud

Alongside the compute-resource spectrum above, cloud computing also varies along a second, independent dimension: *who owns and operates the underlying infrastructure*, and *who else shares it*.

Deployment Model	Description
Public Cloud	Infrastructure owned and operated by a third-party cloud provider (AWS, Azure, Google Cloud) and made available to the general public over the internet, with physical resources shared (multi-tenant) across many different customers, each logically isolated from the others. Every activity in this chapter and the next runs on AWS's public cloud.
Private Cloud	Infrastructure provisioned for exclusive use by a single organisation, whether that infrastructure is physically on the organisation's own premises or hosted by a third party on the organisation's behalf — chosen when an organisation needs the elasticity and self-service benefits of cloud computing but cannot, typically for regulatory or data-sovereignty reasons, share physical infrastructure with other tenants.
Hybrid Cloud	A composition of two or more distinct cloud infrastructures (private and public) that remain separate entities but are bound together by standardised technology enabling data and application portability between them — for example, keeping sensitive customer data in a private cloud while running less sensitive, bursty workloads on public cloud capacity.
Multi-Cloud	The deliberate use of cloud services from <i>more than one</i> public cloud provider (e.g. some workloads on AWS, others on Azure) — distinct from hybrid cloud, which specifically mixes private and public infrastructure; multi-cloud mixes multiple <i>public</i> providers, typically to avoid over-dependence on a single vendor, to access a specific provider's unique service, or to satisfy a regulatory requirement to distribute risk across providers.

Why the Distinction Matters in Practice

“Which model should we use” is rarely an all-or-nothing choice for a real organisation — a large enterprise very often runs a hybrid arrangement (some workloads kept on-premises or in a private cloud for regulatory reasons, others in the public cloud for elasticity) and, increasingly, a genuinely multi-cloud one (chosen deliberately, to avoid a single point of vendor dependency, or accidentally, as different teams within the same organisation independently pick different providers over time).

43.4 AWS Account Fundamentals — the Root User, IAM, and the Free Tier

Before any activity in this chapter can begin, an AWS account itself has to exist. Understanding what actually happens during account creation matters, because it introduces AWS's most fundamental security concept: the distinction between the **root user** and every other kind of user that will ever access the account.

The **root user** is the single identity created automatically the moment an AWS account is signed up for, tied directly to the email address used at sign-up, and it has *unrestricted* access to every resource and every setting in the entire account, with no possibility of restriction. This is precisely why AWS's own security best practices — and the **IAM (Identity and Access Management)** activities covered later in this volume — exist: the root user should be used as rarely as possible (essentially only for the small handful of tasks that genuinely require it, like closing the account or changing the account's support plan), with day-to-day work instead performed by an **IAM user**, a separate identity created *within* the account that can be granted precisely the permissions it needs, and no more.

43.4.1 Activity: Creating an AWS Account

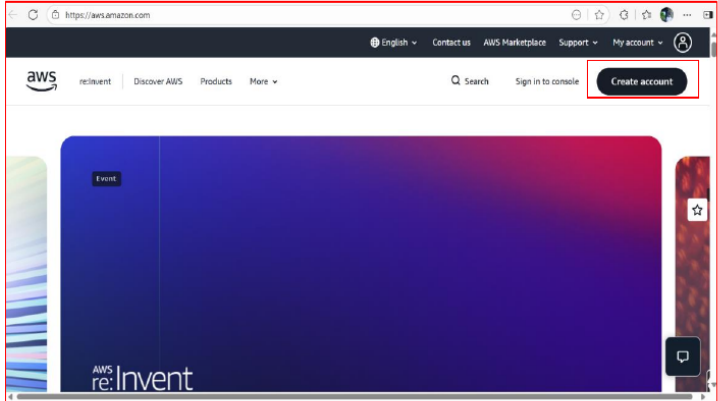
Activity: Sign Up for an AWS Account

1. Visit the AWS home page and click **Create Account**.
2. Enter the **root user email address** and an **AWS account name** (a friendly label for the account, changeable later in account settings), then verify the email address.
3. **Choose an account plan**: the **Free** plan (six months, receiving up to \$200 in credits, with free usage of select services, but without access to every AWS service/feature) versus the **Paid** plan (production-ready, pay-as-you-go pricing from the start, full access to all services). Every activity in this chapter and the next is designed to run comfortably within Free-tier eligible resources.
4. **Create a root user password** (Step 1 of 5).
5. **Fill in contact information** (Step 2 of 5) — whether the account is for Business or Personal use, full name, phone number, and address.
6. **Provide billing information** (Step 3 of 5) — a credit or debit card is required even for the Free plan; AWS makes (and promptly refunds) a small token charge purely to verify the card is valid, not to actually charge for usage yet.
7. **Confirm identity** (Step 4 of 5) — verifying a government-issued ID (PAN card, driving licence, or voter ID, front and back) and confirming a phone number via an SMS or voice-call verification code.
8. **Complete a support plan selection** (Step 5 of 5, not shown in the numbered steps above but the final stage of sign-up) to finish account creation.

Console Walkthrough — Screenshots

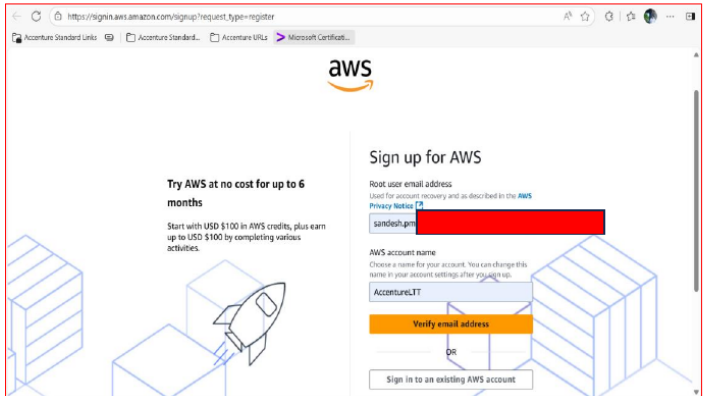
1. Creating AWS account

- 1 Sign up for AWS Account Using the Link below
[Cloud Computing Services - Amazon Web Services \(AWS\)](#)
Click on Create Account



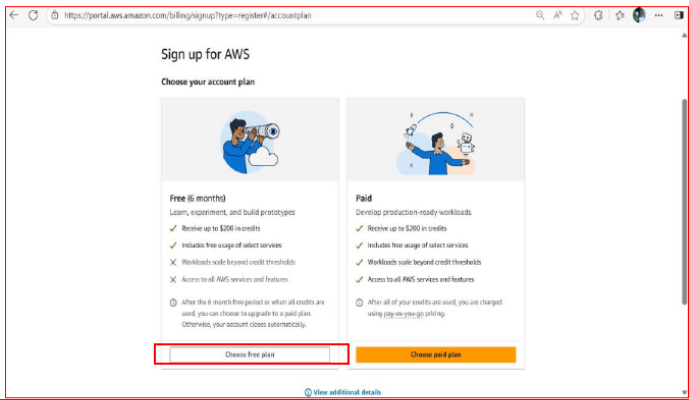
The screenshot shows the AWS homepage (https://aws.amazon.com). The 'Create account' button is highlighted with a red box in the top right corner. The page features the AWS logo, navigation links, and a large banner for 'AWS re:Invent'.

- 2 Enter Email ID and AWS Account Name



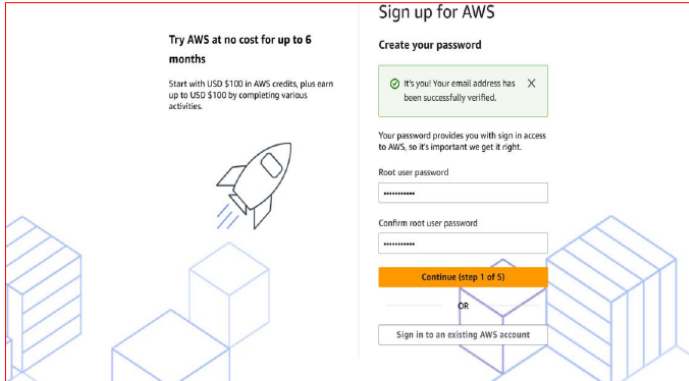
The screenshot shows the AWS sign-up page (https://signin.aws.amazon.com/signup/request?step=register). The 'Root user email address' field is filled with 'sandesh.g...' and the 'AWS account name' field is filled with 'AccentureLIT'. The 'Verify email address' button is highlighted with a red box. The page includes a promotional message about trying AWS at no cost for up to 6 months.

3 Choose free plan



The screenshot shows the 'Sign up for AWS' page with the 'Choose your account plan' section. It compares two plans: 'Free (6 months)' and 'Paid'. The 'Free' plan includes up to \$200 in credits, free usage of select services, and access to all AWS services and features. The 'Paid' plan includes up to \$200 in credits, free usage of select services, and access to all AWS services and features. The 'Free' plan is highlighted with a red box, and the 'Choose free plan' button is also highlighted.

4 Fill the details as it involves 5 steps for Sign up for AWS
Step 1: Enter Password



The screenshot shows the 'Sign up for AWS' page with the 'Create your password' section. It includes a message: 'It's you! Your email address has been successfully verified.' Below this, there are fields for 'Root user password' and 'Confirm root user password'. A 'Continue (step 1 of 5)' button is highlighted in orange. There is also a 'Sign in to an existing AWS account' link.

Step 2: Fill the Contact Information

Earn additional AWS credits

Complete various activities to earn up to an additional USD 100 in credits, such as creating your first AWS budget to monitor costs.



Sign up for AWS

How do you plan to use AWS?

☐ Business: For your work, school, or organization

☒ Personal: For your own projects

Who should we contact about this account?

Full Name

Country Code Phone Number

Country or Region

Address line 1

Address line 2

City

State, Province, or Region

Postal Code

Nearest AWS Region selection - optional

Enabling regional availability for your account resources to provide the lowest latency.

☒ Enable Asia Pacific (Hyderabad) Region

Customers with an Indian contact address are served by Amazon Web Services India Private Limited, the local seller for AWS services in India.

☒ I have read and agree to the terms of the [AWS Customer Agreement](#)

[Sign up and continue \(page 2 of 5\)](#)

Step 3: Provide the Card Details where INR 2 Rupees will be deducted from the Credit Card and Credit Back.

Why this request?

Use an Indian payment method (credit or debit card) to pay for your AWS account. You must provide your card details to activate your account and access the AWS console.

For the fastest activation, please provide your card details and the appropriate billing plan.

Billing Information

Billing method

Cardholder name

Card number

Expiration date

Security code

Cardholder name

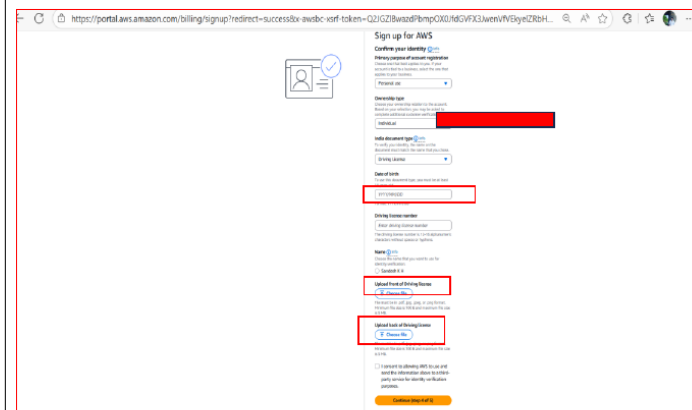
☒ I have read and agree to the terms of the [AWS Customer Agreement](#)

[Sign up and continue \(page 3 of 5\)](#)

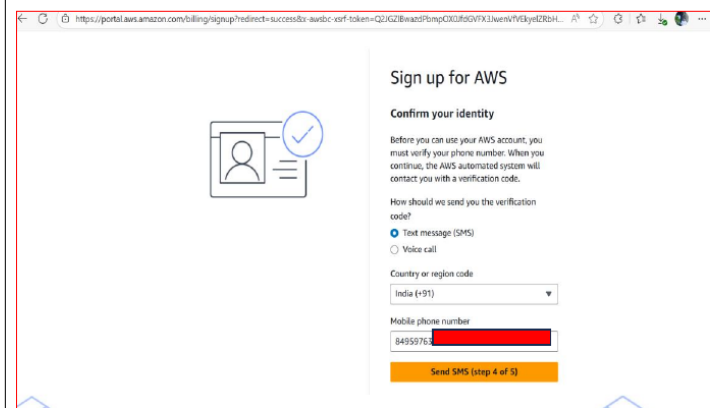
Step 4: Fill the details for Identity, you can Upload EPIC Copy of

1. Pan Card
2. Driving License
3. Voter ID

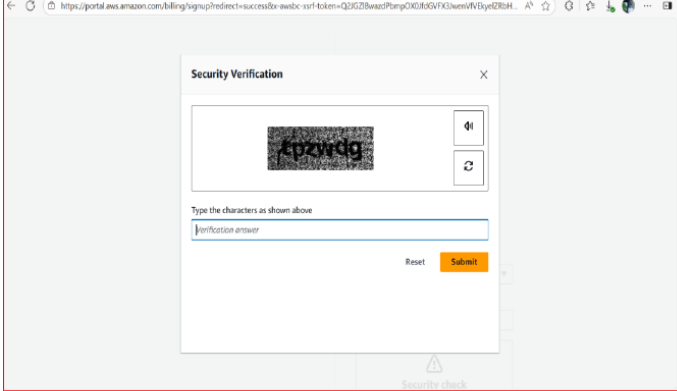
Upload Respective proof of Front Side and Back Side



Step 4: Confirm your Identity by providing your number and Verification Code Sent to Register number



5 Enter Verification



Security Verification

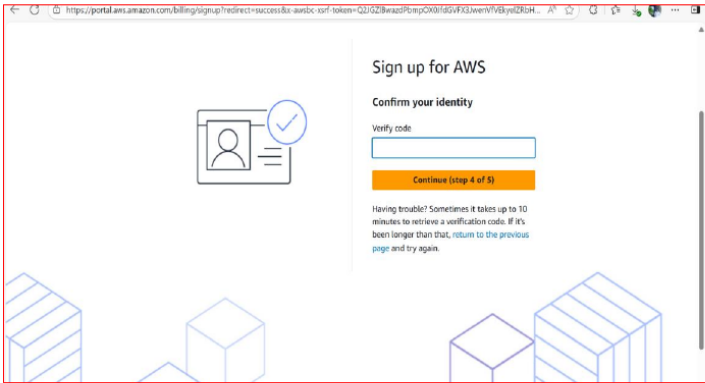
Type the characters as shown above

Verification answer

Reset Submit

Security check

Enter the text or voice verification code that you have received



Sign up for AWS

Confirm your identity

Verify code

Continue (step 4 of 5)

Having trouble? Sometimes it takes up to 10 minutes to retrieve a verification code. If it's been longer than that, [return to the previous page](#) and try again.

Every subsequent activity in this chapter and the next assumes this account already exists and that a specific **AWS Region** (the geographic area where AWS resources are actually provisioned — “Asia Pacific (Mumbai)”, “Europe (Stockholm)”, and so on across this course’s own activities) has been selected, since almost every resource created in AWS — an EC2 instance, a VPC, an RDS database — is scoped to exactly one Region unless explicitly configured otherwise.

43.5 Introduction to Amazon EC2

Amazon Elastic Compute Cloud (EC2) is AWS’s core virtual-machine service, and it is worth learning its vocabulary carefully, because every later activity in this chapter — ECS, EKS, and even, indirectly, Lambda — either runs on top of EC2 instances or was designed as an evolution away from managing them directly.

EC2 Concept	Meaning
Instance	One running virtual machine — the fundamental unit of EC2
AMI (Amazon Machine Image)	A template containing the operating system (and, optionally, pre-installed software) that a new instance is launched from — this chapter’s own activity uses an Ubuntu Server 24.04 LTS AMI
Instance Type	The hardware configuration (vCPU count, memory, network performance) an instance runs with — this chapter’s activity uses t3.micro , part of the <i>burstable</i> instance family, which is Free-tier eligible and appropriate for light, intermittent workloads rather than sustained heavy compute
Key Pair	An asymmetric cryptographic key pair used to securely log in to an instance rather than a traditional password — AWS retains the public key on the instance and gives the private key (a downloadable .pem file, for the Linux-style key pair used with Ubuntu in this chapter) to the user, who must keep it safe, since AWS itself does not retain a copy
Security Group	A virtual firewall controlling inbound and outbound traffic <i>at the instance level</i> — rules are defined by protocol, port range, and source (an IP range, or another security group)
Elastic IP	A static, persistent public IPv4 address that can be associated with an instance, so the instance’s public address does not change across stops/starts the way its default auto-assigned public IP otherwise would

43.5.1 Why a Key Pair Instead of a Password

SSH key-pair authentication is preferred over password authentication for a concrete security reason: a password can be guessed or brute-forced given enough attempts, whereas the private half of an asymmetric key pair is effectively infeasible to derive from the public half alone with any realistic amount of computing power, and— because the private key never has to be transmitted over the network to authenticate, unlike a password, which (even if only sent once, at login) still represents a secret that has to cross the wire — there is no equivalent of a password being intercepted in transit for an attacker to capture in the first place.

43.5.2 Activity: Create an EC2 Instance on AWS

Activity: Launch an EC2 Instance

1. Sign in to the AWS Management Console as the root user (or, in a real deployment, as an IAM user with EC2 permissions) and open the **EC2** service from the search bar.
2. From the EC2 Dashboard, click **Launch instance**.
3. Under **Name and tags**, give the instance a name (e.g. `MyFirstEC2`).
4. Under **Application and OS Images (AMI)**, choose **Ubuntu** (Free-tier eligible).
5. Under **Instance type**, choose **t3.micro** (Free-tier eligible).
6. Under **Key pair (login)**, click **Create new key pair**: since Ubuntu is a Linux AMI, choose key pair type **RSA** and private key file format **.pem**. The **.pem** file downloads automatically once the key pair is created — this is the *only* copy of the private key; AWS does not retain it.
7. Under **Network settings**, either create a new security group or select an existing one. A newly-created security group by default allows SSH traffic (port 22) from a chosen source; additional inbound rules (Custom TCP on a specific application port, or HTTP on port 80) can be added here or edited afterward.
8. Under **Configure storage**, accept the default root volume (e.g. 8 GiB, **gp3** general-purpose SSD).
9. Click **Launch instance**. AWS provisions the instance, which transitions from **Pending** to **Running** within roughly a minute.
10. If additional inbound rules are needed after launch, navigate to **EC2** → **Instances** → (select instance) → **Security tab** → **Security groups** → (the group's ID) → **Inbound rules** → **Edit inbound rules** → **Add rule**, specifying the rule's **Type** (e.g. Custom TCP, SSH, HTTP), **Port range**, and **Source**, then **Save rules**.
11. Click **Connect** on the running instance to open a browser-based SSH session (EC2 Instance Connect) directly against the instance's public IPv4 address, using the default username for the chosen AMI (`ubuntu`, for an Ubuntu AMI).
12. Once connected, the instance is ready for any further software installation via the command line — this is precisely the foundation on which a Jenkins server, a Kubernetes node, or any other self-managed workload would subsequently be installed.
13. **Cost management**: to avoid ongoing charges, stop the instance (**Instance state** → **Stop instance**) when not actively in use, or terminate it entirely if it will not be needed again — a stopped instance still incurs storage charges for its attached EBS volume, while a terminated instance does not.

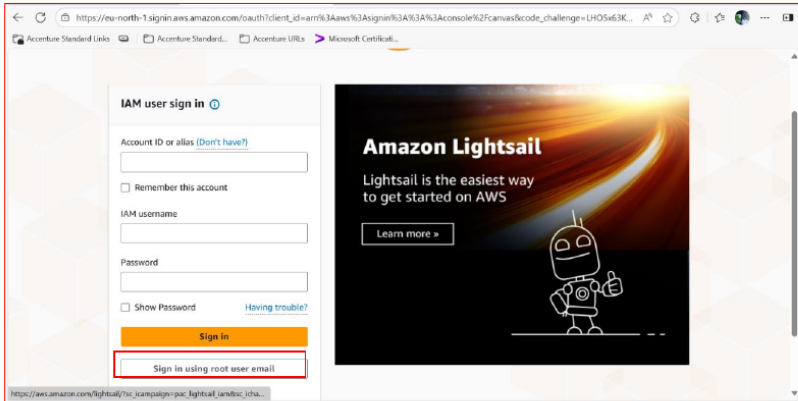
Console Walkthrough — Screenshots

2. Creating EC2 Instance on AWS

1

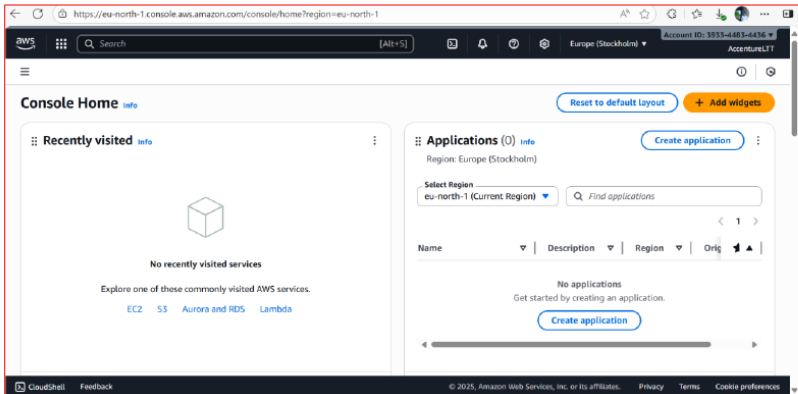
Sign in to AWS Management Console
<https://aws.amazon.com/console/>

1. Click “Sign in to the Console.”
2. Choose:
 - o **Root user**-use your email and password



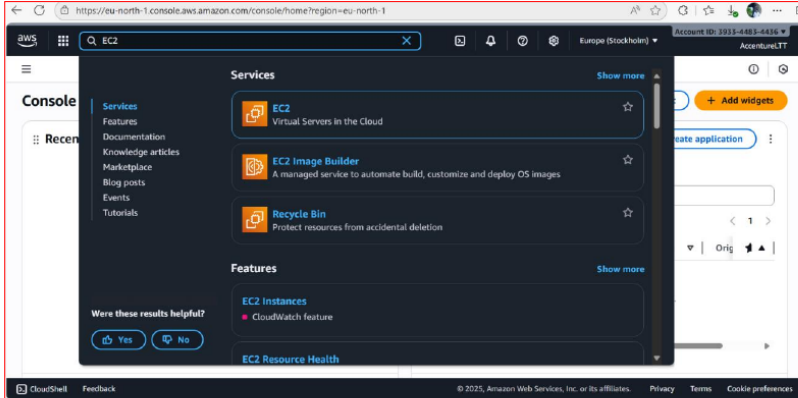
2

AWS Management Console Dashboard.



3 Open EC2 Dashboard

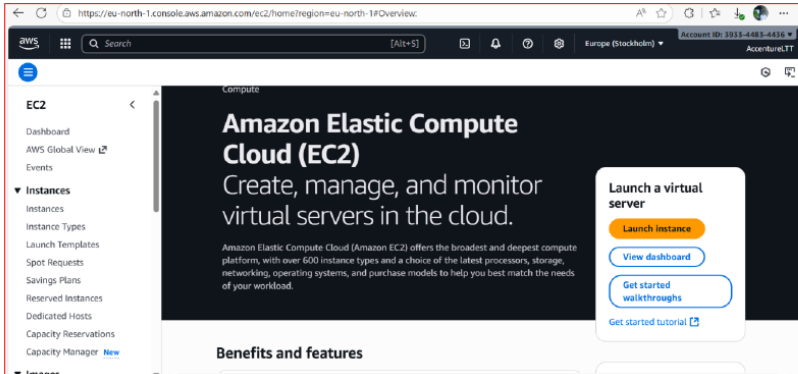
In the search bar at the top, type “EC2”.
Click EC2 under “Services.”



The screenshot shows the AWS console search results for 'EC2'. The search bar at the top contains 'EC2'. Below the search bar, the 'Services' section is expanded, showing 'EC2' as the first result. The 'EC2' service is described as 'Virtual Servers in the Cloud'. Other services shown include 'EC2 Image Builder' and 'Recycle Bin'. The 'Features' section shows 'EC2 Instances' and 'EC2 Resource Health'. At the bottom, there is a feedback prompt: 'Were these results helpful?' with 'Yes' and 'No' buttons.

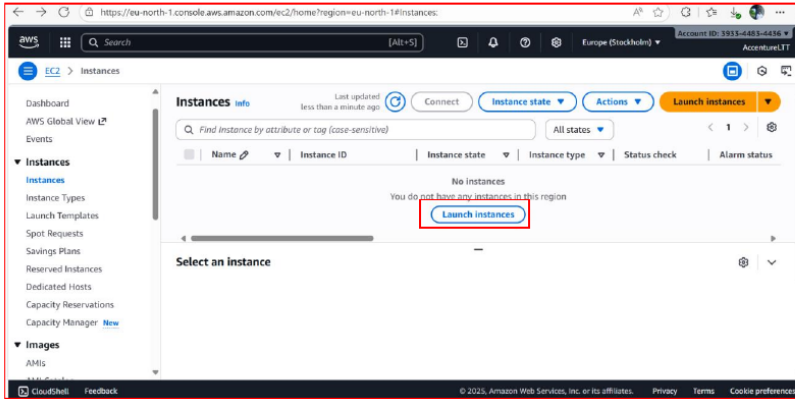
You'll see the **EC2 Dashboard**, showing:

- Instances
- Volumes
- Security Groups
- Key Pairs, etc.
-

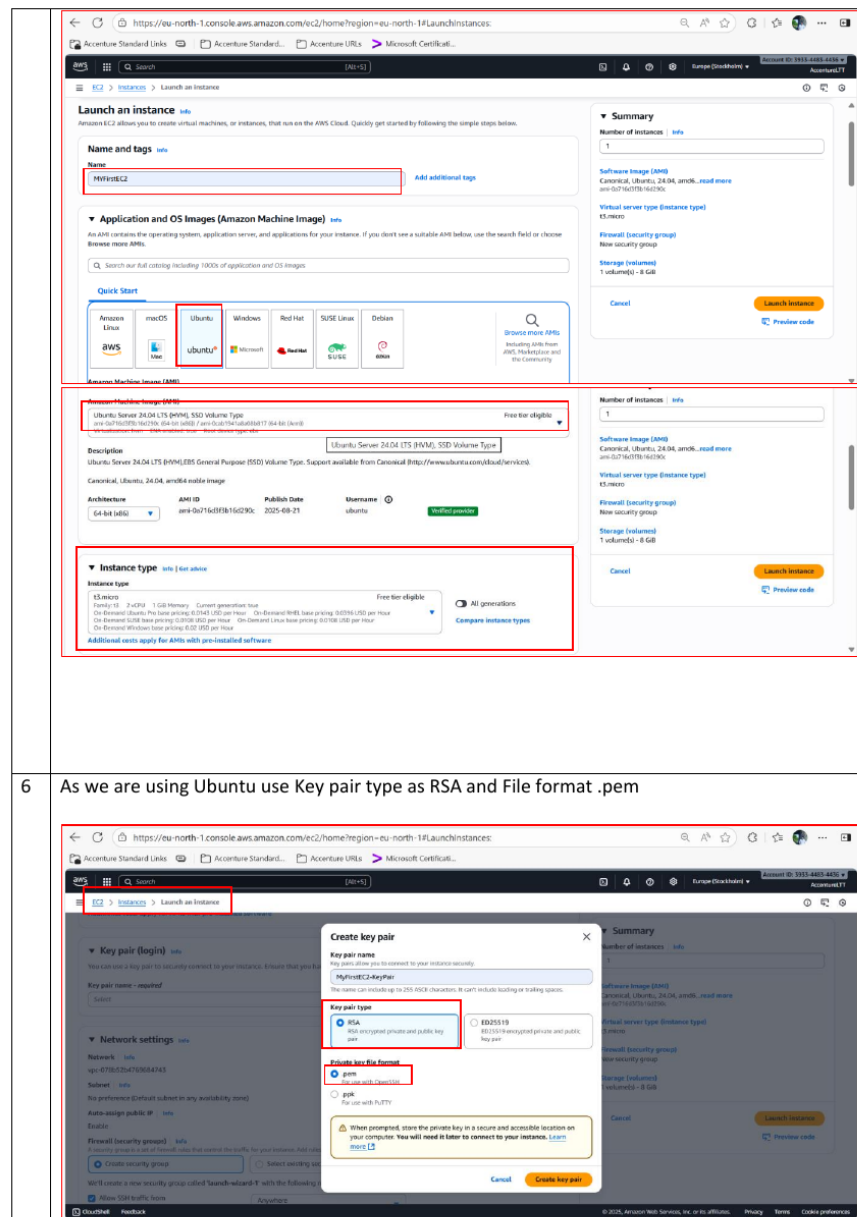


The screenshot shows the Amazon Elastic Compute Cloud (EC2) dashboard. The left sidebar contains the 'EC2' menu with options like 'Dashboard', 'AWS Global View', 'Events', 'Instances', 'Instance Types', 'Launch Templates', 'Spot Requests', 'Savings Plans', 'Reserved Instances', 'Dedicated Hosts', 'Capacity Reservations', and 'Capacity Manager'. The main content area has a heading 'Amazon Elastic Compute Cloud (EC2)' and a subheading 'Create, manage, and monitor virtual servers in the cloud.' Below this, there is a description of EC2 and a 'Benefits and features' section. On the right, there is a 'Launch a virtual server' button and a 'View dashboard' button. At the bottom, there is a 'Get started walkthroughs' button and a 'Get started tutorial' link.

4 Launch a New EC2 Instance
Click “Launch instance.”

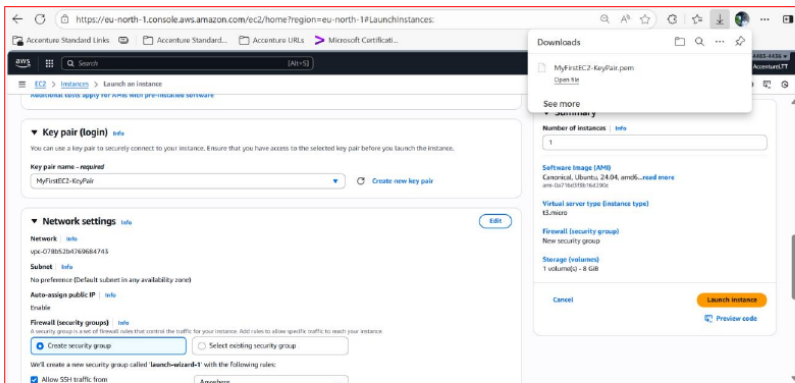


5 Fill the details:
Name and tags → e.g., MyFirstEC2
Application and OS Image (AMI) → Ubuntu. (Free tier eligible)
Instance type → Choose t3.micro (free tier eligible)

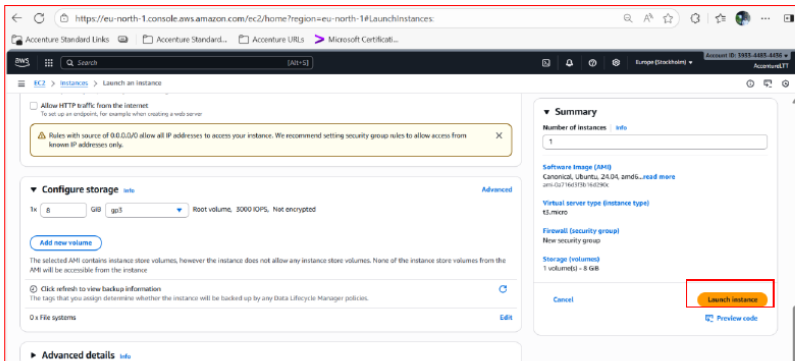


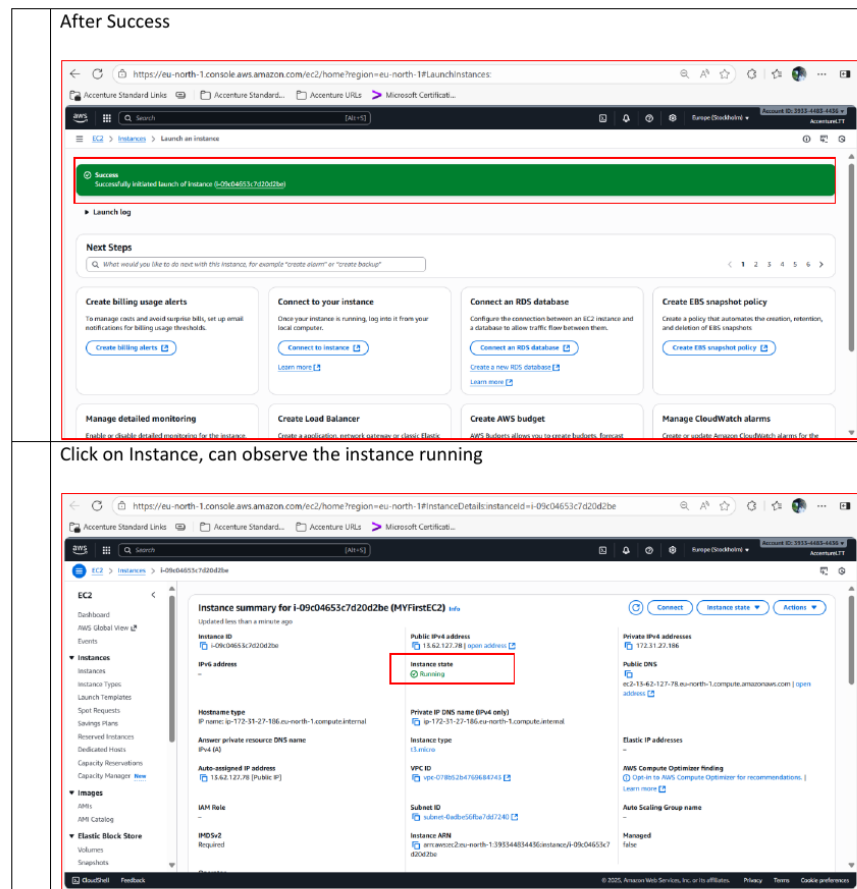
6 As we are using Ubuntu use Key pair type as RSA and File format .pem

.pem File will be downloaded once we key pair is created



7 Launch the instance





By Default, it should have all Inbound rules as shown below

8 If it is not available, you can add Inbound rules,
Go to EC2 Dashboard → Instance → Click on Instance → Security Tab → Security Groups

EC2 Dashboard
AWS Global View
Events
Instances
Instance Types
Launch Templates
Spot Requests
Savings Plans
Reserved Instances
Dedicated Hosts
Capacity Reservations
Capacity Manager
Images
AMI's

Instances (1/1) info

Find instance by attribute or tag (case-sensitive)

Connect Instance state Actions Launch instances

MYFirstEC2 i-09c04653c7d20d2be Stopped t3.micro 8/3 checks passed View alarms eu-north-1a

I-09c04653c7d20d2be (MYFirstEC2)

Name	Security group rule ID	Port range	Protocol	Source	Source type
-	sg-0485c0c0ec8ad9ff0	8080	TCP	0.0.0.0/0	Internet
-	sg-0d4a261a3a86e59	22	TCP	0.0.0.0/0	Internet
-	sg-0d1d17d8b77f566df	80	TCP	0.0.0.0/0	Internet

IAM Role
Owner ID: 39344834435
Launch time: Mon Nov 03 2025 10:56:48 GMT+0530 (India Standard Time)

Security groups
sg-02a4f895e20b7645 (launch-wizard-1)

Inbound rules

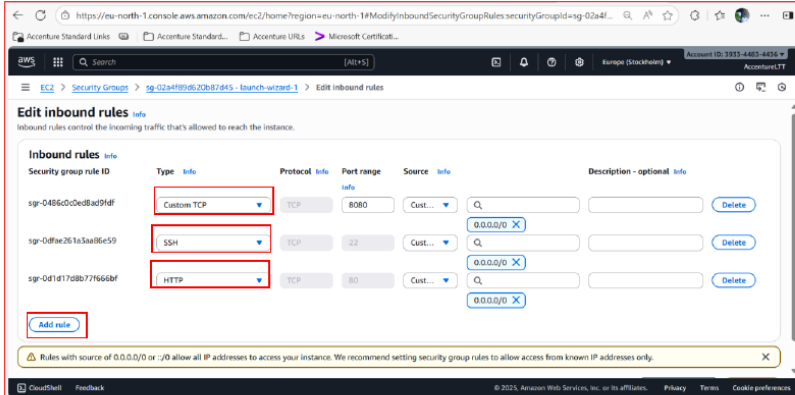
Scroll down

9 Once you click on Security Groups Launch wizard, you will find an option to Edit Inbound rules

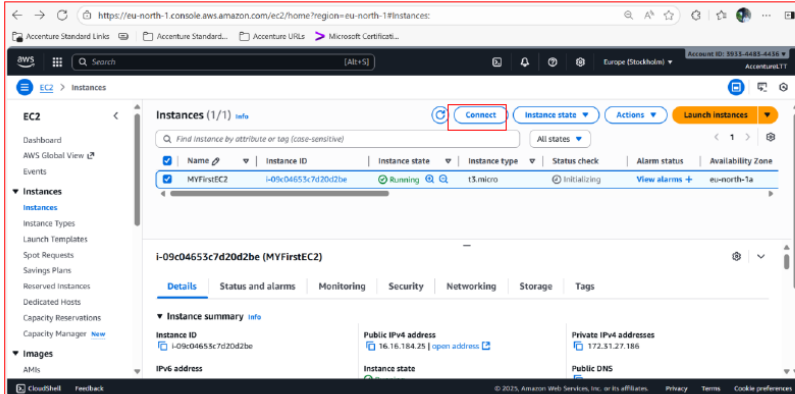
Name	Security group rule ID	Port range	Protocol	Source
-	sg-0486c0e0d8a9f9df	8080	TCP	0.0.0.0/0
-	sg-0dface261a3aa86e59	22	TCP	0.0.0.0/0

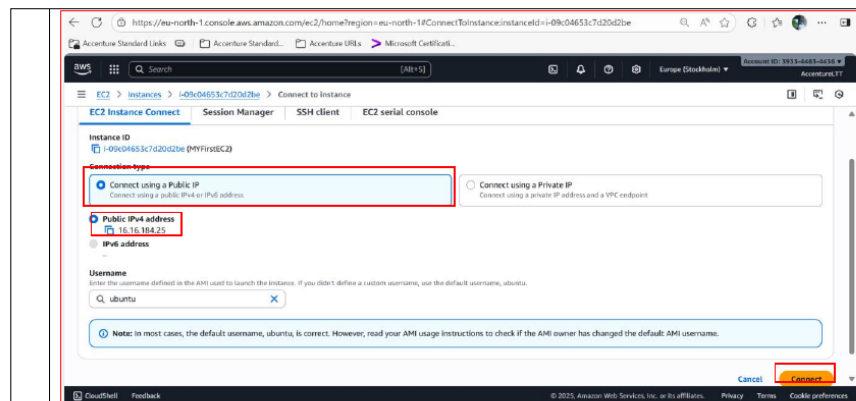
Name	Security group rule ID	IP version	Type	Protocol	Port range
-	sg-0486c0e0d8a9f9df	IPv4	Custom TCP	TCP	8080
-	sg-0dface261a3aa86e59	IPv4	SSH	TCP	22
-	sg-0d101708b07766b0f	IPv4	HTTP	TCP	80

10 Click on Add rule to add respective rules as per requirements.
By default, you can add Custom TCP, SSH and HTTP

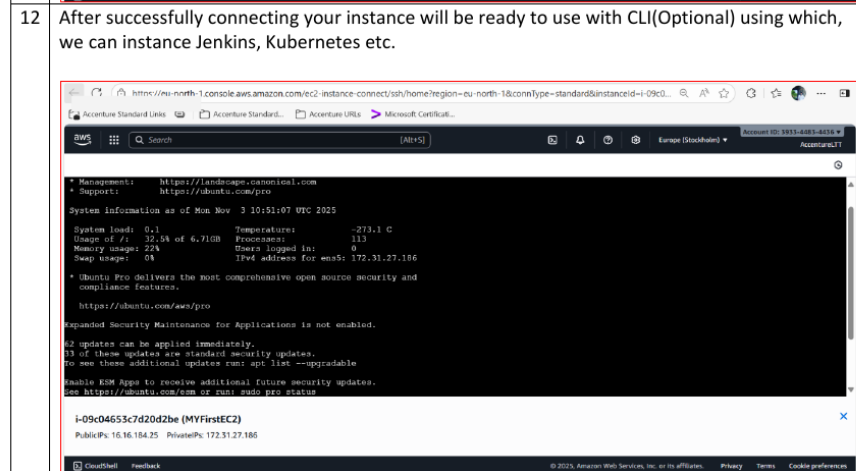


11 You can connect your instance on CLI in AWS account by clicking on Connect





12 After successfully connecting your instance will be ready to use with CLI(Optional) using which, we can instance Jenkins, Kubernetes etc.



```
+ Management: https://landscape.canonical.com
+ Support: https://ubuntu.com/pro

System information as of Mon Nov 3 10:51:07 UTC 2025
System load: 0.1      Temperature: 275.1 C
Usage of /: 32.0k of 6.71GB      Free space: 113
Memory usage: 22k      Swap usage: 0k
IP address for ens5: 172.31.27.186

+ Ubuntu Pro delivers the most comprehensive open source security and compliance features.
https://ubuntu.com/pro

Expanded Security Maintenance for Applications is not enabled.
2 updates can be applied immediately.
11 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable

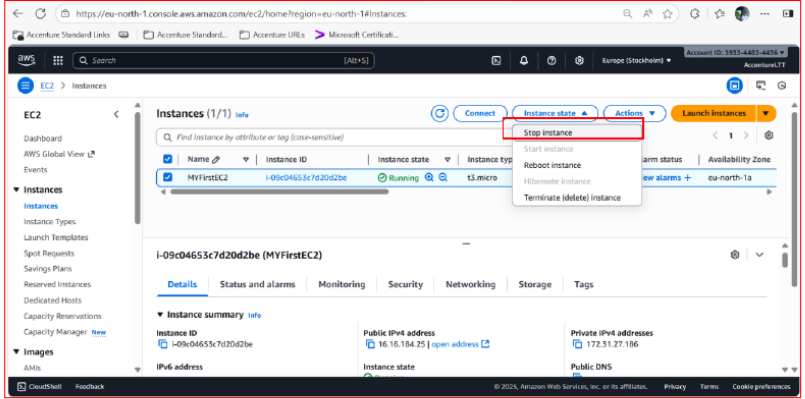
Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

i-09c04653c7d20d2be (MYFirstEC2)
PublicIPs: 16.16.184.25 PrivateIPs: 172.31.27.186
```

13 This is how we can create EC2 on AWS.

Note: - If you don't want any charges to be applied for your AWS account, make sure that you stop or terminate your instance when not in use

Click on Instance State → Stop Instance (In Future if you need) or Terminate Instance if not be used in Future



The screenshot shows the AWS Management Console for the 'eu-north-1' region. The 'Instances' page is active, displaying a table with one instance: 'MYFirstEC2' (Instance ID: i-09c04653c7d20d2be), which is in the 'Running' state. The 'Instance state' dropdown menu is open, showing options: 'Start instance', 'Stop instance', 'Reboot instance', 'Hibernate instance', and 'Terminate (delete) instance'. The 'Stop instance' option is highlighted. Below the table, the details for 'i-09c04653c7d20d2be (MYFirstEC2)' are shown, including the 'Instance summary' with fields for Instance ID, Public IPv4 address, Private IPv4 addresses, Instance state, and Public DNS.

Thank You!

43.6 Introduction to Containerization and Docker

Section 43.2 introduced containers conceptually, in contrast to virtual machines. **Docker** is the tool that made containers practical and mainstream, by standardising both *how* a container image is built and *how* it is run, across any machine that has Docker installed — “build once, run anywhere” in a much more literal sense than was practical before Docker existed.

Docker Concept	Meaning
Dockerfile	A text file of instructions describing how to build a container image — which base image to start from, which files to copy in, which commands to run during the build, and what command to run when a container starts
Image	An immutable, versioned snapshot built from a Dockerfile, containing the application and everything it needs to run — images are typically stored in and pulled from a registry
Container	A running instance of an image — exactly as an EC2 instance is a running instance of an AMI, a container is a running instance of a Docker image
Registry	A storage and distribution service for images — Docker Hub is the most widely used public registry (this chapter’s ECS activity pulls the official <code>nginx:latest</code> image directly from it); AWS’s own equivalent, Amazon ECR (Elastic Container Registry) , is referenced directly in the ECS console’s own navigation

Once an application is packaged as a Docker image, the next problem is running — and keeping running, at scale, across failures — *many* containers reliably in production, which is exactly the problem **container orchestration** solves, and exactly what the next two activities (ECS and EKS) provide as managed AWS services.

43.7 Amazon ECS (Elastic Container Service)

Amazon ECS is AWS’s own, AWS-native container orchestration service: it schedules containers onto a fleet of compute resources, restarts them if they fail, and can scale the number of running copies up or down, all without requiring a Kubernetes cluster to be independently stood up and managed (Kubernetes on AWS is EKS, covered in the next section).

ECS Concept	Meaning
Cluster	A logical grouping of the compute capacity ECS will schedule containers onto — this chapter's activity creates a cluster named MyECSCluster
Task Definition	A blueprint describing what container(s) to run — the image URI, CPU/memory allocation, port mappings, and (for multiple containers working together) how they relate. This chapter's activity defines MySampleTask : 1 vCPU, 3 GB memory, running the nginx:latest image with container port 80 mapped
Task	One running instance of a task definition — analogous to how an EC2 instance is one running instance of an AMI
Service	A construct that maintains a specified <i>number</i> of running tasks continuously, restarting any that stop unexpectedly — appropriate for long-running applications, as opposed to a one-off task run directly (as this chapter's activity does, via Run new task , for a simpler first exposure to the concept)
Launch Type — Fargate vs EC2	Fargate is a serverless launch type: AWS manages the underlying servers the containers run on entirely, and the consumer only specifies the task's own CPU/memory needs — exactly the launch type this chapter's activity uses. The EC2 launch type instead schedules containers onto a fleet of EC2 instances that the consumer provisions and manages directly, trading Fargate's convenience for more control over the underlying host

Why Fargate Specifically, for This Activity

Choosing the Fargate launch type in this chapter's ECS activity means never having to provision, patch, or size a single EC2 instance to actually host the **nginx** container — AWS handles all of that invisibly, in exchange for Fargate's own per-task pricing. This is the container-orchestration equivalent of the serverless trade-off named in Section 43.2: less control over the underlying host, in exchange for far less operational burden.

43.7.1 Activity: Create Amazon ECS (Elastic Container Service)

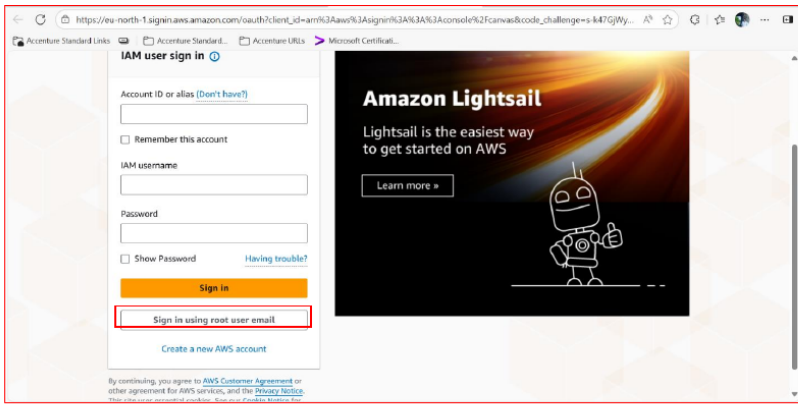
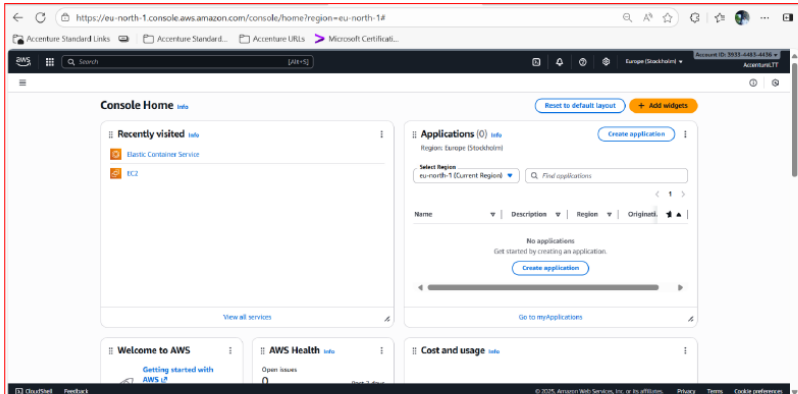
Activity: Run an Nginx Container on ECS Fargate

1. Sign in to the AWS Console and open the **Elastic Container Service**.
2. Navigate to **Clusters** → **Create cluster**, name it (e.g. **MyECSCluster**), and wait for creation to complete — an empty cluster with no running tasks is the result.
3. Navigate to **Task definitions** → **Create new task definition**, select launch type **Fargate**, and configure: **Task name** **MySampleTask**; **Operating system** **Linux**; **Task size** 1 vCPU, 3 GB. Under **Container definitions**, add a container named **nginx**, using image URI **nginx:latest**, with **Port mappings** exposing container port 80. Click **Create**.
4. Return to the cluster, navigate to **Tasks** → **Run new task**, and choose: **Launch type** **Fargate**; **Task definition** **MySampleTask**; **Cluster** **MyECSCluster**; **Number of tasks** 1. Under **VPC and Subnets**, choose the default VPC and at least one public subnet;

- under **Security group**, use one that allows inbound traffic on port 80 (HTTP).
5. Click **Create**. AWS launches the container in Fargate — the task transitions from **Provisioning** to **Running**.
 6. Open the running task's details, find its **Public IP** address, and open that IP address directly in a browser — the default **“Welcome to nginx!”** page confirms the container is genuinely reachable over the internet.
 7. Once finished observing the running task, **Stop** it so it transitions to **Stopped** and no further Fargate compute charges accrue.

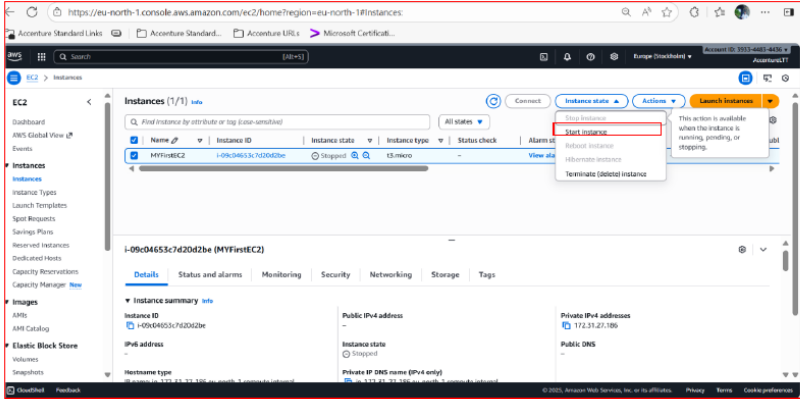
Console Walkthrough — Screenshots

3. Create Amazon ECS (Elastic Container Service)

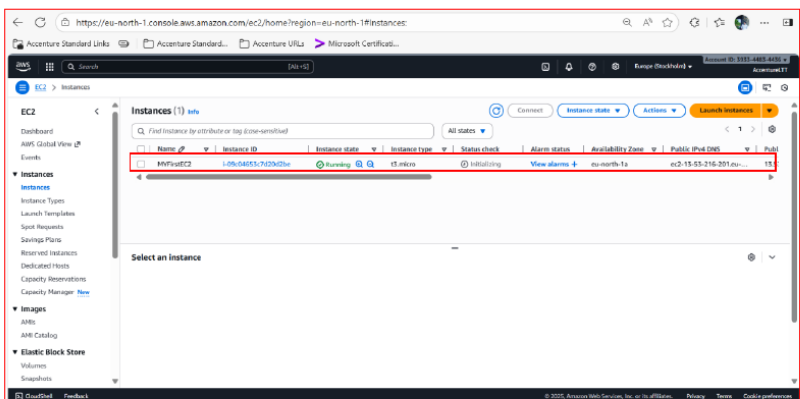
- 1 Use the link below to Sign-In
<https://console.aws.amazon.com/>

- 2 After Successful Sign In you will be launched to the AWS Dashboard


AWS Console — step screenshot (page 1 of 9)

3 Before creating the ECS, launch the EC2 Instance created in previous steps

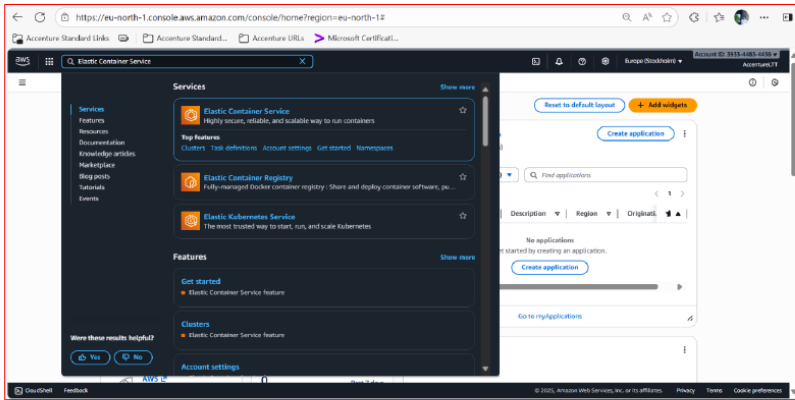


Make Sure that the Instance is running



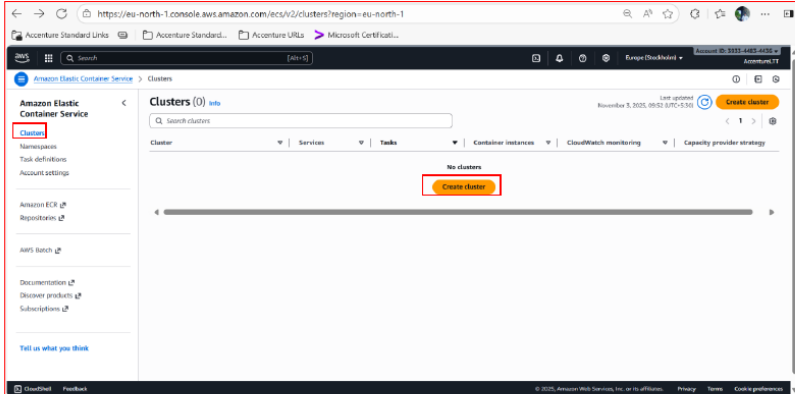
AWS Console — step screenshot (page 2 of 9)

4 Search for Elastic Container Service and Click on the result



The screenshot shows the AWS Management Console search results for 'Elastic Container Service'. The search bar at the top contains 'Elastic Container Service'. The results are displayed in a list format. The first result is 'Elastic Container Service', which is highlighted. Below it are 'Elastic Container Registry' and 'Elastic Kubernetes Service'. The 'Elastic Container Service' result shows a description: 'Highly secure, reliable, and scalable way to run containers'. It also lists 'Key Features' such as 'Clusters', 'Task definitions', 'Account settings', 'Get started', and 'Namespaces'. A 'Get started' button is visible. The 'Elastic Container Registry' result shows a description: 'Fully-managed Docker container registry: store and deploy container software, pa...'. The 'Elastic Kubernetes Service' result shows a description: 'The most trusted way to start, run, and scale Kubernetes'. A 'Get started' button is also visible. The 'Namespaces' result is partially visible at the bottom. The 'Get started' button for 'Elastic Container Service' is highlighted with a red box.

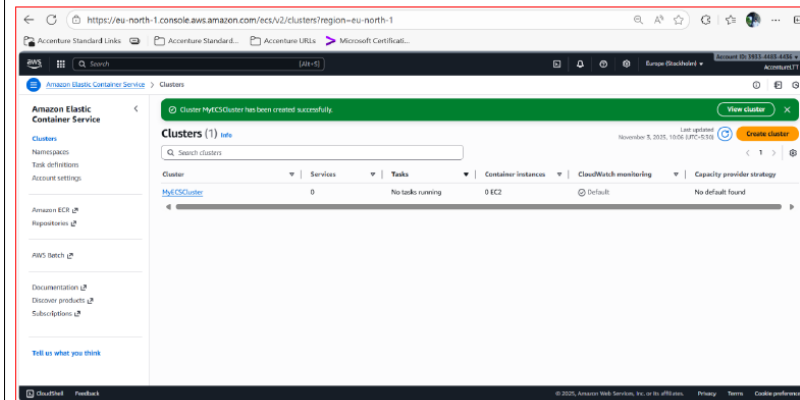
5 Amazon Elastic Container Service → Clusters → Create Cluster



The screenshot shows the Amazon Elastic Container Service (ECS) console. The left sidebar contains the 'Amazon Elastic Container Service' menu. The 'Clusters' link is highlighted with a red box. The main content area shows the 'Clusters' page. The 'Create Cluster' button is highlighted with a red box. The page displays a table with no clusters listed. The 'Create Cluster' button is located at the bottom right of the table.

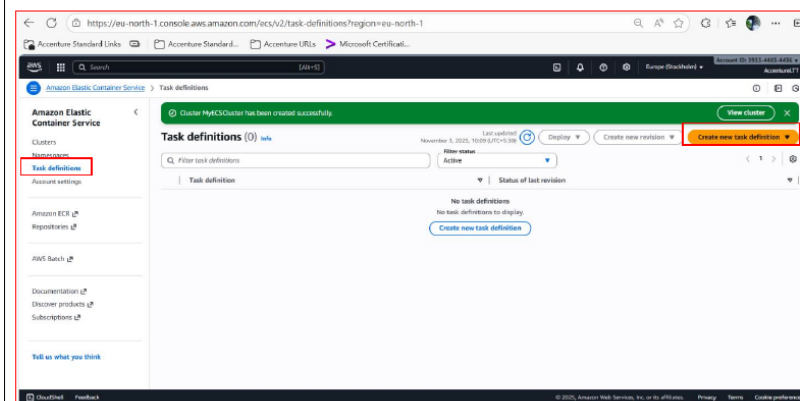
6 Wait for some times don't refresh it may fail, wait until it gets created successfully.

You now have an empty ECS cluster.



7 Next Step: Create a Task Definition

A task definition describes what container(s) to run.



8 In the ECS sidebar, choose **Task definitions** → **Create new task definition**.

Select **Fargate**.

Configure:

Task name: MySampleTask

Task role: Leave default

Operating system: Linux.

Task size: 1 vCPU, 3 GB

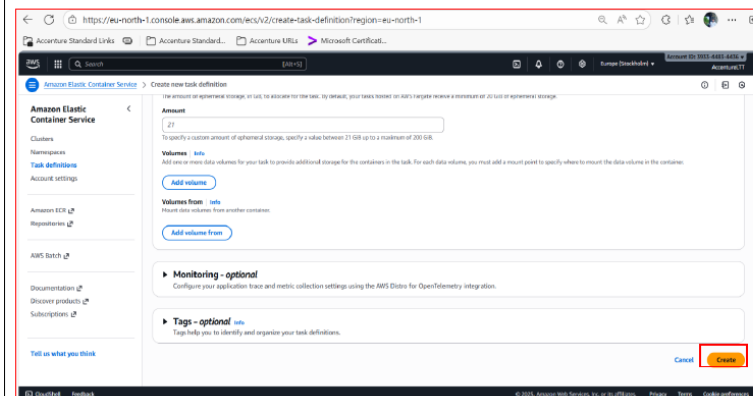
Under **Container definitions**, click **Add container**:

Container name: nginx

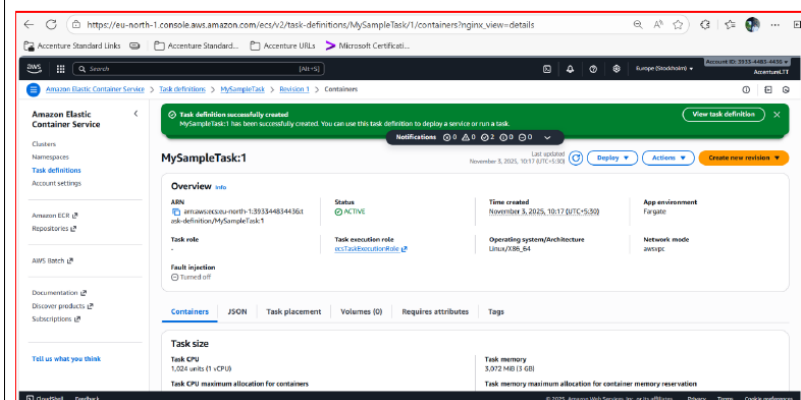
Image URI: nginx:latest (I am using available container)

Port mappings: Container port 80.

Click **Add**, then **Create**.

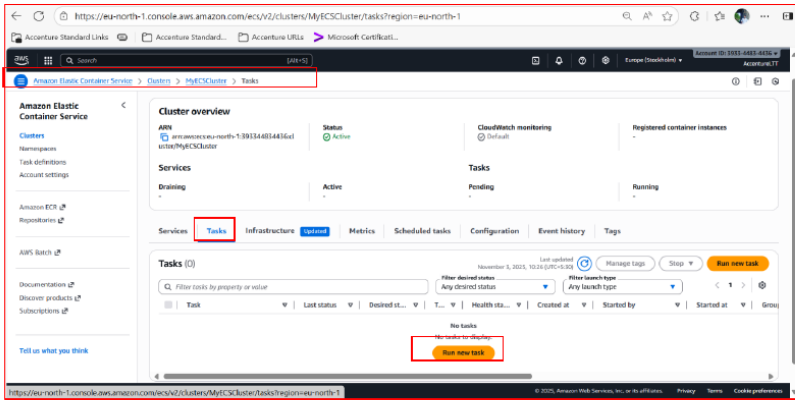


After Successfully Creation you can observe the status of the task created



AWS Console — step screenshot (page 5 of 9)

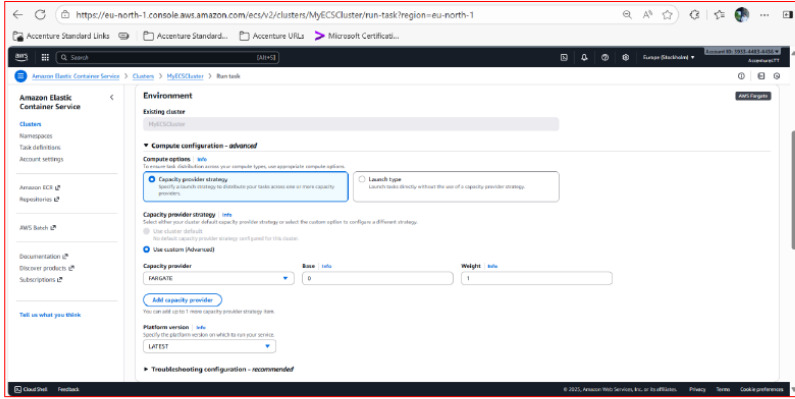
9 Now Go to your cluster → **Tasks** → **Run new task**.



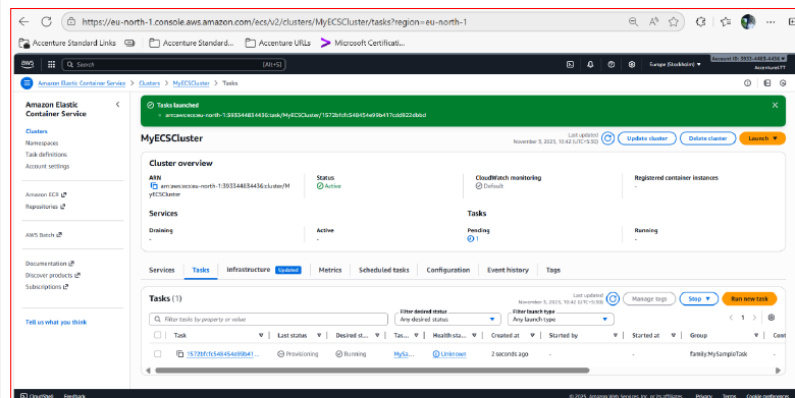
10 Choose:

- **Launch type:** Fargate
- **Task definition:** MySampleTask
- **Cluster:** MyECSCluster
- **Number of tasks:** 1

Under **VPC** and **Subnets**, choose your default VPC and at least one public subnet.
Under **Security group**, use default which allows inbound traffic on port 80 (HTTP).



11 Click Create, AWS will now launch a container in Fargate

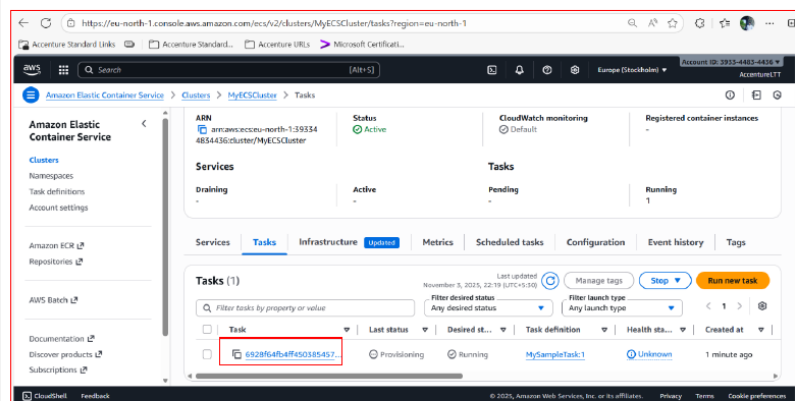


12 Check It's Running

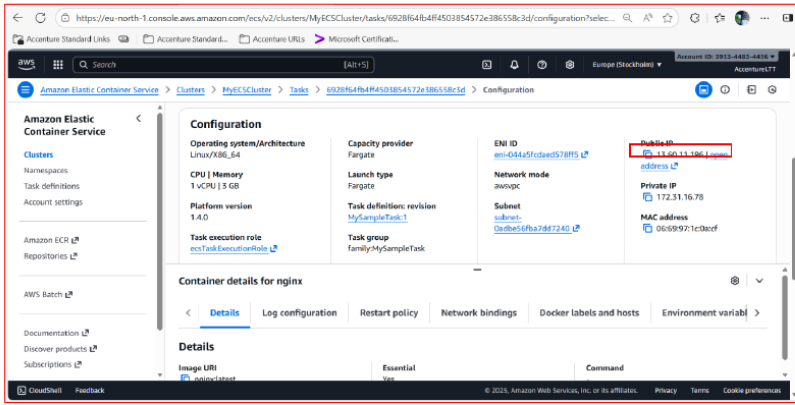
Go to your cluster → Tasks → [task ID].

Find the ENI (Elastic Network Interface) or Public IP address.

Open that IP in your browser → you should see the default Nginx welcome page.

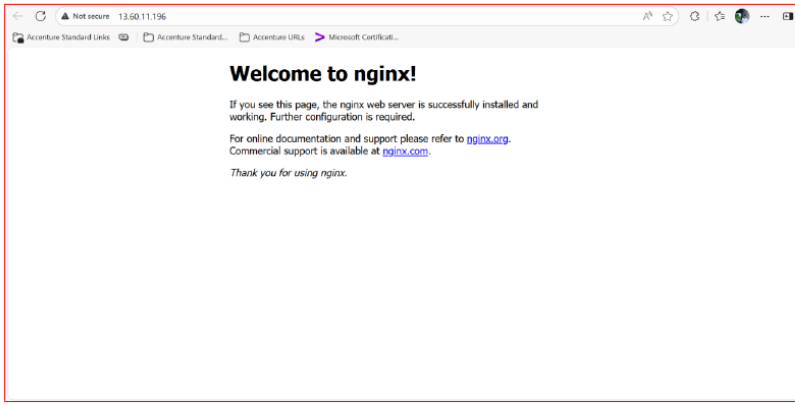


13 After clicking on Created Task
You can observe the public IP address → Click on IP address



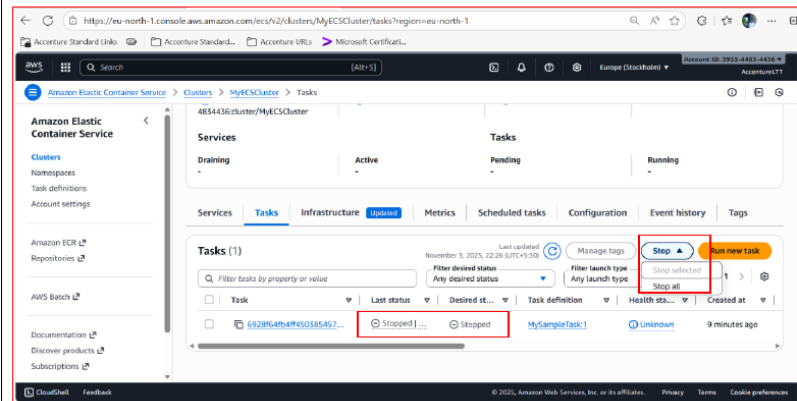
The screenshot shows the AWS Management Console for the Amazon Elastic Container Service (ECS). The left sidebar shows the navigation menu with 'Amazon Elastic Container Service' selected. The main content area displays the 'Configuration' tab for a specific task. The 'Public IP' field is highlighted with a red box, showing the address 172.31.36.196. Other fields include ENI ID, Network mode, Subnet, and MAC address.

14 Default Nginx welcome page will be launched.



The screenshot shows a web browser displaying the default Nginx welcome page. The page has a white background with black text. The heading 'Welcome to nginx!' is prominently displayed. Below it, there is a paragraph stating: 'If you see this page, the nginx web server is successfully installed and working. Further configuration is required.' This is followed by a link to 'For online documentation and support please refer to [nginx.org](\"http://nginx.org\").' and another line: 'Commercial support is available at [nginx.com](\"http://nginx.com\").' The page ends with 'Thank you for using nginx.'

15 Once the tasks are completed, please stop, so you can observe the task status as Stopped



AWS Console — step screenshot (page 9 of 9)

43.8 Amazon EKS (Elastic Kubernetes Service)

Where ECS is AWS's own, proprietary container orchestrator, **Kubernetes** is a vendor-neutral, open-source container orchestration system, originally developed at Google, that has become the de facto industry standard — and **Amazon EKS** is AWS's managed Kubernetes offering, running the same Kubernetes API and concepts a team might otherwise run on any other cloud, or on their own hardware, while letting AWS manage the operationally difficult parts of the underlying Kubernetes **control plane**.

43.8.1 Kubernetes Core Vocabulary

Concept	Meaning
Control Plane	The set of components that make global decisions about the cluster (scheduling, detecting and responding to cluster events) — in EKS, AWS runs and manages this entirely; the consumer never directly administers it
Node	A worker machine (a VM, ultimately an EC2 instance underneath) that actually runs containerised workloads, managed by the control plane
Pod	The smallest deployable unit in Kubernetes — one or more tightly-coupled containers that are always scheduled together, onto the same node

43.8.2 Why EKS Needs So Much IAM Setup Before the Cluster Itself

Unlike this chapter's ECS activity, where a cluster could be created almost immediately, EKS's own activity spends the majority of its steps on **IAM (Identity and Access Management)** roles *before* the cluster is even created. This is not incidental complexity — it reflects a genuine security principle worth understanding on its own: EKS's control plane needs explicit permission to manage other AWS resources (EC2 instances for nodes, load balancers, VPC networking) *on the consumer's behalf*, and Kubernetes worker nodes separately need their own permission to actually register themselves with the cluster and pull container images. AWS deliberately does not grant either of these permissions by default — doing so would violate the **principle of least privilege** (Chapter 40 of the previous volume) at the level of the cloud account itself, so two distinct IAM roles must be created and attached, each scoped only to what it specifically needs:

IAM Role	Attached To	Purpose
Cluster IAM Role (e.g. <code>MyFirstEKSClusterRole</code>)	The EKS cluster itself	Lets the EKS control plane manage AWS resources (EC2, VPC, load balancers) needed to operate the cluster, via the <code>AmazonEKSClusterPolicy</code> and, for EKS Auto Mode specifically, four additional managed policies (<code>AmazonEKSBlockStoragePolicy</code> , <code>AmazonEKSComputePolicy</code> , <code>AmazonEKSLoadBalancingPolicy</code> , and <code>AmazonEKSNetworkPolicy</code>)
Node IAM Role (e.g. <code>AmazonEKSAutoNodeRole</code>)	The worker nodes	Lets nodes connect to the cluster and pull container images from ECR, via <code>AmazonEKSElasticContainerRegistryPolicy</code> , <code>AmazonEKSWorkerNodePolicy</code> , <code>AmazonEKS_CNI_Policy</code> , and <code>AmazonEC2ContainerRegistryPolicy</code>

The **trust policy** on the Cluster IAM Role deserves particular attention: it must explicitly state that the `eks.amazonaws.com` service is allowed to *assume* this role (`sts:AssumeRole`), and — specifically for EKS Auto Mode — must additionally permit `sts:TagSession`, an action that lets the assumed role attach session tags used internally by EKS Auto Mode’s automated node/storage/networking management. This distinction between a role’s **permissions policy** (*what* the role can do) and its **trust policy** (*who*, or which service, is allowed to assume the role in the first place) is a general IAM concept that applies well beyond EKS — every IAM role in AWS has both, and a role is only usable once both are correctly configured.

43.8.3 Networking Prerequisites — Why EKS Needs Its Own VPC

EKS additionally requires a **VPC (Virtual Private Cloud)** — covered in full theoretical depth in Chapter 44 — with subnets spanning *at least two* Availability Zones, because Kubernetes control planes and node groups are designed to be resilient to the loss of any single data centre; the VPC Wizard's **VPC and more** option is the fastest path to this, automatically creating a VPC, public and private subnets across the requested number of Availability Zones, an Internet Gateway, and a NAT Gateway (needed specifically so worker nodes placed in private subnets can still reach the internet to pull container images, without being directly, publicly internet-facing themselves).

43.8.4 Activity: Create an EKS Cluster

Activity: Provision an EKS Cluster with Its Supporting IAM Roles and VPC

1. **Create the VPC:** In the VPC console, choose **Your VPCs** → **Create VPC** → **VPC and more**, set **Number of Availability Zones** to 2, keep default subnet settings (2 public, 2 private), and click **Create VPC** — this provisions the VPC, subnets, an Internet Gateway, and a NAT Gateway together. Note the VPC ID for later steps.
2. **Create the Cluster IAM Role:** In the IAM console, **Roles** → **Create role**, Trusted entity type **AWS service**, Use case **EKS** → **EKS - Cluster**. Accept the pre-selected **AmazonEKSClusterPolicy**, name the role (e.g. **MyFirstEKSClusterRole**), and create it.
3. **Attach the additional EKS Auto Mode policies:** back in IAM, select the newly-created role, **Add permissions** → **Attach policies**, and attach **AmazonEKSBlockStoragePolicy**, **AmazonEKSComputePolicy**, **AmazonEKSLoadBalancingPolicy**, and **AmazonEKSNetworkingPolicy**.
4. **Update the role's trust policy:** on the same role, open the **Trust relationships** tab → **Edit trust policy**, locate the existing statement permitting **eks.amazonaws.com** to assume the role, and add **"sts:TagSession"** alongside the existing **"sts:AssumeRole"** action, then **Update policy**.
5. **Create the Node IAM Role:** back in IAM, **Roles** → **Create role**, Trusted entity type **AWS service**, Use case **EC2** (since nodes are ultimately EC2 instances). Attach **AmazonEKSEKSWorkerNodePolicy**, **AmazonEKS_CNI_Policy**, and **AmazonEC2ContainerRegistryReadOnly**, then create the role.
6. **Create the cluster itself:** in the EKS console, **Clusters** → **Create cluster**. Provide a **Name** (e.g. **MyFirstEKSCluster**), choose a **Kubernetes version**, select the **Cluster IAM role** created above, select the **Node IAM role** created above, and select the **VPC** created above (with its subnets).
7. Click **Create**. The cluster transitions from **Creating** to **Active**, and EKS Auto Mode will create nodes automatically as workloads actually need them.

Console Walkthrough — Screenshots

4. Activity to create EKS

1

Prerequisites (IAM Roles and Networking)

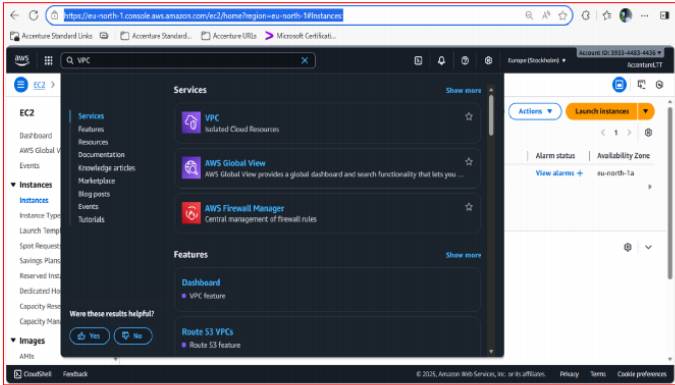
Before creating the cluster, you need foundational resources.

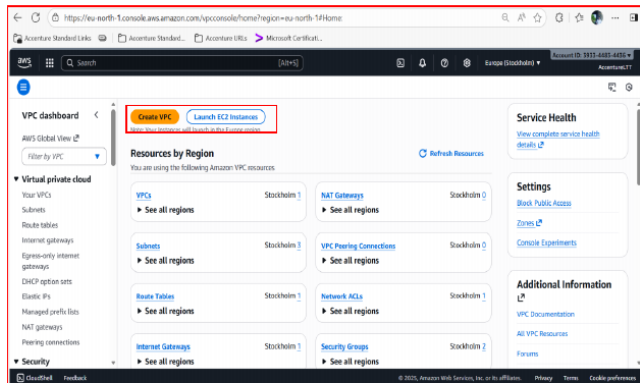
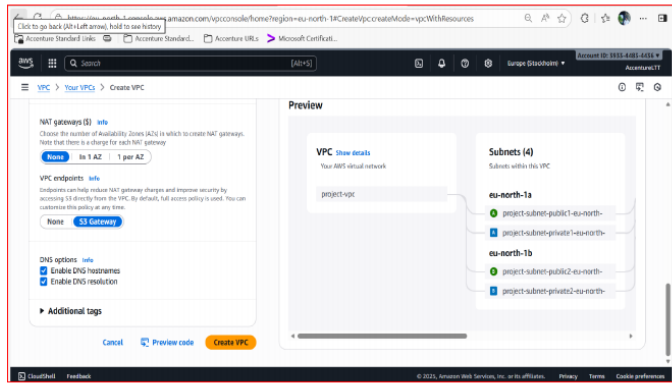
Create a VPC and Subnets

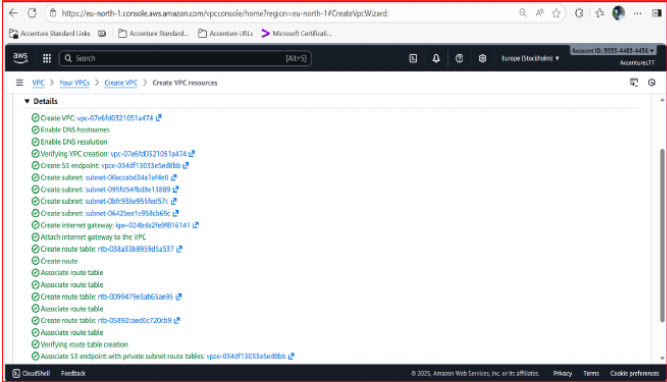
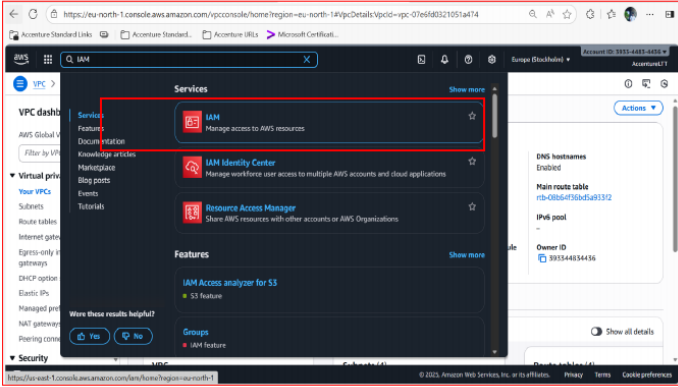
EKS requires a Virtual Private Cloud (VPC) with subnets across at least two Availability Zones (AZs). The easiest way to do this is using the VPC Wizard:

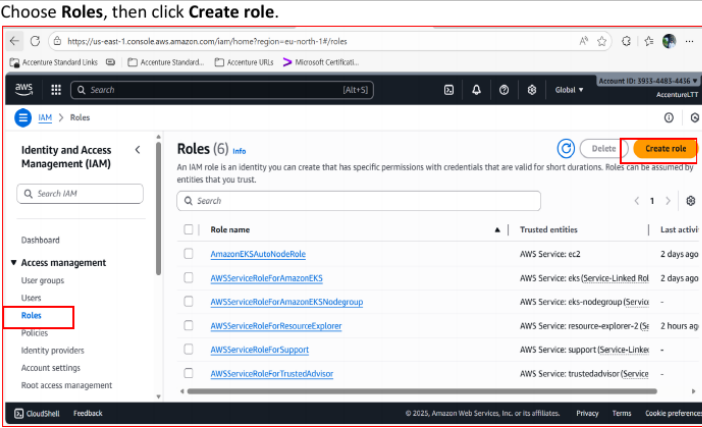
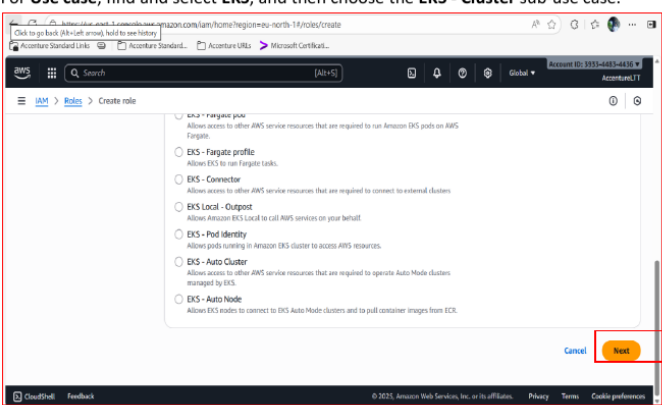
Sign In to AWS

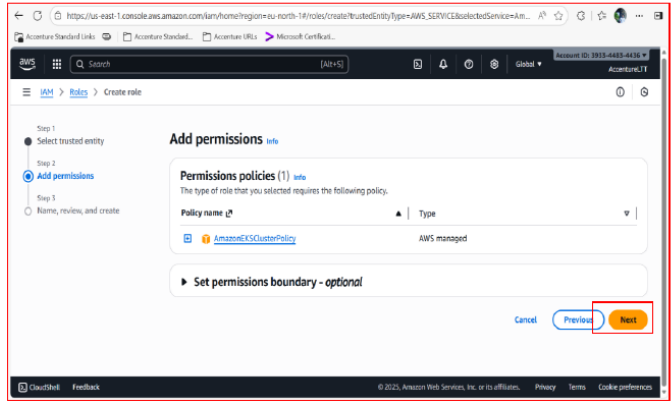
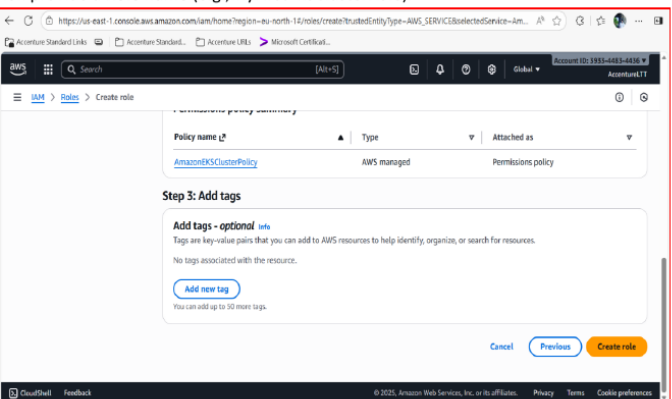
Search for VPC in search bar

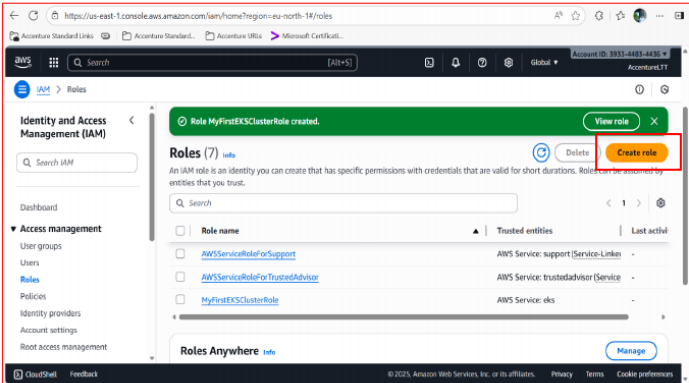
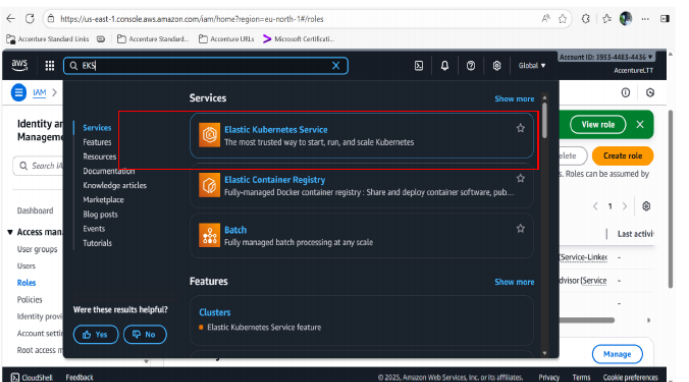
A screenshot of the AWS Management Console interface. The search bar at the top contains the text 'VPC'. Below the search bar, the 'Services' section is expanded, showing 'VPC' as the top result. Other services listed include 'AWS Global View' and 'AWS Firewall Manager'. The 'Features' section is also visible, showing 'Dashboard' and 'Route 53 VPCs'. The left-hand navigation menu is visible, with 'Instances' and 'Images' sections expanded. The right-hand pane shows the 'VPC' service details, including a 'Launch instances' button and a 'View alarms' link.

2	Go to the VPC console. 
3	In the left navigation pane, choose Your VPCs, then click Create VPC. 1. Select VPC and more (This is the fastest method). 2. For Number of Availability Zones (AZs), choose 2. 3. Keep the defaults for subnets (2 public, 2 private). 4. Click Create VPC. This will create a VPC, subnets, Internet Gateway, and NAT Gateway (which costs money but is needed for worker nodes in private subnets to pull images). Note down the VPC ID. 

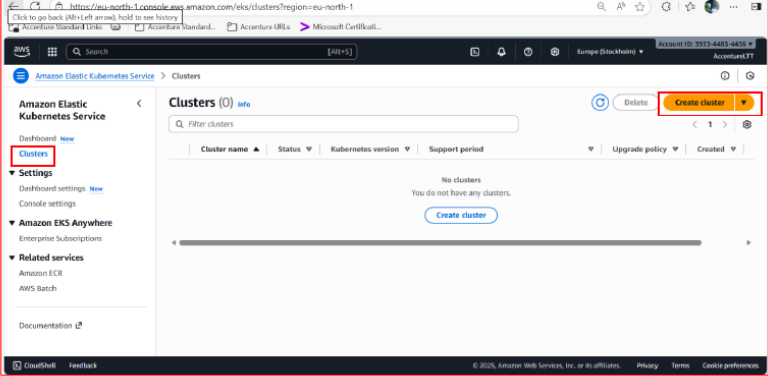
4	View VPC once created successfully. 
5	Create an EKS Cluster IAM Role The EKS control plane needs permission to manage AWS resources (like EC2 instances, VPC, and Load Balancers) on your behalf. Go to the IAM console. 

6	Choose Roles, then click Create role. 
7	For Trusted entity type, select AWS service. For Use case, find and select EKS, and then choose the EKS - Cluster sub-use case. 

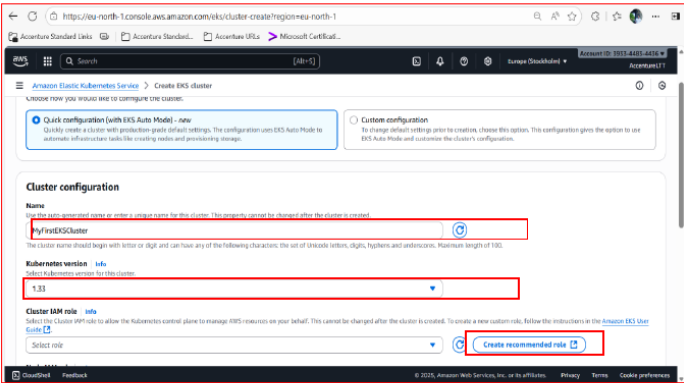
8	Click Next. The AmazonEKSClusterPolicy will be pre-selected. 
9	Click Next, then provide a Role name (e.g., MyFirstEKSClusterRole). 

10	Click Create role.  The screenshot shows the AWS IAM console 'Roles' page. A green banner at the top indicates 'Role MyFirstEKSClusterRole created.' Below this, the 'Roles (7)' section lists several roles, including 'MyFirstEKSClusterRole'. A red rectangular box highlights the 'Create role' button in the top right corner of the console interface.
11	Create the EKS Cluster Control Plane Go to the Amazon EKS console.  The screenshot shows the AWS IAM console 'Services' page. A red rectangular box highlights the 'Elastic Kubernetes Service' option, which is described as 'The most trusted way to start, run, and scale Kubernetes'. Below this, the 'Features' section lists 'Elastic Container Registry' and 'Batch'. The 'Clusters' section lists 'Elastic Kubernetes Service feature'.

12 Click the **Clusters** link in the left pane, then click **Create cluster**.



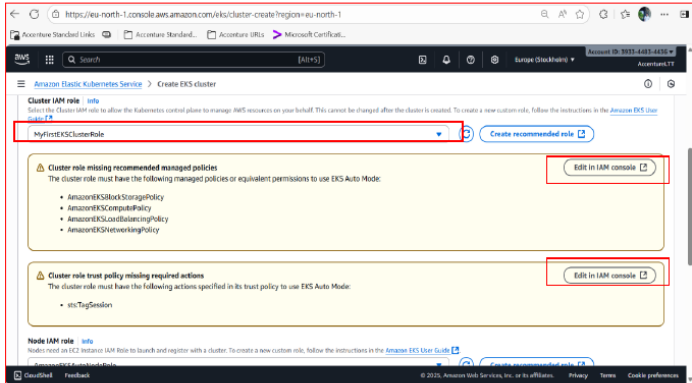
13 Configure cluster:
Name: Enter a unique name (e.g., MyFirstEKSCluster).



14

Kubernetes version: Choose the latest supported version.

Cluster service role: Select the **Cluster IAM Role** you created in previous step (e.g., MyFirstEKSClusterRole).



15

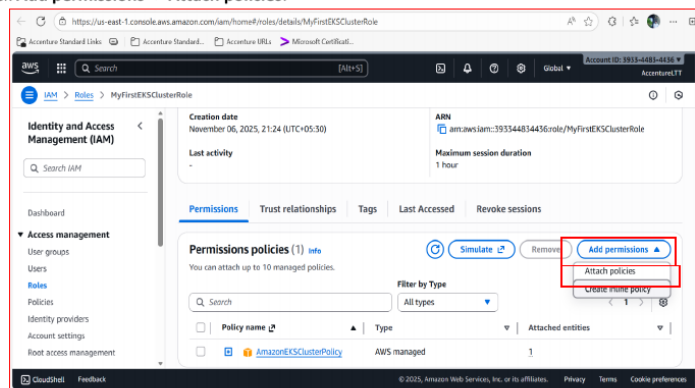
Update the Cluster Role for EKS Auto Mode

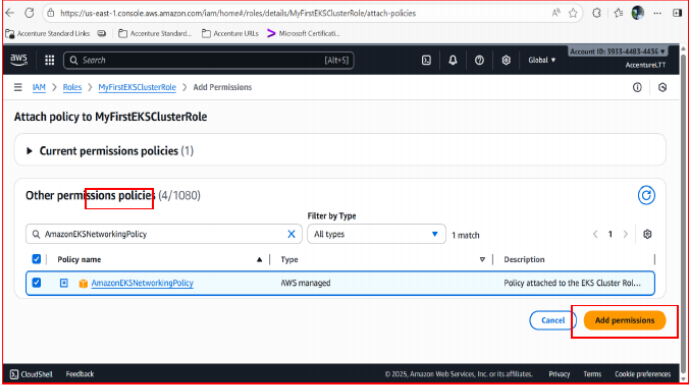
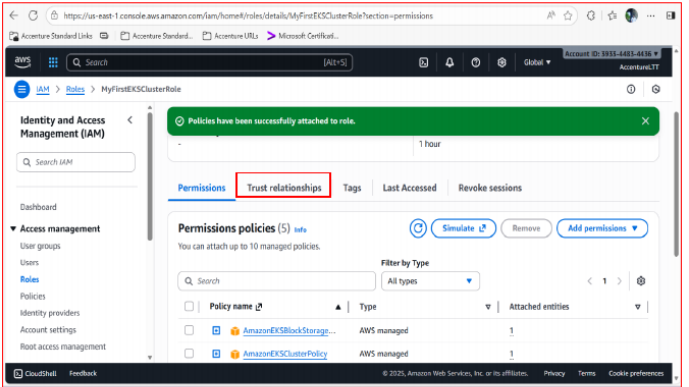
The simplest way to fix this is to use the AWS Console's recommendation feature.

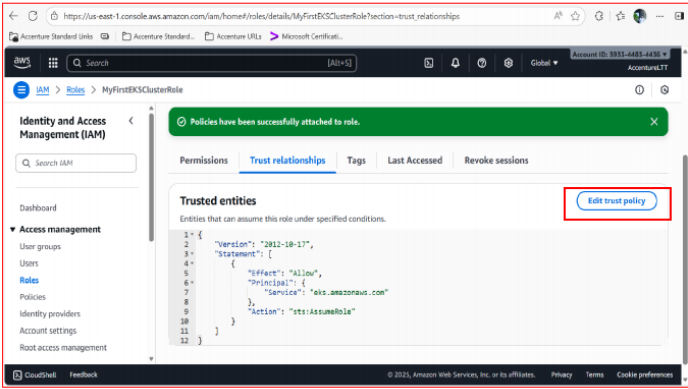
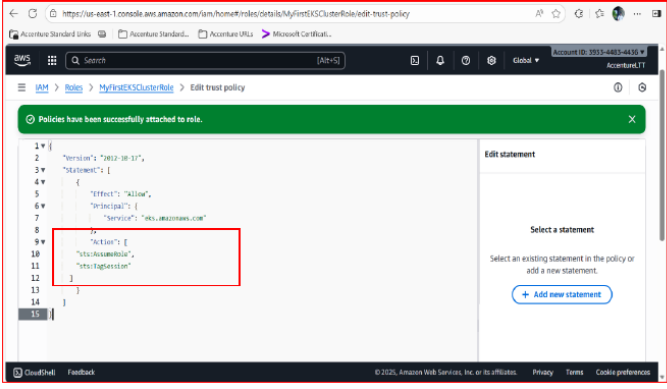
1. Click the **"Create recommended role"** button: In the top right corner of the image you provided, click the **Create recommended role** button.
 - If you're still on the EKS cluster creation page, this button should automatically create a new, compliant IAM role and select it for your cluster.
2. **Manually Edit the IAM Role (If you click "Edit in IAM console"):**
 - If you choose to use your existing role (MyFirstEKSClusterRole), you need to navigate to the **IAM Console** and attach the missing policies and update the trust policy.
3. **Attach Missing Managed Policies**

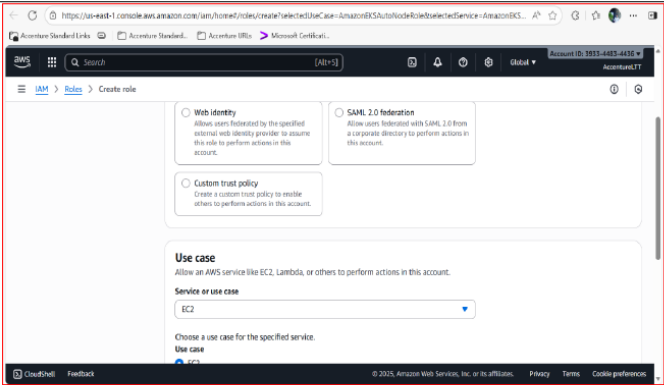
You must attach the four policies listed in the first warning:

 1. Go to the **IAM Console** -> **Roles**.
 2. Search for and select your cluster role (MyFirstEKSClusterRole).
 3. Click **Add permissions** -> **Attach policies**.



16	4. Search for and attach the following four AWS Managed Policies: • AmazonEKSBlockStoragePolicy • AmazonEKSClusterPolicy • AmazonEKSLoadBalancingPolicy • AmazonEKSNetworkingPolicy 
17	Update the Trust Relationship You must add the sts:TagSession action to the trust policy as indicated in the second warning. 1. While still viewing your cluster role (MyFirstEKSClusterRole), click the Trust relationships tab. 

18	2. Click Edit trust policy. 
19	3. Locate the existing statement that allows the EKS service to assume the role (where "Service": "eks.amazonaws.com"). 4. Add "sts:TagSession" to the Action array. 
20	Click Update Policy. Once you have either created the recommended role or manually updated your existing role, click the refresh icon next to the role dropdown in the EKS console.
15	Create the Node IAM Role 1. Navigate to the IAM console in a new browser tab. 2. In the left navigation pane, choose Roles, and then click Create role. 3. For Trusted entity type, select AWS service. 4. For the Use case, select EC2 (because your nodes are essentially EC2 instances). 5. Click Next.

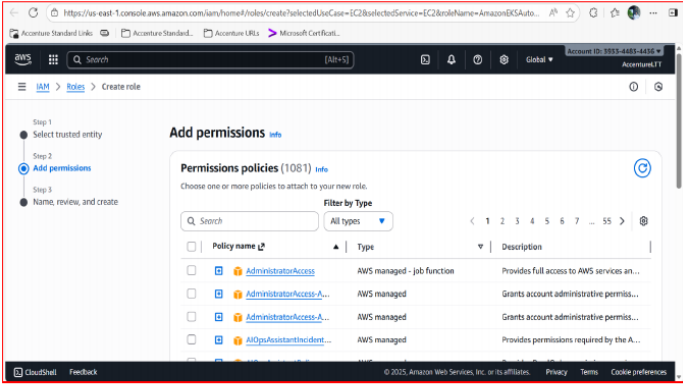


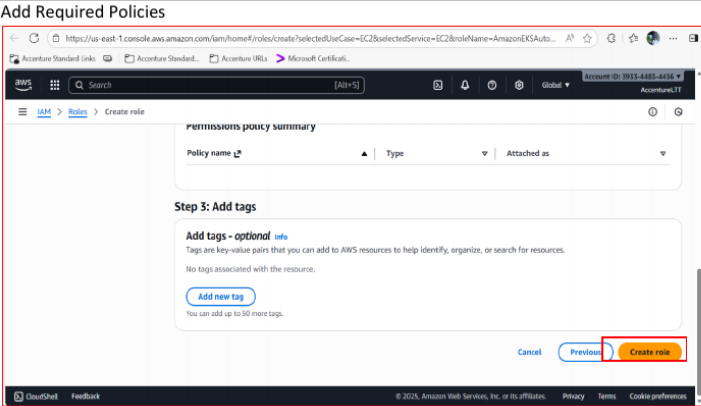
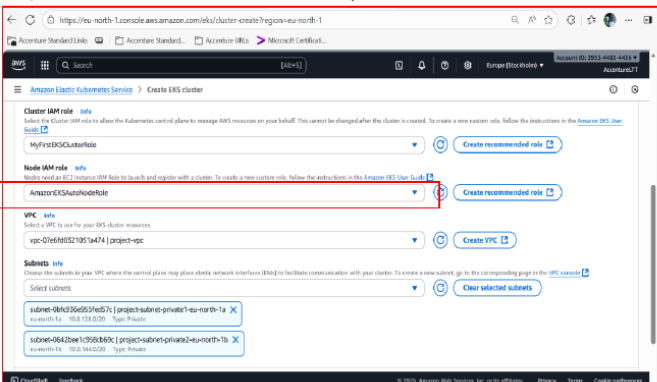
16

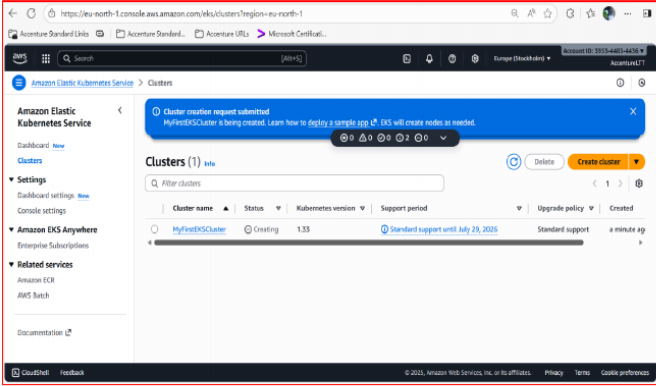
Attach Required Policies

On the "Add permissions" page, you must attach the following three AWS Managed Policies:

- AmazonEKSWorkerNodePolicy
- AmazonEKS_CNI_Policy
- AmazonEC2ContainerRegistryReadOnly



17	Add Required Policies 
	Create role Once you have either created the recommended role or manually updated your existing role, click the refresh icon next to the role dropdown in the EKS console. 

18	Select VPC Created in previous step Click on Create 
	Your Cluster will be created Thank You!

43.9 Serverless Computing on AWS and AWS Lambda

Section 43.2 introduced serverless computing conceptually, as the far end of the compute-abstraction spectrum from virtual machines. **AWS Lambda** is AWS’s Function-as-a-Service offering, and understanding its execution model matters for reasoning correctly about both its cost and its behaviour under load.

43.9.1 The Lambda Execution Model

A Lambda **function** is a self-contained unit of code, together with a declared **runtime** (the language and version it runs in — Python 3.12, Node.js 20, and so on), a **handler** (the specific entry-point method Lambda invokes when the function runs — conventionally `lambda_function.lambda_handler` for Python, matching the pattern of a `main` method acting as an application’s entry point in the Java world covered throughout the rest of this course), and a configured amount of **memory** and **timeout**.

Critically, a Lambda function does *not* run continuously waiting for requests — it runs only in direct response to a **trigger**, an event from another AWS service (an HTTP request via API Gateway, a new object uploaded to an S3 bucket, a scheduled EventBridge rule, and many others) or a manual test invocation. When a trigger fires and no existing execution environment is available to handle it, AWS provisions a fresh one on the spot — this is a **cold start**, and it introduces a small amount of additional latency compared to a **warm** invocation, which reuses an execution environment AWS has kept alive from a recent previous invocation. This on-demand provisioning is exactly what makes Lambda’s pricing model possible: billing is based on the number of requests and the actual compute time consumed while the function runs, rounded to the nearest millisecond, with *no charge at all* while no invocation is in progress — a sharp contrast with an EC2 instance, which (Section 43.4) accrues charges continuously for as long as it remains in the **Running** state, regardless of whether it is actually doing any useful work at any given moment.

43.9.2 Execution Role — IAM for a Function, Not a Server

Exactly as an EKS cluster and its nodes each needed their own scoped IAM role (Section 43.8), a Lambda function needs an **execution role** defining exactly what AWS resources it is permitted to interact with while it runs — at minimum, permission to write logs to **Amazon CloudWatch** (Lambda’s default execution role, “basic Lambda permissions”, grants exactly this and nothing more), and, as this chapter’s final activity shows, additional permissions (such as read/write access to a specific DynamoDB table) attached explicitly as the function’s actual responsibilities grow.

43.9.3 Activity: Create a Lambda Function (Console)

Activity: Author and Trigger a Lambda Function

1. Sign in to the AWS Console and navigate to the **Lambda** service.
2. Click **Create function** and choose **Author from scratch**.
3. Under **Basic information**: give the function a **Function name** (e.g. `MyFirstLambdaFunction`); choose a **Runtime** (e.g. Node.js 20 or Python 3.12); choose an **Architecture** (x86_64, or arm64 for AWS Graviton2, which often provides better performance at lower cost); and, for **Permissions**, choose **Create a new role with basic Lambda permissions**, which creates an execution role that can write logs to CloudWatch.
4. Once created, use the in-console **Code source** editor to write or paste the function’s

code directly (an inline editor is sufficient for a simple function; production deployments more typically upload a zipped deployment package or a container image instead), and confirm the **handler** matches the actual entry-point method in that code.

5. Under the **Configuration** tab, adjust **memory** and **timeout** settings as the function's workload requires.
6. Click **+ Add trigger** in the Function overview to wire the function up to an actual event source — for example, selecting **API Gateway**, choosing to **Create a new API**, and selecting the API type (e.g. REST API) turns the function into an HTTP-invokable endpoint; selecting **S3** and a target bucket instead triggers the function automatically whenever a matching event (e.g. *All object create events*) occurs on that bucket.
7. Use the **Test** tab to invoke the function directly with a sample event payload and confirm its response, independent of whatever trigger was configured.

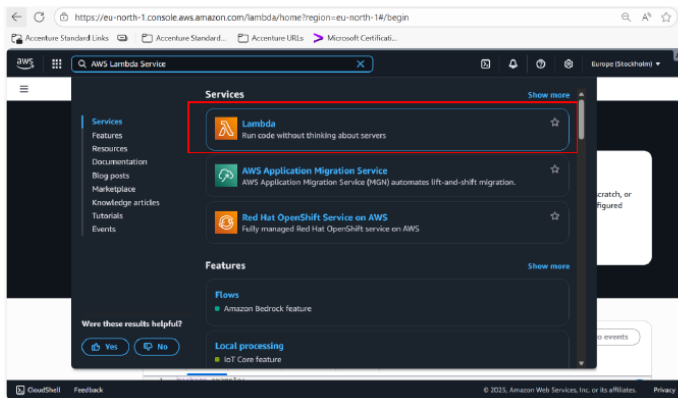
Console Walkthrough — Screenshots

5. Create the Lambda Function (Console)

1 Sign in to the AWS Management Console.

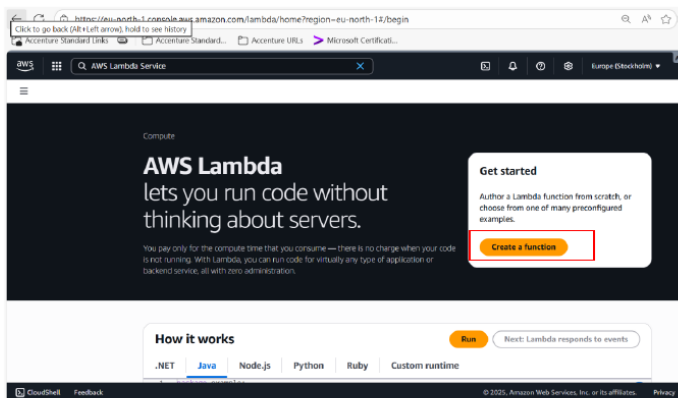
[Console Home](#) | [Console Home](#) | [eu-north-1](#)

Navigate to the **AWS Lambda** service.



The screenshot shows the AWS Management Console interface. The 'Services' list on the left sidebar has 'Lambda' highlighted with a red box. The main content area shows the 'Lambda' service card with the description 'Run code without thinking about servers'. Other services like 'AWS Application Migration Service' and 'Red Hat OpenShift Service on AWS' are also visible.

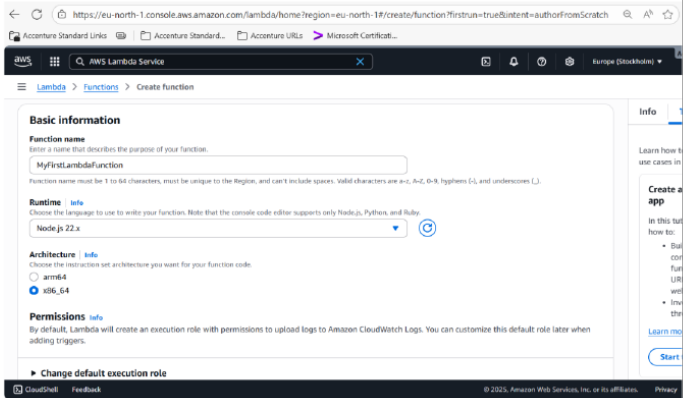
2 Click **Create function**.



The screenshot shows the 'Get started' page for AWS Lambda. The 'Create a function' button is highlighted with a red box. The page includes the text 'AWS Lambda lets you run code without thinking about servers.' and a 'How it works' section with tabs for '.NET', 'Java', 'Node.js', 'Python', 'Ruby', and 'Custom runtime'. The 'Run' button is also visible.

3

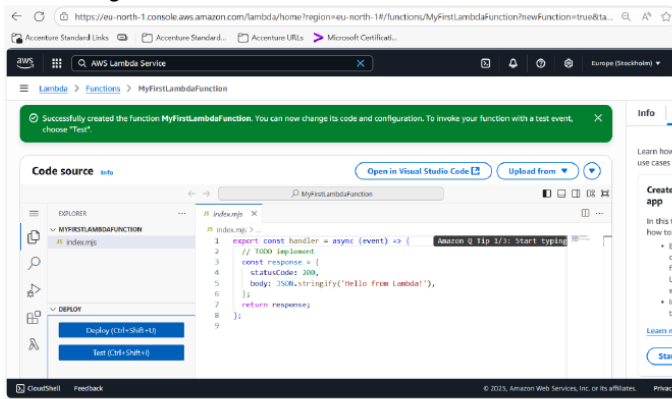
1. Choose **Author from scratch**.
2. **Basic Information:**
 1. **Function name:** eg., MyFirstLambdaFunction.
 2. **Runtime:** Select your desired programming language (e.g., **Python 3.12, Node.js 20**). This defines the execution environment.
 3. **Architecture:** Choose **x86_64** or **arm64** (arm64, powered by AWS Graviton2, often provides better performance and lower cost).
 4. **Permissions:** For the **Execution role**, choose **Create a new role with basic Lambda permissions**. This creates a role allowing the function to write logs to **Amazon CloudWatch**.



4

Configure the Runtime and Code

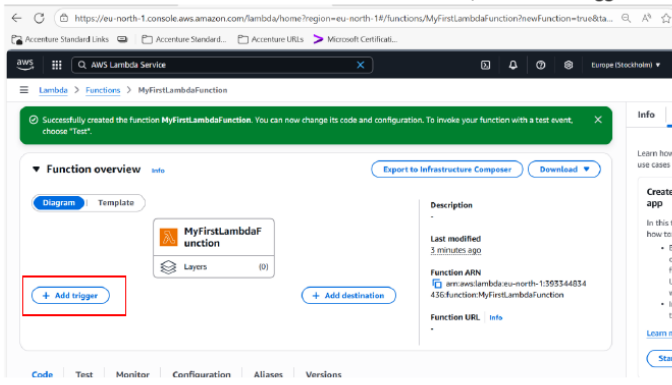
1. After creation, you'll be on the function's configuration page.
2. Use the **Code source** editor to write or paste your function code. The **handler** is the method where execution starts (e.g., `lambda_function.lambda_handler` for Python).
3. **Deployment:** For production, you typically upload a zipped deployment package or use a container image. For a simple exercise, the inline editor is sufficient.
4. **Basic Settings:** Configure memory (MB) and timeout settings under the **Configuration** tab

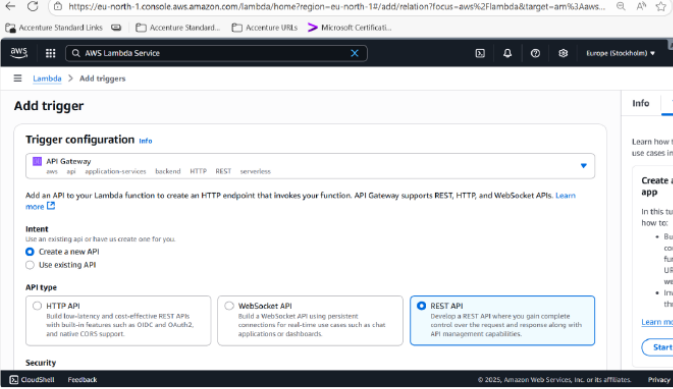
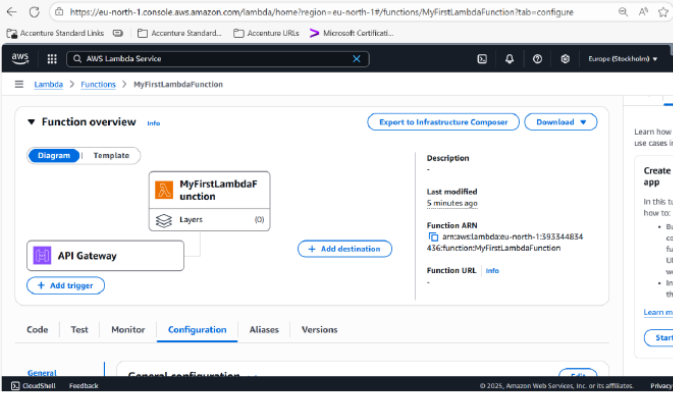


5

Add a Trigger

1. In the **Function overview** on the Lambda console, click **+ Add trigger**.



	1. Select the service that will trigger the function (e.g., API Gateway, S3, or EventBridge). 2. Configure the Trigger: Follow the on-screen prompts, which vary by service. For example: - For S3, select the target bucket and the event type (e.g., All object create events). - For API Gateway, choose to Create an API, select the API type (e.g., REST API), and security settings. 
6	Add 

43.10 Create Resources on AWS — Integrating Lambda with DynamoDB

The final activity of this chapter brings together a Lambda function and a **DynamoDB** table (DynamoDB itself is covered in full theoretical depth in the next chapter, alongside its own dedicated activity) to demonstrate a genuinely common serverless pattern: a function that reads and writes application data to a managed NoSQL database, with neither the function nor the database requiring any server to be provisioned or managed by the consumer at all — a fully serverless data pipeline, end to end.

43.10.1 Why the Lambda Function Needs Its Execution Role Updated

The execution role created in the previous activity only grants CloudWatch logging permissions — nothing about DynamoDB. Attempting to read or write a DynamoDB table from within the function’s code without first granting that permission would fail with an access-denied error at runtime, exactly the same least-privilege principle already seen governing EKS’s IAM roles: AWS never assumes a Lambda function needs broad access just because it exists, and every additional capability (here, DynamoDB access) has to be explicitly granted to the function’s own execution role.

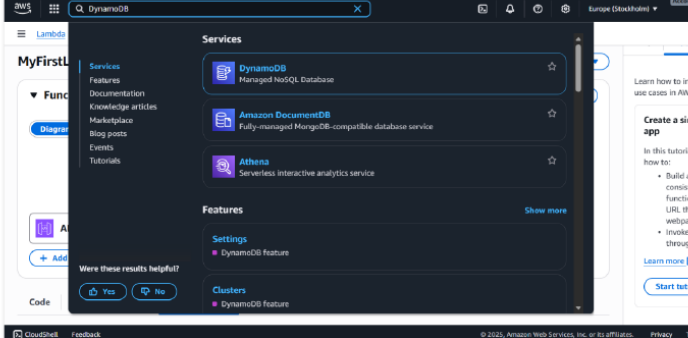
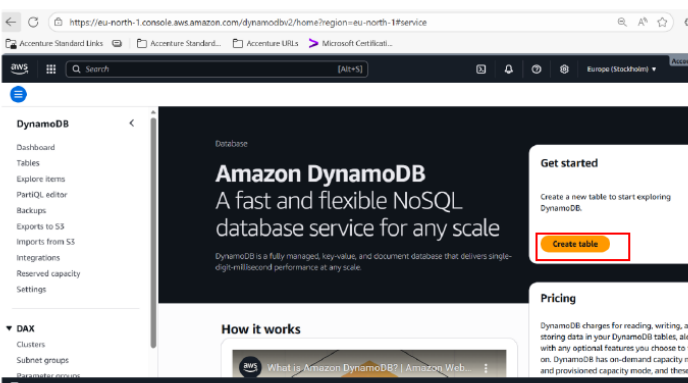
43.10.2 Activity: Create Resources on AWS (DynamoDB + Lambda)

Activity: Connect a Lambda Function to a DynamoDB Table

1. Navigate to the **DynamoDB** service and click **Create table**.
2. Provide a **Table name** (e.g. `MyServerlessToDoList`), a **Partition key** (e.g. `UserID`, string type) defining the primary identifier for items, and, optionally, a **Sort key** (e.g. `ItemID`, string type) for more complex lookups. Accept the default settings for the rest and click **Create table**.
3. Navigate back to the Lambda function created in the previous activity, open its **Configuration** tab, and select **Permissions** — this displays the function’s **Execution role** (e.g. `MyFirstLambdaFunction-role-...`).
4. Click the role name to open it directly in the **IAM console**, then, on its **Permissions** tab, click **Add permissions** → **Attach policies**, search for **AmazonDynamoDBFullAccess** (or a more narrowly-scoped custom policy, preferable for a genuine production deployment), and attach it.
5. Back in the Lambda function, update the function’s code to use the AWS SDK (e.g. **Boto3** for Python) to actually read from and write to the DynamoDB table.
6. Under **Configuration** → **Environment variables**, add a key-value pair (e.g. `TABLE_NAME = MyServerlessToDoList`) so the table’s name is supplied to the function’s code via configuration rather than hardcoded directly into it — the identical externalisation discipline from Chapter 29 (Spring’s `@Value`/properties-file injection) and Chapter 37 (Spring Boot’s `application.properties`) of the earlier volumes, applied here to a serverless function’s own configuration instead.
7. Use the **Test** tab to invoke the function with a sample event, then confirm both that an item was actually written to the DynamoDB table (visible directly in the DynamoDB console’s **Explore table items** view) and that the CloudWatch logs show a successful execution.

Console Walkthrough — Screenshots

6. Create Resources on AWS

1	Create a NoSQL Database (Amazon DynamoDB) Navigate to the DynamoDB service in the console. 
2	Click Create table. 

3

Table details:

Table name: Enter a name (e.g., MyServerlessToDoList).

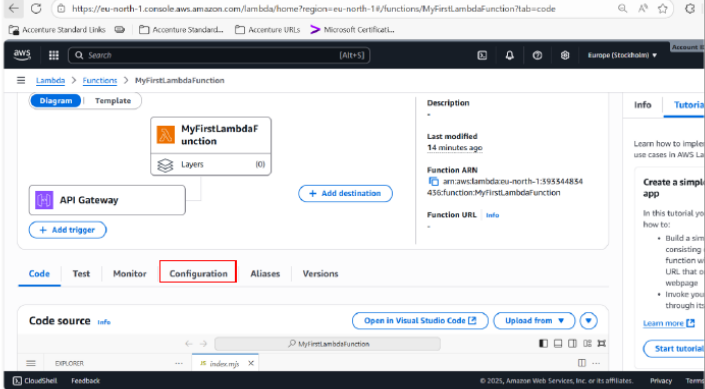
Partition key: Define the primary identifier for your items (e.g., UserID, string type).

Sort key (optional): Define an additional key for more complex lookups (e.g., ItemID, string type).

Use default settings for the rest and click **Create table**.

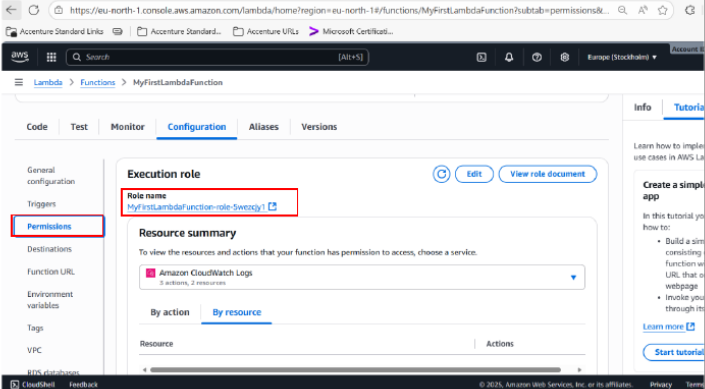
Update Lambda Permissions (IAM Role)

Your Lambda function needs permission to read/write to the new DynamoDB table. Navigate back to your **Lambda function** in the console.

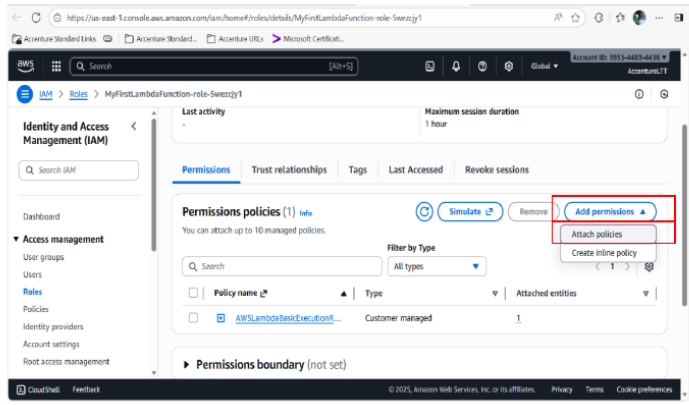


4

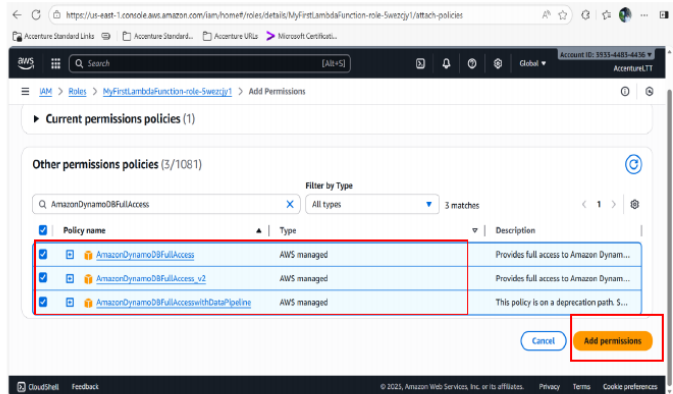
Go to the Configuration tab and select Permissions.



5 Click on the **Role name** to open the IAM Console.
On the Permissions tab of the role, click **Add permissions** → **Attach policies**.



5 Search for a managed policy like **AmazonDynamoDBFullAccess** (or a more restrictive one for production) and attach it.

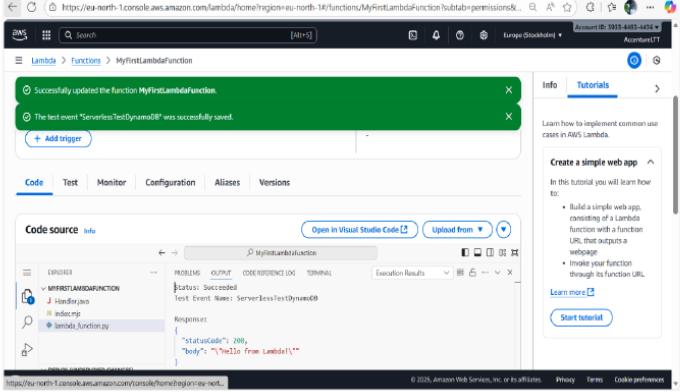


Policy name	Type	Description
AmazonDynamoDBFullAccess	AWS managed	Provides full access to Amazon Dynam...
AmazonDynamoDBFullAccess_v2	AWS managed	Provides full access to Amazon Dynam...
AmazonDynamoDBFullAccesswithDataPipeline	AWS managed	This policy is on a deprecation path. S...

6

Integrate and Test

- Update Lambda Code:** Modify your Lambda function code to use the AWS SDK (e.g., **Boto3** for Python) to interact with the DynamoDB table. Remember to pass the table name using an **Environment Variable** for best practice.
- Add Environment Variable:** In the Lambda configuration, go to **Environment variables** and add a new key-value pair: `TABLE_NAME = ServerlessToDoList`.
- Test:** Use the **Test** tab in the Lambda console to invoke your function and verify that an item is added to the DynamoDB table and that the CloudWatch logs show successful execution.



Viva Voce — Likely Questions & Answers

Q: What are the five essential characteristics of cloud computing according to NIST?

A: On-demand self-service, broad network access, resource pooling, rapid elasticity, and measured service.

Q: What is the fundamental economic shift cloud computing enables compared to on-premises infrastructure?

A: A shift from large upfront capital expenditure (purchasing physical hardware in advance based on a demand forecast) to ongoing, usage-based operating expenditure (paying only for the compute, storage, and networking actually consumed).

Q: What is the key structural difference between a virtual machine and a container?

A: A virtual machine includes its own complete guest operating system, fully isolated by a hypervisor from other VMs on the same physical host; a container shares its host's OS kernel with other containers on the same host, isolated instead by OS-level features, making containers far more lightweight and faster to start than VMs.

Q: What does "serverless" actually mean, given that a server is obviously still involved somewhere?

A: It means the cloud provider handles all provisioning, scaling, and patching of the underlying server completely invisibly to the consumer, whose only remaining responsibility is the application code itself – not that there is literally no server anywhere in the system.

Q: What is the difference between a hybrid cloud and a multi-cloud deployment model?

A: A hybrid cloud combines private and public cloud infrastructure, bound together to allow data and application portability between them; a multi-cloud deployment instead uses multiple different public cloud providers, typically to avoid dependency on a single vendor or to access provider-specific services.

Q: Why should the AWS root user be used as rarely as possible?

A: Because the root user has unrestricted access to every resource and setting in the account with no possibility of restriction; day-to-day work should instead be performed by an IAM user granted only the specific permissions that work actually requires, following the principle of least privilege.

Q: Why is SSH key-pair authentication preferred over password authentication for an EC2 instance?

A: A private key is effectively infeasible to derive from its public counterpart even with significant computing power, unlike a password which can potentially be guessed or brute-forced; the private key also never has to be transmitted over the network during authentication, removing the interception risk a transmitted password carries.

Q: What does a Security Group control, and at what level does it operate?

A: It controls inbound and outbound network traffic at the level of an individual EC2 instance, with rules defined by protocol, port range, and source (an IP range or another security group).

Q: What is the difference between the Fargate and EC2 launch types in Amazon ECS?

A: Fargate is a serverless launch type where AWS manages the underlying servers running

the containers entirely, with the consumer only specifying the task's own CPU/memory needs; the EC2 launch type instead schedules containers onto a fleet of EC2 instances that the consumer must provision and manage directly.

Q: Why does creating an EKS cluster require two separate IAM roles rather than one?

A: Because the EKS control plane and the worker nodes have genuinely different responsibilities and therefore need different permissions: the Cluster IAM Role lets the control plane manage AWS resources like EC2 instances and load balancers, while the Node IAM Role lets worker nodes register with the cluster and pull container images – keeping each role scoped only to what it specifically needs, per the principle of least privilege.

Q: What is the difference between a role's permissions policy and its trust policy?

A: A permissions policy defines what actions the role is allowed to perform; a trust policy defines who (which AWS service, account, or user) is allowed to assume the role in the first place. Both must be correctly configured before a role is usable.

Q: Why does EKS require a VPC with subnets across at least two Availability Zones?

A: Because Kubernetes control planes and node groups are designed to be resilient to the loss of any single data centre, so spreading the underlying infrastructure across multiple Availability Zones is a prerequisite for that resilience.

Q: Why does a Lambda function typically incur no charge while it is not actively handling a request, unlike an EC2 instance?

A: Because Lambda provisions an execution environment on demand only when a trigger actually fires, and bills only for the actual compute time consumed during that invocation; an EC2 instance, in contrast, accrues charges continuously for as long as it remains in the Running state, regardless of whether it is doing any useful work.

Q: What is a "cold start" in the context of AWS Lambda?

A: The additional latency incurred when a trigger fires and no existing execution environment is available to handle it, requiring AWS to provision a fresh one on the spot, as opposed to a "warm" invocation that reuses a recently-used execution environment.

Q: Why does a Lambda function need its execution role explicitly updated before it can read or write a DynamoDB table?

A: Because the default execution role created for a new function only grants permission to write logs to CloudWatch; any further capability, such as DynamoDB access, must be explicitly attached to the role, following the same least-privilege principle that governs IAM roles throughout AWS.

Chapter 44

Cloud Storage & Networking Basics

44.1 Introduction to Storage — Object, Relational, and NoSQL

Every activity in the previous chapter created *compute* — something that runs code. This chapter turns to *storage* — where the data that code reads and writes actually lives — and to the *networking* that keeps cloud resources both reachable and appropriately isolated from one another. AWS (and cloud computing generally) offers several genuinely different storage models, each suited to a different shape of data and a different access pattern, and this chapter’s three storage activities (S3, RDS, DynamoDB) are deliberately chosen to walk through exactly these three models.

Storage Model	Shape of Data	AWS Service (this chapter)
Object Storage	Whole, opaque files (documents, images, backups) addressed by a unique key, with no query language of its own	Amazon S3
Relational	Structured rows in tables with a fixed schema, related to one another via foreign keys, queried with SQL	Amazon RDS
NoSQL	Flexible, often schema-less data — key-value pairs, documents, or wide columns — optimised for scale and simple access patterns rather than complex relational queries	Amazon DynamoDB

None of these three models is a strict upgrade over the others — each is a genuinely different trade-off, and choosing among them is a real architectural decision, not a matter of picking whichever is newest. Object storage excels at storing and serving large, whole files cheaply and durably, but is a poor fit for anything that needs to be queried by its internal structure. A relational database excels at enforcing structure and relationships (exactly as the previous volumes’ JPA/Hibernate chapters relied on) and answering complex, multi-table queries, but that same rigid structure and the overhead of enforcing relational integrity make it harder to scale horizontally across many machines than the simpler access patterns NoSQL databases are built around. A NoSQL database trades away rich relational querying for the ability to scale nearly linearly across many machines and to tolerate a much more flexible, evolving data shape.

44.2 Introduction to Networking — VPC and Subnet, at a Glance

Every resource created in the activities of this chapter and the last — an EC2 instance, an RDS database, an EKS cluster — exists somewhere on a **network**, and by default, that network is a **VPC (Virtual Private Cloud)**: a logically isolated section of the AWS cloud where a consumer can launch resources into a network they themselves define, with their own IP address range, their own **subnets** (subdivisions of that address range), and their own routing rules — entirely separate, by default, from every other AWS customer’s own VPC, even though the underlying physical infrastructure is shared. This chapter’s own VPC activity, at the end of this chapter, covers this in full theoretical and hands-on depth; a brief preview here is worth having in mind while reading the S3, RDS, and DynamoDB sections that follow, since RDS in particular is deployed *inside* a VPC and a specific subnet, exactly like an EC2 instance would be.

44.3 Amazon S3 — Simple Storage Service

Amazon S3 is AWS’s object storage service, and it is worth learning its vocabulary precisely, since “bucket” and “object” mean something specific and are not interchangeable with “folder” and “file” in the traditional filesystem sense S3 deliberately does not use.

S3 Concept	Meaning
Bucket	A top-level container for objects, with a name that must be globally unique across <i>all</i> of AWS (not just within one account), and scoped to a specific AWS Region
Object	An individual stored item — the actual file content, together with a unique key (its full path-like identifier within the bucket) and metadata
Object Ownership	Controls whether objects uploaded by other AWS accounts remain owned by the uploader or are automatically owned by the bucket owner — ACLs disabled (the AWS-recommended default) means the bucket owner owns every object in the bucket, and access is controlled purely through bucket policies rather than legacy per-object Access Control Lists
Block Public Access	A set of four independent settings that, together, prevent a bucket (or its objects) from ever being made publicly accessible via ACLs or bucket policies, regardless of any other configuration that might otherwise permit it — AWS enables this by default for every new bucket, specifically because accidentally-public S3 buckets have historically been one of the most common cloud data-exposure incidents
Versioning	When enabled, S3 preserves every previous version of an object each time it is overwritten or deleted, rather than the new write simply replacing the old data irrecoverably — protecting against both accidental overwrites and accidental deletions
Default Encryption	Server-side encryption automatically applied to every new object as it is stored, using either Amazon S3-managed keys (SSE-S3) or AWS Key Management Service keys (SSE-KMS), so that data at rest is encrypted without every uploading application needing to encrypt it independently beforehand

Why Block Public Access Is On by Default

Section 43.1 named “measured service” and “self-service” as characteristics that make cloud resources fast and convenient to provision — but that same ease of provisioning is exactly why AWS defaults every new S3 bucket to fully private, with all four Block Public Access settings enabled: a bucket that can be created in seconds should not also be a single misconfigured setting away from exposing its contents to the entire internet. Making a bucket (or specific objects within it) intentionally public requires deliberately turning these protections off, which is a conscious, auditable action rather than an accidental default.

44.3.1 Activity: Create and Configure an S3 Bucket

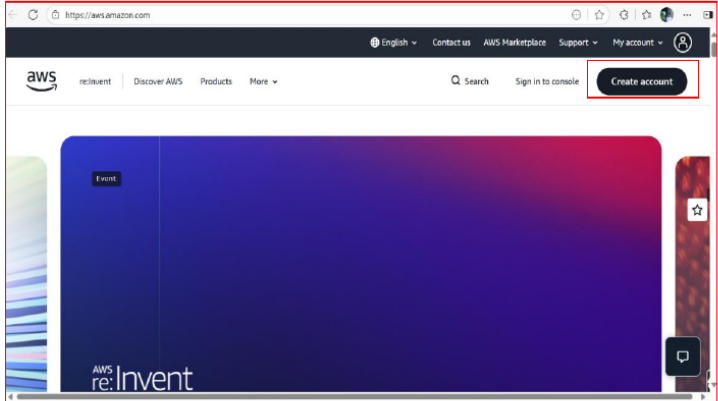
Activity: Create, Configure, and Upload to an S3 Bucket

1. Sign in to the AWS Console and open the **S3** service; select **Buckets** in the left navigation.
2. Click **Create bucket**.
3. Enter a globally unique **Bucket name** and choose the **AWS Region** closest to the bucket’s expected users or the other resources it will interact with.
4. Configure **Object Ownership** (leave at **ACLs disabled**, the recommended default), **Block Public Access settings** (leave all four sub-settings enabled unless the bucket genuinely needs to serve public content), **Bucket Versioning** (Enable if past versions of objects should be recoverable), and **Default encryption** (Server-side encryption with Amazon S3 managed keys, SSE-S3, is enabled by default).
5. Review the settings and click **Create bucket** — a success message confirms the bucket now exists.
6. Click into the newly created bucket, then click **Upload** → **Add files** (or **Add folder**), select one or more files from the local computer, and click **Upload**.
7. A success message, together with a summary of files uploaded and their total size, confirms the objects are now stored in the bucket.

Console Walkthrough — Screenshots

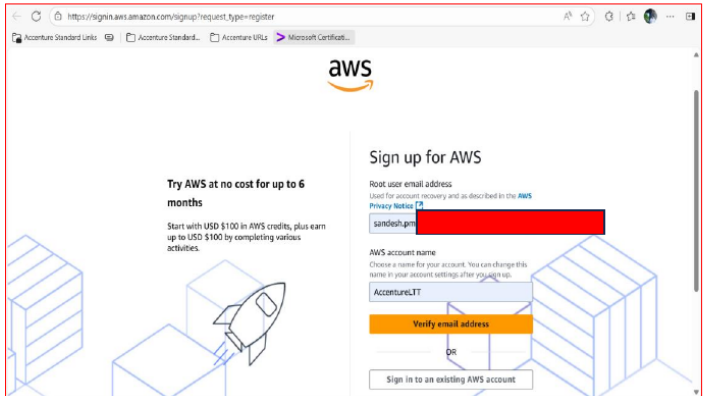
1. Creating AWS account

- 1 Sign up for AWS Account Using the Link below
[Cloud Computing Services - Amazon Web Services \(AWS\)](#)
Click on Create Account



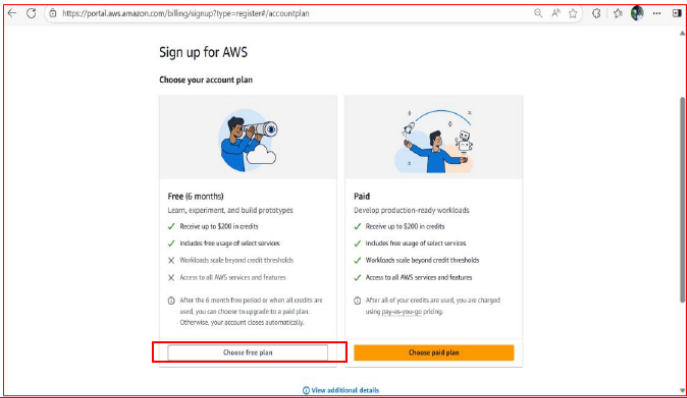
The screenshot shows the AWS homepage at <https://aws.amazon.com>. The navigation bar includes links for English, Contact us, AWS Marketplace, Support, and My account. The main navigation bar has links for re:invent, Discover AWS, Products, and More. The 'Create account' button is highlighted with a red box in the top right corner.

- 2 Enter Email ID and AWS Account Name



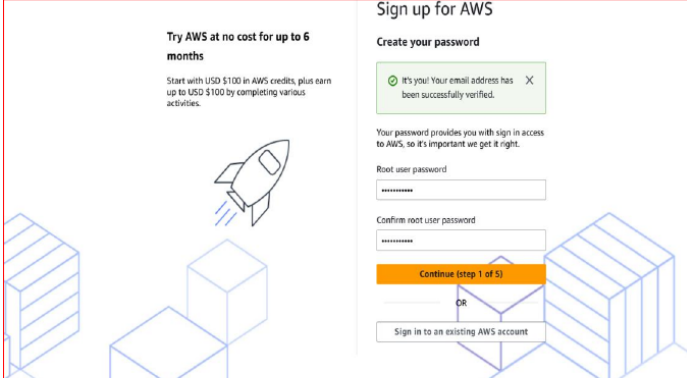
The screenshot shows the AWS sign-up page at <https://signin.aws.amazon.com/signup/request?step=register>. The page includes a promotional banner for 'Try AWS at no cost for up to 6 months'. The 'Sign up for AWS' section contains fields for 'Root user email address' (with a redacted email address) and 'AWS account name' (with the value 'AccentureLT'). The 'Verify email address' button is highlighted with a red box.

3 Choose free plan



The screenshot shows the 'Sign up for AWS' page with the 'Choose your account plan' section. It compares two plans: 'Free (6 months)' and 'Paid'. The 'Free' plan includes up to \$200 in credits, free usage of select services, and access to all AWS services and features. The 'Paid' plan includes up to \$200 in credits, free usage of select services, and access to all AWS services and features. The 'Free' plan is highlighted with a red box around the 'Choose free plan' button.

4 Fill the details as it involves 5 steps for Sign up for AWS
Step 1: Enter Password



The screenshot shows the 'Sign up for AWS' page with the 'Create your password' section. It includes a message: 'It's you! Your email address has been successfully verified.' Below this, there are fields for 'Root user password' and 'Confirm root user password'. A 'Continue (step 1 of 5)' button is highlighted in orange. There is also a 'Sign in to an existing AWS account' link.

Step 2: Fill the Contact Information

Earn additional AWS credits

Complete various activities to earn up to an additional USD 1500 in credits, such as creating your first AWS budget to monitor costs.



Sign up for AWS

Contact Information

How do you plan to use AWS?

☐ Business: For your work, school, or organization

☒ Personal: For your own projects

Who should we contact about this account?

Full Name

Country Code Phone Number

Country or Region

Address line 1

Address line 2

City

State, Province, or Region

Postal Code

Nearby AWS Region selection - optional

Enabling regional availability for your account enables you to provision in specific regions.

☒ Enable Asia Pacific (Hyderabad) Region

Customers with an Indian contact address are served by Amazon Web Services India Private Limited, the local seller for AWS services in India.

☒ I have read and agree to the terms of the [AWS Customer Agreement](#)

[Sign up and continue \(Step 3 of 5\)](#)

Step 3: Provide the Card Details where INR 2 Rupees will be deducted from the Credit Card and Credit Back.

Why this request?

Use an Indian payment method (credit or debit card) to pay for your AWS account. You must provide your card details to activate your account and get started with AWS.

For the fastest activation, please provide your card details and the appropriate card type.



Billing Information

Billing method

Cardholder name

Card number

Expiration date

Security code

Cardholder name

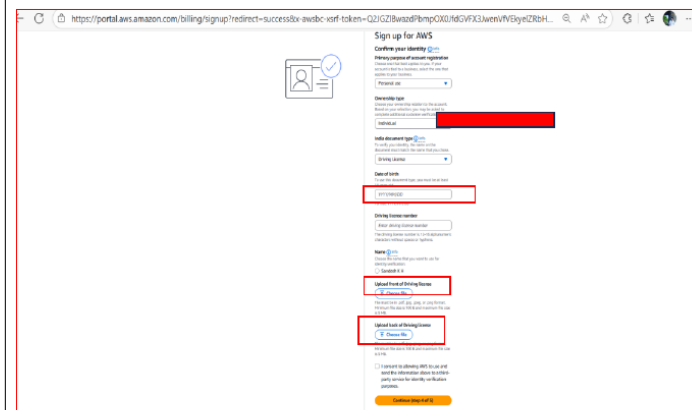
☒ I have read and agree to the terms of the [AWS Customer Agreement](#)

[Sign up and continue \(Step 4 of 5\)](#)

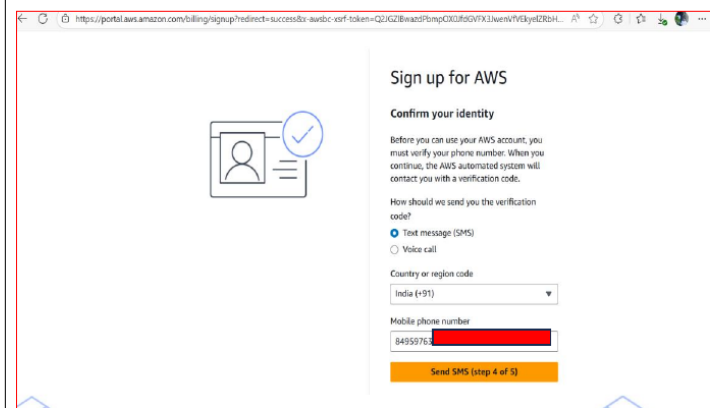
Step 4: Fill the details for Identity, you can Upload EPIC Copy of

1. Pan Card
2. Driving License
3. Voter ID

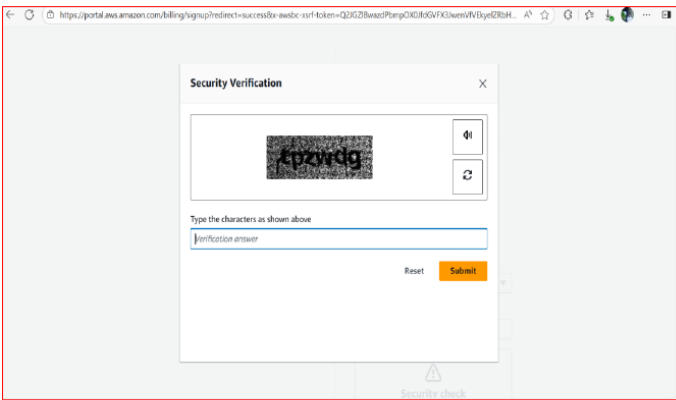
Upload Respective proof of Front Side and Back Side



Step 4: Confirm your Identity by providing your number and Verification Code Sent to Register number



5 Enter Verification



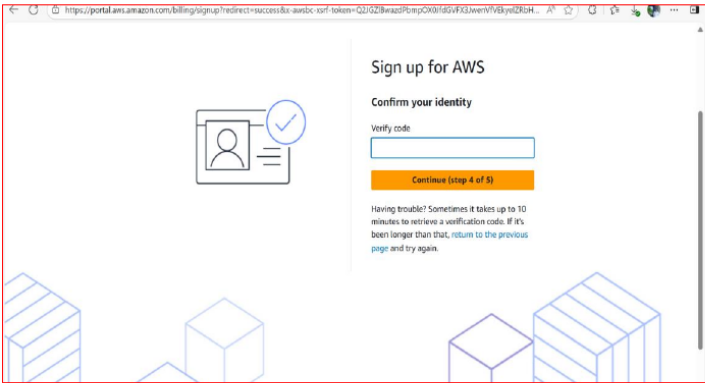
Security Verification

Type the characters as shown above

Verification answer

Reset Submit

Enter the text or voice verification code that you have received



Sign up for AWS

Confirm your identity

Verify code

Continue (step 4 of 5)

Having trouble? Sometimes it takes up to 10 minutes to retrieve a verification code. If it's been longer than that, [return to the previous page](#) and try again.

44.4 Amazon RDS — Relational Database Service

Every JPA/Hibernate demo across the earlier volumes of this course assumed a relational database (MySQL) was already running somewhere, reachable at some JDBC URL — **Amazon RDS** is the AWS-managed service that actually stands that database up, and, critically, takes over the operational burden of running it: automated backups, patching, and (if configured) high availability, none of which a hand-installed MySQL server on a plain EC2 instance would get for free.

RDS Concept	Meaning
DB Instance	One running managed database server — the RDS equivalent of an EC2 instance, but pre-configured specifically to run a chosen relational database engine
Engine	The actual relational database software running inside the instance — MySQL, PostgreSQL, MariaDB, Oracle, SQL Server, or IBM Db2 are all supported; this chapter's activity uses MySQL
Standard vs Easy Create	Standard create exposes every configuration option (availability, security, backups, maintenance) explicitly; Easy create instead applies AWS's own recommended best-practice configuration automatically, with fewer decisions to make up front
Template	A pre-set combination of settings matching a use case — Production (defaults tuned for high availability and consistent performance), Dev/Test (intended for development outside of a production environment, used in this chapter's activity), or Sandbox (for trying out new applications or gaining hands-on experience)
Single-AZ vs Multi-AZ Deployment	A Single-AZ deployment creates one DB instance with no standby — lower cost, no redundancy, appropriate for this chapter's non-production activity. A Multi-AZ deployment additionally creates a synchronised standby instance in a separate Availability Zone, providing automatic failover if the primary instance becomes unavailable — the appropriate choice for a genuine production workload
Instance Class	The compute and memory capacity of the DB instance — this chapter's activity uses the Burstable class <code>db.t3.micro</code> , Free-tier eligible and appropriate for light workloads with variable, intermittent demand
VPC Security Group	The RDS-specific equivalent of an EC2 security group (Section 43.4) — a virtual firewall controlling which sources are permitted to connect to the database instance's network port
Automated Backups	RDS can automatically take a daily snapshot of the database and retain transaction logs, together enabling point-in-time recovery to any second within the configured Backup retention period — an operational safety net a hand-managed database on a plain EC2 instance would not have unless someone specifically built and maintained it

Why RDS Is Still Placed Inside a VPC, Just Like EC2

An RDS instance is not a magically “outside the network” managed service — it is provisioned into a specific VPC and subnet, exactly as an EC2 instance is, and reachability to it is governed by exactly the same security-group mechanism. The “managed” part of “managed relational database service” refers to AWS handling the database engine’s installation, patching, and backups — not to the database somehow existing outside the networking model the rest of this chapter covers.

44.4.1 Activity: Design and Implement a Sample Schema on Amazon RDS

Activity: Provision a MySQL RDS Instance and Build a Schema

1. Sign in to the AWS Console and open **Aurora and RDS**, then click **Create a database**.
2. Choose **Standard create**; under **Engine options**, select **MySQL**.
3. Under **Templates**, select **Dev/Test**.
4. Under **Availability and durability**, select **Single-AZ DB instance deployment (1 instance)**.
5. Under **Settings**, give the instance a **DB instance identifier** (e.g. `sampledb`); under **Credentials Settings**, choose **Self managed** and set a strong **Master password**.
6. Under **Instance Configuration**, choose **Burstable classes (includes t classes)** and select `db.t3.micro`.
7. Under **Storage**, choose **General Purpose SSD (gp3)** with **20 GiB** of allocated storage.
8. Under **Connectivity**, choose **Don’t connect to an EC2 compute resource** and select the **Default VPC**.
9. Under **VPC security group (firewall)**, choose **Create new** and name it (e.g. `rds-sample-sg`).
10. Under **Monitoring**, choose **Database Insights – Standard**.
11. Under **Additional configuration**, give the **Initial database name** (e.g. `school`), and under **Backup**, enable **automated backup** with a **Backup retention period** (e.g. 5 days).
12. Click **Create database**. AWS provisions the instance, transitioning from **Creating** to **Available** after a few minutes.
13. Restrict network access: from the instance’s **Connectivity & security** tab, open its **VPC security group**, and under **Inbound rules** → **Edit inbound rules** → **Add rule**, specify **Type** MySQL/Aurora, **Port** 3306, and **Source** **My IP** — ensuring only the current machine can reach the database.
14. Copy the instance’s **Endpoint** (e.g. `sampledb.cdcmacgsea9b.ap-south-1.rds.amazonaws.com`) from the **Connectivity & security** tab.
15. Connect from a local command line: `mysql -h sampledb.cdcmacgsea9b.ap-south-1.rds.amazonaws.com -u admin -p`, then enter the master password when prompted.

Console Walkthrough — Screenshots

2. Creating EC2 Instance on AWS

1

Sign in to AWS Management Console
<https://aws.amazon.com/console/>

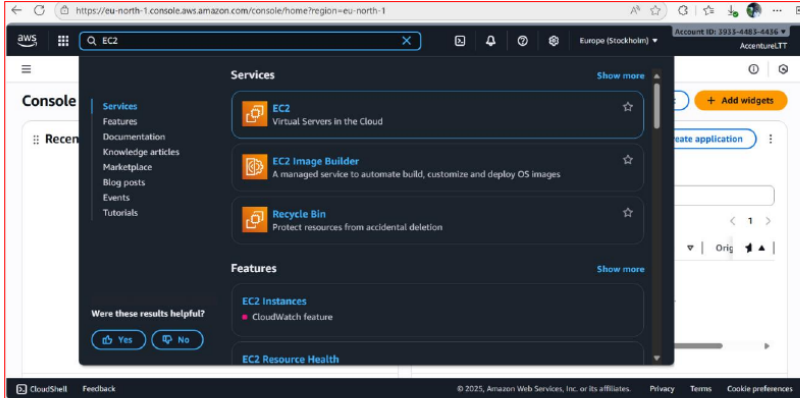
- Click “Sign in to the Console.”
- Choose:
 - Root user-use your email and password

2

AWS Management Console Dashboard.

3 Open EC2 Dashboard

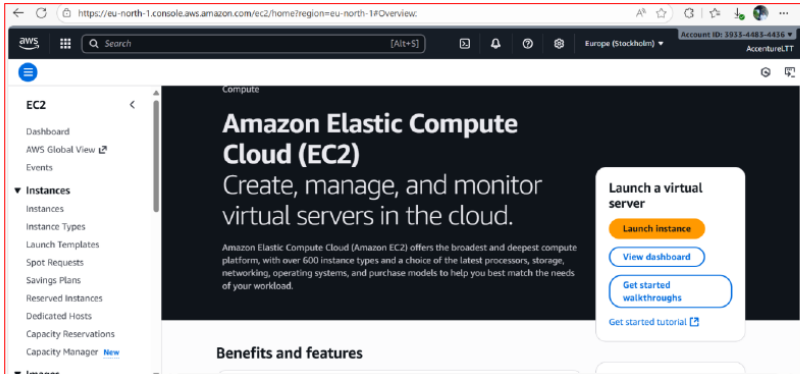
In the search bar at the top, type “EC2”.
Click EC2 under “Services.”



The screenshot shows the AWS Console search results for "EC2". The search bar at the top contains "EC2". Below the search bar, the "Services" section is expanded, showing "EC2" as the first result. The "EC2" service is described as "Virtual Servers in the Cloud". Other services shown include "EC2 Image Builder" and "Recycle Bin". The "Features" section shows "EC2 Instances" and "EC2 Resource Health".

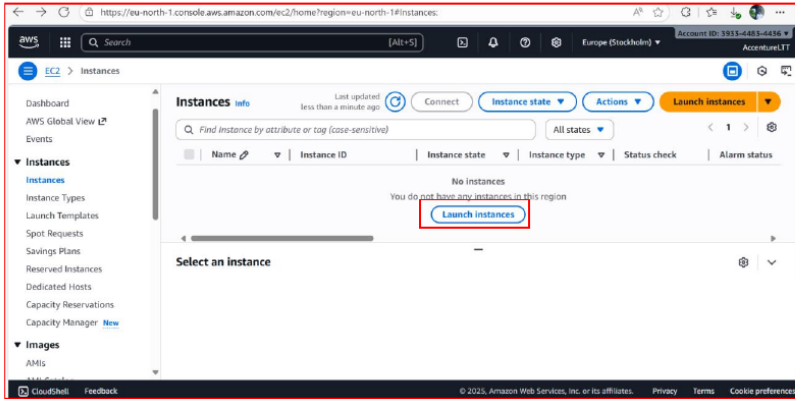
You'll see the **EC2 Dashboard**, showing:

- Instances
- Volumes
- Security Groups
- Key Pairs, etc.

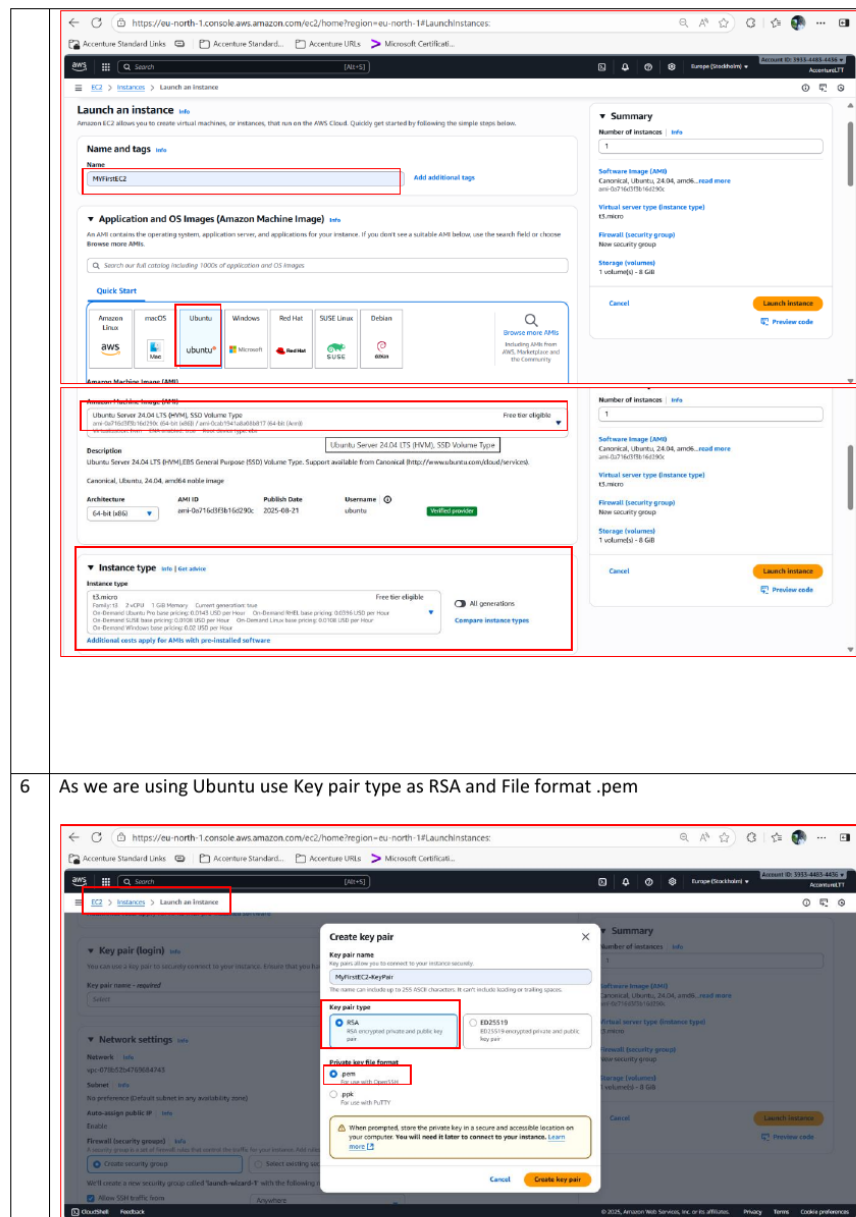


The screenshot shows the Amazon Elastic Compute Cloud (EC2) dashboard. The left sidebar contains the "EC2" menu with options like "Dashboard", "AWS Global View", "Events", "Instances", "Instance Types", "Launch Templates", "Spot Requests", "Savings Plans", "Reserved Instances", "Dedicated Hosts", "Capacity Reservations", and "Capacity Manager". The main content area displays the "Amazon Elastic Compute Cloud (EC2)" title, a description "Create, manage, and monitor virtual servers in the cloud.", and a "Launch a virtual server" button. Below this, there are buttons for "Launch Instance", "View dashboard", "Get started walkthroughs", and "Get started tutorial".

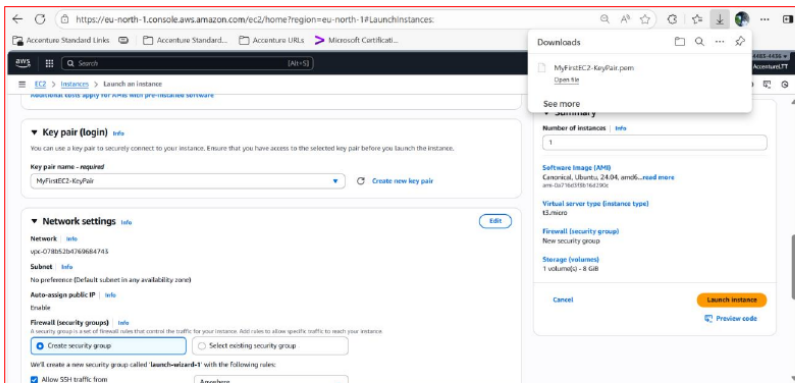
4 Launch a New EC2 Instance
Click “Launch instance.”



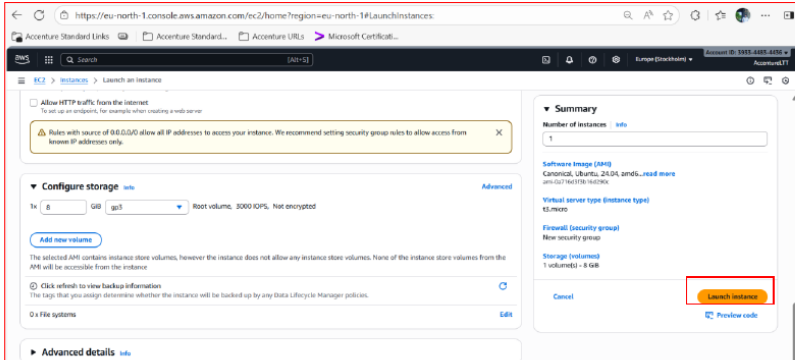
5 Fill the details:
Name and tags → e.g., MyFirstEC2
Application and OS Image (AMI) → Ubuntu. (Free tier eligible)
Instance type → Choose t3.micro (free tier eligible)

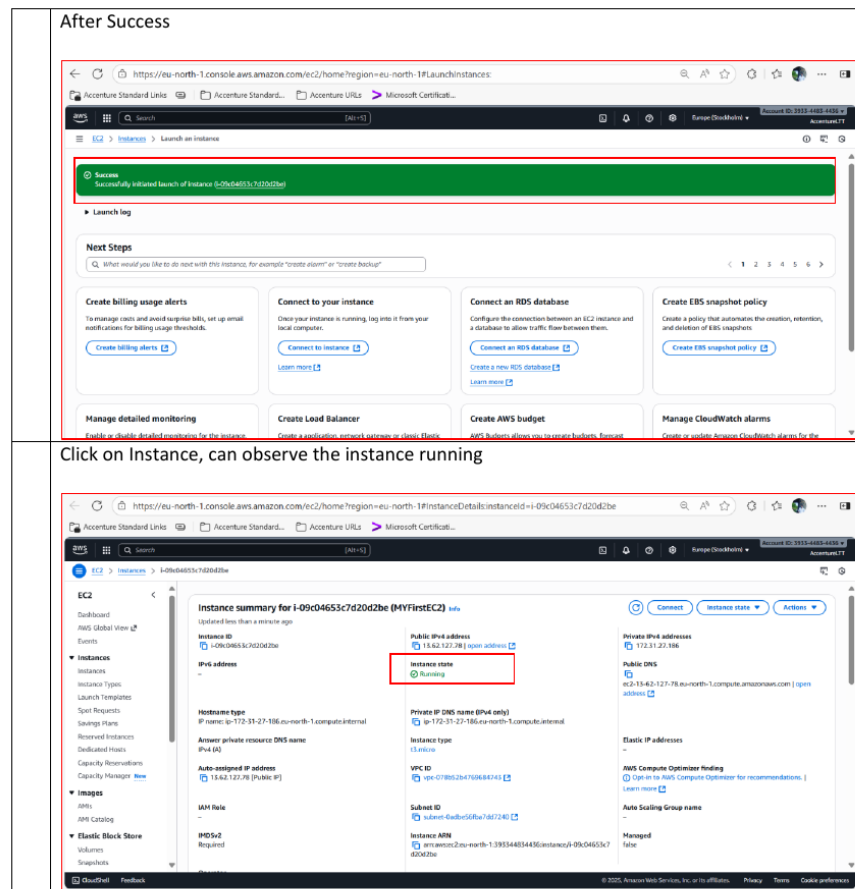


.pem File will be downloaded once we key pair is created

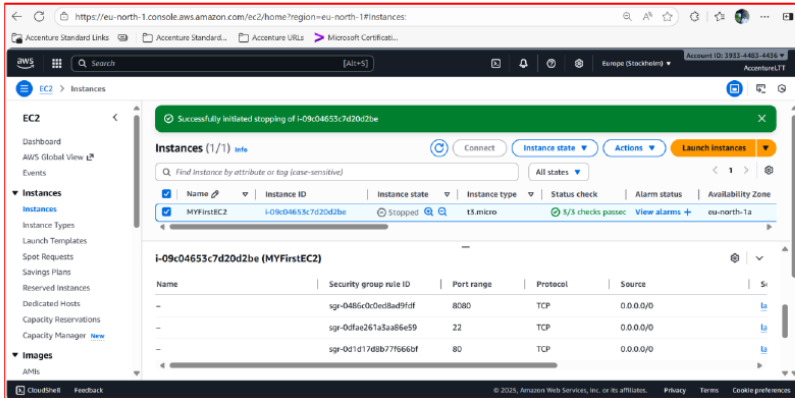


7 Launch the instance



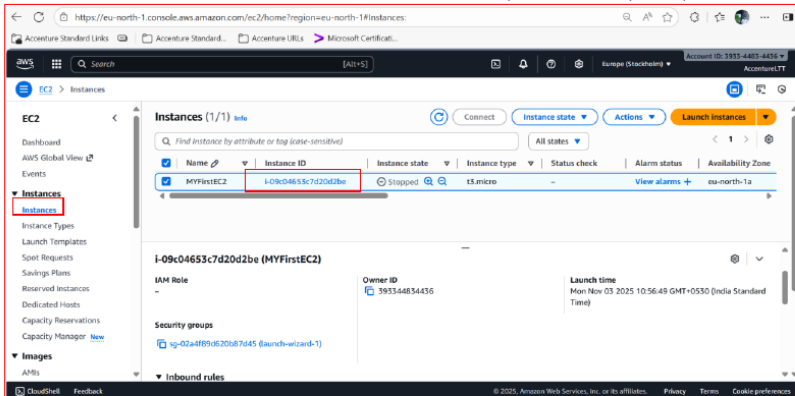


By Default, it should have all Inbound rules as shown below



Name	Security group rule ID	Port range	Protocol	Source	Source type
-	sg-0485c0c0c0ad9ff0f	8080	TCP	0.0.0.0/0	-
-	sg-0d4a261a3a86e59	22	TCP	0.0.0.0/0	-
-	sg-0d1d17d8b77f566df	80	TCP	0.0.0.0/0	-

8 If it is not available, you can add Inbound rules,
Go to EC2 Dashboard→Instance→Click on Instance→Security Tab→Security Groups



Instances (1/1) info

Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone
MYFirstEC2	i-09c04653c7d20d2be	Stopped	t3.micro	✓ 3/3 checks passed	View alarms	eu-north-1a

I-09c04653c7d20d2be (MYFirstEC2)

IAM Role: -

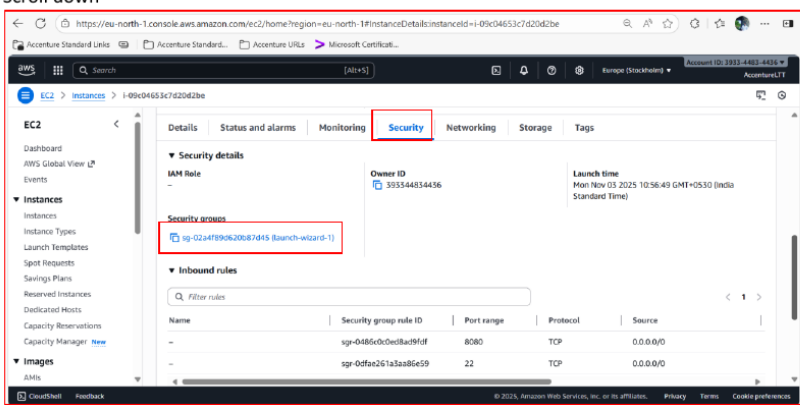
Owner ID: 395344834435

Launch time: Mon Nov 03 2025 10:56:48 GMT+0530 (India Standard Time)

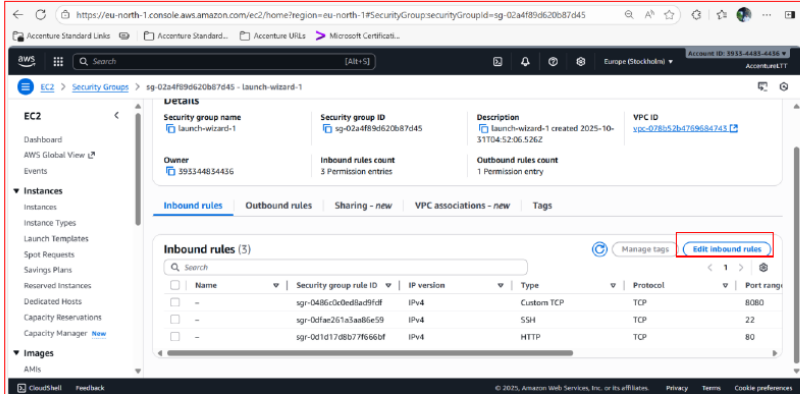
Security groups: sg-02a4f895e20d87d45 (launch-wizard-1)

Inbound rules

Scroll down



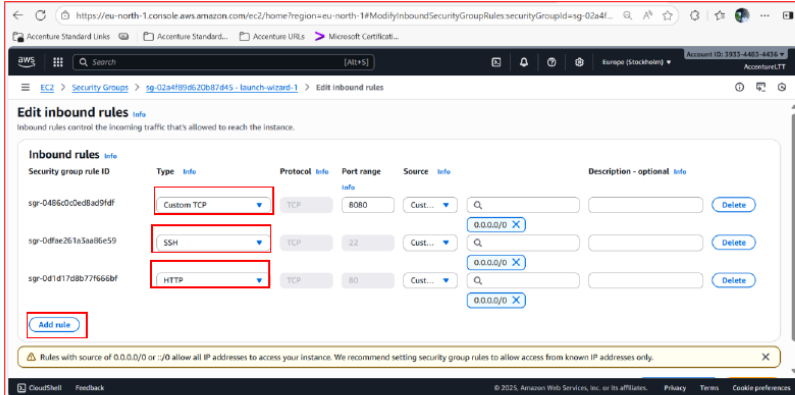
9 Once you click on Security Groups Launch wizard, you will find an option to Edit Inbound rules



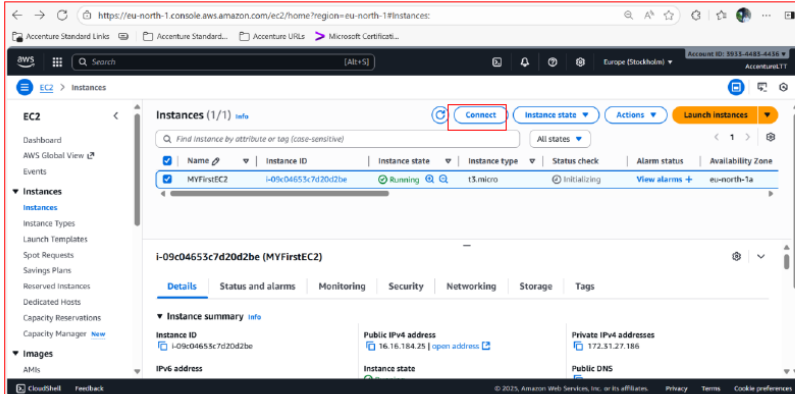
Name	Security group rule ID	Port range	Protocol	Source
-	sg-0486c0e0d8a9f9df	8080	TCP	0.0.0.0/0
-	sg-0dface261a3aa86e59	22	TCP	0.0.0.0/0

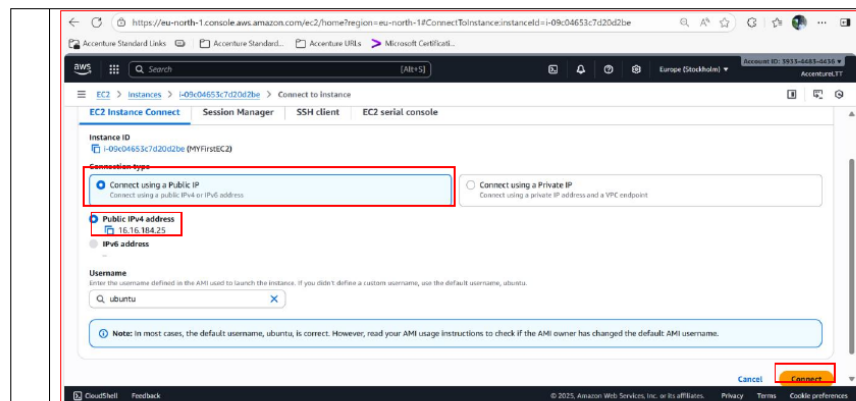
Name	Security group rule ID	IP version	Type	Protocol	Port range
-	sg-0486c0e0d8a9f9df	IPv4	Custom TCP	TCP	8080
-	sg-0dface261a3aa86e59	IPv4	SSH	TCP	22
-	sg-0d101708b07766b0f	IPv4	HTTP	TCP	80

10 Click on Add rule to add respective rules as per requirements.
By default, you can add Custom TCP, SSH and HTTP

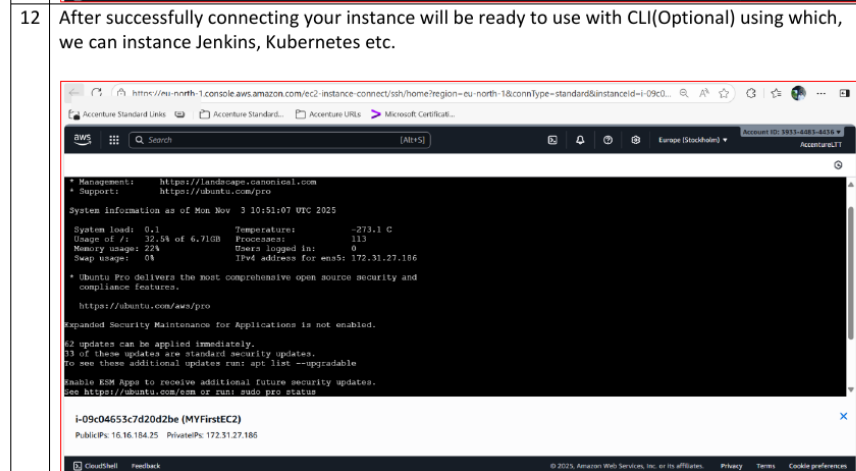


11 You can connect your instance on CLI in AWS account by clicking on Connect





12 After successfully connecting your instance will be ready to use with CLI(Optional) using which, we can instance Jenkins, Kubernetes etc.



```
+ Management: https://landscape.canonical.com
+ Support: https://ubuntu.com/pro

System information as of Mon Nov 3 10:51:07 UTC 2025
System load: 0.1      Temperature: 275.1 C
Usage of /: 32.0k of 6.71GB      Free space: 113
Memory usage: 22k      Swap usage: 0k
IP address for ens5: 172.31.27.186

+ Ubuntu Pro delivers the most comprehensive open source security and
  compliance features.
  https://ubuntu.com/awx/pro

Expanded Security Maintenance for Applications is not enabled.
2 updates can be applied immediately.
11 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable

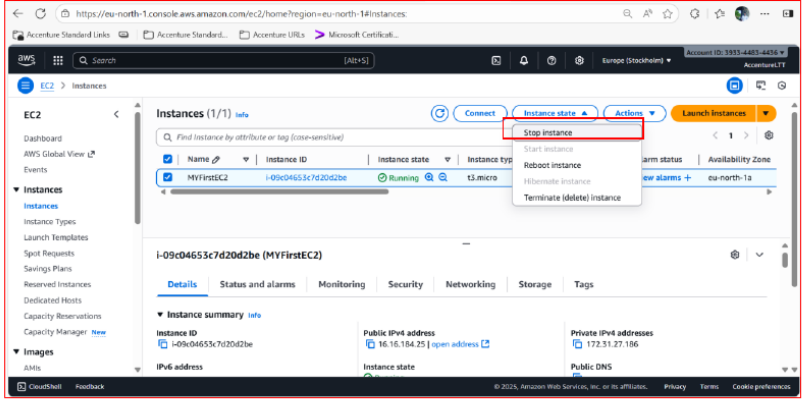
Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

i-09c04653c7d20d2be (MYFirstEC2)
PublicIPs: 16.16.184.25 PrivateIPs: 172.31.27.186
```

13 This is how we can create EC2 on AWS.

Note: - If you don't want any charges to be applied for your AWS account, make sure that you stop or terminate your instance when not in use

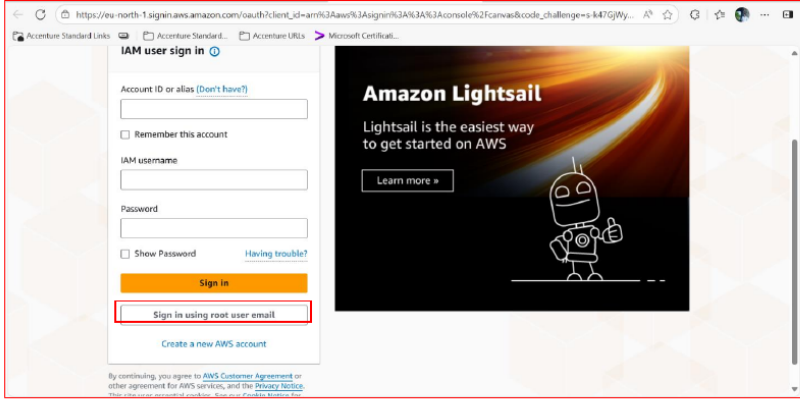
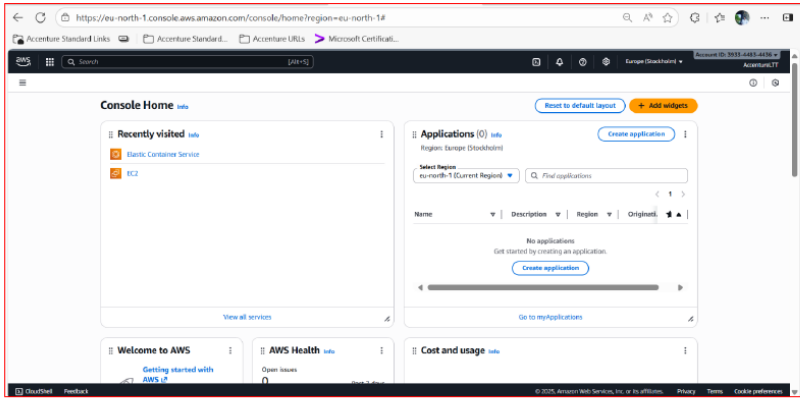
Click on Instance State → Stop Instance (In Future if you need) or Terminate Instance if not be used in Future



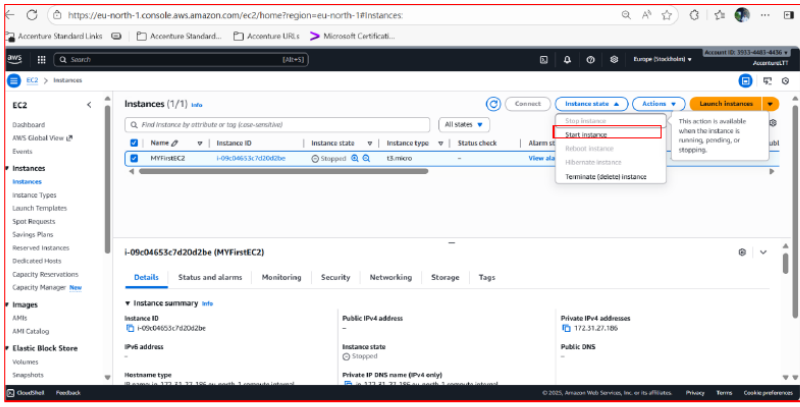
The screenshot shows the AWS Management Console for the 'eu-north-1' region. The 'Instances' page is active, displaying a table with one instance: 'MYFirstEC2' (Instance ID: i-09c04653c7d20d2be), which is in the 'Running' state. The 'Instance state' dropdown menu is open, showing options: 'Start instance', 'Stop instance', 'Reboot instance', 'Hibernate instance', and 'Terminate (delete) instance'. The 'Stop instance' option is highlighted. Below the table, the details for 'i-09c04653c7d20d2be (MYFirstEC2)' are shown, including the 'Instance summary' with fields for Instance ID, Public IPv4 address, Private IPv4 addresses, Instance state, and Public DNS.

Thank You!

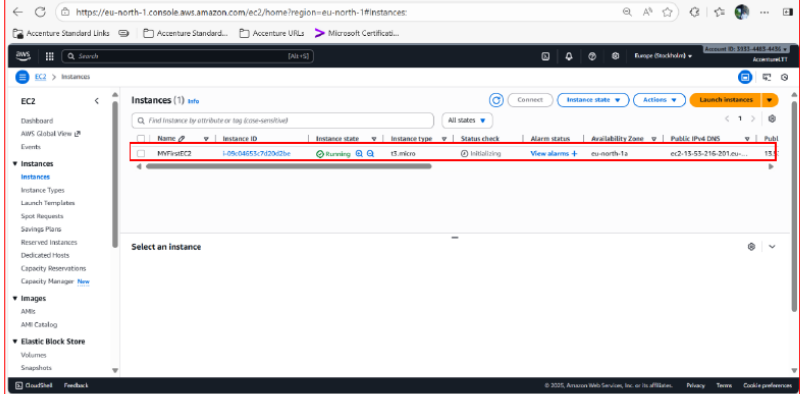
3. Create Amazon ECS (Elastic Container Service)

- 1 Use the link below to Sign-In
<https://console.aws.amazon.com/>

- 2 After Successful Sign In you will be launched to the AWS Dashboard


3 Before creating the ECS, launch the EC2 Instance created in previous steps

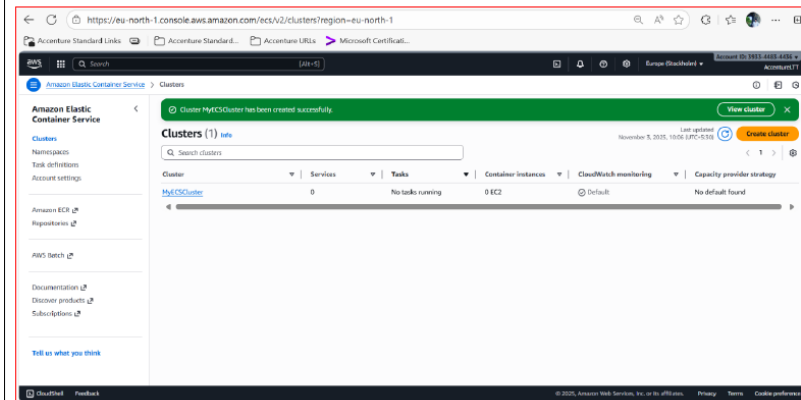


Make Sure that the Instance is running



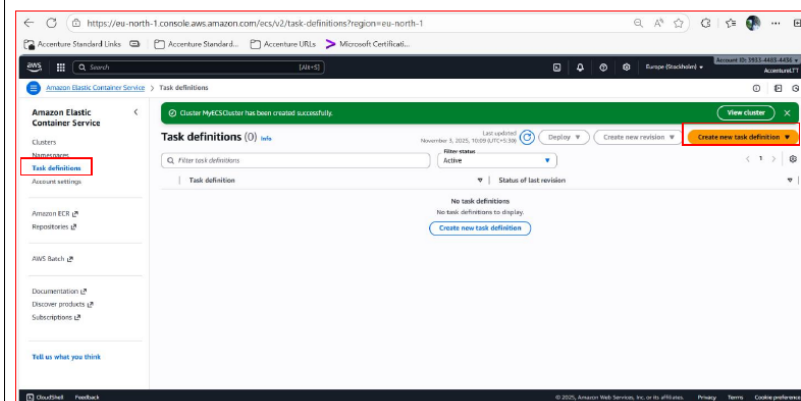
6 Wait for some times don't refresh it may fail, wait until it gets created successfully.

You now have an empty ECS cluster.



7 Next Step: Create a Task Definition

A task definition describes what container(s) to run.



Once connected, the schema is built with ordinary SQL, exactly as in the previous volumes’ JPA/Hibernate chapters — the only difference is that the server these statements run against is now a fully managed RDS instance rather than a locally-installed database:

```
1 CREATE DATABASE school;
2 USE school;
3
4 CREATE TABLE Students (
5     student_id INT AUTO_INCREMENT PRIMARY KEY,
6     name VARCHAR(100),
7     email VARCHAR(100) UNIQUE
8 );
9
10 CREATE TABLE Courses (
11     course_id INT AUTO_INCREMENT PRIMARY KEY,
12     title VARCHAR(100),
13     credits INT
14 );
15
16 CREATE TABLE Enrollments (
17     enrollment_id INT AUTO_INCREMENT PRIMARY KEY,
18     student_id INT,
19     course_id INT,
20     grade CHAR(1),
21     FOREIGN KEY (student_id) REFERENCES Students(student_id),
22     FOREIGN KEY (course_id) REFERENCES Courses(course_id)
23 );
24
25 INSERT INTO Students (name, email) VALUES
26     ('Jack', 'jack@example.com'),
27     ('Eliza', 'eliza@example.com'),
28     ('William', 'william@example.com'),
29     ('Maria', 'maria@example.com');
30
31 INSERT INTO Courses (title, credits) VALUES
32     ('Cloud Computing', 4),
33     ('Database Systems', 3),
34     ('Basic Programming', 4);
35
36 INSERT INTO Enrollments (student_id, course_id, grade) VALUES
37     (1, 1, 'A'),
38     (2, 3, 'B'),
39     (3, 1, 'B');
40
41 -- Confirm the relationships resolve correctly with a join query
42 SELECT s.name, c.title, e.grade
43 FROM Enrollments e
44 JOIN Students s ON e.student_id = s.student_id
45 JOIN Courses c ON e.course_id = c.course_id;
```

This Enrollments table is the same **many-to-many relationship** pattern (a Student can take many Courses; a Course can have many Students, resolved through a junction table carrying its own additional attribute, **grade**) that the earlier volumes’ JPA chapters modelled with @ManyToMany/@JoinTable annotations — the relational modelling concepts transfer directly from a locally-run database onto a managed RDS instance without any change at all.

44.5 Introduction to NoSQL Data Models

Section 44.1 introduced NoSQL as a broad category trading relational rigour for horizontal scalability and schema flexibility. In practice, “NoSQL” actually spans several distinct data models, each suited to a different access pattern:

NoSQL Model	Shape and Typical Use
Key-Value	The simplest model — an opaque value retrieved solely by its unique key, with no query capability into the value's internal structure at all. Extremely fast and simple to scale; appropriate for session storage, caching, or simple lookups
Document	Each record is a self-contained, semi-structured document (typically JSON-like), which <i>can</i> be queried on its internal fields, unlike a pure key-value store — appropriate when related data naturally nests together (an order together with its line items, as one document) rather than being normalised across separate tables
Columnar (Wide-Column)	Data organised by column families rather than rows, optimised for very large datasets where queries typically read a small subset of columns across a huge number of rows — appropriate for time-series and analytics-heavy workloads at massive scale

Amazon DynamoDB, covered next, is primarily a key-value and document store: every item is retrieved by its key (and, optionally, a sort key), but each item's *value* can itself be a rich, nested JSON-like structure, giving DynamoDB a genuinely hybrid position between the pure key-value and document models above.

44.6 Amazon DynamoDB

Amazon DynamoDB is AWS's fully-managed NoSQL database, and, like Lambda (Section 43.9), it is **serverless** — there is no DB instance to size, patch, or manage; capacity and availability are handled entirely by AWS, and (in **on-demand** capacity mode, the default) billing is based purely on the actual read/write requests made and the storage actually consumed.

DynamoDB Concept	Meaning
Table	A collection of items — DynamoDB's rough equivalent of a relational table, except with no fixed schema of columns shared by every row
Item	A single record within a table — DynamoDB's equivalent of a row, except each item can have a genuinely different set of attributes from every other item in the same table
Partition Key	A mandatory attribute that uniquely identifies each item (when no sort key is used) and determines which internal storage partition an item is physically stored on — DynamoDB uses this key's value, hashed, to decide where to place and later retrieve the item, which is why the partition key is central to how DynamoDB actually achieves its scalability
Sort Key (optional)	A second key attribute that, combined with the partition key, allows multiple items to share the same partition key while remaining individually addressable and queryable in sorted order by the sort key's value
Capacity Mode	On-demand (billed per request, no capacity planning needed — this chapter's activity uses this default) versus Provisioned (a fixed read/write throughput reserved and paid for continuously, appropriate for workloads with predictable, steady traffic where reserved capacity works out cheaper)

Why DynamoDB Has No Fixed Schema, and What That Actually Buys You

A relational table (Section 44.3's RDS activity) requires every row to have the same set of columns, defined up front; adding a genuinely new kind of attribute to some rows but not others means either leaving many columns NULL or a schema migration. A DynamoDB table requires nothing beyond its partition key (and optional sort key) to be defined up front — individual items are free to carry entirely different attributes, added or removed at will, with no migration step at all. This flexibility is precisely why DynamoDB fits naturally with fast-changing, non-uniform data shapes, and precisely why it is a poor fit for data whose value strongly depends on being queried or joined across many different, evolving attributes — exactly the strength a relational database provides instead.

44.6.1 Activity: Create Schema on DynamoDB

Activity: Create a DynamoDB Table and Store/Retrieve an Item

1. Sign in to the AWS Console and open the **DynamoDB** service; select **Tables** in the left navigation, then click **Create table**.
2. Enter a **Table name** (e.g. `Customer`) and a **Partition key** (e.g. `customerId`, type `Number`).
3. Under **Table settings**, choose **Default settings** and click **Create table** — the table's status begins as **Creating** and transitions to **Active** once ready.
4. Click on the table's name to view its details — partition key, sort key (if any), capacity mode, and item count are all shown here.
5. **Store an item**: click **Actions** → **Create item**, provide a value for the partition key attribute (e.g. `customerId = 1001`), and optionally click **Add new attribute** to add further attributes (e.g. `customerName`, type `String`, value `John`). Click **Create item**.
6. A success message confirms the item was saved, and it now appears in the table's item list.
7. **Retrieve items**: from the table's **Explore table items** view, use **Scan** (to browse every item in the table) or **Query** (to look up items by a specific partition key value, far more efficient at scale than a full scan) to confirm the stored item is retrievable.

Console Walkthrough — Screenshots

8 In the ECS sidebar, choose **Task definitions** → **Create new task definition**.

Select **Fargate**.

Configure:

Task name: MySampleTask

Task role: Leave default

Operating system: Linux.

Task size: 1 vCPU, 3 GB

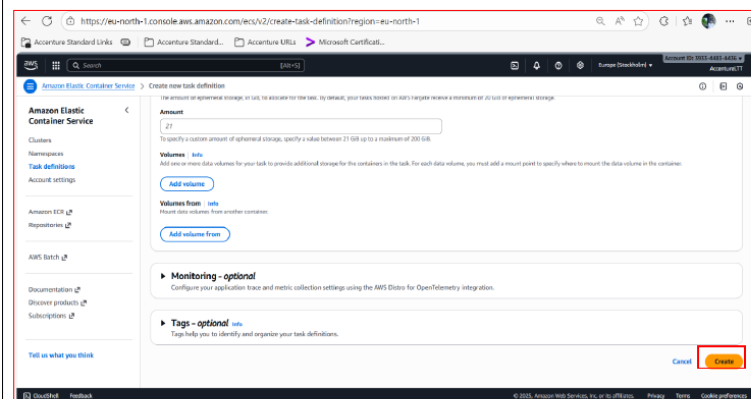
Under **Container definitions**, click **Add container**:

Container name: nginx

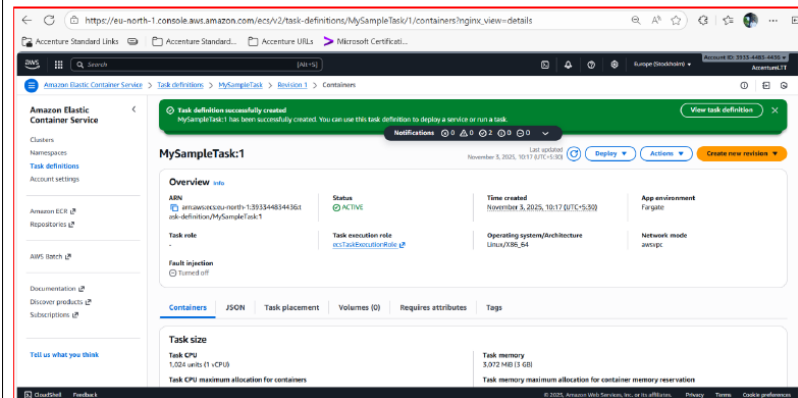
Image URI: nginx:latest (I am using available container)

Port mappings: Container port 80.

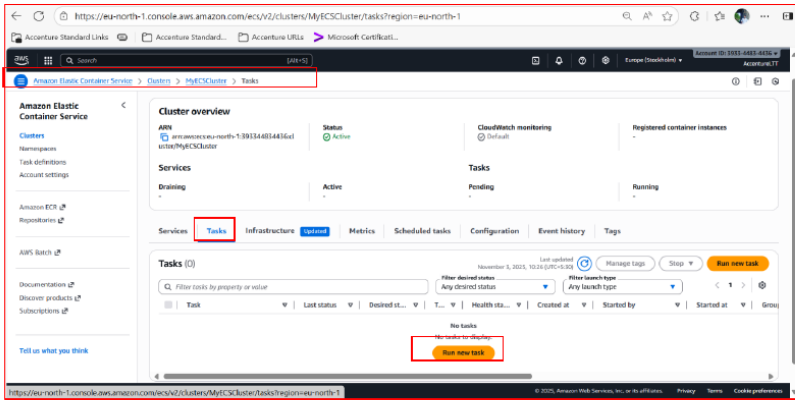
Click **Add**, then **Create**.



After Successfully Creation you can observe the status of the task created



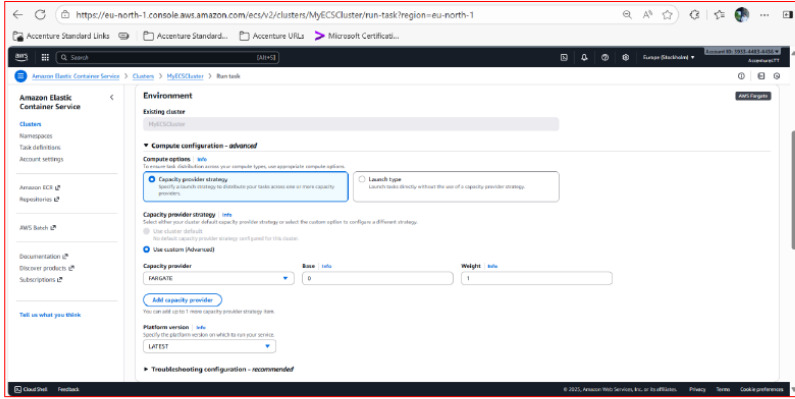
9 Now Go to your cluster → **Tasks** → **Run new task**.



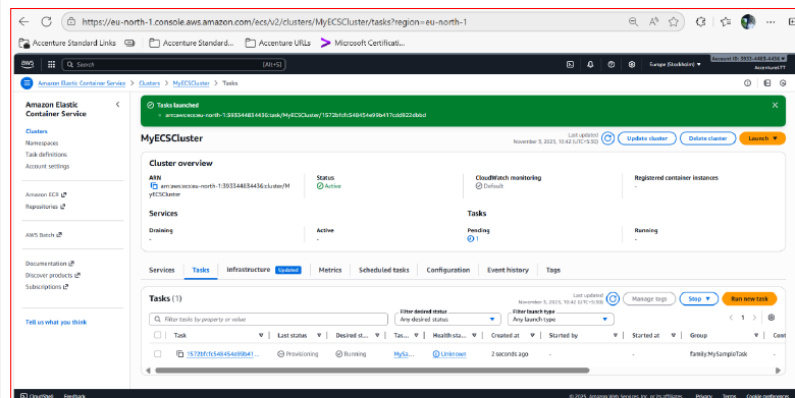
10 Choose:

- **Launch type:** Fargate
- **Task definition:** MySampleTask
- **Cluster:** MyECSCluster
- **Number of tasks:** 1

Under **VPC** and **Subnets**, choose your default VPC and at least one public subnet.
Under **Security group**, use default which allows inbound traffic on port 80 (HTTP).



11 Click Create, AWS will now launch a container in Fargate

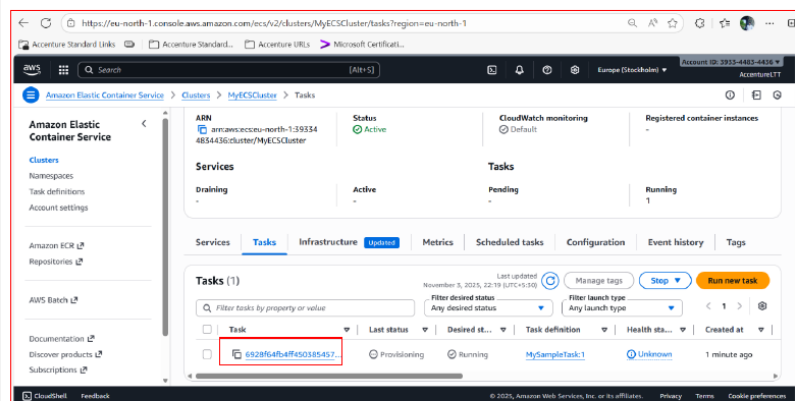


12 Check It's Running

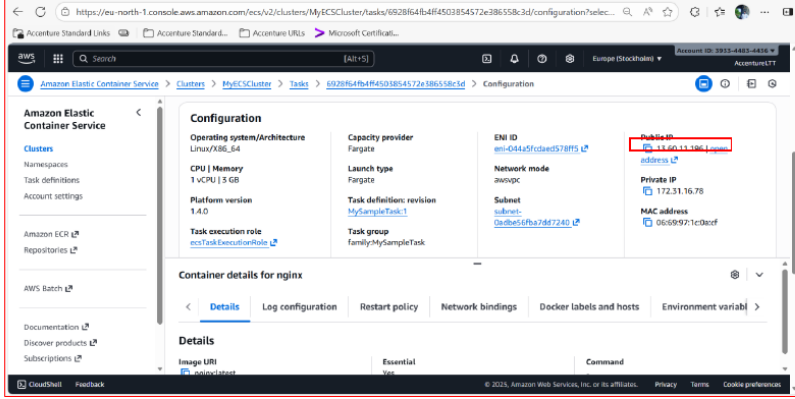
Go to your cluster → Tasks → [task ID].

Find the ENI (Elastic Network Interface) or Public IP address.

Open that IP in your browser → you should see the default Nginx welcome page.

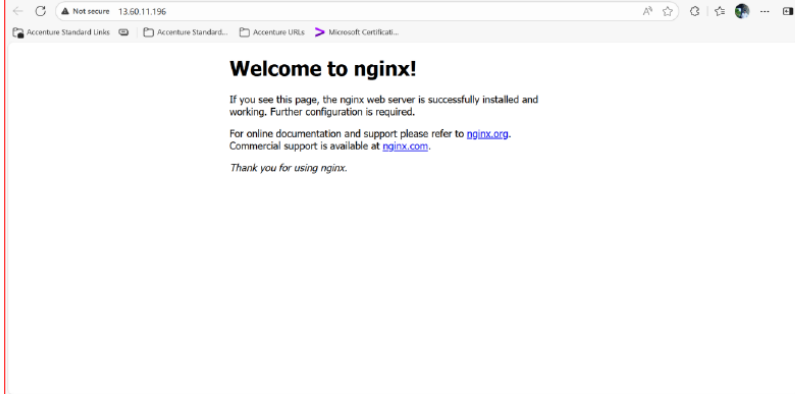


13 After clicking on Created Task
You can observe the public IP address → Click on IP address



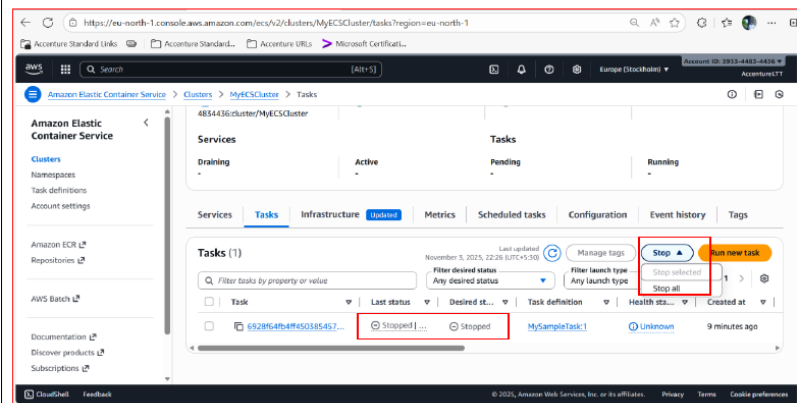
The screenshot shows the AWS Management Console for the Amazon Elastic Container Service (ECS). The left sidebar displays the navigation menu with options like Clusters, Namespaces, Task definitions, Account settings, Amazon ECR, Repositories, AWS Batch, Documentation, Discover products, and Subscriptions. The main content area shows the configuration details for a specific task. The 'Configuration' section includes fields for Operating system/Architecture (Linux/x86_64), CPU/Memory (1 vCPU | 5 GB), Platform version (1.4.0), Task execution role (ecsTaskExecutionRole), Capacity provider (Fargate), Launch type (Fargate), Task definition: revision (MySampleTask:1), Task group (family:MySampleTask), ENI ID (eni-0445fcded578ff5), Network mode (awsvpc), Subnet (subnet-0a8e56fa7d47240), and Public IP (172.31.36.196). The 'Public IP' field is highlighted with a red box. Below the configuration section, there are tabs for Details, Log configuration, Restart policy, Network bindings, Docker labels and hosts, and Environment variables. The 'Details' tab is selected, showing the Image URI (nginx:latest), Essential (Yes), and Command.

14 Default Nginx welcome page will be launched.



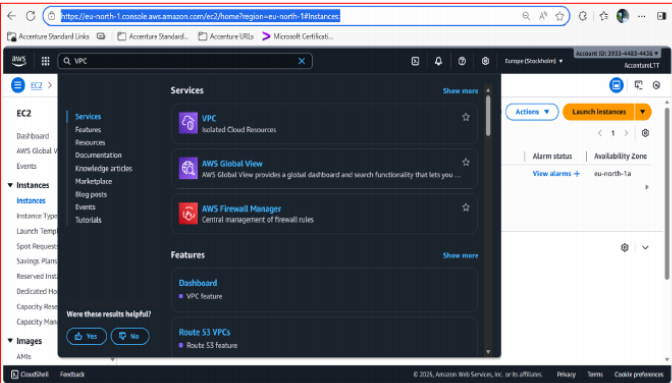
The screenshot shows a web browser window displaying the default Nginx welcome page. The address bar shows the URL 'http://172.31.36.196:80'. The page content includes the heading 'Welcome to nginx!', a message stating 'If you see this page, the nginx web server is successfully installed and working. Further configuration is required.', and links to online documentation and support at 'nginx.org'. The footer of the page says 'Thank you for using nginx.'

15 Once the tasks are completed, please stop, so you can observe the task status as Stopped



AWS Console — step screenshot (page 5 of 6)

4. Activity to create EKS

1	Prerequisites (IAM Roles and Networking) Before creating the cluster, you need foundational resources. Create a VPC and Subnets EKS requires a Virtual Private Cloud (VPC) with subnets across at least two Availability Zones (AZs). The easiest way to do this is using the VPC Wizard: Sign In to AWS Search for VPC in search bar  The screenshot shows the AWS Management Console search results for 'VPC'. The search bar at the top contains 'VPC'. The results are categorized into Services, Features, and Images. Under Services, 'VPC' is listed as 'Isolated Cloud Resources'. Other services shown include 'AWS Global View' and 'AWS Firewall Manager'. Under Features, 'Dashboard' is listed with a 'VPC feature' link, and 'Route 53 VPCs' is listed with a 'Route 53 feature' link. A sidebar on the left shows navigation options like 'Dashboard', 'Instances', and 'Images'. The top right of the console shows the user's account information and the region 'Europe (Frankfurt)'.
---	---

AWS Console — step screenshot (page 6 of 6)

44.7 VPC Components — CIDR, Route Tables, and Gateways

Section 44.2 introduced the VPC at a glance; this section covers the components a VPC is actually built from in enough depth to make sense of the full networking activity that follows, and to explain the earlier EKS activity’s own VPC prerequisite (Section 43.8) properly.

44.7.1 CIDR — Sizing an IP Address Range

Every VPC (and every subnet within it) is defined by a block of IP addresses expressed in **CIDR (Classless Inter-Domain Routing)** notation — an address followed by a slash and a number, such as `10.0.0.0/16`, where the number after the slash indicates how many of the address’s leading bits are fixed as the network portion, with the remaining bits available for individual host addresses within that range. A `/16` block (as commonly used for an entire VPC) leaves 16 bits free, providing 65,536 addresses; a `/24` block (commonly used for one individual subnet carved out of that VPC) leaves only 8 bits free, providing 256 addresses. Subnets within a VPC are carved out of the VPC’s own CIDR block as non-overlapping, smaller ranges — exactly as this chapter’s own activity creates a VPC at `10.0.0.0/16` and then carves a `PublicSubnet` at `10.0.1.0/24` and a `PrivateSubnet` at `10.0.2.0/24` out of it.

44.7.2 Public vs Private Subnets — Defined by Their Route, Not Their Name

It is worth being precise about what actually makes a subnet “public” or “private”, because it is not an inherent property of the subnet itself, but purely a consequence of what its **route table** directs internet-bound traffic to (covered next): a subnet is **public** if its route table sends traffic destined for the internet (`0.0.0.0/0`, meaning “any address not otherwise matched”) to an **Internet Gateway**; a subnet is **private** if it has no such route, or if its internet-bound traffic instead routes through a **NAT Gateway** sitting in a public subnet. Nothing about a subnet’s own CIDR block or its name makes it one or the other — the routing configuration is the entire story.

44.7.3 Internet Gateway

An **Internet Gateway (IGW)** is a horizontally-scaled, redundant VPC component that allows communication between resources in a VPC and the internet — a VPC has, at most, one Internet Gateway attached, and a resource (such as an EC2 instance) can only be reached directly from the internet if it both has a public IP address *and* sits in a subnet whose route table sends internet-bound traffic to this gateway.

44.7.4 NAT Gateway — Outbound Internet Access Without Inbound Exposure

A **NAT (Network Address Translation) Gateway** lets resources in a *private* subnet (with no public IP address of their own) initiate outbound connections to the internet — for example, a private-subnet EC2 instance downloading a software update, or (exactly as named in Section 43.8’s EKS discussion) a private-subnet Kubernetes worker node pulling a container image — while still preventing the internet from initiating any *inbound* connection back to those same private resources. A NAT Gateway is deployed into a *public* subnet (since it needs its own route to the Internet Gateway) and is then referenced by the *private* subnet’s own route table as the target for its own outbound internet-bound traffic — this asymmetry (outbound only, no unsolicited inbound) is precisely what makes a NAT Gateway an appropriate, more secure choice for resources that need internet access without needing to *be* reached from the internet directly.

44.7.5 Route Tables

A **route table** contains a set of rules, called **routes**, that determine where network traffic from a subnet is directed, based on its destination. Every VPC has a **main route table** by default; additional, custom route tables (as this chapter’s activity creates, one for the public subnet and a separate one for the private subnet) can be created and explicitly associated with specific subnets, overriding the main table for just those subnets. A route table entry always has two parts: a **destination** (a CIDR range the rule applies to, such as `10.0.0.0/16` for traffic staying within the VPC, or `0.0.0.0/0` for everything else) and a **target** (where matching traffic should actually be sent — `local` for traffic remaining inside the VPC, an Internet Gateway ID for a public subnet’s internet-bound traffic, or a NAT Gateway ID for a private subnet’s internet-bound traffic).

Route Table	Destination	Target
PublicRouteTable	<code>0.0.0.0/0</code>	Internet Gateway (direct internet access)
PrivateRouteTable	<code>0.0.0.0/0</code>	NAT Gateway (outbound-only internet access)

Tying the Whole Picture Together

A resource’s actual reachability from the internet depends on the combination of *three* independent layers, each covered in this chapter or the last: whether its **subnet’s route table** sends internet-bound traffic to an Internet Gateway at all (this section); whether the resource itself has a **public IP address** to be reached at (Section 43.4); and whether its **security group** (Section 43.4) actually permits the specific inbound traffic in question. All three must align for a resource to be genuinely reachable from the internet — which is exactly why a private-subnet RDS instance (Section 44.4) remains unreachable from the public internet even if its security group were mistakenly opened wide, since its subnet’s own routing never gives inbound internet traffic a path to it in the first place.

44.7.6 Activity: Create a VPC with Public and Private Subnets

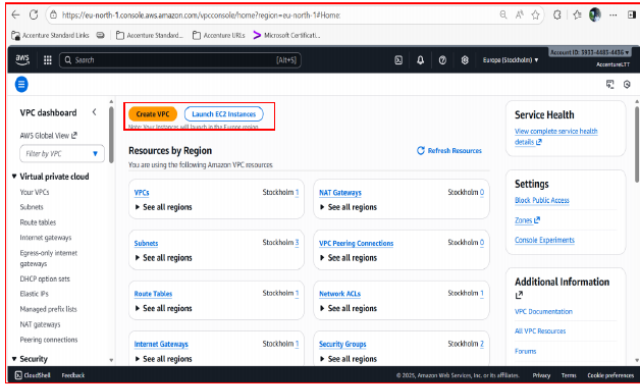
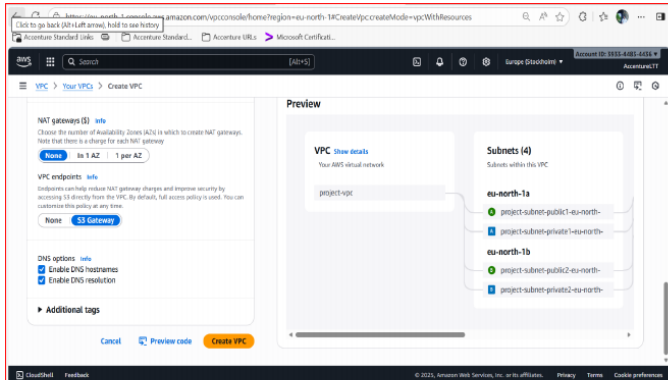
Activity: Build a VPC With Both Subnet Types, End to End

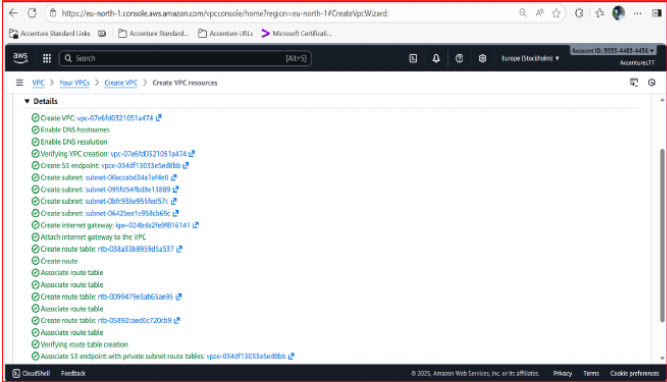
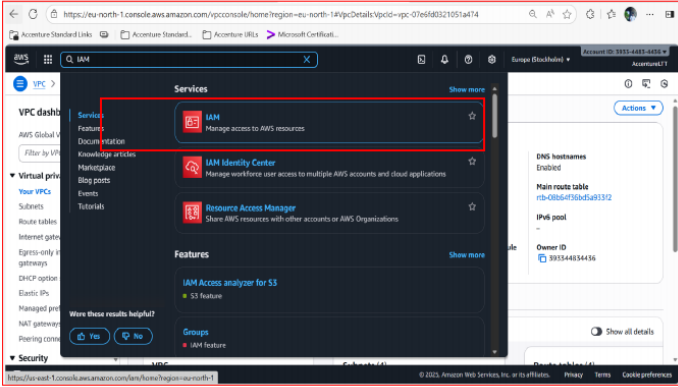
1. **Create the VPC:** in the VPC console, **Your VPCs** → **Create VPC**, provide a **Name tag** (e.g. `MyVPC`) and an **IPv4 CIDR** (e.g. `10.0.0.0/16`), leaving other settings at default, then click **Create VPC**.
2. **Create the public subnet:** **VPC** → **Subnets** → **Create subnet**, select the VPC, name it (e.g. `PublicSubnet`), choose an Availability Zone, and set its **IPv4 subnet CIDR block** (e.g. `10.0.1.0/24`), then **Create subnet**.
3. **Create the private subnet:** repeat the same steps with a different name (e.g. `PrivateSubnet`) and a non-overlapping CIDR block (e.g. `10.0.2.0/24`).
4. **Create and attach an Internet Gateway:** **VPC** → **Internet gateways** → **Create internet gateway**, name it (e.g. `MyIGW`), create it, then **Actions** → **Attach to VPC** and select the VPC created above.
5. **Configure the public route table:** **VPC** → **Route tables** → **Create route table**, name it (e.g. `PublicRouteTable`) and associate it with the VPC. On the new table’s **Routes** tab, **Edit routes** → **Add route**: destination `0.0.0.0/0`, target the Internet Gateway created above, then **Save changes**. On the **Subnet associations** tab, associate this route table explicitly with **PublicSubnet**.
6. **Create a NAT Gateway:** **VPC** → **NAT gateways** → **Create NAT gateway**, select **PublicSubnet** (a NAT Gateway must live in a public subnet), click **Allocate**

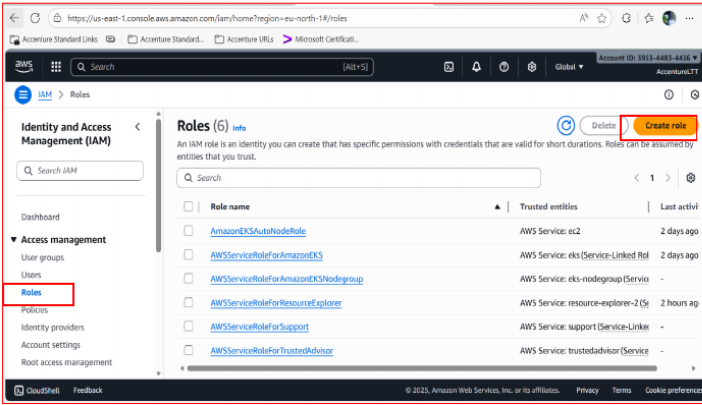
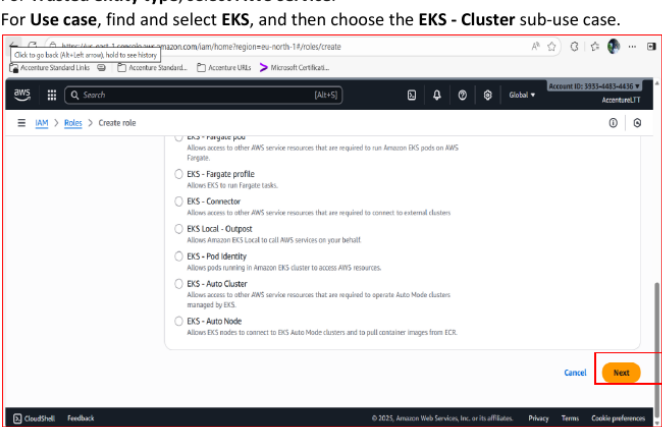
Elastic IP for its required static address, then **Create NAT gateway**.

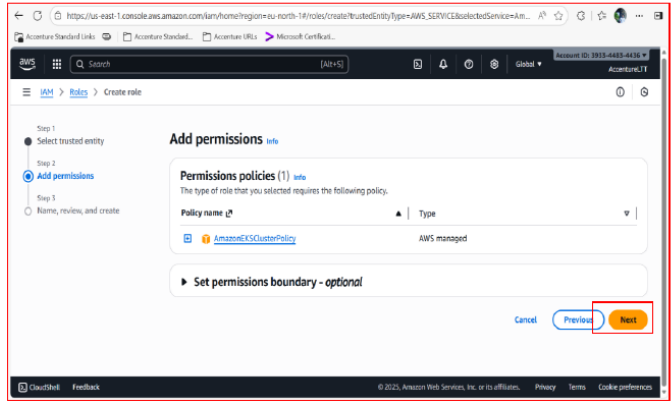
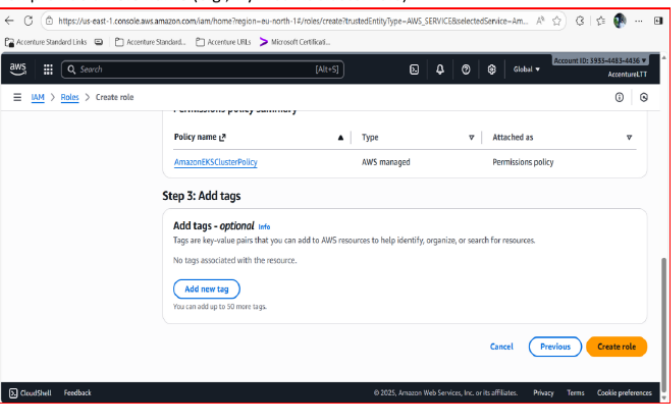
7. **Configure the private route table:** create a second route table (e.g. **PrivateRouteTable**), associated with the same VPC. On its **Routes** tab, add a route with destination **0.0.0.0/0** and target the **NAT Gateway** created above. On its **Subnet associations** tab, associate it explicitly with **PrivateSubnet**.
8. The VPC now has a genuinely public subnet (routed directly to the Internet Gateway) and a genuinely private subnet (routed only outbound, via the NAT Gateway) — exactly the two-subnet-type foundation the EKS activity (Section 43.8) and a production-grade RDS deployment (Section 44.4) would both be built on top of.

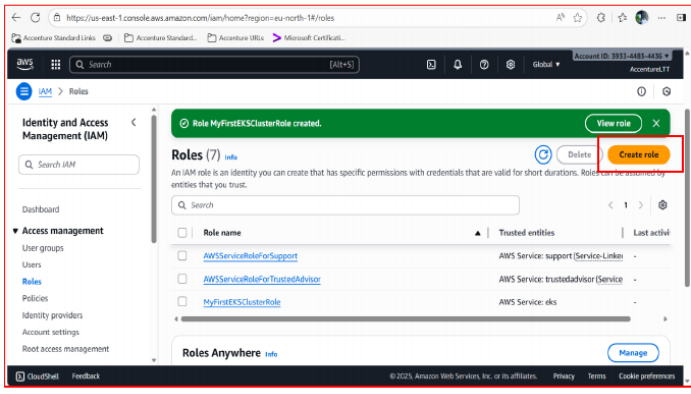
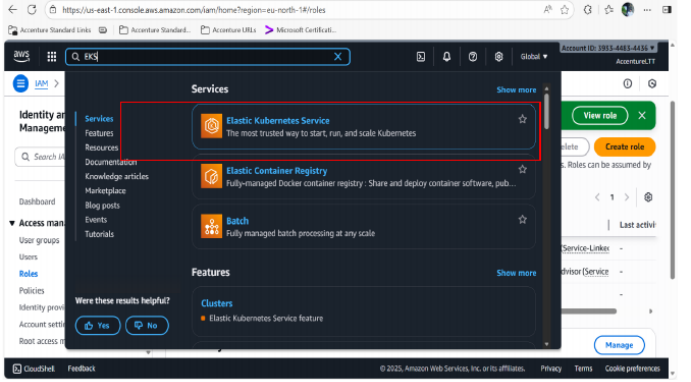
Console Walkthrough — Screenshots

2	Go to the VPC console. 
3	In the left navigation pane, choose Your VPCs, then click Create VPC. 1. Select VPC and more (This is the fastest method). 2. For Number of Availability Zones (AZs), choose 2. 3. Keep the defaults for subnets (2 public, 2 private). 4. Click Create VPC. This will create a VPC, subnets, Internet Gateway, and NAT Gateway (which costs money but is needed for worker nodes in private subnets to pull images). Note down the VPC ID. 

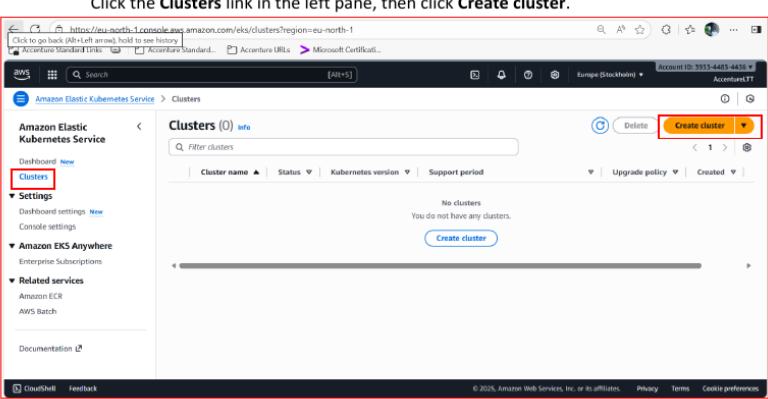
4	View VPC once created successfully. 
5	Create an EKS Cluster IAM Role The EKS control plane needs permission to manage AWS resources (like EC2 instances, VPC, and Load Balancers) on your behalf. Go to the IAM console. 

6	Choose Roles, then click Create role. 
7	For Trusted entity type, select AWS service. For Use case, find and select EKS, and then choose the EKS - Cluster sub-use case. 

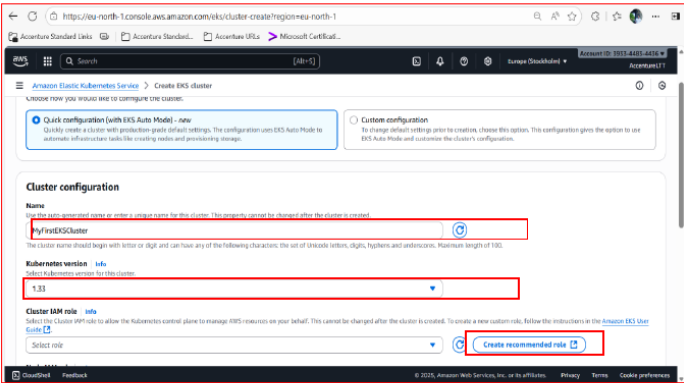
8	Click Next. The AmazonEKSClusterPolicy will be pre-selected. 
9	Click Next, then provide a Role name (e.g., MyFirstEKSClusterRole). 

10	Click Create role.  The screenshot shows the AWS IAM console 'Roles' page. A green banner at the top indicates 'Role MyFirstEKSClusterRole created.' Below this, the 'Roles (7)' section lists several roles, including 'MyFirstEKSClusterRole'. A red rectangular box highlights the 'Create role' button in the top right corner of the console interface.
11	Create the EKS Cluster Control Plane Go to the Amazon EKS console.  The screenshot shows the AWS IAM console 'Services' page. A red rectangular box highlights the 'Elastic Kubernetes Service' option, which is described as 'The most trusted way to start, run, and scale Kubernetes'. The page also lists other services like 'Elastic Container Registry' and 'Batch'.

12 Click the **Clusters** link in the left pane, then click **Create cluster**.



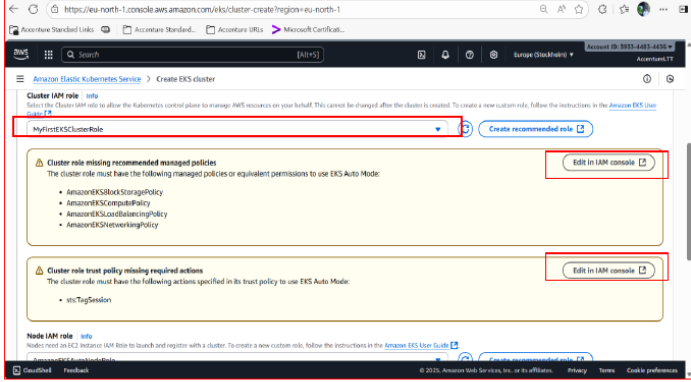
13 Configure cluster:
Name: Enter a unique name (e.g., MyFirstEKSCluster).



14

Kubernetes version: Choose the latest supported version.

Cluster service role: Select the **Cluster IAM Role** you created in previous step (e.g., MyFirstEKSClusterRole).



15

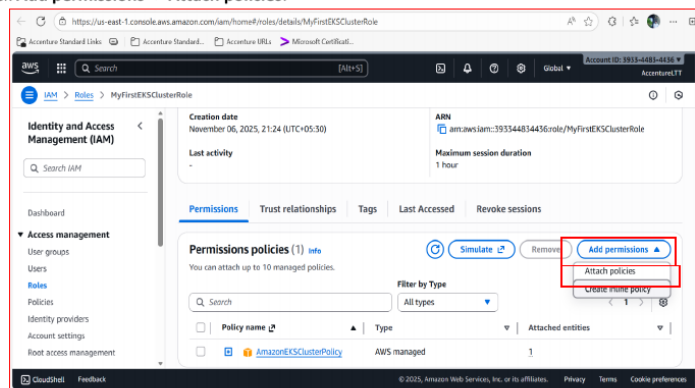
Update the Cluster Role for EKS Auto Mode

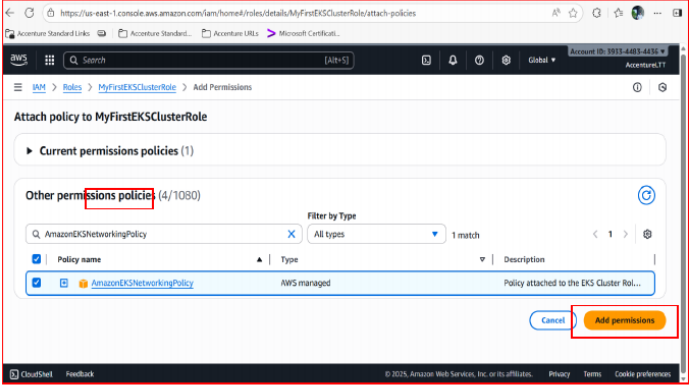
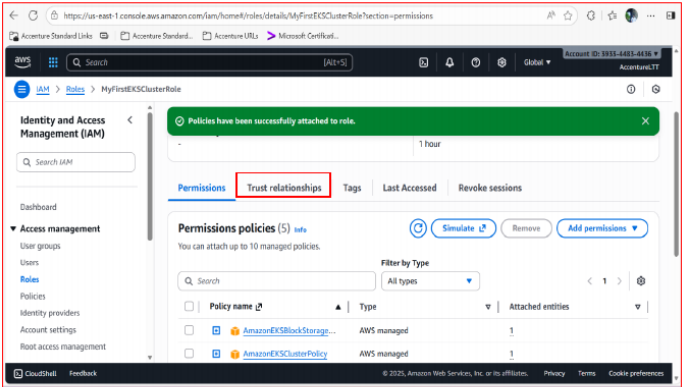
The simplest way to fix this is to use the AWS Console's recommendation feature.

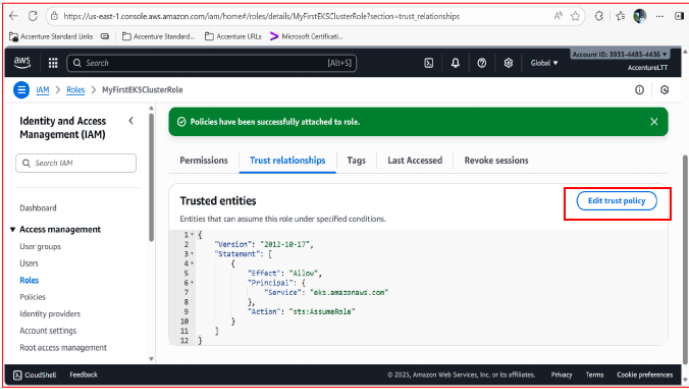
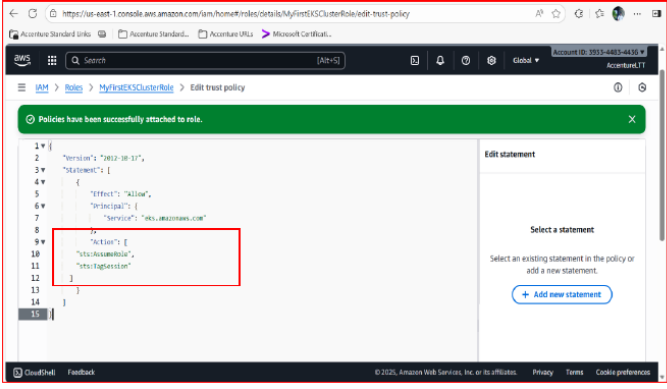
1. Click the **"Create recommended role"** button: In the top right corner of the image you provided, click the **Create recommended role** button.
 - If you're still on the EKS cluster creation page, this button should automatically create a new, compliant IAM role and select it for your cluster.
2. **Manually Edit the IAM Role (If you click "Edit in IAM console"):**
 - If you choose to use your existing role (MyFirstEKSClusterRole), you need to navigate to the **IAM Console** and attach the missing policies and update the trust policy.
3. **Attach Missing Managed Policies**

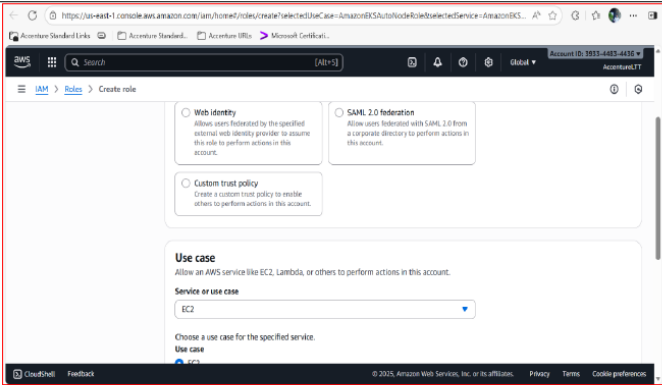
You must attach the four policies listed in the first warning:

 1. Go to the **IAM Console** -> **Roles**.
 2. Search for and select your cluster role (MyFirstEKSClusterRole).
 3. Click **Add permissions** -> **Attach policies**.



16	4. Search for and attach the following four AWS Managed Policies: • AmazonEKSBlockStoragePolicy • AmazonEKSClusterPolicy • AmazonEKSLoadBalancingPolicy • AmazonEKSNetworkingPolicy 
17	Update the Trust Relationship You must add the sts:TagSession action to the trust policy as indicated in the second warning. 1. While still viewing your cluster role (MyFirstEKSClusterRole), click the Trust relationships tab. 

18	2. Click Edit trust policy. 
19	3. Locate the existing statement that allows the EKS service to assume the role (where "Service": "eks.amazonaws.com"). 4. Add "sts:TagSession" to the Action array. 
20	Click Update Policy. Once you have either created the recommended role or manually updated your existing role, click the refresh icon next to the role dropdown in the EKS console.
15	Create the Node IAM Role 1. Navigate to the IAM console in a new browser tab. 2. In the left navigation pane, choose Roles, and then click Create role. 3. For Trusted entity type, select AWS service. 4. For the Use case, select EC2 (because your nodes are essentially EC2 instances). 5. Click Next.

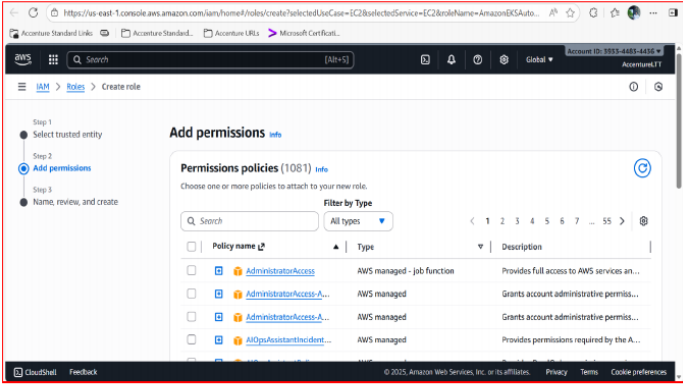


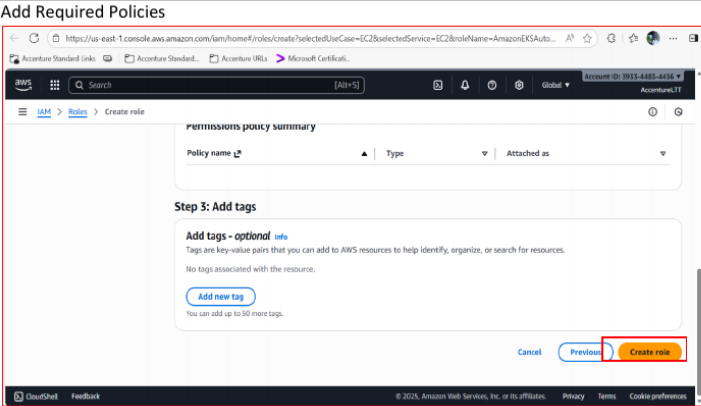
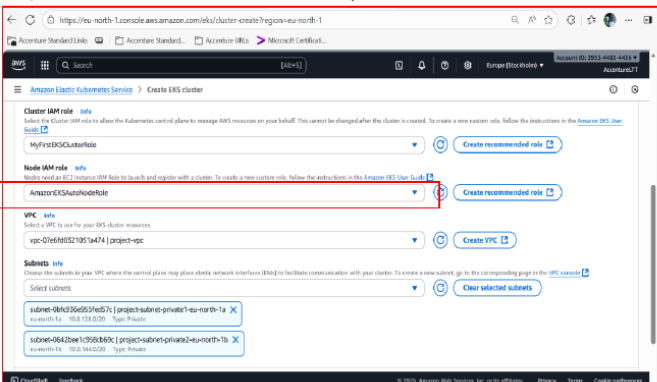
16

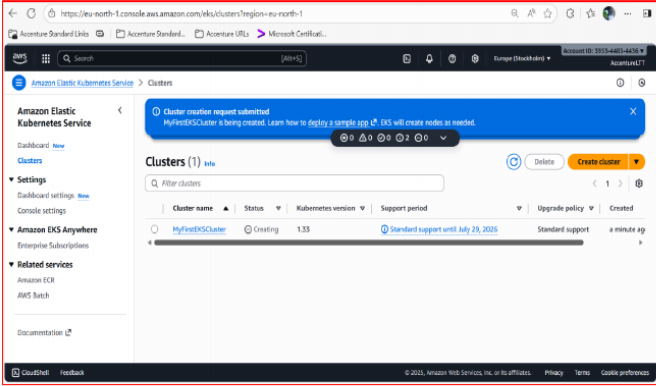
Attach Required Policies

On the "Add permissions" page, you must attach the following three AWS Managed Policies:

- AmazonEKSWorkerNodePolicy
- AmazonEKS_CNI_Policy
- AmazonEC2ContainerRegistryReadOnly



17	Add Required Policies 
	Create role Once you have either created the recommended role or manually updated your existing role, click the refresh icon next to the role dropdown in the EKS console. 

18	Select VPC Created in previous step Click on Create 
	Your Cluster will be created Thank You!

44.8 Identity and Access Management (IAM) and Multi-Factor Authentication

Section 43.4 introduced the distinction between the AWS root user and an IAM user, and every activity in this chapter and the last has, implicitly, been performed by *some* authenticated identity. This final section makes that identity's own security explicit, closing the loop on an idea that has run through this entire volume: least privilege for *resources* (Section 43.8's scoped IAM roles) is only half the security picture; the *identity* performing an action in the first place also needs to be strongly authenticated, which is exactly what **Multi-Factor Authentication (MFA)** provides.

44.8.1 IAM — Users, Groups, Roles, and Policies

IAM Concept	Meaning
IAM User	An identity representing a specific person (or application) that needs to interact with AWS, with its own credentials (a console password, and/or programmatic access keys)
IAM Group	A collection of IAM users, used to attach the same set of permissions to many users at once rather than repeating the same policy attachment individually per user
IAM Role	An identity, similar to a user, but intended to be <i>assumed</i> temporarily by whoever or whatever needs it (a person, an AWS service, or an application) rather than tied permanently to one specific user — exactly the Cluster IAM Role and Node IAM Role from Section 43.8, each assumed by a specific AWS service rather than belonging to any one person
Policy	A JSON document defining a set of permissions — which actions are allowed or denied, on which resources — that can be attached to a user, group, or role

44.8.2 Why MFA Matters, Specifically

A password alone is a **single factor** of authentication — something the user *knows*. If that password is guessed, phished, or leaked in an unrelated data breach (a genuinely common occurrence, given how often people reuse passwords across services), a password alone is the *only* thing standing between an attacker and full access to whatever that password protects. **Multi-Factor Authentication** requires a *second*, independent factor — typically something the user *has* (a mobile device running an authenticator app, or a hardware security key) — so that even a fully compromised password is, by itself, insufficient to authenticate. AWS's own MFA implementation, used in this chapter's activity, is a **virtual MFA device**: rather than a physical hardware token, an authenticator app (such as Microsoft Authenticator) generates a new, six-digit, time-based one-time code roughly every 30 seconds, derived from a secret shared with AWS at setup time (delivered via a scanned QR code) — since both AWS and the app independently compute the same code from the same secret and the same current time, neither side ever has to transmit the actual code across the network for the other to verify it.

Why AWS Requires Two Consecutive Codes During MFA Setup

When first registering a virtual MFA device, AWS asks for *two* consecutive codes, roughly 30 seconds apart, rather than just one. This is a deliberate verification step: entering two correctly-timed, sequential codes proves the device is genuinely correctly synchronised with AWS's own clock and is actually generating valid codes from the shared secret — a single correct code could, in principle, be a lucky coincidence or a stale, cached value; two

consecutive correct codes are a much stronger signal that the setup genuinely succeeded.

44.8.3 Activity: Set Up MFA on an IAM User

Activity: Enable Multi-Factor Authentication for an IAM User

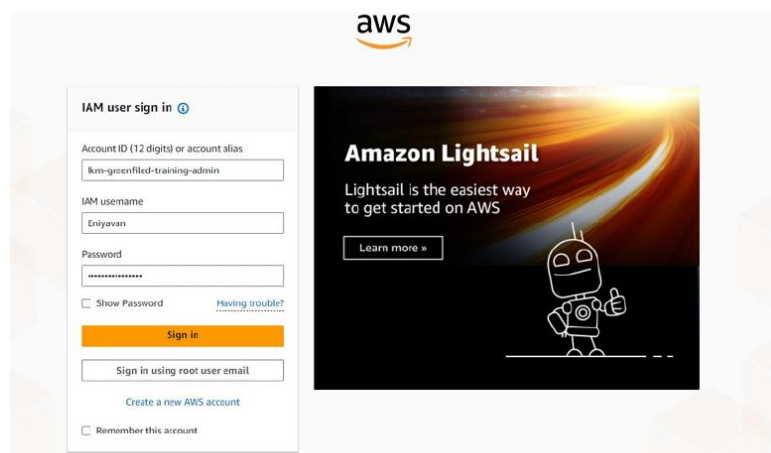
1. Sign in to the AWS Console using the organisation's specific IAM sign-in URL (of the form `https://<account-alias>.signin.aws.amazon.com/console`) with the provided IAM username and password.
2. In the search bar, type **IAM** and select the **IAM** service.
3. Under **Users**, search for and select the relevant IAM username to open its details page.
4. Select the **Security credentials** tab, then, under **Multi-factor authentication (MFA)**, click **Assign MFA device**.
5. Provide an **MFA device name** (e.g. `MyAccentureLab`), select **Authenticator app** as the device option, and click **Next**.
6. On a mobile phone, install an authenticator app (e.g. **Microsoft Authenticator**) from the app store, open it, and choose **Add account** → **Work or school account**.
7. Back in the AWS Console, click **Show QR code**, then scan the displayed QR code using the authenticator app — this transmits the shared secret the app will use to generate codes.
8. The authenticator app immediately begins displaying a six-digit code, refreshing roughly every 30 seconds. Enter the currently displayed code into **MFA Code 1**; wait for the app to generate the next code (roughly 30 seconds later) and enter it into **MFA Code 2**.
9. Click **Add MFA**. A success message (“**MFA device assigned**”) confirms the device is now registered to this IAM user.
10. Sign out of the console and sign back in — the sign-in flow now additionally prompts for a current code from the authenticator app before granting access, confirming MFA is genuinely enforced, not merely configured.

Console Walkthrough — Screenshots

Follow the below listed steps to enable MFA:

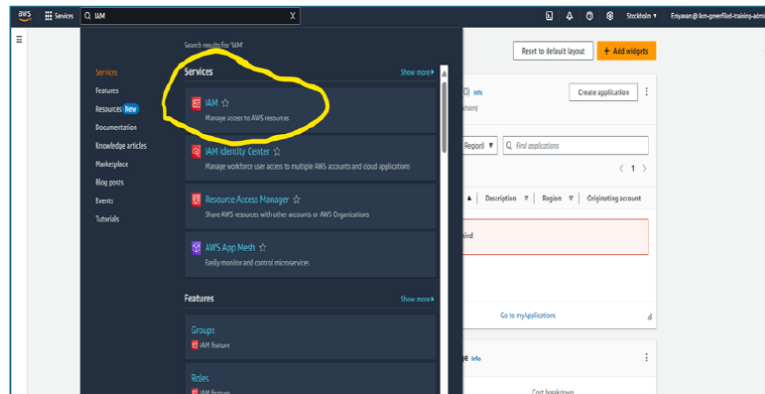
Step 1: Login to Amazon console using the below URL and Enter your IAM username and password provided over an email to sign in.

<https://lkm-greenfiled-training-admin.signin.aws.amazon.com/console>

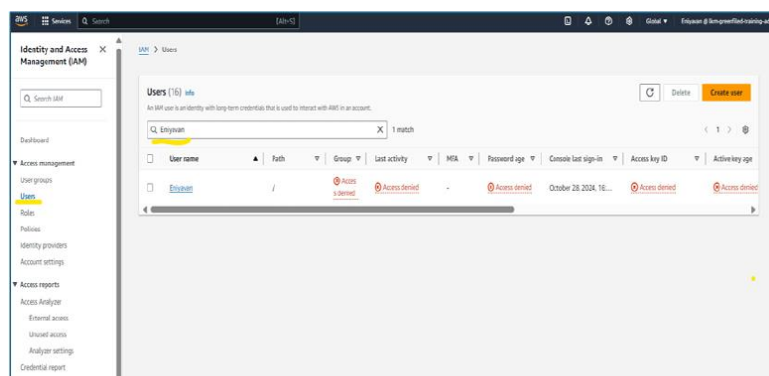


AWS Console — step screenshot (page 1 of 7)

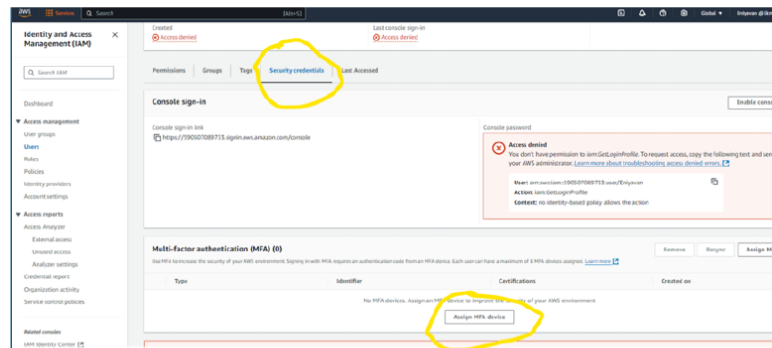
Step 2: After login you can see the below AWS console, in the Search bar type IAM and select the service IAM as shown in the below screenshot.



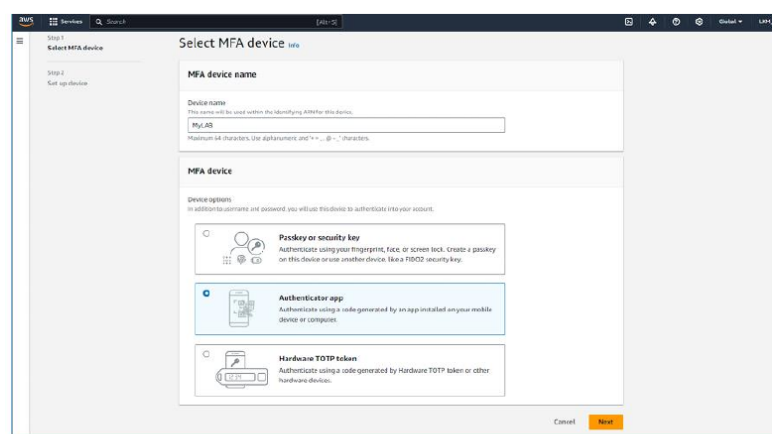
Step 3: Search for your username shared with you over an email. Select the IAM username to open the User window as shown in below screenshot.



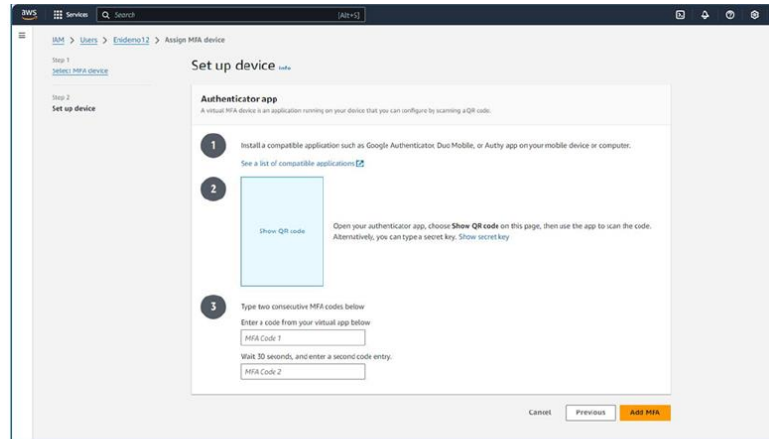
Step 4: Select security credentials and then click on Assign MFA device



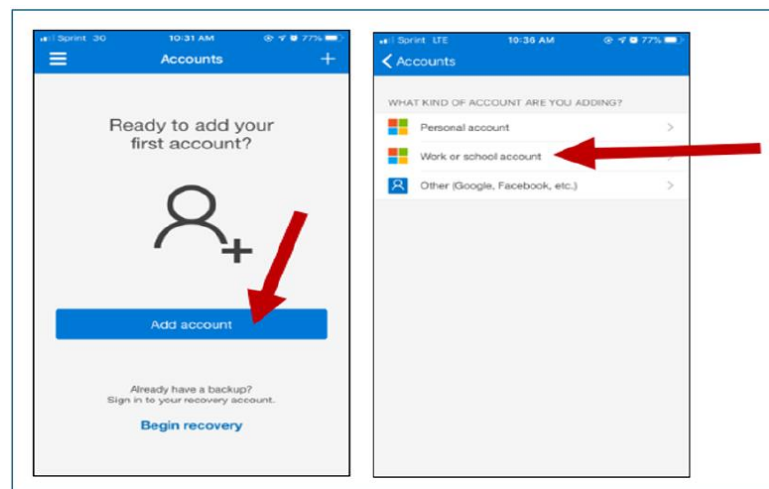
Step 5: Provide any MFA device name and select Authenticator App and click on next.
Example of MFA Device Name : “MyAccentureLab”



Step 6: Install “Microsoft Authenticator” app from Play store on your mobile to add MFA code.



Step 7: Once the Authenticator app has been downloaded to your phone, click Add Account, choose Work or School Account as shown from the below screenshot on the phone.



Step 8: To enable MFA, click Show QR code from step 6, scan the bar code with your phone, then enter the two consecutive MFA codes that show up on your phone.

Set up device Info

Authenticator app

A virtual MFA device is an application running on your device that you can configure by scanning a QR code.

1

Install a compatible application such as Google Authenticator, Duo Mobile, or Authy app on your mobile device or computer.
[See a list of compatible applications](#)

2



Open your authenticator app, choose **Show QR code** on this page, then use the app to scan the code. Alternatively, you can type a secret key. [Show secret key](#)

3

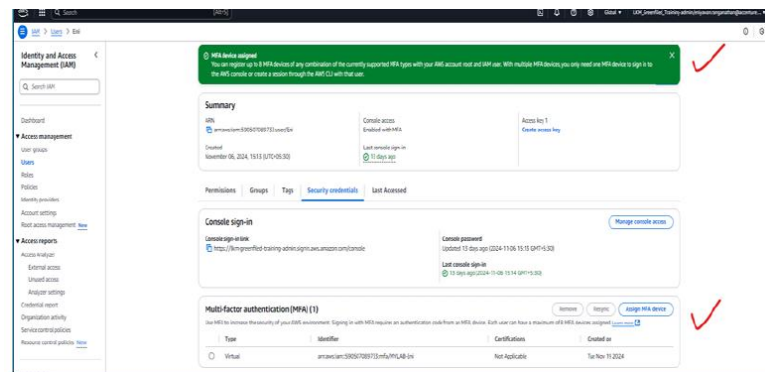
Type two consecutive MFA codes below

Enter a code from your virtual app below

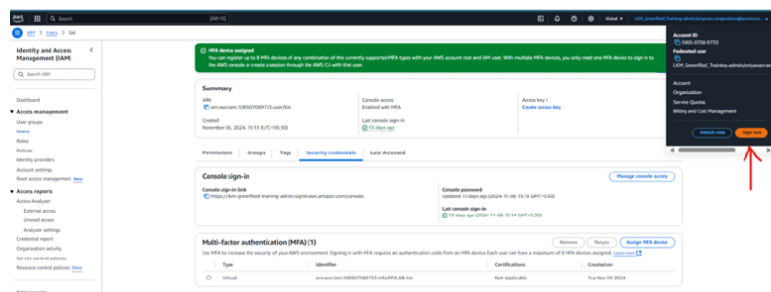
Wait 30 seconds, and enter a second code entry.

Cancel Previous Add MFA

Step 9: MFA device assigned message will appear as shown in below Snapshot (Highlighted text in green).



Step 10: Once the MFA is enabled, please sign out and sign in again with MFA



AWS Console — step screenshot (page 7 of 7)

Viva Voce — Likely Questions & Answers

Q: Why are object storage, relational databases, and NoSQL databases not simply ranked from "worst" to "best", but treated as genuinely different trade-offs?

A: Because each is optimised for a different shape of data and access pattern: object storage suits whole, opaque files with no internal query needs; relational databases suit structured, related data requiring complex multi-table queries; NoSQL databases suit flexible, evolving data at a scale where relational overhead becomes limiting. The right choice depends on the actual workload, not a universal quality ranking.

Q: What does it mean for an S3 bucket name to be "globally unique", and why does that matter?

A: It means the name must be unique across every AWS account in existence, not just within one account, because S3 bucket names form part of a globally shared namespace used to route requests to the correct bucket regardless of which account owns it.

Q: What does Block Public Access do, and why is it enabled by default on every new S3 bucket?

A: It is a set of four independent settings that prevent a bucket or its objects from being made publicly accessible via ACLs or bucket policies, regardless of other configuration. It is on by default because accidentally-public S3 buckets have historically been a common and serious data-exposure incident, so AWS defaults to the safer, private posture.

Q: What operational responsibilities does Amazon RDS take over that a self-managed database on a plain EC2 instance would not get automatically?

A: Automated backups and point-in-time recovery, engine patching, and (in a Multi-AZ deployment) automatic failover to a synchronised standby instance – none of which exist unless someone specifically builds and maintains that capability on a self-managed database.

Q: Is an RDS instance "outside" the VPC networking model that governs EC2 instances?

A: No – an RDS instance is provisioned into a specific VPC and subnet exactly like an EC2 instance, and its reachability is governed by the same security-group mechanism. "Managed" refers to AWS handling the database engine's installation, patching, and backups, not to the instance existing outside the normal networking model.

Q: What is the core structural difference between a relational table and a DynamoDB table?

A: A relational table requires every row to share the same fixed set of columns, defined up front; a DynamoDB table requires only a partition key (and optional sort key) to be defined up front, with individual items free to carry entirely different attributes without any schema migration.

Q: What role does the partition key play in DynamoDB beyond simply identifying an item?

A: Its value is hashed by DynamoDB to determine which internal storage partition the item is physically stored on, which is central to how DynamoDB distributes data and achieves its horizontal scalability.

Q: What actually determines whether a subnet is "public" or "private" in a VPC?

A: Purely its route table's configuration – specifically, whether internet-bound traffic (0.0.0.0/0) is routed to an Internet Gateway (making it public) or has no such route, routing

instead through a NAT Gateway or nowhere at all (making it private). It is not an inherent property of the subnet's name or CIDR block.

Q: What is the key functional difference between an Internet Gateway and a NAT Gateway?

A: An Internet Gateway allows both inbound and outbound internet communication for resources with a public IP in a subnet routed to it; a NAT Gateway allows only outbound-initiated internet communication for resources in a private subnet, without permitting the internet to initiate any inbound connection back to those same resources.

Q: Why must a NAT Gateway itself be placed in a public subnet?

A: Because the NAT Gateway needs its own route to the Internet Gateway to actually reach the internet on behalf of the private-subnet resources using it; placing it in a private subnet would leave it with no path to the internet at all.

Q: What three independent layers must all align for a resource to be reachable directly from the internet?

A: Its subnet's route table must send internet-bound traffic to an Internet Gateway; the resource itself must have a public IP address; and its security group must permit the specific inbound traffic in question. All three conditions must hold simultaneously.

Q: Why does AWS require two consecutive codes, not just one, when first registering a virtual MFA device?

A: Because two correctly-timed, sequential codes prove the device is genuinely synchronised with AWS's clock and is actively generating valid codes from the shared secret; a single correct code could, in principle, be a coincidence or a stale cached value.

Q: What is the fundamental security improvement MFA provides over a password alone?

A: A password alone is a single factor – something the user knows – which, if guessed, phished, or leaked, is by itself sufficient for an attacker to authenticate. MFA requires a second, independent factor (typically something the user has, like an authenticator app), so a compromised password alone is no longer sufficient to gain access.

Maven & Git Fundamentals to Code Quality with SonarQube

Version Control, Build Automation & Static Code Analysis

Extended Theory, Concepts & Hands-On Activities
(Viva / Interview Preparation Edition)

Accenture QuantumCrew Batch — Day-wise Training Reference

Covers Units 35–37 of the Day-wise Training Plan

Chapters 1–3 — Git, Maven, and SonarQube Theory Integrated with Hands-On Activities

Chapter 1

Version Control with Git & GitLab

1.1 The Big Picture — Where These Tools Fit

Before going deep into any one tool, it helps to see where each one sits in the everyday life of a software team. The common shorthand for that life is the **DevOps loop**: an endless cycle of planning work, writing code, building and testing it, releasing it, and then operating and monitoring it in production — with what is learned from monitoring feeding straight back into the next round of planning.

The DevOps Loop — Where Git, Maven, SonarQube & Jenkins Fit

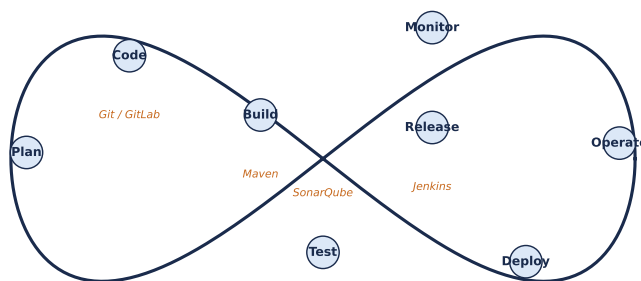


Figure 1.1: The DevOps loop, and where Git, Maven, SonarQube, and Jenkins each sit within it

A Basic Vocabulary Before We Start

- **SDLC (Software Development Life Cycle)** — the broad sequence every piece of software goes through: requirements, design, implementation (coding), testing, deployment, and maintenance. Different teams follow this in different ways (Waterfall does it once, top to bottom; Agile repeats a short version of it every sprint), but

the stages themselves are universal.

- **DevOps** — a set of practices and culture that merges Development and Operations, aiming to shorten the cycle above and deliver changes to production frequently, reliably, and with confidence — largely by automating as much of the cycle as possible.
- **CI/CD** — the automation backbone of DevOps, covered in full in Chapter 3 once SonarQube and Jenkins have been introduced.

As the diagram shows: **Git and GitLab** sit at the *Code* stage, giving a team a safe way to change the same source together; **Maven** sits at the *Build* stage, turning that source into a runnable artifact in a repeatable way; **SonarQube** sits at the *Test* stage, giving an objective answer to whether the newly-built code is good enough; and **Jenkins** sits at the *Release* stage, automating the whole sequence so it happens the same way, every time, without a human having to run each step by hand. The rest of this volume covers each of these four tools in the order they naturally appear in that loop.

1.2 Introduction to Version Control Systems

A **version control system (VCS)** is a tool that records changes to a set of files over time, so that a specific version can be recalled later, changes from multiple people can be combined safely, and the full history of *who changed what, when, and why* is always available. Before VCS tools were common, teams either edited a single shared copy of the code directly (dangerous — one mistake, or two people editing at once, could destroy work) or kept manual copies such as `project_v1`, `project_v2_final`, `project_v2_final_FINAL` — a pattern that scales badly and loses all fine-grained history.

Centralized vs. Distributed Version Control

Centralized VCS (e.g. Subversion, CVS) keeps a single central copy of the entire project history on a server; every developer's working copy is just the current checked-out files, and almost every operation — including viewing history — requires network access to that central server.

Distributed VCS (e.g. Git, Mercurial) gives every developer a full copy of the *entire* repository history on their own machine. Commits, branching, and viewing history all happen locally and instantly; a network connection is only needed to synchronise with other copies (typically a shared *remote* such as GitLab). This is why Git commands like `git log` and `git diff` work even with no internet connection at all.

1.3 Introduction to Git

Git is the distributed version control system created by Linus Torvalds in 2005 (originally to manage the Linux kernel's own source code) and is by far the most widely used VCS in the industry today. Git does not store a list of file-by-file differences the way some older systems do; instead, every commit is a **snapshot** of the entire project at that point in time. If a file has not changed between two commits, Git simply stores a reference to the identical file it already has, rather than storing it again — which is part of why Git repositories stay compact even with a long history.

1.3.1 The Four-Stage Git Architecture

Every Git-tracked project involves up to four distinct areas, and understanding the movement of changes between them is the single most important mental model for using Git correctly.

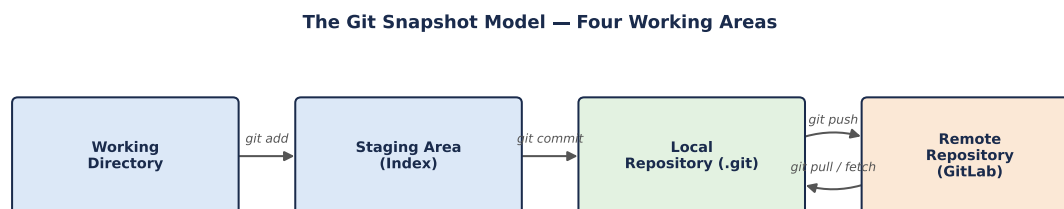


Figure 1.2: The four working areas in Git and the commands that move changes between them

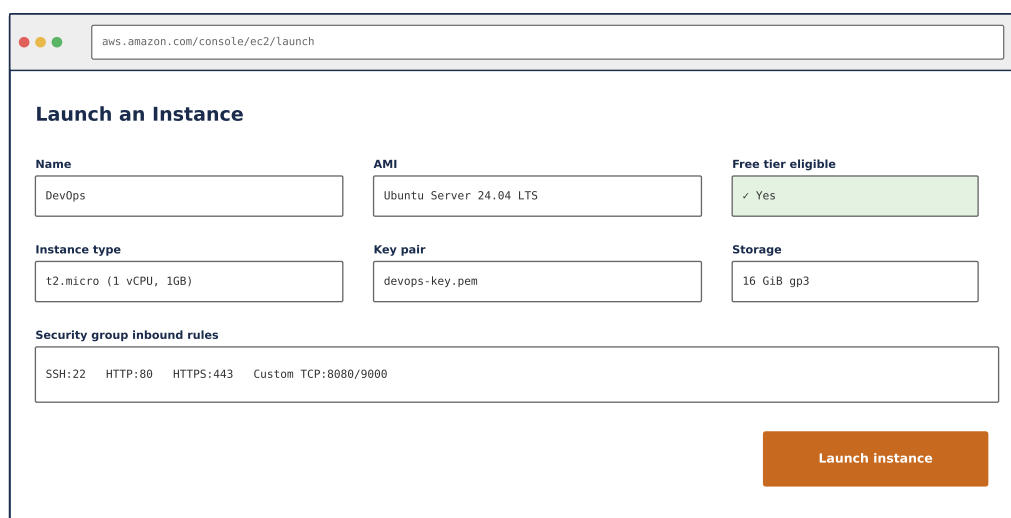
The Four Areas, Explained

- **Working Directory** — the actual files on disk that you edit in your IDE or text editor.
- **Staging Area (Index)** — a holding area where you deliberately choose exactly which changes will go into the *next* commit, using `git add`. This lets you build a commit out of only some of your changed files, or even only part of a file.
- **Local Repository (.git directory)** — the full history of committed snapshots, stored entirely on your own machine. `git commit` moves what is staged into a new permanent snapshot here.
- **Remote Repository** — a copy of the repository hosted elsewhere (GitLab, GitHub, Bitbucket) that other collaborators also synchronise with, using `git push` and `git pull/git fetch`.

1.3.2 Every Commit Has an Identity

Each commit is identified by a SHA-1 hash (a 40-character hexadecimal string) computed from the snapshot's content, its metadata (author, timestamp, commit message), and a pointer to its parent commit(s). Because the hash depends on the parent, commit history in Git forms an immutable, tamper-evident chain — changing any earlier commit changes its hash, which in turn changes every hash after it. This is also why two commits with identical file content but different parents or timestamps get different hashes.

1.3.3 Activity 1: Installing Git on a Cloud Virtual Machine



The image is a mockup of the AWS Management Console 'Launch an Instance' page. At the top, the browser address bar shows 'aws.amazon.com/console/ec2/launch'. The main content area is titled 'Launch an Instance'. It contains several input fields and a 'Launch instance' button. The fields are organized as follows:

Name	AMI	Free tier eligible
DevOps	Ubuntu Server 24.04 LTS	✓ Yes

Instance type	Key pair	Storage
t2.micro (1 vCPU, 1GB)	devops-key.pem	16 GiB gp3

Below these fields is a section for 'Security group inbound rules' with a text box containing: 'SSH:22 HTTP:80 HTTPS:443 Custom TCP:8080/9000'. At the bottom right is an orange 'Launch instance' button.

Illustrative mockup of the AWS EC2 "Launch an Instance" screen

Figure 1.3: Launching the Ubuntu EC2 instance that Git (and, in later chapters, SonarQube and Jenkins) will be installed onto

Activity 1 — Git Installation

1. Launch an Ubuntu 22.04/24.04 LTS EC2 instance on AWS (Free Tier eligible t2.micro), following the same key-pair and security-group process used for any EC2 instance in this course.
2. Connect to the instance (EC2 Instance Connect, or SSH with the downloaded .pem key).
3. Update the package index and install Java (a prerequisite for several later tools in this course):

```
sudo apt update
sudo apt install openjdk-17-jdk -y
```

4. Install Git itself:

```
sudo apt install git -y
```

5. Verify the installation and check the installed version:

```
git --version
```

1.4 Creating a Local Repository

Before looking at the commands, it helps to see the states a single file actually moves through inside Git — this is the concrete, file-level version of the four-area picture shown a moment ago.

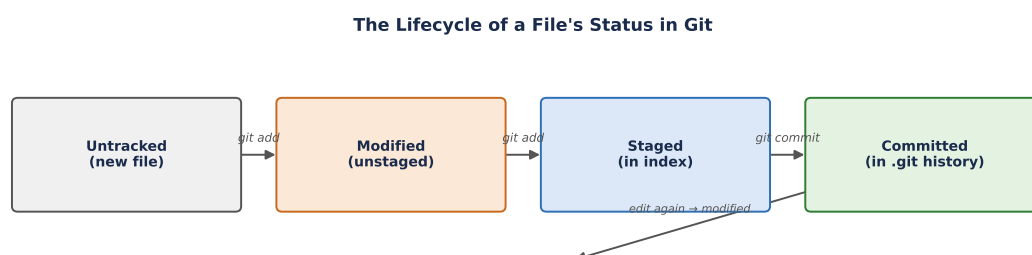


Figure 1.4: How a single file's status changes as it moves through untracked, modified, staged, and committed

Core Local Commands

- `git init` — turns the current directory into a Git repository by creating the hidden `.git` subdirectory that holds all repository metadata and history.
- `git add <file>` — stages a file's current content for the next commit.
- `git commit -m "message"` — creates a new permanent snapshot from whatever is currently staged, tagged with the given commit message.
- `git status` — shows the working directory and staging area state: which files are modified, staged, or untracked.
- `git log` — shows the commit history of the current branch.

A well-written commit message matters: the convention is a short (under ~50 character) summary line, because tools like `git log -oneline` and GitLab's commit list only show that first line.

Quick Reference — Everyday Git Commands

Command	What It Does
<code>git status</code>	Shows which files are modified, staged, or untracked right now
<code>git diff</code>	Shows the exact line-by-line changes not yet staged
<code>git diff -staged</code>	Shows the exact line-by-line changes that <i>are</i> staged, ready to commit
<code>git log -oneline</code>	Shows commit history as one compact line per commit
<code>git branch</code>	Lists all local branches; <code>git branch -a</code> also lists remote-tracking branches
<code>git checkout -b <name></code>	Creates a new branch and switches to it in one step
<code>git stash</code>	Temporarily shelves uncommitted changes so you can switch branches cleanly
<code>git reset -soft HEAD~1</code>	Undoes the last commit but keeps its changes staged
<code>git revert <commit></code>	Creates a new commit that safely undoes a previous one, without rewriting history

Activity 2 — Create Local Repository

1. Create and enter a new project directory: `mkdir My_Project && cd My_Project`
2. Initialise a Git repository: `git init`
3. Create an empty file: `touch My_File`
4. Stage the file: `git add My_File`
5. Commit the change: `git commit -m '<commit_message>'`
6. Confirm nothing is left uncommitted: `git status`

```

ubuntu@devops:~$
$ mkdir My_Project && cd My_Project
$ git init
Initialized empty Git repository in /home/ubuntu/My_Project/.git/
$ touch My_File
$ git add My_File
$ git commit -m 'Initial commit'
[master (root-commit) 3dd6f38] Initial commit
1 file changed, 0 insertions(+), 0 deletions(-)
$ git status
On branch master
nothing to commit, working tree clean

```

Illustrative terminal mockup: creating and committing a local Git repository

Figure 1.5: The terminal output of Activity 2 — the repository moves from an empty directory to one clean, committed snapshot

1.5 Branching in Git

A **branch** in Git is nothing more than a lightweight, movable pointer to a specific commit — this is precisely why creating a branch in Git is instantaneous and cheap, unlike some older VCS tools where branching meant physically copying the entire codebase. The special pointer **HEAD** always points to whichever commit is currently checked out, and normally **HEAD** points indirectly at a branch name, which in turn points at a commit.

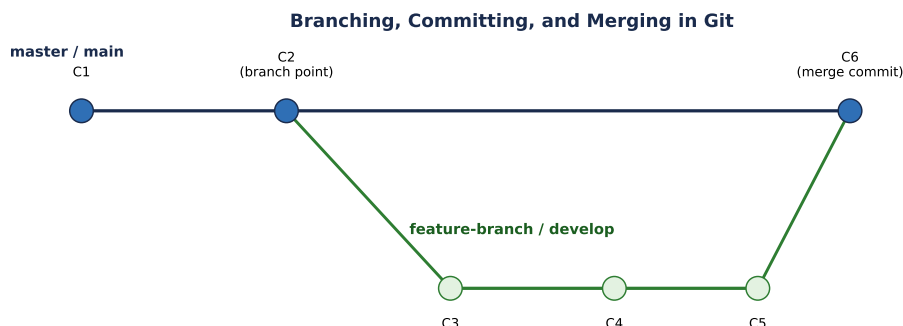


Figure 1.6: A feature branch diverging from and later merging back into the main branch

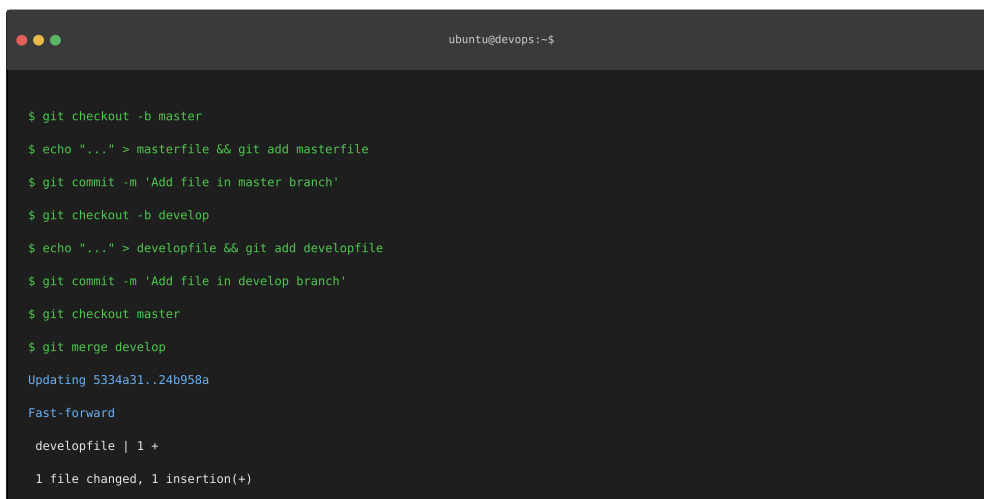
Fast-Forward vs. Three-Way Merge

When the target branch (e.g. **master**) has had no new commits since the feature branch diverged, Git can simply slide the **master** pointer forward to the feature branch's tip — a **fast-forward merge**, with no new merge commit created. When both branches have diverged with independent commits, Git instead performs a **three-way merge**: it compares the common ancestor commit with the tips of both branches and creates a new **merge commit** with two parents, as shown in the diagram above (commit C6).

Activity 3 — Create and Merge a Feature Branch

1. Initialise a repository and switch to (create) the **master** branch: `git checkout -b master`
2. Add and commit a file on **master**: `echo "." > masterfile && git add masterfile && git commit -m "Add file in master branch"`
3. Create and switch to a **develop** branch: `git checkout -b develop`
4. Add and commit a different file there: `echo "." > developfile && git add developfile && git commit -m "Add file in develop branch"`
5. Switch back to **master**: `git checkout master`
6. Merge **develop** into **master**: `git merge develop`
7. Confirm the working tree is clean: `git status`

Because **develop** only added new commits with no conflicting change on **master**, this merge is a clean fast-forward.



```
ubuntu@devops:~$  
  
$ git checkout -b master  
$ echo "..." > masterfile && git add masterfile  
$ git commit -m 'Add file in master branch'  
$ git checkout -b develop  
$ echo "..." > developfile && git add developfile  
$ git commit -m 'Add file in develop branch'  
$ git checkout master  
$ git merge develop  
Updating 5334a31..24b958a  
Fast-forward  
 developfile | 1 +  
 1 file changed, 1 insertion(+)
```

Illustrative terminal mockup: creating master and develop branches, then a clean fast-forward merge

Figure 1.7: The terminal session for Activity 3 — two branches created, and a clean fast-forward merge back into master

1.6 Resolving Merge Conflicts

Why Conflicts Happen

A merge conflict occurs specifically when *both* branches being merged have changed the **same lines** of the same file since they diverged — Git's three-way merge algorithm can automatically combine changes to different parts of a file, but it cannot guess which of two competing changes to the same line should win, so it stops and asks the developer to decide. Git marks the conflicting region directly inside the file using conflict markers:

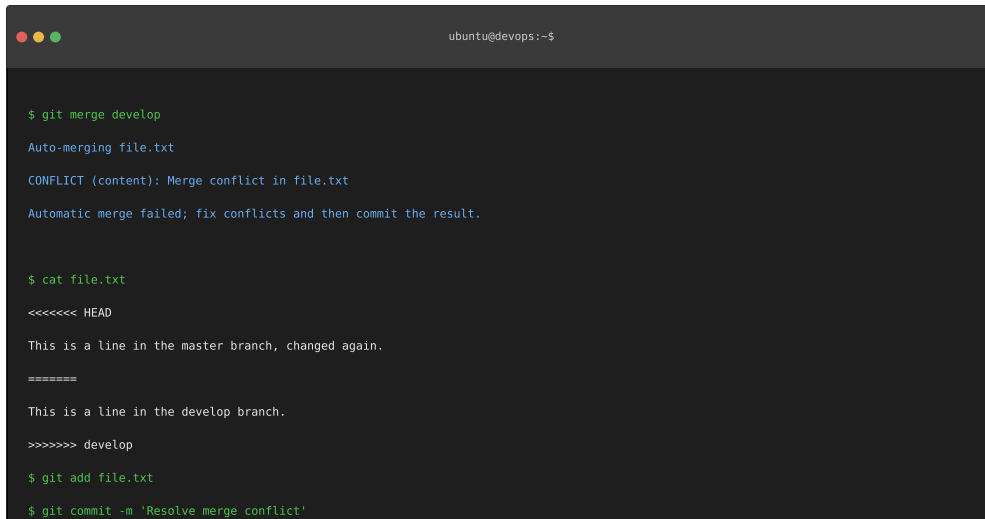
```
<<<<<< HEAD  
(content from the current branch)  
=====  
(content from the branch being merged in)  
>>>>>> develop
```

Resolving the conflict means manually editing the file to the intended final content, removing all three marker lines, then staging and committing the resolved file — which completes the merge.

Activity 4 — Reproducing and Resolving a Merge Conflict

1. Create the repository and **master** branch, then add **file.txt** with one line of content and commit it.
2. Create a **develop** branch and change the *same line* of **file.txt**, then commit.
3. Switch back to **master** and change the same line again to something else, then commit.
4. Attempt the merge: **git merge develop** — Git reports **CONFLICT (content): Merge conflict in file.txt**.

5. Open `file.txt`, manually decide the final content, and remove the conflict markers.
6. Stage and commit the resolution: `git add file.txt && git commit -m "Resolve merge conflict between master and develop branches"`

A terminal window titled 'ubuntu@devops:~\$' showing the following commands and output:

```
$ git merge develop
Auto-merging file.txt
CONFLICT (content): Merge conflict in file.txt
Automatic merge failed; fix conflicts and then commit the result.

$ cat file.txt
<<<<<< HEAD
This is a line in the master branch, changed again.
=====
This is a line in the develop branch.
>>>>>> develop

$ git add file.txt
$ git commit -m 'Resolve merge conflict'
```

Illustrative terminal mockup: a merge conflict appears, is inspected, then resolved and committed

Figure 1.8: The terminal session for Activity 4 — the conflict markers Git inserts, and the resolved, committed result

1.7 GitLab and Remote Repositories

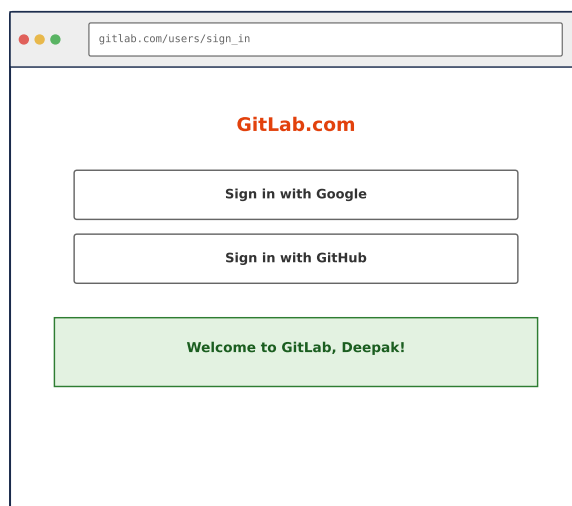
What a *Remote* Is

A **remote** is simply a named reference to another copy of the repository, most commonly hosted on a service such as GitLab, GitHub, or Bitbucket. The default name for the primary remote is conventionally `origin`. A remote-tracking branch such as `origin/main` is a local, read-only bookmark recording where that branch pointed on the remote the last time you communicated with it — it only updates when you **fetch** or **pull**.

1.7.1 Activity 5: Setting Up a GitLab Account

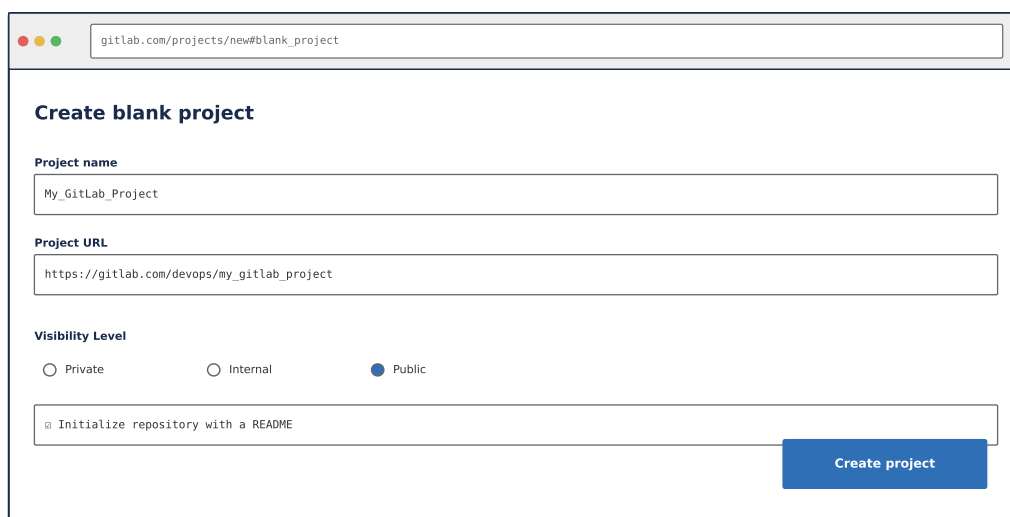
Activity 5 — GitLab Account Setup

1. Navigate to https://gitlab.com/users/sign_in.
2. Choose **Google** as the sign-in provider (or register a new account directly).
3. Select the Google account to use and confirm the OAuth consent screen.
4. On successful verification, GitLab redirects to the account's project dashboard.



Illustrative mockup: signing in to GitLab and landing on the project dashboard

Figure 1.9: Signing in to GitLab and landing on the project dashboard



Illustrative mockup of GitLab's "Create blank project" screen

Figure 1.10: Creating the blank GitLab project that the local repository will be pushed into

1.7.2 Activity 6: Pushing a Local Repository to GitLab

Activity 6 — Push Local Repository to a New GitLab Project

1. In GitLab, click **New project** → **Create blank project**.
2. Name the project (e.g. `My_GitLab_Project`) and choose its visibility level (Private / Internal / Public).
3. Copy the project's HTTPS clone URL from the **Code** dropdown on the project page.
4. Locally, initialise a repository, create `master`, and commit a file:

```
git init
git checkout -b master
echo "This is a file in the master branch." > file.txt
git add file.txt
git commit -m "Add file.txt in master branch"
```

5. Link the local repository to the GitLab remote:

```
git remote add origin https://gitlab.com/<group>/My_GitLab_Project.git
```

6. Push the local branch and set it to track the remote:

```
git push -u origin master
```

7. Refresh the GitLab project page and confirm the files now appear in the repository.

The `-u` (`-set-upstream`) flag links the local `master` branch to `origin/master` permanently, so future plain `git push` / `git pull` commands on this branch no longer need to specify the remote and branch name.

1.7.3 Activity 7: Cloning an Existing GitLab Repository

Activity 7 — Clone, Modify, and Push

1. Create a new blank project on GitLab (e.g. `MyProject_ToClone`) with a README included.
2. Copy its HTTPS clone URL and run: `git clone https://gitlab.com/<user>/repository.git`
3. Change into the cloned directory: `cd myproject_toclone`
4. Create/modify a file, then stage and commit it:

```
touch file.txt
echo "Hello, GitLab!" > file.txt
git add file.txt
git commit -m "file is added in repo"
```

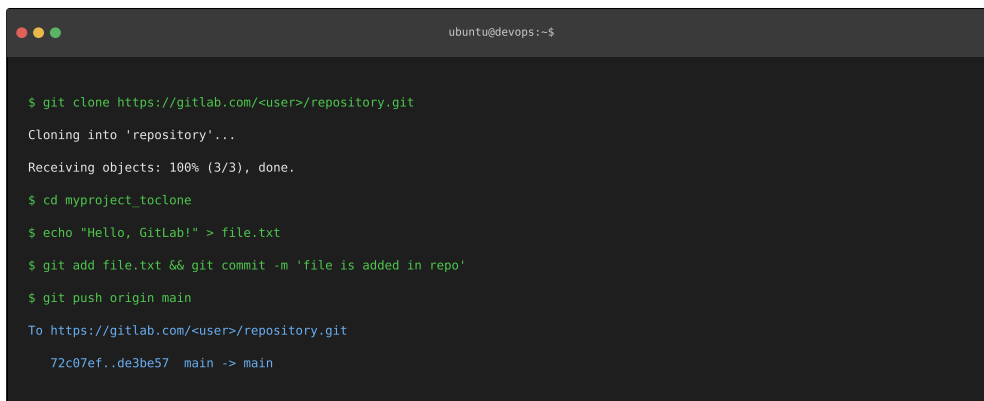
5. Push the change back: `git push origin main`
6. Refresh the GitLab project page to confirm the change is visible remotely.

`git clone` is a single command that both downloads the entire repository history and automatically configures `origin` to point back at the source URL — unlike Activity 6, no manual `git remote add` step is needed after a clone.

1.8 Synchronising with the Remote — Fetch vs. Pull

git fetch vs. git pull

`git fetch origin` downloads any new commits and updates the remote-tracking branches (e.g. `origin/main`) but leaves the current working branch and working directory completely untouched — it is always safe to run. `git pull` is shorthand for `git fetch` immediately followed by `git merge` (merging the fetched remote-tracking branch into the current local branch), so it *does* change local files and can create a merge commit or a conflict. The safer workflow in a team setting is often to `fetch` first, inspect what changed with `git log origin/main`, and only then decide to merge.



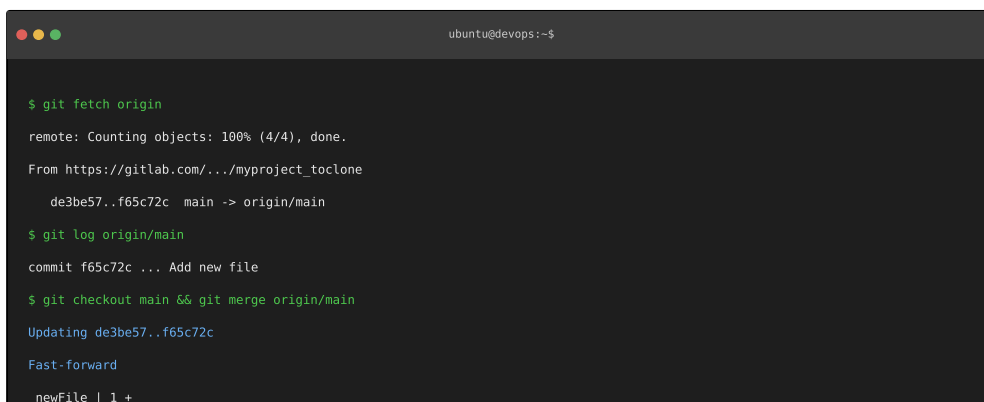
```
ubuntu@devops:~$  
  
$ git clone https://gitlab.com/<user>/repository.git  
Cloning into 'repository'...  
Receiving objects: 100% (3/3), done.  
  
$ cd myproject_tocloned  
  
$ echo "Hello, GitLab!" > file.txt  
  
$ git add file.txt && git commit -m 'file is added in repo'  
  
$ git push origin main  
To https://gitlab.com/<user>/repository.git  
72c07ef..de3be57  main -> main
```

Illustrative terminal mockup: cloning an existing GitLab repository, then committing and pushing a change back

Figure 1.11: The terminal session for Activity 7 — cloning an existing GitLab repository, then committing and pushing a change back

Activity 8 — Making Remote Changes and Syncing Locally

1. On the GitLab web UI, create a new file directly in the repository's **main** branch and commit it there (simulating a teammate's remote change).
2. Locally, download the new commit without touching the working directory: `git fetch origin`
3. Inspect what arrived: `git log origin/main`
4. Switch to **main** and merge the fetched commit in: `git checkout main && git merge origin/main`
5. Confirm the new file now exists locally: `ls`
6. Note the shortcut: `git pull origin main` performs the fetch and merge steps above in a single command.



```
ubuntu@devops:~$  
  
$ git fetch origin  
remote: Counting objects: 100% (4/4), done.  
From https://gitlab.com/.../myproject_tocloned  
de3be57..f65c72c  main -> origin/main  
  
$ git log origin/main  
commit f65c72c ... Add new file  
  
$ git checkout main && git merge origin/main  
Updating de3be57..f65c72c  
Fast-forward  
newFile | 1 +
```

Illustrative terminal mockup: fetching a teammate's remote change, then merging it into the local main branch

Figure 1.12: The terminal session for Activity 8 — fetching a teammate's remote change, then merging it into the local main branch

1.9 Collaborative Workflows — Pull (Merge) Requests

Why Not Push Directly to main?

In a team, pushing straight to the shared **main/master** branch bypasses any review. Instead, a developer pushes their work to a separate **feature branch** and opens a **merge request** (GitLab's term; GitHub calls the identical concept a *pull request*) — a formal proposal to merge that branch into the target branch, which teammates can comment on, request changes to, and must typically approve before the merge is actually performed. This is also the natural point at which automated checks — a CI pipeline, and specifically a SonarQube quality gate as covered in Chapter 3 — are run against the proposed change before it is allowed in.

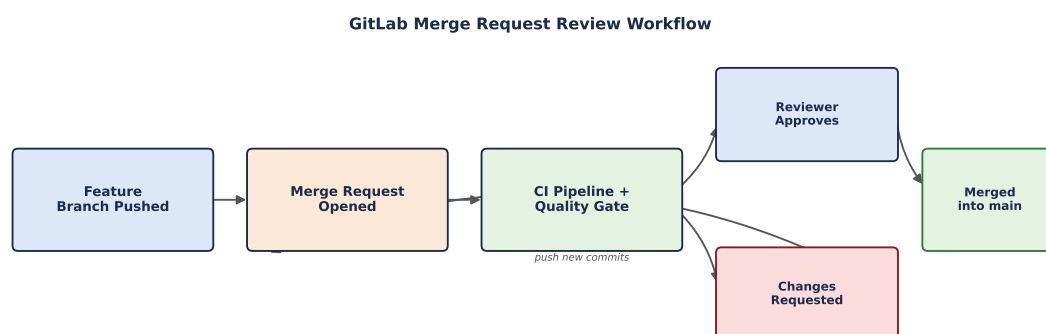


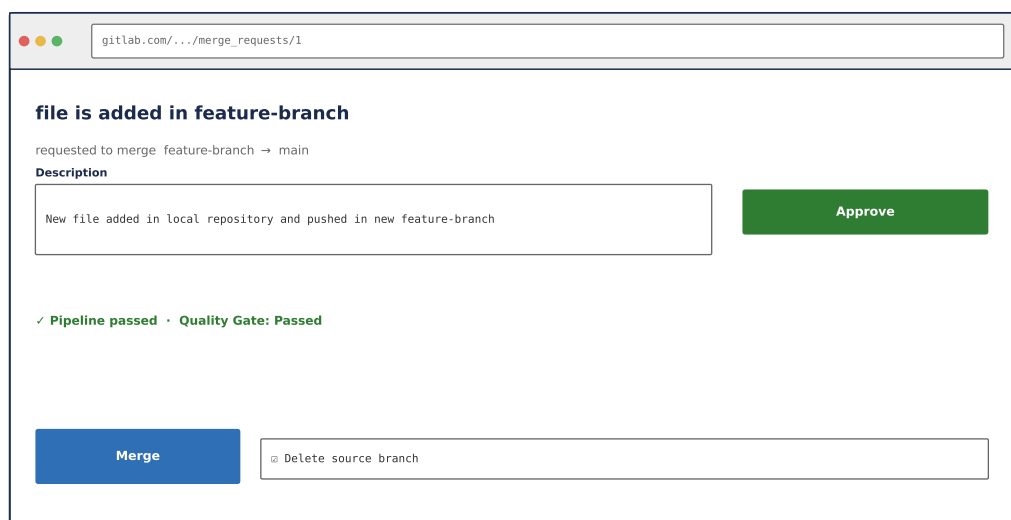
Figure 1.13: The typical review lifecycle of a GitLab merge request, from an opened request through to merging

Activity 9 — Opening and Merging a Pull (Merge) Request

1. Clone the target GitLab repository and create a new branch: `git checkout -b feature-branch`
2. Create and commit a file on that branch:

```
touch file.txt
echo "file is added in feature-branch" > file.txt
git add file.txt
git commit -m "file is added in feature-branch"
```

3. Push the feature branch: `git push origin feature-branch`
4. On the GitLab project page, click **Create merge request** (GitLab surfaces this automatically right after a new branch is pushed).
5. Fill in a clear title and description explaining the change, then click **Create merge request**.
6. Once reviewed, click **Merge**. GitLab merges the branch into **main** and — if the *delete source branch* option was left checked — automatically deletes the now-merged **feature-branch**.



Illustrative mockup of a GitLab merge request, ready to merge

Figure 1.14: The merge request just before it is merged — a passed pipeline and quality gate shown alongside the review

1.10 Git Tags and Release Versioning

Tags vs. Branches

Both tags and branches are pointers to a commit, but a branch pointer is expected to *move* forward as new commits are added, while a tag is meant to be a permanent, unmoving marker for one specific commit — typically used to mark a released version (e.g. `v1.5`). Unlike branches, tags are *not* pushed to a remote automatically by a plain `git push`; they must be pushed explicitly. Most real-world projects follow **Semantic Versioning** (MAJOR.MINOR.PATCH), where the major number increases for breaking changes, the minor number for backward-compatible new features, and the patch number for backward-compatible bug fixes.

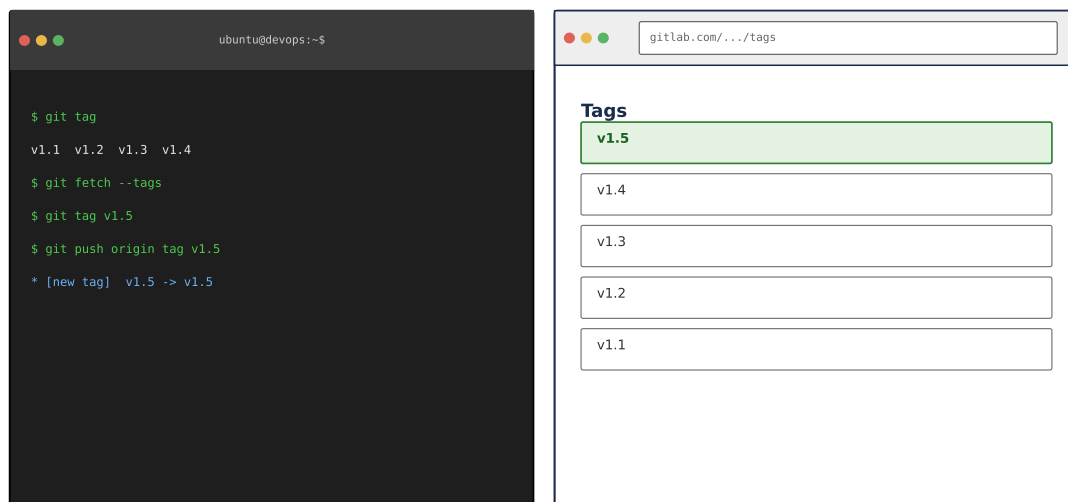
Activity 10 — Creating and Pushing a Git Tag

1. Clone the project repository whose release is being marked.
2. List existing tags: `git tag`
3. If the local tag list may be out of date, refresh it from the remote: `git fetch -tags`
4. Create the new tag locally: `git tag v1.5`
5. Push the new tag to the remote: `git push origin tag v1.5`
6. Confirm on the GitLab project's **Tags** page that `v1.5` is now listed, and optionally use **Create release** against that tag to publish formal release notes.

Viva Voce — Git & GitLab

Q1. What is the SDLC, and how does DevOps relate to it?

A: The SDLC (Software Development Life Cycle) is the universal sequence of require-



Illustrative mockup: pushing a new tag from the terminal, and the resulting GitLab Tags page

Figure 1.15: The terminal session for Activity 10, and the resulting GitLab Tags page showing the new v1.5 tag

ments, design, implementation, testing, deployment, and maintenance that any piece of software goes through; DevOps is a set of practices and culture that merges development and operations to move through that cycle more frequently and reliably, largely through automation.

Q2. In the DevOps loop, which stage does Git sit at, and which does SonarQube sit at?

A: Git (with GitLab) sits at the Code stage, giving a team a safe way to change shared source together; SonarQube sits at the Test stage, giving an objective, automated answer to whether newly built code meets the team's quality bar.

Q3. What is the fundamental difference between a centralized and a distributed version control system?

A: A centralized VCS keeps one authoritative history on a server and requires network access for most operations; a distributed VCS like Git gives every clone the complete history locally, so committing, branching, and browsing history all work offline.

Q4. Why does Git store snapshots rather than file-by-file diffs?

A: Because a full snapshot lets Git reconstruct any version of the project instantly by pointer lookup rather than by replaying a chain of diffs, and unchanged files are simply referenced again rather than duplicated, keeping storage efficient without sacrificing the simplicity of the snapshot model.

Q5. What are the four working areas in Git, and what command moves changes between the first two?

A: Working Directory, Staging Area, Local Repository, and Remote Repository; `git add` moves changes from the Working Directory into the Staging Area, and `git commit` moves staged changes into the Local Repository.

Q6. What, precisely, is a branch in Git?

A: A lightweight, movable pointer to a specific commit — creating one only writes a new small reference file, which is why branching in Git is nearly instantaneous regardless of repository size.

Q7. What is the difference between a fast-forward merge and a three-way merge?

A: A fast-forward merge is possible when the target branch has no new commits since the source branch diverged, so Git simply advances the pointer with no new commit; a three-way merge is required when both branches have diverged independently, and it produces a new merge commit with two parents.

Q8. Why do merge conflicts happen, and how does Git mark them?

A: They happen when both branches being merged changed the same lines of the same file since diverging, so Git cannot automatically decide which change should win; it inserts conflict markers directly into the affected file for the developer to resolve manually.

Q9. What is the practical difference between `git fetch` and `git pull`?

A: `git fetch` only downloads new commits and updates remote-tracking branches, leaving the working directory untouched; `git pull` is a fetch immediately followed by a merge into the current branch, so it can change local files or create a conflict.

Q10. What happens when you run `git clone` that does not happen with a plain `git init`?

A: `git clone` downloads the entire existing history from a remote URL and automatically configures that URL as the `origin` remote; `git init` only creates an empty local repository with no history and no remote configured.

Q11. Why does a team typically work through pull/merge requests instead of pushing straight to `main`?

A: A pull/merge request gives teammates a chance to review and comment on the change and lets automated checks — tests and a SonarQube quality gate — run against the proposed code before it is allowed to merge, rather than letting unreviewed code reach the shared main branch directly.

Q12. What is the key difference between a Git tag and a Git branch?

A: Both are pointers to a commit, but a branch is expected to move forward as new commits are added to it, whereas a tag is meant to permanently mark one specific commit — typically a release — and is not automatically pushed to a remote the way a branch's new commits are.

Chapter 2

Build Automation with Apache Maven

2.1 Why a Build Tool at All?

A non-trivial Java project has to be compiled in the correct dependency order, packaged into a distributable artifact (a JAR or WAR file), have its automated tests run, and pull in exactly the right versions of every third-party library it depends on — and every one of those steps has to produce the *same* result on every developer’s machine and on the CI server. **Apache Maven** is a build automation and project management tool for Java that solves this with two central ideas: a declarative project description (the POM) and a standard, convention-driven build lifecycle that every Maven project shares.

A Short History of Java Build Tools

Tool	Approach
Make (generic, pre-Java)	Build steps and dependencies between files written by hand as rules in a Makefile ; extremely flexible but nothing is standardised between projects.
Apache Ant (2000)	The first Java-specific build tool; build steps are written explicitly, in order, in an XML build.xml file — powerful, but every project reinvents its own structure and conventions.
Apache Maven (2004)	Introduces <i>convention over configuration</i> and a standard lifecycle (this chapter); most build logic is inherited for free simply by following the standard project layout.
Gradle (2009)	Uses a programmable Groovy/Kotlin DSL instead of XML, and builds an explicit task dependency graph; generally faster (incremental, cacheable builds) and more flexible than Maven, at the cost of being less strictly standardised across projects.

Maven remains extremely widely used in enterprise Java — including throughout this course — precisely because its rigid conventions make an unfamiliar Maven project fast to understand: **src/main/java** and **mvn clean install** mean the same thing in every Maven project, everywhere.

Convention Over Configuration

Maven’s guiding philosophy is *convention over configuration*: if a project follows Maven’s standard directory layout (**src/main/java**, **src/test/java**, **src/main/resources**, and

so on) and standard artifact naming, almost no configuration has to be written at all — Maven already knows where to find the source, where to put compiled classes, and how to package the result. This is also precisely why a fresh clone of any well-formed Maven project builds correctly with a single command on any machine that has Maven and the right JDK installed.

2.2 The Project Object Model (POM)

Every Maven project is described by a `pom.xml` file at its root — the **Project Object Model**. At minimum it declares:

- **Coordinates** — `groupId` (organisation/namespace), `artifactId` (the project's name), and `version`, which together uniquely identify this specific artifact, written conventionally as `groupId:artifactId:version` (GAV).
- **Packaging** — how the build output should be packaged, e.g. `jar`, `war`, or `pom` (for a parent/aggregator project with no code of its own).
- **Dependencies** — the other artifacts (by their own GAV coordinates) this project needs at compile time, test time, or runtime.
- **Properties** and **Plugins** — build-time configuration values and the tools that actually perform each build step.

A minimal example:

```
<project>
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.accenture.training</groupId>
  <artifactId>employee-service</artifactId>
  <version>1.0.0</version>
  <packaging>jar</packaging>

  <properties>
    <sonar.projectKey>employee-service</sonar.projectKey>
    <sonar.projectName>employee-service</sonar.projectName>
  </properties>

  <dependencies>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-web</artifactId>
      <version>3.2.0</version>
    </dependency>
  </dependencies>
</project>
```

This is exactly the pattern used when wiring a Spring Boot project into SonarQube in Chapter 3: the `sonar.projectKey` and `sonar.projectName` values are added under `<properties>` so that the SonarQube Maven plugin knows which project on the server to publish results to.

2.3 The Maven Build Lifecycle

Maven organises every build into a small number of named **phases**, executed strictly in order. A crucial rule follows from this: running any phase automatically runs every phase before it, so

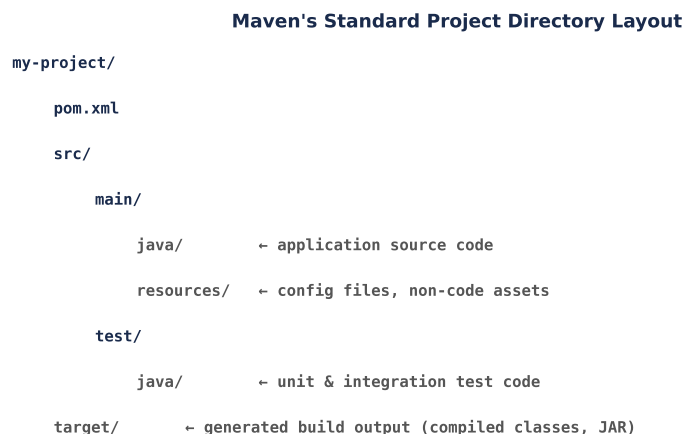


Figure 2.1: The standard directory layout every Maven project follows, so Maven needs almost no explicit path configuration

`mvn package` first validates, compiles, and tests the project before it ever attempts to package anything.

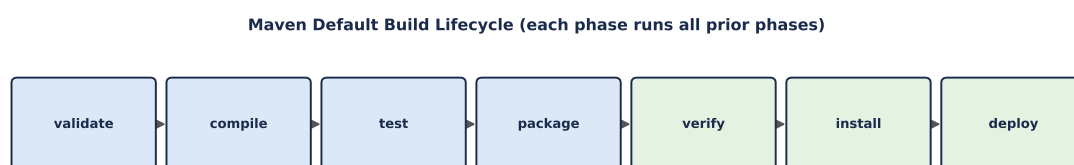


Figure 2.2: The default Maven build lifecycle, phase by phase

What Each Core Phase Does

- **validate** — checks that the project structure and `pom.xml` are correct and all necessary information is available.
- **compile** — compiles the project's main source code (`src/main/java`) into `target/classes`.
- **test** — runs unit tests (`src/test/java`) using a testing framework such as JUnit; a failing test fails the build here, before packaging ever happens.
- **package** — packages the compiled code into its distributable format, e.g. a JAR file in `target/`.
- **verify** — runs any additional checks on the results of integration tests, to confirm the package meets quality criteria.
- **install** — installs the packaged artifact into the *local* repository (`~/.m2/repository`), making it available as a dependency for other projects built on the same machine.
- **deploy** — copies the final package to a *remote* repository, for sharing with other

developers or downstream projects across a team or organisation.

2.4 Dependency Management and Repositories

Maven resolves every declared dependency automatically rather than requiring a developer to hunt down and manually place `.jar` files on the classpath.

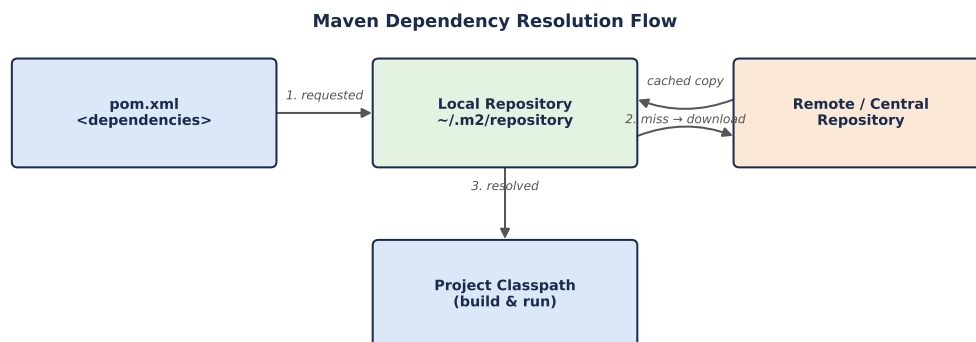


Figure 2.3: How Maven resolves a declared dependency

The Three Repository Types

- **Local repository** — a cache on the developer's own machine, by default `~/.m2/repository`, holding every artifact that project has ever needed.
- **Central repository** — Maven Central, the default public repository that hosts the vast majority of open-source Java libraries; Maven downloads from here automatically the first time an artifact is needed.
- **Remote/private repository** — an organisation's own artifact repository (e.g. Nexus or Artifactory), used to host internally-built artifacts or to proxy/cache Maven Central for a controlled, audited environment.

When a dependency is requested, Maven checks the local repository first; only on a cache miss does it reach out to a remote/central repository, download the artifact, and cache it locally for every future build.

Transitive Dependencies and Scope

If library A depends on library B, a project that depends only on A automatically also receives B on its classpath — a **transitive dependency** — without declaring B explicitly. Each dependency also has a **scope** that controls when it is available: **compile** (the default; available everywhere), **provided** (needed to compile but supplied by the runtime environment, e.g. a servlet container), **runtime** (not needed to compile but needed when running), and **test** (only available while compiling and running tests).

2.5 Multi-Module Projects and POM Inheritance

Real enterprise applications are rarely a single Maven project. A larger application is commonly split into several Maven **modules** (e.g. an API layer, a data-access layer, a shared utilities library), each with its own `pom.xml`, but all inheriting shared configuration from one common parent **POM**.

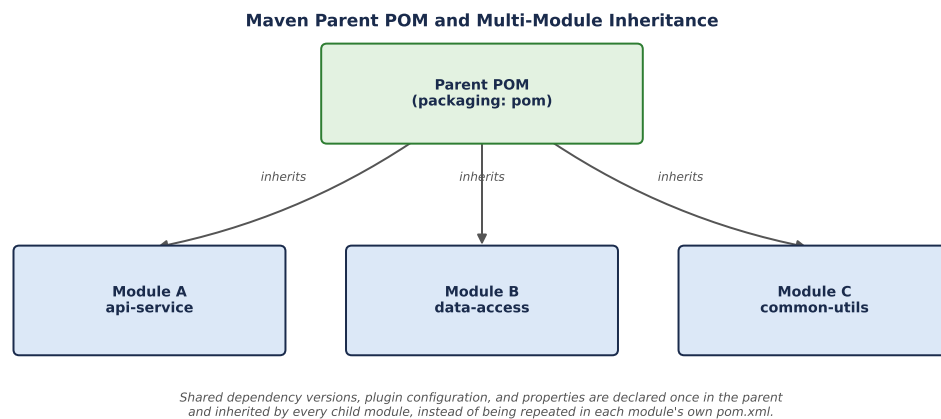


Figure 2.4: A parent POM's configuration is inherited by every child module rather than repeated in each one

What a Parent POM Typically Centralises

- **Dependency versions** — declared once inside `<dependencyManagement>` in the parent, so every child module can reference the same library without re-specifying its version, keeping every module on a consistent, compatible set of versions.
- **Plugin configuration** — build-tool plugin versions and settings (including, in this course's context, the SonarQube Maven plugin configuration) declared once and reused everywhere.
- **Common properties** — values like the Java version target, source encoding, or shared `sonar.*` settings.

The parent POM itself typically has `packaging` set to `pom` (rather than `jar` or `war`), since it exists purely to hold shared configuration and a list of its child `<modules>` — it produces no code of its own.

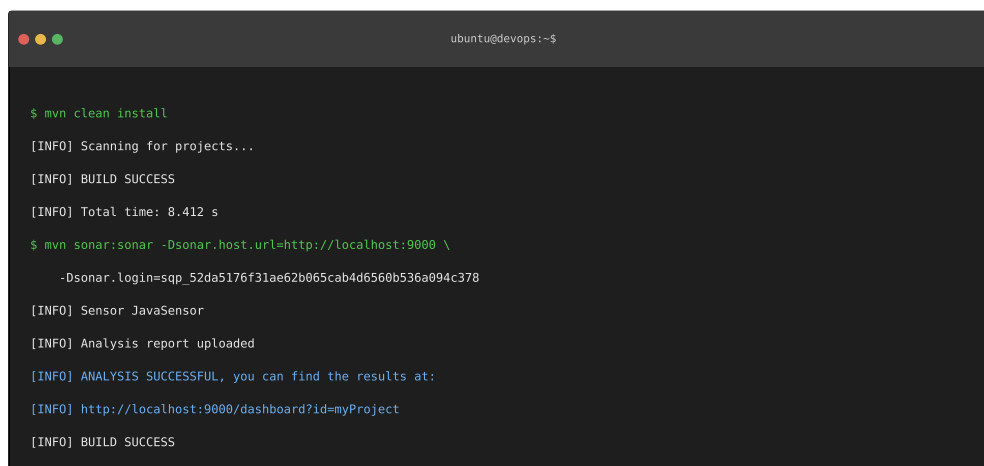
2.6 Maven Plugins and Goals

Every actual unit of work Maven performs — compiling, running tests, even integrating with SonarQube — is implemented by a **plugin**, and each specific task a plugin can perform is one of its **goals**, referenced as `plugin:goal` (e.g. `sonar:sonar`, the goal from the SonarQube Scanner for Maven plugin used throughout Chapter 3). Lifecycle phases are, in effect, just named hooks that plugin goals are bound to — the `compile` phase is bound to the `compiler:compile` goal, the `test` phase to `surefire:test`, and so on. A goal can also be invoked directly without going through the full lifecycle, exactly as `mvn sonar:sonar` does when triggering a SonarQube analysis on demand.

Key Maven Commands Used in This Course

- `mvn clean` — deletes the `target/` directory, removing all previous build output before a fresh build.
- `mvn clean install` — a very common combination: wipe old output, then compile, test, package, and install the artifact into the local repository in one command.
- `mvn clean verify sonar:sonar -Dsonar.host.url=... -Dsonar.token=...`

— runs the full build through `verify`, then explicitly invokes the SonarQube analysis goal, passing the SonarQube server URL and an authentication token as command-line properties (using `-D`).

A terminal window with a dark background and light green text. The prompt is 'ubuntu@devops:~\$'. The first command is '\$ mvn clean install', followed by output: '[INFO] Scanning for projects...', '[INFO] BUILD SUCCESS', and '[INFO] Total time: 8.412 s'. The second command is '\$ mvn sonar:sonar -Dsonar.host.url=http://localhost:9000 \ -Dsonar.login=sqp_52da5176f31ae62b065cab4d6560b536a094c378'. The output for this command is: '[INFO] Sensor JavaSensor', '[INFO] Analysis report uploaded', '[INFO] ANALYSIS SUCCESSFUL, you can find the results at:', '[INFO] http://localhost:9000/dashboard?id=myProject', and '[INFO] BUILD SUCCESS'.

Illustrative terminal mockup: building with Maven, then running the SonarQube analysis goal

Figure 2.5: A Maven build followed by a SonarQube analysis run from the command line — the exact combination used throughout Chapter 3

Viva Voce — Maven

Q1. How does Maven differ from an older tool like Apache Ant?

A: Ant requires every build step to be written out explicitly in an XML file with no standard project structure; Maven instead follows convention over configuration and a standard lifecycle, so an unfamiliar Maven project can usually be built and understood immediately, with far less explicit configuration.

Q2. What is a parent POM used for in a multi-module project?

A: It centralises configuration that would otherwise be repeated in every module — dependency versions, plugin configuration, and shared properties — which every child module then inherits; the parent POM itself typically has packaging set to `pom` since it produces no code of its own.

Q3. What problem does a build tool like Maven solve that manually compiling files does not?

A: It guarantees a repeatable, ordered build — compiling in the correct dependency order, running tests, resolving third-party library versions automatically, and packaging the result consistently on any machine, rather than relying on a developer to remember and repeat every step by hand.

Q4. What does “convention over configuration” mean in the context of Maven?

A: If a project follows Maven's standard directory layout and naming conventions, Maven already knows how to build it with almost no explicit configuration; configuration is only needed to override the defaults.

Q5. What are the three coordinates that uniquely identify a Maven artifact?

A: `groupId`, `artifactId`, and `version` — commonly written together as `groupId:artifactId:version` (GAV).

Q6. If you run `mvn package`, does Maven also run the `compile` and `test` phases?

A: Yes — Maven's build lifecycle is strictly ordered, and running any phase automatically runs every phase that precedes it, so `package` first validates, compiles, and tests the project.

Q7. What is the difference between `mvn install` and `mvn deploy`?

A: `install` copies the built artifact only into the local repository on the developer's own machine, for use by other local projects; `deploy` copies it to a remote/shared repository so other developers or systems can use it too.

Q8. What is a transitive dependency?

A: A dependency that is pulled in automatically because a directly-declared dependency itself depends on it, without the project having to declare it explicitly.

Q9. What is the difference between the `compile` and `provided` dependency scopes?

A: `compile` (the default) is available at compile time, test time, and runtime, and is packaged with the artifact; `provided` is available at compile time but is expected to be supplied by the runtime environment instead, so it is not packaged into the final artifact.

Q10. What is the relationship between a Maven plugin, a goal, and a lifecycle phase?

A: A plugin is a collection of related goals; a goal is one specific unit of work a plugin can perform (referenced as `plugin:goal`); a lifecycle phase is a named point in the build sequence that one or more goals are bound to, so that goal runs automatically whenever that phase runs.

Q11. What does the local Maven repository at `~/.m2/repository` actually store?

A: Every dependency artifact (and its own project's installed artifacts) that has ever been resolved on that machine, so that future builds needing the same artifact can be satisfied instantly from the local cache instead of downloading it again from Maven Central or a remote repository.

Chapter 3

Code Quality with SonarQube

3.1 Introduction to Static Code Analysis

Static code analysis examines source code without actually executing it, looking for patterns that indicate bugs, security vulnerabilities, and maintainability problems — as opposed to **dynamic analysis**, which observes a program’s behaviour while it actually runs (as unit and integration tests do). Static analysis can catch classes of problems that tests routinely miss, precisely because it does not depend on any particular test happening to exercise the buggy code path.

Key Terminology

- **Code smell** — a maintainability issue: code that works correctly today but is needlessly hard to read, extend, or safely modify (e.g. an excessively long method, duplicated logic, or — as flagged repeatedly in this course’s own activities — using `System.out.println` instead of a proper logger).
- **Bug** — code that is demonstrably likely to behave incorrectly at runtime, e.g. a possible null-pointer dereference or a resource that is never closed.
- **Vulnerability** — a weakness that could be exploited to compromise the application’s security, e.g. a SQL query built by string concatenation of user input.
- **Technical debt** — the accumulated, quantifiable cost of the extra future effort needed because of shortcuts, code smells, and unresolved issues left in the codebase today; SonarQube literally estimates this in a unit of time (e.g. “10min effort” on a single issue, as seen directly in this course’s own SonarQube activities).
- **Cyclomatic complexity** — a numeric measure of the number of independent paths through a method’s logic (branches, loops, conditionals); higher complexity correlates strongly with harder-to-test, harder-to-maintain code.

3.2 SonarQube Architecture

SonarQube is a self-hosted (or cloud) platform that continuously inspects code quality and security through static analysis, and presents the results through quality gates, dashboards, and issue-tracking, integrated directly into the build pipeline.

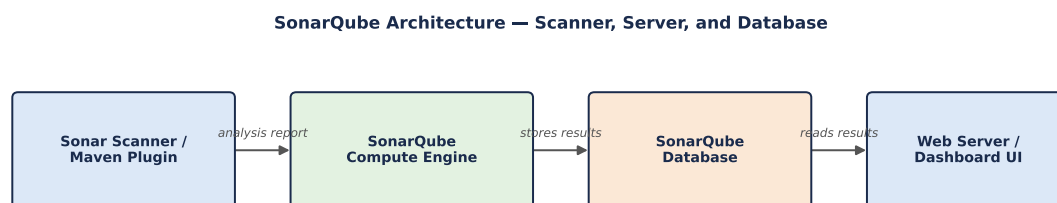


Figure 3.1: The four cooperating components of a SonarQube deployment

The Four Components

- **Scanner** — runs where the source code lives (a developer’s machine or, more commonly, a CI server) and performs the actual analysis, either via the standalone Sonar Scanner or a build-tool plugin (the Maven or Gradle SonarQube plugin), then uploads a compact analysis report to the server.
- **Compute Engine** — receives the uploaded report on the server side and processes it into the metrics, issues, and measures that make up the final analysis.
- **Database** — stores all processed analysis results, historical trend data, and configuration (quality profiles, quality gates, project settings).
- **Web Server / Dashboard** — the browser-based UI where results are reviewed: per-project dashboards, drill-down issue lists, and the pass/fail quality gate status.

Quality Profiles, Rules, and Quality Gates

A **rule** is a single specific check for one pattern (e.g. “Replace this use of `System.out` by a logger”). A **quality profile** is a named, language-specific set of rules that is actually activated for a given project — SonarQube ships sensible default profiles (“Sonar way”) per language. A **quality gate** is a set of pass/fail conditions evaluated against a project’s *latest* analysis — for example, “0 new bugs”, “code coverage on new code $\geq 80\%$ ”, or “duplicated lines $\leq 3\%$ ”. A project’s quality gate status of **Passed** or **Failed** is the single number a CI pipeline checks to decide whether a build may proceed — this is exactly the mechanism used to gate the Jenkins pipeline in the final activity of this chapter.

Severity Levels, Simply Put

Every issue SonarQube reports is assigned one of five severities, from least to most serious: **Info**, **Minor**, **Major**, **Critical**, and **Blocker**. A `System.out.println` left in place instead of a logger — the exact issue flagged repeatedly in this chapter’s own activities — is a routine example of a **Major** maintainability issue: real, worth fixing, but not something that will crash the application. A quality gate condition can be scoped to specific severities (e.g. “zero new Blocker or Critical issues”) so that trivial, low-severity findings do not by themselves block a release.

3.2.1 Code Coverage — A Closer Look

Code coverage is the percentage of a codebase’s lines (or branches) that were actually executed at least once while the automated test suite ran. SonarQube reports coverage by instrumenting

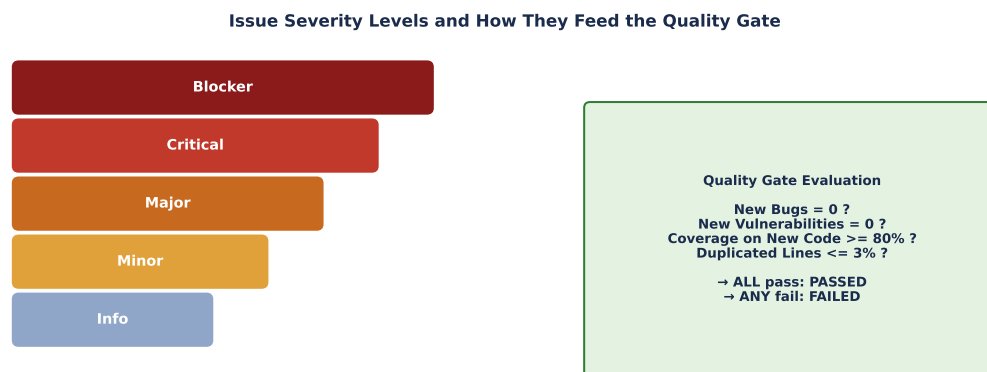


Figure 3.2: SonarQube’s five issue severity levels, and how conditions on the analysis feed into a single pass/fail quality gate decision

the test run (via a coverage tool such as JaCoCo for Java) and comparing which lines were touched against the project’s total lines.

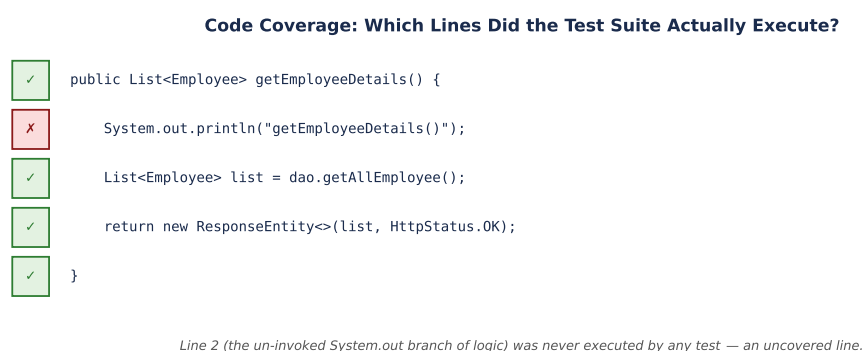


Figure 3.3: Coverage marks each line as executed (covered) or never executed (uncovered) by the test suite

Coverage Is Necessary, but Not Sufficient

High coverage means the test suite *executed* a line; it says nothing about whether the test that executed it actually asserted the right thing. A test that calls a method but checks nothing meaningful about its result still counts that method’s lines as “covered”. This is why a sensible quality gate combines a coverage condition (e.g. “coverage on new code $\geq 80\%$ ”) with separate conditions on bugs, vulnerabilities, and code smells, rather than relying on coverage as the only signal of quality.

“Clean as You Code” and New Code vs. Overall Code

Rather than demanding that an entire legacy codebase be perfect immediately, SonarQube’s default philosophy — **Clean as You Code** — focuses the quality gate primarily on **new code**: code added or changed since a defined baseline (by default, since the previous analysed version). This lets a team progressively improve an existing codebase’s overall quality simply by holding every new change to a high standard, without being

blocked by pre-existing legacy issues that are out of scope for the current change.

3.3 Activity 1: Installing SonarQube on AWS

Activity 1 — SonarQube Installation on an EC2 Instance

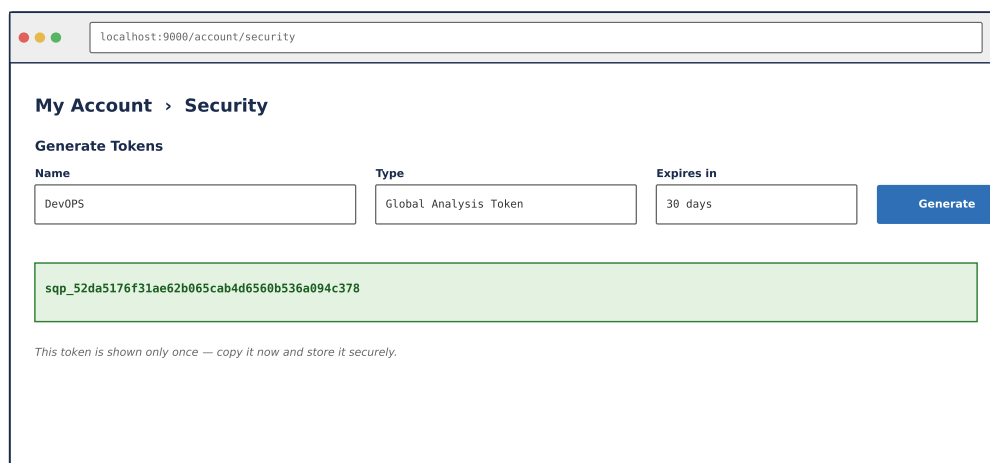
1. Launch an Ubuntu 22.04/24.04 LTS **t2.micro** EC2 instance (as in earlier activities), opening inbound ports 22 (SSH), and 9000 (SonarQube's default web port) in its security group.
2. Connect to the instance and install Java: `sudo apt update && sudo apt install openjdk-17-jdk -y`
3. Install Docker (SonarQube is distributed as an official Docker image):

```
sudo apt install docker.io
sudo usermod -aG docker $USER
sudo reboot
```

4. After reconnecting, run the SonarQube container, mapping port 9000 and persisting its extensions volume:

```
docker run --rm -d -p 9000:9000 \
-v sonarqube_extensions:/opt/sonarqube/extensions sonarqube
```

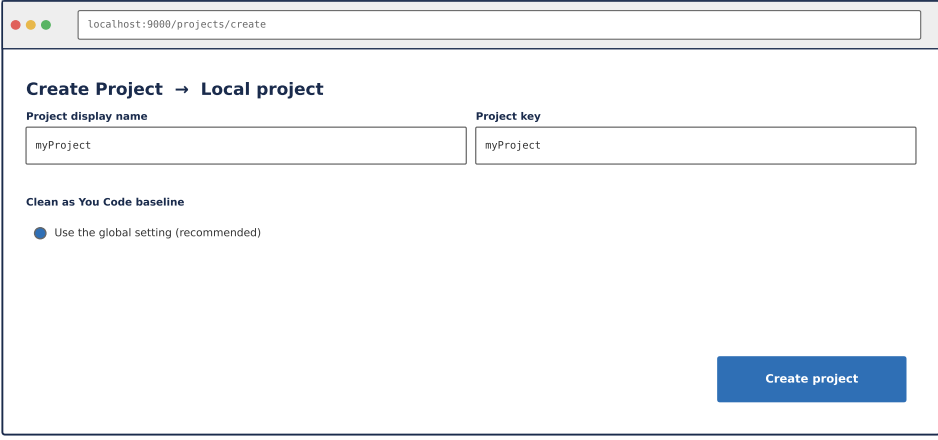
5. Browse to `http://<external-ip>:9000` to reach the SonarQube web UI.
6. Log in with the default credentials (`admin/admin`) and set a new password when prompted.
7. Under **My Account** → **Security**, name and generate a **Global Analysis Token** — this token authenticates a scanner to the server in place of a username/password, and should be copied and stored securely immediately, as it is shown only once.



Illustrative mockup of SonarQube's token-generation screen

Figure 3.4: Generating the analysis token from **My Account** → **Security** — shown once, and required by every scanner command that follows

3.4 Activity 2: Creating a Project in SonarQube



localhost:9000/projects/create

Create Project → Local project

Project display name
myProject

Project key
myProject

Clean as You Code baseline

☒ Use the global setting (recommended)

Create project

Illustrative mockup of SonarQube's "Create Project" screen

Figure 3.5: Creating a new local project in SonarQube, ready for its first analysis

Activity 2 — Project Creation and Analysis Method

1. Start (or reconnect to) the SonarQube server container and log in at <http://localhost:9000>.
2. Click **Create Project → Local project**, and supply a project name and a unique project key — these exact values are reused later as `sonar.projectKey` and `sonar.projectName` in the Spring Boot project's `pom.xml`.
3. On the **Clean as You Code** setup screen, choose **Use the global setting** so that any code changed since the previous version counts as new code, then click **Create project**.
4. On the **Analysis Method** screen, choose **Locally**, provide a token name, and click **Generate** to create a project-scoped analysis token; copy it for use in the next step.
5. Select the appropriate build tool (**Maven**) and copy the suggested analysis command, substituting in the project key, project name, server URL, and token:

```
mvn clean verify sonar:sonar \
  -Dsonar.projectKey=<Project_key> \
  -Dsonar.projectName=<Sonar_Project-Name> \
  -Dsonar.host.url=http://localhost:9000 \
  -Dsonar.token=<Sonar-Token>
```

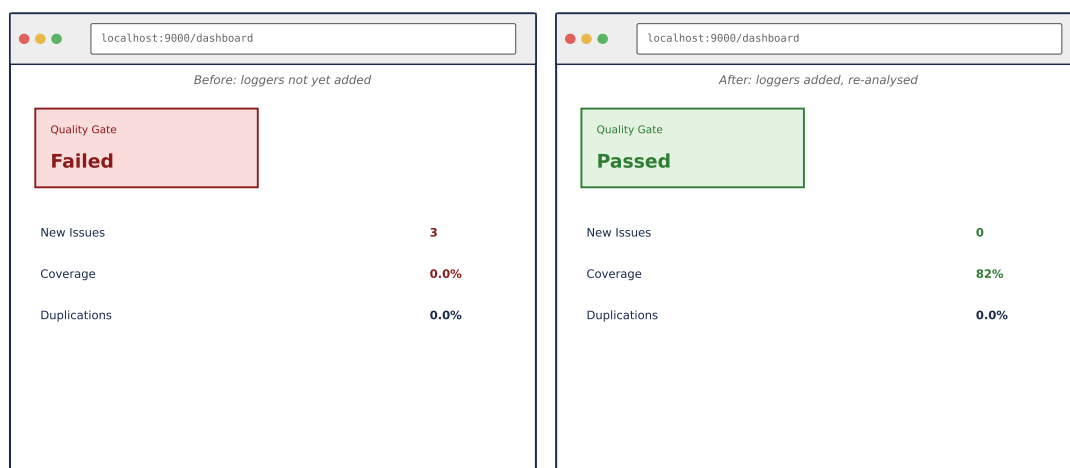
3.5 Activity 3: Static Code Analysis via the Maven SonarQube Plugin

Activity 3 — Analysing a Spring Boot Project with Maven

1. Import (or create) the Spring Boot project in the IDE.
2. Under `<properties>` in `pom.xml`, add the project's SonarQube key and name:

```
<sonar.projectKey>myProject</sonar.projectKey>
<sonar.projectName>myProject</sonar.projectName>
```
3. Build the project: `mvn clean install`
4. Run the analysis: `mvn sonar:sonar -Dsonar.host.url=http://localhost:9000 -Dsonar.login=<sonar_token>`
5. Open the SonarQube UI to review the result: an initial analysis of an unmodified sample controller reports a **Failed** quality gate, with three **Maintainability** code smells — each flagging a direct `System.out.println` call that should be replaced by a logger.
6. Fix the flagged lines in the IDE by introducing a proper logger, then repeat step 4 (re-run the analysis).
7. Re-open the SonarQube UI: the quality gate now reports **Passed**, with 0 new issues.

This before/after cycle is the central workflow of static analysis in practice: analyse, read the issues, fix them at the source, and re-analyse to confirm the fix actually resolved the reported problem.



Illustrative mockup: the same project's dashboard before and after fixing the flagged issues

Figure 3.6: The project dashboard before the fix (quality gate Failed) and after it (quality gate Passed)

3.6 Activity 4: Static Code Analysis via the Standalone Sonar Scanner

Maven Plugin vs. Standalone Sonar Scanner

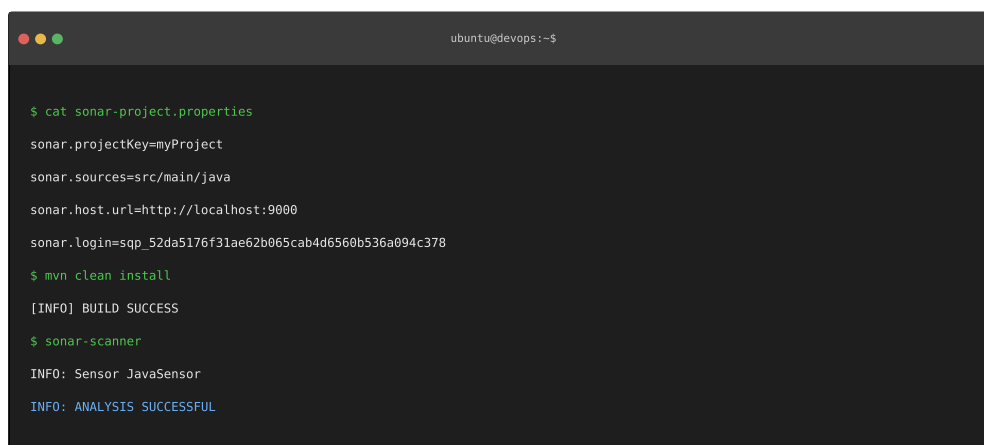
The Maven SonarQube plugin (used in Activity 3) is convenient for Maven projects because it reuses the build's own classpath and configuration automatically. The standalone **Sonar Scanner** instead reads its configuration from a plain `sonar-project.properties` file and can analyse *any* codebase — including non-Maven and non-Java projects — making it the more general-purpose tool of the two.

Activity 4 — Analysing with sonar-scanner

1. In the project's root directory, create `sonar-project.properties`:

```
sonar.projectKey=<Project-Key>
sonar.projectName=<Project-Name>
sonar.projectVersion=1.0
sonar.sources=src/main/java
sonar.java.binaries=target/classes
sonar.exclusions=**/test/**
sonar.sourceEncoding=UTF-8
sonar.host.url=http://localhost:9000
sonar.login=<sonar_token>
```

2. Build the project: `mvn clean install`
3. Run the scanner: `sonar-scanner`
4. Review the result in the SonarQube UI — as before, unaddressed `System.out` calls fail the quality gate; adding loggers and re-running `sonar-scanner` moves the gate to **Passed**.



```
ubuntu@devops:~$
$ cat sonar-project.properties
sonar.projectKey=myProject
sonar.sources=src/main/java
sonar.host.url=http://localhost:9000
sonar.login=sqp_52da5176f31ae62b065cab4d6560b536a094c378
$ mvn clean install
[INFO] BUILD SUCCESS
$ sonar-scanner
INFO: Sensor JavaSensor
INFO: ANALYSIS SUCCESSFUL
```

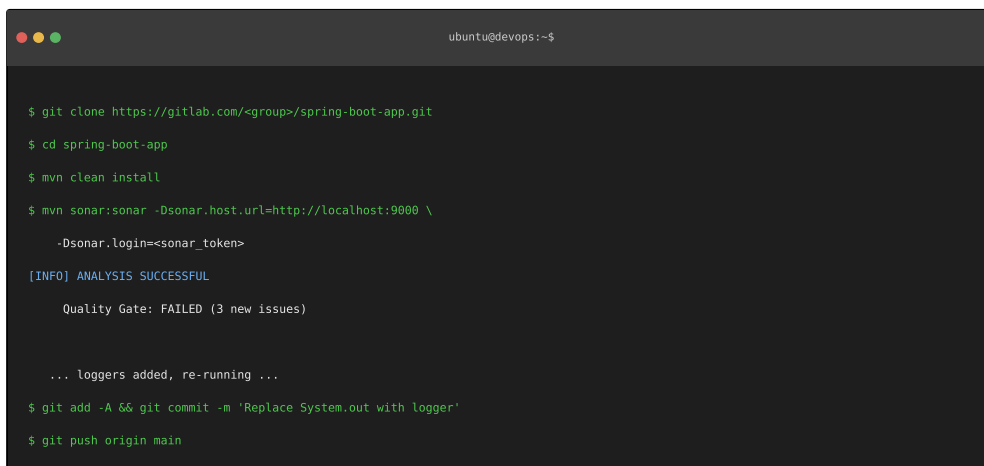
Illustrative terminal mockup: analysing a project using the standalone Sonar Scanner

Figure 3.7: The terminal session for Activity 4 — analysing with the standalone Sonar Scanner instead of the Maven plugin

3.7 Activity 5: Analysing a Project Cloned from GitLab

Activity 5 — End-to-End: Clone, Analyse, Fix, Push

1. Clone the target repository from its GitLab URL to obtain a local working copy.
2. Confirm appropriate test cases already exist in the project to validate functionality alongside the static analysis.
3. From the cloned repository's root, run the analysis using either approach covered above: **Option 1 (Maven plugin)**: `mvn clean install` then `mvn sonar:sonar -Dsonar.host.url=... -Dsonar.login=...`; **Option 2 (Sonar Scanner)**: create `sonar-project.properties` then run `mvn clean install` followed by `sonar-scanner`.
4. Review the SonarQube UI for the resulting quality gate status, coverage, and any reported issues.
5. Apply any necessary code fixes locally, re-run the analysis to confirm the issues clear, then commit and `git push` the fix to the GitLab origin branch — closing the loop between the version-control workflow of Chapter 1 and the quality-analysis workflow of this chapter.
6. Repeat the build/analyse/fix/push cycle until the project consistently passes its quality gate.



```
ubuntu@devops:~$  
  
$ git clone https://gitlab.com/<group>/spring-boot-app.git  
$ cd spring-boot-app  
$ mvn clean install  
$ mvn sonar:sonar -Dsonar.host.url=http://localhost:9000 \  
-Dsonar.login=<sonar_token>  
[INFO] ANALYSIS SUCCESSFUL  
Quality Gate: FAILED (3 new issues)  
  
... loggers added, re-running ...  
$ git add -A && git commit -m 'Replace System.out with logger'  
$ git push origin main
```

Illustrative terminal mockup: cloning a GitLab project, analysing it, fixing the flagged issues, and pushing the fix back

Figure 3.8: The terminal session for Activity 5 — cloning from GitLab, analysing, fixing flagged issues, and pushing the fix back

3.8 Activity 6: Integrating SonarQube and GitLab with Jenkins

Why Automate This in a Pipeline?

Running the analysis manually (Activities 3–5) works for learning the tool, but it depends entirely on a developer remembering to do it. Wiring the same `mvn sonar:sonar` step into a **Jenkins pipeline** means every push automatically triggers a checkout, build, and

analysis with no manual step — and, critically, the pipeline (or a downstream deployment step) can be configured to check the resulting quality gate status and block the release entirely if it fails, which is what actually makes a quality gate an *enforced* gate rather than just an advisory dashboard.

CI, Continuous Delivery, and Continuous Deployment — Not the Same Thing

These three terms are related but distinct, and the difference is entirely about how much of the release process is automated versus gated by a human decision:

- **Continuous Integration (CI)** — every developer's change is automatically built, tested, and analysed (exactly the checkout-build-analyse pipeline in this activity) as soon as it is pushed, so integration problems are caught within minutes rather than being discovered weeks later.
- **Continuous Delivery (CD)** — CI is extended so that a passing build is automatically packaged into a release-ready artifact and deployed as far as a staging environment, with production deployment still requiring a manual approval step.
- **Continuous Deployment** — goes one step further than Continuous Delivery: every change that passes every automated check (including the SonarQube quality gate) is deployed straight to production with no manual approval step at all.

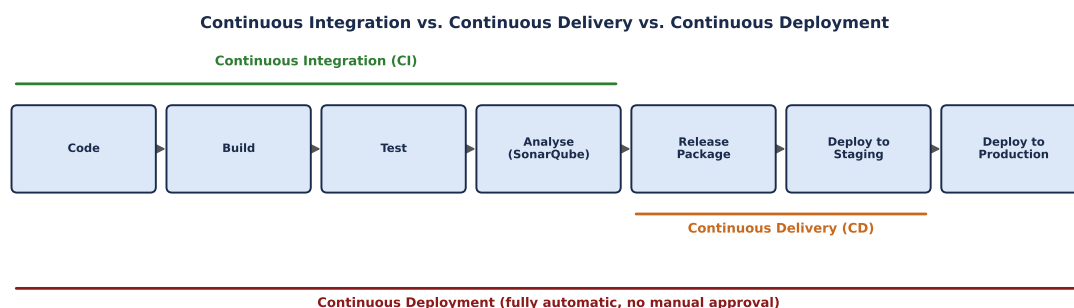


Figure 3.9: The same pipeline stages, showing exactly where CI ends and where Continuous Delivery and Continuous Deployment each extend it

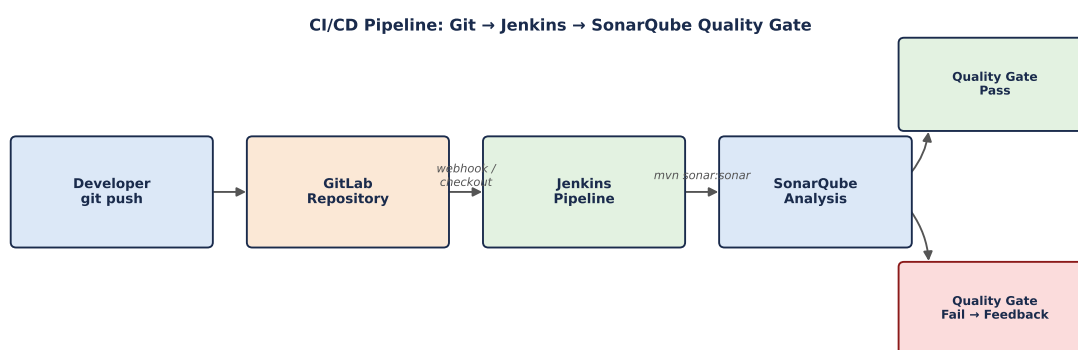
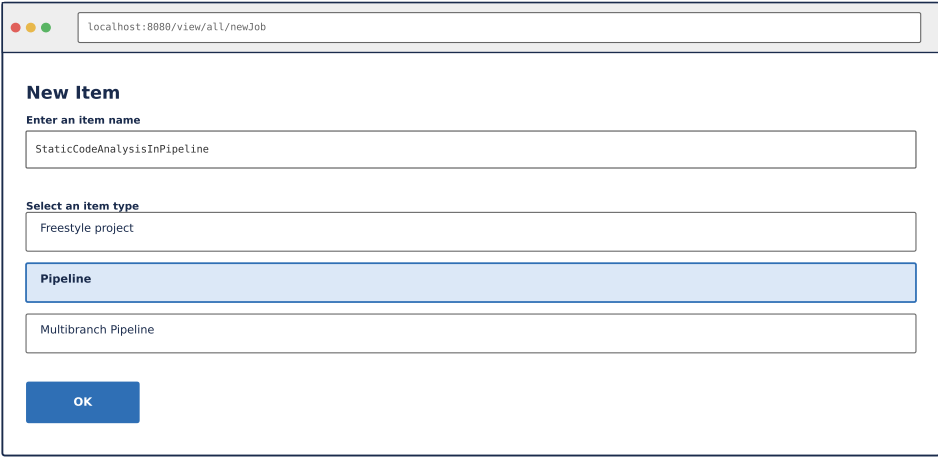


Figure 3.10: The complete CI/CD flow: a Git push triggers Jenkins, which runs the build and SonarQube analysis, and the resulting quality gate decides whether the change proceeds



The image shows a web browser window with the address bar displaying 'localhost:8080/view/all/newJob'. The main content area is titled 'New Item'. It contains a form with the following fields:

- Enter an item name:** A text input field containing 'StaticCodeAnalysisInPipeline'.
- Select an item type:** A dropdown menu with 'Freestyle project' selected.
- Pipeline:** A button labeled 'Pipeline' is highlighted in blue.
- Multibranch Pipeline:** A text input field containing 'Multibranch Pipeline'.
- OK:** A blue button at the bottom left.

Illustrative mockup of Jenkins' "New Item" screen, creating a Pipeline job

Figure 3.11: Creating a new Jenkins Pipeline item

Activity 6 — Building the Jenkins Pipeline

1. Log in to the Jenkins dashboard at <http://localhost:8080>.
2. Click **New Item**, name the item, select **Pipeline** as the project type, and click **OK**.
3. Enter a description explaining the pipeline's purpose.
4. Under the **Pipeline** section, set **Definition** to **Pipeline script** and paste a declarative pipeline with two stages — checkout and static analysis:

```
pipeline {
  agent any
  stages {
    stage('checkout source code') {
      steps {
        git branch: 'main', url: 'https://gitlab.com/<group>/<project>'
      }
    }
    stage('static code analysis') {
      steps {
        withMaven(maven: 'maven3.9.9') {
          sh 'mvn sonar:sonar -Dsonar.host.url=http://localhost:9000/ \
            -Dsonar.login=<SonarQube_token>'
        }
      }
    }
  }
}
```

5. Save the pipeline, then click **Build Now**.
6. Inspect the build's **Stage View** to confirm both the checkout and static-analysis stages completed successfully, and open the SonarQube UI to review the published result.
7. If the analysis reports failures, fix the code, commit and push to the GitLab origin branch, and trigger the build again — in a fully configured pipeline this last trigger step is itself automated via a GitLab webhook, so a push alone starts

the whole pipeline with no manual “Build Now” click required.

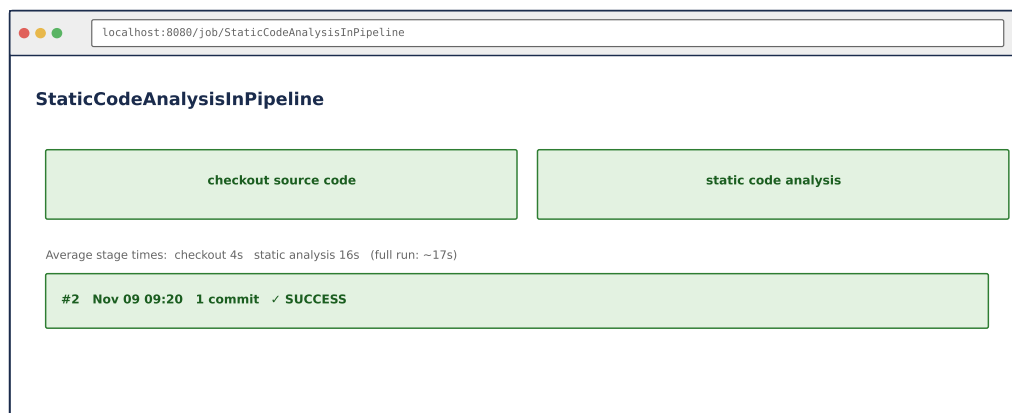


Figure 3.12: The pipeline’s Stage View after a successful run — both stages green, confirming the checkout and SonarQube analysis both completed

Viva Voce — SonarQube & CI/CD Quality Gates

Q1. What are SonarQube’s five issue severity levels, from least to most serious?

A: Info, Minor, Major, Critical, and Blocker; a quality gate condition can be scoped to only the more serious severities so that trivial low-severity findings do not by themselves block a release.

Q2. Does high code coverage guarantee the code is well tested?

A: No — coverage only measures whether a line was *executed* by the test suite, not whether the test that executed it actually asserted the correct behaviour; a test with no meaningful assertions can still produce high coverage, which is why coverage is only one of several quality gate conditions.

Q3. What is the difference between Continuous Integration, Continuous Delivery, and Continuous Deployment?

A: Continuous Integration automatically builds, tests, and analyses every change as it is pushed; Continuous Delivery extends this so a passing build is automatically packaged and deployed as far as staging, with production still requiring manual approval; Continuous Deployment removes that manual approval entirely, so every change that passes every automated check is deployed straight to production.

Q4. What is the difference between static and dynamic code analysis?

A: Static analysis examines source code without executing it, looking for structural issues, bugs, and vulnerabilities; dynamic analysis observes the program’s actual behaviour while it runs, which is what automated tests do.

Q5. What is a “code smell”, and how is it different from a bug?

A: A code smell is code that works correctly but is needlessly hard to read, extend, or safely change (e.g. duplicated logic or a print statement instead of a logger); a bug is code that is demonstrably likely to behave incorrectly at runtime.

Q6. What are the four main components of a SonarQube deployment?

A: The Scanner (runs the analysis and uploads a report), the Compute Engine (processes the report on the server), the Database (stores analysis results and configuration), and the Web Server/Dashboard (the UI where results are reviewed).

Q7. What is a SonarQube quality gate, and why does it matter for CI/CD?

A: A quality gate is a set of pass/fail conditions evaluated against a project's latest analysis (e.g. zero new bugs, minimum coverage on new code); a CI pipeline can check this single pass/fail status to automatically block a release that does not meet the team's quality bar.

Q8. What does SonarQube's "Clean as You Code" approach focus the quality gate on, and why?

A: It focuses primarily on new code — code added or changed since a defined baseline — so a team can progressively raise the quality of an existing codebase by holding every new change to a high standard, rather than being blocked trying to fix an entire legacy codebase at once.

Q9. What is the difference between using the Maven SonarQube plugin and the standalone Sonar Scanner?

A: The Maven plugin reuses the build's own classpath and configuration automatically and is convenient for Maven projects; the standalone Sonar Scanner reads configuration from a `sonar-project.properties` file and can analyse any codebase, including non-Maven and non-Java projects.

Q10. In the Maven analysis command `mvn sonar:sonar -Dsonar.host.url=... -Dsonar.login=...`, what is the purpose of the token passed via `sonar.login`?

A: It authenticates the scanner to the SonarQube server in place of a username and password, and is generated once from the user's account (or project) security settings and then reused in every analysis command.

Q11. Why integrate SonarQube into a Jenkins pipeline rather than running the analysis manually?

A: So the analysis happens automatically and consistently on every change with no manual step to remember, and so the pipeline can act on the resulting quality gate status — automatically blocking a release if the gate fails, which is what actually makes it an enforced gate rather than just an advisory report.

Q12. What does technical debt quantify in SonarQube's terms?

A: The estimated additional effort, expressed as a time value (e.g. minutes of remediation effort per issue), that it would take to fix all the currently outstanding code smells, bugs, and vulnerabilities in the codebase.

Q13. Why does fixing the flagged `System.out.println` calls in the sample project change the quality gate from Failed to Passed?

A: Because the quality gate condition on new code (e.g. “0 new issues” or a maintainability rating threshold) is evaluated against the latest analysis; once the flagged lines are replaced with proper logging and the project is re-analysed, those specific issues no longer exist in the new analysis, so the condition that previously failed now passes.

Java Full Stack Training

DevOps Module Study Guide

Infrastructure as Code with Terraform & Containerization with Docker

Theory • Hands-on Activities with Screenshots • Viva Voce Questions

Aligned to: QuantumCrew Java Full Stack + Gen AI — Day-Wise Plan (Rows 54 & 55)

Contents

Part 1 — Infrastructure as Code with Terraform

- 1. Introduction to Terraform
- 2. Terraform Configuration
- 3. Terraform Architecture
- 4. Hands-on Activities (6 activities)
- 5. Viva Voce Questions

Part 2 — Containerization with Docker

- 1. Introduction to Docker
- 2. Docker Architecture
- 3. Hands-on Activities (13 activities)
- 4. Viva Voce Questions

PART 1

Infrastructure as Code with Terraform

1. Introduction to Terraform

Terraform is an open-source Infrastructure as Code (IaC) tool developed by HashiCorp. It allows engineers to define, provision, and manage cloud and on-premises infrastructure using a simple, human-readable configuration language instead of manually clicking through cloud consoles.

Infrastructure as Code (IaC) is the practice of managing and provisioning infrastructure (servers, networks, databases, storage, IAM roles, etc.) through machine-readable definition files, rather than through manual, interactive configuration tools. This makes infrastructure repeatable, version-controlled, and auditable, just like application source code.

Why Terraform?

- Cloud-agnostic — works with AWS, Azure, GCP, and 100+ other providers using the same workflow.

- Declarative — you describe the desired end state; Terraform figures out how to get there.
- Execution plans — 'terraform plan' shows exactly what will change before anything is touched.
- Resource graph — Terraform builds a dependency graph and creates/updates/destroys resources in the correct order, in parallel where possible.
- State management — Terraform tracks the real-world resources it manages in a state file, enabling drift detection and safe updates.
- Reusability — configurations can be modularized and reused across projects and environments (dev/test/prod).

Key benefits of IaC with Terraform

- Consistency: eliminates configuration drift between environments.
- Speed: infrastructure can be stood up or torn down in minutes.
- Version control: .tf files can be stored in Git, enabling code review, rollback, and audit trails.
- Cost control: easy to destroy temporary/test environments to avoid unnecessary billing.
- Collaboration: teams work off a shared, versioned definition of infrastructure instead of tribal knowledge.

2. Terraform Configuration

Terraform configuration files use HashiCorp Configuration Language (HCL), a declarative language designed to be both human-readable and machine-friendly. Configuration is written in files with the '.tf' extension (commonly main.tf, variables.tf, outputs.tf, terraform.tfvars).

Core building blocks of a Terraform configuration

- Provider block — declares which cloud/platform Terraform should talk to (e.g., aws, azure, google) and its settings such as region and credentials.
- Resource block — the actual infrastructure object to create, e.g. aws_instance, aws_s3_bucket, aws_db_instance, aws_iam_role.
- Data block — reads/references existing infrastructure that Terraform did not create (read-only lookup).
- Variable block — declares input parameters so configuration can be reused with different values.
- Output block — exposes values (like an instance's public IP) after apply, useful for chaining or for the user to read.

- Module — a reusable, self-contained package of .tf configuration that can be called from other configurations.

Example structure

```
provider "aws" {
  region = "ap-south-1"
}

resource "aws_instance" "my_instance" {
  ami = "ami-0f58b397bc5c1f2e8"
  instance_type = "t3.micro"
}
```

Important files

- main.tf — primary configuration (providers + resources).
- variables.tf — input variable declarations.
- terraform.tfvars — actual values assigned to variables.
- outputs.tf — values to display after apply.
- terraform.tfstate — auto-generated file that records the current state of managed infrastructure. This file should never be edited manually.
- .terraform.lock.hcl — records exact provider versions used, ensuring consistent installs across machines.

3. Terraform Architecture

Terraform's architecture is built around a small core engine and a large ecosystem of plugins called providers.

Core components

- Terraform Core — reads the configuration files, builds a resource dependency graph (DAG), and determines the order of operations.
- Providers — plugins (aws, azure, google, mysql, kubernetes, etc.) that translate Terraform's generic resource calls into actual API calls against a specific platform.
- State file (terraform.tfstate) — a JSON file that maps configuration resources to real-world object IDs. Terraform compares this state with the actual configuration to compute the difference (diff) for plan/apply.
- Backend — defines where the state file is stored (local disk by default, or remote backends such as S3 + DynamoDB for team collaboration and locking).

Terraform workflow (the core commands)

- terraform init — downloads/installs the required provider plugins and initializes the working directory & backend.
- terraform plan — creates an execution plan, showing what will be added (+), changed (~), or destroyed (-), without making real changes.
- terraform apply — executes the plan and provisions/updates the actual infrastructure. '-auto-approve' skips the interactive yes/no prompt.
- terraform destroy — tears down all (or, with -target, specific) resources managed by the configuration.

Typical lifecycle diagram (conceptual)

Write configuration (.tf) → terraform init (download providers) → terraform plan (preview changes) → terraform apply (create resources) → manage/update → terraform destroy (clean up).

4. Hands-on Activities

Activity 1: Install Terraform on AWS

Prerequisite	AWS account is ready to use. AWS EC2 instance is ready to use.
Scenario	Launch Shell / EC2 Instance Connect on AWS to install Terraform on an Ubuntu EC2 instance.

Steps:

- Open the AWS Console → EC2 → Instances, search the instance by name/ID, and click Connect → EC2 Instance Connect to open a browser-based terminal.
- Update the system and install curl/unzip:

```
sudo apt update && sudo apt install -y curl unzip
```

- Download the Terraform binary (v1.7.3 for Linux amd64):

```
curl -fsSL https://releases.hashicorp.com/terraform/1.7.3/terraform_1.7.3_linux_amd64.zip -o terraform.zip
```

- Unzip the downloaded archive:

```
unzip terraform.zip
```

- Move the terraform binary into the system PATH:

```
sudo mv terraform /usr/local/bin/
```

- Verify the installation:

terraform -version

- Connect**

The screenshot shows the "EC2 Instance Connect" page. Under "Connection type", the "Connect using a Public IP" option is selected. The public IP address is 13.35.176.174.

Note: If you are using the Ubuntu Desktop (GUI) version, open a Terminal window and execute the commands listed below.
- Open EC2 Instance Connect**

```
i-06d7996c7bbf451e6 ssh -t ubuntu@linux-6-10-2-022-us-east-2a

# Documentation: https://help.ubuntu.com
# Management:   https://landscape.canonical.com
# Support:       https://ubuntu.com/support

System information as of Thu Oct 30 12:30:14 UTC 2020

system info: 0.33          Temperature: -273.1 C
kernel:      5.10.0-rc7    Processor:
memory usage: 22%         Swap: Logged in:
disk space:  25           Free software for: Linux 6.10-2-022-a

Canonical Security Maintenance for Applications is not enabled.

Updates can be applied immediately.

To enable ESM Apps to receive additional future security updates,
run: sudo apt install esm-apps and sudo apt update

i-06d7996c7bbf451e6 (my-ubuntu-vn)
```
- Update system**

Command: `sudo apt update && sudo apt install -y curl unzip`

```
ubuntu@ip-172-31-23-78:~$ sudo apt update && sudo apt install -y curl unzip
Hit:1 http://re-mirror-1.ec2.archive.ubuntu.com/ubuntu noble-updates InRelease [106 kB]
Hit:2 http://re-mirror-1.ec2.archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Hit:3 http://re-mirror-1.ec2.archive.ubuntu.com/ubuntu noble/main amd64 Packages [115.0 MB]
Hit:4 http://re-mirror-1.ec2.archive.ubuntu.com/ubuntu noble-security InRelease [112 kB]
Get:5 http://security.ubuntu.com/ubuntu noble-security InRelease [112 kB]
Get:6 http://re-mirror-1.ec2.archive.ubuntu.com/ubuntu noble/main amd64 Packages Translation-en [5902 kb]
Get:7 http://re-mirror-1.ec2.archive.ubuntu.com/ubuntu noble/main amd64 Components [3911 kb]
Get:8 http://re-mirror-1.ec2.archive.ubuntu.com/ubuntu noble/main amd64 C-s-d Metadata [203 kb]
Get:9 http://re-mirror-1.ec2.archive.ubuntu.com/ubuntu noble/main amd64 Packages [115.0 MB]
Get:10 http://re-mirror-1.ec2.archive.ubuntu.com/ubuntu noble/main amd64 Packages Translation-en [1121 kb]
Get:11 http://re-mirror-1.ec2.archive.ubuntu.com/ubuntu noble/main amd64 Packages [115.0 MB]
Get:12 http://re-mirror-1.ec2.archive.ubuntu.com/ubuntu noble/main amd64 Packages Translation-en [1121 kb]
Get:13 http://re-mirror-1.ec2.archive.ubuntu.com/ubuntu noble-updates/main amd64 Packages [1257 kb]
Get:14 http://re-mirror-1.ec2.archive.ubuntu.com/ubuntu noble-updates/main Translation-en [232 kb]
Get:15 http://re-mirror-1.ec2.archive.ubuntu.com/ubuntu noble-updates/main amd64 Components [1175 kb]
```
- Download Terraform**

Command:

```
curl -fsSL https://releases.hashicorp.com/terraform/1.7.3/terraform_1.7.3_linux_amd64.zip -o terraform.zip
```

Downloading, unzipping and verifying the Terraform installation on the EC2 instance

Activity 2: Authenticate & Configure Terraform with AWS using Terminal

Prerequisite	Terraform is installed on the AWS instance.
Scenario	Authenticate and configure Terraform with AWS credentials so Terraform scripts can create real AWS resources.

Steps:

- Create a service account/IAM user: AWS Console → IAM → Users → Create User (e.g. 'DevOps-Terraform').
- Under Permissions, choose Attach policies directly and attach the AdministratorAccess policy, then create the user.
- Open the new user → Security credentials tab → Access keys → Create access key → choose 'Command Line Interface (CLI)' → Create. Copy/download the Access Key ID and Secret Access Key (shown only once).
- On the terminal, configure the AWS CLI with these keys:

```
aws configure
```

```
AWS Access Key ID [None]: <your-access-key>
```

```
AWS Secret Access Key [None]: <your-secret-key>
```

```
Default region name [None]: ap-south-1
```

```
Default output format [None]: json
```

- Verify the authentication:

```
aws sts get-caller-identity
```

- Create a Terraform configuration file and add the AWS provider block:

```
nano main.tf
```

```
provider "aws" {
```

```
region = "ap-south-1"
```

```
}
```

- Save (CTRL+X → Y → Enter), then initialize and validate the plan:

```
terraform init
```

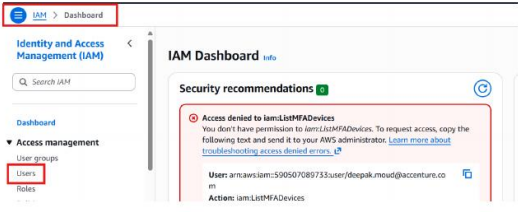
```
terraform plan
```

2.2 Scenario

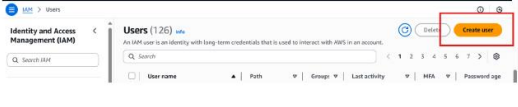
Authenticate & Configure terraform with AWS to execute terraform script to create resources.

2.3 Steps

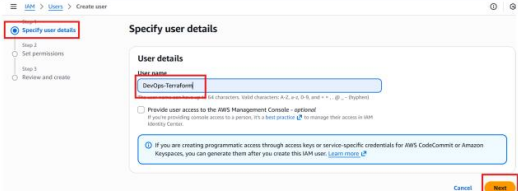
1	Create a service account to authenticate terraform Go to AWS Console-> Click IAM -> Users
2	Click on Create User
3	Enter name-> click Next-> Under Permissions --> Attach policies directly ----> Add this policy: AdministratorAccess -> Click next-> Click on Create User.



The screenshot shows the AWS IAM Dashboard. On the left, the 'IAM' menu is expanded, and 'Users' is highlighted. The main content area shows a 'Security recommendations' card with a red warning icon. The card title is 'Access denied to iam:ListMFADevices'. The text inside says: 'You don't have permission to iam:ListMFADevices. To request access, copy the following text and send it to your AWS administrator. Learn more about troubleshooting access denied errors.' Below this, it shows 'User: arn:aws:iam::59050708973:user/deepak.moud@accenture.co.in' and 'Action: iam:ListMFADevices'.



The screenshot shows the 'Users' page in the AWS IAM console. The page title is 'Users (1/26)'. There is a search bar and a table with columns: User name, Path, Group, Last activity, MFA, and Password age. The 'Create user' button is highlighted in the top right corner.



The screenshot shows the 'Specify user details' page for creating a new IAM user. The page has a sidebar with 'Specify user details' selected. The main content area has a 'User details' section with a text input field for 'User name' containing 'DevOps-Terraform'. Below this, there is a checkbox for 'Provide user access to the AWS Management Console - optional'. At the bottom right, there are 'Cancel' and 'Next' buttons.

Creating an IAM user (DevOps-Terraform) with AdministratorAccess policy

	• AWS Secret Access Key • Default Region Name • Output Format <pre>ubuntu@ip-172-31-23-0:~\$ aws configure AWS Access Key ID [*****]: AKIA6G7UKEJLPAWLAHYVL AWS Secret Access Key [*****]: vnl4329730edgr6cM3hgo/zwhqrQHed8ruw4fo8 Default region name [None]: ap-south-1 Default output format [json]: json ubuntu@ip-172-31-23-0:~\$</pre> The credentials will be saved in the following locations: ~/aws/credentials ~/aws/config
6	Now, Verify Configuration: In the terminal, run the following command: aws sts get-caller-identity If the configuration is successful, you'll see your AWS account and user details displayed: <pre>ubuntu@ip-172-31-23-0:~\$ aws sts get-caller-identity { "UserId": "AKIA6G7UKEJLPAWLAHYVL", "Account": "633458675990", "Arn": "arn:aws:iam::633458675990:user/DevOps-Terraform" }</pre>
7	Create Terraform configuration file with name "main.tf" Command: nano main.tf <pre>ubuntu@ip-172-31-23-0:~\$ nano main.tf ubuntu@ip-172-31-23-0:~\$</pre> That will open editor.
8	Use Terraform with AWS Authentication Paste below terraform script in main.tf through editor <pre>provider "aws" { region = "ap-south-1" }</pre> Replace region in above script.

aws configure, aws sts get-caller-identity, and terraform init/plan output

Activity 3: Create AWS EC2 Instance Resource using Terraform

Prerequisite	AWS cloud account is ready. Terraform is authenticated and configured to create resources.
Scenario	Create an EC2 (virtual machine) instance using a Terraform script.

Steps:

- Create/edit main.tf with the provider and an aws_instance resource block:

provider "aws" {

```

region = "ap-south-1"
}
resource "aws_instance" "ltt_aeh_instance" {
ami = "ami-0f58b397bc5c1f2e8" # Ubuntu 22.04 LTS
instance_type = "t3.micro" # Free-tier eligible
tags = {
Name = "ltt_aeh_instance"
}
}

```

- Note: For organizational AWS accounts, add a `root_block_device` block with `encrypted = true` to satisfy EBS encryption policies.
- Initialize Terraform:

```
terraform init
```

- Preview the plan:

```
terraform plan
```

- Apply and create the instance:

```
terraform apply -auto-approve
```

- Verify the new instance in the AWS Console → EC2 → Instances (switch to the correct region).
- Clean up — destroy only this resource so other infrastructure is untouched:

```
terraform destroy -target=aws_instance.ltt_aeh_instance
```

	<pre> benoit@ip-172-31-23-0:~\$ terraform plan Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols: create Terraform will perform the following actions: # aws_instance.lit_aws_instance will be created resource "aws_instance" "lit_aws_instance" { ami = "ami-0450b337bdc1f2ae" arn = (known after apply) associate_public_ip_address = (known after apply) availability_zone = (known after apply) disable_api_stop = (known after apply) disable_api_termination = (known after apply) elb_optimized = (known after apply) enable_primary_ipv6 = (known after apply) force_destroy = false get_password_data = false host_id = (known after apply) host_resource_group_arn = (known after apply) iam_instance_profile = (known after apply) tags = { "Name" = "lit_aws_instance" } "tags.all" = "lit_aws_instance" } Plan: 1 to add, 0 to change, 0 to destroy. Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions if you run "terraform apply" now. benoit@ip-172-31-23-0:~\$ </pre>
7	Apply Terraform to Create Resources Command: terraform apply -auto-approve <pre> benoit@ip-172-31-23-0:~\$ terraform apply -auto-approve Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols: create Terraform will perform the following actions: # aws_instance.lit_aws_instance will be created resource "aws_instance" "lit_aws_instance" { ami = "ami-0450b337bdc1f2ae" arn = (known after apply) associate_public_ip_address = (known after apply) availability_zone = (known after apply) disable_api_stop = (known after apply) disable_api_termination = (known after apply) elb_optimized = (known after apply) enable_primary_ipv6 = (known after apply) force_destroy = false get_password_data = false host_id = (known after apply) host_resource_group_arn = (known after apply) tags = { "Name" = "lit_aws_instance" } "tags.all" = "lit_aws_instance" } Plan: 1 to add, 0 to change, 0 to destroy. aws_instance.lit_aws_instance: Creating... aws_instance.lit_aws_instance: Still creating... (10s elapsed) aws_instance.lit_aws_instance: Creation complete after 14s [id=i-022e55oda998ffb0e] Apply complete! Resources: 1 added, 0 changed, 0 destroyed. benoit@ip-172-31-23-0:~\$ </pre>
8	check lit_aws_instance resource at AWS cloud console. In the AWS Management Console, navigate to EC2 → Instances, then switch to the region (ap-south-1) where your instance was created.

terraform apply output creating the EC2 instance, and the instance visible in the AWS console

Activity 4: Create AWS S3 Bucket using Terraform

Prerequisite

AWS cloud account is ready.
Terraform is authenticated and configured to create resources.
Create an AWS S3 (object storage) bucket resource using a Terraform script.

Scenario

Steps:

- Create main.tf with the S3 bucket resource:

```

provider "aws" {
  region = "ap-south-1"
}

resource "aws_s3_bucket" "ltd_aeh_bucket" {
  bucket = "ltd-aeh-s3-bucket" # must be globally unique
  acl = "private"
  tags = {
    Name = "ltd_aeh_s3"
    Environment = "Dev"
  }
}

```

- Initialize, plan and apply:

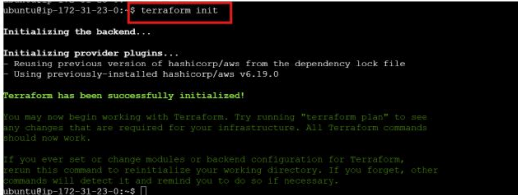
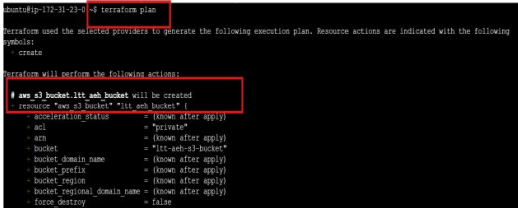
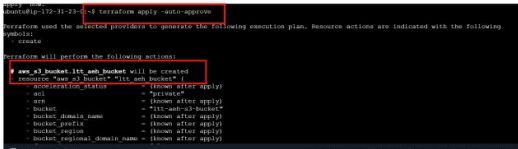
```
terraform init
```

```
terraform plan
```

```
terraform apply -auto-approve
```

- Verify the new bucket in the AWS Console → Amazon S3 (correct region).
- Delete the bucket when done:

```
terraform destroy -target=aws_s3_bucket.ltd_aeh_bucket
```

	Command: terraform init 
6	Check terraform plan. Command: terraform plan 
7	Apply Terraform to Create Resources Command: terraform apply -auto-approve 

terraform apply output creating the S3 bucket, and the bucket visible in the AWS S3 console

Activity 5: Creating AWS RDS using Terraform

Prerequisite

AWS cloud account is ready.
Terraform is authenticated and configured to create resources.

Scenario

Create a simple, minimal Terraform script to launch an AWS RDS (managed MySQL database) instance — analogous to CloudSQL on other clouds.

Steps:

- Create main.tf with the aws_db_instance resource:

```

provider "aws" {
  region = "ap-south-1"
}

resource "aws_db_instance" "ltt_aeh_rds" {
  identifier = "terraform-ltt-aeh-rds"
  allocated_storage = 20
  engine = "mysql"
  engine_version = "8.0"
  instance_class = "db.t3.micro" # free-tier eligible
  username = "admin"
  password = "Admin#12345"
  parameter_group_name = "default.mysql8.0"
  skip_final_snapshot = true
  publicly_accessible = true
  tags = {
    Name = "terraform-ltt-aeh-rds"
  }
}

```

- Note: for organizational AWS accounts, add storage_encrypted = true inside the resource block.
- Initialize, plan and apply:

```
terraform init
```

```
terraform plan
```

```
terraform apply -auto-approve
```

- RDS provisioning is not instant — even a db.t3.micro instance can take 5–15 minutes.
- Verify in the AWS Console → RDS → Databases.
- Delete the RDS instance when finished:

```
terraform destroy -target=aws_db_instance.ltt_aeh_rds
```

	<pre> aws_db_instance.tit_ash_rds will be created resource "aws_db_instance" "tit_ash_rds" { address = (known after apply) allocated_storage = 20 apply_immediately = false arm = true auto_minor_version_upgrade = (known after apply) availability_zone = (known after apply) backup_retention_period = (known after apply) backup_target = (known after apply) backup_window = (known after apply) ca_cert_identifier = (known after apply) character_set_name = (known after apply) copy_tags_to_snapshot = false database_insights_mode = (known after apply) } </pre>
7	Apply & Create the Cloud SQL Instance terraform apply -auto-approve <pre> aws_db_instance.tit_ash_rds will be created resource "aws_db_instance" "tit_ash_rds" { address = (known after apply) allocated_storage = 20 apply_immediately = false arm = true auto_minor_version_upgrade = (known after apply) availability_zone = (known after apply) backup_retention_period = (known after apply) backup_target = (known after apply) backup_window = (known after apply) ca_cert_identifier = (known after apply) character_set_name = (known after apply) copy_tags_to_snapshot = false database_insights_mode = (known after apply) } aws_db_instance.tit_ash_rds: Still creating... [2m0s elapsed] aws_db_instance.tit_ash_rds: Still creating... [2m0s elapsed] aws_db_instance.tit_ash_rds: Still creating... [3m0s elapsed] aws_db_instance.tit_ash_rds: Still creating... [3m0s elapsed] aws_db_instance.tit_ash_rds: Still creating... [3m0s elapsed] aws_db_instance.tit_ash_rds: Still creating... [3m0s elapsed] aws_db_instance.tit_ash_rds: Still creating... [3m0s elapsed] aws_db_instance.tit_ash_rds: Still creating... [4m0s elapsed] aws_db_instance.tit_ash_rds: Still creating... [4m0s elapsed] aws_db_instance.tit_ash_rds: Still creating... [4m0s elapsed] aws_db_instance.tit_ash_rds: Still creating... [4m0s elapsed] aws_db_instance.tit_ash_rds: Still creating... [5m0s elapsed] aws_db_instance.tit_ash_rds: Creation complete after 5m0s [id=db-D8ATV54INQ3Y1TUM37ULPZSQI] Apply complete! Resources: 1 added, 0 changed, 0 destroyed </pre>
8	RDS creation isn't instant — even a small instance like db.t3.micro takes 5-15mins. You can go to: AWS Console → RDS → Databases-> instance
9	Verify the RDS Instance Go to AWS Console → RDS → Databases-> instance You should see your new AWS RDS instance:

RDS instance created by Terraform, verified as 'Available' in the AWS RDS console

Activity 6: Create AWS IAM Role using Terraform

Prerequisite

AWS cloud account is ready.
Terraform is authenticated and configured to create resources.

Scenario

Create a custom AWS IAM role and an attached policy using Terraform, so other AWS services (e.g. EC2) can assume that role.

Steps:

- Create main.tf with a role, a custom policy, and the attachment between them:

```

provider "aws" {
  region = "ap-south-1"
}

resource "aws_iam_role" "ltt_admin_role" {
  name = "ltt-admin-role"
  assume_role_policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Action = "sts:AssumeRole"
      Effect = "Allow"
      Principal = { Service = "ec2.amazonaws.com" }
    }]
  })
}

resource "aws_iam_policy" "ltt_full_policy" {
  name = "ltt-full-access-policy"
  description = "Allows managing S3, EC2, and RDS resources"
  policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Effect = "Allow"
      Action = ["s3:*", "ec2:*", "rds:*", "iam:PassRole"]
      Resource = "*"
    }]
  })
}

resource "aws_iam_role_policy_attachment" "ltt_role_policy_attach" {
  role = aws_iam_role.ltt_admin_role.name
  policy_arn = aws_iam_policy.ltt_full_policy.arn
}

```

}

- Initialize, plan and apply:

terraform init

terraform plan

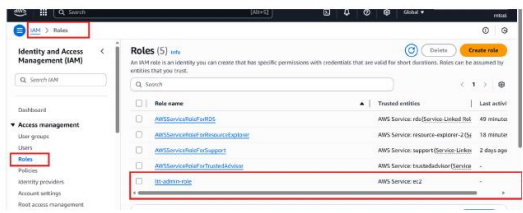
terraform apply -auto-approve

- Verify in the AWS Console → IAM → Roles and IAM → Policies.
- Destroy the created resources:

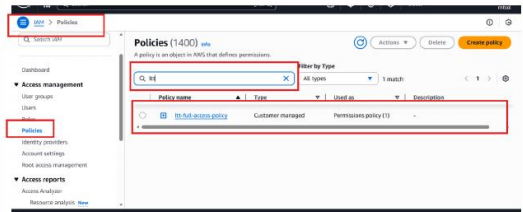
terraform destroy -auto-approve

7 Verify custom role and policy created using terraform script

Go to AWS Console → IAM Dashboard → Roles:



Go to AWS Console → IAM Dashboard → Policies:



You should see the newly created role "my Custom Role and Policy".

8 Destroy the Role:

To delete the custom role, execute following command:

terraform destroy -auto-approve

Custom IAM role 'ltt-admin-role' and policy 'ltt-full-access-policy' visible in the AWS IAM console

5. Viva Voce Questions — Terraform

Q: What is Infrastructure as Code (IaC)?

A: IaC is the practice of managing and provisioning infrastructure through machine-readable configuration files rather than manual processes, making infrastructure repeatable, version-controlled, and auditable.

Q: What is Terraform and who develops it?

A: Terraform is an open-source IaC tool developed by HashiCorp that lets you define and provision infrastructure across many cloud providers using a declarative configuration language (HCL).

Q: Is Terraform declarative or imperative?

A: Terraform is declarative — you describe the desired end state of the infrastructure, and Terraform determines the steps needed to reach that state.

Q: What language does Terraform use?

A: HashiCorp Configuration Language (HCL), a human-readable, declarative language, written in .tf files.

Q: What is a Terraform provider?

A: A plugin that lets Terraform interact with a specific platform's API (e.g. aws, azure, google, mysql, kubernetes) by translating Terraform resource blocks into real API calls.

Q: What is the Terraform state file, and why is it important?

A: terraform.tfstate is a JSON file that maps your configuration to real-world resources. Terraform uses it to determine what already exists and what changes are needed. It should never be edited by hand.

Q: What are the four core Terraform commands, in order?

A: terraform init (download providers/initialize), terraform plan (preview changes), terraform apply (create/update resources), terraform destroy (tear down resources).

Q: What does 'terraform plan' do?

A: It compares the current state with the desired configuration and produces an execution plan showing what will be added, changed, or destroyed — without making any real changes.

Q: What is the purpose of '-auto-approve' in terraform apply?

A: It skips the interactive yes/no confirmation prompt so the apply runs non-interactively (useful for automation/CI pipelines).

Q: How do you destroy only one specific resource instead of the whole configuration?

A: Use terraform destroy -target=<resource_type>.<resource_name>, e.g. terraform destroy -target=aws_instance.ltt_ach_instance.

Q: What is a Terraform module?

A: A reusable, self-contained package of .tf configuration files that can be called by other configurations to avoid duplication.

Q: What is a Terraform backend?

A: The backend defines where the state file is stored and how operations are executed — locally on disk by default, or remotely (e.g. S3 + DynamoDB) for team collaboration and state locking.

Q: What is a resource block used for?

A: To declare an actual piece of infrastructure that Terraform should create and manage, e.g. resource "aws_instance" "my_vm" { ... }.

Q: What is the difference between a variable block and an output block?

A: A variable block declares an input parameter that customizes the configuration; an output block exposes a value (e.g. an instance's IP) after apply for reference or chaining.

Q: Why do we run 'aws configure' or set up an IAM user before using Terraform with AWS?

A: Terraform needs valid AWS credentials (Access Key ID and Secret Access Key) to authenticate API calls on your behalf; without them, init/plan/apply cannot reach AWS.

Q: What does the .terraform.lock.hcl file do?

A: It records the exact provider plugin versions used, ensuring the same versions are installed consistently across different machines/runs.

Q: What happens if you run terraform apply without running terraform init first?

A: It fails, because the required provider plugins have not been downloaded/initialized in the working directory.

Q: What is the benefit of using Terraform over manually creating resources in the AWS console?

A: Speed, repeatability, version control, safer previews via 'plan', consistent environments, and easy teardown of temporary infrastructure to control cost.

PART 2

Containerization with Docker

1. Introduction to Docker

Docker is an open-source containerization platform that packages an application together with all its dependencies (libraries, runtime, configuration files) into a single, lightweight, portable unit called a container. A container runs the same way regardless of where it is deployed — a developer's laptop, a test server, or a production cloud VM — solving the classic 'it works on my machine' problem.

Containers vs. Virtual Machines

- Virtual Machines virtualize an entire operating system (including its own kernel) on top of a hypervisor, making them heavier and slower to start (minutes).
- Containers share the host OS kernel and only virtualize the application layer, making them lightweight, fast to start (seconds), and far more resource-efficient — many containers can run on a single host.

Why use Docker?

- Portability — 'build once, run anywhere' across dev, test, and production environments.
- Isolation — each container runs in its own isolated filesystem, process space, and network namespace.
- Consistency — eliminates environment-configuration drift between machines.
- Efficiency — containers are lightweight compared to VMs, enabling higher density and faster CI/CD pipelines.
- Microservices-friendly — each service can be packaged, versioned, and scaled independently as its own container.

2. Docker Architecture — Managing Docker Images and Containers

Docker uses a client-server architecture.

Core components

- Docker Client — the CLI tool (docker command) used to issue commands such as 'docker build' or 'docker run'.
- Docker Daemon (dockerd) — a background service that does the actual work of building, running, and managing containers and images; it listens for API requests from the client.
- Docker Images — read-only templates built in layers, made up of application code, a runtime, libraries, and configuration. Images are created from a Dockerfile.
- Docker Containers — running (or stopped) instances of an image. A container is an image plus a writable layer on top.

- Dockerfile — a text file with step-by-step instructions (FROM, COPY, RUN, EXPOSE, CMD, etc.) that Docker uses to build an image.
- Docker Registry (e.g. Docker Hub) — a storage and distribution service for Docker images. 'docker push' uploads an image; 'docker pull' downloads one.
- Docker Network — a virtual network that lets containers communicate with each other by name.
- Docker Volume — a mechanism to persist data generated/used by containers outside the container's own writable layer, so data survives container removal.

Typical Dockerfile instructions

- FROM — the base image to build on top of (e.g. openjdk:17-jdk-alpine).
- WORKDIR — sets the working directory inside the container.
- COPY — copies files from the host into the image.
- EXPOSE — documents which port(s) the containerized application listens on.
- CMD — the default command executed when a container starts from the image.

Image vs. Container — the key distinction

An image is a static, immutable blueprint (like a class in OOP). A container is a running instance of that image (like an object). You can start many containers from the same single image.

3. Hands-on Activities

Activity 1: Docker Installation

Prerequisite	AWS cloud account is ready to use.
Scenario	Launch AWS EC2 Instance Connect and install Docker on Ubuntu.

Steps:

- Remove any old Docker versions:

```
sudo apt-get remove -y docker docker-engine docker.io containerd runc
```

- Update package index and install dependencies:

```
sudo apt-get update -y
```

```
sudo apt-get install -y ca-certificates curl gnupg
```


- Add Docker's official GPG key:

```
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor
-o /etc/apt/keyrings/docker.gpg
sudo chmod a+r /etc/apt/keyrings/docker.gpg
```

- Add the Docker apt repository and update:

```
echo \
"deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg]
\
https://download.docker.com/linux/ubuntu \
$(. /etc/os-release && echo "$VERSION_CODENAME") stable" | \
sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
sudo apt-get update -y
```

- Install Docker Engine, CLI, containerd, buildx and compose plugins:

```
sudo apt-get install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin
docker-compose-plugin
```

- Verify installation, then enable and start the service:

```
docker --version
sudo systemctl enable docker
sudo systemctl start docker
```

- Run the classic test container:

```
docker run hello-world
```

- If you see a 'permission denied' error, ensure Docker is active (sudo systemctl status docker), then add your user to the docker group: sudo usermod -aG docker \$USER && newgrp docker, and re-run the test.

- List downloaded images to confirm:

```
docker images
```

	<pre> bhargav_raghavendra@my-ubuntu-vmi-6:~\$ docker run hello-world Unable to find image 'hello-world:latest' locally latest: Pulling from library/hello-world c1ec31eb5944: pull complete Digest: sha256:1b30c95616e3058003c13b7821318369da01da03d8b877aac3026182255c724 Status: Downloaded newer image for hello-world:latest Hello from Docker! This message shows that your installation appears to be working correctly. To generate this message, Docker took the following steps: 1. The Docker client contacted the Docker daemon. 2. The Docker daemon pulled the "hello-world" image from the Docker Hub. (and...) 3. The Docker daemon created a new container from that image which runs the executable that produces the output you are currently reading. 4. The Docker daemon streamed that output to the Docker client, which sent it to your terminal. To try something more ambitious, you can run an Ubuntu container with: \$ docker run -it ubuntu bash Share images, automate workflows, and more with a free Docker ID: https://hub.docker.com/ For more examples and ideas, visit: https://docs.docker.com/get-started/ </pre> Note: If you encounter a "permission denied" error while running Docker commands, follow the steps below to resolve it: 1. Check docker status : execute - <code>sudo systemctl status docker</code> Docker status should be Active. If it's not active then enable and start docker. 2. Docker automatically creates a user group named "docker". To allow your current user to run Docker commands without using <code>sudo</code>, add the user to this group and refresh the group membership: <code>sudo usermod -aG docker \$USER</code> <code>newgrp docker</code> 3. Now execute : <code>docker run hello-world</code>
12	Check docker image "hello-world" status: docker images <pre> ubuntu@ip-172-31-23-0:~\$ docker images REPOSITORY TAG IMAGE ID CREATED SIZE hello-world latest 1b44b5a3e06a 2 months ago 10.1kB ubuntu@ip-172-31-23-0:~\$ </pre>

Verifying Docker installation, enabling the service, and successfully running 'hello-world'

Activity 2: Create Docker File

Prerequisite

Docker is installed; a Spring Boot application is ready for deployment.

Scenario

Create a Dockerfile to containerize a Spring Boot application.

Steps:

- Clone the sample project:

git clone <https://gitlab.com/dmjadhav795/easyfly.git>

- Create a file named Dockerfile with build instructions:

```
# Step 1: Base image (OpenJDK 17)
```

```
FROM openjdk:17-jdk-alpine
```

```
# Step 2: Copy the built JAR file into the container
```

```
COPY target/*.jar app.jar
```

```
# Step 3: Expose the application's port
```

```
EXPOSE 8081
```

```
# Step 4: Run the application
```

```
CMD ["java", "-jar", "/app.jar"]
```

- FROM specifies the base JDK image; COPY places the built jar inside the image; EXPOSE documents the listening port; CMD defines the container's startup command.

Activity 2: Create Docker File

2.1 Prerequisite

1. AWS account is ready to use.
2. Docker is installed.
3. Springboot application is ready for deployment.
4. DockerHub account (optional)
5. Basic knowledge of docker commands.
6. text file to write docker file instructions.

2.2 Scenario

Create a docker file to containerise SpringBoot application.

2.3 Steps

1	Clone git repository git clone https://gitlab.com/dmjadhav795/easyfly.git
2	Open any text editor Write docker file: Docker file will have specific instructions line by line.
3	Creating docker file to dockerise simple spring boot application: # Step 1: Use an official base image (OpenJDK 17) #for x86_64 architecture FROM openjdk:17-jdk-alpine #for ARM64 architecture #FROM --platform=linux/arm64 eclipse-temurin:17-jdk # Step 2: Copy your JAR file into the container COPY target/*.jar app.jar # Step 3: Expose the port your app runs on EXPOSE 8081 # Step 4: Run the application CMD ["java", "-jar", "/app.jar"]
4	Explanation about docker file: FROM: specifying JDK 17 image. COPY: copying application jar into /app container. EXPOSE: specify port number on which you app runs CMD: define commands to run application jar.

Dockerfile content used to containerize a Spring Boot application

Activity 3: Create DockerHub Account

Prerequisite

AWS account, Spring Boot application, and Dockerfile are ready.

Scenario

Create a free Docker Hub account to store and share Docker images.

Steps:

- Browse to <https://app.docker.com/signup> and sign up (email/username/password, or 'Continue with Google'/'Continue with GitHub').
- Sign in and browse <https://hub.docker.com/repositories/<docker-id>> to view your repository and any images you push.

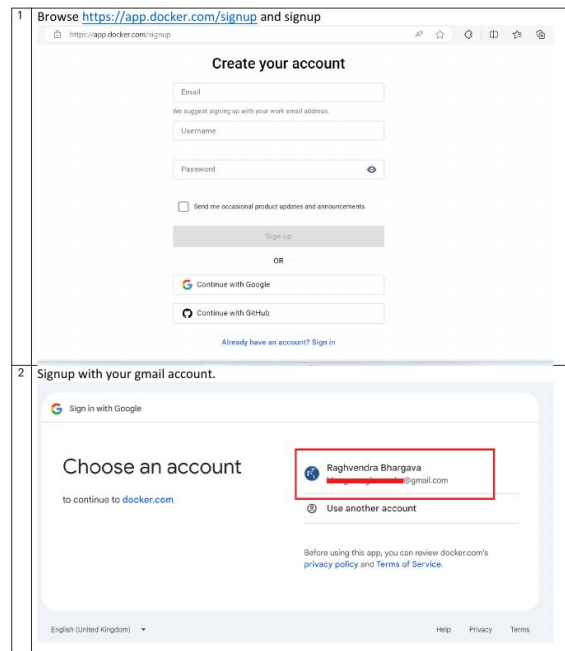
Activity 3: Create DockerHub Account

3.1 Prerequisite

AWS cloud account is ready, Springboot application is ready, Docker file is ready.

3.2 Scenario

3.3 Steps



Docker Hub sign-up page and repository list

Activity 4: Create Docker Image

Prerequisite

AWS EC2 ready; Spring Boot application ready for deployment; Docker Hub account (optional).

Scenario

Build a Docker image for the Spring Boot application on AWS EC2.

Steps:

- Clone the project and build the jar with Maven:

git clone <https://github.com/your-git-hub-id/spring-boot.git>

```
cd spring-boot
mvn clean install
    • Confirm the jar exists inside the target folder:
ls target
    • Create the Dockerfile:
vi Dockerfile
    • Paste the build instructions (adjust jar name):
# Step 1: Base image
FROM openjdk:17-jdk-slim
# Step 2: Working directory
WORKDIR /app
# Step 3: Copy the jar
COPY target/my-spring-app.jar app.jar
# Step 4: Expose port
EXPOSE 8080
# Step 5: Run application
CMD ["java", "-jar", "app.jar"]
    • Save and exit vi with :wq.
    • Build the Docker image (image name/tag must be lowercase):
docker build -t myapp .
    • Confirm the image was created:
docker images
```

	<pre> bhargav_raghavendras@ubuntu-vm:/spring-boot\$ mvn clean install [INFO] Scanning for projects... [INFO] [INFO] -----[com.accenture.spring-boot]----- [INFO] Building spring-boot 1.2 [INFO] -----[jar]----- [INFO] [INFO] --- maven-clean-plugin:3.2.0:clean (default-clean) @ spring-boot --- [INFO] [INFO] --- maven-resources-plugin:3.2.0:resources (default-resources) @ spring-boot --- [INFO] Using 'UTF-8' encoding to copy filtered resources. [INFO] Using 'UTF-8' encoding to copy filtered properties files. [INFO] Copying 1 resource [INFO] Copying 0 resource [INFO] [INFO] --- maven-compiler-plugin:3.10.1:compile (default-compile) @ spring-boot --- [INFO] Changes detected - recompiling the module! [INFO] Compiling 1 source file to /home/bhargav_raghavendra/spring-boot/target/classes [INFO] [INFO] --- maven-resources-plugin:3.2.0:testResources (default-testResources) @ spring-boot --- [INFO] Using 'UTF-8' encoding to copy filtered resources. [INFO] Using 'UTF-8' encoding to copy filtered properties files. [INFO] skip non existing resource directory /home/bhargav_raghavendra/spring-boot/test/resources [INFO] [INFO] --- maven-compiler-plugin:3.10.1:testCompile (default-testCompile) @ spring-boot --- [INFO] Changes detected - recompiling the module! [INFO] Compiling 1 source file to /home/bhargav_raghavendra/spring-boot/target/test-classes [INFO] [INFO] --- maven-surefire-plugin:2.22.2:test (default-test) @ spring-boot --- [INFO] [INFO] [INFO] :: Spring Boot :: [INFO] (v2.7.7) 2025-01-06 10:16:24.053 INFO 222246 --- [main] com.accenture.ApplicationTests : Starting A applicationTests using Java 17.0.13 on my-ubuntu-vm with PID 222246 (started by bhargav_raghavendra in /home/bhar gav_raghavendra/spring-boot) 2025-01-06 10:16:24.096 INFO 222246 --- [main] com.accenture.ApplicationTests : No active profile set, falling back to 1 default profile: "default" 2025-01-06 10:16:26.932 INFO 222246 --- [main] com.accenture.ApplicationTests : Started Ap plicationTests in 3.428 seconds (JVM running for 5.501) [INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 5.271 s - in com.accenture.ApplicationTest s [INFO] [INFO] Results: [INFO] [INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0 [INFO] [INFO] --- maven-jar-plugin:3.1.2:jar (default-jar) @ spring-boot --- [INFO] Building jar: /home/bhargav_raghavendra/spring-boot/target/spring-boot-1.2.jar [INFO] [INFO] --- maven-shade-plugin:3.2.1:shade (repackage) @ spring-boot --- [INFO] Replacing main artifact with repackaged archive [INFO] [INFO] --- maven-install-plugin:3.0.1:install (default-install) @ spring-boot --- [INFO] Installing /home/bhargav_raghavendra/spring-boot/target/spring-boot-1.2.jar to /home/bhargav_raghavendra/ .s2/repository/com/accenture/spring-boot/1.2/spring-boot-1.2.jar [INFO] Installing /home/bhargav_raghavendra/spring-boot/pom.xml to /home/bhargav_raghavendra/.s2/repository/com/ accenture/spring-boot/1.2/spring-boot-1.2.pom [INFO] [INFO] BUILD SUCCESS [INFO] [INFO] Total time: 16.439 s [INFO] Finished at: 2025-01-06T10:16:30Z [INFO] </pre>
5	Browse target folder of spring boot app "ls target" <pre> bhargav_raghavendras@ubuntu-vm:/spring-boot\$ ls target classes generated-test-sources maven-status spring-boot-1.2.jar.original test-classes generated-sources maven-archiver spring-boot-1.2.jar surefire-reports </pre>
6	Open editor to paste docker script by "vi Dockerfile" Save and exit from vi editor (press i for edit mode if required); Press Esc to exit insert mode.

docker build output and the resulting 'myapp' image listed by docker images

Activity 5: Tag, List, Delete Image

Prerequisite

Spring Boot application image is ready.

Scenario

Tag, list, and delete an existing Docker image.

Steps:

- Tag an existing image with a new local tag:

docker tag myapp:latest myapp:sbttag

- Tag it for a Docker Hub repository:

`docker tag myapp my-docker-hub-user/myapp:sbtag`

- List all local images to confirm:

`docker images`

- Remove an image by name:

`docker rmi hello-world`

- Remove an image by name:tag:

`docker rmi myapp:sbtag`

- `docker rmi` permanently removes an image from the local repository; the container(s) built from it are unaffected if already running, but new containers can no longer be created from a removed image.

5.2 Scenario

Tag, List and Delete existing docker image created in previous step.

5.3 Steps

1 To tag an existing Docker image named "myapp" with a new tag "sbtag", use the following command:

```
docker tag myapp:latest myapp:sbtag
```


tag image to docker hub, use following command:

```
docker tag myapp my-docker-hub-user/myapp:sbtag
```


where:
docker tag-> tag an existing image with new tag
myapp-> is existing docker image
myapp:sbtag-. New tag is applied

```
openjdk 11 47a932d998b7 2 years ago 654MB  
sha256:c2c94e256dcb3c5ac279bd32e1248e42e101cbe4841c2234249495f87264ac5a
```

2 To verify new tagging, use following command to list images:

```
docker images
```



```
my-ubuntu-vm1 ~$ docker images
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
myapp	latest	7dd1936ea2f0	4 weeks ago	425MB
myapp	sbtag	7dd1936ea2f0	4 weeks ago	425MB
rntsolutions/myapp	latest	7dd1936ea2f0	4 weeks ago	425MB
rntsolutions/app	latest	0088df118fb7	4 weeks ago	425MB
mysql	latest	56a8c14e1404	3 months ago	603MB
mysql	8	3813a28b4a67	3 months ago	594MB
mysqlapp	latest	012163be7574	3 months ago	594MB
dadaramjadhav/cs	latest	1b53dd69dd5	6 months ago	345MB
hello-world	latest	d2c94e256dcb	21 months ago	13.3kB
openjdk	latest	71260f256d19	24 months ago	470MB
openjdk	11	47a932d998b7	2 years ago	654MB

3 Remove image by image name:

If you have an image named **hello-world**, remove it with:

```
docker rmi hello-world
```


where hello-world is the image name.

```
my-ubuntu-vm1 ~$ docker rmi hello-world
```

```
Untagged: hello-world:latest  
Untagged: hello-world@sha256:b3bccc5e16e3058003c13b7821310369da901dac3bb877aac3c28182255c724  
Deleted: sha256:c2c94e256dcb3c5ac279bd32e1248e42e101cbe4841c2234249495f87264ac5a  
Deleted: sha256:ac28800ec8bb3bdcc3d44d45a6ac4777544941199075df1f6c4b63e093aa81e
```

Tagging an image, listing images, and removing images by name and by tag

Activity 6: Push Image to DockerHub Registry

Prerequisite	Docker Hub account is ready; Spring Boot application image is ready.
Scenario	Tag and push an existing Docker image to the Docker Hub registry.

Steps:

- Log in to Docker Hub:

`docker login`

- Tag the image with your Docker Hub username/repository:

`docker tag myapp:latest my-docker-hub-username/myapp:v1.0`

- List images to confirm the new tag:

`docker images`

- Push the image to Docker Hub:

`docker push my-docker-hub-username/myapp:v1.0`

- Verify the uploaded image at <https://hub.docker.com/>.

1	<pre> khanaputra@phoenix:~/my-ubuntu-vm1\$ docker login Authentication with existing credentials... WARNING! Your password will be stored unencrypted in /home/khanaputra/.docker/config.json. Configure a credential helper to remove this warning. See https://docs.docker.com/engine/reference/commandline/login/#credential-stores Login Succeeded khanaputra@phoenix:~/my-ubuntu-vm1\$ </pre>																																																							
2	Before pushing, you need to tag your image with your Docker Hub username or repository name: docker tag myapp:latest my-docker-hub-username/myapp:v1.0 where: myapp is the docker image v1.0 is tag name in hub repository <pre> khanaputra@phoenix:~/my-ubuntu-vm1\$ docker tag myapp:latest rntoolsdone/myapp:v1.0 </pre>																																																							
3	List repo image using docker images <pre> khanaputra@phoenix:~/my-ubuntu-vm1\$ docker images </pre> <table> <thead> <tr> <th>REPOSITORY</th><th>TAG</th><th>IMAGE ID</th><th>CREATED</th><th>SIZE</th></tr> </thead> <tbody> <tr> <td>myapp</td><td>latest</td><td>7d1f936ea2f0</td><td>4 weeks ago</td><td>425MB</td></tr> <tr> <td>rntoolsdone/myapp</td><td>latest</td><td>7d1f936ea2f0</td><td>4 weeks ago</td><td>425MB</td></tr> <tr> <td>rntoolsdone/myapp</td><td>v1.0</td><td>7d1f936ea2f0</td><td>4 weeks ago</td><td>425MB</td></tr> <tr> <td>rntoolsdone/app</td><td>latest</td><td>0088d11101b7</td><td>5 weeks ago</td><td>425MB</td></tr> <tr> <td>mysql</td><td>latest</td><td>56a8c1ef1d04</td><td>3 months ago</td><td>602MB</td></tr> <tr> <td>mysql</td><td>8</td><td>3818a20b4a67</td><td>3 months ago</td><td>594MB</td></tr> <tr> <td>mysql/app</td><td>latest</td><td>012163ae757d</td><td>3 months ago</td><td>594MB</td></tr> <tr> <td>chidambaramjadhav/cn</td><td>latest</td><td>1b53d856d1d5</td><td>6 months ago</td><td>345MB</td></tr> <tr> <td>openjdk</td><td>latest</td><td>712601256d19</td><td>24 months ago</td><td>470MB</td></tr> <tr> <td>openjdk</td><td>11</td><td>47a932d998b7</td><td>2 years ago</td><td>654MB</td></tr> </tbody> </table> <pre> khanaputra@phoenix:~/my-ubuntu-vm1\$ </pre>	REPOSITORY	TAG	IMAGE ID	CREATED	SIZE	myapp	latest	7d1f936ea2f0	4 weeks ago	425MB	rntoolsdone/myapp	latest	7d1f936ea2f0	4 weeks ago	425MB	rntoolsdone/myapp	v1.0	7d1f936ea2f0	4 weeks ago	425MB	rntoolsdone/app	latest	0088d11101b7	5 weeks ago	425MB	mysql	latest	56a8c1ef1d04	3 months ago	602MB	mysql	8	3818a20b4a67	3 months ago	594MB	mysql/app	latest	012163ae757d	3 months ago	594MB	chidambaramjadhav/cn	latest	1b53d856d1d5	6 months ago	345MB	openjdk	latest	712601256d19	24 months ago	470MB	openjdk	11	47a932d998b7	2 years ago	654MB
REPOSITORY	TAG	IMAGE ID	CREATED	SIZE																																																				
myapp	latest	7d1f936ea2f0	4 weeks ago	425MB																																																				
rntoolsdone/myapp	latest	7d1f936ea2f0	4 weeks ago	425MB																																																				
rntoolsdone/myapp	v1.0	7d1f936ea2f0	4 weeks ago	425MB																																																				
rntoolsdone/app	latest	0088d11101b7	5 weeks ago	425MB																																																				
mysql	latest	56a8c1ef1d04	3 months ago	602MB																																																				
mysql	8	3818a20b4a67	3 months ago	594MB																																																				
mysql/app	latest	012163ae757d	3 months ago	594MB																																																				
chidambaramjadhav/cn	latest	1b53d856d1d5	6 months ago	345MB																																																				
openjdk	latest	712601256d19	24 months ago	470MB																																																				
openjdk	11	47a932d998b7	2 years ago	654MB																																																				
4	Now, push the image to Docker Hub: docker push my-docker-hub-username/myapp:v1.0 <pre> khanaputra@phoenix:~/my-ubuntu-vm1\$ docker push rntoolsdone/myapp:v1.0 The push refers to repository (docker.io/rntoolsdone/myapp) b0ba5984f051: Layer already exists d9d8bd31c400: Layer already exists 0e6992374ef1: Layer already exists 11a3d6e4ff78: Layer already exists 9c1b6d6f1e6f: Layer already exists v1.0: digest: sha256:5d3b5c77f22647f101f583224d2413215177659cd1d8401139366c3c66c87b4a size: 1371 khanaputra@phoenix:~/my-ubuntu-vm1\$ </pre>																																																							
5	Verify pushed image on Docker Hub: https://hub.docker.com/ Go to Docker Hub and check if your image is uploaded.																																																							

docker login, docker push output, and the pushed image with its tags visible on Docker Hub

Activity 7: Pull Image from DockerHub Registry

Prerequisite

Docker Hub account is ready; image has already been pushed.

Scenario

Pull an existing Docker image from Docker Hub to create a container.

Steps:

- List local images before pulling:

docker images

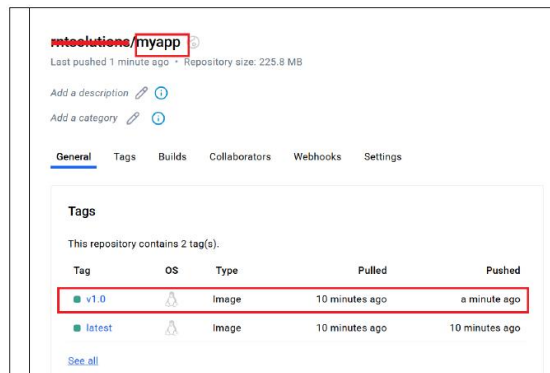
- Log in (if needed) and pull the image by name:tag:

docker login

docker pull my-docker-hub-username/imageName

- Confirm the pulled image is now available locally:

docker images



Activity 7: Pull Image from dockerhub Registry

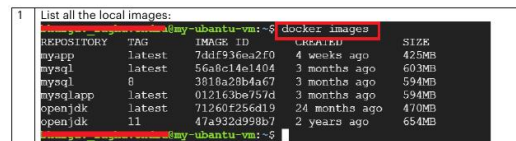
7.1 Prerequisite

- AWS Cloud account is ready to use.
- Springboot application image is ready.
- DockerHub account is ready.
- Basic knowledge of docker commands.

7.2 Scenario

Pull existing docker image from docker hub registry to create container.

7.3 Steps



docker pull output and the newly pulled image listed locally

Activity 8: Create Container

Prerequisite

Docker Hub account ready; Spring Boot application image ready.

Scenario	Create and run a Docker container from a pulled image, then access the app via the public IP.
-----------------	---

Steps:

- Run the image, mapping container port 8080 to host port 8080 in detached mode:

```
docker run -d -p 8080:8080 my-docker-username/myapp:latest
```

- Verify the container is running:

```
docker ps
```

- Copy the EC2 instance's public IP address from the AWS console.
- Access the application in a browser: `http://<public-ip-address>:8080`

2 Make sure you are logged-in to docker hub using "docker login" command. Then use "docker pull" command to pull docker image from registry by name:tag

```
docker pull my-docker-hub-username/imageName
```

```
khargav_raghavendra@my-ubuntu-vm:~$ docker pull rntosolutions/myapp
Using default tag: latest
latest: Pulling from rntosolutions/myapp
Digest: sha256:5d398572f22647e510459324d2413215177659c01d0881139306c3c66c970aa
Status: Downloaded newer image for rntosolutions/myapp:latest
docker.io/rntosolutions/myapp:latest
khargav_raghavendra@my-ubuntu-vm:~$
```

3 Once the image is pulled/downloaded, check if it is available locally, use command:

```
docker images
```

```
khargav_raghavendra@my-ubuntu-vm:~$ docker images
```

REPOSITORY	TAG	IMAGES ID	CREATED	SIZE
myapp	latest	7dd1346aa2f0	4 weeks ago	225MB
rntosolutions/myapp	latest	7dd1346aa2f0	4 weeks ago	225MB
mysql	latest	2ba9c1461404	4 months ago	603MB
mysql	8	3818a28b4a67	3 months ago	594MB
mysqlapp	latest	012163be757d	3 months ago	594MB
openjdk	latest	71260f256d19	24 months ago	470MB
openjdk	11	47a932d998b7	2 years ago	654MB

```
khargav_raghavendra@my-ubuntu-vm:~$
```

Activity 8: Create Container

8.1 Prerequisite

- AWS Cloud account is ready to use.
- Springboot application image is ready.
- DockerHub account is ready.
- Basic knowledge of docker commands.

8.2 Scenario

Now create docker container from pulled docker image in previous step

8.3 Steps

1 To run pulled image my-docker-username/myapp, use following command:
 docker run -d -p 8080:8080 my-docker-username/myapp:latest
 where
 -d runs container in detached mode
 -p 8080:8080 maps the container's port 8080 to your local machine.

```
khargav_raghavendra@my-ubuntu-vm:~$ docker run -d -p 8080:8080 rntosolutions/myapp:latest
d050ea76818158da5adc3dd3c6c43a10454d19dc5f103242c37395abae14c7
khargav_raghavendra@my-ubuntu-vm:~$
```

2 Now verify if container is running, using following command:
 "docker ps"

```
khargav_raghavendra@my-ubuntu-vm:~$ docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
d050ea768181	rntosolutions/myapp:latest	"java -jar app.jar"	36 seconds ago	Up 35 seconds	0.0.0.0:8080->8080/tcp, :::8080->8080/tcp

```
khargav_raghavendra@my-ubuntu-vm:~$
```

docker run / docker ps output, and the Spring Boot application accessible via the public IP and port

Activity 9: Stop, Pause, List, Remove Container

Prerequisite

A running container exists (created in the previous activity).

Scenario

Manage the container's lifecycle: list, pause/unpause, stop/start, and forcefully remove it.

Steps:

- List running/all containers:

docker ps

docker ps -a

- Pause and resume a running container:

docker pause <container-id>

docker unpause <container-id>

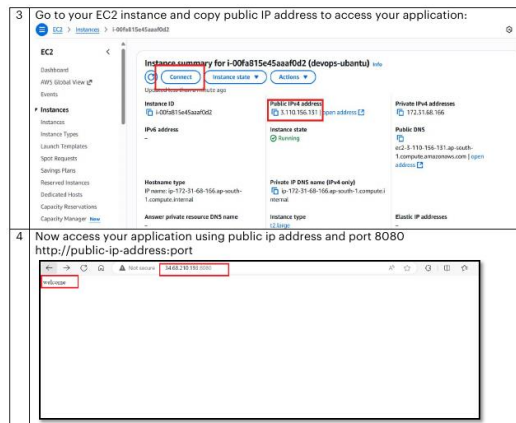
- Stop and start a container:

docker stop <container-id>

docker start <container-id>

- Forcefully remove a container (stops it first, then deletes it):

docker rm <container-id> -f



Activity 9: Stop, Pause, List, Remove Container

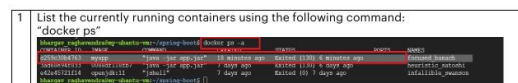
9.1 Prerequisite

- AWS Cloud account is ready to use.
- Springboot application image is ready.
- DockerHub account is ready.
- Basic knowledge of docker commands.

9.2 Scenario

Now, manage the running container we created in the previous step by starting, stopping, pausing, listing, or removing it.

9.3 Steps



docker pause/unpause, stop/start, and forced removal (docker rm -f) of a container

Activity 10: Build and Run MySQL Container

Prerequisite	AWS Cloud account is ready; basic knowledge of docker commands.
Scenario	Pull the official MySQL image, run it as a container, and execute MySQL commands inside it.

Steps:

- Pull the MySQL 8.0 image:

```
docker pull mysql:8.0
```

- Verify the pulled image:

```
docker images
```

- Run a MySQL container with environment variables for root password, database, and user:

```
docker run -d --name sb-mysql-db -e MYSQL_ROOT_PASSWORD=root -e  
MYSQL_DATABASE=mydb -e MYSQL_USER=myuser -e MYSQL_PASSWORD=root  
mysql:8.0
```

- Verify the container is running:

```
docker ps
```

- Access the MySQL shell inside the container:

```
docker exec -it sb-mysql-db mysql -u myuser -p
```

- Run SQL commands (list databases, select mydb, create an employee table, insert and query rows):

```
show databases;
```

```
use mydb;
```

```
CREATE TABLE employee (
```

```
employeeId int,
```

```
firstName varchar(255),
```

```
lastName varchar(255),
```

```
Address varchar(255),
```

```
City varchar(255)
```

```
);

insert into employee (employeeId, firstName, lastName, Address, City)
values (1001, 'Myra', 'Bhargava', 'Indore', 'Indore'),
(1002, 'Ravi', 'Sharma', 'Delhi', 'Delhi');

select * from employee;
```

- Important: If the container is removed (`docker rm -f sb-mysql-db`) and recreated, the employee table and its data are LOST — because MySQL data was stored inside the container's own writable layer, which is destroyed with the container. This motivates the need for Docker volumes (see Activity 12).

	<pre> bhargav_raghavendra@ubuntu-vm:~\$ docker pull mysql:8.0 8.0: Pulling from library/mysql 4473ac30a668: Pull complete 6c3b09f930cc: Pull complete a69038e54211: Pull complete a0e40652ce5a: Pull complete c93c2d4dbb37: Pull complete a628a15d4f35: Pull complete 48394ab09121: Pull complete 8815fc443576: Pull complete b4a711b7267: Pull complete 2af823f1482: Pull complete 07684a158813: Pull complete Digest: sha256:f9657322a50b735bf284dc06a3e3d99e24810ea13d37ef6b7ca6cdd0d54fcd8 Status: Downloaded newer image for mysql:8.0 bhargav_raghavendra@ubuntu-vm:~\$ </pre>
2	Verify pulled mysql image in local repository, execute following command: <pre> docker images </pre> <pre> bhargav_raghavendra@ubuntu-vm:~\$ docker images REPOSITORY TAG IMAGE ID SIZE sb-mysql latest 2d5d7ab71ed5 3 days ago 425MB hello-world latest 74c546c27d64 2 weeks ago 10.1kB mysql 8.0 081ade299f20 2 weeks ago 764MB bhargav_raghavendra@ubuntu-vm:~\$ </pre>
3	Run mysql image to create mysql container using following command: <pre> docker run --name sb-mysql-db -e MYSQL_ROOT_PASSWORD=root -e MYSQL_DATABASE=mydb -e MYSQL_USER=myuser -e MYSQL_PASSWORD=root mysql:8.0 </pre> where: <code>docker run</code> → Creates and starts a new mysql container. <code>-d</code> → Runs the container in detached mode (in the background). <code>--name sb-mysql-db</code> → Assigns the container a name (sb-mysql-db). <code>-e MYSQL_ROOT_PASSWORD=root</code> → Sets the root password for MySQL (root). <code>-e MYSQL_DATABASE=mydb</code> → Creates a database named mydb. <code>-e MYSQL_USER=myuser</code> → Creates a new user (myuser). <code>-e MYSQL_PASSWORD=root</code> → Sets the password for myuser (root). <code>mysql:8.0</code> → Uses the MySQL version 8.0 Docker image. <pre> bhargav_raghavendra@ubuntu-vm:~\$ docker run -d --name sb-mysql-db -e MYSQL_ROOT_PASSWORD=root -e MYSQL_DATABASE=mydb -e MYSQL_USER=myuser -e MYSQL_PASSWORD=root mysql:8.0 cf81d43c240cf30307d8cf13cbb52ab8d0ec64828a7c8f2f44a3d8d423c bhargav_raghavendra@ubuntu-vm:~\$ </pre>
4	Verify if mysql container is running: <pre> docker ps </pre>

Pulling the MySQL image and running the sb-mysql-db container with environment variables

	<pre>mysql> use mydb; Database changed mysql></pre>
8	Now create employee table in selected database mydb: <pre>CREATE TABLE employee (employeeid int, firstName varchar(255), lastName varchar(255), Address varchar(255), City varchar(255));</pre> <pre>mysql> CREATE TABLE employee (-> employeeid int, -> firstName varchar(255), -> lastName varchar(255), -> Address varchar(255), -> City varchar(255) ->); Query OK, 0 rows affected (0.03 sec) mysql></pre>
9	Insert some records in employee table: <pre>insert into employee (employeeid, firstName, lastName, Address, City) values (1001, 'Myra', 'Bhargava', 'Indore', 'Indore'), (1002, 'Ravi', 'Paul', 'New Delhi', 'Delhi');</pre> <pre>mysql> insert into employee -> (employeeid, firstName, lastName, Address, City) -> values -> (1001, 'Myra', 'Bhargava', 'Indore', 'Indore'), -> (1002, 'Ravi', 'Paul', 'New Delhi', 'Delhi'); Query OK, 2 rows affected (0.02 sec) Records: 2 Duplicates: 0 Warnings: 0 mysql></pre>
10	Now retrieve all records from employee table: <pre>select * from employee;</pre>

Creating the employee table and querying inserted records inside the MySQL container

Activity 11: Networking in Docker (Create and Attach Network)

Prerequisite

Spring Boot application image and MySQL image are ready.

Scenario

Create a Spring Boot container and a MySQL container on the same custom Docker network so they can find and communicate with each other by container name.

Steps:

- Create a custom bridge network:

```
docker network create ltt_network
```

- Run the MySQL container (if not already running):

```
docker run -d --name sb-mysql-db -e MYSQL_ROOT_PASSWORD=root -e
MYSQL_DATABASE=mydb -e MYSQL_USER=myuser -e MYSQL_PASSWORD=root
mysql:8.0
```

- Attach the existing MySQL container to the custom network:

```
docker network connect ltt_network sb-mysql-db
```

- Run the Spring Boot container attached to the same network, passing DB connection details as env vars:

```
docker run --name sb-myapp --network ltt_network -e SPRING_DATASOURCE_URL=jdbc:mysql://sb-
mysql-db:3306/mydb -e SPRING_DATASOURCE_USERNAME=myuser -e
SPRING_DATASOURCE_PASSWORD=root -d rntsolutions/myapp
```

- Inspect the network to confirm both containers are attached:

```
docker network inspect ltt_network
```

- Verify connectivity by pinging the MySQL container's name from the Spring Boot container:

```
docker exec -it sb-myapp ping sb-mysql-db
```

- If 'ping' is not found inside the container, install it first: `docker exec -it sb-myapp sh`, then `apt update && apt install -y iputils-ping`, then retry the ping.

Activity 11: Create and Attach Network

11.1 Prerequisite

- AWS Cloud account is ready to use.
- Basic knowledge of docker commands.
- Springboot application image is ready.
- MySQL image is ready to use

11.2 Scenario

Create a Spring Boot application container and a MySQL container on the same Docker network to enable seamless communication between them so containers can find each other by name.

11.3 Steps

1	Create custom network for the SpringBoot and MySQL containers to communicate. <pre>docker network create ltt_network</pre> docker network create -> is used to create custom network. ltt_network-> is the name of custom network <pre>bhargav_raghavendra@my-ubuntu-vm:~\$ docker network create ltt_network 4bf856191ef7fa104780e2fcf9d9792e486b6c4f05bcb9e2fa29f4c0db622ab61 bhargav_raghavendra@my-ubuntu-vm:~\$</pre>
2	Now run MySQL container: <pre>docker run -d --name sb-mysql-db -e MYSQL_ROOT_PASSWORD=root -e MYSQL_DATABASE=mydb -e MYSQL_USER=myuser -e MYSQL_PASSWORD=root mysql:8.0</pre> <pre>bhargav_raghavendra@my-ubuntu-vm:~\$ docker run -d --name sb-mysql-db -e MYSQL_ROOT_PASSWORD=root -e MYSQL_DATABASE=mydb -e MYSQL_USER=myuser -e MYSQL_PASSWORD=root mysql:8.0 3aef8b2c13c1745929f4e7e5df70e24d51ab8114efec0d8d5b3492cfab</pre>
3	List mysql container: <pre>Docker ps</pre> <pre>bhargav_raghavendra@my-ubuntu-vm:~\$ docker ps CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES 3aef8b2c13c1 mysql:8.0 "docker-entrypoint.s..." 4 seconds ago Up 4 seconds 3306/tcp, 33060/tcp sb-mysql</pre>
4	Access the MySQL client inside the container: <pre>docker exec -it sb-mysql-db mysql -u myuser -p</pre> <pre>bhargav_raghavendra@my-ubuntu-vm:~\$ docker exec -it sb-mysql-db mysql -u myuser -p Enter password: Welcome to the MySQL monitor. Commands end with ; or \g. Your MySQL connection id is 10 Server version: 8.0.31 MySQL Community Server - GPL Copyright (c) 2000, 2025, Oracle and/or its affiliates. Oracle is a registered trademark of Oracle Corporation and/or its affiliates. Other names may be trademarks of their respective owners. Type 'help;' or '\h' for help. Type '\q' to clear the current input statement. mysql> \c mysql></pre>
5	Check mydb database which we have created: <pre>show databases;</pre>

Creating ltt_network and connecting the MySQL container to it

	<pre>mysql> show databases; +-----+ Database +-----+ information_schema mysql performance_schema +-----+ 3 rows in set (0.02 sec) mysql> []</pre>
6	Now attach mysql container to the ltt_network: docker network connect ltt_network sb-mysql-db ltt_network-> is custom network sb-mysql-db-> is mysql container which we have created in previous step. <pre>tharqev_raghavendra@my-ubuntu-vm:~\$ docker network connect ltt_network sb-mysql-db tharqev_raghavendra@my-ubuntu-vm:~\$ []</pre>
7	Now, run your Spring Boot container and attach it to the same network, use following command: docker run --name sb-myapp --network ltt_network -e SPRING_DATASOURCE_URL=jdbc:mysql://sb-mysql-db:3306/mydb -e SPRING_DATASOURCE_USERNAME=myuser -e SPRING_DATASOURCE_PASSWORD=root -d rntolutions/myapp sb-myapp container connected to the network ltt_network. Passes the MySQL connection details as environment variables to the Spring Boot application. <pre>tharqev_raghavendra@my-ubuntu-vm:~\$ docker run --name sb-myapp --network ltt_network -e SPRING_DATASOURCE_URL=jdbc:mysql://sb-mysql-db:3306/mydb -e SPRING_DATASOURCE_USERNAME=myuser -e SPRING_DATASOURCE_PASSWORD=root -d rntolutions/myapp 191b165d111146988081e2a280009144149811919450c8a2d5412011d3b5c2</pre>
8	Now list running containers: <pre>tharqev_raghavendra@my-ubuntu-vm:~\$ docker ps CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS names 551f16c47722 rntolutions/myapp "java -jar app.jar" 4 seconds ago Up 7 seconds 8080/tcp sb-myapp baad4b6c412c mysql:5.7 "docker-entrypoint.s..." 14 minutes ago Up 14 minutes 3306/tcp, 3306/tcp</pre>
9	Inspect the network to see the connected containers: docker network inspect ltt_network

docker network inspect output and successful ping between containers on the same network

Activity 12: Create and Mount Volume

Prerequisite

Basic knowledge of docker commands;
MySQL image is ready to use.

Scenario

Set up a MySQL container with a Docker volume so database data persists beyond the container's lifecycle — it is not lost when the container is deleted or restarted.

Steps:

- Create a named Docker volume:

```
docker volume create mysql-data
```

- Remove any existing MySQL container to avoid conflicts:

```
docker rm -f sb-mysql-db
```

- Run a new MySQL container, mounting the volume to MySQL's data directory:

```
docker run -d --name sb-mysql-db -e MYSQL_ROOT_PASSWORD=root -e
MYSQL_DATABASE=mydb -e MYSQL_USER=myuser -e MYSQL_PASSWORD=root
-v mysql-data:/var/lib/mysql mysql:8.0
```

- Verify the mount:

```
docker inspect sb-mysql-db
```

- Create data through the MySQL shell (employee table, inserts) as in Activity 10, then exit.

```
docker exec -it sb-mysql-db mysql -u myuser -p
```

- Stop and forcefully remove the container:

```
docker stop sb-mysql-db
```

```
docker rm -f sb-mysql-db
```

- Recreate the container with the SAME volume attached:

```
docker run -d --name sb-mysql-db -e MYSQL_ROOT_PASSWORD=root -e
MYSQL_DATABASE=mydb -e MYSQL_USER=myuser -e MYSQL_PASSWORD=root
-v mysql-data:/var/lib/mysql mysql:8.0
```

- Confirm the employee table and its rows are still present:

```
docker exec -it sb-mysql-db mysql -u myuser -p
```

```
use mydb;
```

```
select * from employee;
```

- Because the volume stores data outside the container's own writable layer, on the host system, the data survives container deletion/recreation — this is the fix for the data-loss issue seen in Activity 10.

	<pre> shargev_engshavendra@ubuntu-vm:~\$ docker network inspect ltt_network [{ "Name": "ltt_network", "ID": "4bf856191eff7a104780e2fc9d792e48d6d4705bcb9e2fa2979c0b622ab61", "Created": "2025-02-05T08:12:57.980349822Z", "Scope": "local", "Driver": "bridge", "EnableIPV6": false, "IPAM": { "Driver": "default", "Options": {}, "Config": [{ "Subnet": "172.18.0.0/16", "Gateway": "172.18.0.1" }] }, "Internal": false, "Attachable": false, "Ingress": false, "ConfigFrom": { "Network": "" }, "ConfigOnly": false, "Containers": { "391b7da9f72234e86806fa2a2805cc0447498ff9794dc66ac7d94fbb77d085c2": { "Name": "sb-myapp", "EndpointID": "6c6e130d82545128ac953b87083ff1eb2f412a3aaaa40b46a941558eefa5e518", "MacAddress": "02:42:ac:12:00:03", "IPv4Address": "172.18.0.3/16", "IPv6Address": "" } } }] </pre>
10	Verify connectivity between the Spring Boot container and the MySQL container by pinging MySQL from the Spring Boot container, ensuring both containers can communicate within the same network. <pre> docker exec -it sb-myapp ping sb-mysql-db shargev_engshavendra@ubuntu-vm:~\$ docker exec -it sb-myapp ping sb-mysql-db PING sb-mysql-db (172.18.0.2) 56(84) bytes of data: 64 bytes from sb-mysql-db:ltt_network (172.18.0.2): icmp_seq=1 ttl=64 time=0.241 ms 64 bytes from sb-mysql-db:ltt_network (172.18.0.2): icmp_seq=2 ttl=64 time=0.092 ms 64 bytes from sb-mysql-db:ltt_network (172.18.0.2): icmp_seq=3 ttl=64 time=0.099 ms 64 bytes from sb-mysql-db:ltt_network (172.18.0.2): icmp_seq=4 ttl=64 time=0.081 ms 64 bytes from sb-mysql-db:ltt_network (172.18.0.2): icmp_seq=5 ttl=64 time=0.069 ms 64 bytes from sb-mysql-db:ltt_network (172.18.0.2): icmp_seq=6 ttl=64 time=0.067 ms 64 bytes from sb-mysql-db:ltt_network (172.18.0.2): icmp_seq=7 ttl=64 time=0.107 ms 64 bytes from sb-mysql-db:ltt_network (172.18.0.2): icmp_seq=8 ttl=64 time=0.079 ms 64 bytes from sb-mysql-db:ltt_network (172.18.0.2): icmp_seq=9 ttl=64 time=0.065 ms 64 bytes from sb-mysql-db:ltt_network (172.18.0.2): icmp_seq=10 ttl=64 time=0.102 ms 64 bytes from sb-mysql-db:ltt_network (172.18.0.2): icmp_seq=11 ttl=64 time=0.074 ms 64 bytes from sb-mysql-db:ltt_network (172.18.0.2): icmp_seq=12 ttl=64 time=0.091 ms 64 bytes from sb-mysql-db:ltt_network (172.18.0.2): icmp_seq=13 ttl=64 time=0.092 ms 64 bytes from sb-mysql-db:ltt_network (172.18.0.2): icmp_seq=14 ttl=64 time=0.079 ms 64 bytes from sb-mysql-db:ltt_network (172.18.0.2): icmp_seq=15 ttl=64 time=0.063 ms </pre>
11	if ping command is not working then first install ping inside springboot container, using following commands: 1. First, open a shell inside the container <pre>docker exec -it sb-myapp sh</pre> 2. Try installing ping based on your container OS (for ubuntu): <pre>apt update && apt install -y iputils-ping</pre> Exit and Try the ping Command Again <pre>docker exec -it sb-myapp ping sb-mysql-db</pre>

Activity 12: Create and Mount Volume

12.1 Prerequisite

- AWS Cloud account is ready to use.

Creating the mysql-data volume and running the MySQL container mounted to it

	<pre> docker exec -it sb-mysql-db mysql -u myuser -p bharav_zaghavendra@my-ubuntu-vms:~\$ docker exec -it sb-mysql-db mysql -u myuser -p Enter password: Welcome to the MySQL monitor. Commands end with ; or \g. Your MySQL connection id is 8 Server version: 8.0.41 MySQL Community Server - GPL Copyright (c) 2000, 2025, Oracle and/or its affiliates. Oracle is a registered trademark of Oracle Corporation and/or its affiliates. Other names may be trademarks of their respective owners. Type 'help;' or '\h' for help. Type '\c' to clear the current input statement. mysql> </pre>
6	<pre> List databases: Show databases: mysql> show databases; +-----+ Database +-----+ information_schema mydb performance_schema +-----+ 3 rows in set (0.06 sec) mysql> </pre>
7	<pre> Select database mydb: use mydb; mysql> use mydb; Database changed mysql> </pre>
8	<pre> Create a table 'employee': CREATE TABLE employee (employeeId int, firstName varchar(255), lastName varchar(255), Address varchar(255), City varchar(255)); mysql> CREATE TABLE employee (-> employeeId int, -> firstName varchar(255), -> lastName varchar(255), -> Address varchar(255), -> City varchar(255) ->); Query OK, 0 rows affected (0.07 sec) mysql> </pre>
9	<pre> insert some records in the employee table: insert into employee (employeeId, firstName, lastName, Address, City) values (1001, 'Myra', 'Bhargava', 'Indore', 'Indore'), (1002, 'Ravi', 'Sharma', 'Delhi', 'Delhi'); mysql> insert into employee -> (employeeId, firstName, lastName, Address, City) -> values -> (1001, 'Myra', 'Bhargava', 'Indore', 'Indore'), -> (1002, 'Ravi', 'Sharma', 'Delhi', 'Delhi'); Query OK, 2 rows affected (0.07 sec) Records: 2 Duplicates: 0 Warnings: 0 'exit' from MySQL command line. </pre>
10	<pre> Now stop and remove mysql container: docker stop sb-mysql-db docker rm -f sb-mysql-db </pre>
11	<pre> Start new container with the same volume attached: </pre>

After removing and recreating the container, the employee table's data is still intact

Activity 13: Run Spring Boot and MySQL using Docker Compose

Prerequisite

Spring Boot application image ready;
MySQL image ready; basic knowledge
of docker commands.

Scenario

Use Docker Compose to launch the
Spring Boot application and MySQL
containers together with a single
command, instead of running them
individually.

Steps:

- Browse to the project folder containing the Dockerfile:

```
cd spring-boot
```

```
ls
```

- Create a docker-compose.yml file:

```
vi docker-compose.yml
```

- Paste the multi-service definition (MySQL service + Spring Boot service on the same network/volume):

```
version: '3.8'
```

```
services:
```

```
sb-mysql-db:
```

```
image: mysql:8.0
```

```
container_name: sb-mysql-db
```

```
restart: always
```

```
environment:
```

```
MYSQL_ROOT_PASSWORD: root
```

```
MYSQL_DATABASE: mydb
```

```
MYSQL_USER: myuser
```

```
MYSQL_PASSWORD: root
```

```
ports:
```

```
- "3306:3306"
```

```
volumes:
```

```
- mysql-data:/var/lib/mysql
```

```
sb-myapp:
```

```
build:
```

```
context: .
```

```
container_name: sb-myapp
```

```
restart: always
```

```
environment:
```

```
SPRING_DATASOURCE_URL: jdbc:mysql://sb-mysql-db:3306/mydb
```

```
SPRING_DATASOURCE_USERNAME: myuser
```

```
SPRING_DATASOURCE_PASSWORD: root
```



```

ports:
- "8080:8080"
depends_on:
- sb-mysql-db
volumes:
mysql-data:
    • Save and exit (:x in vi).
    • Build and start both containers (or run detached with -d):
docker compose up --build
# or
docker compose up -d
    • List running containers:
docker ps
    • Access the Spring Boot app in a browser: http://<public-ip-address>:8081
    • Verify the MySQL container's data via the MySQL shell:
docker exec -it sb-mysql-db mysql -u myuser -p
show databases;
    • Confirm the two containers can reach each other over the Compose-created
      network:
docker exec -it sb-myapp sh
ping sb-mysql-db

```

	<pre> bhargav_raghavendra@ubuntu-vm:~/spring-boot\$ vi docker-compose.yml </pre>
4	Copy docker-compose.yml script and paste in the vi editor: Refer spring-boot image and mysql image specification in docker compose script. <pre> version: '3.3' # Docker compose version services: db-mysql-db: image: mysql:5.6 # MySQL image version container_name: db-mysql-db # Name of the container restart: always # Automatically restart if it crashes environment: MYSQL_ROOT_PASSWORD: root # Root password for MySQL MYSQL_DATABASE: mydb # Database name MYSQL_USER: myuser # User for the database MYSQL_PASSWORD: root # Password for the user ports: - "3306:3306" # Map MySQL port to localhost volumes: - mysql-data:/var/lib/mysql # Persist MySQL data db-myspring: build: . # Build the app from Dockerfile in the "app" folder container_name: db-myspring # Name of the app container restart: always environment: SPRING_DATASOURCE_URL: jdbc:mysql://db-mysql-db:3306/mydb SPRING_DATASOURCE_USERNAME: myuser SPRING_DATASOURCE_PASSWORD: root ports: - "8080:8080" # Map Spring Boot app port to localhost depends_on: - db-mysql-db # Ensure MySQL starts before the app volumes: - mysql-data: # Named volume for MySQL persistence "docker-compose.yml" 34L, 1312C </pre> Press 'I' to enter edit mode if needed. After making changes, press Esc to exit insert mode. Then, type :x and press Enter to save and exit the vi editor.
5	Now build and run docker compose: docker compose up --build or run containers in detached mode: docker compose up -d (detached mode)
6	List running containers: docker ps <pre> bhargav_raghavendra@ubuntu-vm:~/spring-boot\$ docker ps CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS b89d3d34144f spring-boot-db-myspring "java -jar app.jar" 5 seconds ago Up 4 seconds 8080/tcp, 0.0.0.0:3306->3306/tcp 93ba0432b0 mysql:5.6 "docker-entrypoint.sh" 5 seconds ago Up 4 seconds 0.0.0.0:3306->3306/tcp, 33060/tcp </pre>
7	Access and verify if springboot application is working: http://public-ip-address:port 
8	Verify and access mysql container through mysql command line client: Open mysql command line client: docker exec -it sb-mysql-db mysql -u myuser -p

docker-compose.yml defining the MySQL and Spring Boot services together

```

k8s@k8s-engine:~/spring-boot$ docker exec -it sb-mysql-db mysql -u myuser -p
Enter password:
Welcome to the MySQL monitor. Commands end with ; or \g.
Your MySQL connection id is 2
Server version: 8.0.41 MySQL Community Server - GPL
Copyright (c) 2000, 2025, Oracle and/or its affiliates.
Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.
Type 'help;' or '\h;' for help. Type '\c;' to clear the current input statement.

mysql> show databases;
+-----+
| Database |
+-----+
| information_schema |
| mysql |
| performance_schema |
+-----+
3 rows in set (0.01 sec)

mysql>

```

9 Access the Spring Boot application container shell and ping the MySQL container:
 docker exec -it sb-myapp sh
 and then ping mysql container:
 ping sb-mysql-db

```

k8s@k8s-engine:~/spring-boot$ docker exec -it sb-myapp sh
$ ping sb-mysql-db
sh: 1: ping: not found
$ apt update
Get:1 http://deb.debian.org/debian bullseye InRelease [116 kB]
Get:2 http://security.debian.org/debian-security bullseye-security InRelease [77.2 kB]
Get:3 http://deb.debian.org/debian bullseye-updates InRelease [41.1 kB]
Get:4 http://security.debian.org/debian-security bullseye-security/main amd64 Packages [344 kB]
Get:5 http://deb.debian.org/debian bullseye/main amd64 Packages [804 kB]
Get:6 http://deb.debian.org/debian bullseye-updates/main amd64 Packages [10.6 kB]
Fetched 816 kB in 2s (440 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
44 packages can be upgraded. Run 'apt list --upgradable' to see them.
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  libc-bin libc6 libc6-dev libc6-i386 libc6-rtm libc6-sse2 libc6-x32 libc6-x86-64
  libc6-x86-64-dbg libc6-x86-64-glibc
The following NEW packages will be installed:
  libc-bin libc6 libc6-dev libc6-i386 libc6-rtm libc6-sse2 libc6-x32 libc6-x86-64
  libc6-x86-64-dbg libc6-x86-64-glibc
0 upgraded, 10 newly installed, 0 to remove and 44 not upgraded.
Need to get 10.6 MB of archives.
After this operation, 42.7 MB of additional disk space will be used.
Do you want to continue? [Y/n]

```

Both containers running via Docker Compose, and successful ping between them

4. Viva Voce Questions — Docker

Q: What is Docker?

A: Docker is an open-source platform that packages applications and their dependencies into lightweight, portable units called containers, which run consistently across any environment.

Q: How is a container different from a virtual machine?

A: A VM virtualizes an entire OS (including its kernel) on a hypervisor and is relatively heavy/slow to boot. A container shares the host OS kernel and only isolates the application layer, making it lightweight and fast to start.

Q: What is a Docker image?

A: A read-only, layered template containing application code, runtime, libraries, and configuration, built from a Dockerfile. Containers are created from images.

Q: What is a Docker container?

A: A running (or stopped) instance of a Docker image, consisting of the image's layers plus a thin writable layer on top.

Q: What is a Dockerfile?

A: A text file containing step-by-step instructions (FROM, WORKDIR, COPY, EXPOSE, CMD, etc.) that Docker uses to build an image.

Q: What do the FROM, COPY, EXPOSE, and CMD instructions do?

A: FROM sets the base image; COPY copies files from the host into the image; EXPOSE documents the port the app listens on; CMD specifies the default command run when a container starts.

Q: What is the difference between an image and a container?

A: An image is a static blueprint; a container is a live, running instance of that image. Many containers can be started from a single image.

Q: What is Docker Hub?

A: A public/private cloud registry for storing and distributing Docker images, similar to how GitHub hosts code. 'docker push' uploads images; 'docker pull' downloads them.

Q: What command builds an image from a Dockerfile in the current directory?

A: `docker build -t <image-name> .`

Q: How do you run a container in detached (background) mode and map ports?

A: `docker run -d -p <host-port>:<container-port> <image-name>`, e.g. `docker run -d -p 8080:8080 myapp:latest`.

Q: What command lists running containers? What about all containers (including stopped)?

A: `docker ps` lists running containers; `docker ps -a` lists all containers, running or stopped.

Q: What is the difference between 'docker stop' and 'docker pause'?

A: `docker stop` gracefully terminates the main process in the container (container stops but is not deleted). `docker pause` freezes all processes in the container without stopping them; `docker unpause` resumes them.

Q: How do you forcefully remove a running container?

A: `docker rm <container-id> -f` — the `-f` flag stops the container first and then removes it.

Q: How do you tag an image, and why would you?

A: `docker tag <image>:<tag> <new-name>:<new-tag>`. Tagging lets you version images or prepare them with a registry-qualified name (e.g. `username/repo:tag`) before pushing.

Q: What happens to data stored inside a MySQL container if the container is deleted?

A: All data is lost, because by default it is stored in the container's own writable layer, which is destroyed along with the container.

Q: How does a Docker volume solve the data-persistence problem?

A: A volume stores data outside the container's writable layer, on the Docker host. When mounted with `-v <volume>:<path>`, data written to that path persists even if the container is stopped, removed, or recreated.

Q: What is a Docker network used for?

A: It lets containers communicate with each other, typically by container name, over an isolated virtual network created with `'docker network create'`.

Q: How do two containers on the same custom network find each other?

A: Docker's embedded DNS resolves each container's name to its IP address automatically within that network, so containers can ping/connect using the container's `--name` instead of an IP.

Q: What is Docker Compose and why use it?

A: A tool for defining and running multi-container applications using a single `docker-compose.yml` file, so related services (e.g. an app + its database) can be started, networked, and stopped together with one command instead of many manual `docker run` commands.

Q: What does `'docker compose up --build'` do?

A: It builds images (if a build context is defined) and starts all services defined in `docker-compose.yml`, wiring up their declared networks, volumes, and environment variables.

Q: What does `depends_on` do in `docker-compose.yml`?

A: It controls startup order, ensuring one service (e.g. the database) is started before a dependent service (e.g. the application) — though it does not guarantee the dependency is fully 'ready', only that it has started.

Q: What command shows detailed configuration/runtime info of a container, including mounted volumes?

A: `docker inspect <container-name-or-id>`.

Java Full Stack + Gen AI Training

DevOps to GenAI Module Study Guide

Container Orchestration with Kubernetes → Agile & Scrum → GenAI & Agentic AI

Theory • Hands-on Activities with Screenshots • Demo Programs • Viva Voce Questions

Aligned to: QuantumCrew Java Full Stack + Gen AI — Day-Wise Plan (Rows 56, 58, 59, 60 & 61)

Part 1 — Container Orchestration with Kubernetes

Part 2 — Agile Principles, Scrum Practices & Iterative Development

Part 3 — GenAI & Agentic AI

Contents

Part 1 — Container Orchestration with Kubernetes

- 1. Introduction to Kubernetes
- 2. Kubernetes Architecture
- 3. Basic Concepts You Should Know First
- 4. Hands-on Activities (11 activities, with screenshots)
- 5. Viva Voce Questions

Part 2 — Agile Principles, Scrum Practices & Iterative Development

- 1. Basic Concepts: SDLC and Software Process Models
- 2. Agile Principles
- 3. Scrum Framework (Roles, Events, Artifacts)
- 4. Iterative Development with Jira & Confluence
- 5. Viva Voce Questions

Part 3 — GenAI & Agentic AI

- 1. Basic Concepts: AI, ML, Deep Learning and GenAI
- 2. Introduction to Generative AI
- 3. GenAI in Java
- 4. Agentic AI (and GenAI vs Agentic AI)
- 5. Prompt Engineering
- 6. AI Testing
- 7. GitHub Copilot

- 8. Introduction to Spring AI
- 9. Demo Programs (Vertex AI Gemini x2, Spring AI + Ollama)
- 10. Viva Voce Questions

PART 1

Container Orchestration with Kubernetes

1. Introduction to Kubernetes

Kubernetes (often abbreviated K8s) is an open-source container orchestration platform, originally designed by Google and now maintained by the Cloud Native Computing Foundation (CNCF). It automates the deployment, scaling, networking, and lifecycle management of containerized applications across a cluster of machines.

Once an application is packaged into a Docker image, that image still has to be run reliably in production: it must be restarted if it crashes, scaled up under load, rolled out with zero downtime, and load-balanced across multiple copies. Doing all of this by hand with plain 'docker run' commands does not scale beyond a handful of containers. Kubernetes solves this by letting an engineer describe the desired state of the system in a declarative manifest, and continuously reconciling the real cluster state to match that desired state.

Why Kubernetes?

- Self-healing: automatically restarts failed containers, replaces and reschedules pods when nodes die, and kills containers that do not respond to health checks.
- Horizontal scaling: pods can be scaled manually or automatically (Horizontal Pod Autoscaler) based on CPU/memory or custom metrics.
- Service discovery & load balancing: Kubernetes gives pods their own IP addresses and a single stable DNS name for a set of pods, and load-balances traffic across them.
- Automated rollouts and rollbacks: Kubernetes can progressively roll out changes and roll back automatically if something goes wrong.
- Declarative configuration: the desired state of the cluster is described in YAML manifests that can be version-controlled, reviewed and reused (GitOps).
- Portability: the same manifests can run on-premises, or on any major cloud provider (EKS, AKS, GKE) or on a local single-node cluster such as Minikube.

2. Kubernetes Architecture

A Kubernetes cluster is made up of a Control Plane (the 'brain' of the cluster) and one or more Worker Nodes (the machines that actually run the application containers).

2.1 Control Plane Components

- kube-apiserver: the front door of the cluster. Every kubectl command and every internal component talks to the cluster only through the API server's REST interface.
- etcd: a consistent, highly-available key-value store that holds the entire state and configuration of the cluster (the 'source of truth').
- kube-scheduler: watches for newly created pods that have no node assigned, and picks the best node for them based on resource requirements, affinity rules, taints/tolerations, etc.
- kube-controller-manager: runs controller loops (Node controller, Replication controller, Endpoints controller, etc.) that continuously watch the cluster state and drive it towards the desired state.
- cloud-controller-manager: integrates the cluster with the underlying cloud provider's APIs (load balancers, storage volumes, node lifecycle).

2.2 Node (Worker) Components

- kubelet: an agent that runs on every node, ensures containers described in PodSpecs are running and healthy, and reports node/pod status back to the control plane.
- kube-proxy: maintains network rules on each node that allow network communication to pods from inside or outside the cluster, and implements the Service abstraction.
- Container runtime: the software that actually runs containers (containerd, CRI-O, or Docker Engine through a shim).

2.3 Core Kubernetes Objects

- Pod: the smallest deployable unit in Kubernetes; a group of one or more containers that share storage, network (a single IP address) and a specification for how to run.
- ReplicaSet: ensures a specified number of identical pod replicas are running at any given time.
- Deployment: a higher-level object that manages ReplicaSets for you and provides declarative updates, rolling upgrades and rollbacks for pods.
- Service: a stable network endpoint (virtual IP + DNS name) that load-balances traffic to a dynamic set of pods selected by labels. Common types are ClusterIP (internal only), NodePort (exposes a port on every node)

and LoadBalancer (provisions an external load balancer, e.g. via 'minikube tunnel' locally).

- Namespace: a way to divide cluster resources between multiple users, teams or environments.
- ConfigMap / Secret: externalize configuration data and sensitive values (passwords, tokens) from container images so the same image can run in dev/test/prod with different configuration.
- PersistentVolume (PV) / PersistentVolumeClaim (PVC): abstractions that let pods request and consume durable storage independent of the pod's own lifecycle (used to persist MySQL data, for example).
- HorizontalPodAutoscaler (HPA): automatically increases or decreases the number of pod replicas in a Deployment/ReplicaSet based on observed CPU utilization or other custom metrics.
- Minikube: a lightweight, single-node Kubernetes cluster that runs inside a VM or container on a local/dev machine (used throughout this module's activities); kubectl is the command-line client used to talk to any Kubernetes cluster, local or remote.

2.4 Kubernetes Object Lifecycle (Conceptual)

Write a manifest (YAML) describing the desired state --> 'kubectl apply -f' sends it to the API server --> API server persists it in etcd --> the scheduler assigns pods to nodes --> the kubelet on each node pulls the container image and starts the containers --> controllers continuously reconcile actual state with desired state (restarting/rescheduling pods as needed).

3. Basic Concepts You Should Know First

Kubernetes builds directly on top of containerization concepts. Before working through the activities, it helps to be clear on a few foundational ideas.

3.1 Container vs Virtual Machine

- A Virtual Machine (VM) virtualizes an entire physical machine, including its own guest operating system kernel, and is managed by a hypervisor. VMs are heavier (typically gigabytes in size) and slower to boot (minutes).
- A container virtualizes only at the operating-system level: containers on the same host share the host's kernel but have their own isolated filesystem, process space and network interface. Containers are lightweight (typically megabytes) and start in seconds, which is what makes it practical to run many of them and scale them up/down quickly.
- A container image is a read-only, versioned package (built from a Dockerfile) containing an application and everything it needs to run - code, runtime, libraries, environment variables and default configuration. A container is simply a running instance of that image.

3.2 Why Orchestration Is Needed

- Running a single 'docker run' command on one machine works fine for a demo, but production systems need many container instances spread across many machines, with automatic restart on failure, load balancing, rolling updates and the ability to add/remove capacity - manually managing this quickly becomes unworkable.
- An orchestrator (Kubernetes, Docker Swarm, Amazon ECS, etc.) is the layer of software that automates exactly this: placing containers on machines, keeping the desired number running, routing traffic to them, and healing the system when something fails. Kubernetes has become the de-facto industry standard orchestrator.

3.3 YAML Basics (the language Kubernetes manifests are written in)

- YAML (YAML Ain't Markup Language) is a human-readable, indentation-based data format. Indentation (spaces, never tabs) defines nesting/structure - there are no braces or brackets like JSON.
- key: value defines a field, e.g. 'replicas: 3'. A dash '-' starts a list item, e.g. a list of containers or ports.
- Every Kubernetes manifest conventionally has four top-level fields: apiVersion (which version of the Kubernetes API this object belongs to, e.g. apps/v1), kind (the type of object, e.g. Deployment, Service, Pod), metadata (name, labels, namespace) and spec (the desired state specific to that kind of object).
- Three dashes '---' on their own line separate multiple objects defined inside the same YAML file - this is how Activity 8 defines both a Deployment and a Service in a single deployment.yml.

3.4 Kubernetes Networking Basics

- Every Pod gets its own unique cluster-internal IP address, and all containers inside that Pod share it (they can reach each other via 'localhost').
- Pod IPs are ephemeral - a Pod that is deleted and recreated gets a new IP. This is precisely why Services exist: a Service gives a stable virtual IP and DNS name that automatically routes to whichever healthy Pods currently match its label selector, so other components never need to track individual Pod IPs.
- Inside the cluster, Services are reachable by DNS name (e.g. a Service named 'mysql' in the default namespace can be reached simply as 'mysql' or the fully-qualified 'mysql.default.svc.cluster.local').

3.5 kubectl Command Cheat-Sheet

- kubectl get <resource> - list resources (pods, deployments, svc, nodes, hpa, all, ...); add '-A' or '--all-namespaces' to see every namespace.

- `kubectl describe <resource> <name>` - detailed information and recent Events for one resource (the first place to look when something is failing).
- `kubectl apply -f <file>.yaml` - create or update resources declaratively from a manifest file (the recommended way to manage resources).
- `kubectl delete <resource> <name>` - remove a resource.
- `kubectl logs <pod-name>` - view a container's stdout/stderr output.
- `kubectl exec <pod-name> -it -- <command>` - run a command (e.g. a shell) inside a running container.
- `kubectl scale` / `kubectl autoscale` - manually or automatically change the number of replicas of a Deployment.

3.6 Docker Compose vs Kubernetes

- Docker Compose is a simple tool for defining and running multi-container applications on a single Docker host - convenient for local development, but it has no built-in concept of scheduling across multiple machines, self-healing across nodes, or rolling updates with health-based traffic shifting.
- Kubernetes solves the same 'define my multi-container app declaratively' problem, but is designed from the ground up to run across a cluster of many machines, with self-healing, scaling and load balancing built in - which is why it is the standard choice for production, while Compose remains popular for local development.

4. Hands-on Activities

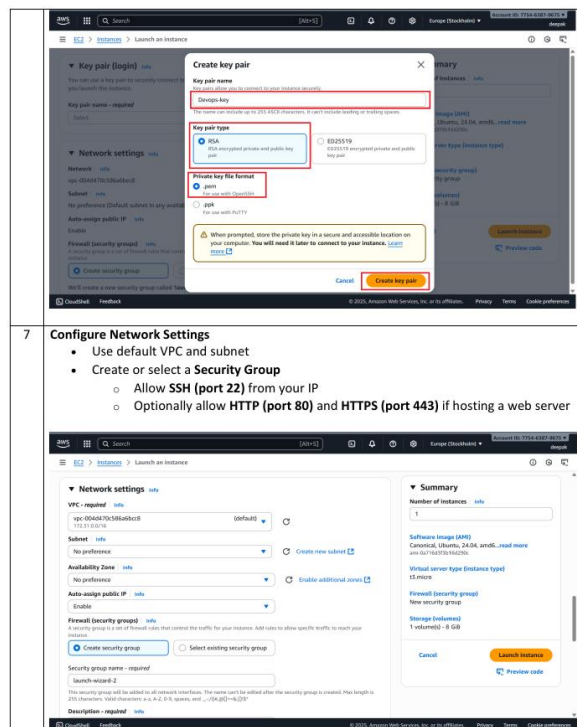
The activities below were performed on an AWS EC2 Ubuntu instance. Screenshots are taken directly from the lab environment used for this module.

Activity 1: Software Setup

Prerequisite	A running EC2 virtual machine with Ubuntu.
Scenario	Install Docker, JDK 17 and Maven on the Ubuntu VM so it can build and containerize the Spring Boot application used in later activities.

- Log in to the AWS Console, open the EC2 dashboard and click Launch Instance.
- Choose an Ubuntu 22.04/24.04 LTS AMI marked Free Tier eligible, and select instance type t2.micro/t3.micro.
- Create/select a key pair, configure the security group (allow SSH 22, and optionally HTTP 80/HTTPS 443), and configure storage (up to 30 GB gp3 under Free Tier).
- Launch the instance, then connect to it (EC2 Instance Connect / SSH).

- Update packages: `sudo apt update`
- Install Docker: `sudo apt install docker.io`, then add the current user to the docker group: `sudo usermod -aG docker $USER`, and reboot: `sudo reboot`
- Verify Docker: `docker --version`
- Install JDK 17: `sudo apt install openjdk-17-jdk-headless -y`
- Install Maven: `sudo apt install maven -y`



Launching the EC2 instance that hosts Docker, JDK 17 and Maven for the Kubernetes labs.

Activity 2: Setup kubectl

Prerequisite Ubuntu VM with Docker installed.

Scenario	Install kubectl, the command-line tool used to interact with any Kubernetes cluster.
----------	--

- Download the latest kubectl release: `curl -LO "https://dl.k8s.io/release/$(curl -L -s https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"`
- Download the checksum file and validate the binary using `sha256sum --check`.
- Install kubectl: `sudo install -o root -g root -m 0755 kubectl /usr/local/bin/kubectl`
- Verify: `kubectl version --client`



Activity 2: Setup kubectl

2.1. Prerequisite

Ubuntu VM with docker installed

2.2. Scenario

Installing kubectl

2.3. Steps

	Go to vm
1	
2	• Download latest release • Command: <code>curl -LO "https://dl.k8s.io/release/\$(curl -L -s https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"</code>
3	• Download kubectl checksum. • Command: <code>curl -LO "https://dl.k8s.io/release/\$(curl -L -s https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl.sha256"</code>
4	• Validate kubectl binary. • Command: <code>echo "\$(cat kubectl.sha256) kubectl" sha256sum --check</code>

Downloading, verifying and installing the kubectl binary.

Activity 3: Setup Minikube Cluster

Prerequisite	Ubuntu VM with Docker and kubectl installed.
Scenario	Install and start a local single-node Kubernetes cluster using Minikube.

- Download the Minikube Debian package: `curl -LO https://github.com/kubernetes/minikube/releases/latest/linux-amd64`
- Install it: `sudo install minikube-linux-amd64 /usr/local/bin/minikube && rm minikube-linux-amd64`
- Start the cluster using the Docker driver: `minikube start --driver=docker --force`
- Verify all system pods are running: `kubectl get po -A`

	
5	• Install kubectl. • Command: <code>sudo install -o root -g root -m 0755 kubectl /usr/local/bin/kubectl</code> 
6	• Check installation • Command: <code>kubectl version --client</code> 

Activity 3: Setup minikube cluster

3.1. Prerequisite

Ubuntu VM with docker, kubectl installed

3.2. Scenario

Installing minikube cluster

3.3. Steps

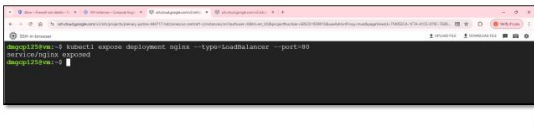
1	Go to vm
2	• Get latest Debian package • Command: <code>curl -LO https://github.com/kubernetes/minikube/releases/latest/download/minikube-linux-amd64</code>

Installing Minikube and starting a single-node cluster with the Docker driver.

Activity 4: Kubernetes Commands

Prerequisite	Minikube cluster started.
Scenario	Practice the fundamental kubectl commands for pods, deployments and services.

- List all pods: `kubectl get pods -A` / `kubectl get po -A`
- Create a stand-alone pod: `kubectl run nginx --image=nginx --restart=Never`, then list it with `kubectl get pods`.
- Delete a pod: `kubectl delete pod nginx`
- Create a Deployment: `kubectl create deployment nginx --image=nginx:latest`, then list it with `kubectl get deployment`.
- Expose the Deployment as a LoadBalancer Service: `kubectl expose deployment nginx --type=LoadBalancer --port=80`
- Start minikube tunnel (in its own terminal) to make the LoadBalancer's external IP reachable, then curl the external IP on port 80.
- Inspect deeper: `kubectl logs <pod-name>`, `kubectl get pods -o wide`, `kubectl describe pod <pod-name>`, `kubectl get rs`, `kubectl get all`.
- Export a live object to YAML: `kubectl get deployment nginx -o yaml » deployment.yaml`
- Create a pod declaratively from a YAML manifest: `vi pod.yml`, then `kubectl apply -f pod.yml`

	
4	Delete pod Command: <code>kubectl delete pod nginx</code> 
5	Create deployment Command: <code>kubectl create deployment nginx --image=nginx:latest</code>  List deployment Command: <code>kubectl get deployment</code> 
6	Expose deployment Command: <code>kubectl expose deployment nginx --type=LoadBalancer --port=80</code>  List services

Working with pods, deployments and the LoadBalancer service for the nginx test workload.

Activity 5: Deploy Spring Boot Application

Prerequisite	Minikube cluster started.
Scenario	Containerize the 'easyfly' Spring Boot application and deploy it to the Minikube cluster.

- Fork and clone the sample Spring Boot repository (`git clone .../easyfly.git`), then build it: `cd easyfly && mvn clean install`
- Author a Dockerfile-dev that copies the built jar and starts it on the dev Spring profile (`server.port=8081`).

- Log in to Docker Hub (docker login) using a personal access token, build the image (docker build -t <user>/easyfly:dev -f Dockerfile-dev .), and push it (docker push <user>/easyfly:dev).
- Create a Deployment directly from the image: kubectl create deployment easyfly-app --image=<user>/easyfly:dev
- Expose it as a LoadBalancer service on port 8081, start minikube tunnel, and access the running REST API (e.g. GET /node_details, GET/POST/PUT/DELETE on /api-v1/flights).

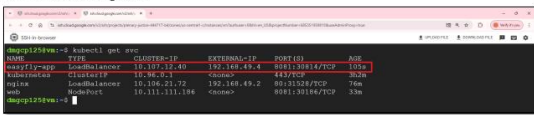
	<pre>mvn clean install</pre> <pre>ls target</pre> 
4	Create dockerfile in root directory. <pre>touch Dockerfile-dev</pre> <pre>vim Dockerfile-dev</pre> To insert text, press 'i'. After typing your content, press 'Esc' to exit insert mode. If you want to save and exit at the same time, type ':wq' and press 'Enter'. <pre>cat Dockerfile-dev</pre> <pre>#this is dockerfile for development</pre> <pre>#use Dockerfile-dev and edit it</pre> <pre>FROM --platform=linux/arm64 eclipse-temurin:17-jdk</pre> <pre>COPY target/*.jar app.jar</pre> <pre>ENV server.port=8081</pre> <pre>ENTRYPOINT ["java", "-Dspring.profiles.active=dev", "-jar", "/app.jar"]</pre>
5	Login to Docker Hub <pre>docker login</pre> enter username of Docker Hub account and enter pass code Build docker image Command: docker build -t dadaramjadhav/easyfly:dev -f Dockerfile-dev 
6	Create personal access token in dockerhub a. Login into docker hub b. Go to https://app.docker.com/settings/personal-access-tokens c. Click on generate new token d. Enter token name e. Choose expiration time f. Choose access permission (Read, Write, Delete) g. Click on generate

Building, pushing and deploying the containerized Spring Boot 'easyfly' service.

Activity 6: Scale the Spring Boot Application

Prerequisite	Spring Boot application running inside the Minikube cluster.
Scenario	Scale the number of running pod replicas up and down manually to handle changing load.

- Scale up: `kubectl scale deploy easyfly-app --replicas=3`, then verify with `kubectl get deploy` / `kubectl get pods`.
- Repeatedly curl the service endpoint - responses come back from different pod hostnames/IPs, proving requests are load-balanced across replicas.
- Scale down: `kubectl scale deploy easyfly-app --replicas=2`, then scale back up: `kubectl scale deploy easyfly-app --replicas=5`, confirming with `kubectl get deploy` each time.

	Command: <code>kubectl get svc</code> 
14	Access application Command: <code>curl 192.168.49.4:8081/node_details</code>  Other endpoints 1. <code>curl 192.168.49.4:8081/api-v1/flights/</code> (GET) 2. <code>curl 192.168.49.4:8081/api-v1/flights/3</code> (GET) 3. <code>curl --header "Content-Type: application/json" --request POST --data '{"flightNumber": "JA-506", "source": "Pune", "destination": "Mumbai", "status": "Running", "duration": 3, "fare": 3000}' 192.168.49.4:8081/api-v1/flights/</code> (POST Request) 4. <code>curl --header "Content-Type: application/json" --request PUT --data '{"flightid": 6, "flightNumber": "JA-506", "source": "Pune", "destination": "Mumbai", "status": "Running", "duration": 3, "fare": 3000}' 192.168.49.4:8081/api-v1/flights/6</code> (PUT request) 5. <code>curl --request DELETE 192.168.49.4:8081/api-v1/flights/6</code> (DELETE)

Activity 6: Scale spring boot application

6.1. Prerequisite

Spring boot application running inside minikube cluster

6.2. Scenario

Scaling spring boot application to handle increasing load

6.3. Steps

1	Go to vm
2	Scale deployment

Scaling the easyfly-app Deployment and confirming traffic is spread across replicas.

Activity 7: Autoscaling the Spring Boot Application

Prerequisite	Spring Boot application running.
Scenario	Configure a Horizontal Pod Autoscaler (HPA) so replica count adjusts automatically to CPU load.

- Create an HPA: `kubectl autoscale deployment easyfly-app --cpu-percent=50 --min=1 --max=10`
- This defines an HPA that keeps between 1 and 10 replicas of easyfly-app, adjusting the count to keep average CPU utilization near 50%.
- Check its status: `kubectl get hpa`



```

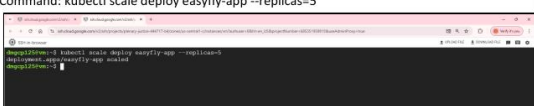
$ kubectl get deploy easyfly-app
NAME          READY   UP-TO-DATE   AVAILABLE   AGE
easyfly-app   2/2     2             2           11m

```

Keep hitting this endpoint to confirm scaling
`curl 192.168.49.4:8081/node_details`

5 Scale up application

Command: `kubectl scale deploy easyfly-app --replicas=5`



```

$ kubectl scale deploy easyfly-app --replicas=5
deployment.apps/easyfly-app scaled

```

List deployment

Command: `kubectl get deploy`



```

$ kubectl get deploy
NAME          READY   UP-TO-DATE   AVAILABLE   AGE
easyfly-app   5/5     5             5           27m

```

Keep hitting this endpoint to confirm scaling
`curl 192.168.49.4:8081/node_details`

Activity 7: Autoscaling spring boot application

7.1. Prerequisite

Spring boot application running

7.2. Scenario

Autoscaling spring boot application to handle dynamic load

7.3. Steps

1	Go to vm
2.	Autoscale application based on CPU utilization

Creating a CPU-based Horizontal Pod Autoscaler for the easyfly-app deployment.

Activity 8: Deploying a Spring Boot Application Using a Manifest File

Prerequisite	Spring Boot application.
Scenario	Move from imperative kubectl commands to a single declarative deployment.yml describing both the Deployment and the Service.

- Clone the source, build it (mvn clean install), build and push the Docker image as before.
- Author deployment.yml containing a Deployment (3 replicas, container port 8081) and a Service of type LoadBalancer, in one file separated by '---'.
- Apply it in one shot: kubectl apply -f deployment.yml
- Verify with kubectl get deployments / kubectl get pods / kubectl get svc, start minikube tunnel, and access the app and its REST endpoints via the LoadBalancer external IP.

```

metadata:
  labels:
    app: easyfly
  spec:
    containers:
      - name: easyfly
        image: dadaramjadhav/easyfly:dev
        ports:
          - containerPort: 8081
---
apiVersion: v1
kind: Service
metadata:
  name: easyfly-svc
spec:
  selector:
    app: easyfly
  ports:
    - protocol: "TCP"
      port: 8081
      targetPort:
        type: LoadBalancer

```

Deployment
Command: `kubectl apply -f deployment.yml`



```

check deployment
kubectl get deployments

check pods
kubectl get pods

check services
kubectl get svc

start minikube tunnel
minikube tunnel

open new terminal

```

Applying a combined Deployment + Service manifest and validating the resulting pods and service.

Activity 9: Changing the Deployment Image

Prerequisite	Spring Boot application.
Scenario	Ship a new application version (api-v2) by updating only the container image referenced in the manifest.

- Pull the updated source code (git pull), rebuild it, and build/push a new image tag (e.g. dadaramjadhav/easyfly:dev-v2).
- Update the 'image:' field inside deployment.yml to point at the new tag, keeping everything else the same.

- Re-apply the manifest: `kubectl apply -f deployment.yml` - Kubernetes performs a rolling update, replacing old pods with new ones without downtime.
- Confirm the new version is live by calling `/node_details` or the new `/api-v2/...` endpoints.

	
5	Build docker image Command: <code>docker build -t dadaramjadhav/easyfly:dev-v2 -f Dockerfile-dev .</code> 
6	Push image into docker hub Command: <code>docker push dadaramjadhav/easyfly:dev-v2</code> 
7	Updated deployment.yml <pre> apiVersion: apps/v1 kind: Deployment metadata: name: easyfly spec: selector: matchLabels: app: easyfly replicas: 5 template: metadata: labels: app: easyfly spec: containers: - name: easyfly image: dadaramjadhav/easyfly:dev-v2 ports: - containerPort: 8081 --- apiVersion: v1 kind: Service metadata: name: easyfly-svc </pre>

Rolling out a new image version (v2) purely by editing and re-applying the manifest.

Activity 10: Deploying a Spring Boot App with MySQL

Prerequisite Spring Boot application with MySQL.

Scenario	Run a stateful MySQL database alongside the stateless Spring Boot app, using a PersistentVolumeClaim so data survives pod restarts.
-----------------	---

- Build and push a 'prod' image of the app (Dockerfile-prod).
- Author db-deployment.yml containing a PersistentVolumeClaim, a MySQL Deployment (image mysql:5.7, env vars for root password and database name, a volume mount at /var/lib/mysql) and a headless Service (clusterIP: None) named 'mysql'.
- Apply it: `kubectl apply -f db-deployment.yml`, then exec into the MySQL pod (`kubectl exec <pod> -it -- /bin/sh`) and confirm the database exists (`mysql -u root -p, SHOW DATABASES;`).
- Deploy the Spring Boot app itself with its own Deployment + LoadBalancer Service (deployment.yml), and access its endpoints, which now read/write to the MySQL pod.


```

accessModes:
- ReadWriteOnce
resources:
  requests:
    storage: 1Gi
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mysql
labels:
  app: mysql
  tier: database
spec:
  selector:
    matchLabels:
      app: mysql
      tier: database
  strategy:
    type: Recreate
  template:
    metadata:
      labels:
        app: mysql
        tier: database
    spec:
      containers:
        - image: mysql:5.7
          args:
            - "--ignore-db-dir=lost+found"
          name: mysql
          env:
            - name: MYSQL_ROOT_PASSWORD
              value: root
            - name: MYSQL_DATABASE
              value: easyfly_db

      ports:
        - containerPort: 3306
          name: mysql
      volumeMounts:
        - name: mysql-persistent-storage
          mountPath: /var/lib/mysql
      volumes:
        - name: mysql-persistent-storage
          persistentVolumeClaim:

```

Verifying the MySQL pod's database and accessing the Spring Boot app backed by it.

Activity 11: Deploying a Spring Boot App with CPU and Memory Limits

Prerequisite	Spring Boot application.
Scenario	Constrain how much CPU and memory each pod may request and consume, to protect the node from a single noisy application.

- Add a 'resources' block under the container spec with requests (cpu: 500m, memory: 1Gi) and limits (cpu: 1000m, memory: 2Gi).

- Apply the manifest: `kubectl apply -f deployment-with-cpu-memory-limits.yml`
- Confirm with `kubectl get deploy` and `kubectl describe deploy <name>`, which prints the configured Limits and Requests for the container.

	<pre>spec: selector: matchLabels: app: easyfly replicas: 2 template: metadata: labels: app: easyfly spec: containers: - name: easyfly image: dadaramjadhav/easyfly:dev ports: - containerPort: 8081 resources: requests: cpu: 500m memory: 1Gi limits: cpu: 1000m memory: 2Gi --- apiVersion: v1 kind: Service metadata: name: easyfly-svc spec: selector: app: easyfly ports: - protocol: "TCP" port: 8081 targetPort: type: LoadBalancer</pre>
5	Create deployment Command: <code>kubectl apply -f deployment-with-cpu-memory-limits.yml</code> 
6	Get deployment Command: <code>kubectl get deploy</code>

Deployment described with explicit CPU and memory requests/limits configured.

5. Viva Voce Questions - Kubernetes

Q: What problem does Kubernetes solve that plain Docker does not?

A: Docker runs individual containers, but does not, by itself, restart failed containers across a fleet of machines, load-balance traffic between replicas, roll out updates without downtime, or schedule containers onto the best available machine. Kubernetes automates all of this by continuously reconciling the actual cluster state with a declared desired state.

Q: What is the difference between a Pod and a Container?

A: A container is a single running process image. A Pod is Kubernetes' smallest deployable unit and wraps one or more tightly-coupled containers that share the same network namespace (one IP) and storage volumes.

Q: What is the role of etcd in a Kubernetes cluster?

A: etcd is a distributed, consistent key-value store that holds the entire state and configuration of the cluster; it is the single source of truth that the API server reads from and writes to.

Q: What does the kube-scheduler do?

A: It watches for pods that have been created but not yet assigned to a node, and selects the best node for each pod based on resource requests, affinity/anti-affinity rules, taints and tolerations, and other constraints.

Q: What is the difference between a ReplicaSet and a Deployment?

A: A ReplicaSet simply guarantees that a specified number of identical pod replicas are running. A Deployment is a higher-level object that manages ReplicaSets for you and adds declarative rolling updates, versioned rollout history and rollback support.

Q: What are the three common Service types and how do they differ?

A: ClusterIP (default) exposes the service only inside the cluster; NodePort exposes a static port on every node's IP, reachable from outside; LoadBalancer provisions an external load balancer (or, locally, is reachable via 'minikube tunnel') and is the most common way to expose an app to the outside world.

Q: Why is 'minikube tunnel' needed when using a LoadBalancer service locally?

A: Minikube runs as a single local VM/container and has no real cloud load balancer to provision. 'minikube tunnel' creates a route on the host machine so LoadBalancer services get a reachable external IP, simulating what a cloud provider would normally do.

Q: What is a Horizontal Pod Autoscaler (HPA) and what does '--cpu-percent=50 --min=1 --max=10' mean?

A: An HPA automatically adjusts the number of pod replicas in a Deployment/ReplicaSet based on observed metrics. The given flags configure it to keep average CPU utilization near 50%, scaling between a minimum of 1 and a maximum of 10 pods.

Q: What is the difference between 'kubectl run' and 'kubectl create deployment'?

A: 'kubectl run' creates a single, unmanaged Pod directly. 'kubectl create deployment' creates a Deployment (and, transitively, a ReplicaSet) that keeps

the desired number of pod replicas running and supports rolling updates - this is what you use for real workloads.

Q: How does Kubernetes achieve a zero-downtime rollout when you change the container image in a Deployment?

A: By default a Deployment uses the RollingUpdate strategy: it gradually creates new pods running the new image while gradually terminating old pods, always keeping enough healthy pods available, so the service never goes fully down.

Q: Why use a manifest (YAML) file instead of individual kubectl commands?

A: A manifest is declarative, version-controllable, repeatable and reviewable; the whole desired state (Deployment + Service, and more) can be created, updated or torn down consistently with a single 'kubectl apply -f', instead of remembering and re-running a sequence of imperative commands.

Q: What is a PersistentVolumeClaim (PVC) and why does the MySQL activity need one?

A: A PVC is a request for durable storage made by a pod, independent of the pod's own lifecycle. MySQL's data files must survive pod restarts/rescheduling, so its Deployment mounts a PVC at /var/lib/mysql instead of relying on the container's writable layer, which is deleted with the container.

Q: Why is the MySQL Service in Activity 10 'headless' (clusterIP: None)?

A: A headless service does not get a single virtual cluster IP; instead DNS resolves directly to the pod's own IP. This is the typical pattern for stateful workloads like databases where clients need to address a specific pod rather than an arbitrarily load-balanced one.

Q: What do 'requests' and 'limits' mean in a container's resources block?

A: 'requests' is the amount of CPU/memory the scheduler guarantees to reserve for the container when placing it on a node. 'limits' is the hard ceiling the container is not allowed to exceed at runtime; exceeding the memory limit gets the container OOM-killed.

Q: What is the difference between kubectl 'describe' and 'get' for a resource?

A: 'kubectl get' gives a short, tabular summary of one or more resources. 'kubectl describe' gives a detailed, human-readable view of a single resource including its full configuration, status, and recent Events, which is very useful for debugging.

Q: What does kubectl logs <pod-name> show, and what if the pod has multiple containers?

A: It streams the stdout/stderr output of the pod's container(s), which is normally where an application writes its logs. If a pod has more than one container you must add '-c <container-name>' to select which container's logs to view.

Q: What is the role of kubelet on a worker node?

A: kubelet is the node agent that talks to the API server, receives PodSpecs assigned to its node, ensures the described containers are actually running and healthy (via the container runtime), and reports pod/node status back to the control plane.

Q: What does kube-proxy do?

A: It runs on every node and maintains the network rules (iptables/IPVS) that implement the Service abstraction, routing traffic addressed to a Service's virtual IP to one of the matching backend pods.

Q: What is a Namespace used for?

A: A Namespace is a way to logically partition a single cluster's resources - for example separating dev/test/prod environments or different teams - so names don't collide and access/quotas can be scoped independently.

Q: How would you roll back a bad Deployment update?

A: Kubernetes Deployments keep a revision history; 'kubectl rollout undo deployment/<name>' reverts the Deployment to the previous (or a specified) revision, and Kubernetes performs the rollback using the same rolling-update mechanism.

PART 2

Agile Principles, Scrum Practices & Iterative Development

1. Basic Concepts: SDLC and Software Process Models

Before looking at Agile and Scrum specifically, it helps to place them in context: they are one family of answers to a much older question - how should a team organize the process of building software?

1.1 What is the SDLC?

The Software Development Life Cycle (SDLC) is the general sequence of stages a piece of software passes through, regardless of which process model is used to organize them: Requirements gathering & analysis, Design, Implementation (coding), Testing, Deployment/Release, and Maintenance. Different process models differ mainly in how these stages are sequenced, repeated and overlapped.

1.2 Waterfall Model

- A linear, sequential model: each SDLC stage must be fully completed and signed off before the next stage begins (requirements -> design -> build -> test -> deploy).

- Advantages: simple to understand and plan, works well when requirements are stable and well understood upfront (e.g. certain regulated/contractual projects).
- Disadvantages: very costly to change direction once a stage is complete; the customer/user does not see working software until very late, so misunderstood requirements are discovered late, when they are most expensive to fix.

1.3 Iterative & Incremental Models

- Iterative: the team repeatedly cycles through the SDLC stages in short passes, refining and improving the software each time, rather than doing each stage once.
- Incremental: the system is built and delivered in small functional pieces (increments), each of which adds usable functionality on top of the last.
- Agile methods (including Scrum and Kanban) combine both ideas: each Sprint/iteration is a short pass through the full SDLC that incrementally adds a working piece of the product.

1.4 Where Agile/Scrum Fits

- Agile is a set of values and principles (the Agile Manifesto, covered below); it is a mindset, not a fixed process.
- Scrum, Kanban, Extreme Programming (XP) and others are concrete frameworks/methodologies that implement Agile values in practice. Scrum is the framework in scope for this module.

2. Agile Principles

Agile is a mindset for delivering software in small, iterative and incremental cycles rather than one large 'big bang' release at the end of a long project. It grew out of dissatisfaction with the traditional Waterfall model, where requirements were frozen up front and problems were only discovered very late, when they are most expensive to fix.

Agile was formalized in 2001 in the Agile Manifesto, which values:

- Individuals and interactions over processes and tools.
- Working software over comprehensive documentation.
- Customer collaboration over contract negotiation.
- Responding to change over following a rigid plan.

Note that the items on the right still have value - Agile does not discard process, documentation, contracts or planning altogether - it simply values the items on the left more, because software requirements are rarely fully known upfront and feedback loops shorten the distance between building the wrong thing and discovering it.

Core Agile Principles Applied Day to Day

- Deliver working software frequently, in short cycles (weeks, not months).
- Welcome changing requirements, even late in development.
- Business people and developers must work together daily throughout the project.
- Build projects around motivated individuals; give them the environment and trust to get the job done.
- Working software is the primary measure of progress.
- Teams reflect regularly on how to become more effective and tune their behaviour accordingly (continuous improvement / retrospectives).

3. Scrum Framework

Scrum is the most widely used framework for implementing Agile. It organizes work into fixed-length iterations called Sprints (commonly 1-4 weeks), each of which produces a potentially shippable increment of the product.

2.1 Scrum Roles

- Product Owner (PO): owns and prioritizes the Product Backlog, represents the voice of the customer/stakeholders, and decides what gets built and in what order.
- Scrum Master: a servant-leader who facilitates Scrum events, removes impediments/blockers for the team, and coaches the team and organization on Agile/Scrum practice - the Scrum Master is not the team's manager.
- Development Team: a small, cross-functional, self-organizing group (developers, testers, designers, etc.) who collectively decide how to turn backlog items into a working increment.

2.2 Scrum Events (Ceremonies)

- Sprint Planning: at the start of a Sprint, the team selects items from the Product Backlog to form the Sprint Backlog and agrees on a Sprint Goal.
- Daily Scrum (Standup): a short (~15 minute), time-boxed daily sync where each team member shares what they did, what they plan to do, and any blockers.
- Sprint Review: at the end of the Sprint, the team demonstrates the completed increment to stakeholders and gathers feedback.
- Sprint Retrospective: the team reflects on how the Sprint went (what went well, what didn't) and agrees on concrete process improvements for the next Sprint.

- Backlog Refinement (Grooming): an ongoing activity where the Product Owner and team clarify, estimate and re-prioritize upcoming backlog items so they are 'ready' for a future Sprint Planning.

2.3 Scrum Artifacts

- Product Backlog: the single, ordered list of everything that might be needed in the product - features, bug fixes, technical work - owned and prioritized by the Product Owner.
- Sprint Backlog: the subset of Product Backlog items selected for the current Sprint, plus the team's plan for delivering them.
- Increment: the sum of all completed Product Backlog items during a Sprint, combined with all previous Sprints' work - it must meet the team's Definition of Done and be potentially releasable.

2.4 Scrum vs Kanban (Brief Comparison)

- Scrum works in fixed-length Sprints with a committed scope; Kanban is a continuous flow with no fixed iterations - work items move across a board as capacity allows.
- Scrum limits work by time-boxing the Sprint; Kanban limits work in progress (WIP limits) per column/stage.
- Scrum prescribes specific roles and ceremonies; Kanban is more lightweight and can be layered on top of any existing process.

4. Iterative Development with Jira & Confluence

Jira is Atlassian's issue-tracking and Agile project-management tool, used to plan, track and report on Agile work (Scrum boards, Kanban boards, backlogs, sprints and releases). Confluence is Atlassian's companion collaborative documentation/wiki tool, commonly used alongside Jira to record requirements, meeting notes, architecture decisions and sprint retrospective notes, often cross-linked directly to Jira issues.

3.1 Core Jira Concepts

- Project: a container for all of the issues related to a piece of work, e.g. a product or a team's workstream.
- Issue types: Epic (a large body of work spanning multiple sprints), Story (a user-facing requirement, usually written as 'As a <user>, I want <goal>, so that <benefit>'), Task (a piece of work not necessarily tied to a user story), Sub-task, and Bug.
- Backlog: the ordered list of not-yet-started issues for a project, groomed and prioritized by the Product Owner.
- Board (Scrum or Kanban): a visual representation of issues moving through workflow states/columns such as To Do, In Progress, In Review and Done.

- **Sprint:** in a Scrum project, a time-boxed iteration containing a committed subset of the backlog; Jira tracks sprint start/end dates, velocity and burndown.
- **Workflow:** the sequence of statuses an issue moves through and the transitions allowed between them, often customized per team.
- **Story Points:** a relative, abstract unit of estimation (often Fibonacci-like: 1, 2, 3, 5, 8, 13) used to size the effort/complexity/uncertainty of a Story rather than estimating in hours/days directly.

3.2 A Typical Iterative Cycle in Jira

- Product Owner adds and refines items in the Product Backlog (Epics broken down into Stories/Tasks).
- During Sprint Planning, the team pulls a set of Stories into the active Sprint, which now appears on the Sprint/Scrum board.
- Developers move issues across the board (To Do -> In Progress -> In Review -> Done) as work progresses; the Daily Scrum uses this board to spot blockers.
- Burndown charts in Jira show remaining work against time, helping the team see if the Sprint is on track.
- At Sprint Review, completed issues are demonstrated; at the Retrospective, process improvements are logged, often in a linked Confluence page.
- The cycle repeats every Sprint, continuously refining the backlog and incrementally delivering working software.

4.3 Writing a Good User Story: the INVEST Checklist

A User Story is a short, plain-language description of a requirement from the end user's perspective, normally written as: 'As a <type of user>, I want <some goal>, so that <some reason/benefit>.' The INVEST checklist is a simple set of criteria for judging whether a story is well written and ready to be picked up in a Sprint:

- **Independent** - can be developed and delivered without being tightly coupled to other stories.
- **Negotiable** - a starting point for a conversation, not a rigid contract; details can be discussed with the Product Owner.
- **Valuable** - delivers clear value to a user or the customer.
- **Estimable** - the team has enough information to size it (in story points).
- **Small** - small enough to be completed within a single Sprint.
- **Testable** - has clear acceptance criteria that let the team verify it is actually done.

5. Viva Voce Questions - Agile & Scrum

Q: What is the core idea behind Agile software development?

A: Deliver working software in small, frequent, iterative increments with continuous feedback and the ability to adapt to changing requirements, instead of trying to fully specify and build the whole system upfront.

Q: What are the four values of the Agile Manifesto?

A: Individuals and interactions over processes and tools; working software over comprehensive documentation; customer collaboration over contract negotiation; and responding to change over following a plan.

Q: What is a Sprint in Scrum?

A: A fixed-length, time-boxed iteration (commonly 1-4 weeks) during which the team builds a potentially shippable increment of the product from a committed subset of the Product Backlog.

Q: What are the three Scrum roles and their responsibilities?

A: The Product Owner owns and prioritizes the Product Backlog; the Scrum Master facilitates the process and removes impediments; the Development Team self-organizes to deliver the increment.

Q: What is the purpose of the Daily Scrum?

A: A short, time-boxed daily sync (typically 15 minutes) where the team inspects progress toward the Sprint Goal and surfaces blockers, so the plan for the next 24 hours can be adjusted.

Q: What is the difference between the Product Backlog and the Sprint Backlog?

A: The Product Backlog is the full, ordered list of everything that might be built for the product, owned by the Product Owner. The Sprint Backlog is the subset selected for the current Sprint, along with the team's plan to deliver it.

Q: What happens in a Sprint Retrospective?

A: The team reflects on how the Sprint went - what worked well, what didn't - and agrees on concrete, actionable process improvements to try in the next Sprint.

Q: What is a Definition of Done?

A: A shared, team-agreed checklist of criteria (e.g. code reviewed, tests passing, documentation updated) that a backlog item must meet before it can be considered complete.

Q: How does Kanban differ from Scrum?

A: Kanban is a continuous flow of work with no fixed iterations and limits work using WIP (work-in-progress) limits per stage, while Scrum organizes work into fixed-length Sprints with a committed scope and prescribed roles/ceremonies.

Q: What is an Epic in Jira, and how does it relate to a Story?

A: An Epic is a large body of work that is too big to fit in a single Sprint; it is broken down into smaller, independently deliverable Stories, which in turn may be broken into Tasks/Sub-tasks.

Q: What are Story Points used for?

A: A relative, abstract unit (often following a Fibonacci-like scale) used to estimate the effort, complexity and uncertainty of a Story, enabling the team to forecast velocity without estimating in exact hours.

Q: What is a burndown chart?

A: A chart that plots remaining work (story points or hours) against time within a Sprint, letting the team see at a glance whether they are on track to complete the Sprint's committed scope.

Q: What is 'velocity' in Scrum?

A: The average amount of work (usually in story points) a team completes per Sprint, calculated from past Sprints and used to help forecast how much work can realistically be taken on in future Sprints.

Q: Why is backlog refinement/grooming important?

A: It keeps upcoming backlog items clear, appropriately sized and estimated ahead of time, so Sprint Planning is fast and the team commits to well-understood work instead of vague, oversized items.

Q: How are Jira and Confluence typically used together?

A: Jira tracks and reports on the actual units of work (Epics/Stories/Tasks, boards, sprints), while Confluence documents the surrounding context - requirements, design decisions, meeting notes and retrospective outcomes - usually cross-linked to the relevant Jira issues.

PART 3

GenAI & Agentic AI

1. Basic Concepts: AI, ML, Deep Learning and GenAI

GenAI and Agentic AI are best understood as the newest, most specialized layer on top of a much older field. It helps to be clear on how the terms nest inside one another before going further.

1.1 The AI → ML → Deep Learning → GenAI Hierarchy

- Artificial Intelligence (AI): the broad field of building systems that perform tasks which normally require human intelligence - reasoning, perception,

language, decision-making. This includes everything from simple rule-based expert systems to modern LLMs.

- Machine Learning (ML): a subset of AI in which a system learns patterns from data instead of being explicitly programmed with rules - for example a model that learns to classify emails as spam by training on many labelled examples.
- Deep Learning: a subset of ML that uses artificial neural networks with many layers ('deep' networks) to automatically learn increasingly abstract features directly from raw data (pixels, audio waveforms, text), which is what makes today's large-scale language and vision models possible.
- Generative AI: a subset of deep learning focused specifically on models that generate new content (text, code, images, audio) rather than only predicting a category or a number. Today's leading GenAI systems for text/code are built on the Transformer architecture and are called Large Language Models (LLMs).

1.2 Key Terms Glossary

- Token: the basic unit an LLM reads and generates - usually a word, part of a word, or punctuation mark, not a whole sentence. 'unbelievable' might be split into tokens like 'un', 'believe', 'able'.
- Context window: the maximum number of tokens (input + output combined) a model can consider at once; anything older than this window is effectively 'forgotten' by the model within that conversation.
- Embedding: a numeric vector representation of a piece of text (a word, sentence or document) such that pieces of text with similar meaning end up as vectors that are numerically close together - the foundation for semantic search and Retrieval-Augmented Generation (RAG).
- Inference: the act of running a trained model on new input to produce an output (as opposed to 'training', which is the earlier, far more expensive process of teaching the model from data).
- Temperature: a parameter (usually 0 to 1 or higher) that controls how random/creative vs. deterministic/focused the model's next-token choices are - low temperature gives more predictable, repeatable answers; high temperature gives more varied, creative ones.
- Foundation model: a large, general-purpose model (e.g. Gemini, GPT, Llama) pre-trained on broad data, which can then be used directly, prompted cleverly, or further specialized for a particular task.
- Fine-tuning vs prompting vs RAG: fine-tuning further trains a foundation model's weights on a specific, usually smaller, dataset so it specializes in a task; prompting instead shapes behaviour purely through the input text at inference time, with no retraining; RAG (Retrieval-Augmented Generation)

retrieves relevant documents/passages (typically via embeddings and a vector database) and injects them into the prompt so the model can answer using up-to-date or private information it was never trained on. The demo programs in this module rely entirely on prompting - no fine-tuning or RAG is involved.

- Hallucination: a confident-sounding but factually incorrect or fabricated output - discussed further in the AI Testing section below.

2. Introduction to Generative AI

Generative AI (GenAI) refers to a class of artificial intelligence models that can create new content - text, code, images, audio or video - rather than simply classifying or predicting a label for existing data. Modern GenAI for text and code is built on Large Language Models (LLMs): neural networks with billions of parameters, trained on massive text/code corpora, that learn statistical patterns of language well enough to generate coherent, contextually relevant output token by token.

How an LLM Generates a Response

- The input prompt is broken into tokens (sub-word units) and converted into numerical embeddings.
- A Transformer neural network (using a mechanism called self-attention) processes the token sequence, weighing how much each token should influence every other token.
- The model predicts a probability distribution over the next token, samples one, appends it to the sequence, and repeats - generating the response one token at a time.
- Techniques like temperature and top-p/top-k sampling control how deterministic or creative/random the output is.

Common GenAI Model Families

- Google Gemini (accessed via Vertex AI) - used in the demo programs in this module.
- Meta Llama (e.g. llama3, run locally via Ollama) - used in the Spring AI demo in this module.
- OpenAI GPT family, Anthropic Claude, and other commercial and open-weight LLMs.

3. GenAI in Java

Java applications typically consume an LLM in one of a few ways: calling a cloud provider's Java SDK directly (e.g. Google's Vertex AI SDK for Java), calling a self-hosted model's REST API (e.g. Ollama running llama3 locally) using a plain RestTemplate/WebClient, or using a higher-level abstraction library such

as Spring AI, which provides a single, provider-agnostic ChatClient API on top of many different model backends.

- Direct SDK integration (e.g. `com.google.cloud.vertexai`) gives full access to a specific provider's features but ties the code to that provider.
- Plain REST calls (e.g. to a local Ollama server's `/api/generate` endpoint) are simple and provider-independent, but require the developer to hand-roll request/response mapping, retries, and error handling.
- Spring AI standardizes prompt construction, model invocation, response parsing, function/tool calling and Retrieval-Augmented Generation (RAG) behind a consistent Spring-idiomatic API, and lets you swap the underlying model provider largely through configuration.

4. Agentic AI

Agentic AI describes AI systems that go beyond producing a single response to a single prompt: an 'agent' can reason about a goal, break it into steps, decide which tools/APIs to call, observe the result of each action, and iterate - autonomously working towards the goal with limited or no human intervention at each step.

A Typical Agent Loop

- Perceive: the agent receives a goal/task and the current state of its environment (conversation history, tool outputs, files, etc.).
- Plan/Reason: the LLM reasons about what to do next - often using patterns like ReAct (Reason + Act) or Chain-of-Thought - and decides on the next action.
- Act: the agent invokes a tool, function or external API (e.g. run a query, call a REST endpoint, execute code, search the web).
- Observe: the agent receives the tool's output and folds it back into its context.
- Repeat/Terminate: the loop continues until the goal is satisfied, a stopping condition is met, or a human is asked to confirm/intervene.

GenAI vs Agentic AI

- GenAI: single-turn (or simple multi-turn chat) content generation - you provide a prompt, the model returns a completion. It has no memory of tools, no independent planning, and no ability to take actions in the world beyond producing text.
- Agentic AI: builds on top of a GenAI model but adds autonomy - planning, tool/function calling, multi-step execution, and often long-running memory - so it can actually carry out a task, not just describe how to.

- In short: GenAI answers 'what should be done'; an agent actually goes and does it, checking its own work along the way.

5. Prompt Engineering

Prompt engineering is the practice of designing the input given to an LLM so that its output is as accurate, relevant and well-formatted as possible. Because LLMs are extremely sensitive to phrasing, structure and examples, a well-engineered prompt can dramatically improve quality without any change to the underlying model.

- Zero-shot prompting: ask the model to perform a task with no examples, relying entirely on its pre-trained knowledge (e.g. the `CodeGeneration.java` demo below simply asks: 'Write a java method that checks if a number is a prime number.').
- Few-shot prompting: include a small number of example input/output pairs in the prompt so the model can infer the desired pattern or format.
- Chain-of-thought prompting: explicitly ask the model to reason step by step before giving a final answer, which improves accuracy on multi-step reasoning or math/logic tasks.
- Role/persona prompting: instruct the model to act as a specific persona (e.g. 'You are a senior Java code reviewer') to bias its tone, vocabulary and focus.
- Providing context/constraints: specifying the target language/framework/version, output format (e.g. 'return only the code, no explanation'), and any constraints (naming conventions, libraries allowed) sharply narrows the space of acceptable answers.
- Iterative refinement: treating the first response as a draft, and following up with a clarifying/refining prompt (as in the `TestCodeGeneration.java` demo, which feeds a method's source code back to the model and asks for a JUnit test class for it).

6. AI Testing

AI testing here covers two related but distinct concerns: testing code that an AI assistant helped write or generate, and evaluating the quality/reliability of an AI model's own output.

- Treat AI-generated code like any other code: it still needs unit tests, code review, static analysis (e.g. SonarQube) and to pass the CI pipeline before merging - AI assistance does not remove the need for verification.
- Watch for hallucinations: an LLM can generate code that references non-existent methods/classes/APIs, or that compiles but is logically wrong; always compile and run generated code, and cross-check any factual claims.

- Test edge cases explicitly, since AI-generated tests (as produced by tools like GitHub Copilot or the TestCodeGeneration demo) commonly cover the 'happy path' well but can miss boundary conditions (e.g. negative numbers, zero, very large inputs) unless specifically asked for.
- Evaluate model output quality along multiple axes: correctness/factual accuracy, relevance to the prompt, consistency across repeated runs, safety (no harmful/biased content), and latency/cost - especially before using a model output in a production decision.
- For agentic systems, additionally test the tool-selection logic itself (did the agent choose the right tool and the right parameters?) and add guardrails/human-in-the-loop checkpoints before any action with real-world side effects.

7. GitHub Copilot

GitHub Copilot is an AI pair-programmer built on OpenAI's models, integrated directly into the developer's editor. It suggests whole lines or blocks of code as you type, can generate functions from a natural-language comment, explain existing code, generate unit tests, and (via 'Copilot Chat') answer questions about the open workspace.

Key Features

- Inline code completion: ghost-text suggestions as you type, accepted with Tab.
- Copilot Chat: a conversational panel to ask questions, request refactors, generate tests, or explain a selected block of code.
- Copilot for Pull Requests: auto-generated PR summaries and code-review suggestions.
- Copilot CLI / Copilot in the terminal: natural-language to shell-command suggestions.

Subscriptions

- Copilot Individual: paid, per-seat plan for individual developers.
- Copilot Business: per-seat plan for organizations, with centralized policy management and no training on the org's private code by default.
- Copilot Enterprise: adds organization-specific customization (indexing an org's own repos/docs for more relevant suggestions) on top of Business.
- A free tier is available with a limited monthly quota of completions and chat interactions.

IDE Integrations

- Available as an extension for Visual Studio Code, Visual Studio, the JetBrains family (IntelliJ IDEA, PyCharm, etc.), Eclipse, Neovim, and directly inside github.com and GitHub Codespaces.

Security & Privacy Concerns

- Suggestions can occasionally reproduce snippets resembling public training data; GitHub provides a filter to block suggestions that match public code, and Business/Enterprise plans do not use customer code to train the underlying model.
- Sensitive data (credentials, internal business logic) should not be pasted into chat prompts, since prompt content may leave the local machine and be logged/processed remotely, depending on plan and settings.
- Organizations can centrally enable/disable Copilot, restrict it to approved repos, and enforce the public-code-matching filter via policy.

Limitations

- Copilot can hallucinate APIs/method signatures that do not exist, and can suggest outdated or insecure patterns - all suggestions must be reviewed, compiled and tested, not accepted blindly.
- It lacks full understanding of the whole codebase/business context unless that context is explicitly provided (open files, comments, chat context, or - on Enterprise - indexed repos).
- It is a productivity aid, not a substitute for code review, testing, or architectural judgement.

8. Introduction to Spring AI

Spring AI is a Spring-ecosystem project that brings the familiar Spring programming model (dependency injection, auto-configuration, starters) to building AI-powered applications in Java. It provides a portable abstraction over many different model providers (OpenAI, Azure OpenAI, Ollama, Vertex AI/Gemini, Anthropic, Amazon Bedrock and more), so application code is largely decoupled from any single vendor's SDK.

Key Capabilities

- ChatClient / ChatModel abstraction: a single, fluent API to send prompts and receive completions, regardless of which model provider is configured underneath.
- Prompt templates: structured, parameterized prompt construction, similar in spirit to a Spring MVC view template.
- Structured output conversion: helpers to map an LLM's free-text response into a strongly-typed Java object (POJO/record).

- Function/tool calling: the model can be given a set of Java methods it is allowed to invoke to fetch data or take action - the foundation for building agentic behaviour in Spring.
- Vector stores & RAG support: integrations with vector databases to store embeddings and retrieve relevant context for Retrieval-Augmented Generation.
- Spring Boot auto-configuration: adding a starter dependency (e.g. spring-ai-starter-model-ollama) and a few properties is usually enough to get a working AI-backed REST endpoint, as shown in the demo below.

9. Demo Programs

The following programs are taken directly from this module's demo projects (GenAIDemos and SpringAI_Demo) and illustrate the concepts above with real, runnable Java code.

8.1 Demo1001 - Vertex AI Gemini: Code Generation

Scenario	Send a plain-text (zero-shot) prompt to Google's Gemini model via the Vertex AI Java SDK and print the generated Java method.
Key API	com.google.cloud.vertexai.VertexAI + GenerativeModel.generateContent(prompt)

```

package com.accenture.lkm;
import java.io.IOException;
import com.google.cloud.vertexai.VertexAI;
import com.google.cloud.vertexai.api.GenerateContentResponse;
import com.google.cloud.vertexai.generativeai.GenerativeModel;
import com.google.cloud.vertexai.generativeai.ResponseHandler;
public class CodeGeneration {
    public static void main(String[] args) throws IOException {
        String projectId = "genaidemo1-423308";
        String modelName = "gemini-1.0-pro-002";
        String location = "asia-south1";
        String prompt = " Write a java method that checks if a number is a prime
number. ";
        generateCode(projectId, location, modelName, prompt);
    }
    public static void generateCode(String projectId, String location, String
modelName, String prompt)
throws IOException {
        try (VertexAI vertexAI = new VertexAI(projectId, location)) {
            GenerativeModel model1 = new GenerativeModel(modelName, vertexAI);
            GenerateContentResponse response = model1.generateContent(prompt);
            System.out.println(ResponseHandler.getText(response));
        }
    }
}

```

How it works: the VertexAI client is created for a given Google Cloud project and region, wrapped in a try-with-resources block so it is closed automatically. A GenerativeModel is instantiated for the chosen Gemini model, and generateContent(prompt) sends the zero-shot prompt to the model. ResponseHandler.getText(...) extracts the plain-text answer from the structured GenerateContentResponse for printing.

8.2 Demo1002 - Vertex AI Gemini: Test Code Generation

Scenario	Feed an existing Java method's source code back into the prompt and ask Gemini to generate a JUnit test class for it - an example of iterative/context-rich prompting.
Key API	Same Vertex AI SDK, with a longer, code-containing prompt string.

```

package com.accenture.lkm;
import java.io.IOException;
import com.google.cloud.vertexai.VertexAI;
import com.google.cloud.vertexai.api.GenerateContentResponse;
import com.google.cloud.vertexai.generativeai.GenerativeModel;
import com.google.cloud.vertexai.generativeai.ResponseHandler;
public class TestCodeGeneration {
    public static void main(String[] args) throws IOException {
        String prompt = "Write a junit test class for this java method: \n"
+ " public static boolean isPrime(int number) { \n"
+ " if (number == 1) { \n"
+ " return false; \n"
+ " } \n"
+ " if (number == 2) { \n"
+ " return true; \n"
+ " } \n"
+ " for (int i = 2; i <= Math.sqrt(number); i++) { \n"
+ " if (number % i == 0) { \n"
+ " return false; \n"
+ " } \n"
+ " } \n"
+ " return true; \n"
+ " } ";
        String projectId = "genaidemo1-423308";
        String modelName = "gemini-1.0-pro-002";
        String location = "asia-south1";
        generateTestCode(projectId, location, modelName, prompt);
    }
    public static void generateTestCode(String projectId, String location, String
modelName, String prompt)
        throws IOException {
        try (VertexAI vertexAI = new VertexAI(projectId, location)) {
            GenerativeModel model1 = new GenerativeModel(modelName, vertexAI);
            GenerateContentResponse response = model1.generateContent(prompt);
            System.out.println(ResponseHandler.getText(response));
        }
    }
}

```

How it works: rather than a bare instruction, the prompt embeds the full source of `isPrime(int)` so the model has the exact contract, edge cases (1 and 2 handled specially) and control flow to test against. This context-rich prompting pattern typically produces far more relevant, compilable JUnit tests than asking the model to 'write tests for a prime-checking method' with no code supplied.

8.3 Spring AI Demo - Basic Static Prompt with a Local Ollama Model

Scenario	A Spring Boot REST API that forwards a user-supplied prompt to a locally running Ollama server (model: llama3) and returns the generated text as JSON.
Key API	Spring Boot + RestTemplate calling Ollama's REST /generate endpoint (wrapped behind a small ChatClient abstraction).

pom.xml (key dependencies):

```
<parent>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-parent</artifactId>
<version>3.5.3</version>
</parent>
<properties>
<java.version>17</java.version>
<spring-ai.version>1.0.0</spring-ai.version>
</properties>
<dependencies>
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-web</artifactId>
</dependency>
<dependency>
<groupId>org.springframework.ai</groupId>
<artifactId>spring-ai-starter-model-ollama</artifactId>
</dependency>
</dependencies>
```

application.properties:

```
spring.application.name=spring-AI
spring.ai.ollama.base-url=http://localhost:11434/api
spring.ai.ollama.model=llama3
server.port=8686
spring.main.allow-bean-definition-overriding=true
```

ChatClient.java (a small provider-agnostic contract for this demo):

```
package com.ai.spring.AI.service;
public interface ChatClient {
String generate(String prompt);
}
```

OllamaChatClient.java (implementation that calls Ollama's REST API):

```
package com.ai.spring.AI.service.impl;
import com.ai.spring.AI.service.ChatClient;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Component;
import org.springframework.web.client.HttpClientErrorException;
import org.springframework.web.client.RestTemplate;
import java.util.HashMap;
import java.util.Map;
@Component("customOllamaChatClient")
public class OllamaChatClient implements ChatClient {
private final RestTemplate restTemplate;
private final String baseUrl;
private final String model;
public OllamaChatClient(RestTemplate restTemplate,
@Value("${spring.ai.ollama.base-url}") String baseUrl,
@Value("${spring.ai.ollama.model}") String model) {
this.restTemplate = restTemplate;
this.baseUrl = baseUrl;
this.model = model;
}
@Override
public String generate(String prompt) {
String url = String.format("%s/generate", baseUrl);
Map<String, Object> request = new HashMap<>();
request.put("model", model);
request.put("prompt", prompt);
request.put("stream", false);
try {
return restTemplate.postForObject(url, request, String.class);
} catch (HttpClientErrorException e) {
System.err.println("Error: " + e.getStatusCode() + " - " +
e.getResponseBodyAsString());
throw e;
}
}
}
```

LlamaAiService.java / LlamaAiServiceImpl.java (service layer with two use cases
- a generic prompt and a canned 'joke' prompt template):

```
public interface LlamaAiService {
    LlamaResponse generateMessage(String prompt);
    LlamaResponse generateJoke(String topic);
}

@Service
public class LlamaAiServiceImpl implements LlamaAiService {
    @Autowired
    private OllamaChatClient chatClient;
    @Override
    public LlamaResponse generateMessage(String promptMessage) {
        final String llamaMessage = chatClient.generate(promptMessage);
        return new LlamaResponse().setMessage(llamaMessage);
    }
    @Override
    public LlamaResponse generateJoke(String topic) {
        final String llamaMessage = chatClient.generate(String.format("Tell me a joke
        about %s", topic));
        return new LlamaResponse().setMessage(llamaMessage);
    }
}
```

LlamaRestController.java (exposes the service as a REST API):

```

@RestController
public class LlamaRestController {
    @Autowired
    private LlamaAiServiceImpl llamaAiService;
    @GetMapping(value = "api/v1/ai/generate")
    public ResponseEntity<LlamaResponse> generate(
        @RequestParam(value = "promptMessage", defaultValue = "Why is the sky
        blue?")
        String promptMessage) {
        final LlamaResponse aiResponse =
            llamaAiService.generateMessage(promptMessage);
        return ResponseEntity.status(HttpStatus.OK).body(aiResponse);
    }
    @PostMapping("api/v1/ai/generate/joke/{topic}")
    public ResponseEntity<LlamaResponse> generateJoke(@PathVariable String
    topic) {
        final LlamaResponse aiResponse = llamaAiService.generateJoke(topic);
        return ResponseEntity.status(HttpStatus.OK).body(aiResponse);
    }
}

```

How it works end-to-end: a client calls GET /api/v1/ai/generate?promptMessage=... (or POST /api/v1/ai/generate/joke/{topic}). The controller delegates to LlamaAiServiceImpl, which calls the injected OllamaChatClient.generate(prompt). That client POSTs a JSON body {model, prompt, stream:false} to the local Ollama server's /generate REST endpoint, receives the generated text, and the response flows back up as a LlamaResponse JSON payload with HTTP 200. This mirrors, at a simplified level, exactly what Spring AI's ChatClient abstraction does for you out of the box - but is hand-rolled here to show what happens 'under the hood'.

10. Viva Voce Questions - GenAI & Agentic AI

Q: What is Generative AI?

A: A class of AI models that create new content - text, code, images, audio, video - rather than only classifying or predicting labels for existing data; modern text/code GenAI is built on Large Language Models (LLMs).

Q: What is a Large Language Model (LLM), at a high level?

A: A neural network (typically a Transformer architecture) with billions of parameters, trained on massive text/code corpora, that generates text one token at a time by predicting the most likely next token given the preceding context.

Q: What is the difference between Generative AI and Agentic AI?

A: GenAI produces a single response/completion for a given prompt with no independent action-taking ability. Agentic AI builds on a GenAI model but adds autonomous planning, tool/function calling and multi-step execution so it can actually carry out a task.

Q: What is the 'agent loop' in Agentic AI?

A: The repeating cycle of perceive (get the goal/state) -> plan/reason (decide the next action) -> act (call a tool/API) -> observe (read the result) -> repeat until the goal is achieved or a stopping condition is hit.

Q: What is prompt engineering and why does it matter?

A: The practice of carefully designing the input given to an LLM (wording, structure, examples, constraints) to get accurate, relevant, well-formatted output - it can significantly change output quality without changing the model itself.

Q: What is the difference between zero-shot and few-shot prompting?

A: Zero-shot gives the model only an instruction, with no examples, relying on its pre-trained knowledge. Few-shot includes a small number of example input/output pairs in the prompt to demonstrate the desired pattern.

Q: What is chain-of-thought prompting?

A: A technique where the model is explicitly asked to reason step by step before giving a final answer, which improves accuracy on multi-step reasoning, math, or logic-heavy tasks.

Q: In the Vertex AI demo, what does `ResponseHandler.getText(response)` do?

A: It extracts the plain-text content from the structured `GenerateContentResponse` object returned by the Gemini model, so it can be printed or used directly as a `String`.

Q: Why does the `TestCodeGeneration` demo include the full source of `isPrime(int)` inside the prompt instead of just describing the method?

A: Providing the exact code gives the model the precise contract, control flow and edge cases (e.g. the special handling of 1 and 2) to write tests against, producing far more accurate and directly usable JUnit tests than a vague natural-language description would.

Q: What is Spring AI?

A: A Spring-ecosystem project that provides a portable, Spring-idiomatic abstraction (`ChatClient/ChatModel`, prompt templates, structured output, tool calling, vector store/RAG support) over many different LLM providers, so application code is largely decoupled from any single vendor's SDK.

Q: In the Spring AI demo, what role does `OllamaChatClient` play?

A: It implements the small ChatClient interface and is the component that actually talks to the local Ollama server, POSTing the model name, prompt and stream flag to Ollama's REST /generate endpoint and returning the generated text.

Q: Why is spring.ai.ollama.base-url configured with @Value in the demo?

A: It externalizes the Ollama server's address (and similarly the model name) into application.properties, so the same code can point at a different Ollama instance or model without any code change - a standard Spring externalized-configuration pattern.

Q: What does the 'stream: false' flag do in the Ollama request payload?

A: It tells Ollama to return the complete generated response in a single JSON payload once generation finishes, instead of streaming partial tokens back incrementally as they are produced.

Q: What is GitHub Copilot and what does it do?

A: An AI pair-programmer, built on OpenAI models and integrated into the editor, that suggests inline code completions, can generate code from natural-language comments, explain existing code, and answer questions via Copilot Chat.

Q: What is the difference between GitHub Copilot Business and Copilot Enterprise?

A: Business is the per-seat organizational plan with centralized policy management and no training on private code by default. Enterprise adds the ability to index and use an organization's own repositories/documentation to make suggestions more context-aware.

Q: What are the main security/privacy concerns with using an AI coding assistant like Copilot?

A: Suggestions can occasionally resemble public training data; prompt content (including anything pasted into chat) may be sent to and logged by a remote service depending on the plan; and sensitive data such as credentials or proprietary business logic should not be shared in prompts.

Q: What is a hallucination in the context of LLM-generated code?

A: A confident but incorrect output - for example, code that references a method, class or API that does not actually exist, or that compiles but implements the wrong logic - which is why generated code must always be compiled and tested.

Q: Why should AI-generated code still go through unit testing and code review?

A: AI assistance does not guarantee correctness: generated code can hallucinate APIs, miss edge cases, or embed outdated/insecure patterns, so the normal

engineering safety net (tests, static analysis, peer review, CI) still applies.

Q: What extra testing consideration applies specifically to agentic systems, beyond testing the code they produce?

A: The agent's tool-selection and decision-making logic itself needs to be tested (did it pick the right tool with the right parameters?), and guardrails or human-in-the-loop checkpoints should be added before any action that has real-world side effects.

Q: Name three model providers that Spring AI can abstract over.

A: Any three of: OpenAI, Azure OpenAI, Ollama (local models such as llama3), Google Vertex AI/Gemini, Anthropic, Amazon Bedrock.